

Principia Softwarica: The Plan 9 Kernel 9
Raspberry Pi (ARM 32 bits) edition
version 0.1

Yoann Padioleau
`yoann.padioleau@gmail.com`

with code from
Ken Thompson, Dave Presotto, and Richard Miller

March 18, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1	Introduction	17
1.1	Motivations	17
1.2	The Plan 9 kernel: 9	18
1.3	Other kernels	18
1.4	Getting started	18
1.5	Requirements	19
1.6	About this document	20
1.7	Copyright	20
1.8	Acknowledgments	20
2	Overview	21
2.1	Operating system principles	21
2.2	9's syscall interface	21
2.3	helloraw.c and helloraw.s	24
2.4	minishell.c	25
2.5	The Raspberry Pi machine (ARM)	26
2.5.1	ARM CPU	26
2.5.2	ARM coprocessors	28
2.5.3	ARM MMU	29
2.5.4	Raspberry Pi Input/Output devices	31
2.6	C and assembly	33
2.7	Code organization	34
2.7.1	Portable versus arch-specific code	34
2.7.2	Main headers	35
2.7.3	Top directories	35
2.8	Software architecture	35
2.8.1	Bootting overview (ARM)	36
2.8.2	Memory overview (ARM)	36
2.8.3	Syscall overview (ARM)	41
2.8.4	FAQ	43
2.8.5	NSFAQ	44
2.9	Book structure	45
3	Core Data Structures	46
3.1	Conf and conf	46
3.2	Cpu and cpu	47
3.3	Proc and up	49
3.4	Chan	51
3.5	Dev and devtab	53
3.6	Soc and soc (ARM)	54

4	Concurrency	55
4.1	Critical regions	55
4.2	Interrupts disabling: <code>arch_splhi()</code> , <code>arch_spllo()</code> (ARM)	55
4.3	Test and set: <code>arch_tas()</code> (ARM)	57
4.4	Atomic count: <code>arch_xinc()</code> and <code>arch_xdec()</code> (ARM)	57
4.5	Spin Lock	58
4.6	Uninterruptable <code>ILock</code>	60
4.7	Waiting <code>QLock</code>	62
4.8	Readers/Writer <code>RWLock</code>	65
4.9	Atomic References and Counter	68
4.10	Safeguards	69
4.10.1	<code>Lock.p</code>	69
4.10.2	<code>Proc.nlocks</code>	69
4.10.3	<code>Cpu.ilockdepth</code>	70
4.11	Synchronization	70
4.11.1	Rendez-vous	71
4.11.2	Producer/Consumer Queue	71
4.11.3	Semaphore	71
I	Virtualizing the Machine	72
5	Memory	73
5.1	Overview	73
5.2	Long-lived structure allocator: <code>xalloc()</code>	74
5.2.1	Data structures	74
5.2.2	<code>xalloc()</code>	76
5.2.3	<code>xfree()</code>	77
5.2.4	<code>xmerge()</code>	78
5.3	Memory pools	79
5.3.1	Pool	79
5.3.2	<code>plock()</code>	80
5.3.3	<code>pmainmem</code>	81
5.3.4	<code>poolprint()</code>	82
5.4	Kernel <code>malloc()</code>	83
5.4.1	<code>malloc()</code>	83
5.4.2	Sleeping <code>smalloc()</code>	84
5.4.3	<code>mallocxxx()</code>	84
5.4.4	<code>malloc</code> tags	85
5.5	User <code>malloc()</code>	86
5.6	Page and <code>palloc</code>	86
5.6.1	Page	87
5.6.2	<code>Palloc</code>	88
5.6.3	<code>newpage()</code>	89
5.7	Segment and <code>Pagetable</code>	92
5.7.1	Segment	92
5.7.2	<code>Pagetable</code>	93
5.7.3	<code>newseg()</code>	95
5.7.4	<code>smallpagedir</code>	96
5.7.5	Adding a page to a segment: <code>segpage()</code>	97

5.7.6	Process segments	97
5.7.7	/proc/x/segment	101
5.8	sysbrk()	102
5.8.1	Extending a segment	103
5.8.2	Shrinking a segment	104
6	Processes	106
6.1	Overview	106
6.2	Process allocation	106
6.2.1	Process ID: Proc.pid	106
6.2.2	Procalloc	107
6.2.3	PID hashing	107
6.2.4	newproc()	108
6.2.5	No more processes	109
6.3	The user	110
6.4	sysrfork()	111
6.4.1	Fork: RFPROC	112
6.4.2	Memory sharing: RFMEM	113
6.4.3	arch_forkchild() (ARM)	114
6.4.4	File descriptors: RFFDG, RFCFDG	115
6.4.5	Namespace: RFNAMEG, RFCNAMEG	116
6.4.6	Environment variables: RFENVG, RFCENVG	116
6.4.7	Rendezvous group: RFREND	116
6.4.8	Note group: RFNOTEg	117
6.4.9	Process sharing settings (no RFPROC)	117
6.5	sysexec()	118
6.5.1	Text segment	122
6.5.2	Data segment	122
6.5.3	Bss segment	122
6.5.4	Stack segment	122
6.5.5	arch_execregs() (ARM)	126
6.6	sysawait()	126
6.6.1	Parent and children	126
6.6.2	Waitmsg and Waitq	127
6.6.3	sysawait()	128
6.6.4	Rendez-vous: Proc.waitr	129
6.7	sysexits()	129
6.7.1	pexit()	130
6.7.2	pexit() and schedinit()	131
6.7.3	Wait message	132
6.7.4	Freeing resources	133
6.7.5	No waiting for child: RFNOWAIT	134
6.8	Kernel processes	134
6.8.1	arch_kprocchild() (ARM)	135
6.9	Process control /proc/#/ctl	136

7	Interrupts	138
7.1	Overview	138
7.2	Exceptions (ARM)	139
7.3	Exception vectors (ARM)	139
7.4	System calls	140
7.4.1	_vsvc() (ARM)	140
7.4.2	Sargs	141
7.4.3	arch__syscall() (ARM)	141
7.5	Exception handlers	143
7.5.1	_vxxx() (ARM)	143
7.5.2	_vswitch() (ARM)	144
7.5.3	arch__trap() (ARM)	146
7.5.4	Undefined instruction fault (ARM)	148
7.5.5	Prefetch fault (ARM)	148
7.5.6	Data fault (ARM)	149
7.6	IRQs (ARM)	151
7.6.1	Vctl (ARM)	152
7.6.2	Intregs (ARM)	152
7.6.3	arch_intrenable() (ARM)	153
7.6.4	Basic IRQs (ARM)	153
7.6.5	irq() (ARM)	154
8	Virtual Memory	155
8.1	Overview	155
8.2	The ARM MMU (ARM)	155
8.3	Pagination and mmupd/mmupt (ARM)	156
8.4	Page fault	159
8.4.1	faultarm() (ARM)	159
8.4.2	fault()	159
8.4.3	fixfault()	160
8.4.4	arch_putmmu() (ARM)	162
8.5	Copy on write	164
8.6	Page cache	165
8.7	Demand loading	168
8.7.1	Kimage and the image cache	168
8.7.2	Segment and KImage	169
8.7.3	Paging in	171
8.7.4	duppage()	173
8.7.5	Collecting images	175
8.8	Swapping	178
8.8.1	Swapalloc	178
8.8.2	/dev/swap	179
8.8.3	Page0rSwap	179
8.8.4	Swapping plan	181
8.8.5	The pager	183
8.8.6	Paging out	187
8.8.7	Paging in	188
8.8.8	Setting up the swap file	188

9	Scheduling	190
9.1	Overview	190
9.2	Process priority	190
9.3	The runqueue	192
9.4	The scheduler	194
9.4.1	sched()	194
9.4.2	schedinit()	195
9.4.3	runproc()	195
9.5	Context switch	197
9.5.1	Label	197
9.5.2	Kernel stacks	197
9.5.3	arch_gotolabel() (ARM)	197
9.5.4	sched() and schedinit() dance	198
9.5.5	Delayed sched()	199
9.5.6	arch_mmuswitch() (ARM)	200
9.5.7	arch_procsave() (ARM)	201
9.6	Cooperative scheduling	202
9.6.1	yield()	202
9.6.2	ready()	202
9.6.3	Rendez	203
9.6.4	sleep()	203
9.6.5	wakeup()	205
9.7	Preemptive scheduling	206
9.7.1	hzsched()	206
9.7.2	hzclock()	207
9.8	Customized scheduling	207
9.9	Real-time scheduling	208
9.10	Adjusting priority	208
9.10.1	reprioritize()	209
9.10.2	rebalance()	211
9.11	Halting (ARM)	212
10	Time	213
10.1	Overview (arm)	213
10.2	Time units	213
10.2.1	Seconds	213
10.2.2	Ticks	213
10.2.3	Fastticks (ARM)	214
10.3	Clock interrupt (ARM)	215
10.4	Timers	216
10.4.1	Timer	216
10.4.2	Timers and CPU timers	216
10.4.3	tadd()/tdel()	217
10.4.4	timeradd()/timerdel()	218
10.4.5	arch_timersset() (arm)	219
10.4.6	timerintr()	220
10.4.7	Clock callbacks: addclock0link()	221
10.4.8	Delay (ARM)	222
10.5	syssleep()	222
10.6	Alarms	224

10.7	<code>sysalarm()</code>	224
10.7.1	<code>alarmkproc()</code>	226
10.7.2	<code>checkalarms()</code>	227
10.8	Time of day	227
10.9	<code>sysnsec()</code>	231
10.10	<code>/dev/time</code>	231
10.11	<code>/dev/bintime</code>	232
10.12	Watchdog	234
11	Initialization	235
11.1	Booting the kernel	235
11.1.1	Loading the kernel in physical memory (ARM)	235
11.1.2	First instructions (ARM)	235
11.1.3	Basic pagination (ARM)	236
11.1.4	Jumping to C <code>main()</code> (ARM)	238
11.2	Kernel initialization (ARM)	238
11.2.1	<code>cpu</code> initialization	240
11.2.2	Memory detection (ARM)	241
11.2.3	<code>conf</code> initialization (ARM)	241
11.2.4	Exception vector initialization (ARM)	243
11.2.5	Clock initialization (ARM)	245
11.2.6	Timer initialization	245
11.2.7	Virtual memory initialisation (ARM)	246
11.2.8	Memory initialisation	246
11.2.9	Device initialisation	250
11.2.10	Displaying hardware information (ARM)	250
11.3	The first process (ARM)	251
11.4	Booting the user	255
II	Everything is a File	256
12	Files	257
12.1	Overview	257
12.2	Channels	257
12.3	Devices table <code>devtab</code>	260
12.4	File descriptors, <code>Fgrp</code>	260
12.5	<code>sysopen()</code> , <code>sysclose()</code>	264
12.6	<code>namec()</code>	266
12.6.1	Checking valid filename string	269
12.6.2	Parsing filename	271
12.6.3	Error management	273
12.7	<code>syspread()</code> , <code>syspwrite()</code>	274
12.8	<code>sysseek()</code>	277
12.9	Collecting unclosed files	278
12.10	Special opening modes	280
12.10.1	Close on exec	281
12.10.2	Remove on close	281

13 Directories	282
13.1 Overview	282
13.2 Path	283
13.3 ..	285
13.4 walk()	286
13.5 syscreate()	290
13.6 sysremove()	293
13.7 syschdir()	294
13.8 sysfd2path() (a.k.a pwd())	294
13.9 DirEntry	295
13.10 sysstat()	296
14 Namespaces	300
14.1 Overview	300
14.2 Mount point	300
14.3 Mount table	302
14.4 Device path	304
14.5 sysbind()	304
14.6 /proc/#/ns	307
14.7 findmount(), domount()	309
14.8 Path mount points and ..	311
14.9 Union mount	313
14.9.1 Union walk	313
14.9.2 ->umh	313
14.9.3 syspread() for directories	314
14.9.4 MCREATE	316
14.9.5 Union of union	316
14.10 Sandboxing	317
14.11 sysmount()	318
14.12 sysumount()	319
14.13 XTODD	321
14.14 XTODD	323
15 Devices	328
15.1 The keyboard	328
15.1.1 Overview	328
15.1.2 Scan code	328
15.1.3 Esc	333
15.1.4 kbd	334
15.1.5 echo()	336
15.1.6 PC Keyboard (x86)	337
15.1.7 Raspberry Pi USB Keyboard (ARM)	337
15.1.8 Changing keyboard: /dev/kbmap	337
15.1.9 External keyboard input: /dev/kbin	338
15.2 The screen	338
15.2.1 Overview	338
15.2.2 Text output on a PC (x86)	338
15.2.3 Text output on a Raspberry Pi (ARM)	338
15.3 The console device /dev/cons	339
15.3.1 Overview	339

15.3.2	Buffered input	340
15.3.3	Output	341
15.3.4	<code>print()</code> , <code>iprint()</code> , <code>pprint()</code>	342
15.3.5	<code>/dev/cons</code>	344
15.3.6	<code>/dev/consctl</code> and raw input	347
15.4	The mouse	348
15.4.1	Mouse and keyboard	348
15.4.2	PS2 mouse (x86)	348
15.4.3	USB mouse (ARM)	348
15.5	Other devices	348
16	Queue	349
17	Filesystems	370
17.1	The root filesystem	370
17.2	The <code>/boot/</code> filesystem	374
17.3	Helpers	375
17.4	SimpleFS	381
17.5	Block	381
17.6	Fileservers	392
17.7	<code>Fcall</code>	392
17.8	<code>XTODO</code>	392
III	Beyond the Basics	393
18	IPC	394
18.1	Notes (signals)	394
18.1.1	<code>sysnotify()</code> (arm)	396
18.1.2	<code>sysnoted()</code> (arm)	398
18.1.3	<code>postnote()</code>	400
18.2	Pipes	401
18.2.1	<code>syspipe()</code>	401
18.2.2	<code>/dev/pipe</code>	403
18.3	Shared segment and semaphore	410
18.3.1	Segment	410
18.3.2	Semaphore	416
18.4	<code>sysrendezvous()</code>	421
18.5	Network	423
19	Debugging Support	424
19.1	Core dumps and broken processes	424
19.2	Stopping a process: <code>/proc/x/ctl</code>	426
19.3	Inspecting process memory: <code>/proc/x/mem</code>	429
19.4	<code>/bin/strace</code>	432
19.5	Scheduler events: <code>/proc/trace</code> and <code>/bin/trace</code>	439
19.6	<code>/bin/db</code>	443
19.7	Other	443

20 Profiling Support	444
20.1 /dev/cputime and /proc/#/status	444
20.2 /bin/time	445
20.3 /bin/tprof	446
20.4 /bin/prof	447
21 Userspace System Programs	450
21.1 /boot/boot	450
21.2 /root/init	450
21.3 The shell	450
21.3.1 #!	450
22 Advanced Topics	453
22.1 Floating point	453
22.1.1 FPSaveStatus	453
22.1.2 Arch_FPsave (ARM)	454
22.1.3 Floating point hooks	454
22.1.4 ARM floating point coprocessors (ARM)	457
22.1.5 coproc.c (ARM)	458
22.1.6 vfp.c (ARM)	460
22.1.7 mathxxx() (ARM)	466
22.1.8 /proc/x/fpregs	467
22.2 CPU cache	467
22.2.1 Instruction and data cache (ARM)	468
22.2.2 Cache policy	468
22.2.3 Paging and caching (ARM)	468
22.2.4 Memory barriers (ARM)	469
22.2.5 Cache operations (ARM)	469
22.2.6 cachexxx() (ARM)	469
22.3 Memory model	472
22.3.1 arch_coherence()	472
22.3.2 Memory barriers (ARM)	473
22.4 The Raspberry Pi 2 machine	473
22.4.1 CPU ID	473
22.4.2 MMU	474
22.4.3 TTB	474
22.4.4 Local registers	474
22.4.5 Timer	475
22.4.6 Performance monitor	475
22.5 SMP (ARM)	476
22.5.1 Initialization adjustments	476
22.5.2 Setting cpu	484
22.5.3 Concurrency adjustments	484
22.5.4 Timer adjustments	485
22.5.5 arch_coherence() (ARM)	486
22.5.6 Cache adjustments	486
22.5.7 IRQ adjustments	491
22.5.8 Cycle counter adjustments	492
22.6 Advanced virtual memory	492
22.6.1 Page coloring	492

22.6.2	Extra segment types	494
22.6.3	arch_kmap()	496
22.6.4	Extra checks	496
22.7	Fast interrupts (FIQ) (ARM)	498
22.7.1	Exception PsrMfiq and associated processor mode	498
22.7.2	IRQfiq and vfiq	499
22.7.3	fiq()	500
22.8	Power management (ARM)	500
22.9	Alignments (ARM)	501
22.10	Real-time scheduling	502
22.11	Fileservers	517
22.11.1	Mnt	517
22.11.2	MntRpc	519
22.11.3	Fcall	520
22.11.4	mntversion()	520
22.11.5	mountrpc()	522
22.11.6	devmnt	524
22.12	Network	527
22.13	Security	527
22.14	Graphics	529
23	Special Fileservers	530
23.1	Cmdtab	530
23.2	/env/	532
23.2.1	Egrp	532
23.2.2	/env/	534
23.3	/proc/	535
23.3.1	/proc/x/ctl	535
23.3.2	/proc/x/mem	535
23.3.3	/proc/x/status	535
23.3.4	/proc/x/args	535
23.3.5	/proc/x/wait	535
23.3.6	Other /proc/x/xxx	536
23.3.7	XXX	536
23.4	/fd/	555
23.4.1	sysdup()	556
23.4.2	/fd/	556
23.5	/sys/	556
23.5.1	/sys/osversion	558
23.5.2	/sys/config	559
23.5.3	/ys/hostowner, /sys/hostdomain	559
23.5.4	/sys/sysname	559
23.5.5	/sys/drivers	560
23.5.6	/sys/reboot	560
23.6	/dev/	567
23.6.1	/dev/random	568
24	Conclusion	571
24.1	Connections to other books	571
24.2	What this book does not cover	572

A	Debugging	573
A.1	sysnop()	573
A.2	Special keys and dumpers	573
A.3	/bin/ktrace (ARM)	579
A.4	/dev/kmesg	581
A.5	/dev/kprint	582
A.6	Remote debugger	583
A.7	Serial line	584
A.7.1	serialoq	584
A.7.2	uartconsinit()	585
A.7.3	uartputs()	585
A.7.4	consuart	585
A.7.5	Uart, PhysUart	586
B	Profiling	587
B.1	/sys/sysstat (ARM)	587
B.2	/dev/kprof	590
B.3	Statistics	591
B.3.1	Lock.lockcycles	591
B.3.2	TaslockStats	591
B.3.3	QlockStats	592
B.3.4	Kimage statistics	592
B.3.5	Scheduling	592
B.3.6	Clock	592
C	Error Management	594
C.1	panic()	594
C.2	error()	595
C.3	waserror(), nexterror(), poperror()	597
C.4	syserrstr()	597
D	Utilities	599
D.1	Memory areas operations	599
D.2	String functions	599
D.3	Conversion functions	601
D.4	Pool allocation	601
D.5	Printf and Fmt	601
D.6	Runes, a.k.a unicode	602
E	Extra Code	604
E.1	arch/	604
E.2	buses/	604
E.3	concurrency/	604
E.3.1	concurrency/portdat_concurrency.h	604
E.3.2	concurrency/portfns_concurrency.h	605
E.3.3	concurrency/taslock.c	605
E.3.4	concurrency/qlock.c	606
E.3.5	concurrency/ref.c	606
E.4	console/	607
E.4.1	console/portdat_console.h	607

E.4.2	console/portfns_console.h	607
E.4.3	console/devcons.c	607
E.5	core/	609
E.5.1	core/portdat_forward.h	609
E.5.2	core/portdat_core.h	609
E.5.3	core/portfns_core.h	610
E.5.4	core/portdat.h	610
E.5.5	core/error.h	610
E.5.6	core/error.c	610
E.5.7	core/portdat.c	610
E.5.8	core/portfns.c	611
E.6	devices/	611
E.6.1	devices/portdat_devices.h	611
E.6.2	devices/portfns_devices.h	612
E.6.3	devices/keyboard/portkbd.c	612
E.6.4	devices/keyboard/latin1.c	613
E.6.5	devices/keyboard/latin1.h	614
E.6.6	TO LP SPLIT devices/keyboard/devkbin.c	614
E.6.7	TO LPSPLIT devices/keyboard/devkbmap.c	616
E.6.8	TO LPSPLIT devices/mouse/portmouse.c	619
E.6.9	TO LPSPLIT devices/mouse/devmouse.c	623
E.6.10	devices/sys/devroot.c	633
E.6.11	devices/sys/devsys.c	635
E.6.12	TO LPSPLIT devices/sys/devenv.c	636
E.6.13	devices/sys/devpipe.c	642
E.6.14	devices/sys/devproc.c	643
E.7	files/	645
E.7.1	files/portdat_files.h	645
E.7.2	files/portfns_files.h	647
E.7.3	files/chan.c	647
E.7.4	files/dev.c	649
E.7.5	files/allocb.c	650
E.7.6	files/env.c	651
E.7.7	files/file.c	651
E.7.8	files/mnt.c	651
E.7.9	files/qio.c	652
E.7.10	files/sysfile.c	654
E.8	filesystems/	655
E.9	init/	655
E.9.1	init/portfns_init.h	655
E.9.2	init/user/preboot/initcode.c	655
E.9.3	init/user/boot/aux.c	655
E.9.4	init/user/boot/boot.c	655
E.9.5	init/user/boot/boot.h	655
E.9.6	init/user/boot/local.c	655
E.10	lib/	655
E.10.1	lib/lib.h	655
E.11	memory/	658
E.11.1	memory/portdat_memory.h	658

E.11.2	memory/portfns_memory.h	660
E.11.3	memory/portdat_memory.c	660
E.11.4	memory/xalloc.c	660
E.11.5	memory/pool.c	661
E.11.6	memory/alloc.c	661
E.11.7	memory/fault.c	663
E.11.8	memory/page.c	664
E.11.9	memory/segment.c	665
E.11.10	memory/swap.c	666
E.11.11	memory/sysmemory.c	667
E.12	misc/	667
E.12.1	misc/portdat_misc.h	667
E.12.2	misc/portfns_misc.h	667
E.12.3	misc/parse.c	667
E.12.4	misc/random.c	668
E.13	interrupts/	668
E.13.1	interrupts/portfns_interrupts.h	668
E.14	processes/	668
E.14.1	processes/portdat_processes.h	668
E.14.2	processes/portfns_processes.h	671
E.14.3	processes/portdat_processes.c	671
E.14.4	time/portfns_time.h	671
E.14.5	time/portdat_time.h	672
E.14.6	time/alarm.c	672
E.14.7	time/portclock.c	673
E.14.8	processes/pgrp.c	673
E.14.9	processes/proc.c	674
E.14.10	time/tod.c	678
E.14.11	processes/sysproc.c	679
E.14.12	processes/syssema.c	679
E.15	security/	681
E.15.1	security/portfns_security.h	681
E.15.2	security/auth.c	681
E.16	syscalls/	682
E.16.1	syscalls/systab.h	682
E.16.2	syscalls/syscallfmt.c	682
E.16.3	syscalls/systab.c	682
E.17	../include/arch/arm/	683
E.17.1	../include/arch/arm/ureg.h	683
E.18	arch/arm/	683
E.18.1	arch/arm/dat_arch.h	683
E.18.2	arch/arm/arm.h	683
E.18.3	arch/arm/arminstr.h	684
E.18.4	arch/arm/io.h	685
E.18.5	arch/arm/coproc.c	685
E.18.6	arch/arm/vcore.c	687
E.19	concurrency/arm/	692
E.19.1	concurrency/arm/spl.s	692
E.19.2	concurrency/arm/tas_raspi1.s	693

E.19.3	concurrency/arm/tas_raspi2.s	693
E.19.4	concurrency/arm/concurrency_raspi1.c	693
E.19.5	concurrency/arm/concurrency_raspi2.c	694
E.20	core/arm/	694
E.20.1	core/arm/dat.h	694
E.20.2	core/arm/dat_core.h	695
E.20.3	core/arm/dat_forward.h	695
E.20.4	core/arm/fns.h	695
E.21	buses/arm/	698
E.22	devices/	698
E.22.1	devices/keyboard/arm/kbd.c	698
E.22.2	devices/mouse/arm/mouse.c	698
E.22.3	devices/sys/arm/devarch.c	699
E.23	init/arm/	703
E.23.1	init/arm/start.s	703
E.23.2	init/arm/startv6.s	703
E.23.3	init/arm/startv7.s	703
E.23.4	init/arm/main.c	704
E.23.5	init/arm/raspi1.c	706
E.23.6	init/arm/raspi2.c	707
E.23.7	init/user/preboot/arm/init9.s	708
E.23.8	init/arm/rebootcode.s	708
E.24	memory/arm/	709
E.24.1	memory/arm/dat_memory.h	709
E.24.2	memory/arm/mem.h	709
E.24.3	memory/arm/mmu.c	710
E.24.4	memory/arm/mmuinvalidate.s	711
E.24.5	memory/arm/cache_raspi1.s	711
E.24.6	memory/arm/cache_raspi2.s	712
E.25	interrupts/arm/	712
E.25.1	interrupts/arm/trap.c	712
E.25.2	interrupts/arm/fault_helpers.s	713
E.25.3	interrupts/arm/lexception.s	714
E.26	time/arm/	714
E.26.1	time/arm/dat_time.h	714
E.26.2	time/arm/clock.c	714
E.26.3	time/arm/time_raspi1.s	716
E.26.4	time/arm/time_raspi2.s	716
E.27	processes/arm/	716
E.27.1	processes/arm/dat_processes.h	716
E.27.2	processes/arm/arch.c	717
E.27.3	processes/arm/vfp3.c	717
E.27.4	processes/arm/lproc.s	719
E.27.5	processes/arm/switch.s	719
E.28	syscalls/arm/	719
E.28.1	syscalls/arm/syscall.c	719
E.29	Extra files	719
E.29.1	init/arm/bootconf.c	719
E.29.2	devices/storage/arm/emmc.c	724

E.29.3	init/arm/watchdog.c	732
E.29.4	buses/arm/dwcotg.h	733
E.29.5	buses/arm/devusb.c	742
E.29.6	buses/arm/usbdwc.c	770
E.29.7	buses/arm/dma.c	789
E.29.8	buses/arm/uartmini.c	794

Glossary	804
-----------------	------------

Indexes	806
----------------	------------

References	806
-------------------	------------

Chapter 1

Introduction

The goal of this book is to explain with full details the source code of a kernel.

1.1 Motivations

Why a kernel? Because I think you are a better programmer if you fully understand how things work under the hood, and the kernel is the program that everything else rely on.

Indeed, every program you write—from a simple `hello` to a web server—relies on the kernel for its most basic operations: reading files, allocating memory, communicating with other programs, and even printing text on the screen. Understanding what happens below those system calls removes an entire layer of mystery and gives you a far more accurate mental model of how software actually executes on hardware.

Moreover, the kernel is a fascinating program in its own right. It must manage concurrency (multiple processes sharing the CPU), virtual memory (giving each process the illusion of having all memory for itself), and devices (keyboards, disks, screens)—all while remaining small enough to be trustworthy and fast enough to stay out of the way. The tension between generality and efficiency produces elegant solutions that are worth studying even if you never write kernel code yourself. Finally, many concepts introduced by the kernel—locks, scheduling, caches, interrupt handling—reappear in user-space programs such as databases, web servers, and even build systems. Understanding the kernel is thus also a way to understand those programs better.

Here are a few questions I hope this book will answer:

- What happens when the user turn on the computer? Which code gets executed? How does the kernel initialise itself?
- What happens when the user types `ls` in a terminal? What are the set of programs involved in such a command? What is the trace of such a command through the different layers of the software stack, from the keyboard interrupt to the display of text glyphs on the screen?
- Where does the kernel store meta data about the user processes?
- Why do the kernel code and some kernel data structures reside in the virtual memory space of every processes?
- What is the trace of a system call?
- What is the memory image of the kernel and the user processes?

1.2 The Plan 9 kernel: 9

Like for most books in Principia Softwarica, I chose a Plan 9 program. The Plan 9 kernel, called 9, was written by Ken Thompson—the creator of UNIX—together with Dave Presotto and others at Bell Labs. If UNIX was “version 1,” then Plan 9 is “version 2”: a chance to redesign an operating system with decades of hindsight. The result is a kernel that is remarkably small, clean, and coherent—a good basis for the other Principia Softwarica books.

I will focus on the code of 9 for the Raspberry Pi, which uses the ARM architecture. ARM is a RISC design, far simpler than x86, making it a better target for a teaching book. Sections covering ARM-specific code are marked with “(ARM)” in their title, but most of the code in this book is portable C—the same source is shared between the ARM and x86 ports of 9.

The graphics stack and the networking stack are covered in separate Principia Softwarica volumes (GRAPHICS book [Pad16d] and NETWORK book [Pad16e]), so this book focuses on the kernel’s core: processes, memory, files, and devices.

1.3 Other kernels

Here are a few kernels that I considered for this book, but which I ultimately discarded:

- Unix V6 (or its modern incarnation xv6) is too simple; there is no graphics, no network. Even though this book also does not cover graphics or networking (those are in GRAPHICS book [Pad16d] and NETWORK book [Pad16e]), the kernel presented here is the same one that supports those subsystems.
- XINU has a network stack but the kernel itself is too simple with no virtual memory for instance
- Minix is fairly small but it does not provide a simple windowing system as it relies on X11 (which is far more complicated than Plan 9 Rio windowing system)
- Linux is really big. Of course the Linux kernel contains thousands of specific device drivers and those things could be discarded when presenting the core of the Linux kernel, but this core is still far bigger than Plan 9 equivalent core.

Figure 1.1 presents a timeline of major kernels. I think 9 represents the best compromise for this book: it implements the essential features of an operating system kernel—processes, virtual memory, file systems, device drivers—while still having a small and understandable codebase. 9 is obviously not as used as the Linux kernel, but it is a production-quality kernel that was used to run all machines at Bell Labs and that still runs today on hobbyist hardware such as the Raspberry Pi.

1.4 Getting started

To get started you first need to get the source of the Plan 9 version described in this book. See <https://www.principia-softwarica.org/getting-started.html>. Then you need to compile using a regular C compiler (e.g. gcc) the Plan 9 C cross compilers written by Ken Thompson: <https://github.com/aryx/goken9cc>. Then you need to cross compile, e.g. from Linux or macOS, the Plan 9 kernel using the Plan 9 C cross compilers installed in the previous step. You can also cross compile the whole Plan 9 operating system with all its libraries and utilities. Finally you can test this compiled kernel and utilities by installing them on a SD card and by booting this SD card on a Raspberry Pi.

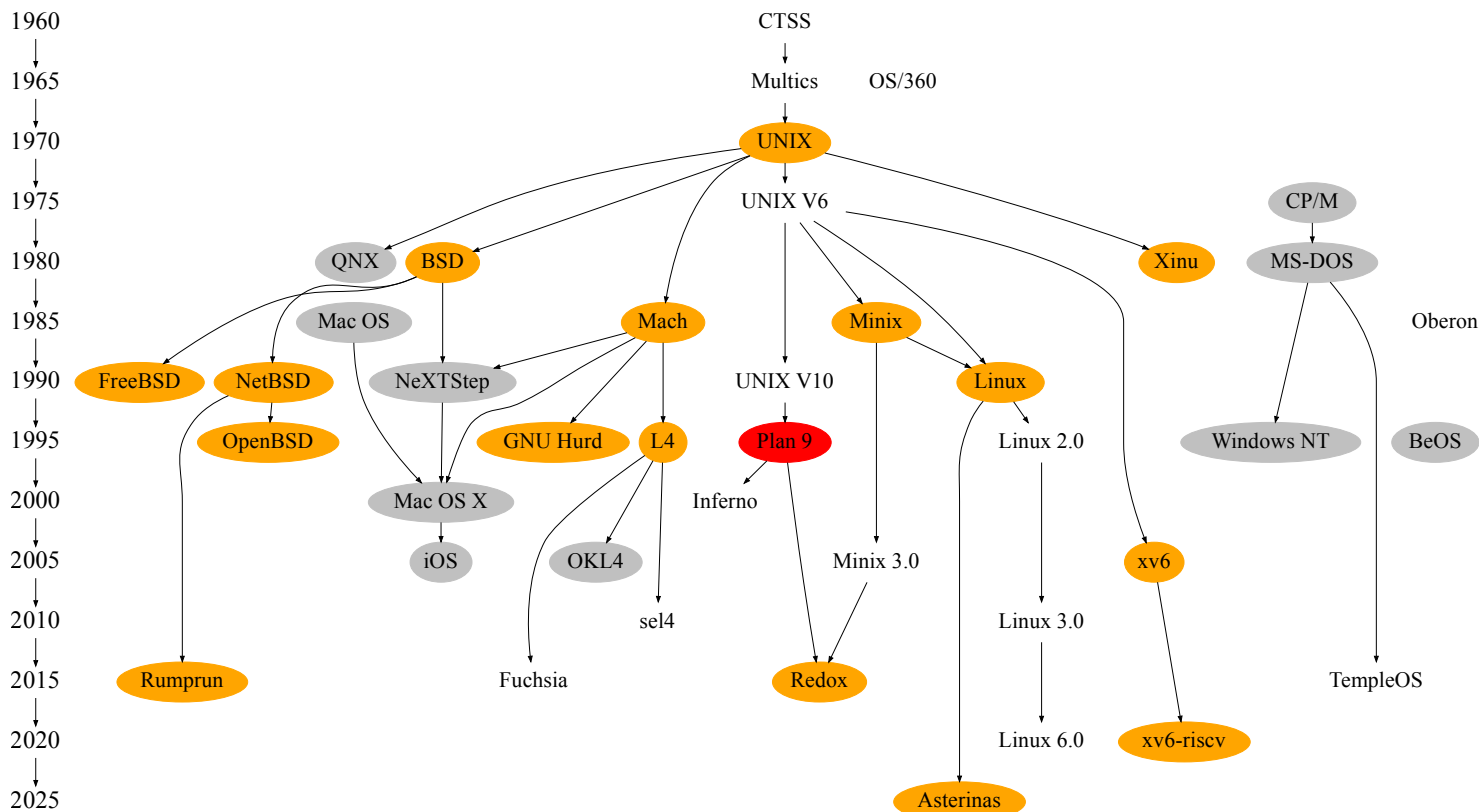


Figure 1.1: Kernels timeline

1.5 Requirements

I assume you actually know a lot:

- **C:** most of this book is made of C source code, so you'll need to have a good knowledge of the C programming language [KR88]. In fact, kernel C is often more demanding than application C: the code makes heavy use of bit manipulation, pointer arithmetic, C idioms for implementing data structures (linked lists via embedded pointers, bitsets, union-based tagged structures), and subtle interactions with the hardware. A very good knowledge of C is required.
- **Operating system:** This book is not an introduction to operating systems. The goal of this book is to explain with full details a kernel but we assume you already have a vague idea of how an operating system works and so that you are familiar with concepts such as virtual memory, critical regions, interrupts, system calls, etc. I assume you already know most of the theory; this book is here to cover the practice.

The following are helpful but not strictly required:

- **The ARM architecture:** some familiarity with the ARM processor helps for the architecture-specific chapters, but most of the kernel code is portable C. The EMULATOR book [Pad15b] and ASSEMBLER book [Pad15a] can serve as references.
- **Assembly:** a few key routines (context switch, interrupt entry, boot code) are written in assembly. The ASSEMBLER book [Pad15a] covers the Asm5 language used here.

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, **syncweb** [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

I would like to acknowledge of course Plan9's authors who wrote most of this book: Ken Thompson, Rob Pike, Dave Presotto, Phil Winterbottom, Russ Cox, and many other people from Bell Labs. Thanks also to Federic Balesteros who wrote a lengthy note about the Plan 9 kernel. Finally, thanks to Richard Miller for porting the Plan 9 kernel to the Raspberry Pi machine.

Chapter 2

Overview

In this chapter, I will give a bird’s-eye view of the 9 kernel before diving into the code in later chapters. I will first recall general operating system principles, then present the syscall interface of 9, the ARM hardware the kernel runs on, the memory layout, and finally the overall software architecture and boot sequence.

2.1 Operating system principles

An operating system is a program that manages other programs. It provides convenient abstractions of the hardware and multiplex the hardware resources for the benefit of user programs. The operating system usually provides:

- a virtual CPU, so that each process thinks it has all the CPU for itself (via preemptive scheduling and the timer interrupt)
- a virtual memory, so that each process thinks it has all memory for itself (via hardware virtual memory features such as paging)
- a virtual screen, so that each process thinks it has all the screen for itself (via the windowing system)
- a file abstraction, so a process can read and store data whether it’s on a disk, network, floppy, etc
- a directory abstraction, so a process can organize data whether it’s on a disk, network, floppy, etc

The common thread is virtualization: each of these abstractions gives a process the illusion of exclusive access to a resource. Making this illusion work is the kernel’s main job, and it requires solving several hard problems: concurrency (multiplexing the CPU safely), memory management (giving each process its own address space), interrupt handling (reacting to hardware events), and persistence (storing data reliably on disk). These are exactly the topics the rest of this book will cover.

Entire books have been written on those subjects. I assume you already know what is an operating system and we will now focus on the concrete services (API) a kernel provides to the user programs.

2.2 9’s syscall interface

Plan 9 is the successor to UNIX and inherits its philosophy that “everything is a file.” Plan 9 pushes this further: “everything is a file server”. Devices, the console, the screen, the process table—all are accessed through `open/read/write/close`, served by in-kernel or user-space file servers. This design lets the syscall interface remain remarkably small.

The entire interface fits in a small single header, `sys.h`, which lives in `lib_core/libc/9syscall/`—it is shared between the kernel and the C library (see LIBCORE book [Pad16c]) because it defines the interface

between them. The same directory also contains the auto-generated assembly wrappers (one per syscall) that user programs link against. Each syscall is a small integer used as an index into a dispatch table (the `systab`^{41f} array shown later).

```
(lib_core/libc/9syscall/sys.h 22a)≡
#define NOP      0
<sys.h process syscalls 22b>
<sys.h memory syscalls 22c>
<sys.h file syscalls 22d>
<sys.h directory syscalls 23a>
<sys.h namespace syscalls 23b>
<sys.h time syscalls 23c>
<sys.h ipc syscalls 23d>
<sys.h concurrency syscalls 23e>
<sys.h special file syscalls 24a>
<sys.h security syscalls 24b>
#define ERRSTR   39
```

Only 39 syscalls! Modern Linux has more than 400. The reason Plan 9 can get away with so few is the “everything is a file server” philosophy: operations that would require dedicated syscalls in other systems (e.g., `getpid`, `ioctl`) are instead handled by reading or writing special files.

The `NOP` syscall at position 0 does nothing useful in production—here it just prints “Hello World” for testing purposes. The `ERRSTR` syscall at position 39 is Plan 9’s error-reporting mechanism. Unlike UNIX, which uses the opaque numeric error code `errno` (where you need a man page to find out what `ENOENT` means), Plan 9 uses human-readable error strings that programs retrieve via `errstr()`—one of the many small cleanups Plan 9 made over UNIX.

The process syscalls mirror the UNIX model: `RFORK` creates a new process (a generalization of UNIX `fork`), `EXEC` replaces the current program, `EXITS` terminates, and `AWAIT` waits for a child (like UNIX `wait`).

```
(sys.h process syscalls 22b)≡ (22a)
#define RFORK      1
#define EXEC       2
#define EXITS      3
#define AWAIT      4
```

Plan 9 cleans up UNIX here too. Instead of having separate `fork()` and `clone()` syscalls (as Linux does), Plan 9 has a single `RFORK` (“resource fork”) that takes flags specifying which resources to copy and which to share: memory, file descriptors, environment, namespace, etc. A traditional `fork` is `RFORK` with everything copied; a thread (shared memory) is `RFORK` with memory shared.

Similarly, `EXITS` takes an error string (or `nil` for success), not a numeric exit code. The parent reads this string via `AWAIT`. No need to memorize that exit code 2 means “file not found”—the string says it directly.

There is only one memory syscall: `BRK` (for “break”), which moves the boundary between the data segment and free memory. The name comes from the traditional UNIX term for the end of the process’s data area. There is no `malloc` or `free` syscall—those are implemented entirely in user space (in the C library, see [LIBCORE](#) book [Pad16c]) on top of `brk`. The library requests large chunks of memory from the kernel via `brk` and then subdivides them.

```
(sys.h memory syscalls 22c)≡ (22a)
#define BRK        5
```

The file syscalls are the workhorses of the interface. Because so many things in Plan 9 are files, most of the kernel’s services are accessed through these five calls. Note `PREAD` and `PWRITE` instead of the traditional UNIX `read/write`: the ‘p’ stands for “positional”—they take an explicit file offset, making seek-then-read atomic. This is another cleanup: in UNIX, `read` and `seek` are separate syscalls, so a concurrent `seek+read` from two threads can race. Plan 9 avoids this by combining them into a single `pread` call.

```
(sys.h file syscalls 22d)≡ (22a)
#define OPEN       6
```

```
#define CLOSE      7
#define PREAD      8
#define PWRITE     9
#define SEEK      10
```

The directory syscalls include `CREATE`, `REMOVE`, `CHDIR`, and a family of `STAT` variants. The proliferation of `STAT`/`FSTAT`/`WSTAT`/`FWSTAT` comes from two axes: `STAT` vs. `FSTAT` (take a path name vs. a file descriptor), and read vs. write (“w” for write, i.e., change metadata). `FD2PATH` is a Plan 9 addition that returns the file name associated with an open file descriptor—useful because in Plan 9, with per-process namespaces, a file descriptor is often more reliable than a path. Note `CREATE` with the full spelling—in UNIX, the syscall was famously called `creat` without the final ‘e’. When asked what he would change about UNIX, Ken Thompson once answered: “I’d spell `creat` with an e.” Plan 9 finally fixed this.

```
<sys.h directory syscalls 23a>≡ (22a)
#define CREATE      11
#define REMOVE      12
#define CHDIR       13
#define FD2PATH     14
#define STAT        15
#define FSTAT       16
#define WSTAT       17
#define FWSTAT      18
```

The namespace syscalls below are specific to Plan 9. `BIND` and `MOUNT` let each process customize its own view of the file system by overlaying directories or attaching file servers at arbitrary mount points. This is what makes `/dev/cons`, `/dev/draw`, etc. work cleanly: each process can have its own `/dev` populated by different servers.

```
<sys.h namespace syscalls 23b>≡ (22a)
#define BIND        19
#define MOUNT       20
#define UNMOUNT     21
```

```
<sys.h time syscalls 23c>≡ (22a)
#define SLEEP       22
#define ALARM       23
```

The IPC syscalls cover several mechanisms. `PIPE` creates a pipe—one of UNIX’s greatest inventions. `NOTIFY`/`NOTED` handle notes, Plan 9’s equivalent of UNIX signals. The `SEG` family (`SEGATTACH`, `SEGDETACH`, etc.) lets processes attach shared memory segments, similar to UNIX `mmap`.

```
<sys.h ipc syscalls 23d>≡ (22a)
#define PIPE        26

#define NOTIFY      24
#define NOTED       25

#define SEGATTACH   27
#define SEGDETACH   28
#define SEGFREE     29
#define SEGFLUSH    30
#define SEGBRK      31
```

The concurrency syscalls provide low-level primitives for user-level synchronization. `RENDEZVOUS` is a meeting-point mechanism, and `SEMACQUIRE`/`SEMRELEASE` provide kernel-backed semaphores. These are the building blocks on top of which the user-level thread library (`libthread`) is implemented.

```
<sys.h concurrency syscalls 23e>≡ (22a)
#define RENDEZVOUS  32
#define SEMACQUIRE 33
#define SEMRELEASE  34
#define TSEMQUIRE  35
```



```
⟨sys.h special file syscalls 24a⟩≡ (22a)
#define DUP          36
```

The two security-related syscalls handle Plan 9’s authentication protocol. **FVERSION** negotiates the 9P protocol version with a file server, and **FAUTH** opens an authentication channel. These are used when mounting remote file servers and are part of Plan 9’s security model, which relies on per-connection authentication rather than on a global user database.

```
⟨sys.h security syscalls 24b⟩≡ (22a)
#define FVERSION      37
#define FAUTH         38
```

And that is the entire syscall interface. There is no **getpid**—a process reads `/dev/pid` instead. There is no **ioctl**—device control is done by writing commands to a file. There is no **select**—**libthread**’s **alt()** uses **rendezvous** instead. The file server abstraction absorbs what would otherwise be dozens of special-purpose syscalls.

2.3 helloraw.c and helloraw.s

To see the syscall interface concretely, it is always good to start with the simplest possible program. Here is a “Hello World” that uses **pwrite()** directly, without going through libc **print()**:

```
⟨helloraw.c 24c⟩≡
#include <u.h>
#include <libc.h>

void main() {
    pwrite(1, "hello world\n", 12, 0LL);
    exits(nil);
}
```

The **pwrite()** call writes 12 bytes to file descriptor 1 (standard output) at offset 0.¹ Under the hood, **pwrite()** is a thin assembly wrapper that places the syscall number (**PWRITE**, which is 9 from **sys.h**) in register **R0** and executes a **SWI** (software interrupt) instruction to trap into the kernel. The assembly version makes this mechanism fully visible:

```
⟨helloraw.s 24d⟩≡
TEXT _main(SB), $20
    /* pwrite(1, &hello, 12, 0LL) */
    MOVW $1, R1
    MOVW R1, 4(R13)
    MOVW $hello(SB), R1
    MOVW R1, 8(R13)
    MOVW $12, R1
    MOVW R1, 12(R13)
    MOVW $0, R1
    MOVW R1, 16(R13)
    MOVW R1, 20(R13)
    MOVW $9 /*PWRITE*/, R0
    SWI $0

    /* exits(0) */
    MOVW $0, R1
    MOVW R1, 4(R13)
    MOVW $3 /*EXITS*/, R0
    SWI $0
```

¹The 0LL suffix makes the offset a 64-bit long long, because Plan 9’s **pwrite** takes a 64-bit file offset, allowing files larger than 4 GiB.

```

GLOBL    hello(SB), $12
DATA     hello+0(SB)/8, $"hello wo"
DATA     hello+8(SB)/4, $"rld\n"

```

To compile and run the C version:

```

$ 5c helloraw.c
$ 5l helloraw.5 -o helloraw
$ ./helloraw
hello world

```

For the assembly version:

```

$ 5a helloraw.s
$ 5l helloraw.5 -o helloraw
$ ./helloraw
hello world

```

You can also use `ratrace`, the Plan 9 equivalent of `strace`, to see the syscalls executed by the program:

```

$ ratrace -c ./helloraw
...
Pwrite 9, "hello world\n", 12
...
Exits  0

```

This confirms that our small program generates exactly the two system calls we expected: `PWRITE` and `EXITS`. We will see in Chapters 7 and 6 how the kernel handles the `SWI` instruction, dispatches the syscall number through `systab`, and executes the corresponding kernel function.

2.4 minishell.c

A slightly more interesting example is a minimal shell that illustrates several syscalls working together:

```

<minishell.c 25>≡
#include <u.h>
#include <libc.h>

void main() {
    char buf[256];
    char *args[2];
    int n;

    for(;;) {
        pwrite(1, "$ ", 2, 0LL);
        n = pread(0, buf, sizeof(buf)-1, 0LL);
        if(n <= 0)
            break;
        buf[n-1] = '\0'; /* strip newline */
        args[0] = buf;
        args[1] = nil;
        if(rfork(RFPROC|RFFDG) == 0) {
            exec(buf, args);
            exits("exec failed");
        }
        await(buf, sizeof(buf));
    }
    exits(0);
}

```

This program reads commands from standard input (file descriptor 0), forks a child with `rfork`, loads the requested program with `exec`, and waits for it to finish with `await`. Running it under `ratrace` reveals the full sequence of syscalls for a single command—a preview of the machinery this book will explain.

2.5 The Raspberry Pi machine (ARM)

The target hardware for this book is the Raspberry Pi Model B, a cheap hobbyist board designed for education. It is built around a Broadcom BCM2835 System-on-Chip (SoC) with an ARM1176JZF-S processor implementing the ARMv6 instruction set (32 bits). In the rest of this book, “Raspberry Pi” and “ARM” refer specifically to this board and this processor unless noted otherwise.

2.5.1 ARM CPU

The ARM is a RISC processor, far simpler than the x86 CISC architecture. This makes it a better target for a teaching book. ARM is also the most popular architecture by number of devices shipped—it powers most smartphones—so understanding it is useful beyond this book.

ARM is a 32 bits architecture, hence the constant definition below “bytes per word”. A word is the natural unit of data for a processor—the size of a general-purpose register and typically the width of the data bus. The term dates back to early computing where a “word” was the fundamental chunk of data a machine could process in one operation. On a 32-bit ARM, a word is 4 bytes.

```
<constant BY2WD(arm) 26a>≡ (709b)
#define BY2WD 4
```

BY2V stands for “bytes per vlong.” A `vlong` (“very long”) is Plan 9’s name for a 64-bit integer. In Plan 9, `long` is always 32 bits regardless of the architecture (unlike `gcc` where `long` is 64 bits on 64-bit systems). So `vlong` (equivalent to C99’s `long long`) is 8 bytes.

```
<constant BY2V(arm) 26b>≡ (709b)
#define BY2V 8 /* only used in xalloc.c */
```

Main registers, Ureg

The ARM has 16 general-purpose 32-bit registers (R0–R15), of which R13 is the stack pointer (SP), R14 the link register (return address), and R15 the program counter (PC). The `Ureg` structure below captures the full register state of a user process. The kernel saves a `Ureg` on the stack every time it takes control from a user process (on a syscall, interrupt, or exception), and restores it when returning.

```
<struct Ureg(arm) 26c>≡ (683a)
//coupling: do not change the order! some assembly code assumes this order.
struct Ureg {
    ulong r0;
    ulong r1;
    ulong r2;
    ulong r3;
    ulong r4;
    ulong r5;
    ulong r6;
    ulong r7;
    ulong r8;
    ulong r9;
    ulong r10;
    ulong r11;
    ulong r12; /* sb */
```

```

union {
    ulong    r13;
    ulong    sp;
};
union {
    ulong    r14;
    ulong    link;
};

ulong    type;    /* of exception */
ulong    psr;

ulong    pc; /* interrupted addr */ // r15
};

⟨function getcallerpc(arm) 27⟩≡ (719b)
TEXT getcallerpc(SB), 1, $-4
    MOVW    0(R13), R0
    RET

```

Program Status Register (PSR)

In addition to the general-purpose registers, the ARM has a Program Status Register (PSR) that contains both the arithmetic flags (negative, zero, carry, overflow) and control bits (processor mode, interrupt disable). Normal user programs never access the PSR directly, but the kernel must save and restore it—this is the `Ureg.psr` field—because the flags are part of the process state. The control bits are critical too: they determine whether the processor is in user or supervisor mode, and whether interrupts are enabled.

The ARM actually has two copies of the PSR. The CPSR (Current Program Status Register) reflects the current state. The SPSR (Saved Program Status Register) is a shadow copy that the processor fills automatically when entering an exception: it saves the old CPSR so the handler can restore it when returning. Both have the same layout:

31	30	29	28	27	...	8	7	6	5	4	0	
+-----+-----+-----+-----+ ... +-----+-----+-----+-----+												
N	Z	C	V			I	F		T	Mode		
+-----+-----+-----+-----+ ... +-----+-----+-----+-----+												
										---	Psrmusr (0x10) = user	
											Psrmsvc (0x13) = supervisor	
											Psrmirq (0x12) = IRQ	
											Psrmabt (0x17) = abort	
											etc.	
										---	Thumb mode (0=ARM, 1=Thumb)	
										---	Psrdfiq (0x40): disable FIQ	
										---	Psrdirq (0x80): disable IRQ	
				---	V: overflow							
			---	C: carry								
		---	Z: zero									
---	N: negative											

Processor modes

The low bits of the PSR encode the processor mode. The ARM supports several modes, but the kernel cares mainly about two: user mode (**PsrMusr**), where normal programs run with restricted privileges, and supervisor mode (**PsrMsvc**), where the kernel runs with full access to hardware. When an exception (e.g., a SWI) occurs, the processor automatically switches to supervisor mode; the kernel must switch back to user mode before returning to the user program. The mode field is 5 bits wide (bits 4–0), but Bit 4 is always set in the defined modes (it distinguishes “32-bit modes” from legacy 26-bit modes). Bits 3–0 select the specific mode.

Bit 5 (the T bit) selects Thumb vs. ARM instruction mode. Thumb is a compact 16-bit instruction encoding that ARM processors can use to reduce code size. The Plan 9 kernel does not use Thumb mode—all kernel and user code runs in ARM mode (32-bit instructions)—so we can ignore the T bit in this book.

```
<type PsrMode(arm) 28a>≡ (683c)
// Last 5 bits of PSR
#define PsrMusr      0x00000010 /* user mode */
#define PsrMsvc      0x00000013 /* ‘protected mode for OS’ */
<PsrMode other cases(arm) 139a>

#define PsrMask      0x0000001F
```

The PSR also controls whether interrupts are enabled. Setting **PsrDirq** disables IRQ interrupts. The kernel uses this to protect critical sections where it must not be preempted—for instance, while manipulating the scheduler’s run queue.

```
<type PsrDisable(arm) 28b>≡ (683c)
#define PsrDirq      0x00000080 /* disable IRQ interrupts */
<PsrDisable other cases(arm) 499b>
```

Exceptions

A note on terminology: the ARM uses the word exception as the general term for any event that diverts the processor from normal execution. This includes interrupts (external hardware signals, called IRQ for Interrupt Request), software interrupts (the SWI instruction used for syscalls), and aborts (page faults, undefined instructions). Each exception type has its own entry in the vector table and its own processor mode.

When an exception occurs, the ARM processor automatically saves the current CPSR into the SPSR, switches to the appropriate privileged mode, and jumps to a fixed address in the exception vector table. This table is normally at physical address 0, but the kernel configures the ARM (via the **CpChv** bit in the system control register) to use “high vectors” at **0xFFFF0000**. This is more convenient both because it lives in the kernel’s virtual address space, and because it frees physical address 0 for the null pointer trap (the first page of user virtual memory is left unmapped).

Of course, **0xFFFF0000** is a virtual address, so once the MMU is enabled the hardware itself relies on the MMU to translate it to the physical memory containing the vector code. This is not a chicken-and-egg problem: the kernel sets up the page tables and copies the vector code to physical page 0 first (while still using physical addresses or identity-mapped memory during early boot), then enables the MMU and **CpChv** together. From that point on, any exception causes the ARM to fetch the handler from virtual **0xFFFF0000**, which the MMU translates to physical 0.

```
<constant HVECTORS(arm) 28c>≡ (683c)
#define HVECTORS      0xffff0000
```

2.5.2 ARM coprocessors

Beyond the main registers and the PSR, the ARM provides access to coprocessors that control hardware features such as the MMU, caches, and performance counters. Coprocessor registers are accessed via special instructions

(MRC to read, MCR to write) and are only available in privileged mode. Normal user programs never touch them—they are the kernel’s private interface to the hardware.

```
<type Coprocessors(arm) 29a>≡ (683c)
#define CpSC      15      /* System Control */
<Coprocessors other cases(arm) 457c>
```

System control coprocessor (CpSC)

The system control coprocessor (CpSC, coprocessor 15) is the most important for the kernel. Its registers control the MMU (CpTTB for the page table base, CpTLB for TLB management), report page faults (CpFSR for fault status, CpFAR for the faulting address), and configure miscellaneous features such as caches and the exception vector location.

```
<type CpSC primary registers(arm) 29b>≡ (683c)
#define CpCONTROL  1      /* miscellaneous control */
#define CpTTB      2      /* Translation Table Base(s) */
#define CpDAC      3      /* Domain Access Control */
#define CpFSR      5      /* Fault Status */
#define CpFAR      6      /* Fault Address */
#define CpTLB      8      /* TLB control */
#define CpSPM      15     /* system performance monitor (arm1176) */
<CpSC primary registers other cases(arm) 468a>
```

```
<type CpCONTROL opcode2(arm) 29c>≡ (683c)
/*
 * CpCONTROL op2 codes, op1==0, Crm==0.
 */
#define CpMainctl  0
<CpCONTROL.opcode2 other cases(arm) 461d>
```

```
<type CpCONTROL CpMainctl(arm) 29d>≡ (683c)
/*
 * CpCONTROL: op1==0, CRm==0, op2==CpMainctl.
 * main control register.
 * cortex/armv7 has more ops and CRm values.
 */
#define CpCmmu     0x00000001 /* M: MMU enable */
#define CpChv      0x00002000 /* V: high vectors */
<CpCONTROL.CpMainctl other cases(arm) 468b>
```

Floating point coprocessors

The ARM has two coprocessors dedicated to floating-point arithmetic: one for single precision and one for double precision. Floating point adds considerable complexity to a kernel because the floating-point registers must be saved and restored on context switches, and some ARM chips lack hardware floating-point entirely (requiring software emulation). These issues are discussed in the Advanced Topics chapter.

2.5.3 ARM MMU

The ARM includes a Memory Management Unit (MMU) that translates virtual addresses to physical addresses using page tables. The page table base address is stored in the CpTTB coprocessor register—this is a physical address, since the MMU itself needs to walk the page table before any virtual-to-physical translation is available. When a translation fails, the MMU generates a fault and records the faulting address in CpFAR and the reason in CpFSR. The kernel uses the MMU to give each process its own virtual address space and to protect kernel memory from user access. More details are in Chapter 8.

The ARM MMU supports two granularities of mapping. An L1 (first-level) table entry can either map a full 1 MiB section directly to a physical address, or point to an L2 (second-level) table that maps individual 4 KiB pages:

L1 entry types:

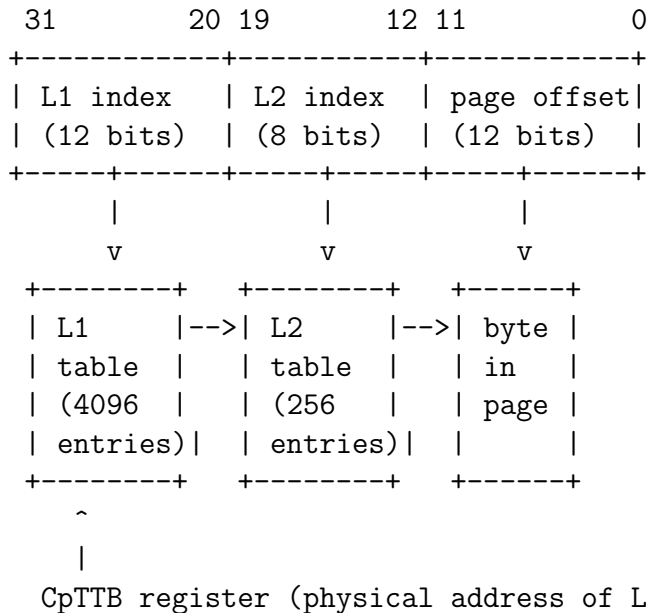
Section --> maps 1 MiB directly (no L2 table needed)

Coarse --> points to an L2 table (256 entries x 4 KiB = 1 MiB)

Sections are efficient for large contiguous mappings (the kernel maps all of RAM this way), while pages are needed for fine-grained control (individual user pages, the exception vector page at `HVECTORS`).

The basic unit of memory managed at the finest level is a page, which on ARM (and on x86) is 4 KiB:

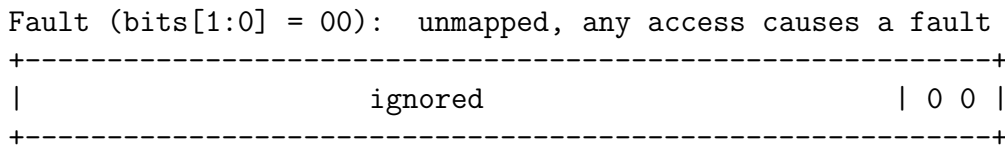
When using pages (L1 entry of type Coarse), the MMU translates virtual addresses using a two-level walk. The virtual address is split into three fields:



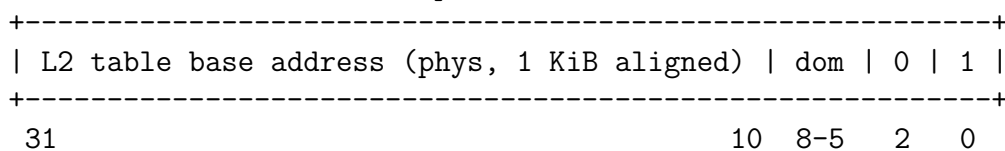
When using sections (L1 entry of type Section), the L2 step is skipped entirely: the L1 entry contains the physical base address of a 1 MiB region directly, and bits 19–0 of the virtual address become the offset within that region.

The lowest 2 bits of each L1 entry determine its type:

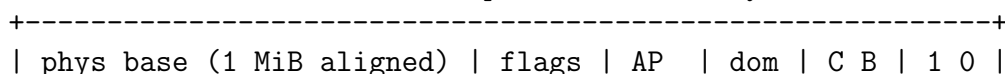
L1 entry (32 bits):

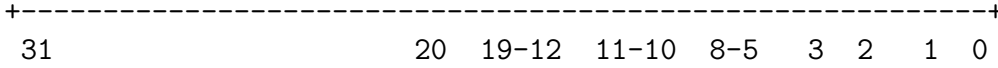


Coarse (bits[1:0] = 01): points to an L2 table



Section (bits[1:0] = 10): maps 1 MiB directly





The **Section** type is used for the kernel’s RAM mapping (512 entries covering 512 MiB) and the I/O register mapping. The **Coarse** type is used for the exception vectors at **HVECTORS** and for user-process pages, where fine-grained 4 KiB mapping is needed.

A few of the bit fields in these entries are worth knowing about now. **AP** (Access Permission, bits 11–10) controls whether the page is readable/writable by user code or only by the kernel—this is how user/kernel memory protection is enforced. **dom** (domain, bits 8–5) selects one of 16 domains; the domain register then says whether accesses to that domain are checked (“client”), unchecked (“manager”), or faulted (“no access”), providing a coarse-grained override. The **C** and **B** bits (bits 3 and 2) control caching: **C**=1 enables the data cache, **B**=1 enables write buffering. Kernel RAM is mapped with both set (write-back cached); I/O registers are mapped with both clear (uncached, so reads/writes go directly to hardware).

```
<constant BY2PG(arm) 31a>≡ (709b)
#define BY2PG      (4*KiB)      /* bytes per page */
```

```
<constant PGSHIFT(arm) 31b>≡ (709b)
#define PGSHIFT    12          /* log(BY2PG) */
```

2.5.4 Raspberry Pi Input/Output devices

The Raspberry Pi is more than just a CPU. The BCM2835 SoC includes a GPU with framebuffer, a USB controller (for keyboard and mouse), an SD card controller (for storage), GPIO pins, timers, and an interrupt controller. The kernel must initialize and manage all of these devices.

Memory-mapped IO

On ARM, devices are accessed through memory-mapped I/O: each device exposes its control registers at fixed physical addresses, and the kernel reads and writes them using ordinary load/store instructions (**MOVW**, which is really **LDR/STR** under the hood). This is simpler than the x86 approach, which uses special **IN/OUT** instructions for I/O ports. The kernel maps the physical I/O region (starting at 0x20000000 on the Raspberry Pi 1) to a virtual address **VIRTIO**, so that all device registers are accessed as offsets from **VIRTIO**:

```
<constant VIRTIO(arm) 31c>≡ (709b)
#define VIRTIO      0x7E000000    /* i/o registers */
```

```
<constant IOSIZE(arm) 31d>≡ (709b)
#define IOSIZE      (16*MiB)
```

The remaining constants define the addresses of specific device register blocks, all at fixed offsets from **VIRTIO**:

```
<constant SYSTIMERS(arm) 31e>≡ (715b)
#define SYSTIMERS (VIRTIO+0x3000)
```

```
<constant ARMTIMER(arm) 31f>≡ (715b)
#define ARMTIMER (VIRTIO+0xB400)
```

```
<constant INTREGS(arm) 31g>≡ (712c)
#define INTREGS   (VIRTIO+0xB200)
```

```
<constant DMAREGS(arm) 31h>≡ (793b)
#define DMAREGS (VIRTIO+0x7000)
```

```
<constant POWERREGS(arm) 31i>≡ (733c)
#define POWERREGS (VIRTIO+0x100000)
```



```

⟨constant AUXREGS(arm) 32a⟩≡ (802)
#define AUXREGS (VIRTIO+0x215000)

```

Output device: VideoCore GPU

The Raspberry Pi’s BCM2835 contains a VideoCore IV GPU that is actually more powerful than the ARM CPU itself. The GPU manages the screen and provides a framebuffer: a region of memory where each pixel is represented by a few bytes. The kernel writes pixels to this framebuffer, and the GPU continuously scans it to produce the video signal. Communication between the ARM CPU and the GPU goes through a mailbox mechanism—a set of hardware registers at a fixed address where the ARM can send requests and receive responses:

```

⟨constant MAILBOX(arm) 32b⟩≡ (691d)
#define MAILBOX (VIRTIO+0xB880)

```

To set up the framebuffer, the kernel fills an `Fbinfo` structure with the desired resolution and pixel depth, sends its physical address to the GPU via the mailbox, and the GPU responds by filling in the `pitch`, `base`, and `screensize` fields. The `base` field is the physical address of the framebuffer memory that the GPU has allocated. Note how the field order in the C struct must exactly match the hardware’s expected memory layout:

```

⟨struct Fbinfo(arm) 32c⟩≡ (691d)
// Framebuffer
// The order matters! the fields match the memory-mapped external registers.
struct Fbinfo {
    u32int xres;
    u32int yres;
    u32int xresvirtual;
    u32int yresvirtual;
    u32int pitch;          /* returned by gpu */
    u32int bpp;
    u32int xoffset;
    u32int yoffset;
    u32int base;           /* returned by gpu */
    u32int screensize;     /* returned by gpu */
};

```

The kernel then maps this GPU-allocated physical memory to the virtual address `FRAMEBUFFER` so it can write pixels directly. See the `GRAPHICS` book [Pad16d] for the full story of how the windowing system uses this framebuffer.

```

⟨constant FRAMEBUFFER(arm) 32d⟩≡ (709b)
#define FRAMEBUFFER 0xC0000000 /* video framebuffer */

```

Input devices: USB keyboard and mouse

```

⟨constant USBREGS(arm) 32e⟩≡ (787b)
#define USBREGS (VIRTIO+0x980000)

```

Storage device: SD card

```

⟨constant EMMCREGS(arm) 32f⟩≡ (731b)
#define EMMCREGS (VIRTIO+0x300000)

```

Other devices

```

⟨constant GPIOREGS(arm) 32g⟩≡ (802)
#define GPIOREGS (VIRTIO+0x200000)

```

The following diagram summarizes the I/O register layout as offsets from `VIRTIO` (`0x7E000000`):

<code>VIRTIO + IOSIZE</code>	<code>(0x7E000000 + 16 MiB)</code>
	+-----+
	MAILBOX (GPU)
<code>VIRTIO + 0xB880</code>	+-----+
	ARMTIMER
<code>VIRTIO + 0xB400</code>	+-----+
	INTREGS
<code>VIRTIO + 0xB200</code>	+-----+
	USBREGS
<code>VIRTIO + 0x980000</code>	+-----+
	EMMCREGS (SD)
<code>VIRTIO + 0x300000</code>	+-----+
	AUXREGS (UART)
<code>VIRTIO + 0x215000</code>	+-----+
	GPIOREGS
<code>VIRTIO + 0x200000</code>	+-----+
	POWERREGS
<code>VIRTIO + 0x100000</code>	+-----+
	DMAREGS
<code>VIRTIO + 0x007000</code>	+-----+
	SYSTIMERS
<code>VIRTIO + 0x003000</code>	+-----+
	(base)
<code>VIRTIO + 0x000000</code>	+-----+

The total I/O space is `IOSIZE` (16 MiB), though only a small fraction of that range is actually used by device registers. The rest is simply unmapped.

2.6 C and assembly

The kernel is written almost entirely in C, compiled by the Plan 9 C compiler `5c` (see the `COMPILER` book [[Pad16b](#)]). A small amount of assembly is necessary for operations that C cannot express: saving and restoring registers, switching between processor modes, manipulating coprocessor registers, and performing the initial boot sequence. The calling convention used by `5c` is straightforward: parameters are passed on the stack, the return value goes in `R0`, and all registers are caller-saved. This simplifies the kernel's context-switch code, because the `Label` structure (shown later) only needs to store `SP` and `PC`—the caller has already saved any registers it cares about. Note that this `Label` (just `SP+PC`) is used for voluntary context switches between kernel threads. When the kernel takes control from a user process (via a syscall or interrupt), it saves all registers into a `Ureg` on the kernel stack—because the user did not follow any calling convention that the kernel could rely on.

Here is the stack layout when a C function `foo(a, b, c)` calls `bar(x, y)`:

	+-----+
c (arg 3)	12(FP) for foo
	+-----+
b (arg 2)	8(FP)
	+-----+
a (arg 1)	4(FP)
	+-----+

```

      | return addr |    0(FP)  = saved R14
SP ----> +-----+
      | y (arg 2)  |    8(SP)  for bar
      +-----+
      | x (arg 1)  |    4(SP)
      +-----+
      | (R14 save) |    0(SP)
      +-----+

```

For syscalls, the convention is different: the syscall number goes in R0 (not on the stack), and the SWI instruction traps into the kernel. The kernel then reads the user’s stack to find the syscall arguments. This is why the `Syscall` typedef takes a `ulong*`—it points to the arguments on the user’s stack.

Kernel code makes heavy use of bit manipulation—rounding addresses to page boundaries, extracting bit fields from hardware registers, etc. The following macros are used throughout the kernel:

```

⟨portdat.h macros 34a⟩≡                                     (? 0—1) 34b▷
#define ROUND(s, sz) (((s)+(sz-1)) & ~(sz-1))

⟨portdat.h macros 34b⟩+≡                                     (? 0—1) ◁34a 34c▷
#define MASK(v) ((1UL << (v)) - 1) /* mask 'v' bits wide */

⟨portdat.h macros 34c⟩+≡                                     (? 0—1) ◁34b 34d▷
#define MIN(a, b) ((a) < (b)? (a): (b))

⟨portdat.h macros 34d⟩+≡                                     (? 0—1) ◁34c 34e▷
// used in bcm/mmu.c
#define FEXT(d, o, w) (((d)>>(o)) & ((1<<(w))-1))

⟨portdat.h macros 34e⟩+≡                                     (? 0—1) ◁34d 100a▷
#define HOWMANY(x, y) (((x)+((y)-1))/(y))
#define ROUNDUP(x, y) (HOWMANY((x), (y))*(y)) /* ceiling */

⟨constant KiB(arm) 34f⟩≡                                     (709b)
#define KiB      1024u          /* Kibi 0x00000000000000400 */

⟨constant MiB(arm) 34g⟩≡                                     (709b)
#define MiB      1048576u       /* Mebi 0x00000000000100000 */

⟨constant GiB(arm) 34h⟩≡                                     (709b)
#define GiB      1073741824u    /* Gibi 000000000040000000 */

```

2.7 Code organization

The kernel source is split across many files organized by subsystem: `core/`, `concurrency/`, `memory/`, `processes/`, `files/`, `filesystems/`, `time/`, `interrupts/`, `devices/`, `init/`, `console/`, etc. These directories map directly to the chapters in this book.

Each subsystem has both portable code (in `port/`) and architecture-specific code (in `bcm/` for ARM).

2.7.1 Portable versus arch-specific code

To make the boundary visible in the code, arch-specific functions called from portable code are prefixed with `arch_` (e.g., `arch_coherence()`), and arch-specific structures embedded in portable ones with `Arch_` (e.g., `Arch_Cpu`, `Arch_Proc`).

2.7.2 Main headers

Every kernel source file starts with the same set of includes:

```
<kernel basic includes 35a>≡ (802 793b 787b 767b 733c 731b 723 719c 717 715b 712c 711a 707 704c 702c 698 694 691d ??????????)
#include <u.h>
#include "../port/lib.h"
#include "../port/error.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
```

2.7.3 Top directories

The master header `dat.h` pulls in all data-structure definitions. In the original Plan 9 source, `dat.h` was a single long file and all portable code lived in a flat `port/` directory. I split it into separate `portdat_XXX.h` headers organized by subsystem (concurrency, memory, files, etc.) and created matching subdirectories. Reading the include list below is a good way to see the overall structure of the kernel:

```
<dat.h includes(arm) 35b>≡ (695a)
#include "dat_core.h"           // Arch_Cpu
#include "../port/portdat_core.h" // Cpu!!, Conf, Label
#include "../port/portdat_concurrency.h" // Lock
#include "dat_memory.h"        // Arch_Proc
#include "../port/portdat_memory.h" // Page, Pagetable, Segment, KImage
#include "../port/portdat_files.h" // Chan!! Dev!!
#include "dat_time.h"          // Arch_HZ
#include "../port/portdat_time.h" // Timer
#include "dat_processes.h"     // Arch_FPsave
#include "../port/portdat_processes.h" // Proc!!!
#include "../port/portdat_console.h" // keyboard queue, consdevtab
#include "../port/portdat_devices.h" // DevConf, DevPort (not that used)
#include "../port/portdat_misc.h"   // Cmd
#include "../port/portdat_buses.h"  // Uart
#include "dat_arch.h"           // Soc
```

```
<fns.h includes(arm) 35c>≡ (695d)
#include "../port/portfns_core.h"
#include "../port/portfns_concurrency.h"
#include "../port/portfns_memory.h"
#include "../port/portfns_processes.h"
#include "../port/portfns_files.h"
#include "../port/portfns_time.h"
#include "../port/portfns_interrupts.h"
#include "../port/portfns_console.h"
#include "../port/portfns_devices.h"
#include "../port/portfns_buses.h"
#include "../port/portfns_security.h"
#include "../port/portfns_network.h"
#include "../port/portfns_syscalls.h"
#include "../port/portfns_misc.h"
#include "../port/portfns_init.h"
```

2.8 Software architecture

Unlike a compiler or an assembler, a kernel is not a pipeline that transforms an input into an output. It is a reactive program: while user programs run, the kernel is dormant. It only wakes up when an event occurs—a syscall, an interrupt, a timer tick—handles it, and then lets the user program resume.

The Plan 9 kernel is monolithic—device drivers, the file system switch, memory management, and scheduling all run in kernel space. But thanks to the 9P protocol, much functionality that would live in the kernel on other systems is pushed out to user-space file servers (e.g., the window system `rio`, the authentication agent `factotum`, the file server `fossil`). This borrows a key idea from microkernel designs: keep the kernel small and move services to user space. In fact, the entire kernel is only about 30 000 lines of C—remarkably small for a complete operating system.

The kernel’s execution can be understood along three axes. First, the boot sequence: how the kernel starts from a bare machine and ends up with a running user shell. Second, the memory layout: how physical memory is divided between the kernel, user processes, devices, and the framebuffer. Third, the steady-state operation: how syscalls flow from user space into the kernel and back, how interrupts propagate from hardware to sleeping processes, and how the scheduler picks which process runs next. The following subsections give an overview of the first two; the normal operation (once the system is up and running) is the subject of the rest of the book.

2.8.1 Booting overview (ARM)

On the Raspberry Pi, the Broadcom firmware (closed source, stored in ROM) loads a bootloader from the SD card, which in turn loads the kernel binary at physical address `0x8000`. The bootloader reads a configuration file (`config.txt`) from the SD card’s boot partition, which specifies the path to the kernel binary. The details of this early firmware stage are specific to the Broadcom SoC and not very interesting; what matters is that all architectures share the same idea of bootstrap code loading the kernel into memory at a known address.

Once the ARM starts executing the kernel, the boot proceeds in three phases: (1) hardware detection, populating the `Conf` structure with the amount of physical memory and other parameters; (2) initialization of the kernel environment (memory allocators, process table, device drivers); and (3) creation of the first user process, which runs `/boot/boot`—the program responsible for mounting the root file system, setting up the user environment, and eventually launching `init` and then `rc` (the shell). See Chapter 11 for the full code.

2.8.2 Memory overview (ARM)

Understanding the kernel’s memory layout requires thinking along several dimensions: physical vs. virtual addresses, kernel vs. user memory, and static vs. dynamic allocation. On the Raspberry Pi, the physical memory starts at address 0.² The kernel binary is loaded at physical `0x8000` (virtual `KTZERO`).³ Everything below `KZERO` (`0x80000000`) in the virtual address space belongs to user processes; everything at or above `KZERO` belongs to the kernel. The kernel’s virtual addresses are simply the physical addresses plus `KZERO`.

The following constants define the virtual address space layout. `KZERO` at `0x80000000` (2 GiB) is the boundary: below is user space, above is kernel space.

<constant KSEG0(arm) 36a> ≡ (709b)

```
#define KSEG0      0x80000000      /* kernel segment */
```

<constant KZERO(arm) 36b> ≡ (709b)

```
#define KZERO      KSEG0          /* kernel address space */
```

<constant KTZERO(arm) 36c> ≡ (709b)

```
#define KTZERO      (KZERO+0x8000) /* kernel text start */
```

²On some architectures, physical memory does not start at 0—for example, on many x86 systems the first megabyte is reserved for legacy BIOS structures. But on the Raspberry Pi’s ARM SoC, physical RAM genuinely begins at address 0.

³Why `0x8000` and not 0? Address 0 is reserved for the ARM exception vector table. But the vectors only occupy a few dozen bytes, so why not `0x1000`? The `0x8000` address is a Broadcom firmware convention—the firmware uses the memory below `0x8000` for its own data structures during boot. The kernel in turn uses some of this space (e.g., `CPUADDR` at `0x2000`) for early kernel structures like the first `Cpu`.

On the user side, the address space starts at `UZERO` (0). The first page is left unmapped so that null pointer dereferences cause a fault. User code starts at `UTZERO` (one page above zero), and the user stack grows downward from `USTKTOP`:

```

<constant UZERO(arm) 37a>≡ (709b)
#define UZERO      0          /* user segment */

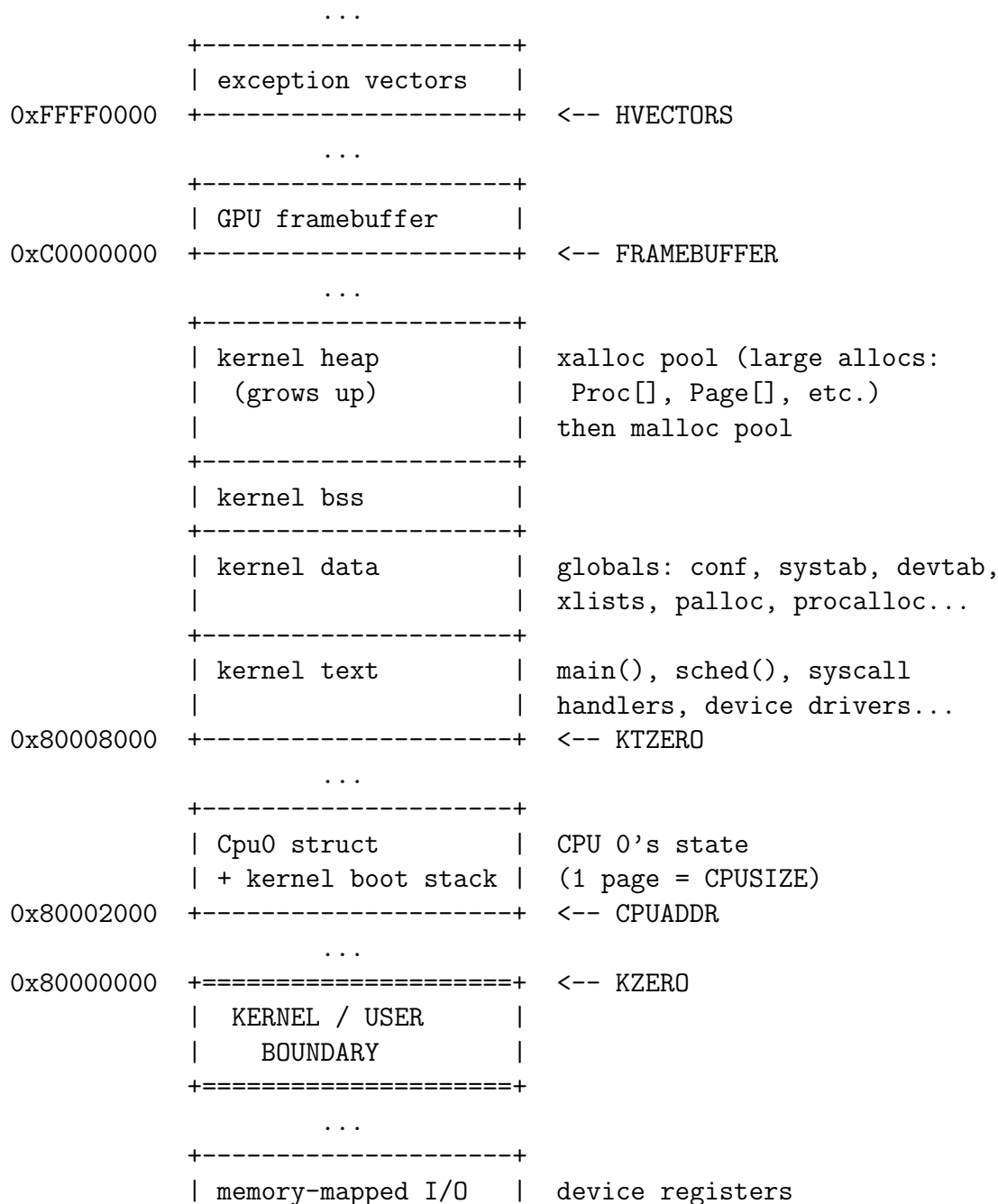
<constant UTZERO(arm) 37b>≡ (709b)
#define UTZERO      (UZERO+BY2PG) /* user text start */

<constant USTKTOP(arm) 37c>≡ (709b)
#define USTKTOP      0x20000000 /* user segment end +1 */

<constant USTKSIZE(arm) 37d>≡ (709b)
#define USTKSIZE      (8*1024*1024) /* user stack size */

```

The following diagram shows the full virtual address space. The kernel and user portions coexist in every process’s page table (though the kernel portion is shared and identical across all processes):



```

0x7E000000 | (16 MiB = IOSIZE) | (SYSTIMERS, INTREGS, ...)
+-----+ <-- VIRTIO

...
(unmapped gap)
...
0x20000000 +-----+ <-- USTKTOP
| user stack | grows down
| (8 MiB max) | (USTKSIZE)
+-----+

...
+-----+
| user heap | managed by brk()
| (grows up) |
+-----+
| user bss |
+-----+
| user data |
+-----+
| user text |
0x00001000 +-----+ <-- UTZERO
| (unmapped page) | null pointer trap
0x00000000 +-----+ <-- UZERO

```

A few things to notice. First, the kernel follows the standard binary layout—text, then data, then bss—just like a user program. The kernel heap (managed by `xalloc` and `malloc`) grows upward from the end of bss. Second, the first page at address zero is intentionally left unmapped so that null pointer dereferences cause an immediate fault. Third, the I/O registers (`VIRTIO`) sit in the gap between the user stack top and `KZERO`—they are mapped here for convenience, even though they are only accessed by the kernel. Fourth, the kernel’s data section contains important global variables such as `systab`^{41f} (the syscall dispatch table), `devtab`^{54b} (the device driver table), `conf`^{46b} (hardware configuration), `palloc`^{88d} (the page allocator state), and `procalloc`^{107b} (the process table). Finally, the `Cpu`^{48a} structure for CPU 0 occupies a single page at `CPUADDR`^{40c}, below the kernel text, in the small gap between `KZERO`^{36b} and `KTZERO`^{36c}.

On multi-processor systems (e.g., Raspberry Pi 2), each additional CPU has its own `Cpu` struct allocated elsewhere; the `cpus`^{49a} array and the dedicated register `R10` allow each processor to find its own.

The physical memory layout is simpler. The Raspberry Pi 1 has 512 MiB of RAM from physical `0x00000000` to `0x20000000`. The I/O registers start immediately after, at `0x20000000` (`Soc.physio`). The GPU allocates the framebuffer somewhere within the RAM. Here is the physical address space:

```

+-----+
| I/O registers | SYSTIMERS, INTREGS,
| (16 MiB) | MAILBOX, USBREGS, ...
0x20000000 +-----+ PHYSIO
|
| free RAM | user pages,
| (user pages, | GPU framebuffer
| GPU framebuf) |
|
+-----+
| kernel: |
| bss + heap | xalloc, malloc pools
+-----+

```

	data	conf, systab, devtab, palloc, procalloc, ...
	+-----+	
	text	main(), sched(), ...
0x00008000	+-----+	
	L1 page table	16 KiB, 4096 entries
	(CpTTB points	(Section or Coarse)
	here)	
0x00004000	+-----+	
	L2 page table	for HVECTORS mapping
0x00003000	+-----+	
	Cpu0 struct +	CPU 0's state
	boot stack	
0x00002000	+-----+	
	vector table	exception entry points
0x00000000	+-----+	

The kernel's `mmuinit()`^{237c} function (Chapter 11) fills the L1 page table at physical 0x4000 and then the boot code stores this address in the CpTTB register and enables the MMU and CpChv together. Here is what the 4096-entry L1 table looks like after `mmuinit()`:

L1 page table (at phys 0x4000, pointed to by CpTTB)
Each entry covers 1 MiB of virtual address space.

index	virt range	entry type	maps to phys
-----	-----	-----	-----
4095	0xFFFF0_0000 HVECTORS	Coarse	--> L2 at 0x3000 (L2[0] = phys page 0)
4094	...	Fault	(unmapped)
...	(unmapped)	...	
3072	0xC000_0000 FRAMEBUF	Section	--> GPU-chosen address
3071	...	Fault	(unmapped)
...	(unmapped)	...	
2559	0x9FF0_0000	Section	--> 0x1FF0_0000
...	(kernel RAM, contiguous)		
2049	0x8010_0000	Section	--> 0x0010_0000
2048	0x8000_0000 KZERO	Section	--> 0x0000_0000
====	kernel / user boundary	=====	
2031	0x7EF0_0000	Section	--> 0x20F0_0000
...	(I/O registers, 16 entries)		
2016	0x7E00_0000 VIRTIO	Section	--> 0x2000_0000 PHYSIO
2015	...	Fault	(unmapped)
...	(unmapped gap)	...	
512	0x2000_0000 USTKTOP	Fault	(unmapped)
511	0x1FF0_0000	Fault initially, then	
...	(user space)	Coarse when process mapped	
1	0x0010_0000	(filled on demand by	
0	0x0000_0000 UZERO	arch_putmmu / fixfault)	

The user entries (indices 0–511) start as Fault after `mmuinit()`. They are filled on demand by `arch_putmmu()`¹⁶² when a process is created or when a page fault occurs: each entry becomes a Coarse pointer to a per-process L2

table. Within the first MiB (entry 0), the L2 table maps pages starting at `UTZERO` (0x1000) but leaves page 0 permanently unmapped—this is the null pointer trap. So it is not the entire first MiB that is inaccessible, only the first 4 KiB page.

The kernel mapping is a simple offset: virtual `KZERO+x` maps to physical `x`, using 512 consecutive L1 Section entries (indices 2048–2559). The L1 table itself, the `Cpu` structure, and the vector table are all within this range, so they are accessible at both their physical addresses and at `KZERO` plus that address. The three special mappings are the I/O registers (16 L1 sections mapping virtual `VIRTIO` to physical `PHYSIO`), the exception vectors (one L1 Coarse entry at index 4095 pointing to an L2 table that maps a single 4 KiB page of physical 0 to virtual `HVECTORS`—the ARM’s `CpChv` bit directs the processor to fetch vectors from `0xFFFF0000`), and the GPU framebuffer (one L1 section at index 3072). User mappings occupy indices 0–511 and are filled per-process with Coarse entries pointing to L2 tables, so that each process has its own pages.

To make the distinction between physical and virtual addresses explicit in the code, the following typedefs are used:

```

<pad memory pointer types(arm) 40a>≡ (695a) 40b▷
// physical address
typedef uintptr phys_addr;
// virtual address (a user_addr or kern_addr)
typedef uintptr virt_addr;

// user address (under KZERO)
typedef uintptr user_addr;
// kernel address (mostly physical + KZERO)
typedef uintptr kern_addr;

<pad memory pointer types(arm) 40b>+≡ (695a) ◁40a
// wp = word pointer, vp = void pointer
typedef ulong* virt_wp;
typedef void* virt_vp;
typedef ulong* kern_wp;
typedef void* kern_vp;
typedef ulong* user_wp;
typedef void* user_vp;

```

The kernel text (code) starts at `KTZERO`, which is `KZERO+0x8000`—because the ARM firmware loads the kernel binary at physical 0x8000. The physical memory below 0x8000 is used for early kernel data structures such as CPU 0’s `Cpu` structure at `CPUADDR`:

```

<constant CPUADDR(arm) 40c>≡ (709b)
#define CPUADDR (KZERO+0x2000) /* Cpu structure */

```

Because the kernel’s virtual addresses are simply the physical addresses plus `KZERO`, converting between the two is a matter of masking bits. `PADDR` strips the high bits to go from virtual to physical; `KADDR` adds `KZERO` to go from physical to virtual:

```

<macro PADDR(arm) 40d>≡ (695d)
#define PADDR(va) PTR2UINT(PHYSDRAM | ((uintptr)(va) & ~KSEGM))

<macro PADDR bis(arm) 40e>≡ (684)
#define PADDR(va) (((va) & ~KSEGM))

<constant PHYSDRAM(arm) 40f>≡ (709b)
#define PHYSDRAM 0

<constant KSEGM(arm) 40g>≡ (709b)
/* mask to check segment; good for 1GB dram */
#define KSEGM 0xC0000000

<macro KADDR(arm) 40h>≡ (695d)
#define KADDR(pa) UINT2PTR(KZERO | ((uintptr)(pa) & ~KSEGM))

```

```

<macro PTR2UINT(arm) 41a>≡ (695d)
#define PTR2UINT(p) ((uintptr)(p))

<macro UINT2PTR(arm) 41b>≡ (695d)
#define UINT2PTR(i) ((void*)(i))

<function arch_cankaddr(arm) 41c>≡ (711a)
/*
 * Return the number of bytes that can be accessed via KADDR(pa).
 * If pa is not a valid argument to KADDR, return 0.
 */
uintptr
arch_cankaddr(phys_addr pa)
{
    if(pa < memsize) /* assumes PHYSDRAM is 0 */
        return memsize - pa;
    return 0;
}

```

2.8.3 Syscall overview (ARM)

We saw in the `helloraw.s` example how a user program invokes a syscall: it places the syscall number in `R0` and executes `SWI $0`. On the kernel side, the interrupt handler extracts the number from `R0` and uses it as an index into `systab`, the syscall dispatch table. Each entry in `systab` is a function pointer with a uniform signature:

```

<typedef Syscall 41d>≡ (41e)
typedef long Syscall(ulong*); // user_wp? or copied in Sargs?

```

The `ulong*` parameter is a pointer into the user's stack, where the syscall arguments have been laid out as a flat array of machine words. For example, `pwrite(fd, buf, n, off)` has four arguments, but on the kernel side they all arrive as consecutive `ulong` values starting at this pointer: `arg[0]` is the file descriptor, `arg[1]` the buffer address, `arg[2]` the count, and `arg[3]` (possibly two words for a 64-bit offset) the file position. Each `sys*()` function casts and extracts what it needs from this generic array.

```

<systab.h 41e>≡
// used by systab.c but also by trap.c for certain codes
#include "../.../lib_core/libc/9syscall/sys.h"

<typedef Syscall 41d>

extern Syscall *systab[];
extern int nsyscall;
extern char *sysctab[];

```

The `systab` array is the central dispatch table. It maps each syscall number from `sys.h` to the corresponding kernel function. This is the only point in the kernel where all syscalls are gathered together:

```

<global systab 41f>≡ (682b)
Syscall *systab[] = {
    [NOP]      sysnop,
    <systab process syscalls 106a>
    <systab memory syscalls 73>
    <systab file syscalls 257a>
    <systab directory syscalls 282a>
    <systab namespace syscalls 300a>
    <systab time syscalls 213a>
    <systab ipc syscalls 394a>
    <systab concurrency syscalls 394b>
    <systab special file syscalls 556a>
    <systab security syscalls 527c>
    [ERRSTR]   syserrstr,
};

```

```

⟨global nsyscall 42a⟩≡ (682b)
int nsyscall = nelem(systab);

```

Here is the simplest possible syscall, useful for testing that the whole trap-and-dispatch machinery works end to end:

```

⟨syscall nop 42b⟩≡ (679a)
// void nop(void);
long
sysnop(ulong*)
{
    print("Hello World\n");
    return 0;
}

```

To tie everything together, here is the complete trace of a NOP syscall, from the user program down to sysnop() and back:

USER SPACE

KERNEL SPACE

```

1. nop()
   libc wrapper:
   MOVW $0, R0    (NOP=0)
   SWI $0
   |
   | ARM hardware:
   | - saves PC in R14_svc
   | - saves CPSR in SPSR_svc
   | - switches to SVC mode
   | - jumps to HVECTORS+0x08
   |   (SWI = vector #2)
   |
+-----> 2. HVECTORS (0xFFFF0000):
           vectors[2]: LDR PC, vtable[2]
           |
           v
3. vtable[2] = _vsvc
           |
           v
4. _vsvc (assembly):
   - saves user regs -> Ureg
     (on kernel stack)
   - sets R12 = kernel SB
   - sets R(CPU), R(UP)
   - calls arch__syscall(ureg)
     |
     v
5. arch__syscall (C):
   - scallnr = ureg->r0  (= 0)
   - sp = user stack pointer
     (the ulong* in Syscall)
   - ret = systab[0](sp)
     |
     v

```

```

6. sysnop(ulong*):
  - print("Hello World\n")
  - return 0
    |
    v
7. back in _vsvc:
  - restores user regs from Ureg
  - restores SPSR -> CPSR
  - RFE (return to user mode)

```

```

|
+<-----+
|
v

```

8. user program continues
after SWI instruction

Note the distinction between `Ureg` and `Label`: when entering the kernel from user space (step 4), `_vsvc` saves all user registers into a `Ureg` because the user program did not follow any calling convention—its registers could hold anything. Later, if the kernel decides to context switch to another process (a voluntary switch between kernel threads), it uses `Label`, which saves only `SP` and `PC`—because kernel code follows the caller-save convention and the caller has already saved any registers it needs.

The exception vector table at `HVECTORS` is set up by `arch__trapinit()`^{243f} during initialization: it copies the `vectors` code (a series of `LDR PC` instructions) and the `vtable` data (an array of addresses like `_vsvc`, `_virq`, etc.) to the page at `0xFFFF0000`. The `CpChv` bit (set during boot) tells the ARM to fetch vectors from there instead of from address 0. The “v” in `_vsvc` stands for “vector” and “svc” for the supervisor call (SWI) exception. Similarly, `_virq` handles IRQ interrupts, `_vdabt` handles data aborts, etc.

A few things about the registers used in step 4. `R(CPU)` is `R10`, which by convention always holds a pointer to the current processor’s `Cpu`^{48a} structure—this is the `extern register Cpu* cpu` declaration we will see soon. `R(UP)` is `R9`, which holds a pointer to the current process’s `Proc`^{49f} structure (`up`^{51d} for “user process”). `R12` is the static base register (`SB`), used by the Plan 9 C compiler to access global variables (see the `LINKER` book [Pad15c] for details). The kernel must reload `R12` because the user program may have used it for something else.

2.8.4 FAQ

The overview above introduced many addresses, registers, tables, and mappings. It is easy to be confused, especially because some aspects are imposed by the ARM hardware (e.g., the exception vector mechanism, the PSR layout, the MMU page table format) while others are design decisions made by the Plan 9 kernel authors (e.g., the value of `KZERO`, the placement of `VIRTIO`, the `systab` dispatch table). The following questions and answers try to clarify the most common sources of confusion.

Q: The exception vector table is at `HVECTORS` and the syscall table is `systab`. How are they related? A: They are different things. The exception vector table at `HVECTORS` is a hardware mechanism: when a `SWI` instruction executes, the ARM jumps to a fixed address in this table. That entry loads the address of `_vsvc` from `vtable`. Then `_vsvc` (assembly code) extracts the syscall number from `R0` and calls `arch__syscall()` (C code), which finally indexes into `systab` to find the right kernel function. So `HVECTORS` is the hardware entry point, and `systab` is the software dispatch table—they are connected by the chain of code in between.

Q: Why `SWI $0`? Does the argument to `SWI` matter? A: On Plan 9, the argument to `SWI` is ignored. The syscall number is passed in `R0` instead. The `SWI` instruction is used purely for its side effect of trapping into supervisor mode. Some other operating systems encode the syscall number in the `SWI` argument, but Plan 9 chose the simpler approach of using a register. Similarly, Linux on x86 uses `int 0x80` (or the faster `syscall` instruction on x86-64) to trap into the kernel, with the syscall number passed in `EAX`.

Q: Why split the 4 GiB address space between kernel and user? Could each have its own full 4 GiB? A: Some systems do give user space the full address range and switch page tables entirely when entering the kernel. For example, xv6 on RISC-V maintains a separate kernel page table (with direct virtual=physical mapping) and per-process user page tables. On every syscall or interrupt, xv6 switches the page table base register from the user's table to the kernel's. To make this switch possible, both tables map a single trampoline page at the same high virtual address—this is where the switching code runs. The Plan 9 approach is different and simpler: the kernel is mapped in the upper half of every process's page table. A syscall or interrupt does not need to switch page tables at all—the kernel's code and data are always mapped and immediately accessible. The trade-off is that user programs get only 2 GiB of virtual space instead of 4 GiB (but on a machine with 512 MiB of physical RAM this is not a practical limitation). The benefits are: trap entry is faster (no TLB flush from a page table switch), no trampoline is needed, and the kernel can access user memory directly (e.g., syscall arguments on the user stack, buffer addresses) without switching page tables first. The kernel still needs to validate user pointers (e.g., check they are below KZERO, handle page faults), but the shared mapping makes the common case simpler than with separate page tables where even reading a user byte requires a special copy routine.

Q: Can a page fault occur inside the kernel? Can kernel memory be swapped out? A: The kernel's own pages are never swapped and are always mapped (via the L1 section entries set up by `mmuinit()`^{237c}). So the kernel does not page-fault when accessing its own data structures. A page fault can only occur for user-space addresses. However, a page fault can occur while the kernel is executing a syscall on behalf of a user process—for example, when the kernel copies data from a user buffer (the user's page might not be resident). The fault handler recognizes this case and handles it.

Q: Why is VIRTIO mapped below KZERO, in the gap between user space and kernel space? A: The kernel RAM mapping (`KZERO+x` = physical `x`) covers exactly `Soc.dramsize` (512 MiB), filling L1 entries 2048–2559. Placing the I/O registers above KZERO would require additional L1 entries in the kernel region, which would overlap with the simple offset mapping. Putting them at `0x7E000000` (below KZERO) avoids this conflict. User processes cannot access these pages because the L1 entries are set with kernel-only permissions.

Q: Why is CPUADDR at 0x2000, below the kernel text at 0x8000? A: The ARM firmware loads the kernel binary at physical `0x8000`. The memory below that is unused by the binary, so the kernel reuses it for the `Cpu` structure (at `0x2000`), the L2 page table (at `0x3000`), and the L1 page table (at `0x4000–0x8000`). This avoids wasting the 32 KiB gap that the firmware leaves.

2.8.5 NSFAQ

The following are more subtle questions that may not occur to every reader on first reading, but that help deepen the understanding of the kernel's design.

Much of this chapter may seem complicated and very ARM-specific. But the essential concepts are the same across architectures. Every kernel needs to:

- split the address space between kernel and user,
- set up page tables (or their equivalent) so the MMU can translate virtual to physical addresses,
- place an exception/interrupt vector table at a known address,
- save and restore user registers on kernel entry/exit,
- dispatch syscalls through a table of function pointers,
- reserve memory for per-CPU state.

The concrete addresses (`KZERO`, `HVECTORS`, etc.), the register conventions (`R10` for `cpu`, `R9` for `up`), and the page table format (L1 sections, L2 pages) are ARM-specific details, but the architecture of the kernel would look very similar on x86 or RISC-V.

2.9 Book structure

The rest of the book is organized as follows. Chapters 3 and 4 present the foundational building blocks: core data structures (`Conf`, `Cpu`, `Label`, etc.) and concurrency primitives (locks, `sleep/wakeup`, `Rendez`). Part I, *Virtualizing the Machine*, covers how the kernel gives each process the illusion of having the hardware to itself: memory allocators (Chapter 5), processes (Chapter 6), interrupts and syscalls (Chapter 7), virtual memory (Chapter 8), scheduling (Chapter 9), time (Chapter 10), and the boot sequence (Chapter 11). Part II, *Everything is a File*, covers Plan 9's central abstraction: files as the uniform interface to data, devices, and services. This includes the `Chan` and `Dev` structures (Chapter 12), directories (Chapter 13), per-process namespaces (Chapter 14), the device driver framework (Chapter 15), streaming I/O queues (Chapter 16), and on-disk file systems (Chapter 17). Part III, *Beyond the Basics*, covers the remaining subsystems: inter-process communication (Chapter 18), debugging and profiling support (Chapters 19 and 20), advanced topics (Chapter 22), and the conclusion (Chapter 24). Some appendices present non-functional code: debugging helpers, error management, and generic utilities.

Chapter 3

Core Data Structures

Show me your code and conceal your data structures, and I shall continue to be mystified. Show me your data structures, and I won't usually need your code; it'll be obvious.

Fred Brooks

In this chapter, I will present the core data structures of the kernel. Some have already been introduced in the Overview: **Ureg**^{26c}, the processor modes (**PsrMode**), the address typedefs, and the syscall dispatch table (**systab**^{41f}). Here I will cover the remaining structures that appear everywhere in the kernel: **Conf**^{46a} (hardware configuration), **Cpu**^{48a} (per-processor state), **Label**^{197d} (context switch), and **Ref**^{68b} (reference counting).

3.1 Conf and conf

The **Conf** structure records what the kernel learned about the hardware during the boot sequence: how many processors, how much physical memory, and how many processes to support. It is filled once by **confinit()**X during initialization and then treated as read-only.

```
<struct Conf 46a>≡ (609)
struct Conf
{
    ulong ncpu;    /* processors */
    Confmem mem[4]; /* physical memory */

    ulong nproc;   /* processes */
    <Conf other fields 164e>
};
```

The **mem** array holds up to 4 **Confmem** entries for non-contiguous regions of available physical memory. On the Raspberry Pi there is just one bank; on a PC there can be two (one below and one above the kernel binary). **confinit()**X excludes the memory occupied by the kernel itself.

```
<global conf 46b>≡ (610b)
Conf conf;
```

Each **Confmem** describes one memory bank—its physical base address and the number of usable pages. These entries are later consumed by the memory allocators (**xalloc** and **palloc**):

```
<struct Confmem 46c>≡ (609)
// memory "bank"
struct Confmem
{
    phys_addr base;
    uintptr limit; // used by bcm only. phys_addr too?
    ulong npage;
    <Confmem other fields 430b>
};
```

On the Raspberry Pi, there is typically a single memory bank. The **base** is the start of usable physical RAM (after the kernel binary and its static allocations), and **npage** is the number of free pages. For example, on a Pi 1 with 512 MiB of RAM:

Physical memory:

```

0x1FFF_FFFF +-----+
            |                               |
            |  Confmem bank 0               |
            |  base = ~end                 |
            |  npage = remaining pages     |
            |                               |
~end        +-----+
            |  kernel text+data+bss       |
0x8000      +-----+
            |  firmware / Cpu struct      |
0x0000      +-----+

```

The exact values depend on the kernel binary size and are computed at boot time by `confinit()`.

The bootloader passes configuration parameters to the kernel as key-value string pairs (similar to a Linux kernel command line). These are stored in the `confname` and `confval` parallel arrays. One important parameter is `*kernelpercent*`, which controls what fraction of physical memory the kernel reserves for its own dynamic allocations (via `xallocX`). The default in the code is 60%, but in practice a value around 20% is more typical—most memory should be available for user processes.

<globals confname and confvar 47a>≡ (610b)

```

// conf (boot) parameters *e.g. { "*kernelpercent*" => "60" }
// hash<string, string>
char *confname[MAXCONF];
char *confval[MAXCONF];

```

<constant MAXCONF 47b>≡ (609)

```

#define MAXCONF      64

```

<global nconf 47c>≡ (610b)

```

// Hashtbl.length(confname)
int nconf;

```

<function getconf 47d>≡

```

char* getconf(char *name)
{
    int i;
    for(i = 0; i < nconf; i++)
        if(cistrncmp(confname[i], name) == 0)
            return confval[i];
    return nil;
}

```

3.2 Cpu and cpu

The `Cpu` structure holds the per-processor state: which process is currently running, the clock tick count, MMU state, and a small kernel stack used during boot and interrupts. On a uniprocessor Raspberry Pi 1 there is a

single Cpu at CPUADDR; on a multiprocessor system each CPU has its own.

```

⟨struct Cpu 48a⟩≡ (609)
//coupling: do not change the order of the first 4 fields!
// Some assembly code assumes this order.
struct Cpu
{
    int cpuno; /* physical id of processor (KNOWN TO ASSEMBLY) */
    ⟨Cpu second field 56f⟩
    // ref<Proc>, or None if halting?
    Proc* proc; /* current process on this processor */

    struct Arch_Cpu;

    ⟨Cpu time fields 48c⟩
    ⟨Cpu stat fields 587a⟩
    ⟨Cpu other fields 70f⟩

    // must be at the end of the structure!
    int stack[1];
};

```

The first fields have a fixed layout because assembly code accesses them by hardcoded offsets. The `stack[1]` at the end is a C idiom: the actual stack extends to the end of the CPUSIZE page, so the array’s “one element” is really the start of a much larger area.

```

⟨constant CPUSIZE(arm) 48b⟩≡ (709b)
#define CPUSIZE BY2PG

```

We will see more about ticks and the clock in the Interrupts chapter (Chapter 7).

```

⟨Cpu time fields 48c⟩≡ (48a) 48d▷
    ulong ticks; /* of the clock since boot time */

⟨Cpu time fields 48d⟩+≡ (48a) ◁48c
    uulong cpuhz;
    int cpumhz; // cpuhz / 1_000_000

```

The Cpu structure shown above is portable across all architectures. But each architecture also has specific fields (MMU state, floating-point context, etc.) that are gathered in Arch_Cpu. Thanks to a Ken C compiler extension, `struct Arch_Cpu;` can be embedded in Cpu without giving it a field name—its fields are accessed directly as if they were part of Cpu itself.

```

⟨struct Arch_Cpu(arm) 48e⟩≡ (695b)
//coupling: do not change the first field! Some assembly code assumes this order
struct Arch_Cpu {
    ⟨Cpu Arch mmu fields(arm) 156k⟩
    ⟨Cpu Arch float fields(arm) 458b⟩
    ⟨Cpu Arch save of first five registers fields(arm) 244c⟩
};

```

The cpu global must be per-processor: on a multiprocessor system, each CPU must see its own Cpu structure. Since each processor has its own set of registers, a dedicated register is a natural way to hold per-processor state. On ARM, this is achieved by dedicating register R10—the 5c compiler’s `extern register` extension reserves R10 so it is never used for anything else. Each processor sets its R10 to point to its own Cpu during boot. Note that R10 is only reserved in kernel code, not in user-space programs; it is a kernel convention. That is why `_vsvc` sets R(CPU) early upon entering kernel mode (visible in the syscall trace diagram above). On x86, where registers are scarce, a different trick involving the page table is used instead.

```

⟨global cpu(arm) 48f⟩≡ (695a)
extern register Cpu* cpu; /* R10 */

```

```

⟨global cpus 49a⟩≡ (610b)
/*
 * Each processor sees its own Cpu structure at address CPUADDR.
 * However, the Cpu structures must also be available via the per-processor
 * MMU information array cpus, mainly for disambiguation and access to
 * the clock which is only maintained by the bootstrap processor (0).
 */
// array<ref<Cpu>>
Cpu* cpus[MAXCPUS];

```

```

⟨constant MAXCPUS (arm) 49b⟩≡ (709b)
#define MAXCPUS 4 /* max # cpus system can run */

```

```

⟨macro CPUS 49c⟩≡ (609)
#define CPUS(n) (cpus[n])

```

The `Active` structure tracks global multiprocessor state that does not belong to any single `Cpu`. The `cpus` bitmap records which processors are online, and `exiting` signals a system-wide shutdown. While `cpus[]` (the array) gives access to each processor’s `Cpu` structure, `Active` answers a different question: which processors are alive, and is the system shutting down? It also carries a `Lock` to serialize concurrent updates from multiple CPUs.

```

⟨struct Active 49d⟩≡ (668c)
struct Active
{
    // array<bool> (coupling: sizeof(int) must be >= MAXCPUS)
    int cpus; /* bitmap of active CPUs */
    bool exiting; /* shutdown */
    ⟨Active other fields 562b⟩
    // extra
    Lock;
};

```

```

⟨global active 49e⟩≡ (671a)
struct Active active;

```

3.3 Proc and up

The `Proc` structure is the largest and most important data structure in the kernel—it is what the kernel knows about a process. Each running program has one `Proc` containing its state, memory map, open files, scheduling priority, parent/child relationships, notes (signals), error handling context, and much more. It is one of the largest structures in the kernel. The structure is organized in sections (visible in the comment banners below), each corresponding roughly to a chapter in this book. We will only show the overview here; the individual fields are introduced gradually in their respective chapters.

```

⟨struct Proc 49f⟩≡ (668c)
struct Proc
{
    //-----
    //coupling: have to be first! Some assembly code assumes this order.
    //-----
    ⟨Proc assembly fields 197e⟩
    //-----
    // State
    //-----
    ⟨Proc state fields 51a⟩
    //-----
    // Memory

```

```
//-----
    <Proc memory fields 98a>

    struct Arch_Proc;
//-----
// Scheduling
//-----
    <Proc scheduling fields 190c>
//-----
// Files
//-----
    <Proc files fields 52a>
//-----
// Notes
//-----
    <Proc notes fields 395h>
//-----
// Process hierarchy
//-----
    <Proc hierarchy fields 127a>
//-----
// Synchronization
//-----
    <Proc synchronization fields 69g>
//-----
// Error management
//-----
    <Proc error managment fields 596b>
//-----
// Stats, profiling
//-----
    <Proc stats and profiling fields 444b>
//-----
// Debugging (the kernel itself)
//-----
    <Proc debugging fields 59b>
//-----
// For debugger, strace
//-----
    <Proc debugger fields 143b>
//-----
// Other
//-----
    <Proc other fields 134c>
//-----
// Extra
//-----
    <Proc extra fields 63c>
};
```

Just as `Cpu` has an `Arch_Cpu`, `Proc` embeds an anonymous `struct Arch_Proc` for architecture-specific fields (on ARM, this is the per-process MMU state). We will detail these fields later.

```
<struct Arch_Proc(arm) 50>≡ (709a)
    struct Arch_Proc
    {
        <Proc Arch memory fields(arm) 157c>
    };
};
```

Perhaps the most important field in `Proc` is its state—whether the process is running, ready to run, sleeping, or dead:

```

⟨Proc state fields 51a⟩≡ (49f) 106b▷
// enum<Procstate>
int state; // Dead, Queuing, etc, (used by /proc/#/status if psstate==nil)

```

A process is always in one of the following states. The most important ones are **Running** (executing on a CPU), **Ready** (runnable, waiting for the scheduler to pick it), and **Wakeme** (sleeping, waiting for an event):

```

⟨enum procstate 51b⟩≡ (668c)
/* Process states, Proc.state */
enum Procstate
{
    Dead = 0,
    Running,
    ⟨Procstate cases 63d⟩
};

⟨global statename 51c⟩≡ (675c)
// hash<enum<Procstate>, string>, coupling: with enum Procstate
char *statename[] =
{
    [Dead]      = "Dead",
    [Running]   = "Running",
    [Queueing]  = "Queueing",
    [QueueingR] = "QueueingR",
    [QueueingW] = "QueueingW",
    [Moribund]  = "Moribund",
    [Ready]     = "Ready",
    [Scheding]  = "Scheding",
    [Wakeme]    = "Wakeme",
    [Broken]    = "Broken",
    [Stopped]   = "Stopped",
    [Rendezvous] = "Rendezvous",
    [Waitrelease] = "Waitrelease",
};

```

Like `cpu`, the `up` pointer (“user process”) is per-processor: it points to the `Proc` currently running on this CPU. On ARM it is stored in R9. The 5c compiler assigns `extern register` variables to registers in decreasing order: the first declaration gets R10 (which is `cpu`), the second gets R9 (which is `up`), and a third would get R8, and so on.

```

⟨global up(arm) 51d⟩≡ (695a)
extern register Proc* up;          /* R9 */

```

3.4 Chan

A **Chan** (“channel”) is the kernel’s representation of an open file. It is the internal object behind a user’s file descriptor—the file descriptor is just an integer index into a per-process table of **Chan** pointers. Because everything in Plan 9 is a file, channels are used everywhere: for real files on disk, for device control files like `/dev/cons`, for network connections, for mounted file servers, etc. The key fields are **type** (which device driver owns this channel), **dev** (a device-specific sub-identifier—together **type** and **dev** play the role of UNIX’s major and minor device numbers), **qid** (the file’s unique identity on that device, like an inode number), **path** (the file name), **offset** (the current read/write position), and **mode** (read, write, or both):

```

⟨struct Chan 51e⟩≡ (645c)
struct Chan
{
    ushort type; // idx in devtab
    ulong dev;
    Qid qid;

```

```

Path* path;

vlong offset;      /* in fd */

//enum<open>, actually restricted to the OREAD|OWRITE|ORDWR of open type
ushort mode;      /* read/write */

bool ismtp; // is a mount point

union {
    void* aux; // generic pointer, for specific usages
    <Chan union other fields 404a>
};
<Chan other fields 257c>

// extra
Ref; /* the Lock in this Ref is also Chan's lock */
<Chan extra fields 258b>
};

```

Every process holds a few **Chan** pointers directly in its **Proc** structure. The two most fundamental ones are **slash** (the root of the process's namespace) and **dot** (its current directory):

```

<Proc files fields 52a>≡ (49f) 261a▷
// ref<Chan>
Chan *slash; // The root! used by namec()
// ref_counted<Chan>
Chan *dot; // The current directory

```

A **Qid** (“q” for “unique”) is Plan 9’s equivalent of a UNIX inode number: it uniquely identifies a file within a device or file server. The **path** field is the unique identifier, **vers** is a version number for cache invalidation, and **type** distinguishes files from directories:

```

<struct Qid 52b>≡ (656e)
struct Qid
{
    // note that this is not a string, but an int! it's kind of an inode
    uulong path;
    // for cache invalidation
    ulong vers;
    // enum<Qidtype>
    byte type;
};

```

Despite its name, **path** is not a string but a 64-bit integer—it is a unique numeric identifier for a file within a device, much like a UNIX inode number. The name is confusing because **Chan** also has a separate **path** field (of type **Path***) that holds the actual string pathname. The **vers** field is more unusual: it is a version number that the file server increments whenever the file’s content changes. The kernel uses it for cache invalidation—if a cached **Qid** has a stale **vers**, the kernel knows it must re-read the file’s metadata.

```

<enum qidtype 52c>≡ (656e)
/* bits in Qid.type */
enum Qidtype {
    QTFILE = 0x00, /* plain file */
    QTDIR = 0x80, /* type bit for directories */
    <Qidtype cases 375a>
};

```

```

<function eqqid 52d>≡ (647)
bool
eqqid(Qid a, Qid b)

```

```

{
    return a.path==b.path && a.vers==b.vers;
}

⟨function mkqid 53a⟩≡ (649)
void
mkqid(Qid *q, vlong path, ulong vers, int type)
{
    q->type = type;
    q->vers = vers;
    q->path = path;
}

⟨function eqchan 53b⟩≡ (647)
bool
eqchan(Chan *a, Chan *b, bool skipvers)
{
    if(a->qid.path != b->qid.path)
        return false;
    if(!skipvers && a->qid.vers!=b->qid.vers)
        return false;

    if(a->type != b->type)
        return false;
    if(a->dev != b->dev)
        return false;
    return true;
}

⟨function eqchantdqid 53c⟩≡ (647)
bool
eqchantdqid(Chan *a, int type, int dev, Qid qid, bool skipvers)
{
    if(a->qid.path != qid.path)
        return false;
    if(!skipvers && a->qid.vers!=qid.vers)
        return false;
    if(a->type != type)
        return false;
    if(a->dev != dev)
        return false;
    return true;
}

```

3.5 Dev and devtab

A **Dev** is a device driver—or more precisely, a file server implemented in the kernel. Each **Dev** provides a set of methods (**open**, **read**, **write**, **close**, etc.) that implement the file interface for a particular kind of resource: the console (**devcons**), the process table (**devproc**), the root file system (**devroot**), the SD card (**devsd**), etc. The single character **dc** (e.g., **'c'** for console, **'p'** for proc) identifies the device in Plan 9's **#c**, **#p** naming convention. In UNIX, devices are identified by major and minor numbers—opaque integers that require a lookup table to understand. Accessing a device requires creating special files with **mknod** and a populated **/dev** directory. Plan 9 replaces the major number with a human-readable Unicode character (a **Rune**): **'c'** for the console, **'p'** for the process table, **'e'** for the environment, etc. You can see these characters in Plan 9's device path syntax: **#c** for the console device, **#p** for the proc device. This is much clearer than memorizing that major 4 means the console. And because **Rune** is a Unicode code point (not an ASCII byte), Plan 9 can support more than 256

different device drivers. While `Chan.type` is still a numeric index internally (into `devtab`), the user-visible name uses the character from `Dev.dc`—so the human never sees the number.

```
⟨struct Dev 54a⟩≡ (645c)
    struct Dev
    {
        Rune dc; // dev character code, e.g. '/' (devroot), 'e' (devenv), ...
        char* name;

        ⟨Dev methods 250b⟩
    };
```

The global `devtab` is an array of pointers to all the `Dev` structures compiled into the kernel—it is the registry of all available device drivers. When a `Chan` is opened on a device, `Chan.type` is the index into `devtab`, so that `devtab[c->type]->read()` calls the right driver’s read method.

```
⟨global devtab 54b⟩≡ (649)
    // array<ref_own<Dev>>
    Dev** devtab = nil;
```

3.6 Soc and soc (ARM)

The `Soc` structure captures the hardware specifics of the System-on-Chip: how much DRAM, where the I/O registers start, and various MMU and bus parameters. It is the single place where the differences between Raspberry Pi 1 and Pi 2 are recorded:

```
⟨struct Soc(arm) 54c⟩≡ (683b)
    struct Soc {          /* SoC dependent configuration */
        ulong   dramsize;
        phys_addr physio;
        ⟨Soc bus fields 790e⟩
        ⟨Soc mmu fields 468i⟩
        ⟨Soc other fields 475d⟩
    };

⟨global soc(raspberry pi1)(arm) 54d⟩≡ (707a)
    Soc soc = {
        .dramsize   = 512*MiB,    // 0 to 0x20000000
        .physio     = 0x20000000, // after RAM
        ⟨soc(raspberry pi1) other fields(arm) 468j⟩
    };
```

Chapter 4

Concurrency

In this chapter, I will present the concurrency primitives used inside the kernel. These are foundational building blocks—almost every chapter that follows will use locks, interrupt disabling, or sleep/wakeup. In fact, we already encountered `Lock`^{58c} (in `Active`^{49d}) and `Ref`^{68b} (in `Chan`^{51e}) in the previous chapter without explaining them—they are defined here.

Kernels were arguably the first concurrent programs, long before user-level threads and multicore CPUs existed. Even on a uniprocessor, the kernel is inherently concurrent: a syscall can be interrupted at any point by a hardware interrupt (a timer tick, a keyboard press, a disk completion), and the interrupt handler may access the same data structures the syscall was modifying. On a multiprocessor, the situation is worse—two CPUs can execute kernel code simultaneously. Concurrency bugs in the kernel are especially dangerous because they can corrupt the data structures that all processes depend on. This chapter covers the kernel’s own concurrency mechanisms. User-level concurrency (semaphores, rendezvous) is covered in Chapter 18.

4.1 Critical regions

A critical region is a section of code that accesses shared data and must not be executed by two threads of control at the same time. If two paths of execution read and modify the same data structure concurrently (for instance, the scheduler’s run queue), the result is a race condition—the data structure can end up in an inconsistent state, leading to crashes or subtle corruption. In the kernel, the concurrent “threads” are not user-level threads but different paths of execution that can interleave:

- A syscall executing on behalf of process A can be interrupted by a timer tick, which triggers the scheduler, which switches to process B, which makes its own syscall.
- On a multiprocessor, two CPUs can execute two different syscalls simultaneously.
- An interrupt handler can fire while the kernel is in the middle of modifying a data structure that the handler also needs.

The kernel uses a hierarchy of mechanisms to protect critical regions, from coarsest to finest: `arch_splhi()` (disable all interrupts—the simplest but most heavy-handed), `Lock` (a spin lock using `arch_tas()`), `ILock` (a spin lock that also disables interrupts, to avoid deadlock with interrupt handlers), `QLock` (a lock where waiters sleep instead of spinning), and `RWLock` (a readers/writer lock). The following sections present each in turn.

4.2 Interrupts disabling: `arch_splhi()`, `arch_spllo()` (ARM)

The simplest way to protect a critical region is to disable interrupts: if no interrupt can fire, no other code can run on this processor. The “spl” name is a legacy from UNIX, standing for “Set Priority Level.” `arch_splhi()`

disables IRQ interrupts by setting the PsrDirq bit in the CPSR; `arch_spllo()` re-enables them. `arch_splhi()` returns the old CPSR value so `arch_splx()` can restore it later. On a uniprocessor, `arch_splhi()` is sufficient for all mutual exclusion. But it does not scale to SMP: disabling interrupts on one CPU does not prevent another CPU from executing kernel code. For SMP, finer-grained locks are needed.

<function arch_splhi(arm) 56a>≡ (692)

```
// bool arch_splhi(void);
TEXT arch_splhi(SB), 1, $-4
    <arch_splxxx() save caller information(arm) 56e>
    MOVW CPSR, R0 /* turn off irqs (but not fiqs) */
    ORR $(PsrDirq), R0, R1
    MOVW R1, CPSR
    RET
```

<function arch_spllo(arm) 56b>≡ (692)

```
// bool arch_spllo(void);
TEXT arch_spllo(SB), 1, $-4
    MOVW CPSR, R0 /* turn on irqs and fiqs */
    BIC $(PsrDirq|PsrDfiq), R0, R1
    MOVW R1, CPSR
    RET
```

<function arch_splx(arm) 56c>≡ (692)

```
// void arch_splx(bool);
TEXT arch_splx(SB), 1, $-4
    <arch_splxxx() save caller information(arm) 56e>
    MOVW R0, R1 /* reset interrupt level */
    MOVW CPSR, R0 //dead
    MOVW R1, CPSR
    RET
```

<function arch_islo(arm) 56d>≡ (692)

```
// bool arch_islo(void);
TEXT arch_islo(SB), 1, $-4
    MOVW CPSR, R0
    AND $(PsrDirq), R0
    EOR $(PsrDirq), R0
    RET
```

When `arch_splhi()` disables interrupts, it also records who called it, for debugging. Since `Cpu` lives at the fixed address `CPUADDR`, the assembly code can store the caller's return address (R14) directly into the second field of `Cpu` at offset 4:

CPUADDR+CPUSIZE	+-----+	
	stack[1] ...	
	+-----+	
	...	
	+-----+	
CPUADDR+0x04	splpc	<- saved here
	+-----+	
CPUADDR	cpuno	
	+-----+	

<arch_splxxx() save caller information(arm) 56e>≡ (56)

```
MOVW $(CPUADDR+4), R2 /* save caller pc in Cpu */
MOVW R14, 0(R2)
```

<Cpu second field 56f>≡ (48a)

```
// must be second field at 0x04, used by arch_splhi()
ulong splpc; /* pc of last caller to arch_splhi */
```

4.3 Test and set: `arch_tas()` (ARM)

Test-and-set is the hardware primitive on which all spin locks are built. The idea is simple: atomically write 1 into a memory word and return the old value. If the old value was 0, the caller has acquired the lock; if it was already 1, someone else holds it. In C terms, `lock()` will call `if(arch_tas(&1->key) == 0)`. The key word is “atomically.” On a RISC architecture like ARM, a naive implementation would need two separate instructions—a load and a store:

```
// hypothetical broken arch_tas:
MOVW 0(R1), R0      // load key into R0
MOVW $1, 0(R1)      // store 1 into key
RET                  // return old value in R0
```

But an interrupt can fire between the load and the store:

```
Process A (syscall):      Process B (interrupt handler):

MOVW 0(R1), R0
  (reads key=0)

                                <--- timer interrupt fires --->
                                MOVW 0(R1), R0
                                (reads key=0, A hasn't stored yet)
                                MOVW $1, 0(R1)
                                (sets key=1)
                                R0==0 -> if() succeeds
                                ... enters critical section ...
                                ... done, returns from interrupt ...

MOVW $1, 0(R1)
  (sets key=1)
R0==0 -> if() succeeds
... also enters critical section! BUG
```

Both see `key=0` and both believe they acquired the lock. On the Raspberry Pi 1, the solution is the `SWPW` (swap word) instruction, which performs the load-and-store in a single indivisible operation—the interrupt sees either the state before or after, never in between. On the Pi 2 (ARMv7, SMP), a different mechanism using `LDREX`/`STREX` is needed because `SWPW` is deprecated and does not provide the memory ordering guarantees required for multiprocessor correctness.

```
<function arch_tas(raspberry pi1)(arm) 57>≡ (693a)
TEXT    arch_tas(SB), $-4
    MOVW    R0,R1
    MOVW    $1,R0
    SWPW    R0,(R1)      /* fix: deprecated in armv6 */
    RET
```

4.4 Atomic count: `arch_xinc()` and `arch_xdec()` (ARM)

The other fundamental atomic operation is increment/decrement. These are used for reference counting (e.g., `Ref`). On the uniprocessor Pi 1, the implementation is trivial: disable interrupts, modify the counter, re-enable.

On SMP, a more sophisticated approach using LDREX/STREX would be needed.

```
⟨function arch_xinc(raspberry pi1)(arm) 58a⟩≡ (694a)
void
arch_xinc(long *p)
{
    int s;

    s = arch_splhi();
    ++*p;
    arch_splx(s);
}
```

```
⟨function arch_xdec(raspberry pi1)(arm) 58b⟩≡ (694a)
long
arch_xdec(long *p)
{
    int s, v;

    s = arch_splhi();
    v = --*p;
    arch_splx(s);
    return v;
}
```

4.5 Spin Lock

A Lock (spin lock) is built on top of `arch_tas()`. Unlike `arch_splhi()`, a spin lock provides mutual exclusion across CPUs, not just against interrupts on one CPU. The caller spins in a tight loop until the lock becomes available. Spin locks are appropriate for very short critical regions (a few lines of code) where the wait is expected to be brief. On a uniprocessor, a spin lock still serves a purpose: while a process holds a lock, the kernel prevents it from being scheduled out (via `Proc.nlocks`), which effectively protects the critical region from other processes. However, a spin lock on a uniprocessor does not protect against interrupts—for that, `ILock` (next section) is needed. Note that `Lock` itself is portable, architecture-independent code (in `port/`)—it provides mutual exclusion on any architecture, uniprocessor or multiprocessor, by relying on the arch-specific `arch_tas()` primitive we just saw.

```
⟨struct Lock 58c⟩≡ (604)
// a.k.a Spinlock
struct Lock
{
    ulong key; // 0 when unset, 1 (or 0xDEADDEAD) when acquired, could be a bool

    ⟨Lock ilock fields 61b⟩
    ⟨Lock debugging fields 59a⟩
    ⟨Lock other fields 59c⟩
};
```

```
⟨function lock 58d⟩≡ (605b)
int
lock(Lock *l)
{
    int i;
    ⟨lock() other locals 60a⟩

    ⟨lock() start of lock, stat 592a⟩
    ⟨lock() increment nlocks 70a⟩
```

```

if(arch_tas(&l->key) == 0){ // lock old value was 0, the lock was not held
    <lock() got the lock, update other fields 60b>
    <lock ifdef LOCKCYCLES 591a>
    return 0;
}
//else

<lock() decrement nlocks 70b>
<lock() when lock already held, stat 592b>

// The lock was already held, need to spin
for(;;){
    <lock() in loop when lock already held, stat 592c>
    i = 0;
    while(l->key){
        <lock() optional priority-inversion for real-time process 504f>
        if(i++ > 100000000){
            i = 0;
            lockloop(l, pc);
        }
    }
    // try again
    <lock() increment nlocks 70a>
    if(arch_tas(&l->key) == 0){
        <lock() got the lock, update other fields 60b>
        <lock ifdef LOCKCYCLES 591a>
        return 1;
    }
    <lock() decrement nlocks 70b>
}
return -1; // unreachable
}

```

The connection between lock() and arch_tas() is straightforward:

```

lock(&l):
    arch_tas(&l->key)
    |
    v
+-----+
| key: 0  | -> arch_tas returns 0, key set to 1: got it!
+-----+
| key: 1  | -> arch_tas returns 1, key stays 1: spin...
+-----+

```

For debugging, each Lock also records the program counter of the caller who acquired it (pc) and the CPU that holds it. Similarly, each Proc remembers its lastlock, making it easier to diagnose deadlocks or lock-ordering violations:

```

<Lock debugging fields 59a>≡ (58c) 69b▷
// for debugging, the caller who did the lock()
kern_addr pc;

```

```

<Proc debugging fields 59b>≡ (49f) 61c▷
Lock* lastlock;

```

```

<Lock other fields 59c>≡ (58c)
// option<ref<Cpu>>, None when key = 0?
Cpu *cpu; // not that used, only in iprintcanlock apparently

```

```
⟨lock() other locals 60a⟩≡ (58d)
```

```
    ulong pc = getcallerpc(&l);
```

```
⟨lock() got the lock, update other fields 60b⟩≡ (58) 62c▷
```

```
    if(up)
        up->lastlock = 1;
    l->pc = pc;
    l->cpu = CPUS(cpu->cpuno); // pad's third bugfix
```

Unlocking is much simpler than locking—no atomicity is needed. The lock holder simply zeroes **key**:

```
⟨function unlock 60c⟩≡ (605b)
```

```
void
unlock(Lock *l)
{
    ⟨unlock ifdef LOCKCYCLES 591c⟩

    ⟨unlock() sanity checks 60d⟩
    l->cpu = nil;
    l->key = 0;
    ⟨unlock() after modified key, call coherence 472a⟩

    ⟨unlock() if delaysched 200c⟩
}
```

```
⟨unlock() sanity checks 60d⟩≡ (60c) 62e▷
```

```
    if(l->key == 0)
        print("unlock: not locked: pc %#p\n", getcallerpc(&l));
```

```
⟨function canlock 60e⟩≡ (605b)
```

```
bool
canlock(Lock *l)
{
    ⟨lock() increment nlocks 70a⟩
    if(arch_tas(&l->key) != 0){ // lock old value != 0, lock was already held
        ⟨lock() decrement nlocks 70b⟩
        return false;
    }
    // else
    ⟨canlock() lock was not already held, update other fields 60f⟩
    ⟨lock ifdef LOCKCYCLES 591a⟩
    return true;
}
```

```
⟨canlock() lock was not already held, update other fields 60f⟩≡ (60e) 69d▷
```

```
    if(up)
        up->lastlock = 1;
    l->pc = getcallerpc(&l);
    l->cpu = CPUS(cpu->cpuno);
```

4.6 Uninterruptable ILock

An **ILock** is a spin lock that also disables interrupts before acquiring the lock. This is necessary when a data structure is accessed both from normal kernel code (syscalls) and from interrupt handlers. Without disabling interrupts, a deadlock can occur: a syscall acquires the lock, an interrupt fires on the same CPU, the interrupt handler tries to acquire the same lock, and spins forever because the interrupted syscall is the one holding it. **ILock** solves this by calling `arch_splhi()` before `arch_tas()`, and saving the old interrupt state in `Lock.sr` so

`iunlock()` can restore it. In practice, `ILock` is used for structures accessed from interrupt handlers: the timer list, device driver queues, etc.

```
<struct ILock 61a>≡ (604)
    typedef Lock ILock;
```

```
<Lock ilock fields 61b>≡ (58c)
    bool_ushort isilock; // false when from lock(), true when from ilock()
    ulong sr; // saved priority level when using ilock() to restore in iunlock()
```

```
<Proc debugging fields 61c>+≡ (49f) <59b 63b>
    Lock *lastilock;
```

```
<function ilock 61d>≡ (605b)
    // To provide mutual exclusion with interrupt code and avoiding deadlock.
    // By using arch_splhi() we disable interrupts while running the critical region
    // code.
    void
    ilock(Lock *l)
    {
        ulong x;
        ulong pc;

        pc = getcallerpc(&l);
        lockstats.locks++;

        x = arch_splhi();
        // no need to take care of up->nlock++ here, we have disabled interrupt
        // so no risk of getting scheduled
        if(arch_tas(&l->key) != 0){
            lockstats.glare++;
            /*
             * Cannot also check l->pc, l->cpu, or l->isilock here
             * because they might just not be set yet, or
             * (for pc and m) the lock might have just been unlocked.
             */
            for(;;){
                lockstats.inglare++;
                arch_splx(x);
                while(l->key)
                    ;
                // let's try again
                x = arch_splhi();
                if(arch_tas(&l->key) == 0)
                    goto acquire;
            }
        }
        acquire:
        <ilock() got the lock, update other fields 61e>
        <lock ifdef LOCKCYCLES 591a>
    }
}
```

```
<ilock() got the lock, update other fields 61e>≡ (61d) 69f>
    if(up)
        up->lastilock = l;
    l->isilock = true;
    l->sr = x;
    l->pc = pc;
    l->cpu = CPUS(cpu->cpuno);
```

```

<function iunlock 62a>≡ (605b)
void
iunlock(Lock *l)
{
    ulong sr;

    <iunlock ifdef LOCKCYCLES 591d>
    <iunlock() sanity checks 62b>
    sr = l->sr;
    l->cpu = nil;
    l->key = 0;
    <iunlock() after modified key, call coherence 472b>
    <iunlock() decrement ilockdepth 70h>
    if(up)
        up->lastilock = nil;
    arch_splx(sr);
}

<iunlock() sanity checks 62b>≡ (62a)
if(l->key == 0)
    print("iunlock: not locked: pc %#p\n", getcallerpc(&l));
if(!l->isilock)
    print("iunlock of lock: pc %#p, held by %#lux\n", getcallerpc(&l), l->pc);
if(arch_islo())
    print("iunlock while lo: pc %#p, held by %#lux\n", getcallerpc(&l), l->pc);

<lock() got the lock, update other fields 62c>+≡ (58) <60b 69c>
l->isilock = false;

<canlock() lock not held, update other fields 62d>≡
l->isilock = false;

<unlock() sanity checks 62e>+≡ (60c) <60d 69e>
if(l->isilock)
    print("unlock of ilock: pc %lux, held by %lux\n", getcallerpc(&l), l->pc);

```

4.7 Waiting QLock

A **QLock** (queuing lock, also known as a mutex) is a lock where the caller goes to sleep instead of spinning when the lock is not available. This is important for critical regions that may take a long time—for instance, I/O operations or file system lookups. Spinning for milliseconds would waste CPU time; sleeping lets the scheduler run other processes. The waiting processes form a queue (**head/tail**), linked through **Proc.qnext**. When the lock holder calls **qunlock()**, it wakes the first waiter in the queue. The internal coordination uses a **Lock** (spin lock) to protect the queue itself, but the spin is only held for the few instructions needed to enqueue or dequeue—the actual wait is a sleep.

```

<struct QLock 62f>≡ (604)
// Kernel basic lock with Queue (renamed to avoid ambiguity with libc.h QLock)
// a.k.a Mutex
struct KQLock
{
    bool locked; /* flag */

    // list<ref<Proc>> (next = Proc.qnext)
    Proc *head; /* next process waiting for object */
    // option<ref<Proc>> (direct access to tail, queue)
    Proc *tail; /* last process waiting for object */
}

```

```

    <QLock debugging fields 63a>

    Lock use;    /* to access Qlock structure */
};

<QLock debugging fields 63a>≡ (62f)
    kern_addr qpc;    /* pc of the holder */ // for debugging?

<Proc debugging fields 63b>+≡ (49f) <61c
    kern_addr qpc;    /* pc calling last blocking qlock */

<Proc extra fields 63c>≡ (49f) 107d▷
    // list<ref<Proc>> QLock.head or RWLock.head (or Procalloc.free)
    Proc *qnext;    /* next process on queue for a QLock */

<Procstate cases 63d>≡ (51b) 65c▷
    Queueing, // see qlock()

```

With the data structure in place, the `qlock()` function itself is straightforward. If the lock is free, take it immediately. Otherwise, add the current process to the wait queue and go to sleep:

```

<function qlock 63e>≡ (606a)
    void
    qlock(QLock *q)
    {
        Proc *p;

        <qlock() sanity checks 64b>
        lock(&q->use);
        <qlock() start of qlock, stat 592g>
        if(!q->locked) {
            q->locked = true;
            q->qpc = getcallerpc(&q);
            unlock(&q->use);
            return;
        }
        // else
        <qlock() sanity check up 64a>
        <qlock() when qlock already held, stat 592h>

        // add_queue(q, up)
        p = q->tail;
        if(p == nil)
            q->head = up;
        else
            p->qnext = up;
        q->tail = up;
        // up->qnext could be non nil before? no otherwise that means
        // the process is already waiting for a lock and so had no occasion
        // to run another qlock() instruction.
        up->qnext = nil;

        up->state = Queueing;
        up->qpc = getcallerpc(&q);
        unlock(&q->use);

        // switch to another process!
        sched();
        // will resume here when another process unlock() the lock and ready() us
        q->qpc = getcallerpc(&q);
    }

```


`<qlock() sanity check up 64a>≡` (63e)

```
    if(up == nil)
        panic("qlock");
```

`<qlock() sanity checks 64b>≡` (63e) 70e▷

```
    if(q->use.key == 0x55555555) // dead code??
        panic("qlock: q %#p, key 5*\n", q);
```

`<function qunlock 64c>≡` (606a)

```
void
qunlock(QLock *q)
{
    Proc *p;

    lock(&q->use);
    <qunlock() sanity checks 64d>

    p = q->head;
    if(p){
        // dequeue(q)
        q->head = p->qnext;
        if(q->head == nil)
            q->tail = nil;

        unlock(&q->use);
        ready(p);
    }else{
        q->locked = false;
        q->qpc = nilptr;
        unlock(&q->use);
    }
}
```

`<qunlock() sanity checks 64d>≡` (64c)

```
    if(!q->locked)
        print("qunlock called with qlock not held, from %#p\n",
            getcallerpc(&q));
```

`<function canqlock 64e>≡` (606a)

```
bool
canqlock(QLock *q)
{
    if(!canlock(&q->use))
        return false;
    if(q->locked){
        unlock(&q->use);
        return false;
    }
    // else

    q->locked = true;
    q->qpc = getcallerpc(&q);
    unlock(&q->use);
    return true;
}
```

4.8 Readers/Writer RWLock

An RWlock allows multiple simultaneous readers or a single exclusive writer. This is useful for data structures that are read frequently but modified rarely, such as mount tables (*Mhead*^{300c}), process groups (*Pgrp*^{302d}), and environment groups (*Egrp*^{533a}). The implementation is subtle: it must avoid starving either readers or writers, and like QLock, waiters sleep rather than spin.

<struct RWlock 65a>≡ (604)

```
struct RWlock
{
    int readers; /* number of readers */
    bool writer; /* have a writer? */

    // list<ref<Proc>> (next = Proc.qnext)
    Proc *head; /* list of waiting processes */
    // option<ref<Proc>> (direct access to tail, queue)
    Proc *tail;
    // option<ref<Proc>>
    Proc *wproc; /* writing proc */

    <RWlock debugging fields 65b>
    // Extra
    Lock use;
};
```

<RWlock debugging fields 65b>≡ (65a)

```
kern_addr wpc; /* pc of writer */
```

Like QLock, waiting for a read lock or a write lock puts the process into a dedicated state so the scheduler knows why it is blocked:

<Procstate cases 65c>+≡ (51b) <63d 66c>

```
QueueingR, // see rlock()
```

<function rlock 65d>≡ (606a)

```
void
rlock(RWlock *q)
{
    Proc *p;

    lock(&q->use);
    rwstats.rlock++;
    if(!q->writer && q->head == nil){
        /* no writer, go for it */
        q->readers++;
        unlock(&q->use);
        return;
    }
    // else, a writer or a waiting queue??

    <rlock() sanity check up 66a>
    rwstats.rlockq++;

    // add_queue(q, up)
    p = q->tail;
    if(p == nil)
        q->head = up;
    else
        p->qnext = up;
    q->tail = up;
    up->qnext = nil;
```

```

    up->state = QueueingR;
    unlock(&q->use);
    sched();
    // will resume here when another process unlock() the lock and ready() us
}

```

<rlock() sanity check up 66a>≡ (65d)

```

    if(up == nil)
        panic("rlock");

```

<function runlock 66b>≡ (606a)

```

void
runlock(RWlock *q)
{
    Proc *p;

    lock(&q->use);
    p = q->head;
    if(--(q->readers) > 0 || p == nil){
        unlock(&q->use);
        return;
    }
    // else, p != nil && q->readers == 0

    /* start waiting writer */
    if(p->state != QueueingW)
        panic("runlock");

    // dequeue(q)
    q->head = p->qnext;
    if(q->head == nil)
        q->tail = nil;

    q->writer = true;
    unlock(&q->use);
    ready(p);
}

```

Writers wait separately from readers:

<Procstate cases 66c>+≡ (51b) <65c 108b>

```

    QueueingW, // see wlock()

```

<function wlock 66d>≡ (606a)

```

void
wlock(RWlock *q)
{
    Proc *p;

    lock(&q->use);
    rwstats.wlock++;
    if(q->readers == 0 && !q->writer){
        /* noone waiting, go for it */
        q->wpc = getcallerpc(&q);
        q->wproc = up;
        q->writer = true;
        unlock(&q->use);
        return;
    }
    // else, already a writer or readers

```

```

/* wait */
⟨wlock() sanity check up 67a⟩
rwstats.wlockq++;

// add_queue(q, up)
p = q->tail;
if(p == nil)
    q->head = up;
else
    p->qnext = up;
q->tail = up;
up->qnext = nil;

up->state = QueueingW;
unlock(&q->use);
sched();
}

⟨wlock() sanity check up 67a⟩≡ (66d)
    if(up == nil)
        panic("wlock");

⟨function wunlock 67b⟩≡ (606a)
    void
    wunlock(RWlock *q)
    {
        Proc *p;

        lock(&q->use);
        p = q->head;
        if(p == nil){
            q->writer = false;
            unlock(&q->use);
            return;
        }
        if(p->state == QueueingW){
            /* start waiting writer */
            // dequeue(q)
            q->head = p->qnext;
            if(q->head == nil)
                q->tail = nil;

            unlock(&q->use);
            ready(p);
            return;
        }

        if(p->state != QueueingR)
            panic("wunlock");

        /* waken waiting readers */
        while(q->head != nil && q->head->state == QueueingR){
            p = q->head;
            q->head = p->qnext;
            q->readers++;
            ready(p);
        }

        if(q->head == nil)

```

```

        q->tail = nil;
        q->writer = false;
        unlock(&q->use);
    }

⟨function canrlock 68a⟩≡ (606a)
/* same as rlock but punts if there are any writers waiting */
bool
canrlock(RWlock *q)
{
    lock(&q->use);
    rwstats.rlock++;
    if(q->writer == false && q->head == nil){
        /* no writer, go for it */
        q->readers++;
        unlock(&q->use);
        return true;
    }
    unlock(&q->use);
    return false;
}

```

4.9 Atomic References and Counter

We already saw `arch_xinc()/arch_xdec()`, which atomically adjust a raw counter. `Ref` below wraps a counter with a `Lock` instead of using `arch_xinc` directly. This may seem redundant. However, the `Lock` does allow `decref()` to additionally and atomically check for underflow and panic. But the main reason is likely historical: `Ref` is portable code (in `port/`) that predates the arch-specific atomic operations, and using a `Lock` keeps it independent of any particular architecture's atomic primitives.

```

⟨struct Ref 68b⟩≡ (604)
// For reference counting shared things (e.g. a Page)
struct Ref
{
    long ref;

    Lock;
};

```

```

⟨function incref 68c⟩≡ (606b)
long
incref(Ref *r)
{
    long x;

    lock(r);
    x = ++r->ref;
    unlock(r);
    return x;
}

```

```

⟨function decref 68d⟩≡ (606b)
long
decref(Ref *r)
{
    long x;

    lock(r);

```

```

    x = --r->ref;
    unlock(r);
    if(x < 0)
        panic("decref pc=%#p", getcallerpc(&r));
    return x;
}

```

Many kernel objects (pages, channels, segments) embed a `Ref` for reference counting—the object is freed when `decref()` returns 0.

```

⟨struct Counter 69a⟩≡ (604)
    typedef struct Ref Counter;

```

4.10 Safeguards

Concurrency bugs are notoriously hard to find, so the kernel includes several runtime checks to catch common mistakes: unlocking from the wrong process, sleeping while holding a spin lock, or taking a `qlock` while interrupts are disabled. We already saw several safeguards earlier: `Lock.pc` and `Proc.lastlock` recording who acquired a lock, `Lock.cpu` tracking which processor holds it, and `splpc` in `Cpu` remembering who disabled interrupts. This section covers additional checks.

4.10.1 Lock.p

Each `Lock` records which process acquired it (in `Lock.p`). On unlock, the kernel checks that the same process is releasing it—a mismatch indicates a bug (for instance, a context switch that should not have happened while the lock was held):

```

⟨Lock debugging fields 69b⟩+≡ (58c) ◁59a 591b▷
    // option⟨ref⟨Proc⟩⟩, None when key == 0
    Proc *p; // the process who did the locking should be the same unlocking

⟨lock() got the lock, update other fields 69c⟩+≡ (58) ◁62c
    l->p = up;

⟨canlock() lock was not already held, update other fields 69d⟩+≡ (60e) ◁60f
    l->p = up;

⟨unlock() sanity checks 69e⟩+≡ (60c) ◁62e
    if(l->p != up)
        print("unlock: up changed: pc %#p, acquired at pc %lux, lock p %#p, unlock up %#p\n", getcallerpc(&l), l->p);

⟨ilock() got the lock, update other fields 69f⟩+≡ (61d) ◁61e 70g▷
    l->p = up;

```

4.10.2 Proc.nlocks

`Proc.nlocks` counts how many spin locks the current process holds. This is crucial: a process holding a spin lock must not be scheduled out, because the lock protects a critical region that must complete without interruption. The scheduler checks `nlocks` and delays rescheduling until all locks are released. Similarly, `qlock()` checks `nlocks` and prints a warning if a process tries to sleep while holding a spin lock—that would deadlock, because no other process could run to release the lock. Note that `nlocks` itself uses `arch_xinc()/arch_xdec()` (via `incbnt()/decnt()`) rather than `incbnt()/decbnt()`, because the latter use `lock()/unlock()`—which would modify `nlocks`, creating a circular dependency.

```

⟨Proc synchronization fields 69g⟩≡ (49f) 203e▷
    // As long as the current process hold spinlocks (to kernel data structures),
    // we will not schedule another process in sched() or unlock();
    // only the last unlock will eventually cause a rescheduling.
    Ref nlocks; /* number of locks held by proc */

```

```

⟨lock() increment nlocks 70a⟩≡ (60e 58)
    if(up)
        incnt(&up->nlocks);    /* prevent being scheded */

⟨lock() decrement nlocks 70b⟩≡ (60e 58)
    if(up)
        decnt(&up->nlocks);

⟨function incnt 70c⟩≡ (605b)
    // See also ref.c incref() and decref(), but we can't use them here as they
    // themselves rely on lock() and unlock().
    static void
    incnt(Ref *r)
    {
        arch_xinc(&r->ref);
    }

⟨function decnt 70d⟩≡ (605b)
    static int
    decnt(Ref *r)
    {
        int x;

        x = arch_xdec(&r->ref);
        if(x < 0)
            panic("decnt pc=%#p", getcallerpc(&r));
        return x;
    }

⟨qlock() sanity checks 70e⟩+≡ (63e) ◁64b 70i▷
    if(up != nil && up->nlocks.ref)
        print("qlock: %#p: nlocks %ld\n", getcallerpc(&q), up->nlocks.ref);

```

4.10.3 Cpu.ilockdepth

Cpu.ilockdepth counts how many ILocks are held on this CPU. Like nlocks, it is checked in qlock(): sleeping while holding an ILock (with interrupts disabled) would deadlock the entire CPU, because the timer interrupt that drives the scheduler could not fire. No atomic operation is needed here because interrupts are already disabled when ilockdepth is modified.

```

⟨Cpu other fields 70f⟩≡ (48a) 195d▷
    int ilockdepth;

⟨ilock() got the lock, update other fields 70g⟩+≡ (61d) ◁69f
    cpu->ilockdepth++;

⟨iunlock() decrement ilockdepth 70h⟩≡ (62a)
    cpu->ilockdepth--;

⟨qlock() sanity checks 70i⟩+≡ (63e) ◁70e
    if(cpu->ilockdepth != 0)
        print("qlock: %#p: ilockdepth %d\n", getcallerpc(&q), cpu->ilockdepth);

```

4.11 Synchronization

The locking primitives above provide mutual exclusion: preventing two threads from accessing the same data at the same time. But they do not solve synchronization: how does a process wait for something to happen (e.g., a disk read to complete, a pipe to have data)? For that, the kernel needs a way to put a process to sleep and wake it up later.

4.11.1 Rendez-vous

The fundamental synchronization mechanism in the kernel is `sleep()/wakeup()`, built around the `Rendez` structure (a “rendezvous point”). A process calls `sleep()` with a condition function; if the condition is not yet true, the process is put to sleep. Later, when another process or an interrupt handler makes the condition true, it calls `wakeup()` to resume the sleeper. The full code is in Chapter 6.

4.11.2 Producer/Consumer Queue

The I/O `Queue` structure (Chapter 16) uses `sleep()/wakeup()` to implement producer/consumer synchronization: a reader sleeps when the queue is empty, a writer sleeps when it is full, and each wakes the other.

4.11.3 Semaphore

The kernel also provides semaphores (`SEMACQUIRE/SEMRELEASE` syscalls), but these are primarily for user-level synchronization. The C library’s `Lock` and `QLock` (in `libcore/libc/port/lock.c`) are implemented on top of these kernel semaphores. Inside the kernel itself, the spin locks and qlocks described above are used instead.

Part I

Virtualizing the Machine

The kernel's central job is to give each process the illusion of having the entire machine to itself. This part covers the mechanisms that make this illusion work: virtual memory (page tables and page faults) backed by a physical page allocator, process creation and management, interrupt and syscall handling (the boundary between the virtual and real machines), scheduling (virtualizing the CPU so each process gets its turn), time keeping, and finally the boot sequence that sets all of this up.

Chapter 5

Memory

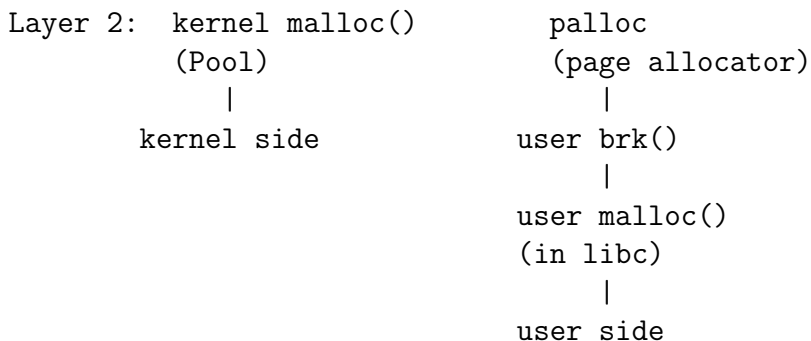
From the user’s point of view, memory is simple: call `brk()` (or `malloc()`) and get memory. Behind this simple interface lies a hierarchy of allocators, each solving a different problem.

`<systab memory syscalls 73>≡` (41f)
`[BRK] sysbrk,`

5.1 Overview

Memory management has a bootstrapping problem: you need memory to track which memory is free, but you do not have an allocator yet. The kernel solves this by building allocators in layers, each bootstrapped from the previous:

0. The kernel binary’s data section contains statically allocated allocator metadata. The boot code also detects physical memory here and records it in `Conf.mem`.
1. `xalloc()`^{76c} carves large structures from those banks. From here the hierarchy forks.
- 2a. On the kernel side, `malloc()`^{83b} provides fine-grained allocation from a `Pool` backed by `xalloc`.
- 2b. On the user side, `palloc`^{88d} manages physical pages. User `brk()` takes pages from `palloc`; user-space `malloc()` (in `libc`) subdivides them.



Layer 1: `xalloc`
(allocates `Proc[]`, `Page[]`,
`Pool` arenas, from `Confmem`)

Layer 0: `kernel binary data section`
(`xlists`, `conf`, `devtab`, etc.)

edata/end mark free memory start

Hardware: physical RAM (detected by confinit)

The following sections present each layer in detail.

5.2 Long-lived structure allocator: `xalloc()`

`xalloc()` is the bottom-level dynamic allocator. It manages the raw physical memory banks from `Conf.mem` using a simple first-fit, non-compacting algorithm. Its own metadata—the `xlists` global, containing the `Hole[128]` array—lives in the kernel’s data section (layer 0), so it needs no memory allocation to get started. `xalloc` is called only during initialization to allocate a few large, long-lived structures—the `Proc` array, the `Page`⁸⁷ array, and the `malloc` pool—that will never be freed. This simplicity is deliberate: the fixed-size hole array cannot handle heavy fragmentation, but it does not need to.

5.2.1 Data structures

Xalloc

The Xalloc structure manages free memory as a sorted linked list of Holes.

```
<struct Xalloc 74a>≡ (658)
// Long lived data structure allocator (singleton)
struct Xalloc
{
    // array<Hole> where each Hole is linked to another hole
    Hole hole[Nhole];

    // list<ref<Hole>> (next = Hole.next) list of free hole entries (addr=top=size=0)
    Hole* unused_slots;

    // list<ref<Hole>> (next = Hole.next) memory holes, sorted by their top addr
    Hole* sorted_holes;

    // extra
    Lock;
};
```

```
<Hole extra fields 74b>≡ (74d)
Hole* next; // list<ref<Hole>> of Xalloc.sorted_holes or Xalloc.unused_slots
```

```
<constants holes 74c>≡ (658)
enum
{
    Nhole = 128,
    <constant Magichole 76b>
};
```

Each `Hole` describes a contiguous region of free memory (between `addr` and `top`). The `Hole` entries themselves are pre-allocated in a fixed-size array (`hole[Nhole]`), and those not currently describing a free region are kept in a separate “unused slots” list.

```
<struct Hole 74d>≡ (658)
struct Hole
{
    // between addr and top the memory is free, this is the "hole"
    phys_addr addr;
```

```

phys_addr top;
ulong size; // top - addr (in bytes)

// Extra
⟨Hole extra fields 74b⟩
};

```

Here is the layout after initialization (one memory bank) and after two allocations:

After `xinit()` with one `Confmem` bank:

```

hole[0]:  addr=A  top=B  size=B-A    (the whole free bank)
hole[1..127]: unused

sorted_holes --> hole[0] --> nil
unused_slots --> hole[1] --> hole[2] --> ... --> hole[127] --> nil

```

Physical memory:

```

  B  +-----+
      |      free (one big hole)      |
  A  +-----+

```

After `xalloc(s1)` and `xalloc(s2)`:

```

hole[0]:  addr=A+s1+s2  top=B  size=B-A-s1-s2  (shrunk)

sorted_holes --> hole[0] --> nil

```

Physical memory:

```

  B  +-----+
      | free (hole[0], shrunk) |
      +-----+
      | Xhdr | data2 (s2 bytes) | <-- second xalloc
      +-----+
      | Xhdr | data1 (s1 bytes) | <-- first xalloc
  A  +-----+

```

Allocations grow upward from the bottom of the hole; the hole shrinks from below. Each allocation is prefixed with an `Xhdr` containing the size and a magic number (for corruption detection via `xmerge()`).

If the first allocation (`s1`) is later freed via `xfree()`, the sorted hole list would then contain two holes—the gap left by `s1` and the original remaining free space at the top—which is why `xalloc` needs a list of holes rather than a single pointer.

xlists

There is a single global instance of `Xalloc`, statically allocated in the kernel's data section—this is layer 0, the bootstrap that makes everything else possible:

```

⟨global xlists 75⟩≡ (660b)
static Xalloc  xlists;

```

Xhdr

Each block returned by `xalloc()` is preceded by an `Xhdr` header that records its size and a magic number. The magic number (0x484F4C45, which spells “HOLE” in ASCII) lets `xmerge()` detect memory corruption when a block is freed:

```
<struct Xhdr 76a>≡ (658)
// What is the connection with Hole? A Hole that get used will gets
// its top and size decremented, and this newly allocated part will describe
// a portion of used memory, and at this memory there will be a header
// and then just after the actual memory xalloc'ed by someone
struct Xhdr
{
    // bookkeeping area
    ulong size;
    ulong magix;

    char data[]; // memory pointer returned by xalloc
};

<constant Magichole 76b>≡ (74c)
Magichole = 0x484F4C45, /* HOLE */
```

5.2.2 xalloc()

`xalloc()` walks the sorted hole list looking for the first hole large enough (first-fit). It then carves the requested amount from the bottom of the hole, prepends an `Xhdr`, and shrinks the hole. If the hole is fully consumed, it is moved back to the unused slots list.

```
<function xalloc 76c>≡ (660b)
kern_vp
xalloc(ulong size)
{
    return xallocz(size, true);
}

<function xallocz 76d>≡ (660b)
kern_vp
xallocz(ulong size, bool zero)
{
    Xhdr *p;
    Hole *h, **l;

    /* add room for magix & size overhead, round up to nearest vlong */
    size += BY2V + offsetof(Xhdr, data[0]);
    size &= ~(BY2V-1);

    ilock(&xlists);
    l = &xlists.sorted_holes;
    for(h = *l; h; h = h->next) {
        // found an appropriate hole
        if(h->size >= size) {
            p = (Xhdr*)KADDR(h->addr);
            h->addr += size; // shrink towards top
            h->size -= size;

            // This hole is now fully used (which is rare because one
            // rarely does an xalloc with the remaining size of a hole).
            // We can put it back in the list of free hole entries.
            if(h->size == 0) {
```

```

        *l = h->next;
        h->next = xlists.unused_slots;
        xlists.unused_slots = h;
    }

    iunlock(&xlists);
    if(zero)
        memset(p, 0, size);
    p->magix = Magichole;
    p->size = size;
    return p->data;
}
l = &h->next;
}
iunlock(&xlists);
return nil; // panic?
}

```

5.2.3 xfree()

```

⟨function xfree 77a⟩≡ (660b)
void
xfree(kern_vp p)
{
    Xhdr *x;

    x = (Xhdr*)((ulong)p - offsetof(Xhdr, data[0]));
    ⟨xfree() sanity check magix mark 77b⟩
    xhole(PADDR((kern_addr)x), x->size);
}

```

```

⟨xfree() sanity check magix mark 77b⟩≡ (77a)
if(x->magix != Magichole) {
    xsummary();
    panic("xfree(%#p) %#ux != %#lux", p, Magichole, x->magix);
}

```

hole() is the inverse of xalloc()^{76c}: it adds a region of free memory to the sorted hole list. It tries to merge with adjacent holes (if the new region's bottom touches an existing hole's top, or vice versa) to avoid fragmentation. This is also the function called by xinit()^{246b} to seed the allocator with the initial Confmem banks.

```

⟨function xhole 77c⟩≡ (660b)
void
xhole(phys_addr addr, ulong size)
{
    phys_addr top;
    Hole *h, *c, **l;

    if(size == 0)
        return;

    top = addr + size;
    ilock(&xlists);
    l = &xlists.sorted_holes;
    for(h = *l; h; h = h->next) {
        if(h->top == addr) {
            h->size += size;
            h->top = h->addr+h->size;

```

```

        c = h->next;
        if(c && h->top == c->addr) {
            h->top += c->size;
            h->size += c->size;
            h->next = c->next;
            c->next = xlists.unused_slots;
            xlists.unused_slots = c;
        }
        iunlock(&xlists);
        return;
    }
    if(h->addr > addr)
        break;
    l = &h->next;
}
if(h && top == h->addr) {
    h->addr -= size;
    h->size += size;
    iunlock(&xlists);
    return;
}

if(xlists.unused_slots == nil) {
    iunlock(&xlists);
    print("xfree: no free holes, leaked %lud bytes\n", size);
    return;
}

h = xlists.unused_slots;
xlists.unused_slots = h->next;
h->addr = addr;
h->top = top;
h->size = size;
h->next = *l;
*l = h;
iunlock(&xlists);
}

```

5.2.4 xmerge()

<function xmerge 78a> ≡ (660b)

```

bool
xmerge(kern_vp vp, kern_vp vq)
{
    Xhdr *p, *q;

    p = (Xhdr*)((ulong)vp - offsetof(Xhdr, data[0]));
    q = (Xhdr*)((ulong)vq - offsetof(Xhdr, data[0]));

    <xmerge() sanity check magix 78b>
    if((byte*)p+p->size == (byte*)q) {
        p->size += q->size;
        return true;
    }
    return false;
}

```

<xmerge() sanity check magix 78b> ≡ (78a)

```

if(p->magix != Magichole || q->magix != Magichole) {

```

```

    <xmerge() debug info when not magichole ??>
    panic("xmerge(%#p, %#p) bad magic %#lux, %#lux",
        vp, vq, p->magix, q->magix);
}

<function xspanalloc 79a>≡ (660b)
kern_vp
xspanalloc(ulong size, int align, ulong span)
{
    ulong a, v, t;
    a = (kern_addr)xalloc(size+align+span);
    <xspanalloc() sanity check a 79b>

    if(span > 2) {
        v = (a + span) & ~(span-1);
        t = v - a;
        if(t > 0)
            xhole(PADDR(a), t);
        t = a + span - v;
        if(t > 0)
            xhole(PADDR(v+size+align), t);
    }
    else
        v = a;

    if(align > 1)
        v = (v + align) & ~(align-1);

    return (kern_vp)v;
}

<xspanalloc() sanity check a 79b>≡ (79a)
if(a == nilptr)
    panic("xspanalloc: %lud %d %lux", size, align, span);

```

5.3 Memory pools

`xalloc` is too coarse for general use—it can only handle a small number of holes (128). The kernel needs a finer-grained allocator for the many small, short-lived structures it creates (and free) during normal operation (channel tables, path strings, etc.). This is provided by the `Pool` allocator, which manages arenas obtained from `xalloc` and subdivides them using a best-fit algorithm.

5.3.1 Pool

The `Pool` structure is defined in `lib_core/include/pool.h`, not in the kernel, because the same allocation machinery is shared between the kernel's `malloc()` and the user-space `malloc()` in `libc`—only the backing allocator differs (`xalloc` for the kernel, `brk` for user space). The full definition is shown below as a verbatim excerpt but see the `LIBCORE` book [Pad16c] for more information:

```

// from pool.h
struct Pool {
    char* name;
    ulong maxsize;

    ulong cursize;

```



```

ulong curfree;
ulong curalloc;

ulong minarena; /* smallest size of new arena */
ulong quantum; /* allocated blocks should be multiple of */
ulong minblock; /* smallest newly allocated block */

void* freeroot; /* actually Free* */
void* arenalist; /* actually Arena* */

void* (*alloc)(ulong);
int (*merge)(void*, void*);
void (*move)(void* from, void* to);

int flags;
int nfree;
int lastcompact;

void (*lock)(Pool*);
void (*unlock)(Pool*);
void (*print)(Pool*, char*, ...);
void (*panic)(Pool*, char*, ...);
void (*logstack)(Pool*);

void* private;
};

```

The important fields are `alloc` (the callback to get more memory—`xalloc` for the kernel), `merge` (to coalesce freed blocks—`xmerge` for the kernel), `freeroot` (the free-block tree), `arenalist` (the list of memory regions obtained from `alloc`), and the `lock/unlock` callbacks for thread safety.

The `Private` structure holds the per-pool lock and a message buffer. The message buffer is needed because the pool code cannot print while holding the lock (it would risk deadlock if `print` itself tries to allocate), so messages are saved and printed after unlock:

```

⟨pool.c struct Private 80a⟩≡ (661b)
struct Private {
    Lock      lk;
    char      msg[256]; /* a rock for messages to be printed at unlock */
};

```

5.3.2 plock()

The pool's lock callback uses `ilock()` (not a regular `lock()`) because interrupt handlers sometimes need to allocate memory. If a regular lock were used, a syscall holding the pool lock could be interrupted, and the interrupt handler's allocation attempt would deadlock.

```

⟨function plock 80b⟩≡ (661b)
static void
plock(Pool *p)
{
    Private *pv;

    pv = p->private;
    ilock(&pv->lk);
}

```

```

    pv->lk.pc = getcallerpc(&p);
    pv->msg[0] = 0;
}

⟨function punlock 81a⟩≡ (661b)
static void
punlock(Pool *p)
{
    Private *pv;
    char msg[sizeof pv->msg];

    pv = p->private;
    if(pv->msg[0] == 0){
        iunlock(&pv->lk);
        return;
    }

    memmove(msg, pv->msg, sizeof msg);
    iunlock(&pv->lk);
    iprint("%.*s", sizeof pv->msg, msg);
}

```

5.3.3 pmainmem

The two pool instances, **pmainmem** (for general kernel allocations, 4 MB) and **pimagmem** (for graphics image data, 16 MB, see GRAPHICS book [Pad16d]), have their **Pool** structures statically allocated in the data section—another layer 0 bootstrap (the pool structures themselves, not the memory they manage).

Their **alloc** callback is **xalloc**, so when the pool needs more memory it asks **xalloc** for a new arena. The 4 MB **maxsize** is a cap, not a reservation—Pool only requests arenas as needed, leaving the remaining **xalloc** memory available for other direct allocations (the **Proc** array, the **Page** array, etc.). Here is how the kernel memory looks after initialization:

xalloc-managed kernel memory (kernelpercent of RAM):

```

+-----+
| free (xalloc hole)          |
+-----+
| pmainmem arena 2           | (Pool, on demand)
+-----+
| pimagem arena 1            | (Pool, on demand)
+-----+
| pmainmem arena 1           | (Pool, on demand)
+-----+
| Page array                  | (direct xalloc)
+-----+
| Proc array (~6 MB)         | (direct xalloc)
+-----+

```

Arenas from different pools and direct **xalloc** calls can interleave—the order depends on when each allocation happens during boot and normal operation. Arenas are never returned to **xalloc** (no **xfree** call), so the pools only grow, which avoids fragmentation in **xalloc**’s hole list.

```

⟨global pmainmem 81b⟩≡ (661b)
static Private pmainpriv;
static Pool pmainmem = {

```

```

.name= "Main",
.maxsize= 4*MB,
.minarena= 128*KB,
.quantum= 32,

.alloc= xalloc,
.merge= xmerge,

.flags= POOL_TOLERANCE,

.lock=plock,
.unlock= punlock,

.print= poolprint,
.panic= ppanic,

.private= &pmainpriv,
};

```

```

⟨global pimagmem 82a⟩≡ (661b)
static Private pimagpriv;
static Pool pimagmem = {
    .name= "Image",
    .maxsize= 16*MB,
    .minarena= 2*MB,
    .quantum= 32,

    .alloc= xalloc,
    .merge= xmerge,

    .flags= 0,

    .lock= plock,
    .unlock= punlock,

    .print= poolprint,
    .panic= ppanic,

    .private= &pimagpriv,
};

```

```

⟨global mainmem and imagmem 82b⟩≡ (661b)
// exported in include/pool.h, defined here!
Pool* mainmem = &pmainmem;
Pool* imagmem = &pimagmem;

```

5.3.4 poolprint()

`poolprint()`^{82c} and `ppanic()`^{83a} are the pool’s diagnostic callbacks. They are called by the pool allocator code (in `libc`) when it detects corruption or wants to log warnings. Because the pool lock is held when these are called, they cannot print directly—`print()` calls `write()` which may acquire locks internally, risking deadlock. Instead, `poolprint()` uses `vseprint()` (which does not allocate or take locks) to append to the message buffer in `Private`, and `punlock()` prints it after releasing the lock:

```

⟨function poolprint 82c⟩≡ (661b)
/*
 * because we can't print while we're holding the locks,
 * we have to save the message and print it once we let go.

```

```

    */
static void
poolprint(Pool *p, char *fmt, ...)
{
    va_list v;
    Private *pv;

    pv = p->private;
    va_start(v, fmt);
    vseprint(pv->msg+strlen(pv->msg), pv->msg+sizeof pv->msg, fmt, v);
    va_end(v);
}

⟨function ppanic 83a⟩≡ (661b)
static void
ppanic(Pool *p, char *fmt, ...)
{
    va_list v;
    Private *pv;
    char msg[sizeof pv->msg];

    pv = p->private;
    va_start(v, fmt);
    vseprint(pv->msg+strlen(pv->msg), pv->msg+sizeof pv->msg, fmt, v);
    va_end(v);
    memmove(msg, pv->msg, sizeof msg);
    iunlock(&pv->lk);
    panic("%s", msg);
}

```

5.4 Kernel malloc()

The kernel's `malloc()` and `free()` are thin wrappers around `poolalloc()/poolfree()` on the `mainmem` pool. Unlike user-space `malloc`, the kernel version always zeroes the memory (via `memset`) and can optionally record the caller's PC in a tag area for debugging (when `Npadlong` is nonzero).

5.4.1 malloc()

```

⟨function malloc 83b⟩≡ (661c)
kern_vp
malloc(ulong size)
{
    kern_vp v;

    v = poolalloc(mainmem, size + Npadlong*sizeof(ulong));
    if(v == nil)
        return nil;
    ⟨malloc() if Npadlong 85b⟩
    memset(v, 0, size);
    return v;
}

⟨function free 83c⟩≡ (661c)
void
free(kern_vp v)
{
    if(v != nil)

```

```

    poolfree(mainmem, (ulong*)v-Npadlong);
}

```

5.4.2 Sleeping smalloc()

`smalloc()` (“sleeping malloc”) is a variant of `malloc()` that never fails: if the pool is full, instead of returning `nil`, it sleeps for 100 ms and retries. This is useful for code paths where an allocation failure would be hard to recover from—for instance, when creating kernel data structures during a syscall where the caller expects success. The trade-off is that `smalloc()` cannot be called from interrupt context (sleeping in an interrupt handler would deadlock the system).

```

⟨function smalloc 84a⟩≡ (661c)
// non failing malloc! will repeat until it can
kern_vp
smalloc(ulong size)
{
    kern_vp v;

    for(;;) {
        v = poolalloc(mainmem, size + Npadlong*sizeof(ulong));
        if(v != nil)
            break;
        // else
        tsleep(&up->sleepr, returnfalse, 0, 100);
    }
    ⟨smalloc() if Npadlong 86a⟩
    memset(v, 0, size); // clear
    return v;
}

```

```

⟨function returnfalse 84b⟩≡ (611a)
// usually used as default callback for sleep/tsleep
bool
returnfalse(void*)
{
    return false;
}

```

5.4.3 mallocxxx()

```

⟨function mallocz 84c⟩≡ (661c)
kern_vp
mallocz(ulong size, bool clr)
{
    kern_vp v;

    v = poolalloc(mainmem, size+Npadlong*sizeof(ulong));
    ⟨mallocz() if Npadlong 86b⟩
    if(clr && v != nil)
        memset(v, 0, size);
    return v;
}

```

```

⟨function mallocalign 84d⟩≡ (661c)
kern_vp
mallocalign(ulong size, ulong align, long offset, ulong span)
{

```

```

kern_vp v;

v = poollocalalign(mainmem, size +Npadlong*sizeof(ulong), align,
                   offset-Npadlong*sizeof(ulong), span);
⟨mallocalign() if Npadlong 86c⟩
if(v)
    memset(v, 0, size);
return v;
}

⟨function realloc 85a⟩≡ (661c)
kern_vp
realloc(kern_vp v, ulong size)
{
    kern_vp nv;

    if(v != nil)
        v = (ulong*)v - Npadlong;
    ⟨realloc() if Npadlong 86d⟩

    if(nv = poolrealloc(mainmem, v, size)){
        ⟨realloc() readjust nv if Npadlong 86e⟩
    }
    return nv;
}

```

5.4.4 malloc tags

When `Npadlong` is nonzero, `malloc()` reserves extra words before the returned pointer to store debugging tags: the PC of the caller who did the `malloc` and the PC of the last `realloc`. This lets the kernel (or a developer inspecting a crash dump) figure out who allocated a given block. The pool allocator in `libc` (see `lib_core/libc/port/pool.c`) provides the underlying bookkeeping; these tags are an additional layer on top.

```

⟨malloc() if Npadlong 85b⟩≡ (83b)
if(Npadlong){
    v = (ulong*)v+Npadlong;
    setmalloctag(v, getcallerpc(&size));
    setrealloctag(v, 0);
}

⟨function setmalloctag 85c⟩≡ (661c)
void
setmalloctag(kern_vp v, kern_addr pc)
{
    kern_wp u;
    USED(v, pc); //??
    if(Npadlong <= MallocOffset || v == nil)
        return;
    u = v;
    u[-Npadlong+MallocOffset] = pc;
}

⟨function setrealloctag 85d⟩≡ (661c)
void
setrealloctag(kern_vp v, kern_addr pc)
{
    kern_wp u;
    USED(v, pc);
    if(Npadlong <= ReallocOffset || v == nil)

```

```

        return;
    u = v;
    u[-Npadlong+ReallocOffset] = pc;
}

⟨salloc() if Npadlong 86a⟩≡ (84a)
    if(Npadlong){
        v = (ulong*)v+Npadlong;
        setmalloctag(v, getcallerpc(&size));
    }

⟨mallocz() if Npadlong 86b⟩≡ (84c)
    if(Npadlong && v != nil){
        v = (ulong*)v+Npadlong;
        setmalloctag(v, getcallerpc(&size));
        setrealloctag(v, 0);
    }

⟨mlocalign() if Npadlong 86c⟩≡ (84d)
    if(Npadlong && v != nil){
        v = (ulong*)v+Npadlong;
        setmalloctag(v, getcallerpc(&size));
        setrealloctag(v, 0);
    }

⟨realloc() if Npadlong 86d⟩≡ (85a)
    if(Npadlong !=0 && size != 0)
        size += Npadlong*sizeof(ulong);

⟨realloc() readjust nv if Npadlong 86e⟩≡ (85a)
    nv = (ulong*)nv+Npadlong;
    setrealloctag(nv, getcallerpc(&v));
    if(v == nil)
        setmalloctag(nv, getcallerpc(&v));

```

5.5 User malloc()

User-space `malloc()` is entirely in the C library (in `lib_core/libc/port/malloc.c`). It uses the same `Pool` allocator but backed by `brk()` instead of `xalloc()`. The kernel only sees page-granularity requests via `brk()`; the library subdivides those pages for the user.

5.6 Page and palloc

The previous allocators (`xalloc`, `Pool`, `malloc`) manage kernel memory. User memory is managed differently: the unit of allocation is a 4 KiB page, and the kernel tracks every physical page with a `Page` metadata structure. The `Palloc` structure maintains a free list of all available pages. When a user calls `brk()` or triggers a page fault, the kernel takes a page from `palloc` and maps it into the process's address space. Here is the relationship between the allocators:

Physical memory (one `Confmemp` bank):

```

+-----+
| user pages (managed by palloc) |
| each page has a Page struct in palloc.pages |

```

```

+-----+
| kernel malloc pool (Pool/mainmem)      |
+-----+
| Page[] array (xalloc'ed)               |
+-----+
| Proc[] array (xalloc'ed)               |
+-----+ <-- end (linker symbol)
| kernel bss                             |
+-----+ <-- edata
| kernel data:                           |
|   xlists (Xalloc + Hole[128]) \        |
|   pmainmem, pimagmem (Pool)   > layer 0 |
|   conf, devtab, soc, etc.     /        |
+-----+
| kernel text                           |
0x8000 +-----+
| L1/L2 tables, Cpu struct              |
0x0   +-----+

```

The kernel data section (between `edata` and the text) contains the statically allocated globals that bootstrap the memory system: `xlists` with its 128-entry `Hole` array, the `pmainmem` and `pimagmem` pool structures, `conf`, `devtab`, etc. These are layer 0—they exist before any dynamic allocator runs. The `end` symbol marks where dynamic memory begins; `xalloc` starts carving from there.

5.6.1 Page

A `Page` is the metadata for one 4 KiB physical page. The kernel allocates one `Page` for every physical page available for user processes—this is a large array (`palloc.pages`), `xalloc`'ed during initialization. The key fields are `pa` (the physical address of the page, set once and never changed) and `ref` (the reference count, because pages can be shared between processes via copy-on-write):

```

⟨struct Page 87⟩≡ (658)
// Page metadata. We will allocate as many Page as to cover all physical memory
// available for the user. xalloc'ed in Palloc.pages
struct Page
{
    phys_addr pa;      /* Physical address in memory */
    user_addr va;      /* Virtual address for user */

    // Why not Ref? to save space (same reason for using 'char' below)
    // but that means needs to use Lock below to access this non-atomic ref.
    ushort ref;        /* Reference count */ // Pages are shared!

    // bitset<enum<Modref>>
    char modref;        /* Simulated modify/reference bits */

    ⟨Page other fields 165c⟩

    // extra
    Lock;
    ⟨Page extra fields 88c⟩
};

```

This structure must remain compact: there is one `Page` per physical page in the system, so if `Page` were 2 KiB, a third of memory would be wasted on bookkeeping for 4 KiB pages. In practice, `Page` is only a few dozen bytes.

The constants below are software modified/referenced bits (the ARM MMU does not provide hardware dirty bits like x86 does). PG_REF is set the first time a page is accessed (in `fixfault()`¹⁶¹); PG_MOD is set when a page is written.¹

```

⟨enum modref 88a⟩≡ (658)
enum Modref
{
    PG_NOTHING = 0x00, // nothing

    PG_MOD      = 0x01, /* software modified bit */
    PG_REF      = 0x02, /* software referenced bit */
};

```

The swapper uses these bits to decide what to do when reclaiming a page: if PG_MOD is not set, the page is clean and can be discarded (or retrieved from the image cache) without writing it to swap. If PG_REF is set, the page was recently used and should be kept longer.

5.6.2 Palloc

The Palloc is a singleton that manages the free page list. Free pages are kept in a doubly-linked list (through `Page.next`/`Page.prev`), ordered so that the most recently used page is at the head and the least recently used at the tail. This LRU ordering is used by the swapper to decide which pages to evict first.

```

⟨struct Palloc 88b⟩≡ (658)
// Page Allocator (singleton)
struct Palloc
{
    // array<Page>, xalloc'ed in pageinit(), huge, cover physical user space
    Page *pages; /* array of all pages */

    // list<ref<Page>> (next = Page.next), list of free pages
    Page *head; /* most recently used */
    // list<ref<Page>> (prev = Page.prev), list of free pages (backward)
    Page *tail; /* least recently used */

    ulong freecount; /* how many pages on free list now */

    ⟨Palloc capacity fields 89a⟩
    ⟨Palloc other fields 165e⟩

    // extra
    Lock; // LOCK ORDERING: always do lock(&palloc); lock(&page)!!
    ⟨Palloc extra fields 184a⟩
};

⟨Page extra fields 88c⟩≡ (87) 165f>
// list<ref<Page>> Palloc.head, or Proc.mmuused or Proc.mmufree, or in mfreeeseg
Page *next; /* Lru free list */
// list<ref<Page>> Palloc.tail
Page *prev;

⟨global palloc 88d⟩≡ (660a)
// Page allocator
struct Palloc palloc;

```

¹How can the kernel track user-space memory accesses? By initially mapping pages as invalid (or read-only for writes). When the user program touches the page, the MMU generates a page fault, giving the kernel a chance to set PG_REF or PG_MOD before mapping the page with the actual permissions.

`Pallocc` has its own `Palloccmem` array, which looks redundant with `Confmem`. The difference is that `Confmem` describes all physical memory (including the kernel's portion), while `Palloccmem` describes only the user pages—the subset left after the kernel has taken its share via `xalloc`. The `user` field is the total count of user pages across all banks.

```

⟨Pallocc capacity fields 89a⟩≡ (88b)
Palloccmem mem[4]; // = Confmem minus memory allocated for the kernel
// sum of mem.npage (which should be conf.upages)
ulong user;      /* how many user pages */

```

```

⟨struct Palloccmem 89b⟩≡ (658)
// memory banks for user memory, similar to Confmem (and RMap)
struct Palloccmem
{
    phys_addr base;
    ulong npage;
};

```

How much memory does all this bookkeeping consume? By default, `confinit()`^X gives 80% of physical pages to user space and 20% to the kernel. On a 512 MiB Raspberry Pi, that is roughly 100 MiB for the kernel side: the `Proc` array (about 6 MiB for 2000 processes at 3 KiB each), the `Page` array (one `Page` struct per user page), the `mainmem` pool (4 MiB), the `imagmem` pool (16 MiB for graphics), etc. The remaining 400 MiB is managed by `pallocc` as user pages.

5.6.3 `newpage()`

`newpage()` allocates a physical page from `pallocc`. The `clear` flag requests the page be zeroed (needed for bss and stack pages). The `s` parameter is the segment the page belongs to—it may be set to `nil` on return if the segment was invalidated while waiting for memory. The `va` is the virtual address where the page will be mapped:

```

⟨constructor newpage 89c⟩≡ (664)
Page*
newpage(bool clear, Segment **s, user_addr va)
{
    Page *p;
    ⟨newpage() other locals 90a⟩

    lock(&pallocc);

    ⟨newpage() loop waiting freecount & highwater 183b⟩

    //when no color: p = pallocc.head;
    ⟨newpage() find free page p, set color and ct 493d⟩

    pageunchain(p);

    lock(p);
    if(p->ref != 0)
        panic("newpage: p->ref %d != 0", p->ref);

    ⟨newpage() uncache page 167b⟩
    p->ref = 1;
    p->va = va;
    p->modref = PG_NOTHING;
    ⟨newpage() set cachectl using ct 493f⟩

    unlock(p);
    unlock(&pallocc);
    ⟨newpage() if clear 90b⟩
}

```

```

    return p;
}

```

The subtle part is the `if(s)/qunlock` dance: if the caller holds a segment lock (`*s != nil`), it must be released before sleeping (to avoid deadlock), and re-acquired after waking up. But while asleep, another process may have invalidated the segment, so `*s` is set to `nil` to signal the caller to retry.

The `Arch_KMap` is needed to temporarily map the physical page into the kernel's virtual address space so `memset()` can zero it. On ARM with less than 2 GiB of RAM, all physical memory is accessible via the `KZERO` offset, so this is trivial; on machines with more memory, a temporary mapping is required.

```

⟨newpage() other locals 90a⟩≡ (89c) 183a▷
    Arch_KMap *k;

```

```

⟨newpage() if clear 90b⟩≡ (89c)
    if(clear) {
        k = arch_kmap(p);
        memset((void*)VA(k), 0, BY2PG);
        arch_kunmap(k);
    }

```

The following functions manage `pallocc`'s doubly-linked free page list via the `prev/next` pointers in `Page` and the `head/tail` pointers in `Pallocc`:

```

⟨function pageunchain 90c⟩≡ (664)
    // assumes pallocc is held
    static void
    pageunchain(Page *p)
    {
        if(canlock(&pallocc))
            panic("pageunchain (pallocc %p)", &pallocc);

        // remove_queue(p, pallocc);
        if(p->prev)
            p->prev->next = p->next;
        else
            pallocc.head = p->next;
        if(p->next)
            p->next->prev = p->prev;
        else
            pallocc.tail = p->prev;
        p->prev = p->next = nil;

        pallocc.freecount--;
    }

```

```

⟨destructor putpage 90d⟩≡ (664)
    void
    putpage(PageOrSwap *p)
    {
        ⟨putpage if p is a swap address 180d⟩

        lock(&pallocc);
        lock(p);

        if(p->ref == 0)
            panic("putpage");

        if(--p->ref > 0) {
            unlock(p);
            unlock(&pallocc);
        }
    }

```

```

        return;
    }
    // else
    ⟨putpage if p has an image 167a⟩
    else
        pagechainhead(p);

    ⟨putpage wakeup process waiting for freepage if any 184b⟩
    unlock(p);
    unlock(&palloc);
}

⟨function pagechainhead 91a⟩≡ (664)
// assumes palloc is held
void
pagechainhead(Page *p)
{
    if(canlock(&palloc))
        panic("pagechainhead");

    // add_head(p, palloc)
    if(palloc.head) {
        p->next = palloc.head;
        palloc.head->prev = p;
    }
    else {
        palloc.tail = p;
        p->next = nil;
    }
    palloc.head = p;
    p->prev = nil;

    palloc.freecount++;
}

⟨function pagechaintail 91b⟩≡ (664)
// assumes palloc is held
void
pagechaintail(Page *p)
{
    if(canlock(&palloc))
        panic("pagechaintail");

    // add_tail(p, palloc)
    if(palloc.tail) {
        p->prev = palloc.tail;
        palloc.tail->next = p;
    }
    else {
        palloc.head = p;
        p->prev = nil;
    }
    palloc.tail = p;
    p->next = nil;

    palloc.freecount++;
}

```

5.7 Segment and Pagetable

A **Segment** represents a contiguous region of a process's virtual address space (text, data, bss, or stack). Each segment contains a two-level page directory that maps virtual addresses within the segment to physical **Pages**. These are the kernel's segment tables—separate from the ARM's hardware L1/L2 page tables, although the two must be kept in sync.

Why maintain segment tables when the hardware already has page tables? Because the hardware tables are architecture-specific (different format on ARM vs. x86), while **Segment/Pagetable** are portable. They also store information the hardware does not: reference counts (for shared pages and copy-on-write), swap addresses (for paged-out pages), and segment types (text vs. data vs. stack). The cost is extra memory, but the segment tables are sparse (only populated on demand via page faults), so the overhead is proportional to the pages actually used, not to the full address space.

```
<enum segtype 92a>≡ (658)
/* Segment types */
enum Segtype
{
    SG_TEXT    = 00,
    SG_DATA    = 01,
    SG_BSS     = 02,
    SG_STACK   = 03,
    <Segtype other cases 494h>
    SG_TYPE    = 07, /* Mask type of segment */

    <Segtype other flags 99a>
};
```

```
<global sname 92b>≡ (643)
// hash<enum<Segtype, string>>
static char *sname[]={
    [SG_TEXT]    = "Text",
    [SG_DATA]    = "Data",
    [SG_BSS]     = "Bss",
    [SG_STACK]   = "Stack",

    [SG_SHARED]  = "Shared",
    [SG_PHYSICAL] = "Phys"
};
```

5.7.1 Segment

Each segment tracks its virtual address range (base–top) and owns a **pagedir** array of **Pagetable**^{93b} pointers:

```
<struct Segment 92c>≡ (658)
// malloc'ed by newseg()
struct Segment
{
    // enum<Segtype>
    ushort type; /* segment type */

    user_addr base; /* virtual base */
    user_addr top; /* virtual top */
    ulong size; /* size in pages */ // top - base / BY2PG

    // array<option<ref_own<Pagetable>>>, malloc'ed (or smallpagedir alias)
    Pagetable **pagedir; // array of PAGEDIRSIZE max
    int pagedirsize; // nelem(pagedir)
```

```

    <Segment other fields 96d>

    // extra
    Ref; // LOCK ORDERING: always do lock(img); lock(s) ??
    QLock lk; // FOR WHAT???
};

```

Here is the structure:

```

Proc
|
+-- seg[NSEG]: array of Segment pointers
    |
    +-- seg[SG_TEXT] --> Segment (base=UTZERO, top=...)
    +-- seg[SG_DATA] --> Segment (base=..., top=...)
    +-- seg[SG_BSS] --> Segment (base=..., top=...)
    +-- seg[SG_STACK] --> Segment (base=USTKTOP-USTKSIZE, top=USTKTOP)
        |
        +-- map[]: array of Pagetable pointers
            |
            +-- Pagetable
                |
                +-- pages[]: array of Page pointers
                    |
                    +-- Page (pa=..., va=...)

```

PAGEDIRSIZE is the maximum number of **Pagetable** pointers in a segment's page directory. Each **Pagetable** covers 1 MiB (256 pages $\times$ 4 KiB), so 1984 entries can map $1984 \times 1 \text{ MiB} \approx 2 \text{ GiB}$ —just under the user address space limit at KZERO. The value 1984 (not 2048) leaves room for the I/O region mapped at VIRTIO just below KZERO.

```

<constant PAGEDIRSIZE(arm) 93a>≡ (709b)
#define PAGEDIRSIZE 1984

```

5.7.2 Pagetable

A **Pagetable** maps 1 MiB of virtual address space (256 pages of 4 KiB each). It is the second level of the software page table—the **Segment.map** array is the first level. The **first/last** pointers optimize iteration: for a stack segment that uses only a few pages at the top of a large range, they avoid scanning hundreds of nil entries:

```

<struct Pagetable 93b>≡ (658)
// ptalloc'ed (malloc'ed)
struct Pagetable
{
    //array<option<ref<PageOrSwap>> will map 1M of memory
    PageOrSwap *pagetab[PAGETABSIZE];

    //to avoid iterate over all entries in pagetab
    // ref<ref<Page>> in Pagetable.pages
    Page **first; /* First used entry */
    // ref<ref<Page>> in Pagetable.pages
    Page **last; /* Last used entry */
};

```

```

<constant PAGETABMAPMEM(arm) 93c>≡ (709b)
#define PAGETABMAPMEM (1024*1024) // 1MB

```

<constant PAGETABSIZE (arm) 94a>≡ (709b)

```
#define PAGETABSIZE 256 // (PAGETABMAPMEM/BY2PG)
```

Here is how the constants relate:

```
Segment.map[0..PAGEDIRSIZE-1]      (up to 1984 entries)
|
+--[0]--> Pagetable.pagetab[0..255] (PAGETABSIZE)
|
|           |
|           +--[0]--> Page (4 KiB = BY2PG)
|           +--[1]--> Page
|           ...
|           +--[255]--> Page
|           covers 256 * 4 KiB = 1 MiB (PAGETABMAPMEM)
|
+--[1]--> Pagetable (another 1 MiB)
...
+--[1983]--> Pagetable
               covers 1984 * 1 MiB ~= 2 GiB (user address space)
```

<constructor ptalloc 94b>≡ (664)

```
    Pagetable*
    ptalloc(void)
    {
        Pagetable *new;

        new = smalloc(sizeof(Pagetable));
        new->first = &new->pagetab[PAGETABSIZE];
        new->last = new->pagetab;
        return new;
    }
```

Note the use of `smalloc()`—a sleeping variant of `malloc()` that blocks if memory is not available instead of returning `nil`.

<destructor freept 94c>≡ (664)

```
    void
    freept(Segment *s, Pagetable *p)
    {
        PageOrSwap **pte;
        <freept() other locals 495d>

        switch(s->type&SG_TYPE) {
        <freept() SG_PHYSICAL case 495e>
        default:
            for(pte = p->first; pte <= p->last; pte++)
                if(*pte) {
                    putpage(*pte);
                    *pte = nil;
                }
        }
        free(p);
    }
```

<function ptcpy 94d>≡ (664)

```
    Pagetable*
    ptcpy(Pagetable *old)
    {
```

```

Pagetable *new;
PageOrSwap **src, **dst;

new = ptalloc();
dst = &new->pagetab[old->first - old->pagetab];
new->first = dst;
for(src = old->first; src <= old->last; src++, dst++)
    if(*src) {
        <ptcpy() if src is a swap page 180g>
        else {
            lock(*src);
            (*src)->ref++;
            unlock(*src);
        }
        new->last = dst;
        *dst = *src;
    }
return new;
}

```

5.7.3 newseg()

We have seen `newpage()`^{89c} for allocating physical pages, and the `Segment` and `Pagetable`^{93b} data structures. Now we can see how a new segment is created:

`newseg()`⁹⁵ creates a new `Segment` of the given type (text, data, bss, stack, or shared), starting at virtual address `base` and covering `size` pages. It is called by `sysexec()`X (to set up a new program's segments) and by `sysbrk()`X (to extend the bss). The page directory is allocated but the actual `Pagetable`s and `Pages` are not—they are populated lazily on page faults:

```

<constructor newseg 95>≡ (665)
Segment *
newseg(int type, user_addr base, ulong size)
{
    Segment *s;
    int pagedirsize;

    <newseg() sanity check size 96a>
    s = smalloc(sizeof(Segment));

    s->ref = 1; // Segment.Ref.ref
    s->type = type;
    s->base = base;
    s->top = base+(size*BY2PG);
    s->size = size;

    <newseg() sema initialization 417b>

    pagedirsize = ROUND(size, PAGETABSIZE)/PAGETABSIZE;

    <newseg() if pagedirsize small 97a>
    else{
        pagedirsize *= 2; // room for growing
        if(pagedirsize > PAGEDIRSIZE)
            pagedirsize = PAGEDIRSIZE; // pad's first bugfix :)
        s->pagedir = smalloc(pagedirsize * sizeof(Pagetable*));
        s->pagedirsize = pagedirsize;
    }
    return s;
}

```



```
}
```

Again `smalloc()` is used, so if the kernel is low on memory the caller will sleep until some is freed.

```
<newseg() sanity check size 96a>≡ (95)
if(size > (PAGEDIRSIZE*PAGETABSIZE))
    error(Enovmem);
```

```
<destructor putseg 96b>≡ (665)
void
putseg(Segment *s)
{
    Pagetable **pde, **emap;
    <putseg() other locals 175a>

    <putseg() sanity check s 96c>
    <putseg() if s has an image 175b>
    else
        lock(s);

    s->ref--;
    if(s->ref != 0) {
        unlock(s);
        return;
    }
    // else
    unlock(s);

    qlock(&s->lk);
    <putseg() if s had an image 176a>

    emap = &s->pagedir[s->pagedirsize];
    for(pde = s->pagedir; pde < emap; pde++)
        if(*pde)
            freept(s, *pde);

    qunlock(&s->lk);

    <putseg() if smallpagedir 97b>
    else
        free(s->pagedir);
    <putseg() free profile 446b>
    free(s);
}
```

```
<putseg() sanity check s 96c>≡ (96b)
if(s == nil)
    return; // TODO: panic("putseg") instead?
```

5.7.4 smallpagedir

As a small optimization, each `Segment` embeds a fixed-size `smallpagedir` array. If the segment is small enough (at most `SMALLPAGEDIRSIZE` entries), `pagedir` points to this embedded array, avoiding an extra `smalloc()` call. Most segments are small enough to benefit from this.

```
<Segment other fields 96d>≡ (92c) 169a▷
// array<option<ref_own<Pagetable>>
Pagetable *smallpagedir[SMALLPAGEDIRSIZE];
```

```
<constant SMALLPAGEDIRSIZE(arm) 96e>≡ (709b)
#define SMALLPAGEDIRSIZE 16
```

```

⟨newseg() if pagedirsize small 97a)≡ (95)
    if(pagedirsize <= nelem(s->smallpagedir)){
        s->pagedir = s->smallpagedir;
        s->pagedirsize = nelem(s->smallpagedir);
    }

```

```

⟨putseg() if smallpagedir 97b)≡ (96b)
    if(s->pagedir == s->smallpagedir) {
    }

```

5.7.5 Adding a page to a segment: segpage()

`segpage()` attaches a physical Page to a segment's page table at the virtual address stored in `p->va`. It is called during process creation (`userinit()`^X), page fault handling (`fixfault()`¹⁶¹), and segment growth (`ibrk()`^{102b} for the bss).

```

⟨function segpage 97c)≡ (665)
    void
    segpage(Segment *s, Page *p)
    {
        Pagetable **pt;
        ulong off;
        Page **pg;

        ⟨segpage() sanity check page in range of segment 97d)

        off = p->va - s->base;
        pt = &s->pagedir[off/PAGETABMAPMEM]; // PDX
        if(*pt == nil)
            *pt = ptalloc();

        pg = &(*pt)->pagetab[(off&(PAGETABMAPMEM-1))/BY2PG]; // PTX
        *pg = p;

        if(pg < (*pt)->first)
            (*pt)->first = pg;
        if(pg > (*pt)->last)
            (*pt)->last = pg;
    }

```

```

⟨segpage() sanity check page in range of segment 97d)≡ (97c)
    if(p->va < s->base || p->va >= s->top)
        panic("segpage");

```

5.7.6 Process segments

We saw `SG_TEXT`, `SG_DATA`, etc. above—those are segment types stored in `Segment.type`. The `Procseg` enum below is different: it defines slot indices into `Proc.seg[]`, the per-process array of segment pointers. `TSEG` is the slot for the text segment, `DSEG` for data, etc. A process always has the four core slots, plus extra slots for `segattach()`'d shared memory:

`Procseg`

```

⟨enum procseg 97e)≡ (668c)
    /*
    * process memory segments - NSEG always last !
    */

```

```
//coupling: must start with SSEG, see sysexec loop
enum Procseg
{
    SSEG, TSEG, DSEG, BSEG, // Stack, Text, Data, Bss

    ESEG, // E = Extra (used for temporary stack segment),
    _SEG0, _SEG1, _SEG2, _SEG3, _SEG4, // free slots for for segattach

    NSEG // to count, see Proc.seg array
};
```

And here is the field that ties it all together—the `seg[]` array in `Proc`, indexed by `Procseg`:

```
<Proc memory fields 98a>≡ (49f) 158f>
// hash<enum<procseg>, option<ref_own<Segment>>>, elt smalloc'ed? ref_counted?
Segment *seg[NSEG];
QLock seglock; /* locked whenever seg[] changes */
```

seg()

Given a virtual address, `seg()` finds which segment it belongs to by scanning `up->seg[]`. This is used throughout the kernel whenever a user-space address must be validated or its segment located (e.g., in page fault handling, read/write syscalls, debugging):

```
<function seg 98b>≡ (663b)
Segment*
seg(Proc *p, user_addr addr, bool dolock)
{
    Segment **s, **et, *sg;

    et = &p->seg[NSEG];
    for(s = p->seg; s < et; s++) {
        sg = *s;
        if(sg == nil)
            continue;
        if(addr >= sg->base && addr < sg->top) {
            if(!dolock)
                return sg;
            // else

            qlock(&sg->lk);
            // can have a race, need to check again
            if(addr >= sg->base && addr < sg->top)
                return sg;
            qunlock(&sg->lk);
        }
    }
    return nil;
}
```

Addresses covering multiple segments, validaddr()

```
<function okaddr 98c>≡ (663b)
/*
 * Called only in a system call
 */
bool
okaddr(user_addr addr, ulong len, bool write)
{

```

```

Segment *s;

if((long)len >= 0) {
    for(;;) {
        s = seg(up, addr, false);
        if(s == nil || (write && (s->type&SG_RDONLY)))
            break;

        if(addr+len > s->top) {
            len -= s->top - addr;
            addr = s->top;
        }else{
            return true;
        }
    }
}
pprint("suicide: invalid address %#lux/%lud in sys call pc=%#lux\n", addr, len, arch_userpc());
return false;
}

```

⟨Segtype other flags 99a⟩≡ (92a) 410b▷

```

SG_RDONLY = 0040, /* Segment is read only */

```

⟨function validaddr 99b⟩≡ (663b)

```

void
validaddr(user_addr addr, ulong len, bool write)
{
    if(!okaddr(addr, len, write)){
        postnote(up, 1, "sys: bad address in syscall", NDebug);
        error(Ebadarg);
    }
}

```

`vmemchr()` is a validated `memchr()`—it searches for character `c` in the user buffer starting at `s` for up to `n` bytes, checking that each page is valid as it crosses page boundaries. This is needed because a user buffer may span multiple pages, some of which might not be mapped yet:

⟨function vmemchr 99c⟩≡ (663b)

```

/*
 * &s[0] is known to be a valid address.
 */
void*
vmemchr(user_vp s, int c, int n)
{
    int m;
    user_addr a;
    user_vp t;

    a = (user_addr)s;
    while(PGROUND(a) != PGROUND(a+n-1)){
        /* spans pages; handle this page */
        m = BY2PG - (a & (BY2PG-1));
        t = memchr((void*)a, c, m);
        if(t)
            return t;
        a += m;
        n -= m;
        if(a < KZERO)
            validaddr(a, 1, false);
    }
}

```

```

    /* fits in one page */
    return memchr((void*)a, c, n);
}

⟨portdat.h macros 100a⟩+≡ ( ? 0—1 ) <34e
// BY2PG is defined in mem.h, which should always be included before "dat.h"!
#define PGROUND(s) ROUNDUP(s, BY2PG)

```

dupseg()

dupseg() duplicates a segment for RFORK. Depending on the segment type, it either creates a full copy or sets up copy-on-write sharing (for data and bss segments):

```

⟨function dupseg 100b⟩≡ (665)
Segment*
dupseg(Segment **seg, int segno, bool share)
{
    Segment *s;
    Segment *n; // when allocate a new segment
    ⟨dupseg() other locals 101a⟩

    SET(n);

    s = seg[segno];

    qlock(&s->lk);
    if(waserror()){
        qunlock(&s->lk);
        nexterror();
    }
    switch(s->type&SG_TYPE) {
    ⟨dupseg() switch segment type cases 100c⟩
    }
    // not same seg, we have allocated a new seg in n above
    ⟨dupseg() when not goto same seg, when allocated a new segment 101b⟩

    poperror();
    qunlock(&s->lk);
    return n;

same seg:
    ⟨dupseg() in same seg, when share a segment 100d⟩

    poperror();
    qunlock(&s->lk);
    return s;
}

⟨dupseg() switch segment type cases 100c⟩≡ (100b) 100e▷
case SG_TEXT:      /* New segment shares pt set */
    goto same seg;

⟨dupseg() in same seg, when share a segment 100d⟩≡ (100b)
    incref(s);

⟨dupseg() switch segment type cases 100e⟩+≡ (100b) <100c 101c▷
case SG_STACK:
    n = newseg(s->type, s->base, s->size);
    break;

```

```

⟨dupseg() other locals 101a⟩≡ (100b)
    int i;
    int size; // pagedir
    Pagetable *pt;

⟨dupseg() when not goto sameseg, when allocated a new segment 101b⟩≡ (100b)
    size = s->pagedirsize;
    for(i = 0; i < size; i++)
        if(pt = s->pagedir[i])
            n->pagedir[i] = ptcpy(pt); // will actually share the pages

⟨dupseg() copy other fields 494d⟩
⟨dupseg() if original segment was shared, flush it 414b⟩

⟨dupseg() switch segment type cases 101c⟩+≡ (100b) <100e 101d>
    case SG_DATA: /* Copy on write plus demand load info */
        ⟨dupseg() when SG_DATA, if Text segment 171b⟩
        if(share)
            goto sameseg; // threads! clone()
        // else
        n = newseg(s->type, s->base, s->size);
        ⟨dupseg() SG_DATA case, attach image to new segment n 171a⟩
        break;

⟨dupseg() switch segment type cases 101d⟩+≡ (100b) <101c 494j>
    case SG_BSS: /* Just copy on write */
        if(share)
            goto sameseg; // threads! clone()
        // else
        n = newseg(s->type, s->base, s->size);
        break;

```

5.7.7 /proc/x/segment

Reading `/proc/\emphpid/segment` shows the memory layout of a process—the base, top, and type of each segment. This is implemented in `devprocX` (see Chapter 6):

```

⟨procread() locals 101e⟩≡ (540c) 307b▷
    Segment *sg;

⟨procread() cases 101f⟩≡ (540c) 308c▷
    case Qsegment:
        j = 0;
        for(i = 0; i < NSEG; i++) {
            sg = p->seg[i];
            if(sg == nil)
                continue;
            j += snprintf(statbuf+j, sizeof statbuf - j,
                "%-6s %c%c %.8lux %.8lux %4ld\n",
                sname[sg->type&SG_TYPE],
                sg->type&SG_RDONLY ? 'R' : ' ',
                sg->profile ? 'P' : ' ',
                sg->base, sg->top, sg->ref);
        }
        if(offset >= j)
            return 0;
        if(offset+n > j)
            n = j-offset;
        if(n == 0 && offset == 0)

```

```

    exhausted("segments");
    memmove(a, &statbuf[offset], n);
    return n;

```

5.8 sysbrk()

We now have all the pieces: `xalloc` for bulk allocation, `pallo`c for page management, `Segment` and `Pagetable` for mapping virtual addresses to pages. We can finally see how the `brk()` syscall ties them together. This is the only memory-related syscall. When a user program calls `brk(addr)`, the kernel extends (or shrinks) the BSS segment to the requested address by allocating (or freeing) pages from `pallo`c and updating the segment's page directory. The user-space `malloc()` library calls `brk()` when it needs more memory from the kernel.

<syscall brk 102a>≡ (679a)

```

// int brk(void*);
long
sysbrk(ulong* arg)
{
    return ibrk(arg[0], BSEG); // BSS, the heap size is changed
}

```

<function ibrk 102b>≡ (665)

```

long
ibrk(user_addr addr, int seg)
{
    Segment *s;
    user_addr newtop;
    ulong newsize;
    <ibrk() other locals 103b>

    s = up->seg[seg];
    <ibrk() sanity check s 102c>
    <ibrk() if addr nil 102d>

    qlock(&s->lk);

    <ibrk() if addr below base 103a>

    newtop = PGROUND(addr);
    newsize = (newtop - s->base)/BY2PG; // in nb pages

    <ibrk() if newtop lower than oldtop, shrink the segment 104a>
    // else
    <ibrk() if newtop more than oldtop, extend the segment 103c>

}

```

<ibrk() sanity check s 102c>≡ (102b)

```

if(s == nil)
    error(Ebadarg);

```

<ibrk() if addr nil 102d>≡ (102b)

```

if(addr == nilptr)
    return s->base;

```

```

<ibrk() if addr below base 103a>≡ (102b)
/* We may start with the bss overlapping the data */
if(addr < s->base) {
    if(seg != BSEG || up->seg[DSEG] == nil || addr < up->seg[DSEG]->base) {
        qunlock(&s->lk);
        error(Enovmem);
    }
    addr = s->base;
}

```

5.8.1 Extending a segment

The rest of `ibrk()` handles two cases: extending a segment (growing `top` upward, allocating new `Pagetable` entries in `pagedir`) and shrinking it (freeing pages and page tables):

```

<ibrk() other locals 103b>≡ (102b) 103d▷
int mapsize;
Pagetable **map;

```

```

<ibrk() if newtop more than oldtop, extend the segment 103c>≡ (102b)
<ibrk() sanity check newtop does not overlap other segments 103e>
<ibrk() sanity check newsize 103f>

```

```

// similar to code in newseg()
mapsize = ROUND(newsize, PAGETABSIZE)/PAGETABSIZE;
// realloc
if(mapsize > s->pagedirsize){
    map = smalloc(mapsize*sizeof(Pagetable*));
    memmove(map, s->pagedir, s->pagedirsize*sizeof(Pagetable*));
    if(s->pagedir != s->smallpagedir)
        free(s->pagedir);
    s->pagedir = map;
    s->pagedirsize = mapsize;
}

```

```

s->top = newtop;
s->size = newsize;
qunlock(&s->lk);
return OK_0;

```

```

<ibrk() other locals 103d>+≡ (102b) ◁103b
Segment *ns;
int i;

```

```

<ibrk() sanity check newtop does not overlap other segments 103e>≡ (103c)
for(i = 0; i < NSEG; i++) {
    ns = up->seg[i];
    if(ns == nil || ns == s)
        continue;
    if(newtop >= ns->base && newtop < ns->top) {
        qunlock(&s->lk);
        error(Esoverlap);
    }
}

```

```

<ibrk() sanity check newsize 103f>≡ (103c)
if(newsize > (PAGEDIRSIZE*PAGETABSIZE)) {
    qunlock(&s->lk);
    error(Enovmem);
}

```


5.8.2 Shrinking a segment

(ibrk() if newtop lower than oldtop, shrink the segment 104a)≡ (102b)

```
if(newtop < s->top) {
    /*
     * do not shrink a segment shared with other procs, as the
     * to-be-freed address space may have been passed to the kernel
     * already by another proc and is past the validaddr stage.
     */
    if(s->ref > 1){
        qunlock(&s->lk);
        error(Einuse);
    }
    mfreeseg(s, newtop, (s->top-newtop)/BY2PG);
    s->top = newtop;
    s->size = newsize;
    qunlock(&s->lk);
    arch_flushmmu();
    return OK_0;
}
```

(function mfreeseg 104b)≡ (665)

```
/*
 * called with s->lk locked
 */
void
mfreeseg(Segment *s, ulong start, int pages)
{
    int i, j, size;
    ulong soff;
    PageOrSwap *pg;
    Page *list;

    soff = start-s->base;
    j = (soff&(PAGETABMAPMEM-1))/BY2PG; // PTX

    size = s->pagedirsize;
    list = nil;
    for(i = soff/PAGETABMAPMEM; i < size; i++) { // PDX
        if(pages <= 0)
            break;
        if(s->pagedir[i] == nil) { // space was never accessed, good, easier
            pages -= PAGETABSIZE-j;
            j = 0;
            continue;
        }
        while(j < PAGETABSIZE) {
            pg = s->pagedir[i]->pagetab[j];
            /*
             * We want to zero s->pagedir[i]->page[j] and putpage(pg),
             * but we have to make sure other processors flush the
             * entry from their TLBs before the page is freed.
             * We construct a list of the pages to be freed, zero
             * the entries, then (below) call procflushseg, and call
             * putpage on the whole list.
             */
            *
            * Swapped-out pages don't appear in TLBs, so it's okay
            * to putswap those pages before procflushseg.
            */
            if(pg){
```

```

        <mfreeseg if pg is a swap address 180e>
        else{
            pg->next = list;
            list = pg;
        }
        s->pagedir[i]->pagetab[j] = nil;
    }
    if(--pages == 0)
        goto out;
    j++;
}
j = 0;
}
out:
<mfreeseg() if segment was shared, flush it 414c>
/* free the pages */
for(pg = list; pg != nil; pg = list){
    list = list->next;
    putpage(pg);
}
}

```

Chapter 6

Processes

In this chapter, I will present the process lifecycle: how processes are allocated, created (**rfork**), loaded with a new program (**exec**), how a parent waits for children (**await**), and how processes terminate (**exits**). These four syscalls are the foundation of Plan 9’s process model.

```
<systab process syscalls 106a>≡ (41f)
[RFORK]    sysrfork,
[EXEC]     sysexec,
[AWAIT]    sysawait,
[EXITS]    sysexits,
```

6.1 Overview

Plan 9 inherits the UNIX fork/exec model: **rfork** creates a child process (a copy of the parent), and **exec** replaces the child’s memory with a new program. This two-step approach is powerful—between the fork and the exec, the child can set up pipes, redirect files, or modify its namespace before running the new program. Plan 9 generalizes UNIX **fork** into **rfork** (“resource fork”): instead of always copying everything, **rfork** takes flags that independently control which resources to copy, share, or create fresh—memory, file descriptors, namespace, environment, notes, and rendezvous group. A traditional **fork** is **rfork** with everything copied; a thread (shared memory) is **rfork** with **RFMEM** set. Sandboxing is trivial with **RFNOMNT**: the child gets a fresh namespace where **mount** and **bind** are forbidden, so it cannot reach any file server the parent does not explicitly pass. This achieves what Linux needs **chroot** jails or Docker containers for—but as a simple flag to **rfork**, with no extra kernel infrastructure.

6.2 Process allocation

The **Proc** structure is large and long-lived, so processes are pre-allocated in a pool during boot rather than malloc’d on each fork. The pool is an array of **conf.nproc** entries (up to 2000), allocated by **xalloc** during initialization. A free list threads through unused entries, and a hash table provides fast pid-to-process lookup.

6.2.1 Process ID: **Proc.pid**

Every process needs a unique identifier. The **pid** field holds it, and **pidalloc** (a **Counter**, i.e., a **Ref**) ensures uniqueness: each **newproc()** call atomically increments it to get a fresh value.

```
<Proc state fields 106b>+≡ (49f) <51a 109c>
    ulong pid;
```

```
<global pidalloc 106c>≡ (675c)
    static Counter pidalloc;
```

6.2.2 Procalloc

The process allocator is a simple pool: an `xalloc`'ed array of `Proc` structures (`arena`) and a free list (`free`) threaded through `Proc.qnext`. When a process exits, it is returned to the free list for reuse:

```
<struct Procalloc 107a>≡ (668c)
// Process allocator
struct Procalloc
{
    // array<Proc>, xalloc'ed in procinit() (conf.nproc)
    Proc* arena;
    // list<ref<Proc>> (next = Proc.qnext)
    Proc* free;

    <Procalloc other fields 107c>

    // extra
    Lock;
};

<global procalloc 107b>≡ (675c)
static struct Procalloc procalloc;
```

6.2.3 PID hashing

The kernel needs to look up a `Proc` by pid (e.g., when accessing `/proc/x/...`). A linear scan of the arena would be slow, so `Procalloc` includes a hash table of 128 buckets chained through `Proc.pidhash`:

```
<Procalloc other fields 107c>≡ (107a)
// hash<Proc.pid, ref<Proc>> (next = Proc.pidhash)>
Proc* ht[128];

<Proc extra fields 107d>+≡ (49f) <63c 192d>
// hash<Proc.pid, ref<Proc>> Procalloc.ht
Proc *pidhash; /* next proc in pid hash */

<function pidhash 107e>≡ (675c)
static void
pidhash(Proc *p)
{
    int h;

    h = p->pid % nelem(procalloc.ht);
    lock(&procalloc);
    // add_hash(procalloc.ht, p->pid, p)
    p->pidhash = procalloc.ht[h];
    procalloc.ht[h] = p;
    unlock(&procalloc);
}

<function pidunhash 107f>≡ (675c)
static void
pidunhash(Proc *p)
{
    int h;
    Proc **l;

    h = p->pid % nelem(procalloc.ht);
    lock(&procalloc);
    // remove_hash(procalloc.ht, p->pid, p)
```

```

    for(l = &procalloc.ht[h]; *l != nil; l = &(*l)->pidhash)
        if(*l == p){
            *l = p->pidhash;
            break;
        }
    unlock(&procalloc);
}

```

`procindex()` translates a pid into an index in the `procalloc.arena` array. It is used by `devproc` to implement `/proc/\emphpid/...`:

```

⟨function procindex 108a⟩≡ (675c)
int
procindex(ulong pid)
{
    Proc *p;
    int h;
    int s;

    s = -1;
    // hash_lookup(pid, procalloc.ht)
    h = pid % nelem(procalloc.ht);
    lock(&procalloc);
    for(p = procalloc.ht[h]; p != nil; p = p->pidhash)
        if(p->pid == pid){
            s = p - procalloc.arena;
            break;
        }
    unlock(&procalloc);
    return s;
}

```

6.2.4 newproc()

`newproc()` pops a `Proc` from the free list, assigns it a fresh pid (by incrementing the global `pidalloc` counter), and initializes its fields to safe defaults. The new process starts in state **Scheduling**—not **Ready**—because its segments are not yet set up; the caller (`sysrfork()`X or `kproc()`^{135b}) must finish the initialization before calling `ready()`^{202c}.

```

⟨Procstate cases 108b⟩+≡ (51b) <66c 131a>
    Scheduling,

```

```

⟨constructor newproc 108c⟩≡ (675c)
Proc*
newproc(void)
{
    Proc *p;
    ⟨newproc() other locals 109f⟩

    lock(&procalloc);
    // ~ p = procalloc.free
    ⟨newproc() set p to procalloc.free and wait if nil 109g⟩
    procalloc.free = p->qnext;
    unlock(&procalloc);
    p->qnext = nil;

    p->state = Scheduling;
    p->pid = incref(&pidalloc);
    ⟨newproc() sanity check pid 109a⟩
}

```

```

pidhash(p);

⟨newproc() set fields of p 109b⟩

return p;
}

```

```

⟨newproc() sanity check pid 109a⟩≡ (108c)
if(p->pid==0)
    panic("pidalloc");

```

```

⟨newproc() set fields of p 109b⟩≡ (108c) 109d▷
memset(p->seg, nilptr, sizeof p->seg);

```

Two string fields help with debugging and the `ps` command: `psstate` is a human-readable description of what the process is doing (e.g., “New”, “PageOut”, or the name of the syscall it is blocked in), and `text` is the name of the executable. Both appear in `/proc/emphpid/status`.

```

⟨Proc state fields 109c⟩+≡ (49f) <106b 111a▷
// some debugging information, e.g. "New", "PageOut", or name of syscall
char *psstate; /* used by /proc/#/status */
// e.g. "*init*", or name of executable
char *text;

```

```

⟨newproc() set fields of p 109d⟩+≡ (108c) <109b 109e▷
p->psstate = "New";
kstrdup(&p->text, "*notext");

```

```

⟨newproc() set fields of p 109e⟩+≡ (108c) <109d 111b▷
p->fgrp = nil;
p->pgrp = nil;
p->egrp = nil;
p->rgrp = nil;

```

6.2.5 No more processes

If the free list is empty when `newproc()`^{108c} is called, the kernel sleeps and retries, hoping another process will exit soon. If the situation persists, it panics and reboots.

```

⟨newproc() other locals 109f⟩≡ (108c)
char msg[64];

```

```

⟨newproc() set p to procalloc.free and wait if nil 109g⟩≡ (108c)
while((p = procalloc.free) == nil) {
    unlock(&procalloc);

```

```

    snprintf(msg, sizeof msg, "no procs; %s forking", up?up->text:"kernel");
    /*
     * the situation is unlikely to heal itself.
     * dump the proc table and restart by default.
     * *noprocspersist in plan9.ini will yield the old
     * behaviour of trying forever.
     */
    if(getconf("*noprocspersist") == nil)
        noproccpanic(msg);
    resrcwait(msg);
    lock(&procalloc);
}

```

`resrcwait()` (defined later in this chapter) prints a warning and sleeps for 300 ms before retrying—it is the kernel’s standard “resource temporarily unavailable” back-off.

```
⟨function noprocpanic 110a⟩≡ (675c)
void
noprocpanic(char *msg)
{
    /*
     * setting exiting will make hzclock() on each processor call exit(0).
     * clearing our bit in cpus avoids calling exit(0) from hzclock()
     * on this processor.
     */
    lock(&active);
    active.cpus &= ~(1<<cpu->cpuno);
    active.exiting = true;
    unlock(&active);

    procdump();
    arch_delay(1000);
    panic(msg);
}
```

```
⟨function resrcwait 110b⟩≡ (673b)
void
resrcwait(char *reason)
{
    ulong now;
    char *p;
    static ulong lastwhine;

    if(up == nil)
        panic("resrcwait");

    p = up->psstate;
    if(reason) {
        up->psstate = reason;
        now = seconds();
        /* don't tie up the console with complaints */
        if(now - lastwhine > Whinsecs) {
            lastwhine = now;
            print("%s\n", reason);
        }
    }
    tsleep(&up->sleepr, returnfalse, 0, 300);
    up->psstate = p;
}
```

```
⟨constant Whinsecs 110c⟩≡ (673b)
Whinsecs = 10, /* frequency of out-of-resources printing */
```

6.3 The user

In UNIX, the process owner is a numeric uid. In Plan 9, it is a string (`Proc.user`): typically “eve” for the system owner. There is no `/etc/passwd`—authentication is handled per-connection by the file server protocol (9P). In Plan 9, file permissions also use strings: `struct Dir` has `uid` and `gid` fields that are character strings, not

numeric ids. The file server itself decides whether to grant access by comparing the connection’s authenticated user name against these strings—there is no kernel-level uid/gid translation.

```
⟨Proc state fields 111a⟩+≡ (49f) <109c 114c>
// e.g.. "eve" (no uid/gid in plan9, because of its distributed nature?)
char *user;
```

```
⟨newproc() set fields of p 111b⟩+≡ (108c) <109e 123a>
kstrdup(&p->user, "nouser");
```

hostownerwrite() is called during boot (from boot.rc) to set the “host owner”—the user who owns the terminal. This is the Plan 9 equivalent of setting the system’s primary user account.

```
⟨function hostownerwrite 111c⟩≡ (681)
/*
 * called by devcons() for host owner/domain
 *
 * writing hostowner also sets user
 */
long
hostownerwrite(char *a, int n)
{
    char buf[128];

    if(!iseve())
        error(Eperm);
    if(n <= 0 || n >= sizeof buf)
        error(Ebadarg);
    memmove(buf, a, n);
    buf[n] = 0;

    renameuser(eve, buf);
    kstrdup(&eve, buf);
    kstrdup(&up->user, buf);
    up->basepri = PriNormal;
    return n;
}
```

```
⟨function renameuser 111d⟩≡ (675c)
/*
 * change ownership to 'new' of all processes owned by 'old'. Used when
 * eve changes.
 */
void
renameuser(char *old, char *new)
{
    Proc *p, *ep;

    ep = procalloc.arena + conf.nproc;
    for(p = procalloc.arena; p < ep; p++)
        if(p->user!=nil && strcmp(old, p->user)==0)
            kstrdup(&p->user, new);
}
```

6.4 sysrfork()

sysrfork()X is the most complex syscall in the kernel. It takes a bitmask of flags that independently control which resources to fork, share, or create fresh. If RFPROC is set, a new child process is created; otherwise, the

flags modify the current process's own resources (e.g., detach from the parent's namespace).

```

⟨enum rfork 112a⟩≡ (679a)
//coupling: with libc.h
enum Rfork
{
    RFPROC      = (1<<4), // fork a new process!! (if unset then set props for up)
    RFMEM       = (1<<5), // share data and bss (kinda thread, a la Linux clone)

    RFNOWAIT    = (1<<6), // child will not leave a waitmsg

    RFNAMEG     = (1<<0), // copy namespace (if unset then share)
    RFENVG      = (1<<1), // copy environment variables (if unset then share)
    RFFDG       = (1<<2), // copy file descriptor table (if unset then share)

    RFCNAMEG    = (1<<10), // clean new namespace
    RFCENVG     = (1<<11), // clean new empty environment variables
    RFCFDG      = (1<<12), // clean new file descriptor table

    RFNOTEG     = (1<<3), // start new group for notes
    RFREND      = (1<<13), // start a new group for rendezvous

    RFNOMNT     = (1<<14), // # paths forbidden, sandboxing
};

```

```

⟨syscall rfork 112b⟩≡ (679a)
// int rfork(int flags);
long
sysrfork(ulong* arg)
{
    ulong flag;
    ⟨sysrfork() other locals 112c⟩

    flag = arg[0];
    /* Check flags before we commit */
    ⟨sysrfork() sanity check flags, clean or copy 115a⟩

    ⟨sysrfork() if no RFPROC, no new proc but property settings 117c⟩
    // else
    ⟨sysrfork() when RFPROC, fork a new process 112d⟩
}

```

6.4.1 Fork: RFPROC

When RFPROC is set, the kernel allocates a new process, copies or shares the parent's resources according to the other flags, sets up the child's kernel stack so it will return 0 from the syscall (via `arch_forkchild()`^{114b}), and makes it runnable. The parent returns the child's pid.

```

⟨sysrfork() other locals 112c⟩≡ (112b) 113c▷
    Proc *p;
    ulong pid;

⟨sysrfork() when RFPROC, fork a new process 112d⟩≡ (112b)
    p = newproc();
    pid = p->pid;

    ⟨sysrfork() when RFPROC propagate fields from up to p 113a⟩

    /* Make a new set of memory segments */
    ⟨sysrfork() copy or share memory segments 114a⟩

```

```

/* File descriptors */
⟨sysrfork() copy, clean, or share file descriptors 115b⟩
/* Process groups */ // Namespace
⟨sysrfork() copy, clean, or share namespace 116b⟩
// Rendez vous group
⟨sysrfork() new or share rendezvous group 116e⟩
/* Environment group */
⟨sysrfork() copy, clean, or share environment 116d⟩
// Note group
⟨sysrfork() share or not note group 117a⟩

/* Craft a return frame which will cause the child to pop out of
 * the scheduler in user mode with the return register zero
 */
arch_forkchild(p, up->dbgreg);

p->parent = up;
p->parentpid = up->pid;
⟨sysrfork() wait or not for child 134b⟩

⟨sysrfork() setting time field 444c⟩

/*
 * since the bss/data segments are now shareable,
 * any mmu info about this process is now stale
 * (i.e. has bad properties) and has to be discarded.
 */
arch_flushmmu();

⟨sysrfork() if parent is a wired proc 208c⟩

ready(p);
sched(); // !!!
return pid;

```

The `ready(p)` call puts the child on the run queue, and `sched()` immediately reschedules—the parent may or may not continue first. The `arch_forkchild()` call (defined further below) sets up the child’s kernel stack so that when the scheduler picks it, it returns from `rfork` with a return value of 0 (how the child knows it is the child).

```

⟨sysrfork() when RFPROC propagate fields from up to p 113a⟩≡ (112d) 113b▷
    kstrdup(&p->text, up->text);
    kstrdup(&p->user, up->user);

⟨sysrfork() when RFPROC propagate fields from up to p 113b⟩+≡ (112d) <113a 141d>
    p->slash = up->slash;
    p->dot = up->dot;
    incref(p->dot);

```

6.4.2 Memory sharing: RFMEM

If `RFMEM` is set, the child shares the parent’s data and bss segments (thread-like behavior). Otherwise, `dupseg()`^{100b} makes copy-on-write copies. The text segment is always shared (it is read-only).

```

⟨sysrfork() other locals 113c⟩+≡ (112b) <112c 117b>▷
    bool share;
    int i;

```

```

⟨sysrfork() copy or share memory segments 114a⟩≡ (112d)
    share = flag & RFMEM;

    qlock(&p->seglock);
    if(waserror()){
        qunlock(&p->seglock);
        nexterror();
    }
    for(i = 0; i < NSEG; i++)
        if(up->seg[i])
            p->seg[i] = dupseg(up->seg, i, share);
    qunlock(&p->seglock);
    poperror();

```

6.4.3 arch_forkchild() (ARM)

arch_forkchild() ^{114b} is the key architecture-specific piece of fork: it sets up the child's kernel stack so that when the scheduler picks the child, it will "return" from the syscall with R0=0 (the child's return value). It copies the parent's saved user registers (Ureg) onto the child's kernel stack and sets p->sched (the Label) to point to arch__forkretX, which restores these registers and returns to user mode.

```

⟨function arch_forkchild(arm) 114b⟩≡ (719c)
/*
 * Craft a return frame which will cause the child to pop out of
 * the scheduler in user mode with the return register zero. Set
 * pc to point to a l.s return function.
 */
void
arch_forkchild(Proc *p, Ureg *ureg)
{
    Ureg *cureg;

    p->sched.sp = (ulong)p->kstack + KSTACK - sizeof(Ureg);
    p->sched.pc = (ulong)arch__forkret;

    cureg = (Ureg*)(p->sched.sp);
    memmove(cureg, ureg, sizeof(Ureg));
    /* syscall returns 0 for child */ // adjust
    cureg->r0 = 0;

    /* Things from bottom of syscall which were never executed */
    p->psstate = nil;
    p->insyscall = false;

    ⟨arch_forkchild() floating point(arm) 455a⟩
}

```

```

⟨Proc state fields 114c⟩+≡ (49f) <111a 122d>
    bool insyscall; // true when process inside a syscall

```

```

⟨function arch__forkret(arm) 114d⟩≡ (719a)
/*
 * here to jump to a newly forked process
 */
TEXT arch__forkret(SB), 1, $-4
    ADD $(4*15), R13    /* make r13 point to ureg->type */
    MOVW    8(R13), R14  /* restore link */
    MOVW    4(R13), R0   /* restore SPSR */
    MOVW    R0, SPSR    /* ... */

```

```

MOVM.DB.S (R13), [R0-R14] /* restore registers */
ADD $8, R13 /* pop past ureg->{type+psr} */
RFE /* MOVM.IA.S.W (R13), [R15] */

```

Here is the memory layout after `arch_forkchild()` has set up the child, showing how the parent and child relate:

PARENT (up)	CHILD (p)
user stack: (shared or COW copy of parent) +-----+ user code/data +-----+	user stack: (shared or COW copy of parent) +-----+ user code/data +-----+
kernel stack (up->kstack, 8 KiB): +-----+ <-- kstack+KSTACK Ureg (saved by _vsvc r0..r14 on syscall entry) type,psr pc (return to user) +-----+ <-- up->dbgreg (used by syscall handler) +-----+ <-- kstack	kernel stack (p->kstack, 8 KiB): +-----+ <-- kstack+KSTACK Ureg (copied from parent, r0=0! r0 set to 0) type,psr pc (same user PC) +-----+ <-- p->sched.sp (empty) +-----+ <-- kstack
up->sched.pc = (in sched code) up->sched.sp = (in sched code)	p->sched.pc = <code>arch__forkret</code> p->sched.sp = <code>&Ureg</code> on kstack

When scheduler picks child:

1. `gotolabel(p->sched) -> jump to arch__forkret`
2. `arch__forkret` restores `Ureg` from kstack
3. `RFE -> user mode, R0=0` (child sees `rfork()`=0)

6.4.4 File descriptors: RFFDG, RFCFDG

The next subsections handle resource sharing. They all follow the same three-way pattern: if the “G” flag is set, copy the parent’s resource; if the “CG” flag is set, create a clean (empty) one; if neither, share the parent’s (just increment the reference count). The two flags are mutually exclusive.

```

<sysrfork() sanity check flags, clean or copy 115a>≡ (112b) 116a▷
if((flag & (RFFDG|RFCFDG)) == (RFFDG|RFCFDG))
    error(Ebadarg);

```

```

<sysrfork() copy, clean, or share file descriptors 115b>≡ (112d)
if(flag & (RFFDG|RFCFDG)) {
    if(flag & RFFDG)
        p->fgrp = dupfgrp(up->fgrp);
    else
        p->fgrp = dupfgrp(nil);
}

```

```

}
else {
    p->fgrp = up->fgrp;
    incref(p->fgrp);
}

```

6.4.5 Namespace: RENAMEG, RFCNAMEG

Same pattern as file descriptors, but for the process's view of the file system (the namespace). RFNOMNT is an additional flag that, when set, prevents the child from mounting anything—useful for sandboxing untrusted programs.

```

<sysrfork() sanity check flags, clean or copy 116a>+≡ (112b) <115a 116c>
    if((flag & (RENAMEG|RFCNAMEG)) == (RENAMEG|RFCNAMEG))
        error(Ebadarg);

<sysrfork() copy, clean, or share namespace 116b>≡ (112d)
    if(flag & (RENAMEG|RFCNAMEG)) {
        p->pgrp = newpgrp();
        if(flag & RENAMEG)
            pgrpcpy(p->pgrp, up->pgrp);
        <sysrfork() inherit noattach, RFPROC==1 case 318d>
    }
    else {
        p->pgrp = up->pgrp;
        incref(p->pgrp);
    }
    <sysrfork() set noattach to true when RFNOMNT, RFPROC==1 case 318f>

```

6.4.6 Environment variables: RFENVG, RFCENVG

```

<sysrfork() sanity check flags, clean or copy 116c>+≡ (112b) <116a
    if((flag & (RFENVG|RFCENVG)) == (RFENVG|RFCENVG))
        error(Ebadarg);

<sysrfork() copy, clean, or share environment 116d>≡ (112d)
    if(flag & (RFENVG|RFCENVG)) {
        p->egrp = smalloc(sizeof(Egrp)); // newegrp
        p->egrp->ref = 1;
        if(flag & RFENVG)
            envcpy(p->egrp, up->egrp);
    }
    else {
        p->egrp = up->egrp;
        incref(p->egrp);
    }

```

6.4.7 Rendezvous group: RFREND

Rendezvous is Plan 9's lightweight IPC mechanism: two processes that share a rendezvous group can synchronize by calling `rendezvous()` with a matching tag (see Chapter 18). RFREND gives the child a fresh group, isolating it from the parent's rendezvous partners.

```

<sysrfork() new or share rendezvous group 116e>≡ (112d)
    if(flag & RFREND)
        p->rgrp = newrgrp();
    else {

```

```

    incref(up->rgrp);
    p->rgrp = up->rgrp;
}

```

6.4.8 Note group: RFNOTEG

Notes are Plan 9's equivalent of UNIX signals (see Chapter 7). Processes that share a note group can send notes to each other; RFNOTEG gives the child its own group, so notes sent to the parent's group will not reach the child.

```

<sysrfork() share or not note group 117a>≡ (112d)
    if((flag&RFNOTEG) == 0)
        p->noteid = up->noteid; // rectify what was done in newproc

```

6.4.9 Process sharing settings (no RFPROC)

When RFPROC is not set, `rfork` does not create a child. Instead, it modifies the current process's own resources—for example, a program can call `rfork(RFNAMEG)` to get its own private copy of the namespace before mounting something, without forking a new process. The code below mirrors the RFPROC branch but operates on `up` instead of a new `p`, and must carefully close the old resource groups after switching.

```

<sysrfork() other locals 117b>+≡ (112b) <113c 208b>
    Fgrp *ofg;
    Pgrp *opg;
    Egrp *oeg;
    Rgrp *org;

```

```

<sysrfork() if no RFPROC, no new proc but property settings 117c>≡ (112b)
    if((flag&RFPROC) == 0) { // not a fork, just setting properties for up
        <sysrfork() when no RFPROC, sanity check flag 117d>

        <sysrfork() when no RFPROC, clean or copy file descriptors 117e>

        <sysrfork() when no RFPROC, clean or copy namespace 118a>

        <sysrfork() when no RFPROC, clean or copy environment 118b>

        <sysrfork() when no RFPROC, new rendez vous group 118c>

        <sysrfork() when no RFPROC, new note group 118d>

        return 0;
    }

```

```

<sysrfork() when no RFPROC, sanity check flag 117d>≡ (117c)
    if(flag & (RFMEM|RFNOWAIT))
        error(Ebadarg);

```

```

<sysrfork() when no RFPROC, clean or copy file descriptors 117e>≡ (117c)
    if(flag & (RFFDG|RFCFDG)) {
        ofg = up->fgrp;
        if(flag & RFFDG)
            up->fgrp = dupfgrp(ofg);
        else
            up->fgrp = dupfgrp(nil);
        closefgrp(ofg);
    }

```

```

⟨sysrfork() when no RFPROC, clean or copy namespace 118a⟩≡ (117c)
    if(flag & (RFNAMEG|RFCNAMEG)) {
        opg = up->pgrp;
        up->pgrp = newpgrp();
        if(flag & RFNAMEG)
            pgrpncpy(up->pgrp, opg);
        ⟨sysrfork() inherit noattach, RFPROC==0 case 318c⟩
        closepgrp(opg);
    }
    ⟨sysrfork() set noattach to true when RFNOMNT, RFPROC==0 case 318e⟩

⟨sysrfork() when no RFPROC, clean or copy environment 118b⟩≡ (117c)
    if(flag & (RFENVG|RFCENVG)) {
        oeg = up->egrp;
        up->egrp = smalloc(sizeof(Egrp)); // newegrp()
        up->egrp->ref = 1;
        if(flag & RFENVG)
            envcpy(up->egrp, oeg);
        closeegrp(oeg);
    }

⟨sysrfork() when no RFPROC, new rendez vous group 118c⟩≡ (117c)
    if(flag & RFREND) {
        org = up->rgrp;
        up->rgrp = newrgrp();
        closergrp(org);
    }

⟨sysrfork() when no RFPROC, new note group 118d⟩≡ (117c)
    if(flag & RFNOTE)
        up->noteid = incref(&noteidalloc);

```

6.5 sysexec()

sysexec()X is the second half of the fork/exec pair. It replaces the current process's memory image with a new program loaded from a file. This is a complex operation because the old program's memory must be freed while building the new one—and if anything goes wrong (bad binary, out of memory), the process must survive with its old image intact. The logical chain is: file name → open channel → read **a.out** header → create text/data/bss segments → count arguments → build arguments on temporary stack → free old segments → relocate stack to final position → set PC and SP → return to user mode in new program. Script files starting with **#!** are handled by re-opening with the interpreter.

```

⟨constant AOUT_MAGIC(arm) 118e⟩≡ (716d)
/*
 * parameters for sysproc.c
 */
// E_MAGIC is defined in include/a.out.h
#define AOUT_MAGIC (E_MAGIC)

```

The logical chain—filename string → channel → binary header → segments—is reflected in the local variables:

```

⟨sysexec() locals 118f⟩≡ (119d) 118g▷
    char *file;
    char *elem; // last element of binary, for up->text
    char *file0;

⟨sysexec() locals 118g⟩+≡ (119d) ◁118f 119a▷
    Chan *tc;

```

```

⟨sysexec() locals 119a⟩+≡ (119d) <118g 119b>
    Exec exec;
    ulong magic;
    user_addr text, entry, data, bss;

⟨sysexec() locals 119b⟩+≡ (119d) <119a 119c>
    user_addr t, d, b; // end of text/data/bss in address rounded to page
    Segment *s, *ts;

⟨sysexec() locals 119c⟩+≡ (119d) <119b ??>
    ulong ssize;
    ulong nargs;

⟨syscall exec 119d⟩≡ (679a)
    // void* exec(char *name, char* argv[]);
    long
    sysexec(ulong* arg)
    {
        ⟨sysexec() locals 118f⟩

        elem = nil;
        validaddr(arg[0], 1, false);
        file0 = validnamedup((char*)arg[0], true);
        if(waserror()){
            free(file0);
            free(elem);
            nexterror();
        }
        file = file0;
        for(;;){
            // this will also adjust up->genbuf to contain the last elt of file path
            ⟨sysexec() call namec() to get a channel in tc from file 267a⟩
            if(waserror()){
                cclose(tc);
                nexterror();
            }
            if(!indir)
                kstrdup(&elem, up->genbuf);

            n = devtab[tc->type]->read(tc, &exec, sizeof(Exec), 0);
            if(n < 2) // need at least 2 bytes to decide if a #! or real binary
                error(Ebadexec);
            magic = l2be(exec.magic);
            text = l2be(exec.text);
            entry = l2be(exec.entry);

            if(n==sizeof(Exec) && (magic == AOUT_MAGIC)){
                if(text >= USTKTOP-UTZERO
                || entry < UTZERO+sizeof(Exec)
                || entry >= UTZERO+sizeof(Exec)+text)
                    error(Ebadexec);

                // else
                break; /* for binary */
            }
            ⟨sysexec() process sharpbang 450b⟩
        }
        // from break above
        data = l2be(exec.data);
        bss = l2be(exec.bss);
    }

```


<sysexec() compute end of text, data, bss segments 121a>
 <sysexec() build args count 123c>
 <sysexec() build temporary stack segment and qlock seglock 124a>
 <sysexec() build args 124c>
 <sysexec() free old memory segments 121d>
 <sysexec() close files marked as opened with close on exec 281f>
 <sysexec() set Text segment 122a>
 <sysexec() set Data segment 122b>
 <sysexec() set Bss segment 122c>
 <sysexec() relocate temporary stack segment, unlock seglock 125a>
 <sysexec() set prioriry of root processes 191h>

```

poperror();
cclose(tc); // tc has still a reference in the img

/*
 * At this point, the mmu contains info about the old address
 * space and needs to be flushed
 */
arch_flushmmu();

qlock(&up->debug);
<sysexec() when hold debug lock, reset some fields 396c>
arch_procsetup(up);
qunlock(&up->debug);

<sysexec() if hang 429e>

return arch_execregs(entry, ssize, nargs);
}

```

sysexec() is one of the longest functions in the kernel. The key steps are: open the binary via **namec()**^{267g} (the name-to-channel resolution function, covered in Chapter 14), read its header to get the text/data/bss sizes, tear down the old segments, create new ones, and build the stack.

The binary file (a.out format) starts with an **Exec** header followed by the text and data sections. On disk the layout is contiguous, but in memory the segments are placed at different virtual addresses:

On disk (a.out file):		In memory (virtual):
+-----+ Exec header	USTKTOP	+-----+ stack
+-----+ text (code)	b	+-----+ bss (zero)
+-----+ data	d	+-----+



The constants `t`, `d`, `b` below are the virtual addresses of the boundaries between segments, all page-aligned:

<sysexec() compute end of text, data, bss segments 121a>≡ (119d)

```
t = UTROUND(UTZERO+sizeof(Exec)+text);
// data is put at page boundary after text
d = ROUND(t + data, BY2PG);
// note that this is not d + bss here, but t + data + bss!
b = ROUND(t + data + bss, BY2PG);
```

```
if(t >= KZERO || d >= KZERO || b >= KZERO)
    error(Ebadexec);
```

<function l2be 121b>≡ (679a)

```
ulong
l2be(long l)
{
    byte *cp;

    cp = (byte*)&l;
    return (cp[0]<<24) | (cp[1]<<16) | (cp[2]<<8) | cp[3];
}
```

<macro UTROUND(arm) 121c>≡ (709b)

```
#define UTROUND(t)    ROUNDUP((t), BY2PG)
```

<sysexec() free old memory segments 121d>≡ (119d)

```
/*
 * Committed.
 * Free old memory.
 * Special segments are maintained across exec
 */
for(i = SSEG; i <= BSEG; i++) {
    putseg(up->seg[i]);
    /* prevent a second free if we have an error */
    up->seg[i] = nil;
}
```

<sysexec() free special segments if close on exec 410c>

`arch_procsetup()` initializes the architecture-specific parts of a process after `exec`: it sets up the MMU page table for the new address space and initializes the floating-point state.

<function arch_procsetup(arm) 121e>≡ (717b)

```
/*
 * set mach dependent process state for a new process
 */
void
arch_procsetup(Proc* p)
{
    <arch_procsetup() floating point(arm) 455c>
}
```

6.5.1 Text segment

The text (code) segment is shared and demand-loaded: the pages are not read from disk now but will be faulted in on first access. If another process is already running the same executable, the text pages are shared via a cache image.

```
<sysexec() set Text segment 122a>≡ (119d)
/* Text. Shared. Attaches to cache image if possible */
<sysexec() get text segment ts via demand loading on tc 170d>
up->seg[TSEG] = ts;
```

6.5.2 Data segment

```
<sysexec() set Data segment 122b>≡ (119d)
/* Data. Shared. */
s = newseg(SG_DATA, t, (d-t)>>PGSHIFT);
up->seg[DSEG] = s;
<sysexec() adjust data segment s for demand loading on tc 170e>
```

6.5.3 Bss segment

The bss is the simplest: it starts where data ends, and its pages are zero-filled on demand by the page fault handler—no disk I/O needed.

```
<sysexec() set Bss segment 122c>≡ (119d)
/* BSS. Zero fill on demand */
up->seg[BSEG] = newseg(SG_BSS, d, (b-d)>>PGSHIFT);
```

6.5.4 Stack segment

Each process has two stacks: a user stack (in the SSEG segment, growing down from USTKTOP) and a kernel stack (the per-process **kstack**, used during syscalls and interrupts). The exec code builds the new user stack with the program's arguments:

```
USTKTOP +-----+
| Tos                | (time-of-stack info)
+-----+
| argv strings       | "ls\0" "-l\0" ...
+-----+
| argv[n]=0          | (nil terminator)
| argv[1]             | (pointer to "-l")
| argv[0]             | (pointer to "ls")
+-----+
| nargs              | (argument count)
SP --> +-----+
|                   |
| (free stack space) |
|                   |
+-----+
```

Arguments count

```
<Proc state fields 122d>+≡ (49f) <114c 127c>
char *args;
int nargs; /* number of bytes of args */
```

```

⟨newproc() set fields of p 123a⟩+≡ (108c) ◁111b 127b▷
    kstrdup(&p->args, "");
    p->nargs = 0;

⟨sysexec() locals 123b⟩+≡ (119d) ◁?? 123f▷
    char **argp;
    char *a, *args;
    ulong nbytes;

⟨sysexec() build args count 123c⟩≡ (119d)
    /*
     * Args: pass 1: count
     */
    nargs = 0;
    nbytes = 0;

    ⟨sysexec() nbytes tos adjustments 449g⟩
    ⟨sysexec() if indir arg adjustments 451b⟩

    arch_validalign(arg[1], sizeof(char**));
    argp = (char**)arg[1];
    validaddr((ulong)argp, BY2WD, false);
    while(*argp){
        a = *argp++;
        if(((ulong)argp&(BY2PG-1)) < BY2WD)
            validaddr((ulong)argp, BY2WD, false);
        validaddr((ulong)a, 1, false);
        nbytes += ((char*)vmemchr(a, 0, 0x7FFFFFFF) - a) + 1;
        nargs++;
    }
    ssize = BY2WD*(nargs+1) + ROUND(nbytes, BY2WD);

```

Temporary stack segment

Building the new stack is the trickiest part of exec. The kernel must write the argument strings and `argv` pointers into the new program's stack, but it cannot overwrite the current stack—the old program's syscall frame is still there. So the kernel creates a temporary stack segment at a different virtual address (`TSTKTOP`), builds the arguments there, and only after the old segments are freed does it relocate this segment to its final position at `USTKTOP`.

```

⟨constant TSTKTOP(arm) 123d⟩≡ (709b)
    // end of new stack in sysexec
    #define TSTKTOP      (USTKTOP-USTKSIZE) /* sysexec temporary stack */

⟨constant TSTKSIZ(arm) 123e⟩≡ (709b)
    #define TSTKSIZ      256 // pages in new stack; limits exec args

```

The kernel allocates a temporary stack segment at `TSTKTOP` (well below `USTKTOP`) so the old and new stacks coexist briefly:

```

USTKTOP  +-----+
         | old stack (SSEG) |  <- still in use by syscall
         +-----+
         |      ...      |
TSTKTOP  +-----+
         | temp stack (ESEG) |  <- new args built here
         +-----+

```

```

⟨sysexec() locals 123f⟩+≡ (119d) ◁123b 124b▷
    ulong spage;

```

```

<sysexec() build temporary stack segment and qlock seglock 124a>≡ (119d)
/*
 * 8-byte align SP for those (e.g. sparc) that need it.
 * arch_execregs() will subtract another 4 bytes for argc.
 */
if((ssize+4) & 7)
    ssize += 4;
spage = (ssize+(BY2PG-1)) >> PGSHIFT;

/*
 * Build the stack segment, putting it in kernel virtual for the moment
 */
if(spage > TSTKSIZ)
    error(Enovmem);

qlock(&up->seglock);
if(waserror()){
    qunlock(&up->seglock);
    nexterror();
}
// why ESEG? and why not TSTKTOP?? ESEG because will free later
// the current SSEG. If we have an error in sysexec we don't want
// to have messed up with the current stack. TSTKTOP-USTKSIZE for
// the same reason, because we don't want overwrite old stack
// all of that will be relocated later.
up->seg[ESEG] = newseg(SG_STACK, TSTKTOP-USTKSIZE, USTKSIZE/BY2PG);

```

Arguments

The arguments are copied into the temporary stack in two passes: first the string data, then the `argv` pointer array pointing back into the strings.

```

<sysexec() locals 124b>+≡ (119d) <123f 170c>
char **argv;
char *charp;

```

```

<sysexec() build args 124c>≡ (119d)
/*
 * Args: pass 2: assemble; the pages will be faulted in
 */
<sysexec() tos settings 448i>

argv = (char**)(TSTKTOP - ssize);
charp = (char*)(TSTKTOP - nbytes);
args = charp;
<sysexec() if indir argp adjustments 451c>
else
    argp = (char**)arg[1];

for(i=0; i<nargs; i++){
    <sysexec() if indir argp adjustments again 452>
    *argv++ = charp + (USTKTOP-TSTKTOP);
    n = strlen(*argp) + 1;
    memmove(charp, *argp++, n);
    charp += n;
}
free(file0);
// file0 = nil? to avoid double free?

```

```

free(up->text);
up->text = elem;
elem = nil; /* so waserror() won't free elem */
USED(elem);

/* copy args; easiest from new process's stack */
n = charp - args;
if(n > 128) /* don't waste too much space on huge arg lists */
    n = 128;
a = up->args;
up->args = nil;
free(a);
up->args = smalloc(n);
memmove(up->args, args, n);

if(n>0 && up->args[n-1]!='\0'){
    /* make sure last arg is NUL-terminated */
    /* put NUL at UTF-8 character boundary */
    for(i=n-1; i>0; --i)
        if(fullrune(up->args+i, n-i))
            break;
    up->args[i] = 0;
    n = i+1;
}
up->nargs = n;

```

Note another use of `smalloc()` above for the argument buffer—it sleeps if memory is unavailable.

Relocate temporary stack segment

```

⟨sysexec() relocate temporary stack segment, unlock seglock 125a⟩≡ (119d)
/*
 * Move the stack
 */
s = up->seg[ESEG];
up->seg[ESEG] = nil;
up->seg[SSEG] = s;
qunlock(&up->seglock);

poperror(); /* seglock */
poperror(); /* elem */ // really? I think this matches more the cclose(tc)
s->base = USTKTOP-USTKSIZE;
s->top = USTKTOP;
relocateseg(s, USTKTOP-TSTKTOP);

⟨function relocateseg 125b⟩≡ (665)
void
relocateseg(Segment *s, ulong offset)
{
    Page **pg, *x;
    Pagetable *pt, **p, **endpt;

    endpt = &s->pagedir[s->pagedirsize];
    for(p = s->pagedir; p < endpt; p++) {
        if(*p == nil)
            continue;
        pt = *p;
        for(pg = pt->first; pg <= pt->last; pg++) {
            if(x = *pg)
                x->va += offset;
        }
    }
}

```

```

    }
}
}

```

6.5.5 arch_execregs() (ARM)

The final call in `sysexec()` is `arch_execregs()`, which sets the user registers (PC to the entry point, SP to the new stack) so that when the kernel returns to user mode, the new program starts. It also reserves space at the top of the stack for the `Tos` (Time of Stack) structure, which the kernel uses to pass timing and profiling information to user space (described in Chapter 10).

```

⟨function arch_execregs(arm) 126⟩≡ (719c)
long
arch_execregs(user_addr entry, ulong ssize, ulong nargs)
{
    user_wp sp;
    Ureg *ureg;

    sp = (user_wp)(USTKTOP - ssize);
    *--sp = nargs;

    ureg = up->dbgreg;
    ureg->r13 = (ulong)sp;
    ureg->pc = entry;

    /*
     * return the address of kernel/user shared data
     * (e.g. clock stuff)
     */
    return USTKTOP-sizeof(Tos);
}

```

6.6 sysawait()

We now look at the parent's side of process termination before seeing the child's (`sysexits()`). It is easier to understand the exit code once you know what the parent expects to receive. When a child exits, it leaves a wait message in the parent's queue (`Proc.waitq`). There is no race: even if the child exits before the parent calls `await()`, the message is preserved (up to 128 per parent). The parent blocks on `Proc.waitr` until a message is available, then returns it as a formatted string.

There are no zombie processes in Plan 9. When a child exits, it writes a wait message (containing its exit string and resource usage) to the parent's queue and immediately dies. The parent collects these messages via `await()`. If the parent has already exited, the messages are simply discarded. In UNIX, a zombie is created when a child exits but its parent has not yet called `wait()` —the child's `Proc` entry lingers, holding a process slot, until the parent collects the exit status. If the parent never calls `wait`, zombies accumulate. Plan 9 avoids this entirely: the exit status is copied into the parent's wait queue (a bounded buffer, not a `Proc` slot), and the child's `Proc` is freed immediately.

6.6.1 Parent and children

Each process keeps a pointer to its parent and a count of living children. But the `parent` pointer alone is not safe: if the parent exits, its `Proc` slot is reused for a new process. The `parentpid` field solves this—before accessing `parent`, the child checks that `parent->pid == parentpid`. If not, the parent has died and been replaced, so the child acts as an orphan. Orphans and zombies are different concepts: a zombie is a dead child whose exit

status has not been collected; an orphan is a living child whose parent has died. Plan 9 has orphans but no zombies.

```

<Proc hierarchy fields 127a>≡ (49f) 127d▷
    // option<ref<Proc>> (nil for the boot process)
    Proc *parent;
    int nchild; /* Number of living children */

<newproc() set fields of p 127b>+≡ (108c) <123a 127e▷
    p->parent = nil;
    p->nchild = 0;

<Proc state fields 127c>+≡ (49f) <122d
    // option<pid> (None = 0)
    ulong parentpid;

```

6.6.2 Waitmsg and Waitq

This is the data structure behind Plan 9’s “no zombies” design. Instead of keeping the dead child’s `Proc` around (as UNIX does with zombie processes), the child writes a small `Waitmsg` (containing its pid, CPU time, and exit string) to the parent’s `waitq` and then frees its `Proc` immediately. The parent collects these messages at its leisure via `await()`.

```

<Proc hierarchy fields 127d>+≡ (49f) <127a 129b▷
    // list<ref_own<Waitq>> =~ list<ref_own<Waitmsg>>
    Waitq *waitq; /* Exited processes wait children */
    int nwait; /* Number of uncollected wait records */ // len(waitq)

    Lock exl; /* Lock count and waitq */ // and nchild

<newproc() set fields of p 127e>+≡ (108c) <127b 135a▷
    p->waitq = nil;
    p->nwait = 0;

<struct Waitmsg 127f>≡ (656e)
    struct Waitmsg
    {
        int pid; /* of loved one */ // pid of the child
        <Waitmsg time field 445c>
        char msg[ERRMAX]; /* actually variable-size in user mode */
    };

<struct Waitq 127g>≡ (668c)
    // essentially a stack<ref_own<Waitmsg>>
    struct Waitq
    {
        Waitmsg w;

        // extra
        // list<ref_own<Waitq>> Proc.waitq
        Waitq *next;
    };

```

A `Waitq` entry is allocated (via `smalloc`) by the dying child in `pexit()`^{130b} and linked into the parent’s `Proc.waitq` list. The 128 limit is enforced by `Proc.nwait`—if it reaches the maximum, the parent must drain some messages before more can be queued.

6.6.3 sysawait()

With the data structures in place, `sysawait()` itself is straightforward: it sleeps until a `Waitq` entry appears in the parent's queue, then formats it into the user-provided buffer:

<syscall await 128a>≡ (679a)

```
// int await(char *s, int n);
long
sysawait(ulong* arg)
{
    int i;
    int pid;
    Waitmsg w; // allocated in stack!
    ulong n;

    n = arg[1];
    validaddr(arg[0], n, true);
    pid = pwait(&w);
    <sysawait() sanity check pid 128b>
    i = snprintf((char*)arg[0], n, "%d %lud %lud %lud %q",
        w.pid,
        <sysawait() snprint time field arguments 445g>
        w.msg);

    return i;
}
```

<sysawait() sanity check pid 128b>≡ (128a)

```
if(pid < 0)
    return -1;
```

<function pwait 128c>≡ (675c)

```
ulong
pwait(Waitmsg *w)
{
    ulong cpid; // child pid
    Waitq *wq;

    <pwait() lock qwaitr 536b>

    <pwait() sanity check process has children 129a>

    // sleep!
    <pwait() wait until has a waitq 129c>
    // ok, got a waitmsg

    lock(&up->exl);
    // wq = pop(up->waitq), // can't be null, see haswaitq()
    wq = up->waitq;
    up->waitq = wq->next;
    up->nwait--;
    unlock(&up->exl);

    <pwait() unlock qwaitr 536c>

    if(w)
        memmove(w, &wq->w, sizeof(Waitmsg));
    cpid = wq->w.pid;
    free(wq);
    return cpid;
}
```

```

⟨pwait() sanity check process has children 129a⟩≡ (128c)
    lock(&up->exl);
    if(up->nchild == 0 && up->waitq == nil) {
        unlock(&up->exl);
        error(Enochild);
    }
    unlock(&up->exl);

```

6.6.4 Rendez-vous: Proc.waitr

Here is the first concrete use of **Rendez** (introduced in the Concurrency chapter): **waitr** is the rendezvous point where a parent sleeps in **sysawait()** until a child exits and wakes it up.

```

⟨Proc hierarchy fields 129b⟩+≡ (49f) <127d 535a>
    Rendez waitr; /* Place to hang out in wait */

```

```

⟨pwait() wait until has a waitq 129c⟩≡ (128c)
    sleep(&up->waitr, haswaitq, up); // qwaitr is still locked

```

```

⟨function haswaitq 129d⟩≡ (675c)
    int
    haswaitq(void *x)
    {
        Proc *p;
        p = (Proc *)x;
        return p->waitq != nil;
    }

```

The **sleep()** call (from the Concurrency chapter) takes a **Rendez** and a condition function. It acquires the **Rendez**'s internal lock, checks the condition, and if false, puts the process to sleep. When another process calls **wakeup()** on the same **Rendez**, the sleeping process is woken and re-checks the condition. Here the condition is **haswaitq()**—"is there a **Waitq** entry in my queue?"

6.7 sysexits()

sysexits() is a thin wrapper that copies the exit string from user space and calls **pexit()**^{130b}, the real termination function. The exit string is optional (**nil** means success). **pexit()** never returns—the process is destroyed.

```

⟨syscall exits 129e⟩≡ (679a)
    // void exits(char *msg);
    long
    sysexits(ulong* arg)
    {
        char *status;
        ⟨sysexits() other locals 129f⟩

        status = (char*)arg[0];
        ⟨sysexits() sanity check status, copy string in buf 130a⟩
        pexit(status, /*freemem*/true);

        panic("pexit: should never reach this point");
        return -1; // unreachable
    }

```

```

⟨sysexits() other locals 129f⟩≡ (129e)
    char *inval = "invalid exit string";
    char buf[ERRMAX];

```

```

⟨sysexits() sanity check status, copy string in buf 130a⟩≡ (129e)
    if(status){
        if(waserror()){
            status = inval;
        }
        else{
            // validaddr() and vmemchr() can generate error()
            validaddr((ulong)status, 1, false);
            if(vmemchr(status, '\0', ERRMAX) == 0){
                memmove(buf, status, ERRMAX);
                buf[ERRMAX-1] = '\0';
                status = buf;
            }
            poperror();
        }
    }
}

```

6.7.1 pexit()

`pexit()` is the real termination function, and its design is split into two phases for a subtle reason. Phase 1 runs on the dying process's own kernel stack: it frees resources (file descriptors, namespace, segments), builds the wait message for the parent, and wakes the parent up. Phase 2 runs on the CPU's main stack (inside `schedinit()` ^{195e}): it releases the MMU state and returns the `Proc` to the free pool. The split is necessary because the process cannot free its own kernel stack while still using it—it must switch to another stack first. The process's state goes through `Moribund` (between the two phases) and finally `Dead`.

The two-phase design is necessary because phase 1 runs on the process's own kernel stack (`Proc.kstack`), which cannot be freed while it is in use. Phase 2 runs on the CPU's boot stack (inside the `Cpu` structure at `CPUADDR`):

Phase 1 (<code>Proc.kstack</code>):	Phase 2 (<code>Cpu.stack</code>):
<code>kstack+KSTACK</code>	<code>CPUADDR+CPUSIZE</code>
<pre> +-----+ pexit() frame free fds, segments, namespace... +-----+ </pre>	<pre> +-----+ schedinit() frees kstack sets Dead recycles Proc +-----+ </pre>
<code>kstack</code>	<code>CPUADDR</code>

```

⟨function pexit 130b⟩≡ (675c)
void
proc_pexit(char *exitstr, bool freemem)
{
    Proc *p; // parent
    ⟨pexit() other locals 132a⟩

    // free 1
    ⟨pexit() at the start, free resources 219b⟩
    ⟨pexit() hook proctrace 441b⟩
    // free 2
    ⟨pexit() nil out resources under lock and free them 134a⟩

    ⟨pexit() if a kernel process, do nothing about parent and child 135c⟩
    else {

```

```

/*
 * if not a kernel process and have a parent,
 * do some housekeeping.
 */
p = up->parent;
⟨pexit() sanity check parent pointer p 131b⟩
while(waserror())
    ; // ???

// exit/await and your parent
⟨pexit() build waitq message wq 132b⟩
⟨pexit() add wq message in parent process p waitq 132c⟩
}
⟨pexit() if not freemem, add process to broken processes list 425d⟩

// free 3, must be after built waitq because exitstr in one of the segment
⟨pexit() free memory segments 133e⟩

// exit/await and your (orphan) children
⟨pexit() wakeup process waiting on waitr and unhash pid 133a⟩

// free 4
⟨pexit() free waitq 133c⟩
// free 5
⟨pexit() release debuggers 427f⟩

/* Sched must not loop for these locks */
lock(&procalloc);
lock(&palloc);

⟨pexit() optional edfstop() for real-time scheduling 504b⟩
up->state = Moribund;
// will arch_gotolabel() to schedinit() which has special code around Moribund
sched();
panic("pexit: should never reach this point");
}

⟨Procstate cases 131a⟩+≡ (51b) <108b 202b>
Moribund,

⟨pexit() sanity check parent pointer p 131b⟩≡ (130b)
// no parent pointer, must be the very first process
if(p == nil) {
    if(exitstr == nil)
        exitstr = "unknown";
    panic("boot process died: %s", exitstr);
}

```

6.7.2 pexit() and schedinit()

Here is phase 2: `pexit()`^{130b} set the state to `Moribund` and called `sched()`^{194c}, which jumped to `schedinit()`^{195e} on the CPU's main stack. Now `schedinit()` finishes the cleanup: release the MMU state, push the `Proc` back onto the free list, and unlock the locks that `pexit()` was holding.

```

⟨schedinit() switch process state cases 131c⟩≡ (199a) 199b▷
case Moribund:
    up->state = Dead;
    ⟨schedinit() optional real-time edfstop() 502h⟩
/*

```

```

    * Holding locks from pexit:
    *  procalloc
    *  palloc
    */
arch_mmurelease(up);

up->qnext = procalloc.free;
procalloc.free = up;

unlock(&palloc);
unlock(&procalloc);
break;

```

6.7.3 Wait message

We saw the data structures for wait messages earlier, but the code that creates them lives inside `pexit()`—this is where the dying process builds the `Waitq` entry and links it into its parent’s queue.

Wait message and your parent

```

⟨pexit() other locals 132a⟩≡ (130b) 133b▷
    Waitq *wq;

⟨pexit() build waitq message wq 132b⟩≡ (130b)
    wq = smalloc(sizeof(Waitq));
    poperror(); // of ???

    wq->w.pid = up->pid;
    ⟨pexit() set wait msg time field 445e⟩
    if(exitstr && exitstr[0])
        snprintf(wq->w.msg, sizeof(wq->w.msg), "%s %lud: %s", up->text, up->pid, exitstr);
    else
        wq->w.msg[0] = '\0';

```

Again `smalloc()`—the dying process may sleep here if memory is tight.

```

⟨pexit() add wq message in parent process p waitq 132c⟩≡ (130b)
    lock(&p->exl);
    /*
     * Check that parent is still alive.
     */
    // the parent pointer may not be your parent anymore, the parent
    // could have died and its slot reallocated to another process
    if(p->pid == up->parentpid && p->state != Broken) {
        p->nchild--;
        ⟨pexit() update TC time of parent 445f⟩
        /*
         * If there would be more than 128 wait records
         * processes for my parent, then don't leave a wait
         * record behind. This helps prevent badly written
         * daemon processes from accumulating lots of wait
         * records.
         */
        if(p->nwait < 128) {
            // push(wq, p->waitq)
            wq->next = p->waitq;
            p->waitq = wq;
            p->nwait++;
        }
    }

```

```

        wq = nil; // the parent will do the free
        wakeup(&p->waitr); // haswaitq() is true now
    }
}
unlock(&p->exl);
if(wq)
    free(wq);

```

Wait and my (orphan) children

When a process exits, it must also deal with its own children: wake up anyone waiting on this process's `waitr`, unhash its pid, and discard any uncollected wait messages from its own children.

```

⟨pexit() wakeup process waiting on waitr and unhash pid 133a⟩≡ (130b)
    lock(&up->exl);      /* Prevent my children from leaving waits */
    pidunhash(up);
    // so my children will not generate a waitq, I will not be here anymore
    up->pid = 0;
    wakeup(&up->waitr);
    unlock(&up->exl);

```

```

⟨pexit() other locals 133b⟩+≡ (130b) <132a 133d>
    Waitq *f, *next;

```

```

⟨pexit() free waitq 133c⟩≡ (130b)
    for(f = up->waitq; f; f = next) {
        next = f->next;
        free(f);
    }

```

6.7.4 Freeing resources

With the wait message safely queued in the parent, the dying process can now free its own resources: segments, file descriptors, namespace, environment, and rendezvous group. This must happen after building the wait message, because the exit string lives in one of the process's segments.

```

⟨pexit() other locals 133d⟩+≡ (130b) <133b 133f>
    Segment **s, **es;

```

```

⟨pexit() free memory segments 133e⟩≡ (130b)
    qlock(&up->seglock);
    es = &up->seg[NSEG];
    for(s = up->seg; s < es; s++) {
        if(*s) {
            putseg(*s);
            *s = nil;
        }
    }
    qunlock(&up->seglock);

```

```

⟨pexit() other locals 133f⟩+≡ (130b) <133d 441a>
    Fgrp *fgrp;
    Egrp *egrp;
    Rgrp *rgrp;
    Pgrp *pgrp;
    Chan *dot;

```

```

⟨pexit() nil out resources under lock and free them 134a⟩≡ (130b)
/* nil out all the resources under lock (free later) */
qlock(&up->debug);
fgrp = up->fgrp;
up->fgrp = nil;
egrp = up->egrp;
up->egrp = nil;
rgrp = up->rgrp;
up->rgrp = nil;
pgrp = up->pgrp;
up->pgrp = nil;
dot = up->dot;
up->dot = nil;
qunlock(&up->debug);

if(fgrp)
    closefgrp(fgrp);
if(egrp)
    closeegrp(egrp);
if(rgrp)
    closergrp(rgrp);
if(dot)
    cclose(dot);
if(pgrp)
    closepgrp(pgrp);

```

6.7.5 No waiting for child: RFNOWAIT

Now that we have seen `sysawait()` and `pexit()`, we can understand the `RFNOWAIT` flag in `sysrfork()`. When set, `parentpid` is cleared to 0, so when the child exits, `pexit()`^{130b} will see that the parent's pid does not match `parentpid` and skip the wait message entirely. The child becomes an orphan: no one will collect its exit status. If `RFNOWAIT` is not set, the parent's `nchild` count is incremented normally. This is used by programs that spawn helper processes they never need to wait for—for example, `rio` (the window system) and `lib9p` use `RFNOWAIT` when forking worker processes.

```

⟨sysrfork() wait or not for child 134b⟩≡ (112d)
if(flag&RFNOWAIT)
    p->parentpid = 0;
else {
    lock(&up->exl);
    up->nchild++;
    unlock(&up->exl);
}

```

6.8 Kernel processes

Not all processes run user programs. The kernel creates a few background kernel processes for housekeeping tasks: the pager (reclaims pages when memory is low), the alarm handler, etc. A kernel process has no user segments—it runs entirely in kernel memory. It is created by `kproc()`^{135b}, which is simpler than `sysrfork()`X: it calls `newproc()`^{108c}, sets `kp=true`, stores the entry function in `kpfun`, and makes the process runnable. When scheduled, the process enters `linkproc()`^{136b}, which enables interrupts and calls the function.

```

⟨Proc other fields 134c⟩≡ (49f) 141c▷
bool kp; /* true if a kernel process */
void (*kpfun)(void*);
void *kparg;

```

```

⟨newproc() set fields of p 135a⟩+≡ (108c) <127e 137b>
    p->kp = false;

⟨function kproc 135b⟩≡ (675c)
    // kernel process (aka kernel_thread in Linux?)
    void
    kproc(char *name, void (*func)(void *), void *arg)
    {
        Proc *p;
        static Pgrp *kpgrp;

        p = newproc();
        p->kp = true; // Kernel Process!

        p->psstate = nil;
        p->procmode = 0640;
        p->ureg = nil;
        p->noswap = true;

        p->slash = up->slash;
        p->dot = up->dot;
        if(p->dot)
            incref(p->dot);

        p->fpsave = up->fpsave;
        p->sargs = up->sargs;

        memmove(p->note, up->note, sizeof(p->note));
        p->nnote = up->nnote;
        p->lastnote = up->lastnote;
        p->notify = up->notify;

        procpriority(p, PriKproc, false);

        arch_kprocchild(p, func, arg);

        kstrdup(&p->user, eve);
        kstrdup(&p->text, name);

        if(kpgrp == nil)
            kpgrp = newpgrp();
        p->pgrp = kpgrp;
        incref(kpgrp);

        ⟨kproc() setting time field 444d⟩
        ready(p);
    }

```

Unlike `sysrfork()`, `kproc()` calls `ready(p)` but not `sched()`—it does not voluntarily yield, so the new kernel process will run when the scheduler next picks it. The `kp` flag marks it as a kernel process.

Kernel processes have no user segments—they run entirely in kernel address space. This is why `pexit()` checks `up->kp` early and skips the parent/child wait message machinery:

```

⟨pexit() if a kernel process, do nothing about parent and child 135c⟩≡ (130b)
    if(up->kp) {
    }

```

6.8.1 `arch_kprocchild()` (ARM)

`arch_kprocchild()` is the kernel-process counterpart of `arch_forkchild()`—it sets up the child’s kernel stack

so that it starts executing `func(arg)` when scheduled. It is simpler because there is no user-mode return to arrange:

```
⟨function arch_kprocchild(arm) 136a⟩≡ (717a)
/*
 * setup stack and initial PC for a new kernel proc. This is architecture
 * dependent because of the starting stack location
 */
void
arch_kprocchild(Proc *p, void (*func)(void*), void *arg)
{
    p->sched.pc = PTR2UINT(linkproc);
    p->sched.sp = PTR2UINT(p->kstack+KSTACK);

    p->kpfun = func;
    p->kparg = arg;
}
```

`linkproc()` is the entry point for all kernel processes—`arch_kprocchild()` sets the child’s PC to `linkproc` above. It calls the actual function, then `pexit()` when it returns:

```
⟨function linkproc(arm) 136b⟩≡ (717a)
/*
 * this is the body for all kproc’s
 */
static void
linkproc(void)
{
    arch_spllo();
    up->kpfun(up->kparg);
    // should never reach this place?? kernel processes are supposed
    // to run forever??
    pexit("kproc exiting", false);
}
```

6.9 Process control `/proc/#/ctl`

Processes can be controlled from outside by writing commands to `/proc/#/ctl`. For example, `echo kill > /proc/42/ctl` sends a “sys: killed” note to process 42. This is how the shell implements job control and how administrators kill runaway processes. The full `/proc` file system is covered in Chapter 15. The shell `rc` itself does not use `/proc/\emphpid/ctl` directly—it sends notes via `postnote()`—but tools like `kill(1)` and `slay(1)` do write to `ctl`.

```
⟨Procctl cases 136c⟩≡ (426e) 432a▷
Proc_exitme,
```

The `Procctl` enum values correspond to commands that can be written to `/proc/\emphpid/ctl`. The full `/proc` device is covered in Chapter 15, but `procctl()` is shown here because its cases use `pexit()` and other helpers defined in this chapter. It is fine to skim this code for now.

```
⟨procctl() Proc_exitme case 136d⟩≡ (427b)
case Proc_exitme:
    arch_spllo(); /* pexit has locks in it */
    pexit("Killed", true);
```

```

⟨procctlreq() CMkill case 137a⟩≡ (554b)
case CMkill:
    switch(p->state) {
        ⟨procctlreq() CMkill case, Broken case 426c⟩
        case Stopped:
            p->procctl = Proc_exitme;
            postnote(p, 0, "sys: killed", NExit);
            ready(p);
            break;
        default:
            p->procctl = Proc_exitme;
            postnote(p, 0, "sys: killed", NExit);
    }
    break;

⟨newproc() set fields of p 137b⟩+≡ (108c) ◁135a 186b▷
p->pdbg = nil;
p->procctl = Proc_nothing;
p->syscalltrace = nil;
p->trace = false;
p->dbgreg = nil;

```

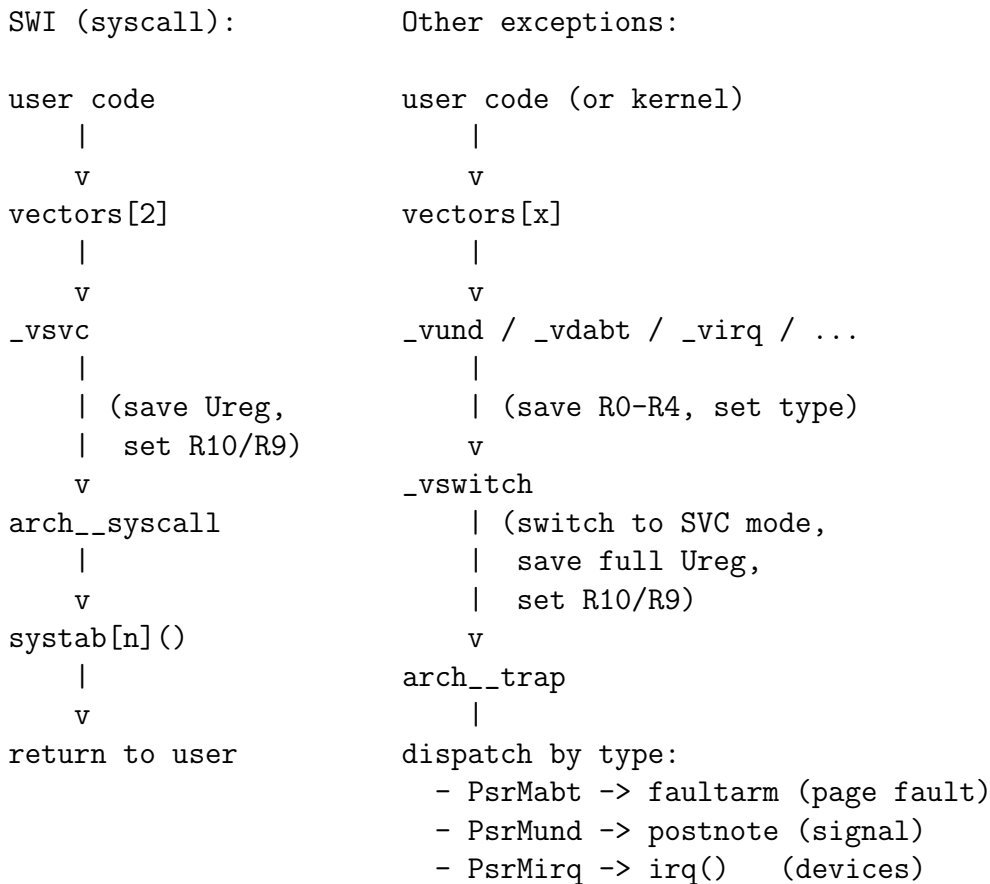
Chapter 7

Interrupts

This chapter shows how the kernel handles events that interrupt normal execution: system calls (which we already traced in the Overview), hardware interrupts (timer, keyboard, disk), and faults (page faults, undefined instructions). Most of the code is architecture-specific, but the overall structure is the same on any architecture: save registers, dispatch to a handler, restore registers, return.

7.1 Overview

There are two main paths through this chapter. The syscall path (a software interrupt via **SWI**) goes through `_vsvc` → `arch__syscall` → `systab[n]` → return. The exception/IRQ path goes through `_vxxx` → `_vswitch` → `arch__trap`, which dispatches based on the exception type: page faults, undefined instructions, or hardware IRQs.



7.2 Exceptions (ARM)

We saw `PsrMusr` and `PsrMsvc` in the Overview. Here are the remaining processor modes, one for each exception type. Each mode has its own banked R13/R14, which is why `_vswitch` must save those registers before switching to SVC mode:

```
<PsrMode other cases(arm) 139a>≡ (28a) 498b▷
#define PsrMirq      0x00000012
#define PsrMabt      0x00000017
#define PsrMund      0x0000001B

<global trapnames(arm) 139b>≡ (712c)
static char *trapnames[PsrMask+1] = {
    [ PsrMusr ] "user mode",
    [ PsrMsvc ] "svc/swi exception",
    [ PsrMirq ] "irq interrupt",
    [ PsrMabt ] "prefetch abort/data abort",
    [ PsrMabt+1 ] "data abort",
    [ PsrMund ] "undefined instruction",
    <trapnames() other entries(arm) 499a>
};
```

7.3 Exception vectors (ARM)

The ARM has 8 exception entries at fixed offsets (0x00 for reset, 0x04 for undefined, 0x08 for SWI, etc.). Each entry is a single instruction that loads the PC from a nearby table (`vtable`). The `vectors` and `vtable` code below is copied to `HVECTORS` (0xFFFF0000) by `arch__trapinit()`^{243f} during boot. The indirection through `vtable` is necessary because the ARM vectors are only one instruction wide—not enough room for a full address. Each vector entry does `MOVW 0x18(R15), R15`, R15, which loads the PC from 8 instructions ahead (where `vtable` starts), effectively jumping to the handler:

```
<global vectors(arm) 139c>≡ (714a)
/*
 * exception vectors, copied by trapinit() to somewhere useful
 */
TEXT vectors(SB), 1, $-4
    MOVW    0x18(R15), R15    /* reset */
    MOVW    0x18(R15), R15    /* undefined instr. */
    MOVW    0x18(R15), R15    /* SWI & SMC */
    MOVW    0x18(R15), R15    /* prefetch abort */
    MOVW    0x18(R15), R15    /* data abort */
    MOVW    0x18(R15), R15    /* reserved */
    MOVW    0x18(R15), R15    /* IRQ */
    MOVW    0x18(R15), R15    /* FIQ */

<global vtable(arm) 139d>≡ (714a)
//coupling: must be just after vectors in memory (at +0x18 = 8 instructions)
TEXT vtable(SB), 1, $-4
    WORD    $_vsvc(SB)        /* reset, in svc mode already */
    WORD    $_vund(SB)        /* undefined, switch to svc mode */
    WORD    $_vsvc(SB)        /* swi, in svc mode already */
    WORD    $_vpabt(SB)       /* prefetch abort, switch to svc mode */
    WORD    $_vdabt(SB)       /* data abort, switch to svc mode */
    WORD    $_vsvc(SB)        /* reserved */
    WORD    $_virq(SB)        /* IRQ, switch to svc mode */
    WORD    $_vfiq(SB)        /* FIQ, switch to svc mode */
```

7.4 System calls

We already traced the syscall path in the Overview chapter. Here we see the actual code. The SWI instruction puts the ARM in SVC mode and jumps to `vectors[2]`, which loads `_vsvc` from `vtable[2]`.

7.4.1 `_vsvc()` (ARM)

`_vsvc` builds a Ureg on the kernel stack by pushing registers in the right order, then calls `arch__syscall()` in C. On return, it restores the registers and executes RFE (Return From Exception) to resume user mode. The key steps are:

1. Push `ureg->pc` (= R14, the return address)
2. Push `ureg->psr` (= SPSR, the saved user CPSR)
3. Push `ureg->type` (= `PsrMsvc`, to identify this as a syscall)
4. Push R0-R14 (all user registers, via `MOVM.DB.S`)
5. Set R12 = kernel SB, R10 = `cpu`, R9 = `up`
6. Call `arch__syscall(ureg)`
7. Restore R0-R14, SPSR from the Ureg
8. RFE -> back to user mode

<constant CPU(arm) 140a>≡ (709b)

```
#define CPU      10      /* R10 is cpu-> */
```

<constant UP(arm) 140b>≡ (709b)

```
#define UP        9      /* R9 is up-> */
```

<function _vsvc(arm) 140c>≡ (714a)

```
TEXT _vsvc(SB), 1, $-4      /* SWI */
    CLREX
    MOVW.W  R14, -4(R13)     /* ureg->pc = interrupted PC */
    MOVW    SPSR, R14        /* ureg->psr = SPSR */
    MOVW.W  R14, -4(R13)     /* ... */
    MOVW    $PsrMsvc, R14    /* ureg->type = PsrMsvc */
    MOVW.W  R14, -4(R13)     /* ... */

    /* avoid the ambiguity described in notes/movm.w. */
    MOVM.DB.S [R0-R14], (R13) /* save user level registers */
    SUB $(15*4), R13         /* r13 now points to ureg */

    MOVW    $setR12(SB), R12 /* Make sure we've got the kernel's SB loaded */

    /* get R(CPU) for this cpu */
    <_vsvc() set cpu register(arm) 484b>

    MOVW    8(R(CPU)), R(UP) /* up */ // Cpu->proc

    MOVW    R13, R0          /* first arg is pointer to ureg */
    SUB $8, R13              /* space for argument+link */

    BL arch__syscall(SB)

    ADD $(8+4*15), R13       /* make r13 point to ureg->type */
    MOVW    8(R13), R14      /* restore link */
    MOVW    4(R13), R0       /* restore SPSR */
    MOVW    R0, SPSR        /* ... */
    MOVM.DB.S (R13), [R0-R14] /* restore registers */
    ADD $8, R13              /* pop past ureg->{type+psr} */
    RFE                     /* MOVM.IA.S.W (R13), [R15] */
```

7.4.2 Sargs

The syscall arguments live on the user stack, but the kernel copies them into `Proc.sargs` before dispatching. This avoids repeated user-space accesses (which could fault) and ensures the arguments cannot be modified by another thread while the syscall is running. `MAXSYSARG` is 5 because `mount()` has the most parameters of any syscall: `mount(fd, afd, mpt, flag, arg)`.

```
<constant MAXSYSARG 141a>≡ (668c)
#define MAXSYSARG 5 /* for mount(fd, afd, mpt, flag, arg) */
```

```
<struct Sargs 141b>≡ (668c)
// syscall arguments copied from user stack
struct Sargs
{
    ulong args[MAXSYSARG];
};
```

```
<Proc other fields 141c>+≡ (49f) <134c 223f>
Sargs sargs; /* address of this is known by db */
```

```
<sysrfork() when RFPROC propagate fields from up to p 141d>+≡ (112d) <113b 186c>
p->sargs = up->sargs;
```

7.4.3 arch__syscall() (ARM)

`arch__syscall()` is the C-level syscall dispatcher, called by `_vsvc` with a pointer to the saved `Ureg`. It extracts the syscall number from `ureg->r0`, copies the arguments from the user stack into `up->sargs`, calls `sysstab[scallnr]()`, stores the return value in `ureg->r0`, and handles errors. Before returning, it checks for pending notes (`notify`) and deferred scheduling.

```
<function arch__syscall(arm) 141e>≡ (719c)
void
arch__syscall(Ureg* ureg)
{
    int scallnr;
    long ret;
    user_addr sp;
    <syscall() other locals(arm) 142a>

    <syscall() sanity check call from userspace(arm) 143d>
    <syscall() adjust kentry 449c>

    cpu->syscall++;
    up->insyscall = true;

    up->pc = ureg->pc;
    up->dbgreg = ureg;

    // syscall number!
    scallnr = ureg->r0;
    //up->scallnr = scallnr;

    <syscall() if RFORK, handle floating point(arm) 456c>

    arch_spllo();
    sp = ureg->sp;

    <syscall() Proc_tracesyscall if, syscall entry(arm) 432c>
```

```

up->nerrlab = 0;
ret = -1;

startns = todget(nil);
if(!waserror()){
    <syscall() sanity check scallnr(arm) 142b>
    <syscall() sanity check sp(arm) 142c>

    // copy syscall arguments from user stack to up->sargs
    up->sargs = *((Sargs*)(sp+BY2WD)); // extra BY2WD for?
    up->psstate = systab[scallnr];

    //!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
    //IMPORTANT: The actual system call
    ret = systab[scallnr](up->sargs.args);
    //!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

    poperror();
}else{
    /* failure: save the error buffer for errstr */
    e = up->syserrstr;
    up->syserrstr = up->errstr;
    up->errstr = e;
}
<syscall() if nerrlab(arm) 143a>

/*
 * Put return value in frame. On the x86 the syscall is
 * just another trap and the return value from syscall is
 * ignored. On other machines the return value is put into
 * the results register by caller of syscall.
 */
ureg->r0 = ret;

<syscall() Proc_tracesyscall if, syscall exit(arm) 433b>

up->insyscall = false;
up->psstate = nil;

<syscall() call noted() 398b>
<syscall() call notify(arm) 396g>

<syscall() if delaysched(arm) 200e>
arch__kexit(ureg);
}

<syscall() other locals(arm) 142a>≡ (141e) 433a▷
char *e;
u32int s;
int i;

<syscall() sanity check scallnr(arm) 142b>≡ (141e)
if(scallnr >= nsyscall){
    pprint("bad sys call number %d pc %#lux\n",
        scallnr, ureg->pc);
    postnote(up, 1, "sys: bad sys call", NDebug);
    error(Ebadarg);
}

<syscall() sanity check sp(arm) 142c>≡ (141e)

```

```

    if(sp < (USTKT0P-BY2PG) || sp > (USTKT0P-sizeof(Sargs)-BY2WD))
        validaddr(sp, sizeof(Sargs)+BY2WD, 0);

⟨syscall() if nerrlab(arm) 143a⟩≡ (141e)
    if(up->nerrlab){
        print("bad errstack [%d]: %d extra\n", scallnr, up->nerrlab);
        for(i = 0; i < NERR; i++)
            print("sp=%#p pc=%#p\n",
                up->errlab[i].sp, up->errlab[i].pc);
        panic("error stack");
    }

⟨Proc debugger fields 143b⟩≡ (49f) 427a▷
    void *dbgreg; /* User registers for devproc */

⟨function arch_userpc(arm) 143c⟩≡ (717a)
    /*
     * return the userpc the last exception happened at
     */
    uintptr
    arch_userpc(void)
    {
        Ureg *ureg = up->dbgreg;
        return ureg->pc;
    }

```

Before dispatching, `arch__syscall()` checks that the SWI came from user mode—a syscall from kernel mode would be a bug. `arch_userureg()` tests whether the saved processor mode in `ureg->psr` is `PsrMusr`:

```

⟨syscall() sanity check call from userspace(arm) 143d⟩≡ (141e)
    if(!arch_userureg(ureg))
        panic("syscall: from kernel: pc %#lux r14 %#lux psr %#lux",
            ureg->pc, ureg->r14, ureg->psr);

⟨function arch_userureg(arm) 143e⟩≡ (717a)
    bool
    arch_userureg(Ureg* ureg)
    {
        return (ureg->psr & PsrMask) == PsrMusr;
    }

```

7.5 Exception handlers

All exceptions other than SWI (which has its own fast path via `_vsvc`) go through the same two-step mechanism: a short handler `_vxxx` that saves a few registers and records the exception type, then a common `_vswitch` routine that switches to SVC mode and builds the full `Ureg`.

7.5.1 `_vxxx()` (ARM)

Each exception type has a tiny entry point that saves R0–R4 to make working room, loads the exception type constant into R0, and falls through to `_vswitch`. Note that each exception mode has its own banked R13, so the `MOVMI.A` writes to that mode’s stack—a small scratch area set up during boot, not the kernel’s SVC stack:

```

⟨function _vund(arm) 143f⟩≡ (714a)
    TEXT _vund(SB), 1, $-4          /* undefined */
    MOVMI.A [R0-R4], (R13)        /* free some working space */
    MOVW    $PsrMund, R0
    B       _vswitch

```



```

⟨function _vpabt(arm) 144a⟩≡ (714a)
TEXT _vpabt(SB), 1, $-4      /* prefetch abort */
    MOVMI.A [R0-R4], (R13)  /* free some working space */
    MOVW    $PsrMabt, R0    /* r0 = type */
    B       _vswitch

```

```

⟨function _vdabt(arm) 144b⟩≡ (714a)
TEXT _vdabt(SB), 1, $-4      /* data abort */
    MOVMI.A [R0-R4], (R13)  /* free some working space */
    MOVW    $(PsrMabt+1), R0 /* r0 = type */
    B       _vswitch

```

```

⟨function _virq(arm) 144c⟩≡ (714a)
TEXT _virq(SB), 1, $-4      /* IRQ */
    MOVMI.A [R0-R4], (R13)  /* free some working space */
    MOVW    $PsrMirq, R0    /* r0 = type */
    B       _vswitch

```

7.5.2 _vswitch() (ARM)

_vswitch is the heart of the exception mechanism. It must switch from the exception's processor mode (IRQ, ABT, UND) to SVC mode, because the kernel runs in SVC mode and needs access to the SVC-mode stack. But switching modes changes the banked R13 and R14, so _vswitch saves them first (SPSR, R14, R13) into general registers (R1–R3), then switches:

In exception mode (e.g., IRQ):

```

R0 = exception type (set by _vxxx)
R1 = SPSR (saved user/kernel CPSR)
R2 = R14 (interrupted PC)
R3 = R13 (pointer to saved R0-R4)
|
v  switch CPSR to SVC mode
|

```

Now in SVC mode:

```

R13 = SVC stack (kernel stack)
Build Ureg on this stack, then call arch__trap()

```

It then checks whether the exception interrupted user code or kernel code (`AND.S \0xf, R1`)—the handling differs because a kernel-mode exception must preserve the kernel's R10 (CPU) and R9 (up), and may have been nested (an interrupt inside an interrupt handler).

```

⟨label _vswitch(arm) 144d⟩≡ (714a)
/*
 * come here with type in R0 and R13 pointing above saved [r0-r4].
 * we'll switch to SVC mode and then call trap.
 */
_vswitch:
    CLREX
    MOVW    SPSR, R1        /* save SPSR for ureg */
    MOVW    R14, R2        /* save interrupted pc for ureg */
    MOVW    R13, R3        /* save pointer to where the original [R0-R4] are */

/*
 * switch processor to svc mode. this switches the banked registers
 * (r13 [sp] and r14 [link]) to those of svc mode.
 */

```

```

MOVW    CPSR, R14
BIC $PsrMask, R14
ORR $(PsrDirq|PsrMsvc), R14
MOVW    R14, CPSR      /* switch! */

AND.S    $0xf, R1, R4      /* interrupted code kernel or user? */
BEQ _userexcep
// else
/* here for trap from SVC mode */
<_vswitch trap from SVC mode 145b>
/* here for trap from USER mode */
<_vswitch trap from USER mode 145a>
<_vswitch trap from USER mode 145a>≡ (144d)
_userexcep:
    MOVW.DB.W [R0-R2], (R13) /* set ureg->{type, psr, pc}; r13 points to ureg->type */
    MOVW.IA (R3), [R0-R4] /* restore [R0-R4] from previous mode's stack */

/* avoid the ambiguity described in notes/movm.w. */
MOVW.DB.S [R0-R14], (R13) /* save kernel level registers */
SUB $(15*4), R13 /* r13 now points to ureg */

MOVW    $setR12(SB), R12 /* Make sure we've got the kernel's SB loaded */

/* get R(CPU) for this cpu */
<_vswitch() set cpu register(arm) 484c>

MOVW    8(R(CPU)), R(UP) /* up */

MOVW    R13, R0 /* first arg is pointer to ureg */
SUB $(4*2), R13 /* space for argument+link (for debugger) */

BL arch__trap(SB)

ADD $(4*2+4*15), R13 /* make r13 point to ureg->type */
MOVW    8(R13), R14 /* restore link */
MOVW    4(R13), R0 /* restore SPSR */
MOVW    R0, SPSR /* ... */
MOVW.DB.S (R13), [R0-R14] /* restore registers */
ADD $(4*2), R13 /* pop past ureg->{type+psr} */
RFE /* MOVW.IA.S.W (R13), [R15] */

```

The SVC-mode (kernel) trap path is trickier. Since the kernel was already in SVC mode when the exception occurred, R13 and R14 were not banked—the saved R0–R4 are on the exception mode’s stack, not the SVC stack. The code restores them, then saves all registers onto the SVC stack. On return, it must also overwrite the saved R10 with the current R(CPU), because if a reschedule happened during the trap, this processor might now be running a different process whose R10 was stale.

```

<_vswitch trap from SVC mode 145b>≡ (144d)
    MOVW.DB.W [R0-R2], (R13) /* set ureg->{type, psr, pc}; r13 points to ureg->type */
    MOVW.IA (R3), [R0-R4] /* restore [R0-R4] from previous mode's stack */

/*
 * avoid the ambiguity described in notes/movm.w.
 * In order to get a predictable value in R13 after the stores,
 * separate the store-multiple from the stack-pointer adjustment.
 * We'll assume that the old value of R13 should be stored on the stack.
 */
/* save kernel level registers, at end r13 points to ureg */
MOVW.DB [R0-R14], (R13)

```

```

SUB $(15*4), R13          /* SP now points to saved R0 */

MOVW    $setR12(SB), R12   /* Make sure we've got the kernel's SB loaded */

MOVW    R13, R0            /* first arg is pointer to ureg */
SUB $(4*2), R13            /* space for argument+link (for debugger) */
MOVW    $0xdeaddead, R11   /* marker */

BL arch__trap(SB)

MOVW    $setR12(SB), R12   /* reload kernel's SB (ONLY?) */
ADD $(4*2+4*15), R13       /* make r13 point to ureg->type */
/*
 * if we interrupted a previous trap's handler and are now
 * returning to it, we need to propagate the current R(CPU) (R10)
 * by overriding the saved one on the stack, since we may have
 * been rescheduled and be on a different processor now than
 * at entry.
 */
MOVW    R(CPU), (-(15-CPU)*4)(R13) /* restore current cpu's CPU */
MOVW    8(R13), R14        /* restore link */
MOVW    4(R13), R0         /* restore SPSR */
MOVW    R0, SPSR          /* ... */

MOVM.DB (R13), [R0-R14]    /* restore registers */

ADD $(4*2), R13            /* pop past ureg->{type+psr} to pc */
RFE                       /* MOVM.IA.S.W (R13), [R15] */

```

7.5.3 arch__trap() (ARM)

`arch__trap()` is the C-level exception dispatcher, analogous to `arch__syscall()`^{141e} for system calls. It receives the saved `Ureg` and dispatches based on `ureg->type`: data abort and prefetch abort go to the page fault handler, undefined instruction to a trap handler, and IRQ to the interrupt dispatcher. Before returning to user mode, it checks for pending notes (signals) and deferred scheduling, just like `arch__syscall()`. One subtle detail: the ARM pipeline means the saved PC points past the faulting instruction. For most exceptions, the interrupted PC is at `ureg->pc - 4`; for data aborts it is `ureg->pc - 8` (because the ARM has already advanced past the faulting instruction and the one after it). The code below corrects for this.

```

<function arch__trap(arm) 146>≡ (712c)
/*
 * here on all exceptions other than syscall (SWI) and fiq
 */
void
arch__trap(Ureg *ureg)
{
    bool user;
    bool clockintr;
    <trap() other locals(arm) 147c>

    <trap() sanity check interrupts are disabled(arm) 147b>
    <trap() sanity check enough space in kernel stack(arm) 147d>

    user = (ureg->psr & PsrMask) == PsrMusr; // TODO: arch_userureg
    if(user){
        up->dbgreg = ureg;
        <trap() adjust kentry when interrupt user 449d>
    }
}

```

```

/*
 * All interrupts/exceptions should be resumed at ureg->pc-4,
 * except for Data Abort which resumes at ureg->pc-8.
 */
if(ureg->type == (PsrMabt+1))
    ureg->pc -= 8;
else
    ureg->pc -= 4;

clockintr = false;      /* if set, may call sched() before return */

switch(ureg->type){
<trap() switch exception type cases(arm) 148b>
default:
    panic("unknown trap; type %#lux, psr mode %#lux pc %#lux", ureg->type,
        ureg->psr & PsrMask, ureg->pc);
    break;
}
arch_splhi();

<trap() if delaysched and clockintr 200d>

if(user){
    <trap() call possibly notify(arm) 396e>
    arch__kexit(ureg);
}
}

```

arch__kexit() is called just before returning from the kernel to user space (after both syscalls and exceptions). It updates the Tos structure with the current CPU frequency and flushes the write-back cache so that user code sees consistent memory.

```

<function arch__kexit(arm) 147a>≡ (717a)
/* go to user space */
void
arch__kexit(Ureg*)
{
    <arch__kexit() tos adjustments 449a>
    tos->cyclefreq = cpu->cpuhz;
    <arch__kexit() write back cache(arm) 470l>
}

<trap() sanity check interrupts are disabled(arm) 147b>≡ (146)
assert(!arch_islo());

<trap() other locals(arm) 147c>≡ (146) 148a▷
int rem;

<trap() sanity check enough space in kernel stack(arm) 147d>≡ (146)
if(up != nil)
    rem = ((char*)ureg)-up->kstack;
else
    rem = ((char*)ureg)-((char*)cpu + sizeof(Cpu));
if(rem < 256) {
    iprint("trap: %d stack bytes left, up %#p ureg %#p at pc %#lux\n",
        rem, up, ureg, ureg->pc);
    arch_delay(1000);
    arch_dumpstack();
    panic("trap: %d stack bytes left, up %#p ureg %#p at pc %#lux",
        rem, up, ureg, ureg->pc);
}
}

```

7.5.4 Undefined instruction fault (ARM)

An undefined instruction in user mode sends a note to the process (which can handle it or die). In kernel mode, it is a bug and causes a panic. The handler also checks for floating-point instructions, which may trap if the FPU was not enabled for this process.

```
<trap() other locals(arm) 148a>+≡ (146) <147c 148d>
char buf[ERRMAX];

<trap() switch exception type cases(arm) 148b>≡ (146) 148e>
case PsrMund: /* undefined instruction */
    if(user){
        <trap() when undefined instruction, if breakpoint(arm) 424a>
        else{
            <trap() when undefined instruction, if float instr(arm) 466a>
            if(rv == 0){
                snprintf(buf, sizeof buf,
                    "undefined instruction: pc %#lux\n",
                    ureg->pc);
                postnote(up, 1, buf, NDebug);
            }
        }
    }else{
        <trap() when undefined instruction, round pc(arm) 148c>
        iprint("undefined instruction: pc %#lux inst %#ux\n",
            ureg->pc, *(u32int*)ureg->pc);
        panic("undefined instruction");
    }
    break;

<trap() when undefined instruction, round pc(arm) 148c>≡ (148b)
if (ureg->pc & 3) {
    iprint("rounding fault pc %#lux down to word\n",
        ureg->pc);
    ureg->pc &= ~3;
}
```

7.5.5 Prefetch fault (ARM)

A prefetch abort means the CPU tried to fetch an instruction from an unmapped or protected address. If it is a valid user address, this triggers a page fault (handled by `faultarm()`¹⁵⁹, which calls the virtual memory system). Otherwise, it is a fatal error. The Instruction Fault Status Register (IFSR) encodes the reason; the code extracts a 4-bit fault code by concatenating non-contiguous bits (an ARMv6/v7 encoding quirk).

```
<trap() other locals(arm) 148d>+≡ (146) <148a 149c>
int x;
ulong fsr;

<trap() switch exception type cases(arm) 148e>+≡ (146) <148b 149d>
case PsrMabt: /* prefetch fault */
    x = ifsrget();
    fsr = (x>>7) & 0x8 | x & 0x7;
    switch(fsr){
        <trap() when prefetch fault, switch fst cases(arm) 424b>
    default:
        // Page fault on code!!
        faultarm(ureg, ureg->pc, user, true);
        break;
    }
    break;
```

```

⟨function ifsrget(arm) 149a⟩≡ (713)
TEXT ifsrget(SB), 1, $-4          /* instruction fault status */
    MRC CpSC, 0, R0, C(CpFSR), C(0), CpFSRinst
    RET

```

```

⟨type CpFSR(arm) 149b⟩≡ (683c)
/*
 * CpFSR opcode2 values.
 */
#define CpFSRdata    0          /* armv6, armv7 */
#define CpFSRinst    1          /* armv6, armv7 */

```

7.5.6 Data fault (ARM)

A data abort is the most common exception: it fires when a load or store accesses an unmapped page. This is the entry point for demand paging—the page fault handler (`faultarm()`¹⁵⁹) checks whether the address belongs to a valid segment, allocates a page if needed, fills it from disk or zeros it, and returns. The faulting instruction is then re-executed. Data aborts can also occur when the kernel accesses a user address during a syscall (e.g., copying arguments); this case is handled via the error/waserror mechanism. The Fault Status Register (FSR) tells the kernel why the fault occurred. The code extracts a 6-bit fault code from the FSR and dispatches on it: translation faults (0x5, 0x7) and permission faults (0xd, 0xf) go to the page fault handler; domain faults (0x9, 0xb) are access violations; alignment faults and external aborts are fatal errors.

```

⟨trap() other locals(arm) 149c⟩+≡ (146) <148d 465d>
    virt_addr va; // can be a kern_addr?
    ulong inst;

```

```

⟨trap() switch exception type cases(arm) 149d⟩+≡ (146) <148e 154b>
case PsrMabt+1:          /* data fault */
    va = farget();
    inst = *(ulong*)(ureg->pc);
    /* bits 12 and 10 have to be concatenated with status */
    x = fsrget();
    fsr = (x>>7) & 0x20 | (x>>6) & 0x10 | x & 0xf;
    switch(fsr){
    ⟨trap() when data fault, switch fst cases(arm) 149g⟩
    default:
    case 0xa:             /* ? was under external abort */
        panic("unknown data fault, 6b fsr %#lux", fsr);
        break;
    }
    break;

```

```

⟨function farget(arm) 149e⟩≡ (713)
TEXT farget(SB), 1, $-4          /* fault address */
    MRC CpSC, 0, R0, C(CpFAR), C(0x0)
    RET

```

```

⟨function fstget(arm) 149f⟩≡ (713)
TEXT fsrget(SB), 1, $-4          /* data fault status */
    MRC CpSC, 0, R0, C(CpFSR), C(0), CpFSRdata
    RET

```

```

⟨trap() when data fault, switch fst cases(arm) 149g⟩≡ (149d) 150b>
case 0xd:
case 0xf:
    /* permission error, copy on write or real permission error */
    faultarm(ureg, va, user, !writetomem(inst));
    break;

```

The page fault handler needs to know whether the faulting instruction was a load or a store, to set `PG_MOD` correctly. `writetomem()` decodes the faulting ARM instruction: bit 20 distinguishes loads (1) from stores (0) for most instructions, and swap instructions (detected by a specific bit pattern) always write.

```

⟨function writetomem(arm) 150a⟩≡ (712c)
/*
 * returns true if the instruction writes memory, false otherwise
 */
bool
writetomem(ulong inst)
{
    /* swap always write memory */
    if((inst & 0x0FC00000) == 0x01000000)
        return true;

    /* loads and stores are distinguished by bit 20 */
    if(inst & (1<<20))
        return false;

    return true;
}

⟨trap() when data fault, switch fst cases(arm) 150b⟩+≡ (149d) <149g 150c>
case 0x5:      /* translation fault, no section entry */
case 0x7:      /* translation fault, no page entry */
    // Page fault on data!!
    faultarm(ureg, va, user, !writetomem(inst));
    break;

⟨trap() when data fault, switch fst cases(arm) 150c⟩+≡ (149d) <150b 150d>
case 0x9:
case 0xb:
    /* domain fault, accessing something we shouldn't */
    if(user){
        snprintf(buf, sizeof buf,
            "sys: access violation: pc %#lux va %#p\n",
            ureg->pc, va);
        postnote(up, 1, buf, NDebug);
    } else
        panic("kernel access violation: pc %#lux va %#p",
            ureg->pc, va);
    break;

⟨trap() when data fault, switch fst cases(arm) 150d⟩+≡ (149d) <150c 150e>
case 0x0:
    panic("vector exception at %#lux", ureg->pc);
    break;

⟨trap() when data fault, switch fst cases(arm) 150e⟩+≡ (149d) <150d 151a>
case 0x1:      /* alignment fault */
case 0x3:      /* access flag fault (section) */
    if(user){
        snprintf(buf, sizeof buf,
            "sys: alignment: pc %#lux va %#p\n",
            ureg->pc, va);
        postnote(up, 1, buf, NDebug);
    } else
        panic("kernel alignment: pc %#lux va %#p", ureg->pc, va);
    break;

```

```

⟨trap() when data fault, switch fst cases(arm) 151a⟩+≡ (149d) <150e 151b>
    case 0x2:
        panic("terminal exception at %#lux", ureg->pc);
        break;

⟨trap() when data fault, switch fst cases(arm) 151b⟩+≡ (149d) <151a 151c>
    case 0x4: /* icache maint fault */
    case 0x6: /* access flag fault (page) */
    case 0x8: /* precise external abort, non-xlat'n */
    case 0x28:
    case 0xc: /* l1 translation, precise ext. abort */
    case 0x2c:
    case 0xe: /* l2 translation, precise ext. abort */
    case 0x2e:
    case 0x16: /* imprecise ext. abort, non-xlt'n */
    case 0x36:
        panic("external abort %#lux pc %#lux addr %#p",
            fsr, ureg->pc, va);
        break;

⟨trap() when data fault, switch fst cases(arm) 151c⟩+≡ (149d) <151b
    case 0x1c: /* l1 translation, precise parity err */
    case 0x1e: /* l2 translation, precise parity err */
    case 0x18: /* imprecise parity or ecc err */
        panic("translation parity error %#lux pc %#lux addr %#p",
            fsr, ureg->pc, va);
        break;

```

7.6 IRQs (ARM)

Hardware interrupts (IRQs) come from external devices: the timer, USB controller, SD card, UART, etc. On the Raspberry Pi, the BCM2835 has an interrupt controller at INTREGS with three 32-bit pending registers (covering up to 72 interrupt sources). Each device driver registers its handler via `arch_intrenable()`^{153a}, and when an IRQ fires, `irq()`^{154c} scans the pending bits and dispatches to the registered handler. The overall pattern is simple: at boot, each device driver calls `arch_intrenable(IRQxxx, handler, arg)` to register itself. At runtime, when a device raises an interrupt, the hardware sets a bit in the pending register, the ARM jumps to `_virq`, which builds a `Ureg` and calls `arch__trap`, which calls `irq()`, which walks the `vctl` list and calls every handler whose pending bit is set.

```

⟨enum IRQ(arm) 151d⟩≡ (685a)
enum IRQ {
    IRQtimer0 = 0, // reserved by GPU?
    IRQtimer1 = 1, // reserved by GPU?
    IRQtimer2 = 2, // reserved by GPU?
    IRQtimer3 = 3,

    IRQclock = IRQtimer3,

    IRQusb = 9, // also IRQfiq
    IRQdma0 = 16, // IRQdma1, IRQdma2, ... via IRQDMA() macro

    IRQaux = 29,
    IRQi2c = 53,
    IRQspi = 54,
    IRQmmc = 62,

    ⟨IRQ basic cases(arm) 153d⟩

```



```

    <IRQ local cases(arm) 491a>
    <IRQ other cases(arm) 499f>
};

```

7.6.1 Vctl (ARM)

A Vctl (“vector control”) records one registered interrupt handler: which IRQ number it handles, the hardware register and bitmask to check whether this IRQ is pending, the handler function, and its argument. All registered handlers form a linked list (vctl):

```

<struct Vctl(arm) 152a>≡ (712c)
    struct Vctl {
        // enum<IRQ>
        int irq;

        u32int *reg; // VIRTIO address
        u32int mask;

        void (*f)(Ureg*, void*); /* handler to call */
        void *a; /* argument to call it with */

        <Vctl other fields(arm) 491b>

        // Extra
        <Vctl extra fields(arm) 152c>
    };

<global vctl(arm) 152b>≡ (712c)
    // list<ref_own<Vctl>>> (next = Vctl.next)
    static Vctl *vctl;

<Vctl extra fields(arm) 152c>≡ (152a)
    Vctl *next;

<global vctllock(arm) 152d>≡ (712c)
    static Lock vctllock;

```

7.6.2 Intregs (ARM)

The BCM2835 interrupt controller is accessed via memory-mapped registers at INTREGS. It has separate pending/enable/disable registers for ARM-specific and GPU-shared interrupts. The structure fields are laid out to match the hardware registers exactly—writing to GPUenable[0] directly enables the corresponding interrupt in hardware:

```

<struct Intregs(arm) 152e>≡ (712c)
/*
 * interrupt control registers
 */
// The order matters! the fields match the memory-mapped external registers.
struct Intregs {
    u32int ARMPending;
    u32int GPUpending[2];

    u32int FIQctl;

    u32int GPUenable[2];
    u32int ARMenable;

```

```

    u32int GPUdisable[2];
    u32int ARMdisable;
};

```

7.6.3 arch_intrenable() (ARM)

Device drivers call `arch_intrenable()` ^{153a} during initialization to register their interrupt handler. It allocates a `Vctl`, records the handler and the appropriate pending register/mask, and enables the interrupt in the hardware controller:

```

<macro arch_intrenable(arm) 153a>≡ (695d)
    #define arch_intrenable(i, f, a, b, n) irqenable((i), (f), (a))

<function irqenable(arm) 153b>≡ (712c)
    void
    irqenable(int irq, void (*f)(Ureg*, void*), void* a)
    {
        Vctl *v;
        Intregs *ip;
        u32int *enable;

        ip = (Intregs*)INTREGS;
        v = (Vctl*)malloc(sizeof(Vctl));
        <irqenable() sanity check v(arm) 153c>
        v->irq = irq;
        v->cpu = 0;
        <irqenable() if IRQlocal(arm) 491c>
        <irqenable() if IRQbasic(arm) 154a>
        else{
            enable = &ip->GPUenable[irq/32];
            v->reg = &ip->GPUpending[irq/32];
            v->mask = 1 << (irq % 32);
        }
        v->f = f;
        v->a = a;
        lock(&vctllock);
        <irqenable() if IRQfiq(arm) 499g>
        else{
            v->next = vctl;
            vctl = v;
            <irqenable() if IRQlocal adjust enable(arm) 491d>
            else
                *enable = v->mask;
        }
        unlock(&vctllock);
    }

<irqenable() sanity check v(arm) 153c>≡ (153b)
    if(v == nil)
        panic("irqenable: no mem");

```

7.6.4 Basic IRQs (ARM)

The BCM2835 splits interrupts into two groups: 64 GPU interrupts (in `GPUpending[0..1]`) and a handful of ARM-specific interrupts (in `ARMpending`, starting at number 64). The ARM timer interrupt (`IRQtimerArm`) is in this second group:

```

<IRQ basic cases(arm) 153d>≡ (151d)
    IRQbasic      = 64,
    IRQtimerArm   = IRQbasic + 0,

```

```

⟨irqenable() if IRQbasic(arm) 154a⟩≡ (153b)
    else if(irq >= IRQbasic){
        enable = &ip->ARMenable;
        v->reg = &ip->ARMPending;
        v->mask = 1 << (irq - IRQbasic);
    }

```

7.6.5 irq() (ARM)

`irq()`^{154c} is called from `arch__trap` when the exception type is `PsrMirq`. It walks the `vctl` list and, for each handler whose pending bit is set, calls the handler. It returns `true` if a clock interrupt was among them—the caller uses this to decide whether to trigger a reschedule (the clock is what drives preemptive scheduling).

```

⟨trap() switch exception type cases(arm) 154b⟩+≡ (146) ◁149d
    case PsrMirq:
        clockintr = irq(ureg);
        cpu->intr++;
        break;

```

The dispatch loop is brute-force: walk the entire `vctl` list and check each handler’s pending bit. This is acceptable because the list is short (a handful of device drivers). If no handler matches, the interrupt is counted as spurious. The return value tells `arch__trap()` whether the clock was among the interrupts, which triggers a possible reschedule.

```

⟨function irq(arm) 154c⟩≡ (712c)
/*
 * called by trap to handle irq interrupts.
 * returns true iff a clock interrupt, thus maybe reschedule.
 */
static bool
irq(Ureg* ureg)
{
    Vctl *v;
    bool clockintr;
    bool found;

    cpu->perf.intrts = arch_perfticks();
    clockintr = false;
    found = false;
    for(v = vctl; v; v = v->next)
        if(v->cpu == cpu->cpuno && (*v->reg & v->mask) != 0){
            found = true;
            ⟨irq() before dispatch(arm) 472c⟩
            // Dispatch
            v->f(ureg, v->a);
            ⟨irq() after dispatch(arm) 473a⟩
            if(v->irq == IRQclock || v->irq == IRQcntps || v->irq == IRQcntpns)
                clockintr = true;
        }
    if(!found)
        cpu->spuriousintr++;
    intrtime();
    return clockintr;
}

```

Chapter 8

Virtual Memory

8.1 Overview

This chapter connects the interrupt machinery of the previous chapter to the memory structures of the Memory chapter. When a process accesses an unmapped page, the ARM MMU generates a data abort (or prefetch abort), which reaches `arch__trap()` and dispatches to `faultarm()`¹⁵⁹. The page fault handler then walks the software segment tables to decide what to do: allocate a zero page (bss), load from disk (demand loading), copy a shared page (copy on write), or page in from swap. The chapter is organized in layers: first the ARM MMU constants and hardware page table manipulation (`arch_putmmu`), then the portable page fault handler (`fault/fixfault`), then the higher-level mechanisms built on top: copy on write, the page cache, demand loading from executables, and swapping.

8.2 The ARM MMU (ARM)

We saw the ARM's two-level page table structure in the Overview chapter. Here are the macros that extract the L1 index (bits 31–20, selecting one of 4096 entries in the page directory) and the L2 index (bits 19–12, selecting one of 256 entries in a page table):

```
<macro L1X(arm) 155a>≡ (711a)
#define L1X(va)      FEXT((va), 20, 12)
```

```
<macro L2X(arm) 155b>≡ (711a)
#define L2X(va)      FEXT((va), 12, 8)
```

```
<macro L1X_bis(arm) 155c>≡ (684)
#define L1X(va)      (((((va))>>20) & 0x0fff)<<2)
```

The L1 entry type determines the granularity of the mapping. `Section` maps 1 MiB directly (used for kernel RAM). `Coarse` points to an L2 table of 256 entries, each mapping a 4 KiB page (used for user process pages):

```
<type PageDirGranularity(arm) 155d>≡ (683c)
#define Coarse      (Mbz|1)          /* L1 */
#define Section     (Mbz|2)          /* L1 1MB */
```

```
<constant Mbz(arm) 155e>≡ (683c)
/* Mbz (0x10) bit is implementation-defined and must be 0 on the cortex. */
#define Mbz         (0<<4)
```

```
<type PageTableGranularity(arm) 155f>≡ (683c)
#define Large       0x00000001        /* L2 64KB */
#define Small       0x00000002        /* L2 4KB */
```

The Domain Access Control (DAC) and Access Permission (AP) bits together control who can access a page. **Client** means the MMU checks the AP bits; **Manager** means the domain bypasses all permission checks. For AP: **Krw** = kernel read/write only, **Uro** = user read-only, **Urw** = user read/write:

```

⟨type DAC(arm) 156a⟩≡ (683c)
    #define Client      1          /* DAC */
    #define Manager     3          /* DAC */

⟨type PageTableEntryAttribute3(arm) 156b⟩≡ (683c)
    #define Noaccess    0          /* AP, DAC */
    #define Krw         1          /* AP */
    /* armv7 deprecates AP[2] == 1 & AP[1:0] == 2 (Uro), prefers 3 (new in v7) */
    #define Uro         2          /* AP */
    #define Urw         3          /* AP */

⟨constant Fault(arm) 156c⟩≡ (683c)
    #define Fault       0x00000000 /* L[12] pte: unmapped */

⟨constant Dom0(arm) 156d⟩≡ (683c)
    #define Dom0        0

⟨macro L1AP(arm) 156e⟩≡ (683c)
    #define L1AP(ap)    (AP(3, (ap)))

⟨macro AP(arm) 156f⟩≡ (683c)
    #define AP(n, v)    F((v), ((n)*2)+4, 2)

⟨macro F(arm) 156g⟩≡ (683c)
    #define F(v, o, w)  (((v) & ((1<<(w))-1))<<(o))

⟨macro DAC(arm) 156h⟩≡ (683c)
    #define DAC(n, v)   F((v), (n)*2, 2)

⟨macro L2AP(arm) 156i⟩≡ (711a)
    #define L2AP(ap)    l2ap(ap)

⟨function l2ap(raspberrypi)(arm) 156j⟩≡ (707a)
    int
    l2ap(int ap)
    {
        return (AP(3, (ap))|AP(2, (ap))|AP(1, (ap))|AP(0, (ap)));
    }

```

8.3 Pagination and mmupd/mmupt (ARM)

Now we see how Plan 9 uses the ARM MMU hardware. Each CPU has a single L1 page directory (`cpu->mmul1`), shared between the kernel (upper half, mapped with **Section** entries during boot and never changed) and the current user process (lower half, populated on demand by `arch_putmmu()` and cleared on context switch by `mmul1empty()`). Each process keeps its own list of L2 page table pages in `Proc.mmul2`. When the scheduler switches to a new process, `arch_mmuswitch()` invalidates the old user mappings from the L1 and the TLB; the new process's pages are faulted in on demand.

```

⟨Cpu Arch mmu fields(arm) 156k⟩≡ (48e) 163c▷
    PTE*    mmul1; /* l1 for this processor */

⟨type PTE(arm) 156l⟩≡ (695b)
    typedef u32int PTE;

```

L1lo and L1hi delimit the range of L1 entries used for user-space mappings. Only entries between L1lo (index 0, at UZERO) and L1hi (just past USTKTOP) are cleared on context switch; the kernel entries above KZERO are left untouched:

```
<constant L1lo(arm) 157a>≡ (710)
L1lo      = UZERO/MiB,      /* L1X(UZERO)? */
```

```
<constant L1hi(arm) 157b>≡ (710)
L1hi      = (USTKTOP+MiB-1)/MiB, /* L1X(USTKTOP+MiB-1)? */
```

```
<Proc Arch memory fields(arm) 157c>≡ (50) 158b▷
// list<ref<Page> > pages used to store page tables
Page*    mmul2;
```

```
<macro PPN(arm) 157d>≡ (709b)
#define PPN(x)      ((x)&~(BY2PG-1))
```

```
<macro VA(arm) 157e>≡ (709a)
#define VA(k)      ((uintptr)(k))
```

When a process dies (Moribund state), schedinit() calls arch_mmurelease() to free all its hardware page table resources: the L2 pages are zeroed and returned to the free page list, the L1 user entries are cleared, and the TLB is invalidated.

```
<function arch_mmurelease(arm) 157f>≡ (711a)
void
arch_mmurelease(Proc* proc)
{
    Page *page, *next;

    <arch_mmurelease() invalidate cache before adjusting page tables(arm) 470h>
    mmul2empty(proc, false);
    <arch_mmurelease() free l2 page cache(arm) 158d>
    mmul1empty();
    <arch_mmurelease() write back cache after adjusting page tables(arm) 470a>
    /* lose any possible stale tlb entries */
    mmuinvalidate();
}
```

```
<function mmuinvalidate(arm) 157g>≡ (711b)
/*
 * invalidate tlb
 */
TEXT mmuinvalidate(SB), 1, $-4
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinv
    BARRIERS
    RET
```

```
<type CpTLB secondary registers(arm) 157h>≡ (683c)
/*
 * CpTLB Secondary (CRm) registers and opcode2 fields.
 */
#define CpTLBinvu    7      /* unified */
```

```
<type CpTLB opcode2(arm) 157i>≡ (683c)
#define CpTLBinv     0      /* invalidate all */
#define CpTLBinvse   1      /* invalidate single entry */
```

```

⟨function mmul2empty(arm) 158a⟩≡ (711a)
static void
mmul2empty(Proc* proc, bool clear)
{
    PTE *l1;
    Page **l2, *page;

    l1 = cpu->mmul1;
    l2 = &proc->mmul2;
    for(page = *l2; page != nil; page = page->next){
        if(clear)
            memset(UINT2PTR(page->va), 0, BY2PG);
        l1[page->daddr] = Fault;
        l2 = &page->next;
    }
    ⟨mmul2empty() remember free pages(arm) 158c⟩
}

⟨Proc Arch memory fields(arm) 158b⟩+≡ (50) ◁157c
// list<ref<Page> > (next = Page.next)
Page* mmul2cache; /* free mmu pages */

⟨mmul2empty() remember free pages(arm) 158c⟩≡ (158a)
*l2 = proc->mmul2cache;
proc->mmul2cache = proc->mmul2;
proc->mmul2 = nil;

⟨arch_mmurelease() free l2 page cache(arm) 158d⟩≡ (157f)
for(page = proc->mmul2cache; page != nil; page = next){
    next = page->next;
    if(--page->ref)
        panic("mmurelease: page->ref %d", page->ref);
    pagechainhead(page);
}
if(proc->mmul2cache && palloc.freememr.p)
    wakeup(&palloc.freememr);
proc->mmul2cache = nil;

⟨function mmul1empty(arm) 158e⟩≡ (711a)
static void
mmul1empty(void)
{
    memset(&cpu->mmul1[L1lo], 0, (L1hi - L1lo)*sizeof(PTE));
}

```

When the pager modifies a process's page tables (e.g., swapping out a page), it sets `newtlb` to signal that the process must flush its TLB entries next time it runs. `arch_flushmmu()` forces an immediate flush:

```

⟨Proc memory fields 158f⟩+≡ (49f) ◁98a 186a▷
bool newtlb; /* Pager has changed my pte's, I must flush */

⟨function arch_flushmmu(arm) 158g⟩≡ (711a)
void
arch_flushmmu(void)
{
    int s;

    s = arch_splhi();
    up->newtlb = true;
    arch_mmuswitch(up);
    arch_splx(s);
}

```

8.4 Page fault

This is where interrupt handling meets memory management. When a data abort or prefetch abort reaches `arch__trap()`, it calls `faultarm()` with the faulting address. The call chain is:

```
faultarm(ARM) -> fault(portable) -> fixfault(portable)
                                     -> arch_putmmu(ARM)
```

The ARM-specific code extracts the faulting address and determines read vs. write; the portable code decides what to do about it.

8.4.1 `faultarm()` (ARM)

`faultarm()` is the ARM-specific wrapper: it calls the portable `fault()` function with the faulting address. If `fault()` returns an error in user mode, a note is posted (the process can handle it or will be killed); if it is a kernel fault, the system panics.

```
<function faultarm(arm) 159>≡ (712c)
/*
 * called by trap to handle access faults
 */
static void
faultarm(Ureg *ureg, virt_addr va, bool user, bool read)
{
    bool insyscall;
    int n;
    char buf[ERRMAX];

    if(up == nil) {
        //dumpregs(ureg);
        panic("fault: nil up in faultarm, pc %#p accessing %#p", ureg->pc, va);
    }
    insyscall = up->insyscall;
    up->insyscall = true; // ???
    <faultarm() if debug 579c>
    n = fault(va, read); // portable code

    if(n < 0){
        if(!user){
            dumpregs(ureg);
            panic("fault: kernel accessing %#p", va);
        }
        /* don't dump registers; programs suicide all the time */
        snprintf(buf, sizeof buf, "sys: trap: fault %s va=%#p",
            read? "read": "write", va);
        postnote(up, 1, buf, NDebug);
    }
    up->insyscall = insyscall;
}
```

8.4.2 `fault()`

The portable `fault()` function finds which segment contains the faulting address (via `seg()`^{98b}) and calls `fixfault()` to resolve it. The `for(;;)` loop handles the case where `fixfault()` returns an error because the

segment was invalidated while waiting for I/O—it simply retries.

```
⟨function fault 160a⟩≡ (663b)
int
fault(virt_addr addr, bool read)
{
    Segment *s;
    char *sps;

    if(up == nil)
        panic("fault: nil up");
    if(up->nlocks.ref)
        print("fault: addr %#p: nlocks %ld\n", addr, up->nlocks.ref);

    sps = up->psstate;
    up->psstate = "Fault";
    arch_spllo();

    cpu->pfault++;
    for(;;) {
        s = seg(up, addr, /*dolock*/true); /* leaves s->lk qlocked if seg != nil */
        if(s == nil) {
            up->psstate = sps;
            return -1;
        }
        if(!read && (s->type&SG_RDONLY)) {
            qunlock(&s->lk);
            up->psstate = sps;
            return -1;
        }

        if(fixfault(s, addr, read, /*putmmu*/true) == 0) /* qunlocks s->lk */
            break;

        // else? try again?
    }

    up->psstate = sps;
    return 0;
}
```

8.4.3 fixfault()

`fixfault()` is the heart of virtual memory. It walks the software segment tables to find (or create) the Page for the faulting address, then calls `arch_putmmu()` to install the mapping in the hardware page table. What it does depends on the segment type:

- SG_TEXT: demand-load from executable (read-only)
- SG_BSS: allocate a zero-filled page
- SG_STACK: allocate a zero-filled page
- SG_DATA: demand-load, or copy-on-write if shared

The PTEVALID, PTERONLY, and PTEWRITE flags are software-defined bits that `fixfault()` passes to `arch_putmmu()`, which translates them into the hardware AP bits (Uro or Urw):

```
⟨type PTExxx 160b⟩≡ (658)
#define PTEVALID    (1<<0)
#define PTERONLY    0
#define PTEWRITE    (1<<1)
⟨PTExxx other cases 496b⟩
```

```

⟨function fixfault 161⟩≡ (663b)
    errorneg1
    fixfault(Segment *s, virt_addr addr, bool read, bool doputmmu)
    {
        int type;
        int ref;
        Pagetable **pde, *pt;
        Page **pte, *lkp, *new;
        ulong mmuppte = nilptr;
        ulong soff;

        addr &= ~(BY2PG-1);
        soff = addr - s->base;

        // walk
        pde = &s->pagedir[soff/PAGETABMAPMEM];
        if(*pde == nil)
            *pde = ptalloc();
        pt = *pde;
        pte = &pt->pagetab[(soff&(PAGETABMAPMEM-1))/BY2PG];
        if(pte < pt->first)
            pt->first = pte;
        if(pte > pt->last)
            pt->last = pte;

        type = s->type&SG_TYPE;

        switch(type) {
        case SG_TEXT:                /* Demand load */
            ⟨fixfault() page in for SG_TEXT pte if pagedout 171e⟩

            mmuppte = PPN((*pte)->pa) | PTERONLY|PTEVALID;
            (*pte)->modref = PG_REF;
            break;

        case SG_BSS:
        case SG_SHARED:              /* Zero fill on demand */
        case SG_STACK:
            if(*pte == nil) {
                new = newpage(true, &s, addr); // true so clear! zeroed BSS & heap!
                if(s == nil) ///? when can be nil at exit?
                    return ERROR_NEG1;
                *pte = new;
            }
            goto common;

        case SG_DATA:
        common:                      /* Demand load/pagein/copy on write */
            ⟨fixfault() page in for SG_DATA or swapin (SG_BSS, etc) pte if pagedout 171f⟩

            ⟨fixfault() if read and copy on write, adjust mmuppte and break 164f⟩
            // else

            lkp = *pte;

            lock(lkp);

            if(lkp->image == &swapimage)
                ref = lkp->ref + swapcount(lkp->daddr);
            else

```

```

    ref = lkp->ref;
    <fixfault() if one ref and page has an image 174a>
    unlock(lkp);

    <fixfault() if write and more than one ref then copy page 165a>

    mmupte = PPN((*pte)->pa) | PTEWRITE | PTEVALID;
    (*pte)->modref = PG_MOD|PG_REF;
    break;

    <fixfault() SG_PHYSICAL case 496a>

default:
    panic("fault");
    break;
}
qunlock(&s->lk);

if(doputmmu)
    arch_putmmu(addr, mmupte, *pte);

return OK_0;
}

```

8.4.4 arch_putmmu() (ARM)

arch_putmmu() installs a mapping in the hardware page table. If the L1 entry for this virtual address is Fault (no L2 table yet), it allocates a page for the L2 table (reusing one from the per-process mmul2cache if available). It then writes the L2 entry with the physical page address and the appropriate permission bits, and invalidates the TLB entry for this address so the MMU picks up the new mapping.

```

<function arch_putmmu(arm) 162>≡ (711a)
void
arch_putmmu(virt_addr va, phys_addr pa, Page* page)
{
    int x;
    Page *pg;
    PTE *l1, *pte;

    x = L1X(va);
    l1 = &cpu->mmul1[x];
    if(*l1 == Fault){
        /* wasteful - l2 pages only have 256 entries - fix */
        <arch_putmmu() if mmul2cache not empty(arm) 163a>
    } else{
        /* auxpg since we don't need much? memset if so */
        pg = newpage(true, nil, 0);
        pg->va = VA(arch_kmap(pg));
    }
    pg->daddr = x;

    //add_list(pg, up->mmul2)
    pg->next = up->mmul2;
    up->mmul2 = pg;

    <arch_putmmu() write back cache before fixing va fault(arm) 470g>

    *l1 = PPN(pg->pa) | Dom0 | Coarse;
    <arch_putmmu() write back cache after changing pde(arm) 470e>
}

```

```

    <arch_putmmu() maintain mmul1lo and mmul1hi(arm) 164a>
}
pte = UINT2PTR(KADDR(PPN(*l1)));

/* protection bits are
 * PTERONLY|PTEVALID;
 * PTEWRITE|PTEVALID;
 * PTEWRITE|PTEUNCACHED|PTEVALID;
 */
x = Small;
<arch_putmmu() if PTEUNCACHED, do not change x(arm) 496c>
else
    x |= L2ptedramattrs;

if(pa & PTEWRITE)
    x |= L2AP(Urw);
else
    x |= L2AP(Uro);

pte[L2X(va)] = PPN(pa)|x;
<arch_putmmu() write back cache after changing pte(arm) 470f>

/* clear out the current entry */
mmuinvalidateaddr(PPN(va));

/* write back dirty entries - we need this because the pio() in
 * fault.c is writing via a different virt addr and won't clean
 * its changes out of the dcache. Page coloring doesn't work
 * on this mmu because the virtual cache is set associative
 * rather than direct mapped.
 */
<arch_putmmu() if PG_TXTFLUSH, invalidate cache(arm) 493g>
arch_checkmmu(va, PPN(pa));
}

<arch_putmmu() if mmul2cache not empty(arm) 163a>≡ (162)
if(up->mmul2cache != nil){
    pg = up->mmul2cache;
    up->mmul2cache = pg->next;
    memset(UINT2PTR(pg->va), 0, BY2PG);
}

<function mmuinvalidateaddr(arm) 163b>≡ (711b)
/*
 * mmuinvalidateaddr(va)
 * invalidate tlb entry for virtual page address va, ASID 0
 */
TEXT mmuinvalidateaddr(SB), 1, $-4
    MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinvs
    BARRIERS
    RET

<Cpu Arch mmu fields(arm) 163c>+≡ (48e) <156k 200f>
// index in mmul1
int mmul1lo;
int mmul1hi;

```

```

⟨arch_putmmu() maintain mmul1lo and mmul1hi(arm) 164a⟩≡ (162)
    if(x >= cpu->mmul1lo && x < cpu->mmul1hi){
        if(x+1 - cpu->mmul1lo < cpu->mmul1hi - x)
            cpu->mmul1lo = x+1;
        else
            cpu->mmul1hi = x;
    }

```

```

⟨function arch_checkmmu(arm) 164b⟩≡ (711a)
    void
    arch_checkmmu(virt_addr va, phys_addr pa)
    {
        USED(va);
        USED(pa);
    }

```

`arch_kmap()` makes a physical page accessible to the kernel. On ARM with less than 2 GiB RAM, this is trivial: add `KZERO` to get the virtual address. `arch_kunmap()` flushes the cache for that page:

```

⟨function arch_kmap(arm) 164c⟩≡ (711a)
    Arch_KMap*
    arch_kmap(Page *p) {
        return (Arch_KMap*)((p)->pa|KZERO);
    }

```

```

⟨function arch_kunmap(arm) 164d⟩≡ (711a)
    void
    arch_kunmap(Arch_KMap *k)
    {
        ⟨arch_kunmap() write back cache for page(arm) 470i⟩
    }

```

8.5 Copy on write

When `rfork` duplicates a data segment, it does not immediately copy all pages. Instead, both parent and child share the same physical pages, marked read-only. When either process writes to a shared page, the write triggers a permission fault. `fixfault()` sees that the page has `ref > 1` (multiple users), allocates a fresh page, copies the content, and gives the writer the private copy with write permission. This is copy on write (COW)—it avoids copying pages that are never modified. There is a subtlety: if the segment has only one user (`s->ref == 1`) and the access is a read, the page can be mapped read-only without copying, even if `conf.copymode` would normally trigger a copy. This optimization is controlled by `conf.copymode`:

```

⟨Conf other fields 164e⟩≡ (46a) 178c▷
    int copymode; /* 0 is copy on write, 1 is copy on reference */

```

```

⟨fixfault() if read and copy on write, adjust mmuppte and break 164f⟩≡ (161)
    /*
     * It's only possible to copy on write if
     * we're the only user of the segment.
     */
    if(read && conf.copymode == false && s->ref == 1) {
        mmuppte = PPN((*pte)->pa)|PTERONLY|PTEVALID;
        (*pte)->modref |= PG_REF;
        break;
    }

```

```

⟨fixfault() if write and more than one ref then copy page 165a⟩≡ (161)
    if(ref > 1){
        new = newpage(false, &s, addr);
        if(s == nil)
            return -1;
        *pte = new;
        ccopypage(lkp, *pte);
        putpage(lkp); //why??
    }

```

ccopypage() performs the actual page copy via temporary kernel mappings (arch_kmap):

```

⟨function ccopypage 165b⟩≡ (664)
    void
    ccopypage(Page *f, Page *t)
    {
        Arch_KMap *ks, *kd;

        ks = arch_kmap(f);
        kd = arch_kmap(t);
        memmove((void*)VA(kd), (void*)VA(ks), BY2PG);
        arch_kunmap(ks);
        arch_kunmap(kd);
    }

```

8.6 Page cache

The page cache avoids redundant disk reads. When the kernel loads a page from an executable (demand loading) or from the swap file, it registers the page in a hash table keyed by (image, disk address). If the same page is needed again—e.g., two instances of rc sharing the same text—lookpage() finds it in the cache instead of reading from disk. Pages in the cache can be simultaneously in use (referenced by a segment) or free (on palloc’s free list but still in the hash table, ready to be reclaimed if needed before being recycled).

Each Page can be associated with a KImage (an open executable or the swap file) and a disk address (daddr). Together, these form the cache key:

```

⟨Page other fields 165c⟩≡ (87) 187c▷
    // option<ref<Kimage>>
    KImage *image; /* Associated binary image or swap image */
    ulong daddr; /* Disc address on image */

```

```

⟨function pghash 165d⟩≡ (658)
    enum
    {
        PGHLOG = 9, // 2^9 = 512
        PGHSIZE = 1<<PGHLOG, /* Page hash for image lookup */
    };
    #define pghash(daddr) palloc.hash[(daddr>>PGSHIFT)&(PGHSIZE-1)]

```

```

⟨Palloc other fields 165e⟩≡ (88b)
    // hash<Page.daddr, ref<Page>> (next = Page.hash)>
    Page *hash[PGHSIZE];
    Lock hashlock;

```

```

⟨Page extra fields 165f⟩+≡ (87) <88c
    // hash<daddr, ref<Page>> Palloc.hash
    Page *hash; /* Image hash chains */

```

`cachepage()` adds a page to the cache after loading it from disk. `lookpage()` searches the cache before attempting a disk read. `uncachepage()` removes a page when it is recycled for a different purpose:

```

<function cachepage 166a>≡ (664)
void
cachepage(Page *p, KImage *i)
{
    Page **l; // list

    /* If this ever happens it should be fixed by calling
     * uncachepage instead of panic. I think there is a race
     * with pio in which this can happen. Calling uncachepage is
     * correct - I just wanted to see if we got here.
     */
    if(p->image)
        panic("cachepage");

    incref(i);
    lock(&palloc.hashlock);
    p->image = i;
    // add_hash(palloc.hash, p->daddr, p)
    l = &pghash(p->daddr);
    p->hash = *l;
    *l = p;
    unlock(&palloc.hashlock);
}

```

`lookpage()` searches the hash table for a cached page. If found and the page was free (ref was 0), it unchains it from the free list—the page is now in use again without any disk I/O:

```

<function lookpage 166b>≡ (664)
Page *
lookpage(KImage *i, ulong daddr)
{
    Page *pg;

    lock(&palloc.hashlock);
    for(pg = pghash(daddr); pg; pg = pg->hash) {
        if(pg->image == i && pg->daddr == daddr) {
            unlock(&palloc.hashlock);

            lock(&palloc);
            lock(pg);
            if(pg->image != i || pg->daddr != daddr) { // race, not the one anymore
                unlock(pg);
                unlock(&palloc);
                return nil;
            }
            if(++pg->ref == 1)
                pageunchain(pg);
            unlock(&palloc);
            unlock(pg);

            return pg;
        }
    }
    unlock(&palloc.hashlock);

    return nil;
}

```

<putpage if p has an image 167a>≡ (90d)

```
if(p->image && p->image != &swapimage)
    pagechaintail(p);
```

<newpage() uncacheimage 167b>≡ (89c)

```
uncacheimage(p);
```

<function uncacheimage 167c>≡ (664)

```
/* Always called with a locked page */
void
uncacheimage(Page *p)
{
    Page **l, *f;

    if(p->image == nil)
        return;

    lock(&palloc.hashlock);
    // remove_hash(palloc.hash, p->daddr, p)
    l = &pghash(p->daddr);
    for(f = *l; f; f = f->hash) {
        if(f == p) {
            *l = p->hash;
            break;
        }
        l = &f->hash;
    }
    unlock(&palloc.hashlock);
    putimage(p->image);
    p->image = nil;
    p->daddr = 0;
}
```

<function cachedel 167d>≡ (664)

```
void
cachedel(KImage *i, ulong daddr)
{
    Page *f, **l;

    lock(&palloc.hashlock);
    l = &pghash(daddr);
    for(f = *l; f; f = f->hash) {
        if(f->image == i && f->daddr == daddr) {
            lock(f);
            // can have a race? things could have changed, so rested under lock
            if(f->image == i && f->daddr == daddr){
                *l = f->hash;
                putimage(f->image); // ~ decref
                f->image = nil;
                f->daddr = 0;
            }
            unlock(f);
            break;
        }
        l = &f->hash;
    }
    unlock(&palloc.hashlock);
}
```


8.7 Demand loading

When `sysexec()` creates the text and data segments, it does not load any pages from disk. Instead, the segment tables are left empty (`nil` page entries), and the segment records which file to read from (`Segment.image`, `Segment.fstart`, `Segment.flen`). When the process first touches a page, `fixfault()` sees `*pte == nil`, calls `pio()` to load the page from the executable on disk, and installs it. This is demand loading—pages are loaded only when actually accessed. Demand loading also enables text sharing: if two processes run the same executable, `attachimage()` finds the existing `KImage` in the image cache and shares its segment. The pages loaded by the first process are found via `lookuppage()` by the second, avoiding duplicate reads.

8.7.1 Kimage and the image cache

A `KImage` wraps a `Chan` (open file handle) to an executable binary or the swap file. It also holds a back pointer to the `Segment` that uses it, enabling text sharing: when a second process execs the same binary, `attachimage()` finds the existing `KImage` and shares its segment (and thus its already-loaded pages):

```
<struct KImage 168a>≡ (658)
// a KImage is essentially a channel to an executable or swapfile
struct KImage
{
    Chan *c; /* channel to text file */
    bool notext; /* no file associated */ // for swapfile
    <KImage other fields 170a>

    // extra
    Ref;
    <KImage extra fields 168c>
};

<struct Imagealloc 168b>≡ (658)
// Image allocator (internal to segment.c, but important so here, singleton)
struct Imagealloc
{
    // hash<qid.path, ref<KImage>> (next = KImage.hash)
    KImage *hash[IHASHSIZE];

    // list<ref<KImage>> (next = KImage.next)
    KImage *free; // originally xalloc'ed in imageinit() (conf.nimage)
    QLock ireclaim; /* mutex on reclaiming free images */

    <Imagealloc other fields 176c>

    // extra
    Lock;
};

<KImage extra fields 168c>≡ (168a) 169b▷
// list<ref<KImage>> of Imagealloc.free
KImage *next; /* Free list */
// hash<qid.path, ref<KImage>> Imagealloc.hash
KImage *hash; /* Qid hash chains */

<function ihash 168d>≡ (658)
#define IHASHSIZE 64
// actually internal to page.c, but important so here
#define ihash(qidpath) imagealloc.hash[qidpath%IHASHSIZE]

<global imagealloc 168e>≡ (665)
static struct Imagealloc imagealloc;
```

8.7.2 Segment and KImage

Each segment that supports demand loading has a pointer to its KImage and records the file offset (**fstart**) and length (**flen**) of the segment's data within the executable:

```
⟨Segment other fields 169a⟩+≡ (92c) <96d 170b>
    KImage *image; /* text in file attached to this segment */

⟨KImage extra fields 169b⟩+≡ (168a) <168c
    // option<ref<Segment>>?
    Segment *s; /* TEXT segment for image if running */

⟨constructor attachimage 169c⟩≡ (665)
    KImage*
    attachimage(int type, Chan *c, virt_addr base, ulong len)
    {
        KImage *img, **l;

        /* reclaim any free channels from reclaimed segments */
        if(imagealloc.nfreechan)
            imagechanreclaim();

        lock(&imagealloc);

        /*
         * Search the image cache for remains of the text from a previous
         * or currently running incarnation
         */
        for(img = ihash(c->qid.path); img; img = img->hash) {
            if(c->qid.path == img->qid.path) {
                lock(img);
                if(eqqid(c->qid, img->qid) &&
                    eqqid(c->mqid, img->mqid) &&
                    c->mchan == img->mchan &&
                    c->type == img->type) {
                    goto found;
                }
                unlock(img);
            }
        }

        /*
         * imagereclaim dumps pages from the free list which are cached by image
         * structures. This should free some image structures.
         */
        while(!(img = imagealloc.free)) {
            unlock(&imagealloc);
            imagereclaim();
            sched();
            lock(&imagealloc);
        }

        imagealloc.free = img->next;

        lock(img);
        incref(c);
        img->c = c;
        img->type = c->type;
        img->qid = c->qid;
        img->mqid = c->mqid;
        img->mchan = c->mchan;
```

```

//add_hash(imagealloc.hash, c->qid.path, img)
l = &ihash(c->qid.path);
img->hash = *l;
*l = img;

found:
    unlock(&imagealloc);

    if(img->s == nil) {
        /* Disaster after commit in exec */
        if(waserror()) {
            unlock(img);
            pexit(Enovmem, /*freemem*/true);
        }
        img->s = newseg(type, base, len);
        img->s->image = img;
        img->ref++;
        poperror();
    }
    else
        incref(img->s);

    return img;
}

```

⟨KImage other fields 170a⟩≡ (168a)

```

Qid    qid;      /* Qid for page cache coherence */
Chan   *mchan;
Qid     mqid;
ushort  type;     /* Device type of owning channel */

```

`attachimage()` is called by `sysexec()` to find or create a `KImage` for the executable. It searches the image hash table by `Qid`—if the same binary is already cached (from a previous or concurrent `exec`), the existing `KImage` and its shared segment are reused. Otherwise, a fresh `KImage` is allocated from the free list and a new segment is created.

⟨Segment other fields 170b⟩+≡ (92c) <169a 417a>

```

ulong fstart; /* start address in file for demand load */
ulong flen;   /* length of segment in file */

```

⟨sysexec() locals 170c⟩+≡ (119d) <124b 281e>

```

KImage *img;

```

⟨sysexec() get text segment ts via demand loading on tc 170d⟩≡ (122a)

```

/* attachimage returns a locked cache image */
img = attachimage(SG_TEXT|SG_RDONLY, tc, UTZERO, (t-UTZERO)>>PGSHIFT);
ts = img->s;
ts->fstart = 0;
ts->flen = sizeof(Exec)+text;
⟨sysexec() initialize other fields 493j⟩
unlock(img);

```

⟨sysexec() adjust data segment s for demand loading on tc 170e⟩≡ (122b)

```

/* Attached by hand */
incref(img);
s->image = img;
s->fstart = ts->fstart+ts->flen;
s->flen = data;
// data is also in binary

```

```

⟨dupseg() SG_DATA case, attach image to new segment n 171a⟩≡ (101c)
    incref(s->image); // how sure non nil? data always attached to an img
    n->image = s->image;
    n->fstart = s->fstart;
    n->flen = s->flen;

```

```

⟨dupseg() when SG_DATA, if Text segment 171b⟩≡ (101c)
    if(segno == TSEG){ // why not SG_TEXT then?
        poperror();
        qunlock(&s->lk);
        return data2txt(s); // ???
    }

```

```

⟨function data2txt 171c⟩≡ (665)
    Segment*
    data2txt(Segment *s)
    {
        Segment *ps;

        ps = newseg(SG_TEXT, s->base, s->size);
        ps->image = s->image;
        incref(ps->image);
        ps->fstart = s->fstart;
        ps->flen = s->flen;
        ⟨data2txt() initialize other segment fields 494b⟩

        return ps;
    }

```

8.7.3 Paging in

A page table entry is “paged out” if it is either `nil` (never loaded—demand loading) or marked as a swap address (`onswap`). In both cases, `pio()` must read data from disk before the page can be used:

```

⟨function pagedout 171d⟩≡ (658)
    #define pagedout(s) (((ulong)s)==0 || onswap(s))

```

```

⟨fixfault() page in for SG_TEXT pte if pagedout 171e⟩≡ (161)
    if(pagedout(*pte))
        pio(s, addr, soff, pte);

```

```

⟨fixfault() page in for SG_DATA or swapin (SG_BSS, etc) pte if pagedout 171f⟩≡ (161)
    if(pagedout(*pte))
        pio(s, addr, soff, pte);

```

`pio()` (“page I/O”) loads a page from disk. It first checks the page cache (`lookpage()`)—if the page is already cached, no I/O is needed. Otherwise, it allocates a fresh page, reads the data from the executable’s channel (or the swap file), and adds the page to the cache. The segment lock (`s->lk`) is released during the I/O to avoid blocking other processes; this creates a race (another process might fault on the same page), which is handled by rechecking after re-acquiring the lock.

```

⟨function pio 171g⟩≡ (663b)
    void
    pio(Segment *s, virt_addr addr, ulong soff, PageOrSwap **p)
    {
        Page *new;
        Arch_KMap *k;
        Chan *c;
        int n, ask;

```

```

char *kaddr;
ulong daddr; // disk address
Page *loadrec;

retry:
    loadrec = *p;

    if(loadrec == nil) { /* from a text/data image */
        daddr = s->fstart+soff;
        new = lookpage(s->image, daddr);
        if(new != nil) {
            *p = new;
            return;
        }

        c = s->image->c;
        ask = s->flen-soff;
        if(ask > BY2PG)
            ask = BY2PG;
    }else{ /* from a swap image */
        daddr = swapaddr(loadrec);
        new = lookpage(&swapimage, daddr);
        if(new != nil) {
            putswap(loadrec);
            *p = new;
            return;
        }
        c = swapimage.c;
        ask = BY2PG;
    }
    qunlock(&s->lk);

    new = newpage(false, nil, addr);
    k = arch_kmap(new);
    kaddr = (char*)VA(k);

    while(waserror()) {
        if(strcmp(up->errstr, EINTR) == 0)
            continue;
        arch_kunmap(k);
        putpage(new);
        faulterror(Eioload, c, false);
    }

    // reading the Page!! slow! which is why it's done without s->lk locked
    n = devtab[c->type]->read(c, kaddr, ask, daddr);

    if(n != ask)
        faulterror(Eioload, c, false);
    if(ask < BY2PG)
        memset(kaddr+ask, 0, BY2PG-ask);

    poperror();
    arch_kunmap(k);
    qlock(&s->lk);

    if(loadrec == nil) { /* This is demand load */
        /*
         * race, another proc may have gotten here first while
         * s->lk was unlocked

```

```

        */
        if(*p == nil) {
            new->daddr = daddr;
            cachepage(new, s->image);
            *p = new;
        }
        else
            putpage(new);
    }else{ /* This is paged out */
        /*
         * race, another proc may have gotten here first
         * (and the pager may have run on that page) while
         * s->lk was unlocked
         */
        if(*p != loadrec){
            if(!pagedout(*p)){
                /* another process did it for me */
                putpage(new);
                goto done; // ~ return, just need adjust cachectl before
            } else {
                /* another process and the pager got in */
                putpage(new);
                goto retry;
            }
        }

        new->daddr = daddr;
        cachepage(new, &swapimage);
        *p = new;
        putswap(loadrec);
    }
done:
<pio() set cachectl if flushme segment 494f>
}

<function faulterror 173>≡ (663b)
static void
faulterror(char *s, Chan *c, bool freemem)
{
    char buf[ERRMAX];

    if(c && c->path){
        snprintf(buf, sizeof buf, "%s accessing %s: %s", s, c->path->s, up->errstr);
        s = buf;
    }
    if(up->nerrlab) {
        postnote(up, 1, s, NDebug);
        error(s);
    }
    pexit(s, freemem);
}

```

8.7.4 duppage()

`duppage()` handles a subtle case in copy-on-write. When a page has `ref == 1` but is cached (has an `image`), the page is about to be modified—but the cache still holds a reference to the original content. `duppage()` makes a copy of the page for the cache, so the original can be safely modified. This preserves the invariant that cached

pages always reflect their disk content.

```
<fixfault() if one ref and page has an image 174a>≡ (161)
    if(ref == 1 && lkp->image){
        /* save a copy of the original for the image cache */
        duppage(lkp);
        ref = lkp->ref;
    }
```

```
<global dupretries 174b>≡ (664)
    static int dupretries = 15000;
```

```
<function duppage 174c>≡ (664)
    /* Always call with p locked */
    int
    duppage(Page *p)
    {
        Page *np;
        int retries;
        int color;

        retries = 0;
    retry:

        if(retries++ > dupretries){
            print("duppage %d, up %p\n", retries, up);
            dupretries += 100;
            if(dupretries > 100000)
                panic("duppage\n");
            uncacheimage(p);
            return 1;
        }

        /* don't dup pages with no image */
        if(p->ref == 0 || p->image == nil || p->image->notext)
            return 0;

        /*
         * normal lock ordering is to call
         * lock(&palloc) before lock(p).
         * To avoid deadlock, we have to drop
         * our locks and try again.
         */
        if(!canlock(&palloc)){
            unlock(p);
            if(up)
                sched();
            lock(p);
            goto retry;
        }

        /* No freelist cache when memory is very low */
        if(palloc.freecount < swappalloc.highwater) {
            unlock(&palloc);
            uncacheimage(p);
            return 1;
        }

        color = getpgcolor(p->va);
        for(np = palloc.head; np; np = np->next)
```

```

    if(np->color == color)
        break;

/* No page of the correct color */
if(np == 0) {
    unlock(&palloc);
    uncache_page(p);
    return 1;
}

//when no color: np = palloc.head;

    page_unchain(np);
    page_chain_tail(np);
/*
* XXX - here's a bug? - np is on the freelist but it's not really free.
* when we unlock palloc someone else can come in, decide to
* use np, and then try to lock it. they succeed after we've
* run copy_page and cache_page and unlock(np). then what?
* they call page_unchain before locking(np), so it's removed
* from the freelist, but still in the cache because of
* cache_page below. if someone else looks in the cache
* before they remove it, the page will have a nonzero ref
* once they finally lock(np).
*/
    lock(np);
    unlock(&palloc);

/* Cache the new version */
    uncache_page(np);
    np->va = p->va;
    np->daddr = p->daddr;
    copy_page(p, np);
    cache_page(np, p->image);
    unlock(np);
    uncache_page(p);

    return 0;
}

```

8.7.5 Collecting images

When a segment is freed (`putseg()`) and it was the last reference, the `KImage` is detached (its `s` pointer set to `nil`) and its reference count decremented. If the `KImage` itself reaches zero references, it is removed from the image hash table and returned to the free list. Channel closing is deferred to avoid blocking while holding spin locks.

$\langle \text{putseg}() \text{ other locals } 175a \rangle \equiv$ (96b)

```

    KImage *img;

```

$\langle \text{putseg}() \text{ if } s \text{ has an image } 175b \rangle \equiv$ (96b)

```

    img = s->image;
    if(img != nil) {
        lock(img);
        lock(s);
        if(img->s == s && s->ref == 1) // race?
            img->s = nil;
        unlock(img);
    }

```


<putseg() if s had an image 176a>≡ (96b)

```
if(img)
    putimage(img);
```

<destructor putimage 176b>≡ (665)

```
void
putimage(KImage *img)
{
    Chan *c, **cp;
    KImage *f, **l;

    if(img->notext)
        return;

    lock(img);
    if(--img->ref == 0) {
        l = &ihash(img->qid.path);
        mkqid(&img->qid, ~0, ~0, QTFILE);
        unlock(img);
        c = img->c;

        lock(&imagealloc);
        //remove_hash(imagealloc.hash, img)
        for(f = *l; f; f = f->hash) {
            if(f == img) {
                *l = img->hash;
                break;
            }
            l = &f->hash;
        }

        //add_list(imagealloc.free, img)
        img->next = imagealloc.free;
        imagealloc.free = img;

        /* defer freeing channel till we're out of spin lock's */

        if(imagealloc.nfreechan == imagealloc.szfreechan){
            //realloc(imagealloc.freechan, szfreechan+NFREECHAN)
            imagealloc.szfreechan += NFREECHAN;
            cp = malloc(imagealloc.szfreechan*sizeof(Chan*));
            if(cp == nil)
                panic("putimage");
            memmove(cp, imagealloc.freechan, imagealloc.nfreechan*sizeof(Chan*));
            free(imagealloc.freechan);
            imagealloc.freechan = cp;
        }

        imagealloc.freechan[imagealloc.nfreechan++] = c;
        unlock(&imagealloc);

        return;
    }
    unlock(img);
}
```

<Imagealloc other fields 176c>≡ (168b)

```
Chan **freechan; /* free image channels */
int nfreechan; /* number of free channels */
int szfreechan; /* size of freechan array */
QLock fcreclaim; /* mutex on reclaiming free channels */
```

<function imagechanreclaim 177a>≡ (665)

```
/*
 * since close can block, this has to be called outside of
 * spin locks.
 */
static void
imagechanreclaim(void)
{
    Chan *c;

    /* Somebody is already cleaning the image chans */
    if(!canqlock(&imagealloc.fcreclaim))
        return;

    /*
     * We don't have to recheck that nfreechan > 0 after we
     * acquire the lock, because we're the only ones who decrement
     * it (the other lock contender increments it), and there's only
     * one of us thanks to the qlock above.
     */
    while(imagealloc.nfreechan > 0){
        lock(&imagealloc);
        imagealloc.nfreechan--;
        c = imagealloc.freechan[imagealloc.nfreechan];
        unlock(&imagealloc);
        cclose(c);
    }

    qunlock(&imagealloc.fcreclaim);
}
```

<function imagereclaim 177b>≡ (665)

```
static void
imagereclaim(void)
{
    int n;
    Page *p;
    uvlong ticks; // fastticks

    irstats.calls++;
    /* Somebody is already cleaning the page cache */
    if(!canqlock(&imagealloc.ireclaim))
        return;

    lock(&palloc);
    ticks = arch_fastticks(nil);
    n = 0;
    /*
     * All the pages with images backing them are at the
     * end of the list (see putpage) so start there and work
     * backward.
     */
    for(p = palloc.tail; p && p->image && n<1000; p = p->prev) {
        if(p->ref == 0 && canlock(p)) {
            if(p->ref == 0) {
                n++;
                uncacheimage(p); // will call putimage()
            }
            unlock(p);
        }
    }
}
```

```

}
ticks = arch_fastticks(nil) - ticks;
unlock(&palloc);
irstats.loops++;
irstats.ticks += ticks;
if(ticks > irstats.maxt)
    irstats.maxt = ticks;
//print("T%lld+", ticks);
qunlock(&imagealloc.ireclaim);
}

```

8.8 Swapping

When physical memory runs low, the kernel must evict pages to disk to make room. This is swapping (or paging out). The evicted page's content is written to a swap file, and the page table entry is replaced with the swap address. When the process touches that page again, a page fault triggers `pio()` to read it back. The swap subsystem mirrors the page allocator: where `Palloc` manages physical pages, `Swapalloc` manages slots in the swap file. Each slot is one page (4 KiB), and the `swmap` array is a reference count for each slot (because forked processes can share swap entries via copy-on-write).

8.8.1 Swapalloc

```

⟨struct Swapalloc 178a⟩≡ (658)
// Swap allocator (singleton)
struct Swapalloc
{
    // array<byte> xalloc'ed in swapinit()
    // each idx represents a chunk of swapimage, each value a ref count
    byte* swmap; /* Base of swap map in memory */

    int free; /* currently free swap pages */

    // ref<byte> in swmap
    byte* alloc; /* Round robin allocator */
    // ref<byte> in swmap
    byte* top; /* Top of swap map */

    // ref<byte> in swmap
    byte* last; /* Speed swap allocation */

    ulong highwater; /* Pager start threshold */ // = 5% conf.upages
    ulong headroom; /* Space pager frees under highwater */ // = 1.25*hw

    //extra
    Lock; /* Free map lock */
    Rendez r; /* Pager kproc idle sleep */ // needpages()
};

⟨global swapalloc 178b⟩≡ (660a)
// Swap allocator
struct Swapalloc swapalloc;

⟨Conf other fields 178c⟩+≡ (46a) <164e 182a>
ulong nswap; /* number of swap pages */

⟨global swapimage 178d⟩≡ (660a)
KImage swapimage;

```

```

⟨function newswap 179a⟩≡ (666)
    ulong
    newswap(void)
    {
        byte *look;

        lock(&swapalloc);

        if(swapalloc.free == 0){
            unlock(&swapalloc);
            return ~0; //???
        }

        look = memchr(swapalloc.last, 0, swapalloc.top-swapalloc.last);
        if(look == nil)
            panic("inconsistent swap"); // swapalloc.free != 0, should find a page

        *look = 1;
        swapalloc.last = look;
        swapalloc.free--;
        unlock(&swapalloc);
        return (look-swapalloc.swmap) * BY2PG; // offset in swapfile
    }

```

8.8.2 /dev/swap

Reading /dev/swap gives a summary of all the memory subsystems we have seen: total physical memory, page size, kernel vs. user pages, swap usage, and malloc pool sizes. It is a convenient diagnostic:

```

⟨consread() cases 179b⟩≡ (345c) 231c▷
    case Qswap:
        snprintf(tmp, sizeof tmp,
            "%lud memory\n"
            "%d pagesize\n"
            "%lud kernel\n"
            "%lud/%lud user\n"
            "%lud/%lud swap\n"
            "%lud/%lud kernel malloc\n"
            "%lud/%lud kernel draw\n",
            conf.npage*BY2PG,
            BY2PG,
            conf.npage-conf.upages,
            palloc.user - palloc.freecount, palloc.user,
            conf.nswap - swapalloc.free, conf.nswap,
            mainmem->cursize, mainmem->maxsize,
            imagmem->cursize, imagmem->maxsize);

        return readstr((ulong)offset, buf, n, tmp);

```

8.8.3 PageOrSwap

How does a page table entry indicate “this page is on swap”? By a clever encoding trick: since `Page*` pointers are word-aligned, their lowest bit is always 0. Setting bit 0 (`PG_ONSWAP`) marks the value as a swap address instead of a `Page` pointer. The remaining bits encode the offset in the swap file. This is the `PageOrSwap` type—a union disguised as a pointer:

```

⟨constant PG_ONSWAP 179c⟩≡ (658)
    #define PG_ONSWAP 1

```

```

⟨type PageOrSwap 180a⟩≡ (658)
    typedef Page PageOrSwap;

⟨function onswap 180b⟩≡ (658)
    #define onswap(s) (((kern_addr)s)&PG_ONSWAP)

⟨function putswap 180c⟩≡ (666)
    void
    putswap(Page *p)
    {
        byte *idx;

        lock(&swapalloc);
        idx = &swapalloc.swmap[((ulong)p)/BY2PG];
        if(--(*idx) == 0) {
            swapalloc.free++;
            if(idx < swapalloc.last)
                swapalloc.last = idx;
        }
        if(*idx >= 254)
            panic("putswap %#p == %ud", p, *idx);
        unlock(&swapalloc);
    }

⟨putpage if p is a swap address 180d⟩≡ (90d)
    if(onswap(p)) {
        putswap(p);
        return;
    }

⟨mfreeeseg if pg is a swap address 180e⟩≡ (104b)
    if(onswap(pg))
        putswap(pg);

⟨function dupswap 180f⟩≡ (666)
    void
    dupswap(Page *p)
    {
        lock(&swapalloc);
        if(++swapalloc.swmap[((ulong)p)/BY2PG] == 0)
            panic("dupswap");
        unlock(&swapalloc);
    }

⟨ptcpy() if src is a swap page 180g⟩≡ (94d)
    if(onswap(*src))
        dupswap(*src);

⟨function swapcount 180h⟩≡ (666)
    int
    swapcount(ulong daddr)
    {
        return swapalloc.swmap[daddr/BY2PG];
    }

```

8.8.4 Swapping plan

When the pager decides to evict a page, `pagepte()` handles the details per segment type. For text pages, eviction is free: the page is simply discarded (it can be demand-loaded again from the executable). For data, bss, and stack pages, the content must be written to the swap file: `pagepte()` allocates a swap slot, caches the page under the swap image (so a concurrent fault can find it during the write), replaces the `Page*` in the page table with the swap address, and adds the page to `iolist` for the actual disk write.

```
(function pagepte 181)≡ (666)
static void
pagepte(int type, Page **pte)
{
    ulong daddr;
    Page *outp;

    outp = *pte;
    switch(type) {
    case SG_TEXT:                /* Revert to demand load */
        putpage(outp);
        *pte = nil;
        break;

    case SG_DATA:
    case SG_BSS:
    case SG_STACK:
    case SG_SHARED:
        /*
         * get a new swap address and clear any pages
         * referring to it from the cache
         */
        daddr = newswap();
        if(daddr == ~0)
            break; // return;
        cachedel(&swapimage, daddr);

        lock(outp);

        /* forget anything that it used to cache */
        uncachepte(outp);

        /*
         * incr the reference count to make sure it sticks around while
         * being written
         */
        outp->ref++;

        /*
         * enter it into the cache so that a fault happening
         * during the write will grab the page from the cache
         * rather than one partially written to the disk
         */
        outp->daddr = daddr;
        cachepte(outp, &swapimage);
        *pte = (Page*)(daddr|PG_ONSWAP); // turn a Page pte into a Swap pte!
        unlock(outp);

        /* Add page to IO transaction list */
        iolist[ioptr++] = outp;
        break;
    }
}
```

```

}

<Conf other fields 182a>+≡ (46a) <178c 241d>
    int nswppo;    /* max # of pageouts per segment pass */

<global iolist 182b>≡ (666)
    // array<option<ref<Page>>>, xalloc'ed in swapinit(), size = Conf.nswppo
    static Page    **iolist;

<global ioptr 182c>≡ (666)
    // index in iolist
    static int ioptr;

```

pageout() walks a segment's page tables looking for pages to evict. It uses an aging algorithm: each page records when it was last referenced (*gen*). Pages older than *genage* are candidates for eviction. Recently referenced pages (PG_REF set) get their generation reset, giving them a second chance.

```

<function pageout 182d>≡ (666)
    static void
    pageout(Proc *p, Segment *s)
    {
        int type, i, size;
        ulong age;
        Pagetable *pt;
        Page **pg, *entry;

        if(!canqlock(&s->lk))    /* We cannot afford to wait, we will surely deadlock */
            return;

        if(s->steal) {            /* Protected by /dev/proc */
            qunlock(&s->lk);
            return;
        }

        if(!canflush(p, s)) {    /* Able to invalidate all tlbs with references */
            qunlock(&s->lk);
            putseg(s);
            return;
        }

        if(waserror()) {
            qunlock(&s->lk);
            putseg(s);
            return;
        }

        /* Pass through the tables looking for memory pages to swap out */
        type = s->type&SG_TYPE;
        size = s->pagedirsize;
        for(i = 0; i < size; i++) {
            pt = s->pagedir[i];
            if(pt == nil)
                continue;
            for(pg = pt->first; pg < pt->last; pg++) {
                entry = *pg;
                if(pagedout(entry)) // already swapped, or nil
                    continue;

                if(entry->modref & PG_REF) {
                    entry->modref &= ~PG_REF;
                    entry->gen = genclock;
                }
            }
        }
    }

```

```

    }

    if(genclock < entry->gen)
        age = ~(entry->gen - genclock);
    else
        age = genclock - entry->gen;
    gensum += age;
    gencount++;
    if(age <= genage)
        continue;

    pagepte(type, pg);

    if(ioptr >= conf.nswppo)
        goto out;
}
}
out:
    poperror();
    qunlock(&s->lk);
    putseg(s);
}

```

8.8.5 The pager

The pager is a kernel process (started by `kproc()` during initialization) that runs when free memory drops below `swalloc.highwater`. When `newpage()` cannot find enough free pages, it calls `kickpager()` to wake the pager, which scans all processes' segments looking for old pages to evict. Once enough pages are freed, the pager goes back to sleep. This is the classic “page daemon” found in all UNIX-like systems.

```

<newpage() other locals 183a>+= (89c) <90a 493c>
    bool dontalloc;

```

```

<newpage() loop waiting freecount > highwater 183b>≡ (89c)
    for(;;) {
        if(palloc.freecount > swalloc.highwater)
            break;
        if(up->kp && palloc.freecount > 0)
            break;

        // in highwater, not so many free pages, need to wait

        unlock(&palloc);
        dontalloc = false;
        if(s && *s) {
            qunlock(&((*s)->lk));
            *s = nil; // !!
            dontalloc = true;
        }
        qlock(&palloc.pwait); /* Hold memory requesters here */

        while(waserror()) /* Ignore interrupts */
            ;

        kickpager();
        tsleep(&palloc.freememr, hasfreepages, 0, 1000);
    }

```



```

    poperror();
    qunlock(&palloc.pwait);

    /*
     * If called from fault and we lost the segment from
     * underneath don't waste time allocating and freeing
     * a page. Fault will call newpage again when it has
     * reacquired the segment locks
     */
    if(dontalloc)
        return nil;

    lock(&palloc);
}

⟨Palloc extra fields 184a⟩≡ (88b)
    Rendez freememr; /* Sleep for free mem */ // hasfreepages()
    QLock pwait; /* Queue of procs waiting for memory */

⟨putpage wakeup process waiting for freepage if any 184b⟩≡ (90d)
    if(palloc.freememr.p != nil)
        wakeup(&palloc.freememr);

⟨function hasfreepages 184c⟩≡ (664)
    int
    hasfreepages(void*)
    {
        return palloc.freecount >= swapalloc.highwater;
    }

⟨function kickpager 184d⟩≡ (666)
    void
    kickpager(void)
    {
        static bool started;

        if(started)
            wakeup(&swapalloc.r);
        else {
            kproc("kpager", pager, nil);
            started = true;
        }
    }

⟨function pager 184e⟩≡ (666)
    static void
    pager(void *junk)
    {
        int i;
        Segment *s;
        Proc *p, *ep;

        if(waserror())
            panic("pager: os error");

        p = proctab(0);
        ep = &p[conf.nproc];

loop:
        up->psstate = "Idle";

```

```

wakeup(&palloc.freememr);
sleep(&swapalloc.r, needpages, nil);

while(needpages(junk)) {
    if(swapimage.c) {
        p++;
        if(p >= ep){
            p = proctab(0);
            gentick();
        }

        if(p->state == Dead || p->noswap)
            continue;

        if(!canqlock(&p->seglock))
            continue;          /* process changing its segments */

        for(i = 0; i < NSEG; i++) {
            if(!needpages(junk)){
                qunlock(&p->seglock);
                goto loop;
            }

            if(s = p->seg[i]) {
                switch(s->type&SG_TYPE) {
                    case SG_TEXT:
                        pageout(p, s);
                        break;
                    case SG_DATA:
                    case SG_BSS:
                    case SG_STACK:
                    case SG_SHARED:
                        up->psstate = "Pageout";
                        pageout(p, s);
                        if(ioptr != 0) {
                            up->psstate = "I/O";
                            executeio();
                        }
                        break;
                    default:
                        break;
                }
            }
        }
        qunlock(&p->seglock);
    } else {
        print("out of memory\n");
        killbig("out of memory");
        freebroken();          /* can use the memory */

        /* Emulate the old system if no swap channel */
        if(!swapimage.c)
            tsleep(&up->sleepr, returnfalse, 0, 5000);
    }
}
goto loop;
}

```

⟨function needpages 185⟩≡
static bool

(666)

```

needpages(void*)
{
    return palloc.freecount < swapalloc.headroom;
}

⟨Proc memory fields 186a⟩+≡ (49f) <158f
    bool noswap; /* process is not swappable */

⟨newproc() set fields of p 186b⟩+≡ (108c) <137b 190d>
    p->noswap = false;

⟨sysrfork() when RFPROC propagate fields from up to p 186c⟩+≡ (112d) <141d 191c>
    p->noswap = up->noswap;

⟨procctlreq() CMnoswap case 186d⟩≡ (554b)
    case CMnoswap:
        p->noswap = true;
        break;

⟨function canflush 186e⟩≡ (666)
    static bool
    canflush(Proc *p, Segment *s)
    {
        int i;
        Proc *ep;

        lock(s);
        if(s->ref == 1) { /* Easy if we are the only user */
            s->ref++;
            unlock(s);
            return canpage(p);
        }
        s->ref++;
        unlock(s);

        /* Now we must do hardwork to ensure all processes which have tlb
         * entries for this segment will be flushed if we succeed in paging it out
         */
        p = proctab(0);
        ep = &p[conf.nproc];
        while(p < ep) {
            if(p->state != Dead) {
                for(i = 0; i < NSEG; i++)
                    if(p->seg[i] == s)
                        if(!canpage(p))
                            return false;
            }
            p++;
        }
        return true;
    }

⟨function canpage 186f⟩≡ (675c)
    int
    canpage(Proc *p)
    {
        bool ok = false;

        arch_splhi();
        lock(runq);

```

```

/* Only reliable way to see if we are Running */
if(p->cpu == nil) {
    p->newtlb = true;
    ok = true;
}
unlock(runq);
arch_spllo();

return ok;
}

⟨function gentick 187a⟩≡ (666)
static void
gentick(void)
{
    genclock++;
    if(gencount)
        genage = gensum / gencount;
    else
        genage = 0;
    gensum = gencount = 0;
}

⟨global genxxx 187b⟩≡ (666)
static  ulong  genage, genclock, gencount;
static  uulong  gensum;

⟨Page other fields 187c⟩+≡ (87) <165c 492c>
ulong gen;      /* Generation counter for swap */

```

8.8.6 Paging out

```

⟨function executeio 187d⟩≡ (666)
static void
executeio(void)
{
    Page *out;
    int i, n;
    Chan *c;
    char *kaddr;
    Arch_KMap *k;

    c = swapimage.c;
    qsort(iolist, ioptr, sizeof iolist[0], pageiocomp);
    for(i = 0; i < ioptr; i++) {
        if(ioptr > conf.nswppo)
            panic("executeio: ioptr %d > %d", ioptr, conf.nswppo);
        out = iolist[i];
        k = arch_kmap(out);
        kaddr = (char*)VA(k);

        if(waserror())
            panic("executeio: page out I/O error");

        n = devtab[c->type]->write(c, kaddr, BY2PG, out->daddr); // swap 1 page
        if(n != BY2PG)
            nexterror();

        arch_kunmap(k);
    }
}

```

```

        poperror();

        /* Free up the page after I/O */
        lock(out);
        out->ref--;
        unlock(out);
        putpage(out);
    }
    ioptr = 0;
}

⟨function pageiocomp 188a⟩≡ (666)
static int
pageiocomp(void *a, void *b)
{
    Page *p1, *p2;

    p1 = *(Page **)a;
    p2 = *(Page **)b;
    if(p1->daddr > p2->daddr)
        return 1;
    else
        return -1;
}

```

8.8.7 Paging in

```

⟨function swapaddr 188b⟩≡ (658)
#define swapaddr(s) (((ulong)s)&~PG_ONSWAP)

```

8.8.8 Setting up the swap file

```

⟨conswrite() cases 188c⟩≡ (346a) 231e▷
case Qswap:
    if(n >= sizeof buf)
        error(Egreg);
    memmove(buf, va, n); /* so we can NUL-terminate */
    buf[n] = 0;
    /* start a pager if not already started */
    if(strncmp(buf, "start", 5) == 0){
        kickpager();
        break;
    }
    if(!iseve())
        error(Eperm);
    if(buf[0]<'0' || '9'<buf[0])
        error(Ebadarg);
    fd = strtoul(buf, 0, 0);
    swc = fdtochan(fd, -1, true, true);
    setswapchan(swc);
    break;

⟨function setswapchan 188d⟩≡ (666)
void
setswapchan(Chan *c)
{
    byte dirbuf[sizeof(DirEntry)+100];
    DirEntry d;

```

```

int n;

if(swapimage.c) {
    if(swapalloc.free != conf.nswap){
        cclose(c);
        error(Einuse);
    }
    cclose(swapimage.c);
}

/*
 * if this isn't a file, set the swap space
 * to be at most the size of the partition
 */
if(devtab[c->type]->dc != L'M'){
    n = devtab[c->type]->stat(c, dirbuf, sizeof dirbuf);
    if(n <= 0){
        cclose(c);
        error("stat failed in setswapchan");
    }
    convM2D(dirbuf, n, &d, nil);
    if(d.length < conf.nswap*BY2PG){
        conf.nswap = d.length/BY2PG;
        swapalloc.top = &swapalloc.smap[conf.nswap];
        swapalloc.free = conf.nswap;
    }
}

swapimage.c = c;
}

```

Chapter 9

Scheduling

9.1 Overview

The scheduler decides which process runs next. Its central function is `sched()` (and its twin `schedinit()`). The chapter has two halves: cooperative scheduling, where kernel code voluntarily calls `sched()` or `sleep()/wakeup()` to yield the processor; and preemptive scheduling, where the clock interrupt forces a context switch after a time quantum (100 ms by default). Both halves ultimately funnel through the same mechanism: save the current process's context, pick the next one from the run queue, and restore its context. Processes are organized into priority queues. The scheduler always picks the highest-priority ready process, but priorities are dynamically adjusted to prevent starvation.

9.2 Process priority

There are 20 priority levels (`Npriq`). Normal user processes start at priority 10 (`PriNormal`), kernel processes at 13 (`PriKproc`). Higher numbers mean higher priority. The priority can be adjusted at runtime via `/proc/\emphpid/ctl`:

```
<constant Npriq 190a>≡ (668c)
    Npriq    = 20,    /* number of scheduler priority levels */
```

```
<enum priority 190b>≡ (668c)
    enum Priority
    {
        PriNormal = 10,    /* base priority for normal processes */
        PriKproc   = 13,    /* base priority for kernel processes */
        <Priority cases 191g>
        <Priority cases for real-time priority 502a>
    };
```

```
<Proc scheduling fields 190c>≡ (49f) 191b▷
    // enum<Priority>
    along priority; /* priority level */
```

```
<newproc() set fields of p 190d>+≡ (108c) <186b 195b▷
    /* sched params */
    procpriority(p, PriNormal, false);
```

```

⟨function procpriority 191a⟩≡ (675c)
void
procpriority(Proc *p, int pri, bool fixed)
{
    ⟨procpriority() sanity check pri 191d⟩
    p->priority = pri;

    p->basepri = pri;
    p->fixedpri = fixed;
}

⟨Proc scheduling fields 191b⟩+≡ (49f) <190c 195a>
// enum<Priority>
ulong basepri; /* base priority level */
bool fixedpri; /* priority level doesn't change */

⟨sysrfork() when RFPROC propagate fields from up to p 191c⟩+≡ (112d) <186c 197c>
p->priority = up->basepri;
p->basepri = up->basepri;
p->fixedpri = up->fixedpri;

⟨procpriority() sanity check pri 191d⟩≡ (191a)
if(pri >= Npriq)
    pri = Npriq - 1;
else if(pri < 0)
    pri = 0;

⟨procctlreq() CMpri case 191e⟩≡ (554b)
case CMpri:
    pri = atoi(cb->f[1]);
    if(pri > PriNormal && !iseve())
        error(Eperm);
    procpriority(p, pri, false);
    break;

⟨procctlreq() CMfixedpri case 191f⟩≡ (554b)
case CMfixedpri:
    pri = atoi(cb->f[1]);
    if(pri > PriNormal && !iseve())
        error(Eperm);
    procpriority(p, pri, true);
    break;

⟨Priority cases 191g⟩≡ (190b)
PriRoot    = 13, /* base priority for root processes */

⟨sysexec() set priority of root processes 191h⟩≡ (119d)
/*
 * '/' processes are higher priority (hack to make /ip more responsive).
 */
if(devtab[tc->type]->dc == L'/')
    up->basepri = PriRoot;
up->priority = up->basepri;

```


9.3 The runqueue

The run queue is an array of `Nrq` queues, one per priority level. Each queue is a FIFO linked list of `Proc` pointers. The bitmap `runvec` has bit i set if `runq[i]` is non-empty, allowing `runproc()` to quickly find the highest-priority non-empty queue:

```
runq[19]: (highest priority)
runq[18]:
...
runq[13]: kproc1 -> kproc2
...
runq[10]: proc_A -> proc_B -> proc_C
...
runq[0]: (lowest priority)

runvec = ... 0010_0000_0100_0000_0000 (bits 10, 13 set)

⟨constant Nrq 192a⟩≡ (668c)
    Nrq = Npriq+2, /* number of priority levels including real time */

⟨global runq 192b⟩≡ (675c)
    // The run queue!!
    // hash<enum<priority>, queue<ref<Proc>>>
    Schedq runq[Nrq];

⟨struct Schedq 192c⟩≡ (668c)
    // essentially a queue<ref<Proc>>
    struct Schedq
    {
        // list<ref<Proc>> (next = Proc.rnext)
        Proc* head;
        // ref<Proc>, the tail
        Proc* tail;
        // size of list
        int n;

        // extra
        Lock; // protects also the globals nrdy and runvec
    };

⟨Proc extra fields 192d⟩+≡ (49f) <107d 224e>
    // list<ref<Proc>> of Schedq.head
    Proc *rnext; /* next process in run queue */

⟨global runvec 192e⟩≡ (675c)
    // array<bool>, each bit i represents whether the runq at pri i has processes
    along runvec; // coupling: sizeof(ulong) must be >= Nrq

⟨function anyready 192f⟩≡ (675c)
    bool
    anyready(void)
    {
        return runvec;
    }

⟨global nrdy 192g⟩≡ (675c)
    int nrdy;
```

<function queueproc 193a>≡ (675c)

```
/*
 * add a process to a scheduling queue
 */
void
queueproc(Schedq *rq, Proc *p)
{
    int pri;

    pri = rq - runq;
    lock(runq);
    p->priority = pri; // because a priority can evolve

    // add_queue(p, rq)
    p->rnext = nil;
    if(rq->tail)
        rq->tail->rnext = p;
    else
        rq->head = p;
    rq->tail = p;
    rq->n++;

    nrdy++;
    runvec |= 1<<pri;
    unlock(runq);
}
```

<function dequeueproc 193b>≡ (675c)

```
/*
 * try to remove a process from a scheduling queue (called arch_splhi)
 */
Proc*
dequeueproc(Schedq *rq, Proc *tp)
{
    Proc *l, *p;

    if(!canlock(runq))
        return nil;

    /*
     * the queue may have changed before we locked runq,
     * refind the target process.
     */
    l = nil;
    for(p = rq->head; p; p = p->rnext){
        if(p == tp)
            break;
        l = p;
    }
    <dequeueproc() return if process not found or already associated 194a>

    // remove_queue(p, rq)
    if(p->rnext == nil)
        rq->tail = l;
    if(l)
        l->rnext = p->rnext;
    else
        rq->head = p->rnext;

    if(rq->head == nil)
```

```

        runvec &= ~(1<<(rq-runq));
    rq->n--;
    nrdy--;
    ⟨dequeueproc() sanity check p 194b⟩
    unlock(runq);
    return p;
}

⟨dequeueproc() return if process not found or already associated 194a⟩≡      (193b)
/*
 * p->cpu==nil only when process state is saved
 */
if(p == nil || p->cpu){
    unlock(runq);
    return nil;
}

⟨dequeueproc() sanity check p 194b⟩≡      (193b)
if(p->state != Ready)
    print("dequeueproc %s %lud %s\n", p->text, p->pid, statename[p->state]);

```

9.4 The scheduler

9.4.1 sched()

`sched()` is the central function. When called with `up != nil` (a running process wants to yield), it saves the current process's context and jumps to `schedinit()`. When called with `up == nil` (from `schedinit()`, on the CPU's boot stack), it picks the next process via `runproc()` and jumps to it.

```

⟨function sched 194c⟩≡      (675c)
/*
 * If changing this routine, look also at sleep(). It
 * contains a copy of the guts of sched().
 */
void
proc_sched(void)
{
    ⟨sched() locals 194d⟩

    ⟨sched() sanity check ilockdepth 195c⟩
    ⟨sched() if up 198b⟩
    // else
    ⟨sched() if no up 194e⟩
}

⟨sched() locals 194d⟩≡      (194c)
Proc *p;

⟨sched() if no up 194e⟩≡      (194c)
// We should execute this code using the main kernel stack, as
// we should arrive here from schedinit()

p = runproc();
⟨sched() after found process p to schedule, possibly update priority 209a⟩
⟨sched() if readied process, do not adjust schedticks 206g⟩
else {
    ⟨sched() after found a new proc, adjust schedticks 206d⟩
}

```

```

⟨sched() reset readied 203c⟩

cpu->proc = p;
up = p; // same instruction than previous line on some archi (e.g. x86)

up->state = Running;
up->cpu = CPUS(cpu->cpuno);

arch_mmuswitch(up);
arch_gotolabel(&up->sched);

⟨Proc scheduling fields 195a⟩+≡ (49f) <191b 197a>
// option<ref<Cpu>>, null when not associated to a processor
Cpu *cpu; /* processor running this proc */

⟨newproc() set fields of p 195b⟩+≡ (108c) <190d 197b>
p->cpu = nil;

⟨sched() sanity check ilockdepth 195c⟩≡ (194c)
if(cpu->ilockdepth)
    panic("cpu%d: ilockdepth %d, last lock %#p at %#p, sched called from %#p",
        cpu->cpuno, cpu->ilockdepth, up? up->lastilock: nil,
        (up && up->lastilock)? up->lastilock->pc: 0, getcallerpc(&p+2));

```

9.4.2 schedinit()

`schedinit()` runs on the CPU's boot stack (set up during initialization). It calls `arch_setlabel()` to save the CPU's "return here" point in `cpu->sched`, then handles the outgoing process (putting it back on the run queue if `Running`, freeing it if `Moribund`), and falls through to `sched()` with `up == nil`. When `sched()` later finds a new process, it calls `arch_gotolabel(&cpu->sched)` to return here.

```

⟨Cpu other fields 195d⟩+≡ (48a) <70f 203a>
Label sched; /* scheduler wakeup */ // address of schedinit()

⟨function schedinit 195e⟩≡ (675c)
/*
 * Always arch_splhi()'ed.
 */
void
schedinit(void) /* never returns */
{
    ⟨schedinit() locals 502f⟩

    arch_setlabel(&cpu->sched);
    ⟨schedinit() after setlabel, if up 199a⟩
    // ok at this point up is nil
    sched();
}

```

9.4.3 runproc()

`runproc()` finds the next process to run. It scans the run queue from highest to lowest priority, looking for a process with CPU affinity to this processor (or any process on a second pass). If no process is ready, it calls

arch_idlehands() to halt the CPU until an interrupt arrives.

```
<function runproc 196>≡ (675c)
/*
 * pick a process to run
 */
Proc*
runproc(void)
{
    Proc *p;
    Schedq *rq;
    int i;
    <runproc() other locals 440h>

    <runproc() start 587d>
    <runproc() cooperative scheduling if process readied 203b>
    //else

    preempts++;

loop:
    /*
     * find a process that last ran on this processor (affinity),
     * or one that hasn't moved in a while (load balancing). Every
     * time around the loop affinity goes down.
     */
    arch_spllo(); // from schedinit()

    for(i = 0;; i++){
        /*
         * find the highest priority target process that this
         * processor can run given affinity constraints.
         */
        for(rq = &runq[Nrq-1]; rq >= runq; rq--){
            for(p = rq->head; p; p = p->rnext){
                if(p->lastcpu == nil || p->lastcpu == CPUS(cpu->cpuno)
                    || (p->wired == nil && i > 0)) // favor affinity for first round
                    goto found;
            }
        }
        // nothing found
        /* waste time or halt the CPU */
        arch_idlehands();
        <runproc() after arch_idlehands, before restarting loop 587e>
    }

found:
    arch_splhi();
    p = dequeueproc(rq, p);
    if(p == nil) // the runq may have changed between spllo and splhi
        goto loop;

    p->state = Scheduling;
    p->lastcpu = CPUS(cpu->cpuno);

    <runproc() test if p is a real-time process 502c>
    <runproc() hook proctrace 440i>
    return p;
}
```

```

⟨Proc scheduling fields 197a⟩+≡ (49f) <195a 200a>
    Cpu *lastcpu;    /* processor this process last ran on */

⟨newproc() set fields of p 197b⟩+≡ (108c) <195b 197f>
    p->lastcpu = nil;

⟨sysrfork() when RFPROC propagate fields from up to p 197c⟩+≡ (112d) <191c 395f>
    p->lastcpu = up->lastcpu;

```

9.5 Context switch

A context switch saves one process’s execution state and restores another’s. Unlike the full register save done by `_vsvc` on a syscall (saving all 15 registers into a `Ureg`), a kernel-to-kernel context switch only needs to save SP and PC—because kernel code follows the calling convention, and the compiler has already saved any registers it needs.

9.5.1 Label

```

⟨struct Label 197d⟩≡ (609)
    // =~ a jumpbuf in C, for coroutines
    struct Label
    {
        // or virt_addr? used also for saving context of user code?
        kern_addr sp;
        kern_addr pc;
    };

⟨Proc assembly fields 197e⟩≡ (49f)
    Label sched;    /* known to l.s */
    char *kstack;   /* known to l.s */

```

9.5.2 Kernel stacks

Each process has its own 8 KiB kernel stack (`kstack`), allocated once when the `Proc` is first used and reused across process lifetimes. This per-process stack is where syscall handlers, page fault handlers, and `sched()` run. It is separate from the CPU’s boot stack (inside the `Cpu` structure), which is used only by `schedinit()`.

```

⟨newproc() set fields of p 197f⟩+≡ (108c) <197b 200b>
    if(p->kstack == nil)
        p->kstack = smalloc(KSTACK);

⟨constant KSTACK(arm) 197g⟩≡ (709b)
    #define KSTACK    (8*KiB)

```

9.5.3 arch_gotolabel() (ARM)

`arch_setlabel()` saves SP and PC into a `Label` and returns 0. `arch_gotolabel()` restores them and returns 1. This is the kernel’s equivalent of C’s `setjmp/longjmp`—it is how the scheduler switches between processes. The different return values let the caller distinguish “just saved” from “resumed after being switched out”:

```

⟨function arch_gotolabel(arm) 197h⟩≡ (719b)
    TEXT arch_gotolabel(SB), 1, $-4
        MOVW    0(R0), R13    /* sp */
        MOVW    4(R0), R14    /* pc */
        MOVW    $1, R0
        RET

```

```

⟨function arch_setlabel(arm) 198a)≡ (719b)
TEXT arch_setlabel(SB), 1, $-4
    MOVW    R13, 0(R0)      /* sp */
    MOVW    R14, 4(R0)      /* pc */
    MOVW    $0, R0
    RET

```

9.5.4 sched() and schedinit() dance

The interplay between `sched()` and `schedinit()` is the trickiest part of the scheduler. Here is the flow when a running process calls `sched()`:

Process A's kstack:	CPU's boot stack:
<pre> 1. sched() called arch_setlabel(&up->sched) returns 0 (first time) v 2. arch_gotolabel(&cpu->sched) ----jumps to-----> </pre>	
	<pre> 3. schedinit() resumes up->state == Running -> ready(up) up = nil -> sched() v 4. p = runproc() (picks process B) arch_gotolabel(&p->sched) ----jumps to-----> </pre>
	Process B resumes from its setlabel

When process A is eventually picked again by `runproc()`, `arch_gotolabel(&A->sched)` returns 1 (into step 1 above), and `sched()` calls `arch_procrestore()` and returns—process A continues where it left off.

```

⟨sched() if up 198b)≡ (194c)
if(up){
    ⟨sched() when up, possibly increment delaysched and return 199c)
    arch_splhi(); // schedinit requires this (also sensitive moment)
    cpu->cs++;

    arch_procsave(up);
    if(arch_setlabel(&up->sched)){
        //
        // here when the process has been scheduled back
        // from a arch_gotolabel(up->sched) by another process, see below
        //
        arch_procrestore(up);
        arch_spllo();
        return;
    }else{
        //
        // here to go to schedinit() (which will call sched() back)
    }
}

```

```

    //
    arch_gotolabel(&cpu->sched); // goto schedinit()
    panic("sched: should never reach this point");
}
}

⟨schedinit() after setlabel, if up 199a⟩≡ (195e)
if(up) {
    ⟨schedinit() optional real-time edfrecord() 502g⟩
    //old: cpu->proc = nil;
    // but now that on x86 up = cpu->proc and not cpu->externup
    // we can't do that anymore. Is there a place that rely on
    // cpu->proc and cpu->externup to not be in sync?
    switch(up->state) {
        ⟨schedinit() switch process state cases 131c⟩
    }
    up->cpu = nil;
    ⟨schedinit() updatecpu before resetting up 209c⟩
    cpu->proc = nil;
    up = nil; // same instruction than previous line on some archi (e.g. PC)
}

⟨schedinit() switch process state cases 199b⟩+≡ (199a) <131c
case Running:
    ready(up);
    break;

```

9.5.5 Delayed sched()

A process holding spin locks must not be switched out—the lock would remain held, potentially deadlocking other CPUs. So when `sched()` is called while `up->nlocks > 0`, it increments `delaysched` and returns immediately. The actual reschedule happens later, when `unlock()` sees `delaysched > 0` and calls `sched()` after releasing the last lock. As a safety valve, if `delaysched` exceeds 20 (a lock was probably leaked), the scheduler forces a switch anyway.

```

⟨sched() when up, possibly increment delaysched and return 199c⟩≡ (198b)
/*
 * Delay the sched until the process gives up the locks
 * it is holding. This avoids dumb lock loops.
 *
 * But don't delay if the process is Moribund.
 * It called sched to die.
 * But do sched eventually. This avoids a missing unlock
 * from hanging the entire kernel.
 * But don't reschedule procs holding palloc or procalloc.
 * Those are far too important to be holding while asleep.
 *
 * This test is not exact. There can still be a few instructions
 * in the middle of taslock when a process holds a lock
 * but Lock.p has not yet been initialized.
 */
if(up->nlocks.ref)
    if(up->state != Moribund)
        if(up->delaysched < 20
            || palloc.Lock.p == up
            || procalloc.Lock.p == up){

            up->delaysched++;

```



```

        delayed_scheds++;
        return;
    }
    // else
    up->delaysched = 0;

<Proc scheduling fields 200a>+≡ (49f) <197a 207e>
    ulong delaysched;

<newproc() set fields of p 200b>+≡ (108c) <197f 207f>
    p->nlocks.ref = 0;
    p->delaysched = 0;

<unlock() if delaysched 200c>≡ (60c)
    if(up && decCnt(&up->nlocks) == 0 && up->delaysched && arch_islo()){
        /*
         * Call sched if the need arose while locks were held
         * But, don't do it from interrupt routines, hence the arch_islo() test
         */
        sched();
    }

<trap() if delaysched and clockintr 200d>≡ (146)
    /* delaysched set because we held a lock or because our quantum ended */
    if(up && up->delaysched && clockintr){
        sched();
        arch_splhi();
    }

<syscall() if delaysched(arm) 200e>≡ (141e)
    /* if we delayed sched because we held a lock, sched now */
    if(up->delaysched){
        sched();
        arch_splhi();
    }

```

9.5.6 arch_mmuswitch() (ARM)

On each context switch, `arch_mmuswitch()` installs the new process's hardware page tables. It clears the user portion of the L1, repopulates it from the process's `mmul2` list, and invalidates the TLB. If the pager modified the process's pages while it was asleep (`newtlb` flag), the L2 pages are also cleared.

```

<Cpu Arch mmu fields(arm) 200f>+≡ (48e) <163c
    int mmupid;

<function arch_mmuswitch(arm) 200g>≡ (711a)
    void
    arch_mmuswitch(Proc* proc)
    {
        int x;
        PTE *l1;
        Page *page;

        /* do kprocs get here and if so, do they need to? */
        /** "This is plausible, but wrong" - Charles Forsyth 1 Mar 2015
         * if(cpu->mmupid == proc->pid && !proc->newtlb)
         *     return;
         */
        cpu->mmupid = proc->pid;
    }

```

```

⟨arch_mmuswitch() write back and invalidate cache before switch(arm) 469f⟩

if(proc->newtlb){
    mmul2empty(proc, 1);
    proc->newtlb = false;
}

mmul1empty();

/* move in new map */
l1 = cpu->mmul1;
for(page = proc->mmul2; page != nil; page = page->next){
    x = page->daddr;
    l1[x] = PPN(page->pa)|Dom0|Coarse;
    /* know here that L1lo < x < L1hi */
    if(x+1 - cpu->mmul1lo < cpu->mmul1hi - x)
        cpu->mmul1lo = x+1;
    else
        cpu->mmul1hi = x;
}

⟨arch_mmuswitch() write back cache after adjusting page tables(arm) 470b⟩

/* lose any possible stale tlb entries */
mmuinvalidate();
}

```

9.5.7 arch_procsave() (ARM)

arch_procsave() saves architecture-specific state before a context switch (floating-point registers, cycle counters). arch_procrestore() restores them when the process resumes:

```

⟨function arch_procsave(arm) 201a⟩≡ (717b)
/*
 * Save the mach dependent part of the process state.
 */
void
arch_procsave(Proc* p)
{
    ⟨arch_procsave() cycles adjustments 449e⟩
    ⟨arch_procsave() floating point(arm) 455e⟩
}

⟨function arch_procrestore(arm) 201b⟩≡ (717b)
void
arch_procrestore(Proc* p)
{
    uulong t;

    if(p->kp)
        return;
    ⟨arch_procrestore cycles adjustments 449f⟩
    ⟨arch_procrestore() floating point(arm) 456a⟩
}

```

9.6 Cooperative scheduling

In cooperative scheduling, a process voluntarily gives up the CPU by calling `sched()`, `sleep()`, or `yield()`. This is the most common path: syscalls that block (waiting for I/O, a lock, or a child process) call `sleep()` on a `Rendez`, which calls `sched()` internally.

9.6.1 `yield()`

```
<function yield 202a>≡ (675c)
/*
 * yield the processor and drop our priority
 */
void
yield(void)
{
    if(anyready()){
        <yield() adjust lastupdate before scheduling another process 209h>
        sched();
    }
}
```

9.6.2 `ready()`

`ready()` puts a process on the run queue at its current priority. It is called from many places: `schedinit()` (when the current process is preempted), `wakeup()` (when a sleeping process's condition becomes true), `qunlock()` (when a waiting process gets the lock), `sysrfork()` (for the child), and `kproc()` (for new kernel processes).

```
<Procstate cases 202b>+≡ (51b) <131a 203g>
    Ready,

<function ready 202c>≡ (675c)
/*
 * ready(p) picks a new priority for a process and sticks it in the
 * runq for that priority.
 */
void
proc_ready(Proc *p)
{
    int pri;
    int s; // spl
    Schedq *rq;
    <ready() other locals 440f>

    s = arch_splhi();
    <ready() optional edfready() for real-time scheduling 502i>

    <ready() possibly set cpu->readied 202d>
    <ready() update priority 209b>

    rq = &runq[pri];
    p->state = Ready;
    queueproc(rq, p);
    <ready() hook proctrace 440g>
    arch_splx(s);
}

<ready() possibly set cpu->readied 202d>≡ (202c)
    if(up != p && (p->wired == nil || p->wired == CPUS(cpu->cpuno))) // pad fix
        cpu->readied = p; /* group scheduling */
```

```

⟨Cpu other fields 203a⟩+≡ (48a) <195d 206c>
    Proc* readied; /* for runproc */

⟨runproc() cooperative scheduling if process readied 203b⟩≡ (196)
    /* cooperative scheduling until the clock ticks */
    p=cpu->readied;
    if(p && p->cpu==nil && p->state==Ready &&
        (p->wired == nil || p->wired == CPUS(cpu->cpuno)) && // pad's bugfix!
        ⟨runproc() test for empty real-time scheduling queue 502b⟩
    ){
        skipscheds++;
        rq = &runq[p->priority];
        goto found;
    }

⟨sched() reset readied 203c⟩≡ (194e)
    cpu->readied = nil;

```

9.6.3 Rendez

A **Rendez** (rendezvous point) is a condition variable for kernel code. It pairs a sleeping process (**r->p**) with a lock. **sleep()** puts the current process to sleep on a **Rendez** if a condition is false; **wakeup()** wakes it when the condition becomes true. This is how all blocking in the kernel works: waiting for a page, a lock, a child exit, a pipe read, etc.

```

⟨struct Rendez 203d⟩≡ (604)
    struct Rendez
    {
        // option<ref<Proc>>
        Proc *p; // sleeping process

        Lock; // LOCK ORDERING: always do lock(r); lock(p->rlock);
    };

⟨Proc synchronization fields 203e⟩+≡ (49f) <69g 203f>
    // option<ref<Rendez>>, can point to waitr, freememr, sleep, etc
    Rendez *r; /* rendezvous point slept on */

⟨Proc synchronization fields 203f⟩+≡ (49f) <203e 222f>
    Lock rlock; /* sync sleep/wakeup with postnote */

```

9.6.4 sleep()

sleep() checks the condition function **f(arg)**. If true (or a note is pending), it returns immediately. Otherwise, it sets the process state to **Wakeme**, records the **Rendez**, and performs a context switch (the same **arch_setlabel/arch_gotolabel** dance as **sched()**). The locks on both the **Rendez** and the process's **rlock** are held while committing to sleep, then released just before jumping to **schedinit()**.

```

⟨Procstate cases 203g⟩+≡ (51b) <202b 422d>
    Wakeme,

```

```

⟨function sleep 204⟩≡ (675c)
/*
 * sleep if a condition is not true. Another process will
 * awaken us after it sets the condition. When we awaken
 * the condition may no longer be true.
 *
 * we lock both the process and the rendezvous to keep r->p
 * and p->r synchronized.
 */
void
proc_sleep(Rendez *r, bool (*f)(void*), void *arg)
{
    int s; // spl
    ⟨sleep() other locals 441c⟩

    s = arch_splhi();
    ⟨sleep() sanity check nlocks 205a⟩
    lock(r);
    lock(&up->rlock);
    ⟨sleep() sanity check double sleep on this rendez vous 205b⟩

    /*
     * Wakeup only knows there may be something to do by testing
     * r->p in order to get something to lock on.
     * Flush that information out to memory in case the sleep is
     * committed.
     */
    r->p = up;

    if((*f)(arg) || up->notepending){
        /*
         * if condition happened or a note is pending
         * never mind
         */
        r->p = nil;
        unlock(&up->rlock);
        unlock(r);
    } else {
        /*
         * now we are committed to
         * change state and call scheduler
         */
        ⟨sleep() hook proctrace 441d⟩
        up->state = Wakeme;
        up->r = r;

        // similar code to sched(), why not call sched()? because of the unlock?

        /* statistics */
        cpu->cs++;

        arch_procsave(up);
        if(arch_setlabel(&up->sched)) {
            /*
             * here when the process is awakened
             */
            arch_procrestore(up);
            arch_spllo();
        } else {
            /*

```

```

        * here to go to sleep (i.e. stop Running)
        */
        unlock(&up->rlock);
        unlock(r);

        arch_gotolabel(&cpu->sched);
        panic("sleep: should never reach this point");
    }
}
⟨sleep() if notepending 395g⟩
arch_splx(s);
}

⟨sleep() sanity check nlocks 205a⟩≡ (204)
    if(up->nlocks.ref)
        print("process %lud sleeps with %lud locks held, last lock %#p locked at pc %#lux, sleep called from %#p\n"
            up->pid, up->nlocks.ref, up->lastlock, up->lastlock->pc, getcallerpc(&r));

⟨sleep() sanity check double sleep on this rendez vous 205b⟩≡ (204)
    if(r->p){
        print("double sleep called from %#p, %lud %lud\n", getcallerpc(&r), r->p->pid, up->pid);
        arch_dumpstack();
    }
}

```

9.6.5 wakeup()

wakeup() checks if anyone is sleeping on the Rendez. If so, it clears the Rendez link, calls ready() to put the process back on the run queue, and returns the process. The woken process will resume inside sleep() at the arch_setlabel return point:

```

⟨function wakeup 205c⟩≡ (675c)
/*
 * Expects that only one process can call wakeup for any given Rendez.
 * We hold both locks to ensure that r->p and p->r remain consistent.
 * Richard Miller has a better solution that doesn't require both to
 * be held simultaneously, but I'm a paranoid - presotto.
 */
Proc*
proc_wakeup(Rendez *r)
{
    Proc *p;
    int s; // spl

    s = arch_splhi();
    lock(r);
    p = r->p;

    if(p != nil){
        lock(&p->rlock);
        ⟨wakeup() sanity check process state 206a⟩
        r->p = nil;
        p->r = nil;
        ready(p);
        unlock(&p->rlock);
    }
    unlock(r);
    arch_splx(s);
    return p;
}

```

```

⟨wakeup() sanity check process state 206a⟩≡ (205c)
    if(p->state != Wakeme || p->r != r){
        iprint("%p %p %d\n", p->r, r, p->state);
        panic("wakeup: state");
    }

```

9.7 Preemptive scheduling

Cooperative scheduling is not enough—a CPU-bound process that never calls `sleep()` would monopolize the processor. Preemptive scheduling uses the clock interrupt (firing `Arch_HZ` times per second, i.e., every 10 ms) to force context switches. Each process gets a time quantum of 100 ms (`Arch_HZ/10` ticks); when the quantum expires, `hzsched()` sets `delaysched` to trigger a reschedule at the next safe point.

```

⟨constant Arch_HZ(arm) 206b⟩≡ (714b)
#define Arch_HZ      100          /* clock frequency */

```

9.7.1 hzsched()

`hzsched()` is called from `hzclock()` on every timer tick while a process is running. It checks two conditions for preemption: is there a higher-priority process ready (`anyhigher()`)? or has the current process's quantum expired (`cpu->ticks > cpu->schedticks`)? If either is true, it sets `delaysched` rather than calling `sched()` directly—because we are inside an interrupt handler and cannot safely switch context here. The actual switch happens when the interrupt returns and checks `delaysched`.

```

⟨Cpu other fields 206c⟩+≡ (48a) <203a 415b>
    ulong schedticks; /* next forced context switch */

```

```

⟨sched() after found a new proc, adjust schedticks 206d⟩≡ (194e)
    cpu->schedticks = cpu->ticks + Arch_HZ/10; // 100ms of allocated time

```

```

⟨function hzsched 206e⟩≡ (675c)
/*
 * here once per clock tick to see if we should resched
 */
void
hzsched(void)
{
    ⟨hzsched() rebalance ready procs 211a⟩
    /* unless preempted, get to run for at least 100ms */
    if(anyhigher()
        || (!up->fixedpri && cpu->ticks > cpu->schedticks && anyready())){

        cpu->readied = nil; /* avoid cooperative scheduling */
        up->delaysched++;
    }
}

```

```

⟨function anyhigher 206f⟩≡ (675c)
int
anyhigher(void)
{
    return runvec & ~((1<<(up->priority+1))-1);
}

```

```

⟨sched() if readied process, do not adjust schedticks 206g⟩≡ (194e)
    if(p == cpu->readied) {
    }

```

9.7.2 hzclock()

hzclock() is the clock interrupt handler, called 100 times per second. It increments `cpu->ticks`, accounts CPU time to the current process, checks alarms, and calls `hzsched()` to decide whether to preempt.

<clock callback hzclock 207a>≡ (673a)

```
void
hzclock(Ureg *ur)
{
    cpu->ticks++;
    <hzclock() adjust pc of proc 207b>
    <hzclock() if flushmmu 415a>
    <hzclock() accounttime 588a>
    <hzclock() if kproftimer 590b>

    <hzclock() if not active cpu, return 207c>
    // else
    <hzclock() if exiting, exit 207d>
    // else

    <hzclock() check the alarms 227a>
    if(up && up->state == Running)
        hzsched(); /* in proc.c */
}
```

<hzclock() adjust pc of proc 207b>≡ (207a)

```
if(cpu->proc) // why not using up here? why cpu->proc?
    cpu->proc->pc = ur->pc;
```

<hzclock() if not active cpu, return 207c>≡ (207a)

```
if((active.cpus & (1<<cpu->cpuno)) == 0)
    return;
```

<hzclock() if exiting, exit 207d>≡ (207a)

```
if(active.exiting) {
    print("someone's exiting\n");
    arch_exit(0);
}
```

9.8 Customized scheduling

A process can be wired to a specific CPU via `echo wired \emphn > /proc/\emphpid/ctl`. Once wired, `runproc()` will only run it on that CPU. This is used for I/O-intensive processes that benefit from cache locality on a particular processor:

<Proc scheduling fields 207e>+≡ (49f) <200a 209d>

```
Cpu *wired;
```

<newproc() set fields of p 207f>+≡ (108c) <200b 209e>

```
p->wired = nil;
```

<procctlreq() CMwired case 207g>≡ (554b)

```
case CMwired:
    procwired(p, atoi(cb->f[1]));
    break;
```



```

⟨function procwired 208a⟩≡ (675c)
/*
 * wire this proc to a processor
 */
void
procwired(Proc *p, int bm)
{
    Proc *pp;
    int i;
    char nwired[MAXCPUS];
    Cpu *wm;

    if(bm < 0){
        /* pick a processor to wire to */
        memset(nwired, 0, sizeof(nwired));
        p->wired = nil;
        pp = proctab(0);
        for(i=0; i<conf.nproc; i++, pp++){
            wm = pp->wired;
            if(wm && pp->pid)
                nwired[wm->cpuno]++;
        }
        bm = 0;
        for(i=0; i<conf.ncpu; i++)
            if(nwired[i] < nwired[bm])
                bm = i;
    } else {
        /* use the virtual processor requested */
        bm = bm % conf.ncpu;
    }

    p->wired = CPUS(bm);
    p->lastcpu = p->wired;
}

⟨sysrfork() other locals 208b⟩+≡ (112b) <117b
    Cpu *wm;

⟨sysrfork() if parent is a wired proc 208c⟩≡ (112d)
    wm = up->wired;
    if(wm)
        procwired(p, wm->cpuno);

```

9.9 Real-time scheduling

9.10 Adjusting priority

A fixed-priority scheduler would starve low-priority processes. To prevent this, the kernel dynamically adjusts each process's priority based on how much CPU time it has used recently. `updatecpu()` decays the process's CPU usage counter, and `reprioritize()` maps that usage to a priority level. Processes that have used a lot of CPU time recently get a lower priority; those that have been sleeping get a higher one. This ensures that I/O-bound processes (which sleep often) remain responsive, while CPU-bound processes gradually drop in priority.

9.10.1 reprioritize()

<sched() after found process p to schedule, possibly update priority 209a>≡ (194e)
<sched() if real-time process then do nothing, do not reprioritize 504a>
else {
 updatecpu(p);
 p->priority = reprioritize(p);
}

<ready() update priority 209b>≡ (202c)
updatecpu(p);
pri = reprioritize(p);
p->priority = pri;

<schedinit() updatecpu before resetting up 209c>≡ (199a)
updatecpu(up);

<Proc scheduling fields 209d>+≡ (49f) <207e 504c>
 ulong lastupdate; // dimension?? ticks * Scaling;
 ulong cpuavg; /* cpu average */

<newproc() set fields of p 209e>+≡ (108c) <207f 395d>
p->cpuavg = 0;
p->lastupdate = CPUS(0)->ticks * Scaling;

<constant Scaling 209f>≡ (675c)
Scaling=2,

<constant Schedagain 209g>≡ (675c)
schedgain = 30, /* units in seconds */

<yield() adjust lastupdate before scheduling another process 209h>≡ (202a)
/* pretend we just used 1/2 tick */
up->lastupdate -= Scaling/2;

<function updatecpu 209i>≡ (675c)
/*
 * Update the cpu time average for this particular process,
 * which is about to change from up -> not up or vice versa.
 * p->lastupdate is the last time an updatecpu happened.
 *
 * The cpu time average is a decaying average that lasts
 * about D clock ticks. D is chosen to be approximately
 * the cpu time of a cpu-intensive "quick job". A job has to run
 * for approximately D clock ticks before we home in on its
 * actual cpu usage. Thus if you manage to get in and get out
 * quickly, you won't be penalized during your burst. Once you
 * start using your share of the cpu for more than about D
 * clock ticks though, your p->cpu hits 1000 (1.0) and you end up
 * below all the other quick jobs. Interactive tasks, because
 * they basically always use less than their fair share of cpu,
 * will be rewarded.
 *
 * If the process has not been running, then we want to
 * apply the filter
 *
 * cpu = cpu * (D-1)/D
 *
 * n times, yielding
 *
 * cpu = cpu * ((D-1)/D)^n
*/

```

*
* but D is big enough that this is approximately
*
*   cpu = cpu * (D-n)/D
*
* so we use that instead.
*
* If the process has been running, we apply the filter to
* 1 - cpu, yielding a similar equation. Note that cpu is
* stored in fixed point (* 1000).
*
* Updatecpu must be called before changing up, in order
* to maintain accurate cpu usage statistics. It can be called
* at any time to bring the stats for a given proc up-to-date.
*/
void
updatecpu(Proc *p)
{
    int n, t, ocpu;
    int D = schedgain * Arch_HZ * Scaling;

    <updatecpu() return if real-time process 502e>

    t = CPUS(0)->ticks*Scaling + Scaling/2;
    n = t - p->lastupdate;
    p->lastupdate = t;

    if(n == 0)
        return;
    if(n > D)
        n = D;

    ocpu = p->cpuavg;
    if(p != up)
        p->cpuavg = (ocpu*(D-n))/D;
    else{
        t = 1000 - ocpu;
        t = (t*(D-n))/D;
        p->cpuavg = 1000 - t;
    }
}

}

<function reprioritize 210>≡ (675c)
/*
* On average, p has used p->cpuavg of a cpu recently.
* Its fair share is conf.ncpu/cpu->load of a cpu. If it has been getting
* too much, penalize it. If it has been getting not enough, reward it.
* I don't think you can get much more than your fair share that
* often, so most of the queues are for using less. Having a priority
* of 3 means you're just right. Having a higher priority (up to p->basepri)
* means you're not using as much as you could.
*/
int
reprioritize(Proc *p)
{
    int fairshare, n, load, ratio;

    load = CPUS(0)->load;
    if(load == 0)

```

```

    return p->basepri;

/*
 * fairshare = 1.000 * conf.nproc * 1.000/load,
 * except the decimal point is moved three places
 * on both load and fairshare.
 */
fairshare = (conf.ncpu*1000*1000)/load;
n = p->cpuavg;
if(n == 0)
    n = 1;
ratio = (fairshare+n/2) / n;
if(ratio > p->basepri)
    ratio = p->basepri;
if(ratio < 0)
    panic("reprioritize");
//iprint("pid %d cpu %d load %d fair %d pri %d\n", p->pid, p->cpuavg, load, fairshare, ratio);
return ratio;
}

```

9.10.2 rebalance()

<hzsched() rebalance ready procs 211a>≡ (206e)

```

/* once a second, rebalance will reprioritize ready procs */
if(cpu->cpuno == 0)
    rebalance();

```

<function rebalance 211b>≡ (675c)

```

static void
rebalance(void)
{
    int pri, npri, t, x;
    Schedq *rq;
    Proc *p;

```

<rebalance() return if less than one second since last rebalance 212b>

```

for(pri=0, rq=runq; pri<Npriq; pri++, rq++){
another:
    p = rq->head;
    if(p == nil)
        continue;
    if(p->lastcpu != CPUS(cpu->cpuno)) // this has been the source of comments on 9fans, todo look if bug (
        continue;
    if(pri == p->basepri)
        continue;
    updatecpu(p);
    npri = reprioritize(p);
    if(npri != pri){
        x = arch_splhi();
        p = dequeueproc(rq, p);
        if(p)
            queueproc(&runq[npri], p);
        arch_splx(x);
        goto another;
    }
}
}

```

```

⟨global balancetime 212a⟩≡ (675c)
/*
 * recalculate priorities once a second. We need to do this
 * since priorities will otherwise only be recalculated when
 * the running process blocks.
 */
ulong balancetime;

⟨rebalance() return if less than one second since last rebalance 212b⟩≡ (211b)
t = cpu->ticks;
if(t - balancetime < Arch_HZ)
    return;
balancetime = t;

```

9.11 Halting (ARM)

When `runproc()` finds no ready process, it calls `arch_idlehands()`, which enables interrupts and executes the ARM WFI (Wait For Interrupt) instruction. The CPU halts until the next interrupt (typically the clock) arrives, saving power:

```

⟨function arch_idlehands(raspberry pi1)(arm) 212c⟩≡ (703b)
TEXT arch_idlehands(SB), $-4
    MOVW    CPSR, R3
    ORR $(PsrDirq|PsrDfiq), R3, R1    /* splfhi */
    MOVW    R1, CPSR

    DSB
    MOVW    nrdy(SB), R0
    CMP $0, R0
    MCR.EQ  CpSC, 0, R0, C(CpCACHE), C(CpCACHEintr), CpCACHEwait
    DSB

    MOVW    R3, CPSR    /* splx */
    RET

```

Chapter 10

Time

```
⟨systab time syscalls 213a⟩≡ (41f)
[SLEEP]      syssleep,
[ALARM]      sysalarm,
```

10.1 Overview (arm)

This chapter covers the kernel's time infrastructure: how hardware timers generate interrupts, how the kernel multiplexes a single hardware timer into many software timers, and the time-related syscalls (**sleep** and **alarm**). We saw **hzclock()** in the Scheduling chapter; here we see how it gets called and what else runs on each clock tick. The Raspberry Pi has several hardware timers running at different frequencies. The kernel uses system timer 3 (1 MHz, 64-bit) for **hzclock** interrupts and **fastticks**, and the ARM timer (250 MHz, 32-bit) for performance measurement.

10.2 Time units

The kernel uses several time units internally. The conversion between them is a frequent source of bugs, so the author introduced explicit typedefs:

10.2.1 Seconds

```
⟨type Txxx 213b⟩≡ (672a) 213d>
typedef vlong    Tsec; // seconds
typedef int      Tms;  // milliseconds
typedef vlong    Tmicro; // microseconds
typedef vlong    Tnano; // nanoseconds
```

```
⟨macro MS2NS 213c⟩≡ (671b)
#define MS2NS(n) (((vlong)(n))*1000000LL)
```

10.2.2 Ticks

A tick is the kernel's basic time unit: **cpu->ticks** is incremented once per **hzclock()** interrupt, i.e., **Arch_HZ** times per second (every 10 ms on the Raspberry Pi). Ticks are coarse but cheap to read. The **TK2MS** macro converts ticks to milliseconds:

```
⟨type Txxx 213d⟩+≡ (672a) <213b 214f>
typedef vlong    Tval; // ticks
```

`<macro TK2MS 214a>≡` (671b)

```
#define TK2MS(x) ((x)*(1000/Arch_HZ))
```

`<function tk2ms 214b>≡` (673a)

```
/*
 * This tk2ms avoids overflows that the macro version is prone to.
 * It is a LOT slower so shouldn't be used if you're just converting
 * a delta.
 */
ulong
tk2ms(ulong ticks)
{
    uulong t, hz;

    t = ticks;
    hz = Arch_HZ;
    t *= 1000L;
    t = t/hz;
    ticks = t;
    return ticks;
}
```

`<function ms2tk 214c>≡` (673a)

```
ulong
ms2tk(ulong ms)
{
    <ms2tk() overflow approximation 214d>
    return (ms * Arch_HZ + 500) / 1000;
}
```

`<ms2tk() overflow approximation 214d>≡` (214c)

```
/* avoid overflows at the cost of precision */
if(ms >= 1000000000 / Arch_HZ)
    return (ms / 1000) * Arch_HZ;
```

10.2.3 Fastticks (ARM)

Fastticks are much finer-grained than ticks: on the Raspberry Pi, the system timer runs at 1 MHz, so one fasttick is one microsecond (vs. 10 ms for a tick—10,000 times finer). Fastticks are used for the software timer system (`Timer.twhen`) and for precise time measurement.

`<constant SystimerFreq(arm) 214e>≡` (714c)

```
SystimerFreq = 1*Mhz,
```

`<type Txxx 214f>+≡` (672a) <213d

```
typedef vlong Tfast; // fastticks
typedef uulong Tufast; // fastticks
```

`<function arch_us(arm) 214g>≡` (715b)

```
ulong // Tmicro
arch_us(void)
{
    <arch_us() if non-standard systimer frequency 230c>
    return arch_fastticks(nil);
}
```

```

⟨function clock_arch_fastticks(arm) 215a⟩≡ (715b)
Tufast
clock_arch_fastticks(uvlong *hz)
{
    Systimers *tn;
    ulong lo, hi;
    Tufast now;
    int s;

    ⟨arch_fastticks() if hz pointer given 215b⟩

    tn = (Systimers*)SYSTIMERS;
    s = arch_splhi();

    do{
        hi = tn->chi;
        lo = tn->clo;
    }while(tn->chi != hi);

    now = (uvlong)hi<<32 | lo;
    arch_splx(s);
    return now;
}

⟨arch_fastticks() if hz pointer given 215b⟩≡ (215a)
if(hz)
    *hz = SystimerFreq;

```

10.3 Clock interrupt (ARM)

The clock interrupt is the heartbeat of the kernel. On the Raspberry Pi, system timer 3 fires an IRQ (IRQclock) which calls `clockintr()`. This dismisses the interrupt by writing to the timer's control/status register, then calls the portable `timerintr()` which processes all elapsed software timers (including `hzclock`).

```

⟨struct Systimers(arm) 215c⟩≡ (715b)
// The order matters! the fields match the memory-mapped external registers.
struct Systimers {
    u32int cs;

    u32int clo;
    u32int chi;

    u32int c0;
    u32int c1;
    u32int c2;
    u32int c3;
};

⟨function clockintr(arm) 215d⟩≡ (715b)
static void
clockintr(Ureg *ureg, void*)
{
    Systimers *tn;

    tn = (Systimers*)SYSTIMERS;
    /* dismiss interrupt */
    tn->cs = 1<<3;
    // portable code

```



```

    timerintr(ureg, 0);
}

```

10.4 Timers

The hardware provides a single compare register per timer channel—it can only trigger at one point in time. But the kernel needs many independent timers (for `sleep`, alarms, periodic callbacks, etc.). The solution is a sorted linked list of software `Timer` structures: only the earliest one is programmed into the hardware. When it fires, `timerintr()` processes it and programs the next.

10.4.1 Timer

```

⟨struct Timer 216a⟩≡ (672a)
    struct Timer
    {
        /* Public interface */
        // enum<Timermode>
        int tmode;    /* See above */
        Tnano tns;    /* meaning defined by mode */ //nanosecond

        void (*tf)(Ureg*, Timer*);
        void *ta; // extra argument (generic pointer)

        /* Internal */
        Tfast twhen;    /* ns represented in fastticks */
        ILock; // LOCK ORDERING: ilock(timer); ilock(timers)

        ⟨Timer extra fields 216d⟩
    };

⟨enum timermode 216b⟩≡ (672a)
    /*
     * fasttick timer interrupts
     */
    enum Timermode
    {
        Trelative, /* timer programmed in ns from now */
        Tperiodic, /* periodic timer, period in ns */
    };

```

10.4.2 Timers and CPU timers

Each CPU has its own sorted timer list (`timers[cpuno]`), so timers fire on the CPU that created them. The list is protected by an `ILock` (not a regular `Lock`) because timer manipulation happens in interrupt context:

```

⟨struct Timers 216c⟩≡ (672a)
    struct Timers
    {
        // list<Timer> (next = Timer.tnext)
        Timer *head;
        // extra
        ILock;
    };

⟨Timer extra fields 216d⟩≡ (216a) 217a▷
    // list<Timer> of Timers.head
    Timer *tnext;

```

```

<Timer extra fields 217a>+≡ (216a) <216d
// option<ref<list<Timer>> Timers.head (of one of global timers[MAXCPUS])
Timers *tt; /* Timers queue this timer runs on */

<global timers 217b>≡ (673a)
static Timers timers[MAXCPUS];

```

10.4.3 tadd()/tdel()

tadd() inserts a timer into the sorted list (by **twhen**). If the new timer becomes the head (earliest), it returns the new deadline so the caller can reprogram the hardware. **tdel()** removes a timer:

```

<function tadd 217c>≡ (673a)
static Tfast
tadd(Timers *tt, Timer *nt)
{
    Timer *t, **last;

    /* Called with tt locked */
    <tadd() sanity check nt 217d>
    <tadd() set nt->twhen 217e>

    // insert_sorted(nt, tt, (a->twhen <=> b->twhen))
    for(last = &tt->head; t = *last; last = &t->tnext){
        if(t->twhen > nt->twhen)
            break;
    }
    nt->tnext = *last;
    *last = nt;

    nt->tt = tt;
    if(last == &tt->head)
        return nt->twhen;
    return 0;
}

```

```

<tadd() sanity check nt 217d>≡ (217c)
assert(nt->tt == nil);

```

```

<tadd() set nt->twhen 217e>≡ (217c)
switch(nt->tmode){
case Trelative:
    if(nt->tns <= 0)
        nt->tns = 1;
    nt->twhen = arch_fastticks(nil) + ns2fastticks(nt->tns);
    break;
case Tperiodic:
    assert(nt->tns >= 100000); /* At least 100 Âµs period */
    if(nt->twhen == 0){
        <tadd() look if can combine with identical timer 218a>
        else
            nt->twhen = arch_fastticks(nil);
    }
    nt->twhen += ns2fastticks(nt->tns);
    break;
default:
    panic("timer: impossible");
    break;
}

```

```

⟨tadd() look if can combine with identical timer 218a⟩≡ (217e)
/* look for another timer at same frequency for combining */
for(t = tt->head; t; t = t->tnext){
    if(t->tmode == Tperiodic && t->tns == nt->tns)
        break;
}
if (t)
    nt->twhen = t->twhen;

```

```

⟨function tdel 218b⟩≡ (673a)
static Tfast
tdel(Timer *dt)
{
    Timer *t, **last;
    Timers *tt;

    tt = dt->tt;
    ⟨tdel() sanity check tt 218c⟩
    // remove_list(dt, tt)
    for(last = &tt->head; t = *last; last = &t->tnext){
        if(t == dt){
            assert(dt->tt);
            dt->tt = nil;
            *last = t->tnext;
            break;
        }
    }
    if(last == &tt->head && tt->head)
        return tt->head->twhen;
    return 0;
}

```

```

⟨tdel() sanity check tt 218c⟩≡ (218b)
if (tt == nil)
    return 0; //todo: panic instead

```

10.4.4 timeradd()//timerdel()

timeradd() and timerdel() are the public interface. They lock the `Timer` first, then the `Timers` list (respecting the lock ordering), and call `tadd()`/`tdel()`. If the new timer becomes the earliest, `arch_timerset()` is called to reprogram the hardware:

```

⟨function timeradd 218d⟩≡ (673a)
/* add or modify a timer */
void
timeradd(Timer *nt)
{
    Timers *tt;
    Tfast when;

    /* Must lock Timer struct before Timers struct */
    ilock(nt);

    ⟨timeradd() if timer was already in a list 219a⟩
    tt = &timers[cpu->cpuno];
    ilock(tt);
    when = tadd(tt, nt);
    if(when)
        arch_timerset(when);
}

```

```

        iunlock(tt);
        iunlock(nt);
    }

⟨timeradd() if timer was already in a list 219a⟩≡ (218d)
    if(tt = nt->tt){
        ilock(tt);
        tdel(nt);
        iunlock(tt);
    }

⟨pexit() at the start, free resources 219b⟩≡ (130b) 225b▷
    if (up->tt)
        timerdel(up);

⟨function timerdel 219c⟩≡ (673a)
    void
    timerdel(Timer *dt)
    {
        Timers *tt;
        Tfast when;

        ilock(dt);
        if(tt = dt->tt){
            ilock(tt);
            when = tdel(dt);
            if(when && tt == &timers[cpu->cpuno])
                arch_timerset(tt->head->twhen);
            iunlock(tt);
        }
        iunlock(dt);
    }

```

10.4.5 arch_timerset() (arm)

`arch_timerset()` programs the hardware timer to fire at the given fasttick value by writing to system timer 3's compare register (c3). The period is clamped to avoid either firing too fast (overwhelming the CPU) or too slow (missing the next hzclock):

```

⟨function arch_timerset(arm) 219d⟩≡ (715b)
    void
    arch_timerset(Tfast next)
    {
        Systimers *tn;
        Tufast now;
        long period;

        now = arch_fastticks(nil);
        period = next - now;
        ⟨arch_timerset() sanitize period(arm) 220a⟩
        ⟨arch_timerset() if not cpu0(arm) 486c⟩
        else{
            tn = (Systimers*)SYSTIMERS;
            tn->c3 = (ulong)(now + period);
        }
    }

```

```
<arch_timerset() sanitize period(arm) 220a>≡ (219d)
```

```
if(period < MinPeriod)
    period = MinPeriod;
else if(period > MaxPeriod)
    period = MaxPeriod;
```

```
<constant MaxPeriod(arm) 220b>≡ (714c)
```

```
MaxPeriod = SystimerFreq / Arch_HZ,
```

```
<constant MinPeriod(arm) 220c>≡ (714c)
```

```
MinPeriod = SystimerFreq / (100*Arch_HZ),
```

10.4.6 timerintr()

timerintr() is the portable timer interrupt handler. It walks the sorted timer list, popping and calling every timer whose twhen has passed. Periodic timers are re-added to the list after firing. hzclock() is called last (via the callhzclock flag) to ensure all other timer callbacks complete first:

```
<interrupt callback timerintr 220d>≡ (673a)
```

```
void
timerintr(Ureg *u, Tval)
{
    Timer *t;
    Timers *tt;
    Tufast when, now;
    bool callhzclock = false;
    <timerintr() other locals 221b>

    <timerintr() start, stat 593a>
    tt = &timers[cpu->cpuno];
    now = arch_fastticks(nil);
    <timerintr() sanity check now 221a>
    ilock(tt);
    <timerintr() before loop, initialize count 221c>
    while((t = tt->head) != nil){
        /*
         * No need to ilock t here: any manipulation of t
         * requires tdel(t) and this must be done with a
         * lock to tt held. We have tt, so the tdel will
         * wait until we're done
         */
        when = t->twhen;
        if(when > now){
            arch_timerset(when);
            iunlock(tt);
            if(callhzclock)
                hzclock(u);
            return;
        }
        // else
        // pop_list(tt)
        tt->head = t->tnext;
        assert(t->tt == tt);
        t->tt = nil;
        <timerintr() in timers loop, stat 593b>
        iunlock(tt);

        // call the callback
        if(t->tf)
```

```

        (*t->tf)(u, t);
    else
        callhzclock = true;

        ilock(tt);
        if(t->tmode == Tperiodic)
            tadd(tt, t);
        <timerintr() end of timers loop, sanity check count 221e>
    }
    iunlock(tt); // unreachable?
}

<timerintr() sanity check now 221a>≡ (220d)
    if(now == 0)
        panic("timerintr: zero arch_fastticks()");

<timerintr() other locals 221b>≡ (220d)
    int count;

<timerintr() before loop, initialize count 221c>≡ (220d)
    count = Maxtimerloops;

<constant Maxtimerloops 221d>≡ (673a)
    Maxtimerloops = 20*1000,

<timerintr() end of timers loop, sanity check count 221e>≡ (220d)
    if (--count <= 0) {
        count = Maxtimerloops;
        iprint("timerintr: probably stuck in while loop; "
            "scrutinise clock.c or use faster cycle "
            "counter\n");
    }
}

```

10.4.7 Clock callbacks: addclock0link()

`addclock0link()` registers a periodic callback on CPU 0's timer list. Device drivers use this to run polling functions at regular intervals (e.g., checking the Ethernet controller, updating the console). If `ms` is 0, the callback runs at the `hzclock` frequency (every 10 ms):

```

<function addclock0link 221f>≡ (673a)
    Timer*
    addclock0link(void (*f)(void), Tms ms)
    {
        Timer *nt;
        Tfast when;

        <addclock0link() sanity check timersinit 222a>

        nt = malloc(sizeof(Timer));
        <addclock0link() sanity check nt 222b>
        <addclock0link() sanitize ms 222c>

        nt->tns = (Tnano)ms * 1000000LL; // MS2NS(ms)
        nt->tmode = Tperiodic;
        nt->tt = nil;
        nt->tf = (void (*)(Ureg*, Timer*))f;

        // those clock callbacks are all done on the bootstrap processor
        ilock(&timers[0]);
    }
}

```

```

    when = tadd(&timers[0], nt);
    if(when)
        arch_timeraset(when);
    iunlock(&timers[0]);
    return nt;
}

⟨addclock0link() sanity check timersinit 222a⟩≡ (221f)
    if(!timersinit)
        panic("addclock0link: timersinit not called yet");

⟨addclock0link() sanity check nt 222b⟩≡ (221f)
    if(nt == nil)
        error(Enomem);

⟨addclock0link() sanitize ms 222c⟩≡ (221f)
    /* Synchronize to hztimer if ms is 0 */
    if(ms == 0)
        ms = 1000 / Arch_HZ; // tk2ms(1)

```

10.4.8 Delay (ARM)

`arch_delay()` and `arch_microdelay()` are busy-wait loops used by device drivers that need to wait for hardware to respond (e.g., after writing a control register). They spin-wait on the system timer counter—no interrupt is involved:

```

⟨function clock_arch_microdelay(arm) 222d⟩≡ (715b)
    void
    clock_arch_microdelay(int n) // Tmicro
    {
        Systimers *tn;
        u32int now, diff;

        diff = n + 1;
        tn = (Systimers*)SYSTIMERS;
        now = tn->clo;
        while(tn->clo - now < diff)
            ;
    }

⟨function clock_arch_delay(arm) 222e⟩≡ (715b)
    void
    clock_arch_delay(Tms n)
    {
        while(--n >= 0)
            arch_microdelay(1000);
    }

```

10.5 syssleep()

The `sleep(ms)` syscall puts the calling process to sleep for `ms` milliseconds. It works by combining the Rendez/`sleep()`/`wakeup()` mechanism from the Scheduling chapter with a one-shot timer: `tsleep()` adds a timer that will call `twakeup()` after the requested delay, then calls `sleep()` with a condition that is always false (so the process sleeps until the timer wakes it).

```

⟨Proc synchronization fields 222f⟩+≡ (49f) <203f 422a>
    Rendez sleep; /* place for syssleep/debug/tsleep */

```

```

⟨syscall sleep 223a⟩≡ (679a)
// int sleep(long millisecs);
long
sysssleep(ulong* arg)
{
    int n;
    n = arg[0];
    if(n <= 0) {
        ⟨sysssleep() optional edfyield() for real-time scheduling 503a⟩
        yield();
        return 0;
    }
    ⟨sysssleep() sanitize n 223b⟩
    tsleep(&up->sleepr, returnfalse, 0, n);
    return 0;
}

⟨sysssleep() sanitize n 223b⟩≡ (223a)
if(n < TK2MS(1))
    n = TK2MS(1);

⟨function tsleep 223c⟩≡ (675c)
void
proc_tsleep(Rendez *r, int (*fn)(void*), void *arg, ulong ms)
{
    ⟨tsleep() sanity check up timer 223e⟩
    up->tns = MS2NS(ms);
    up->tmode = Trelative;
    up->tf = twakeup;

    up->ta = up; // so can access r from wakeup
    up->trend = r;

    up->tfn = fn; // needed?

    timeradd(up);

    if(waserror()){
        timerdel(up);
        nexterror();
    }
    sleep(r, tfn, arg);
    // back after sleep
    ⟨tsleep() reset timer after sleep 223d⟩
    poperror();
}

⟨tsleep() reset timer after sleep 223d⟩≡ (223c)
if(up->tt)
    timerdel(up);
up->twhen = 0;

⟨tsleep() sanity check up timer 223e⟩≡ (223c)
if (up->tt){
    print("tsleep: timer active: mode %d, tf %#p\n", up->tmode, up->tf);
    timerdel(up);
}

⟨Proc other fields 223f⟩+≡ (49f) <141c 224a>
Timer; /* For tsleep and real-time */

```



```

⟨Proc other fields 224a⟩+≡ (49f) ◁223f 225a▷
    Rendez *trend;
    int (*tfn)(void*);

⟨function tfn 224b⟩≡ (675c)
    static int
    tfn(void *arg)
    {
        return up->trend == nil || up->tfn(arg);
    }

⟨function twakeup 224c⟩≡ (675c)
    void
    twakeup(Ureg*, Timer *t)
    {
        Proc *p;
        Rendez *trend;

        p = t->ta;
        trend = p->trend;
        p->trend = nil;
        if(trend)
            wakeup(trend);
    }

```

10.6 Alarms

Alarms are a simpler, coarser mechanism than timers. The `alarm(ms)` syscall arranges for a “sys: alarm” note to be posted to the calling process after `ms` milliseconds. Unlike `sleep()`, the process continues running—the note arrives asynchronously, like a UNIX `SIGALRM`. Alarms are stored as a sorted linked list of `Proc` pointers, checked on every `hzclock()` tick by `checkalarms()`:

```

⟨struct Alarms 224d⟩≡ (668c)
    struct Alarms
    {
        // sorted_list<ref<Proc> (next = Proc.palarm, sort_key = Proc.alarm)
        Proc *head;

        // Extra
        QLock;
    };

⟨Proc extra fields 224e⟩+≡ (49f) ◁192d
    // Alarms.head chain?
    Proc *palarm; /* Next alarm time */

⟨global alarms 224f⟩≡ (672b)
    static Alarms alarms;

```

10.7 sysalarm()

```

⟨syscall alarm 224g⟩≡ (679a)
    // long alarm(unsigned long millisecs);
    long
    sysalarm(ulong* arg)
    {
        return procalarm(arg[0]);
    }

```

```

<Proc other fields 225a>+≡ (49f) <224a 269c>
    // option<ticks>
    ulong alarm;    /* Time of call */ // Tval ticks

<pexit() at the start, free resources 225b>+≡ (130b) <219b 433e>
    up->alarm = 0;

<function procalarm 225c>≡ (672b)
    ulong
    procalarm(ulong time) // Tms
    {
        Proc *l, *f;
        ulong when, old; // Tval ticks

        <procalarm() compute remaining time in old alarm 225d>
        <procalarm() if time is zero, reset alarm 226a>
        when = ms2tk(time) + CPUS(0)->ticks;
        <procalarm() sanitize when 226b>

        qlock(&alarms);
        l = &alarms.head;
        // remove_list(up, alarms)
        for(f = *l; f; f = f->palarm) {
            if(up == f){
                *l = up->palarm;
                break;
            }
            l = &f->palarm;
        }
        up->palarm = nil;

        // add_sorted_list(up, alarms)
        if(alarms.head) {
            l = &alarms.head;
            for(f = *l; f; f = f->palarm) {
                if((long)(f->alarm - when) >= 0) {
                    up->palarm = f;
                    *l = up;
                    goto done;
                }
                l = &f->palarm;
            }
            *l = up;
        }
        else
            alarms.head = up;

    done:
        up->alarm = when;
        qunlock(&alarms);

        return old;
    }

<procalarm() compute remaining time in old alarm 225d>≡ (225c)
    if(up->alarm)
        old = tk2ms(up->alarm - CPUS(0)->ticks);
    else
        old = 0;

```

```

⟨procalarm() if time is zero, reset alarm 226a⟩≡ (225c)
    if(time == 0) {
        up->alarm = 0;
        return old;
    }

```

```

⟨procalarm() sanitize when 226b⟩≡ (225c)
    if(when == 0) /* ticks have wrapped to 0? */
        when = 1; /* distinguish a wrapped alarm from no alarm */

```

10.7.1 alarmkproc()

The alarm kernel process runs in an infinite loop: it sleeps until `checkalarms()` (called from `hzclock()`) wakes it, then walks the sorted alarm list posting “alarm” notes to processes whose alarm time has passed. The indirection through a kernel process avoids doing the `postnote()` work inside the interrupt handler:

```

⟨global alarmr 226c⟩≡ (672b)
    static Rendez alarmr;

```

```

⟨kernel process alarmkproc 226d⟩≡ (672b)
    // Kernel Process for alarm management
    void
    alarmkproc(void*)
    {
        Proc *rp;
        ulong now;

        for(;;){
            now = CPUS(0)->ticks;

            qlock(&alarms);
            /*
             * the odd test of now vs. rp->alarm is to cope with
             * now wrapping around.
             */
            while((rp = alarms.head) && (long)(now - rp->alarm) >= 0){
                if(rp->alarm != 0L){
                    if(canqlock(&rp->debug)){
                        if(!waserror()){
                            postnote(rp, 0, "alarm", NUser);
                            poperror();
                        }
                        qunlock(&rp->debug);
                        rp->alarm = 0L;
                    }else
                        break;
                }
                alarms.head = rp->palarm;
            }
            qunlock(&alarms);

            sleep(&alarmr, returnfalse, 0);
        }
    }

```

10.7.2 checkalarms()

`checkalarms()` is called on every clock tick. It just checks whether the earliest alarm has expired and, if so, wakes up `alarmkproc()` to do the actual work. This keeps the interrupt handler fast:

```
<hzclock() check the alarms 227a>≡ (207a)
    checkalarms();
```

```
<function checkalarms 227b>≡ (672b)
/*
 * called every clock tick
 */
void
checkalarms(void)
{
    Proc *p;
    ulong now;

    p = alarms.head;
    now = CPUS(0)->ticks;

    if(p && (long)(now - p->alarm) >= 0)
        wakeup(&alarmr);
}
```

10.8 Time of day

The time of day (wall clock) is maintained by the TOD structure, which converts fastticks into nanoseconds since the epoch using a fixed-point multiplier. The multiplier is calibrated from `cpu->cpuhz` (the fasttick frequency). A periodic callback (`todfix`) prevents the 64-bit intermediate values from overflowing, and `todset()` allows the time to be adjusted gradually (via a delta applied over several ticks) to avoid jumps.

```
<struct TOD 227c>≡ (678)
struct TOD {
    bool init;          /* true if initialized */

    ulong    cnt;

    uulong   multiplier; /* ns = off + (multiplier*ticks)>>31 */
    uulong   divider;    /* ticks = (divider*(ns-off))>>31 */
    uulong   umultiplier; /* Âµs = (Âµmultiplier*ticks)>>31 */
    uulong   udivider;    /* ticks = (Âµdivider*Âµs)>>31 */

    vlong    hz;         /* frequency of fast clock */
    vlong    last;       /* last reading of fast clock */

    vlong    off;        /* offset from epoch to last */
    vlong    lasttime;   /* last return value from todget */
    vlong    delta;      /* add 'delta' each slow clock tick from sstart to send */

    ulong    sstart;     /* ... */
    ulong    send;       /* ... */

    // Extra
    Lock;
};

<global tod 227d>≡ (678)
struct TOD tod;
```

<devcons.c Exxx errors 228a>≡ (607)

```
char *Ebadtimectl = "bad time control";
```

<constant VLNUMSIZE 228b>≡ (607)

```
VLNUMSIZE= 22,
```

<function todinit 228c>≡ (678)

```
void
todinit(void)
{
    if(tod.init)
        return;
    ilock(&tod);
    tod.init = true;          /* prevent reentry via arch_fastticks */
    tod.last = arch_fastticks((uvlong *)&tod.hz);
    iunlock(&tod);
    todsetfreq(tod.hz);
    addclock0link(todfix, 100);
}
```

<function todsetfreq 228d>≡ (678)

```
/*
 * calculate multiplier
 */
void
todsetfreq(vlong f)
{
    if (f <= 0)
        panic("todsetfreq: freq %lld <= 0", f);
    ilock(&tod);
    tod.hz = f;

    /* calculate multiplier for time conversion */
    tod.multiplier = mk64fract(TODFREQ, f);
    tod.divider    = mk64fract(f, TODFREQ) + 1;
    tod.umultiplier = mk64fract(MicroFREQ, f);
    tod.udivider   = mk64fract(f, MicroFREQ) + 1;
    iunlock(&tod);
}
```

<function todset 228e>≡ (678)

```
/*
 * Set the time of day struct
 */
void
todset(vlong t, vlong delta, int n)
{
    if(!tod.init)
        todinit();

    ilock(&tod);
    if(t >= 0){
        tod.off = t;
        tod.last = arch_fastticks(nil);
        tod.lasttime = 0;
        tod.delta = 0;
        tod.sstart = tod.send;
    } else {
        if(n <= 0)
            n = 1;
```

```

    n *= Arch_HZ;
    if(delta < 0 && n > -delta)
        n = -delta;
    if(delta > 0 && n > delta)
        n = delta;
    if (n == 0) {
        iprint("todset: n == 0, delta == %lld\n", delta);
        delta = 0;
    } else
        delta /= n;
    tod.sstart = CPUS(0)->ticks;
    tod.send = tod.sstart + n;
    tod.delta = delta;
}
iunlock(&tod);
}

```

⟨function todget 229⟩≡ (678)

```

/*
 * get time of day
 */
vlong
todget(vlong *ticksp)
{
    uulong x;
    vlong ticks, diff;
    ulong t;

    if(!tod.init)
        todinit();

    /*
     * we don't want time to pass twixt the measuring of arch_fastticks
     * and grabbing tod.last. Also none of the vlongs are atomic so
     * we have to look at them inside the lock.
     */
    ilock(&tod);
    tod.cnt++;
    ticks = arch_fastticks(nil);

    /* add in correction */
    if(tod.sstart != tod.send){
        t = CPUS(0)->ticks;
        if(t >= tod.send)
            t = tod.send;
        tod.off = tod.off + tod.delta*(t - tod.sstart);
        tod.sstart = t;
    }

    /* convert to epoch */
    diff = ticks - tod.last;
    if(diff < 0)
        diff = 0;
    arch_mul64fract(&x, diff, tod.multiplier);
    x += tod.off;

    /* time can't go backwards */
    if(x < tod.lasttime)
        x = tod.lasttime;
    else

```

```

        tod.lasttime = x;

        iunlock(&tod);

        if(ticksp != nil)
            *ticksp = ticks;

        return x;
    }

⟨function todfix 230a⟩≡ (678)
/*
 * called regularly to avoid calculation overflows
 */
static void
todfix(void)
{
    vlong ticks, diff;
    uvlong x;

    ticks = arch_fastticks(nil);

    diff = ticks - tod.last;
    if(diff > tod.hz){
        ilock(&tod);

        /* convert to epoch */
        arch_mul64fract(&x, diff, tod.multiplier);
    if(x > 30000000000ULL) iprint("todfix %lld\n", x);
        x += tod.off;

        /* protect against overflows */
        tod.last = ticks;
        tod.off = x;

        iunlock(&tod);
    }
}

⟨function seconds 230b⟩≡ (678)
long
seconds(void)
{
    return (vlong)todget(nil) / TODFREQ;
}

⟨arch_us() if non-standard systimer frequency 230c⟩≡ (214g)
if(SystimerFreq != 1*Mhz)
    return fastticks2us(arch_fastticks(nil));

⟨function fastticks2us 230d⟩≡ (678)
uvlong
fastticks2us(uvlong ticks)
{
    uvlong res;

    if(!tod.init)
        todinit();
    arch_mul64fract(&res, ticks, tod.umultiplier);
    return res;
}

```

```

⟨function ns2fastticks 231a⟩≡ (678)
/*
 * convert nanoseconds to fast ticks
 */
uvlong
ns2fastticks(uvlong ns)
{
    uvlong res;

    if(!tod.init)
        todinit();
    arch_mul64fract(&res, ns, tod.divider);
    return res;
}

```

```

⟨function mk64fract 231b⟩≡ (678)
/*
 * Make a 64 bit fixed point number that has a decimal point
 * to the left of the low order 32 bits. This is used with
 * arch_mul64fract for converting twixt nanoseconds and arch_fastticks.
 *
 * multiplier = (to<<32)/from
 */
uvlong
mk64fract(uvlong to, uvlong from)
{
    return (to<<32) / from;
}

```

10.9 sysnsec()

10.10 /dev/time

Reading /dev/time returns the current time in seconds since the epoch (used by `date(1)`). Writing to it sets the time—this is how the system clock is initialized from the network or a real-time clock at boot:

```

⟨consread() cases 231c⟩+≡ (345c) <179b 232b>
    case Qtime:
        return readtime((ulong)offset, buf, n);

```

```

⟨global fasthz 231d⟩≡ (607)
    vlong    fasthz;

```

```

⟨conswrite() cases 231e⟩+≡ (346a) <188c 232c>
    case Qtime:
        if(!iseve())
            error(Eperm);
        return writetime(a, n);

```

```

⟨function readtime 231f⟩≡ (607)
/*
 * like the old #c/time but with added info. Return
 *
 * secs    nanosecs    fastticks    fasthz
 */
static int
readtime(ulong off, char *buf, int n)
{

```



```

vlong  nsec, ticks;
long  sec;
char  str[7*NUMSIZE];

nsec = todget(&ticks);
if(fasthz == 0LL)
    arch_fastticks((uvlong*)&fasthz);
sec = nsec/1000000000ULL;
snprint(str, sizeof(str), "%*lud %*llud %*llud %*llud ",
        NUMSIZE-1, sec,
        VLNUMSIZE-1, nsec,
        VLNUMSIZE-1, ticks,
        VLNUMSIZE-1, fasthz);
return readstr(off, buf, n, str);
}

```

```

⟨function writetime 232a⟩≡ (607)
/*
 * set the time in seconds
 */
static int
writetime(char *buf, int n)
{
    char b[13];
    long i;
    vlong now;

    if(n >= sizeof(b))
        error(Ebadtimectl);
    strncpy(b, buf, n);
    b[n] = 0;
    i = strtol(b, 0, 0);
    if(i <= 0)
        error(Ebadtimectl);
    now = i*1000000000LL;
    todset(now, 0, 0);
    return n;
}

```

10.11 /dev/bintime

```

⟨consread() cases 232b⟩+≡ (345c) ◁231c 445a▷
case Qbintime:
    return readbintime(buf, n);

```

```

⟨conswrite() cases 232c⟩+≡ (346a) ◁231e 347f▷
case Qbintime:
    if(!iseve())
        error(Eperm);
    return writebintime(a, n);

```

```

⟨function readbintime 232d⟩≡ (607)
/*
 * read binary time info.  all numbers are little endian.
 * ticks and nsec are synchronized.
 */
static int
readbintime(char *buf, int n)

```

```

{
    int i;
    vlong nsec, ticks;
    byte *b = (byte*)buf;

    i = 0;
    if(fasthz == 0LL)
        arch_fastticks((uvlong*)&fasthz);
    nsec = todget(&ticks);
    if(n >= 3*sizeof(uvlong)){
        vlong2le(b+2*sizeof(uvlong), fasthz);
        i += sizeof(uvlong);
    }
    if(n >= 2*sizeof(uvlong)){
        vlong2le(b+sizeof(uvlong), ticks);
        i += sizeof(uvlong);
    }
    if(n >= 8){
        vlong2le(b, nsec);
        i += sizeof(vlong);
    }
    return i;
}

```

⟨function writebintime 233⟩≡

(607)

```

/*
 * set any of the following
 * - time in nsec
 * - nsec trim applied over some seconds
 * - clock frequency
 */
static int
writebintime(char *buf, int n)
{
    byte *p;
    vlong delta;
    long period;

    n--;
    p = (byte*)buf + 1;
    switch(*buf){
    case 'n':
        if(n < sizeof(vlong))
            error(Ebadtimectl);
        le2vlong(&delta, p);
        todset(delta, 0, 0);
        break;
    case 'd':
        if(n < sizeof(vlong)+sizeof(long))
            error(Ebadtimectl);
        p = le2vlong(&delta, p);
        le2long(&period, p);
        todset(-1, delta, period);
        break;
    case 'f':
        if(n < sizeof(uvlong))
            error(Ebadtimectl);
        le2vlong(&fasthz, p);
        if(fasthz <= 0)
            error(Ebadtimectl);
    }
}

```

```

        todsetfreq(fasthz);
        break;
    }
    return n;
}

⟨global uvorder 234a⟩≡ (607)
    static uvlong uvorder = 0x0001020304050607ULL;

⟨function le2vlong 234b⟩≡ (607)
    static byte*
    le2vlong(vlong *to, byte *f)
    {
        byte *t, *o;
        int i;

        t = (byte*)to;
        o = (byte*)&uvorder;
        for(i = 0; i < sizeof(vlong); i++)
            t[o[i]] = f[i];
        return f+sizeof(vlong);
    }

⟨function vlong2le 234c⟩≡ (607)
    static byte*
    vlong2le(byte *t, vlong from)
    {
        byte *f, *o;
        int i;

        f = (byte*)&from;
        o = (byte*)&uvorder;
        for(i = 0; i < sizeof(vlong); i++)
            t[i] = f[o[i]];
        return t+sizeof(vlong);
    }

⟨global order 234d⟩≡ (607)
    static long order = 0x00010203;

⟨function le2long 234e⟩≡ (607)
    static byte*
    le2long(long *to, byte *f)
    {
        byte *t, *o;
        int i;

        t = (byte*)to;
        o = (byte*)&order;
        for(i = 0; i < sizeof(long); i++)
            t[o[i]] = f[i];
        return f+sizeof(long);
    }

```

10.12 Watchdog

A watchdog timer is a hardware safety net: if the kernel does not periodically “pet” it (reset its counter), the hardware assumes the system is hung and forces a reboot. On the Raspberry Pi, the watchdog is also used to trigger a deliberate reboot via `arch_reboot()`.

Chapter 11

Initialization

11.1 Booting the kernel

The boot process has three stages: the Broadcom firmware loads the kernel binary at physical 0x8000, the assembly startup code (`_start`) sets up a minimal environment (SVC mode, stack, basic page tables), and then `main()` in C initializes every kernel subsystem we have seen in the previous chapters—memory, processes, devices, timers—and creates the first user process.

11.1.1 Loading the kernel in physical memory (ARM)

11.1.2 First instructions (ARM)

The very first instructions set up the minimum needed to run C code: the static base register R12 (adjusted for physical addresses since the MMU is still off), SVC mode with interrupts disabled, and a stack at the top of the `Cpu` structure at `CPUADDR`. Then it jumps to `armstart` to set up the page tables:

```
<function _start(arm) 235>≡ (703a)
/*
 * on bcm2836, only cpu0 starts here
 * other cpus enter at cpureset in armv7.s
 */
TEXT _start(SB), 1, $-4
/*
 * load physical base for SB addressing while mmu is off
 * keep a handy zero in R0 until first function call
 */
MOVW    $setR12(SB), R12
SUB     $KZERO, R12
MOVW    $0, R0

/*
 * SVC mode, interrupts disabled
 */
MOVW    $(PsrDirq|PsrDfiq|PsrMsvc), R1
MOVW    R1, CPSR

/*
 * start stack at top of 'cpu' (physical addr)
 */
MOVW    $PADDR(CPUADDR+CPUSIZE-4), R13

/*
 * do arch-dependent startup (no return)
```

```

    */
BL    armstart(SB)

// Unreached
B     0(PC)
RET

```

11.1.3 Basic pagination (ARM)

Before calling C `main()`, `armstart` sets up a minimal page table so that the MMU can be enabled. It first disables the MMU and caches, clears the memory between `CPUADDR` and `KTZERO` (which will hold the `Cpu` struct and the L1/L2 tables), then calls `mmuinit()` to populate the L1 page directory with three sets of mappings:

1. RAM at KZERO: pa 0 -> va KZERO (Section entries)
2. Identity map: pa 0 -> va 0 (so PC works after MMU on)
3. I/O at VIRTIO: pa soc.physio -> va VIRTIO
4. Vectors: pa 0 -> va HVECTORS (via L2, Coarse entry)

After enabling the MMU and caches, `armstart` adjusts `R12` (SB) and `R13` (SP) to their virtual (KZERO) addresses and jumps to `_startpg`, which calls C `main()`.

```

⟨function armstart(raspberry pi1)(arm) 236⟩≡ (703b)
TEXT armstart(SB), 1, $-4

```

```

/*
 * disable the mmu and L1 caches
 * invalidate caches and tlb
 */
MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
BIC $(CpCmmu), R1
⟨armstart() disable features(arm) 468c⟩
MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl

⟨armstart() invalidate caches(arm) 468d⟩
MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinv
ISB

/*
 * clear mach and page tables
 */
MOVW    $PADDR(CPUADDR), R1
MOVW    $PADDR(KTZERO), R2
_ramZ:
MOVW    R0, (R1)
ADD $4, R1
CMP R1, R2
BNE _ramZ

/*
 * start stack at top of CPUADDR (physical addr)
 * set up page tables for kernel
 */
MOVW    $PADDR(CPUADDR+CPUSIZE-4), R13 // done already in start.s
MOVW    $PADDR(L1), R0

BL    mmuinit(SB)

/*

```

```

    * set up domain access control and page table base
    */
    MOVW    $Client, R1
    MCR CpSC, 0, R1, C(CpDAC), C(0)

    MOVW    $PADDR(L1), R1
    MCR CpSC, 0, R1, C(CpTTB), C(0)

/*
    * enable caches, mmu, and high vectors
    */
    MRC CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
    ORR $(CpCmmu|CpChv), R0
    <armstart() reenable features(arm) 468e>
    MCR CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
    ISB

/*
    * switch SB, SP, and PC into KZERO space
    */
    MOVW    $setR12(SB), R12
    MOVW    $(CPUADDR+CPUSIZE-4), R13

    MOVW    $_startpg(SB), R15

<constant L1(arm) 237a>≡ (709b)
#define L1            (KZERO+0x4000)    /* ttb ptes: 16KiB aligned */

<constant L2(arm) 237b>≡ (709b)
#define L2            (KZERO+0x3000)    /* L2 ptes for vectors etc */

<function mmuinit(arm) 237c>≡ (711a)
/*
    * Set up initial PTEs for this cpu (called with mmu off)
    */
void
mmuinit(void *a)
{
    PTE *l1, *l2;
    phys_addr pa;
    kern_addr va;

    l1 = (PTE*)a; // PADDR(L1) for cpu0

/*
    * map all of ram at KZERO
    */
    va = KZERO;
    for(pa = 0; pa < soc.dramsize; pa += MiB, va += MiB)
    {
        l1[L1X(va)] = pa|Dom0|L1AP(Krw)|Section|L1ptedramattrs;
    }

/*
    * identity map first MB of ram so mmu can be enabled
    */
    // ???
    l1[L1X(0)] = 0|Dom0|L1AP(Krw)|Section|L1ptedramattrs;

/*

```

```

    * map i/o registers
    */
va = VIRTIO;
for(pa = soc.physio; pa < soc.physio + IOSIZE; pa += MiB, va += MiB){
    // No L1ptedramattrs (Cached | Buffered) here, because volatile!
    l1[L1X(va)] = pa|Dom0|L1AP(Krw)|Section;
}
⟨mmuinit() set armlocal virtual memory mapping(arm) 475g⟩

/*
 * double map exception vectors at top of virtual memory
 */
// ???
l2 = (PTE*)PADDR(L2);
va = HVECTORS;
l1[L1X(va)] = (uintptr)l2|Dom0|Coarse;
l2[L2X(va)] = L2AP(Krw)|Small|L2ptedramattrs;
}

```

11.1.4 Jumping to C main() (ARM)

```

⟨function _startpg(raspberry pi1)(arm) 238a⟩≡ (703b)
TEXT _startpg(SB), 1, $-4

⟨_startpg() enable cycle counter(raspberry pi1)(arm) 448f⟩
/*
 * call main and loop forever if it returns
 */
BL main(SB)

// Unreached
B 0(PC)
BL _div(SB) /* hack to load _div, etc. */

```

11.2 Kernel initialization (ARM)

`main()` initializes every kernel subsystem in the right order—a chicken-and-egg problem, since each subsystem depends on others. The sequence is roughly: (1) CPU and memory detection, (2) kernel memory allocator (`xinit`), (3) console output (`uartconsinit`, `screeninit`), (4) exception vectors and clock interrupts, (5) process and page allocators (`procinit`, `pageinit`), (6) device drivers (`chandeveset`), (7) first user process (`userinit`), (8) enter the scheduler (`schedinit`).

```

⟨function main(arm) 238b⟩≡ (704c)
void
main(void)
{
    ⟨main() locals(arm) 250c⟩

    ⟨main() initial assignments for backward deps(arm) ??⟩

    ⟨main() clear bss(arm) 239a⟩

    // Let's go!

    cpu = (Cpu*)CPUADDR;
    arch__cpu0init(); // cpu0 initialization (calls arch__cpuinit())
    mmuinit1((void*)L1); // finish mmu initialization started in mmuinit0

```

```

machon(0); // enable cpu0 for scheduling

⟨main() parsing options and boot arguments(arm) 241c⟩
arch__confinit(); /* figures out amount of memory */
xinit();

uartconsinit();

arch__screeninit(); // screenputs = swconsole_screenputs
quotefmtinstall(); // libc printf initialization

print("\nPlan 9 from Bell Labs\n"); // yeah!

⟨main() print board and firmware information(arm) 250d⟩
/* set clock rate to arm_freq from config.txt (default pi1:700Mhz pi2:900MHz) */
setclkrate(ClkArm, 0);

arch__trapinit();
clockinit(); // enable timer interrupts

//TODO? kbdqinit(); // setup kbdq
lineqinit(); // setup lineq

timersinit(); // set initial timer for timer interrupt
swcursor_init();

arch_cpuidprint();
archreset();

procinit();
imageinit();

links();

// initialize all devices
chandevreset(); /* most devices are discovered here */

pageinit(); // setup palloc.pages and swapalloc.highwater
swapinit(); // setup swapalloc

// let's craft our first process (that will then exec("boot/boot"))
arch__userinit();

⟨main() start the other processors(arm) 478e⟩

// schedule the only ready user process (the one created by userinit)
schedinit();

assert(0); /* shouldn't have returned */
}

⟨main() clear bss(arm) 239a⟩≡ (238b)
memset(edata, 0, end - edata); /* clear bss */

⟨lib.h xxx decl 239b⟩≡ (656e)
extern char etext[];
//@Scheck: Assembly, not dead used by 386/l.s and arm/main.c
extern char edata[];
extern char end[];

```


11.2.1 cpu initialization

`arch__cpu0init()` initializes the boot CPU's `Cpu` structure: sets `cpuno` to 0, registers it in the `cpus` array, and clears up (no process running yet). `machon()` then marks this CPU as active in the `Active` bitmap so the scheduler knows it exists:

```
<function arch__cpu0init(arm) 240a>≡ (704c)
void
arch__cpu0init(void)
{
    conf.ncpu = 0; // set in machon() instead (machon() is called after cpuint)

    cpu->cpuno = 0;
    cpus[cpu->cpuno] = cpu;

    arch__cpuinit();
    active.exiting = false;

    up = nil;
}
```

```
<function arch__cpuinit(arm) 240b>≡ (704c)
void
arch__cpuinit(void)
{
    Cpu *cpu0;

    cpu->ticks = 1;
    cpu->perf.period = 1;

    cpu0 = CPUS(0);
    <arch__cpuinit() it not cpu0 483a>
}
```

```
<function machon(arm) 240c>≡ (704c)
/* enable scheduling of this cpu */
void
machon(uint xcpu)
{
    ulong cpubit;

    cpubit = 1 << xcpu;
    lock(&active);
    if ((active.cpus & cpubit) == 0) { /* currently off? */
        conf.ncpu++;
        active.cpus |= cpubit;
    }
    unlock(&active);
}
```

```
<function arch_cpuidprint(raspberry pi1)(arm) 240d>≡ (707a)
void
arch_cpuidprint(void)
{
    char name[64];

    cputype2name(name, sizeof name);
    arch_delay(50); /* let uart catch up */
    print("cpu%d: %dMHz ARM %s\n", cpu->cpuno, cpu->cpumhz, name);
}
```

```

⟨function cputype2name(raspberry pi1)(arm) 241a⟩≡ (707a)
char*
cputype2name(char *buf, int size)
{
    seprint(buf, buf + size, "1176JZF-S");
    return buf;
}

```

11.2.2 Memory detection (ARM)

The Raspberry Pi does not have a BIOS memory map like x86. Instead, `getramsize()` queries the Broadcom GPU via the mailbox interface (`vcreq`) to find the base and limit of ARM-accessible RAM:

```

⟨function getramsize(arm) 241b⟩≡ (691d)
/*
 * Get ARM ram
 */
void
getramsize(Confmem *mem)
{
    u32int buf[2];

    if(vcreq(TagGetram, buf, 0, sizeof buf) != sizeof buf)
        return;
    mem->base = buf[0];
    mem->limit = buf[1];
}

```

11.2.3 conf initialization (ARM)

`arch__confinit()` determines how much physical memory is available (via the Broadcom mailbox interface), splits it into kernel and user portions (80% user by default), and computes the sizes of all the large kernel data structures: the Proc array, Page array, KImage array, swap map, and malloc pools.

```

⟨main() parsing options and boot arguments(arm) 241c⟩≡ (238b)
//optionsinit("/boot/boot boot"); // setup values for getconf()
//ataginit((Atag*)BOOTARGS); // from bootloader config
// example of manual config:
// TODO confname("console") = 1?

⟨Conf other fields 241d⟩+≡ (46a) <182a 403f>
ulong npage;    /* total physical pages of memory */

ulong upages;   /* user page pool */
// kpages = npage - upages

ulong nimage;   /* number of page cache image headers */
ulong ialloc;   /* max interrupt time allocation in bytes */

⟨global memsize(arm) 241e⟩≡ (704c)
// used also in mmu.c
ulong memsize = 128*1024*1024;

```

```

⟨function arch__confinit(arm) 242⟩≡ (704c)
void
arch__confinit(void)
{
    int i;
    char *p;
    phys_addr pa;
    ulong kpages;
    ulong kmem;

    ⟨arch__confinit() if maxmem boot parameter(arm) 243a⟩
    // simpler than for x86 :)
    getramsize(&conf.mem[0]);
    ⟨arch__confinit() sanity check conf.mem[0]/(arm) 243b⟩

    conf.npage = 0;
    pa = PADDR(PGROUND(PTR2UINT(end)));

    /*
     * we assume that the kernel is at the beginning of one of the
     * contiguous chunks of memory and fits therein.
     */
    for(i=0; i<nelem(conf.mem); i++){
        /* take kernel out of allocatable space */
        if(pa > conf.mem[i].base && pa < conf.mem[i].limit)
            conf.mem[i].base = pa;

        conf.mem[i].npage = (conf.mem[i].limit - conf.mem[i].base)/BY2PG;
        conf.npage += conf.mem[i].npage;
    }

    conf.upages = (conf.npage*80)/100;
    kpages = conf.npage - conf.upages;

    /* set up other configuration parameters */
    conf.ialloc = (kpages/2)*BY2PG; // max bytes for iallocb

    conf.nproc = 100 + ((conf.npage*BY2PG)/MB)*5;
    if(cpuserver)
        conf.nproc *= 3;
    if(conf.nproc > 2000)
        conf.nproc = 2000;

    conf.nswap = conf.npage*3;
    conf.nswppo = 4096;
    conf.nimage = 200;

    conf.copymode = 1;      /* copy on reference, not copy on write */

    /*
     * Guess how much is taken by the large permanent
     * datastructures. Mntcache and Mntrpc are not accounted for
     * (probably ~300KB).
     */
    kmem = kpages * BY2PG;
    kmem -=
        conf.upages * sizeof(Page)
        + conf.nproc * sizeof(Proc)
        + conf.nimage * sizeof(KImage)
        + conf.nswap

```

```

    + conf.nswppo * sizeof(Page*); // pad's second bugfix :)

// memory pool
mainmem->maxsize = kmem;

if(!cpuserver)
    /*
     * give terminals lots of image memory, too; the dynamic
     * allocation will balance the load properly, hopefully.
     * be careful with 32-bit overflow.
     */
    imagmem->maxsize = kmem;
}

⟨arch__confinit() if maxmem boot parameter(arm) 243a⟩≡ (242)
    if((p = getconf("maxmem")) != nil){
        memsize = strtoul(p, 0, 0);
        if (memsize < 16*MB) /* sanity */
            memsize = 16*MB;
    }

⟨arch__confinit() sanity check conf.mem[0](arm) 243b⟩≡ (242)
    if(conf.mem[0].limit == 0){
        conf.mem[0].base = 0;
        conf.mem[0].limit = memsize;
    }
    ⟨arch__confinit() if maxmem parameter and limit mem is zero(arm) 243c⟩

⟨arch__confinit() if maxmem parameter and limit mem is zero(arm) 243c⟩≡ (243b)
    else if(p != nil)
        conf.mem[0].limit = conf.mem[0].base + memsize;

```

11.2.4 Exception vector initialization (ARM)

`arch__trapinit()` copies the `vectors` and `vtable` code (from the Interrupts chapter) to `HVECTORS` (`0xFFFF0000`), disables all interrupts, and sets up scratch stacks for each exception mode (IRQ, ABT, UND) so that `_vxxx` handlers have somewhere to save R0–R4:

```

⟨struct Vpage0(arm) 243d⟩≡ (712c)
/*
 * Layout at virtual address KZERO (double mapped at HVECTORS).
 */
struct Vpage0 {
    void (*vectors[Nvec])(void);
    u32int vtable[Nvec];
};

⟨constant Nvec(arm) 243e⟩≡ (712b)
    Nvec = 8, /* # of vectors at start of lexception.s */

⟨function arch__trapinit(arm) 243f⟩≡ (712c)
/*
 * set up for exceptions
 */
void
arch__trapinit(void)
{
    Vpage0 *vpage0;

```

```

    if (cpu->cpuno == 0) {
        /* disable everything */
        intrsoff();
        /* set up the exception vectors */
        vpage0 = (Vpage0*)HVECTORS;
        memmove(vpage0->vectors, vectors, sizeof(vpage0->vectors));
        memmove(vpage0->vtable, vtable, sizeof(vpage0->vtable));
        <arch__trapinit() invalidate cache after adjust vectors(arm) 470j>
    }

    /* set up the stacks for the interrupt modes */
    <arch__trapinit() set stack for other exception/processor-modes 244b>
    <arch__trapinit() coherence(arm) 473b>
}

<function intrsoff(arm) 244a>≡ (712c)
void
intrsoff(void)
{
    Intregs *ip;
    int disable;

    ip = (Intregs*)INTREGS;
    disable = ~0;
    ip->GPUdisable[0] = disable;
    ip->GPUdisable[1] = disable;
    ip->ARMdisable = disable;
    ip->FIQctl = 0;
}

<arch__trapinit() set stack for other exception/processor-modes 244b>≡ (243f) 499d▷
    setr13(PsrMirq, cpu->sirq);
    setr13(PsrMabt, cpu->sabt);
    setr13(PsrMund, cpu->sund);

<Cpu Arch save of first five registers fields(arm) 244c>≡ (48e)
    /* save areas for exceptions, hold R0-R4 */
    u32int  sirq[5];
    u32int  sund[5];
    u32int  sabt[5];

<function setr13(arm) 244d>≡ (714a)
    /*
     * set the stack value for the mode passed in R0
     */
    TEXT setr13(SB), 1, $-4
        MOVW    4(FP), R1

        MOVW    CPSR, R2
        BIC $PsrMask, R2, R3
        ORR $(PsrDirq|PsrDfiq), R3
        ORR R0, R3
        MOVW    R3, CPSR          /* switch to new mode */

        MOVW    R13, R0          /* return old sp */
        MOVW    R1, R13         /* install new one */

        MOVW    R2, CPSR          /* switch back to old mode */
        RET

```

11.2.5 Clock initialization (ARM)

`clockinit()` measures the CPU frequency by timing cycles against the 1 MHz system timer, then registers `clockintr` as the handler for `IRQtimer3`. After this, the hardware will generate an interrupt when system timer 3's compare register matches the counter:

```
<function clockinit(arm) 245a>≡ (715b)
void
clockinit(void)
{
    Systimers *tn;
    Armtimer *tm;
    u32int t0, t1, tstart, tend;

    <clockinit() if many processors 485d>

    tn = (Systimers*)SYSTIMERS;
    tstart = tn->clo;
    do{
        t0 = arch_lcycles();
    }while(tn->clo == tstart);
    tend = tstart + 10000;
    do{
        t1 = arch_lcycles();
    }while(tn->clo != tend);
    t1 -= t0;

    cpu->cpuhz = 100 * t1;
    cpu->cpumhz = (cpu->cpuhz + Mhz/2 - 1) / Mhz;

    cpu->cyclefreq = cpu->cpuhz;

    if(cpu->cpuno == 0){
        tn->c3 = tn->clo - 1;

        tm = (Armtimer*)ARMTIMER;
        tm->load = 0;
        tm->ctl = TmrPrescale1|CntEnable|CntWidth32;

        arch_intrenable(IRQtimer3, clockintr, nil, 0, "clock");
    }
    <clockinit() if not cpu0 485e>
}
```

11.2.6 Timer initialization

`timersinit()` creates the periodic timer that drives `hzclock()`. The timer's `tf` callback is set to `nil`, which `timerintr()` interprets as “call `hzclock()`”. This is the timer that makes the kernel tick:

```
<global timersinit 245b>≡ (673a)
static bool timersinit;

<function timersinit 245c>≡ (673a)
void
timersinit(void)
{
    Timer *t;

    timersinit = true;
```

```

todinit();

t = malloc(sizeof(Timer));
if(t == nil)
    error(Enomem);
t->tmode = Tperiodic;
t->tt = nil;
t->tns = 1000000000 / Arch_HZ;
/*
 * T->tf == nil means the HZ clock for this processor.
 */
t->tf = nil;
timeradd(t);
}

```

11.2.7 Virtual memory initialisation (ARM)

`mmuinit1()` finishes what `mmuinit()` started: it records the L1 table in `cpu->mmul1` and removes the identity mapping of physical address 0 that was needed only while the MMU was being enabled:

```

⟨function mmuinit1(arm) 246a⟩≡ (711a)
void
mmuinit1(void *a)
{
    PTE *l1;

    l1 = (PTE*)a;
    cpu->mmul1 = l1;

    /*
     * undo identity map of first MB of ram
     */
    l1[L1X(0)] = 0;
    ⟨mmuinit1() write back cache after adjusting page tables(arm) 470c⟩
    mmuinvalidateaddr(0);
}

```

11.2.8 Memory initialisation

The memory initialization functions allocate the large kernel data structures using `xalloc` from the Memory chapter. `xinit()` splits physical memory between the kernel (`xhole`) and user pages (`palloc.mem`). Then `procinit()`, `pageinit()`, `swapinit()`, and `imageinit()` each allocate their respective arrays:

```

xinit()

⟨function xinit 246b⟩≡ (660b)
void
xinit(void)
{
    int i;
    int nkpages;
    int kpages;
    ulong maxpages;
    Confmem *m;
    Pallocmem *pm;
    Hole *h, *eh;
}

```

```

eh = &xlists.hole[Nhole-1];
for(h = xlists.hole; h < eh; h++)
    h->next = h+1;

xlists.unused_slots = xlists.hole;

kpages = conf.npage - conf.upages;

pm = palloc.mem;
for(i=0; i<nelem(conf.mem); i++){
    m = &conf.mem[i];
    nkpages = m->npage;
    if(nkpages > kpages)
        nkpages = kpages; // will be zero once kpages has been filled
    /* don't try to use non-KADDR-able memory for kernel */
    maxpages = arch_cankaddr(m->base)/BY2PG;
    if(nkpages > maxpages)
        nkpages = maxpages;

    /* first give to kernel */
    if(nkpages > 0){
        xhole(m->base, nkpages*BY2PG);
        kpages -= nkpages;
        <xinit() nkpages kernel memory in m 430a>
    }

    /* if anything left over, give to user */
    if(m->npage > nkpages){
        if(pm >= palloc.mem+nelem(palloc.mem)){
            print("xinit: losing %lud pages\n", m->npage-nkpages);
            continue;
        }
        pm->base = m->base+nkpages*BY2PG;
        pm->npage = m->npage - nkpages;
        pm++;
    }
}
}

```

procinit()

```

<function procinit 247a>≡ (675c)
void
procinit(void)
{
    Proc *p;
    int i;

    procalloc.free = xalloc(conf.nproc * sizeof(Proc));
    <procinit() sanity check procalloc.free 247b>
    procalloc.arena = procalloc.free;

    p = procalloc.free;
    for(i=0; i<conf.nproc-1; i++,p++)
        p->qnext = p+1;
    p->qnext = nil;
}

```

<procinit() sanity check procalloc.free 247b>≡ (247a)


```

if(procalloc.free == nil){
    xsummary();
    panic("cannot allocate %lud procs (%ludMB)\n",
          conf.nproc, conf.nproc*sizeof(Proc)/MB);
}

```

pageinit()

```

⟨function pageinit 248⟩≡ (664)
void
pageinit(void)
{
    int i, j;
    Page *p;
    Pallocmem *pm;
    ulong m, np, k, vkb, pkb;
    ⟨pageinit() other locals 492d⟩

    np = 0;
    for(i=0; i<nelem(palloc.mem); i++){
        pm = &palloc.mem[i];
        np += pm->npage;
    }

    palloc.pages = xalloc(np * sizeof(Page));
    ⟨pageinit() sanity check palloc.pages 249a⟩

    palloc.head = palloc.pages;
    p = palloc.head;
    for(i=0; i<nelem(palloc.mem); i++){
        pm = &palloc.mem[i];
        for(j=0; j < pm->npage; j++){
            p->prev = p-1;
            p->next = p+1;
            p->pa = pm->base + j*BY2PG;
            palloc.freecount++;
            ⟨pageinit() set page color 492e⟩
            p++;
        }
    }
    palloc.tail = p - 1;
    palloc.head->prev = nil;
    palloc.tail->next = nil;

    palloc.user = p - palloc.pages;
    assert(palloc.user == np);

    pkb = palloc.user*BY2PG/KB;
    vkb = pkb + (conf.nswap*BY2PG)/KB;

    /* Paging numbers */
    swapalloc.highwater = (palloc.user*5)/100;
    swapalloc.headroom = swapalloc.highwater + (swapalloc.highwater/4);

    // diagnostic
    m = 0;
    for(i=0; i<nelem(conf.mem); i++)
        if(conf.mem[i].npage)
            m += conf.mem[i].npage*BY2PG;

```

```

    k = PGROUND(end - (char*)KTZERO);
    print("%ldM memory: ", (m+k+1024*1024-1)/(1024*1024));
    print("%ldM kernel data, ", (m+k-pkb*1024+1024*1024-1)/(1024*1024));
    print("%ldM user, ", pkb/1024);
    print("%ldM swap\n", vkb/1024);
}

⟨pageinit() sanity check palloc.pages 249a⟩≡ (248)
    if(palloc.pages == nil)
        panic("pageinit");

swapinit()

⟨function swapinit 249b⟩≡ (666)
    void
    swapinit(void)
    {
        swapalloc.swmap = xalloc(conf.nswap);

        swapalloc.top = &swapalloc.swmap[conf.nswap];
        swapalloc.alloc = swapalloc.swmap;
        swapalloc.last = swapalloc.swmap;
        swapalloc.free = conf.nswap;

        iolist = xalloc(conf.nswppo*sizeof(Page*));

        ⟨swapinit() sanity check xalloc return values 249c⟩
        swapimage.notext = true;
    }

⟨swapinit() sanity check xalloc return values 249c⟩≡ (249b)
    if(swapalloc.swmap == nil || iolist == nil)
        panic("swapinit: not enough memory");

imageinit()

⟨constant NFEECHAN 249d⟩≡ (658)
    #define NFEECHAN 64

⟨function imageinit 249e⟩≡ (665)
    void
    imageinit(void)
    {
        KImage *i, *ie;

        imagealloc.free = xalloc(conf.nimage * sizeof(KImage));
        ⟨imageinit() sanity check imagealloc.free 249f⟩
        ie = &imagealloc.free[conf.nimage-1];
        for(i = imagealloc.free; i < ie; i++)
            i->next = i+1;
        i->next = nil;
        imagealloc.freechan = malloc(NFEECHAN * sizeof(Chan*));
        imagealloc.szfreechan = NFEECHAN;
    }

⟨imageinit() sanity check imagealloc.free 249f⟩≡ (249e)
    if (imagealloc.free == nil)
        panic("imageinit: no memory");

```

11.2.9 Device initialisation

`chandevreset()` iterates over `devtab` and calls each device driver's `reset()` method. This is where most hardware is discovered and initialized: the USB controller, SD card, Ethernet, console, etc. A second pass via `chandevinit()` (called later from `init0()`) runs each driver's `init()` method in a process context:

```
<function chandevreset 250a>≡ (647)
void
chandevreset(void)
{
    int i;

    todinit(); /* avoid later reentry causing infinite recursion */

    for(i=0; devtab[i] != nil; i++) {
        print("reset %d, %s\n", i, devtab[i]->name);
        devtab[i]->reset();
    }
}

<Dev methods 250b>≡ (54a) 253b▷
void (*reset)(void); // done once at boot time
```

11.2.10 Displaying hardware information (ARM)

```
<main() locals(arm) 250c>≡ (238b)
uint firmware, board;

<main() print board and firmware information(arm) 250d>≡ (238b)
board = getboardrev();
firmware = getfirmware();
print("board rev: %#ux firmware rev: %d\n", board, firmware);
if(firmware < Minfirmrev){
    print("Sorry, firmware (start*.elf) must be at least rev %d"
          " or newer than %s\n", Minfirmrev, Minfirmdate);
    for(;;)
        ;
}

<constant Minfirmrev(arm) 250e>≡ (704c)
/* Firmware compatibility */
#define Minfirmrev 326770

<constant Minfirmdate(arm) 250f>≡ (704c)
#define Minfirmdate "19 Aug 2013"

<function getboardrev(arm) 250g>≡ (691d)
/*
 * Get board revision
 */
uint
getboardrev(void)
{
    u32int buf[1];

    if(vcreq(TagGetrev, buf, 0, sizeof buf) != sizeof buf)
        return 0;
    return buf[0];
}
```

```

⟨function getfirmware(arm) 251a⟩≡ (691d)
/*
 * Get firmware revision
 */
uint
getfirmware(void)
{
    u32int buf[1];

    if(vcreq(TagGetfwrev, buf, 0, sizeof buf) != sizeof buf)
        return 0;
    return buf[0];
}

```

11.3 The first process (ARM)

`arch__userinit()` creates the first process by hand—there is no parent to `rfork` from. It allocates a `Proc`, creates a text segment containing `initcode` (a tiny assembly program that calls `exec("/boot/boot")`), creates a stack segment, sets `sched.pc` to `init0` (the kernel-side startup code), and calls `ready()`. When `schedinit()` picks this process, it jumps to `init0`, which sets up the root namespace (`/`), initializes environment variables, starts the alarm kernel process, and finally calls `arch_touser()` to drop into user mode and execute `initcode`.

```

⟨global sp(arm) 251b⟩≡ (704c)
    static user_addr sp;      /* XXX - must go - user stack of init proc */

⟨macro STACKALIGN(arm) 251c⟩≡ (709b)
    #define STACKALIGN(sp) ((sp) & ~3)    /* bug: assure with alloc */

⟨function arch__userinit(arm) 251d⟩≡ (704c)
/*
 * create the first process
 */
void
arch__userinit(void)
{
    Proc *p;
    Segment *s;
    Arch_KMap *k;
    Page *pg;

    /* no processes yet */
    up = nil;

    p = newproc();
    p->pgrp = newpgrp();
    p->egrp = smalloc(sizeof(Egrp));
    p->egrp->ref = 1;
    p->fgrp = dupfgrp(nil);
    p->rgrp = newrgrp();
    p->procmode = 0640;

    kstrdup(&eve, "");
    kstrdup(&p->text, "init*");
    kstrdup(&p->user, eve);

    /*
     * Kernel Stack
     */
}

```

```

p->sched.pc = PTR2UINT(init0);
p->sched.sp = PTR2UINT(p->kstack + KSTACK
                      - sizeof(up->sargs.args)
                      - sizeof(uintptr));
p->sched.sp = STACKALIGN(p->sched.sp);

/*
 * User Stack
 *
 * Technically, newpage can't be called here because it
 * should only be called when in a user context as it may
 * try to sleep if there are no pages available, but that
 * shouldn't be the case here.
 */
s = newseg(SG_STACK, USTKTOP-USTKSIZE, USTKSIZE/BY2PG);
⟨arch__userinit() set flushme for stack segment 494a⟩
p->seg[SSEG] = s;
pg = newpage(true, nil, USTKTOP-BY2PG);
segspace(s, pg);

k = arch_kmap(pg);
sp = bootargs(VA(k));
arch_kunmap(k);

/*
 * Text
 */
s = newseg(SG_TEXT, UTZERO, 1); // initcode needs only 1 page
p->seg[TSEG] = s;
pg = newpage(true, nil, UTZERO);
⟨arch__userinit() set cachectl(arm) 493h⟩
segspace(s, pg);

k = arch_kmap(s->pagedir[0]->pagetab[0]);
memmove(UINT2PTR(VA(k)), initcode, sizeof initcode);
arch_kunmap(k);

// ready to go!
ready(p);
}

⟨function init0(arm) 252⟩≡ (704c)
/*
 * starting place for first process
 */
void
init0(void)
{
    int i;
    char buf[2*KNAMELEN]; // has to be the same than in ksetenv?

    up->nerrlab = 0;

    ⟨init0() coherence(arm) 473c⟩
    arch_spllo(); // Allow interrupts!!!

    /*
     * These are o.k. because rootinit is null.
     * Then early kproc's will have a root and dot.
     */

```

```

up->slash = namec("#/", Atodir, 0, 0);
pathclose(up->slash->path);
up->slash->path = newpath("/");
up->dot = cclone(up->slash);

chandevinit();

if(!waserror()){
    snprintf(buf, sizeof(buf), "%s %s", "ARM", conffile);
    ksetenv("terminal", buf, 0);
    ksetenv("cputype", "arm", 0); // used by mkfile!
    if(cpuserver)
        ksetenv("service", "cpu", 0);
    else
        ksetenv("service", "terminal", 0);
    snprintf(buf, sizeof(buf), "-a %s", getethermac());
    ksetenv("etherargs", buf, 0);

    /* convert plan9.ini variables to #e and #ec */
    for(i = 0; i < nconf; i++) {
        ksetenv(confname[i], confval[i], 0);
        ksetenv(confname[i], confval[i], 1);
    }
    poperror();
}
kproc("alarm", alarmkproc, nil); // ??

// !! Go to user space !!
arch_touser(sp);

assert(0);          /* shouldn't have returned */
}

```

⟨function chandevinit 253a⟩≡ (647)

```

void
chandevinit(void)
{
    int i;
    for(i=0; devtab[i] != nil; i++) {
        devtab[i]->init();
    }
}

```

⟨Dev methods 253b⟩+≡ (54a) ◁250b 266b▷

```

void (*init)(void);

```

⟨function arch_touser(arm) 253c⟩≡ (719a)

```

/*
 * This is the first jump from kernel to user mode.
 * Fake a return from interrupt.
 *
 * Enter with R0 containing the user stack pointer.
 * UTZERO + 0x20 is always the entry point.
 *
 */
TEXT arch_touser(SB), 1, $-4
    /* store the user stack pointer into the USR_r13 */
    MOVM.DB.W [R0], (R13)
    /* avoid the ambiguity described in notes/movm.w. */
    // MOVM.S.IA.W (R13), [R13]

```

```

    MOVM.S (R13), [R13]
    ADD $4, R13

    /* set up a PSR for user level */
    MOVW $(PsrMusr), R0
    MOVW R0, SPSR

    /* save the PC on the stack */
    MOVW $(UTZERO+0x20), R0
    MOVM.DB.W [R0], (R13)

    /*
     * note that 5a's RFE is not the v6 arch. instruction (0xe89d0a00,
     * I think), which loads CPSR from the word after the PC at (R13),
     * but rather the pre-v6 simulation 'MOVM.IA.S.W (R13), [R15]'
     * (0xe8fd8000 since MOVM is LDM in this case), which loads CPSR
     * not from memory but from SPSR due to '.S'.
     */
    RFE

<init9.c 254a>≡
    /*@Scheck: def can actually be found initcode.c, but this file is in skip list
    extern void startboot(char*, char**);

    /*@Scheck: entry point looked for by the linker 8l
    void
    _main(char *argv0)
    {
        startboot(argv0, &argv0);
    }

<init/user/preboot/arm/init9.s 254b>≡
    /*
     * This is the same as the C program:
     *
     * void
     * main(char* argv0)
     * {
     *     startboot(argv0, &argv0);
     * }
     *
     * It is in assembler because SB needs to be
     * set and doing this in C drags in too many
     * other routines.
     */
    TEXT main(SB), 1, $8
        MOVW $setR12(SB), R12        /* load the SB */
        MOVW $boot(SB), R0

        ADD $12, R13, R1            /* pointer to 0(FP) */

        MOVW R0, 4(R13)             /* pass argc, argv */
        MOVW R1, 8(R13)

        BL startboot(SB)
    _loop:
        B _loop

<initcode.c 254c>≡
    /*

```

```

* IMPORTANT! DO NOT ADD LIBRARY CALLS TO THIS FILE.
* The entire text image must fit on one page
* (and there's no data segment, so any read/write data must be on the stack).
*/

```

```

#include <u.h>
#include <libc.h>

char cons[] = "#c/cons";
char boot[] = "/boot/boot";
char dev[] = "/dev";
char c[] = "#c";
char e[] = "#e";
char ec[] = "#ec";
char s[] = "#s";
char srv[] = "/srv";
char env[] = "/env";

// called from init9.s
void
startboot(char *argv0, char **argv)
{
    char buf[200]; /* keep this fairly large to capture error details */

    /* in case boot is a shell script */
    open(cons, OREAD);
    open(cons, OWRITE);
    open(cons, OWRITE);

    bind(c, dev, MAFTER);
    bind(ec, env, MAFTER);
    bind(e, env, MCREATE|MAFTER);
    bind(s, srv, MREPL|MCREATE);

    USED(argv0);
    exec(boot, argv);

    rerrstr(buf, sizeof buf);
    buf[sizeof buf - 1] = '\0';
    _exits(buf);
}

```

11.4 Booting the user

Once `arch_touser()` drops into user mode, `initcode` calls `exec("/boot/boot")`, which replaces the first process with the boot program. `boot.rc` is a simple `rc` script that mounts the root file system, sets up the user environment, and eventually launches `init` and then the shell. At this point, the kernel is fully operational—all the machinery from the previous chapters is running.

```

#!/boot/rc -m /boot/rcmain
/boot/echo booooooooooting...
...

```


Part II

Everything is a File

In Plan 9, the file is not just a storage abstraction—it is the universal interface to everything: devices, network connections, the process table, even the windowing system. This part covers how the kernel implements this idea: the **Chan** and **Dev** structures that represent open files and device drivers, the directory and namespace operations that let each process assemble its own view of the file system, the **Queue** structure used for streaming I/O, and the on-disk file system.

Chapter 12

Files

```
<systab file syscalls 257a>≡ (41f)
[OPEN]      sysopen,
[CLOSE]     sysclose,
[PREAD]     syspread,
[PWRITE]    syspwrite,
[SEEK]      sysseek,
```

12.1 Overview

In this chapter, I present the file-related system calls: `sysopen()`, `sysclose()`, `syspread()`, `syspwrite()`, and `sysseek()`. In Plan 9, files are the universal abstraction — not just for storage but for devices, network connections, and system services. The key data structures were introduced in Chapter ??: `\hlinkChan` is the kernel’s representation of an open file, `\hlinkDev` defines the file server interface, and `devtab` maps device types to their method tables. This chapter shows how user-visible file descriptors connect to channels, how the syscalls dispatch through the device table, and how the `\hlinkPath` structure tracks human-readable filenames.

12.2 Channels

A `\hlinkChan` carries a `flag` bitset that tracks its lifecycle state. The two fundamental flags are `COPEN` (the channel has been opened for I/O via a device’s `open()` method) and `CFREE` (the channel is on the free list and available for reuse). Other flags like `CCEXEC` and `CRCLOSE` are added later via the open mode bits.

```
<enum channelflag 257b>≡ (645c)
/*
 * channel flags
 */
enum ChannelFlag
{
    COPEN = 0x0001,    /* for i/o */
    CFREE = 0x0010,    /* not in use */
    <ChannelFlag cases 281b>
};

<Chan other fields 257c>≡ (51e) 314a▷
// enum<channelflag> (actually a bitset for certain properties)
ushort flag;
```

Channels are managed by a custom allocator rather than being individually malloced and freed. The `\hlinkChanalloc` structure maintains two linked lists: `free` holds recycled channels ready for reuse, and `list` holds every channel ever allocated (used for debugging and the `fid` counter). Channels are never truly freed

back to the heap — once allocated, they cycle between the free list and active use. This avoids the cost of repeated `malloc/free` pairs for what is one of the most frequently allocated kernel objects.

```
<struct Chanalloc 258a>≡ (647)
// Channel allocator
struct Chanalloc
{
    //list<ref<Chan>> (next = Chan.next), the free one
    Chan    *free;
    //list<ref<Chan>> (next = Chan.link), the used one
    Chan    *list;

    int fid; // type could be 'Counter', but already have a Lock anyway

    // extra
    Lock;
};
```

```
<Chan extra fields 258b>≡ (51e)
// list<ref<Chan> of Chanalloc.free
Chan* next;    /* allocation */
// list<ref<Chan> of Chanalloc.list
Chan* link;
```

```
<global chanalloc 258c>≡ (647)
struct Chanalloc chanalloc;
```

`\hlinknewchan` first tries the free list; if empty, it allocates a fresh channel and assigns it a unique `fid` (used by the mount device `devmnt` to multiplex 9P sessions). The channel is initialized with `type 0` (the root device), so if an error occurs before the caller associates it with a real device, `cclose()` will call `rootclose()`, which is a no-op — a safe default.

```
<constructor newchan 258d>≡ (647)
Chan*
newchan(void)
{
    Chan *c;

    lock(&chanalloc);
    c = chanalloc.free;
    if(c != nil)
        chanalloc.free = c->next;
    unlock(&chanalloc);

    if(c == nil){
        c = smalloc(sizeof(Chan));
        lock(&chanalloc);
        c->fid = ++chanalloc.fid;
        c->link = chanalloc.list;
        chanalloc.list = c;
        unlock(&chanalloc);
    }

    /* if you get an error before associating with a dev,
       close calls rootclose, a nop */
    c->type = 0;
    c->dev = 0;

    c->path = nil;

    c->offset = 0;
```

```

    c->devoffset = 0;
    // c->mode?

    c->flag = 0;
    c->ref = 1;

    c->ismtpt = false;
    c->umh = nil;
    // c->umc?
    c->uri = 0;

    c->mux = nil;
    c->mchan = nil;
    memset(&c->mqid, 0, sizeof(c->mqid));
    c->mcp = nil;
    c->iounit = 0;
    c->dri = 0;

    c->aux = nil;

    return c;
}

```

```

⟨destructor chanfree 259a⟩≡ (647)
void
chanfree(Chan *c)
{
    c->flag = CFREE;

    ⟨chanfree() optional free 314b⟩

    pathclose(c->path);
    c->path = nil;

    lock(&chanalloc);
    c->next = chanalloc.free;
    chanalloc.free = c;
    unlock(&chanalloc);
}

```

`\hlinkuniquechan` implements copy-on-write for channels: if the channel has multiple references, it clones it to get an exclusive copy. This is needed before operations that modify the channel in place (like `open` or `create`).

```

⟨function uniquechan 259b⟩≡ (647)
/*
 * Make sure we have the only copy of c.  (Copy on write.)
 */
Chan*
uniquechan(Chan *c)
{
    Chan *nc;

    if(c->ref != 1){
        nc = cclone(c);
        cclose(c);
        return nc;
    }else{
        return c;
    }
}

```

12.3 Devices table devtab

Each device driver provides a standard `attach()` method. Most drivers implement it by calling the generic helper `\hlinkdevattach`, which allocates a fresh channel, sets its `type` to the index of the driver in `devtab`, and constructs a path string like `"#c"` or `"#I"` from the device's rune character.

`<function devattach 260a>≡` (649)

```
Chan*
devattach(Rune tc, char *spec)
{
    int n;
    Chan *c;
    char *buf;

    c = newchan();
    mkqid(&c->qid, 0, 0, QTDIR); // root of a device is a QTDIR
    c->type = devno(tc, false);

    if(spec == nil)
        spec = "";
    n = 1+UTFmax+strlen(spec)+1; // '#' + Rune + spec + '\0'
    buf = smalloc(n);
    snprintf(buf, n, "%C%s", tc, spec);
    c->path = newpath(buf);
    free(buf);

    return c;
}
```

`<function devno 260b>≡` (649)

```
int
devno(Rune c, bool user)
{
    int i;

    for(i = 0; devtab[i] != nil; i++) {
        if(devtab[i]->dc == c)
            return i;
    }
    if(!user)
        panic("devno %C %#ux", c, c);

    return -1;
}
```

12.4 File descriptors, Fgrp

A file descriptor is simply an integer index into the `fd` array of an `\hlinkFgrp` (file descriptor group). Each slot is either `nil` (unused) or a reference-counted pointer to a `\hlinkChan`. The array grows dynamically in `DELTA`FD increments, capped at 5000. The `\hlinkFgrp` itself is reference-counted and stored in `Proc.fgrp`, allowing processes created with `RFFDG` to share file descriptor tables.

`<struct Fgrp 260c>≡` (668c)

```
struct Fgrp
{
    // array<option<ref_counted<Chan>>>
    Chan **fd;
    // nelem(fd)
```

```

    int nfd;        /* number allocated */
    int maxfd;      /* highest fd in use */ // <= nfd

    <Fgrp other fields 263b>
    // extra
    Ref;
};

<Proc files fields 261a>+≡ (49f) <52a 278a>
    // ref_counted<Fgrp>
    Fgrp *fgrp;    /* File descriptor group */

    \hlinkdupfgrp serves double duty: called with nil it acts as a constructor (allocating a fresh table of
    DELTAxFD slots), and called with an existing \hlinkFgrp it duplicates the table, incrementing the reference
    count on each channel. This is the function behind RFFDG: when a child process gets its own file descriptor
    group, all current fds are shared (via reference counting on the channels), but future opens in either process are
    independent.

    <function dupfgrp 261b>≡ (673b)
    Fgrp*
    dupfgrp(Fgrp *f)
    {
        Fgrp *new;
        Chan *c;
        int i;

        new = smalloc(sizeof(Fgrp));
        if(f == nil){
            new->fd = smalloc(DELTAxFD*sizeof(Chan*));
            new->nfd = DELTAxFD;
            new->ref = 1;
            return new;
        }

        lock(f);
        /* Make new fd list shorter if possible, preserving quantization */
        new->nfd = f->maxfd+1;
        i = new->nfd%DELTAxFD;
        if(i != 0)
            new->nfd += DELTAxFD - i;
        new->fd = malloc(new->nfd*sizeof(Chan*));
        if(new->fd == nil){
            unlock(f);
            free(new);
            error("no memory for fgrp");
        }
        new->ref = 1;

        new->maxfd = f->maxfd;
        for(i = 0; i <= f->maxfd; i++) {
            if(c = f->fd[i]){
                incref(c);
                new->fd[i] = c;
            }
        }
        unlock(f);

        return new;
    }

```

```
⟨constant DELTAFD 262a⟩≡ (668c)
    DELTAFD = 20    /* incremental increase in Fgrp.fd's */
```

```
⟨function closefgrp 262b⟩≡ (673b)
    void
    closefgrp(Fgrp *f)
    {
        int i;
        Chan *c;

        if(f == nil)
            return;

        if(decref(f) != 0)
            return;

        /*
         * If we get into trouble, forceclosefgrp
         * will bail us out.
         */
        up->closingfgrp = f;
        for(i = 0; i <= f->maxfd; i++)
            if(c = f->fd[i]){
                f->fd[i] = nil;
                cclose(c);
            }
        up->closingfgrp = nil;

        free(f->fd);
        free(f);
    }
```

\hlinknewfd assigns a channel to the lowest available file descriptor. It scans the fd array for a nil slot via \hlinkfindfreefd, growing the array if needed. The fd table uses the same amortized growth strategy (allocate in DELTAFD-sized chunks) seen throughout the kernel.

```
⟨function newfd 262c⟩≡ (654)
    int
    newfd(Chan *c)
    {
        int fd;
        Fgrp *f;

        f = up->fgrp;
        lock(f);
        fd = findfreefd(f, 0);
        if(fd < 0){
            unlockfgrp(f);
            return -1;
        }
        if(fd > f->maxfd)
            f->maxfd = fd;
        f->fd[fd] = c;
        unlockfgrp(f);
        return fd;
    }
```

```
⟨function findfreefd 262d⟩≡ (654)
    /*
     * this assumes that the fgrp is locked
```

```

*/
int
findfreefd(Fgrp *f, int start)
{
    int fd;

    for(fd=start; fd<f->nfd; fd++)
        if(f->fd[fd] == nil)
            break;
    if(fd >= f->nfd && growfd(f, fd) < 0)
        return -1;
    return fd;
}

```

```

⟨function growfd 263a⟩≡ (654)
int
growfd(Fgrp *f, int fd) /* fd is always >= 0 */
{
    Chan **newfd, **oldfd;

    if(fd < f->nfd)
        return 0; // should never happen no?
    if(fd >= f->nfd+DELTAxFD)
        return -1; /* out of range */ // when can this happen? from sysdup?
    /*
     * Unbounded allocation is unwise
     */
    if(f->nfd >= 5000){
    Exhausted:
        print("no free file descriptors\n");
        return -1;
    }
    // realloc
    newfd = malloc((f->nfd+DELTAxFD)*sizeof(Chan*));
    if(newfd == nil)
        goto Exhausted;
    oldfd = f->fd;
    memmove(newfd, oldfd, f->nfd*sizeof(Chan*));
    f->fd = newfd;
    free(oldfd);
    f->nfd += DELTAxFD;
    if(fd > f->maxfd){
        if(fd/100 > f->maxfd/100)
            f->exceed = (fd/100)*100;
        f->maxfd = fd;
    }
    return 1;
}

```

```

⟨Fgrp other fields 263b⟩≡ (260c)
int exceed; /* debugging */

```

```

⟨function unlockfgrp 263c⟩≡ (654)
static void
unlockfgrp(Fgrp *f)
{
    int ex;

    ex = f->exceed;
    f->exceed = 0;
}

```



```

unlock(f);
if(ex)
    pprintf("warning: process exceeds %d file descriptors\n", ex);
}

```

12.5 sysopen(), sysclose()

The open syscall is deceptively simple: it calls `\hlinknamec` to resolve the filename into a channel (walking through mount points and device tables), then assigns that channel to the lowest available fd. The heavy lifting is in `namec()` (presented later in this chapter) and in the device's `open()` method. Close is even simpler: it checks the fd is valid via `\hlinkfdtochan`, then calls `\hlinkfdclose` which removes it from the table and decrements the channel's reference count.

```

⟨enum open 264a⟩≡ (656e)
enum OpenFlags {
    OREAD = 0, /* open for read */
    OWRITE = 1, /* write */
    ORDWR = 2, /* read and write */
    ⟨OpenFlags cases 267b⟩
};

```

```

⟨syscall open 264b⟩≡ (654)
// int open(char *file, int omode);
long
sysopen(ulong* arg)
{
    int fd;
    Chan *c;

    openmode(arg[1]); /* error check only */
    validaddr(arg[0], 1, false);
    c = namec((char*)arg[0], Aopen, arg[1], 0);
    if(waserror()){
        cclose(c);
        nexterror();
    }
    fd = newfd(c);
    if(fd < 0)
        error(Enofd);
    poperror();
    return fd;
}

```

```

⟨function openmode 264c⟩≡ (651b)
int
openmode(ulong o)
{
    o &= ~(OTRUNC|OEXEC|ORCLOSE);
    if(o > OEXEC)
        error(Ebadarg);
    if(o == OEXEC)
        return OREAD;
    return o;
}

```

```

⟨syscall close 265a⟩≡ (654)
// int close(int fd);
long
sysclose(ulong* arg)
{
    fdtochan(arg[0], -1, false, false); // for its error checking side effect
    fdclose(arg[0], 0);
    return 0;
}

```

`\hlinkfdtochan` converts a user-space file descriptor integer into a kernel `\hlinkChan` pointer, with bounds checking and optional access mode validation. The `mode` parameter of `-1` means “skip mode checking”. When `iref` is true, it increments the channel’s reference count so the caller can safely use the channel even if another thread closes the `fd` concurrently.

```

⟨function fdtochan 265b⟩≡ (651b)
Chan*
fdtochan(int fd, int mode, bool chkmnt, bool iref)
{
    Chan *c;
    Fgrp *f;

    c = nil;
    f = up->fgrp;

    lock(f);
    if(fd<0 || f->nfd<=fd || (c = f->fd[fd])==nil) {
        unlock(f);
        error(Ebadfd);
    }
    // c = f->fd[fd] != nil

    if(iref)
        incref(c);
    unlock(f);

    if(chkmnt && (c->flag&CMMSG)) {
        ⟨fdtochan() undo iref incref 265c⟩
        error(Ebadusefd);
    }

    if(mode<0 || c->mode==ORDWR) // mode < 0? mean no check?
        return c;

    if((mode&OTRUNC) && c->mode==OREAD) { // BUG impossible, dead code
        ⟨fdtochan() undo iref incref 265c⟩
        error(Ebadusefd);
    }
    // the access mode must match the opening mode
    if((mode&~OTRUNC) != c->mode) {
        ⟨fdtochan() undo iref incref 265c⟩
        error(Ebadusefd);
    }

    return c;
}

```

```

⟨fdtochan() undo iref incref 265c⟩≡ (265)
if(iref)
    cclose(c);

```

⟨function fdclose 266a⟩≡ (651b)

```
void
fdclose(int fd, int flag)
{
    int i;
    Chan *c;
    Fgrp *f = up->fgrp;

    lock(f);
    c = f->fd[fd]; // bound checking??
    if(c == nil){
        /* can happen for users with shared fd tables */
        unlock(f);
        return;
    }
    if(flag){
        if(c==nil || !(c->flag&flag)){ // dead test c==nil? see above
            unlock(f);
            return;
        }
    }
    f->fd[fd] = nil;

    if(fd == f->maxfd)
        for(i=fd; --i>=0 && f->fd[i]==nil; )
            f->maxfd = i;

    unlock(f);
    cclose(c);
}
```

⟨Dev methods 266b⟩+≡ (54a) ◁253b 269a▷

```
void (*close)(Chan*);
```

`\hlinkcclose` is the central channel destructor. It decrements the reference count; if it reaches zero, it calls the device's `close()` method (wrapped in `waserror` since the close may involve I/O to a remote file server that could fail) and then returns the channel to the free list via `\hlinkchanfree`.

⟨function cclose 266c⟩≡ (647)

```
void
chan_cclose(Chan *c)
{
    if(c->flag&CFREE)
        panic("cclose %p", getcallerpc(&c));

    DBG("cclose %p name=%s ref=%ld\n", c, c->path->s, c->ref);
    if(decRef(c))
        return;

    if(!waserror()){
        devtab[c->type]->close(c);
        poperror();
    }
    chanfree(c);
}
```

12.6 namec()

`\hlinknamec` (“name to channel”) is the central function that translates a file path string into a `\hlinkChan`. It

is called by every file syscall: `sysopen()`, `syscreate()`, `sysstat()`, `sysremove()`, `sysbind()`, `syschdir()`, and `sysexec()`. The `amode` parameter (`Aopen`, `Acreate`, `Aremove`, `Aaccess`, `Atodir`, `Abind`, `Amount`) tells `namec()` what the caller intends to do with the result, which affects how mount points are traversed. The overall flow is: (1) pick a starting channel (`up->slash` for “/”, `up->dot` for relative paths, or a device attach for “#” paths), (2) parse the name into path elements, (3) call `\hlinkwalk` to resolve those elements through devices and mount points, and (4) call the device’s `open()` or `create()` method.

```
<sysexec() call namec() to get a channel in tc from file 267a>≡ (119d)
    tc = namec(file, Aopen, OEXEC, 0);
```

```
<OpenFlags cases 267b>≡ (264a) 281a▷
    OEXEC = 3, /* execute, == read but check execute permission */
```

```
<enum accessnamec 267c>≡ (645c)
/*
 * Access types in namec
 */
enum AccessFlags
{
    Aopen,      /* for i/o */
    Acreate,    /* is to be created */
    Aremove,    /* will be removed by caller */
    Aaccess,    /* as in stat, wstat */
    Atodir,     /* as in chdir */
    <AccessFlags cases 305c>
};
```

```
<namec() locals 267d>≡ (267g) 267e▷
    char *name;
```

```
<namec() locals 267e>+≡ (267g) <267d 267f▷
    Elemlist e;
```

```
<namec() locals 267f>+≡ (267g) <267e ??▷
    Chan *c;
```

```
<function namec 267g>≡ (647)
/*
 * Turn a name into a channel.
 * &name[0] is known to be a valid address. It may be a kernel address.
 *
 * Opening with amode Aopen, Acreate, Aremove, or Aaccess guarantees
 * that the result will be the only reference to that particular fid.
 * This is necessary since we might pass the result to
 * devtab[]->remove().
 *
 * Opening Atodir or Amount does not guarantee this.
 *
 * Under certain circumstances, opening Aaccess will cause
 * an unnecessary clone in order to get a uniquechan Chan so it
 * can attach the correct name. Sysstat and sys_stat need the
 * correct name so they can rewrite the stat info.
 */
Chan*
namec(char *aname, int amode, int omode, ulong perm)
{
    <namec() locals 267d>

    if(aname[0] == '\0')
        error("empty file name");
```

```

aname = validnamedup(aname, true);

if(waserror()){
    free(aname);
    nexterror();
}
DBG("namec %s %d %d\n", aname, amode, omode);
name = aname;

sharppath = (name[0] == '#');

/*
 * Find the starting off point (the current slash, the root of
 * a device tree, or the current dot) as well as the name to
 * evaluate starting there.
 */
switch(name[0]){
<namec() if name[0] is a sharp 304b>
case '/':
    c = up->slash;
    incref(c);
    break;

default:
    c = up->dot;
    incref(c);
    break;
}

<namec() initializes Elemlist e 271c>

if(waserror()){
    cclose(c);
    <namec() if waserror free Elemlist e and prepare nice error 273c>
}

/*
 * Build a list of elements in the name.
 */
parsename(name, &e);

/*
 * On create, ....
 */
<namec() adjust Elemlist e if Acreate 291a>

if(walk(&c, e.elems, e.nelems, sharppath, &e.nerror) < 0){
    if(e.nerror < 0 || e.nerror > e.nelems){
        print("namec %s walk error nerror=%d\n", aname, e.nerror);
        e.nerror = 0;
    }
    nexterror();
}

<namec() error if not a directory or cannot exec directory 283a>

switch(amode){
case Aopen:
case Aremove:

```

```

case Aaccess:
Open:
    <namec() case Aopen, Acreate, Aremove, Aaccess, handle mountpoint part1 310b>

    switch(amode){
    case Aopen:
    case Acreate:
        <namec() print error if c->umh != nil 314c>
        <namec() case Aopen, Acreate, handle mountpoint part2 313c>

        <namec() set channel flag before open ??>
        c = devtab[c->type]->open(c, omode&~OCEXEC);
        <namec() set channel flag after open 281c>
        break;

    case Aaccess:
    case Aremove:
        <namec() case Aremove, Aaccess, handle mountpoint part2 310c>
        break;

    }
    break;
<namec() other cases 291d>
default:
    panic("unknown namec access %d\n", amode);
}

<namec() set genbuf from Elemlist e 269b>

<namec() free Elemlist e 272a>
poperror(); /* e c */
free(aname);
poperror(); /* aname */

return c;
}

<Dev methods 269a>+≡ (54a) <266b 275a>
    Chan* (*open)(Chan*, int);

<namec() set genbuf from Elemlist e 269b>≡ (267g)
    /* place final element in genbuf for e.g. exec */
    if(e.nelems > 0)
        kstrcpy(up->genbuf, e.elems[e.nelems-1], sizeof up->genbuf);
    else
        kstrcpy(up->genbuf, ".", sizeof up->genbuf);

<Proc other fields 269c>+≡ (49f) <225a 453b>
    char genbuf[128]; /* buffer used e.g. for last name element from namec */

```

12.6.1 Checking valid filename string

Before parsing, `namec()` validates the filename string: it must be shorter than 2^{16} bytes (to fit in a 9P string field) and must not contain control characters or other “frog” characters. The `dup` variant copies the string out of user space before the second scan, preventing a malicious thread from modifying the string after validation.

```

<function validnamedup 269d>≡ (647)
char*
validnamedup(char *aname, bool slashok)

```

```

{
    return validname0(aname, slashok, true, getcallerpc(&aname));
}

⟨function validname 270a⟩≡ (647)
void
validname(char *aname, bool slashok)
{
    validname0(aname, slashok, false, getcallerpc(&aname));
}

⟨function validname0 270b⟩≡ (647)
/*
 * Check that the name
 * a) is in valid memory.
 * b) is shorter than 216 bytes, so it can fit in a 9P string field.
 * c) contains no frogs.
 * The first byte is known to be addressible by the requester, so the
 * routine works for kernel and user memory both.
 * The parameter slashok flags whether a slash character is an error
 * or a valid character.
 *
 * The parameter dup flags whether the string should be copied
 * out of user space before being scanned the second time.
 * (Otherwise a malicious thread could remove the NUL, causing us
 * to access unchecked addresses.)
 */
static char*
validname0(char *aname, bool slashok, bool dup, kern_addr pc)
{
    char *ename, *name, *s;
    int n;
    int c;
    Rune r;

    name = aname;

    if((virt_addr)name < KZERO){
        if(!dup)
            print("warning: validname called from %#p with user pointer", pc);
        ename = vmemchr(name, 0, (1<<16));
    }else
        ename = memchr(name, 0, (1<<16));

    if(ename==nil || ename-name>=(1<<16))
        error("name too long");

    s = nil;
    if(dup){
        n = ename-name;
        s = smalloc(n+1);
        memmove(s, name, n);
        s[n] = '\0';
        aname = s;
        name = s;
        setmalloctag(s, pc);
    }

    // checking validity of name
    while(*name){

```

```

/* all characters above '~' are ok */
c = *(byte*)name;
if(c >= Runeself)
    name += chartorune(&r, name);
else{
    if(isfrog[c])
        if(!slashok || c!='/' ){
            snprintf(up->genbuf, sizeof(up->genbuf), "%s: %q", Ebadchar, aname);
            free(s);
            error(up->genbuf);
        }
        name++;
    }
}
return s;
}

```

<global isfrog 271a>≡ (647)

```

char isfrog[256]={
    /*NUL*/ 1, 1, 1, 1, 1, 1, 1, 1,
    /*BKS*/ 1, 1, 1, 1, 1, 1, 1, 1,
    /*DLE*/ 1, 1, 1, 1, 1, 1, 1, 1,
    /*CAN*/ 1, 1, 1, 1, 1, 1, 1, 1,
    ['/' ] 1,
    [0x7f] 1,
};

```

12.6.2 Parsing filename

Before walking, `namec()` parses the path string into an `\hlinkElemlist`: an array of individual name components (“usr”, “pad”, “foo.c”) with offsets back into the original string for error reporting. The `mustbedir` flag records whether the path ended with “/”, which means the final element must be a directory.

<struct Elemlist 271b>≡ (647)

```

struct Elemlist
{
    char    *aname; /* original name */
    char    *name;  /* copy of name, so '/' can be overwritten */

    //array<string> pointing to subparts of name
    char    **elems; // subparts
    // size(elems)
    int nelems;
    //array<int>, offset in name for the subparts of name in elems
    int *off;

    bool mustbedir; // if last part of filename is a '/'

    int nerror;
    int prefix;
};

```

<namec() initializes Elemlist e 271c>≡ (267g)

```

e.aname = aname;
e.prefix = name - aname;
e.name = nil;
e.elems = nil;
e.off = nil;
e.nelems = 0;
e.nerror = 0;

```


`<namec() free Elemlist e 272a>≡` (267g)

```
free(e.name);
free(e.elems);
free(e.off);
```

`<function parsename 272b>≡` (647)

```
/*
 * The name is known to be valid.
 * Copy the name so slashes can be overwritten.
 * An empty string will set nelem=0.
 * A path ending in / or /. or /././ etc. will have
 * e->mustbedir = true, so that we correctly
 * reject, e.g., "/adm/users/." when /adm/users is a file
 * rather than a directory.
 */
static void
parsename(char *aname, Elemlist *e)
{
    char *name, *slash;

    kstrdup(&e->name, aname);
    name = e->name;
    e->nelems = 0;
    e->elems = nil;
    e->off = smalloc(sizeof(int));
    e->off[0] = skipslash(name) - name;
    for(;;){
        name = skipslash(name);
        if(*name == '\0'){
            e->off[e->nelems] = name+strlen(name) - e->name;
            e->mustbedir = true;
            break;
        }
        growparse(e);
        e->elems[e->nelems++] = name;
        slash = utfrune(name, '/');
        if(slash == nil){
            e->off[e->nelems] = name+strlen(name) - e->name;
            e->mustbedir = false;
            break;
        }
        e->off[e->nelems] = slash - e->name;
        *slash++ = '\0';
        name = slash;
    }
}
```

`<function growparse 272c>≡` (647)

```
static void
growparse(Elemlist *e)
{
    char **new;
    int *inew;
    enum { Delta = 8 };

    if(e->nelems % Delta == 0){
        new = smalloc((e->nelems+Delta) * sizeof(char*));
        memmove(new, e->elems, e->nelems*sizeof(char*));
        free(e->elems);
        e->elems = new;
    }
```

```

        inew = smalloc((e->nelems+Delta+1) * sizeof(int));
        memmove(inew, e->off, (e->nelems+1)*sizeof(int));
        free(e->off);
        e->off = inew;
    }
}

```

⟨function skipslash 273a⟩≡ (647)

```

/*
 * name is valid. skip leading / and ./ as much as possible
 */
char*
skipslash(char *name)
{
    while(name[0]=='/' || (name[0]=='.' && (name[1]!='\0' || name[1]=='/')))
        name++;
    return name;
}

```

12.6.3 Error management

When `namec()` fails partway through a path, it constructs an error message showing the prefix of the path that was successfully resolved, helping the user identify which component was not found.

⟨namec() locals 273b⟩+≡ (267g) ◁?? 286h▷

```

    char tmperrbuf[ERRMAX];

```

⟨namec() if waserror free Elemlist e and prepare nice error 273c⟩≡ (267g)

```

    free(e.name);
    free(e.elems);
/*
 * Prepare nice error, showing first e.nerror elements of name.
 */
if(e.nerror == 0)
    nexterror();
strcpy(tmperrbuf, up->errstr);
if(e.off[e.nerror]==0)
    print("nerror=%d but off=%d\n",e.nerror, e.off[e.nerror]);
len = e.prefix+e.off[e.nerror];
free(e.off);
namelerror(aname, len, tmperrbuf);

```

⟨function namelerror 273d⟩≡ (647)

```

void
namelerror(char *aname, int len, char *err)
{
    char *ename, *name, *next;
    int i, errlen;

/*
 * If the name is short enough, just use the whole thing.
 */
errlen = strlen(err);
if(len < ERRMAX/3 || len+errlen < 2*ERRMAX/3)
    snprintf(up->genbuf, sizeof up->genbuf, "%.s",
        utfnlen(aname, len), aname);
else{
/*
 * Print a suffix of the name, but try to get a little info.

```

```

    */
    ename = aname+len;
    next = ename;
    do{
        name = next;
        next = memrchr(aname, '/', name-aname);
        if(next == nil)
            next = aname;
        len = ename-next;
    }while(len < ERRMAX/3 || len + errlen < 2*ERRMAX/3);

    /*
     * If the name is ridiculously long, chop it.
     */
    if(name == ename){
        name = ename-ERRMAX/4;
        if(name <= aname)
            panic("bad math in namelerror");
        /* walk out of current UTF sequence */
        for(i=0; (*name&0xC0)==0x80 && i<UTFmax; i++)
            name++;
    }
    snprintf(up->genbuf, sizeof up->genbuf, "...%.s",
             utfnlen(name, ename-name), name);
}
snprintf(up->errstr, ERRMAX, "%#q %s", up->genbuf, err);
nexterror();
}

```

```

⟨function nameerror 274a⟩≡ (647)
void
nameerror(char *name, char *err)
{
    namelerror(name, strlen(name), err);
}

```

12.7 syspread(), syspwrite()

Plan 9 uses `pread` and `pwrite` (with an explicit offset) rather than separate `read/write/seek` syscalls. This matches the 9P protocol, where every read and write message carries an offset, making the operations atomic and stateless. When the offset argument is `0ULL` (“no offset”), the kernel falls back to maintaining the channel’s internal `c->offset`. The actual I/O dispatches through `devtab[c->type]->read()` or `devtab[c->type]->write()`, reaching the appropriate device driver.

```

⟨syscall pread 274b⟩≡ (654)
// long pread(int fd, void *buf, long nbytes, vlong offset);
long
syspread(ulong* arg)
{
    vlong v;
    va_list list;

    /* use varargs to guarantee alignment of vlong */
    va_start(list, arg[2]);
    v = va_arg(list, vlong);
    va_end(list);

    if(v == ~0ULL)

```

```

        return read(arg, nil);

    return read(arg, &v);
}

<Dev methods 275a>+≡ (54a) <269a 276b>
    long (*read)(Chan*, void*, long, vlong);

<function read 275b>≡ (654)
    // long pread(int fd, void *buf, long nbytes, vlong offset);
    static long
    read(ulong *arg, vlong *offp)
    {
        long n, nn, nnn;
        byte *p;
        Chan *c;
        vlong off;

        n = arg[2];
        validaddr(arg[1], n, true);
        p = (void*)arg[1];
        c = fdtochan(arg[0], OREAD, true, true);

        if(waserror()){
            cclose(c);
            nexterror();
        }

        if(offp == nil) /* use and maintain channel's offset */
            off = c->offset;
        else
            off = *offp;

        if(off < 0)
            error(Enegoff);

        if(off == 0){ /* rewind to the beginning of the directory */
            if(offp == nil){
                c->offset = 0;
                c->devoffset = 0;
            }
            <read() rewind when off == 0 315b>
        }

        <read() if c is a QTDIR 314g>
        else
            nnn = nn = devtab[c->type]->read(c, p, n, off);

        lock(c);
        c->offset += nnn;
        c->devoffset += nn;
        unlock(c);

        poperror();
        cclose(c);

        return nnn;
    }
}

```

⟨syscall pwrite 276a⟩≡ (654)

```
// long pwrite(int fd, void *buf, long nbytes, vlong offset);
long
syspwrite(ulong* arg)
{
    vlong v;
    va_list list;

    /* use varargs to guarantee alignment of vlong */
    va_start(list, arg[2]);
    v = va_arg(list, vlong);
    va_end(list);

    if(v == ~0ULL)
        return write(arg, nil);

    return write(arg, &v);
}
```

⟨Dev methods 276b⟩+≡ (54a) <275a 286a>

```
long (*write)(Chan*, void*, long, vlong);
```

⟨function write 276c⟩≡ (654)

```
// long pwrite(int fd, void *buf, long nbytes, vlong offset);
static long
write(ulong *arg, vlong *offp)
{
    Chan *c;
    long m, n;
    vlong off;

    validaddr(arg[1], arg[2], false);
    n = 0;
    c = fdtochan(arg[0], OWRITE, true, true);
    if(waserror()) {
        if(offp == nil){
            lock(c);
            c->offset -= n;
            unlock(c);
        }
        cclose(c);
        nexterror();
    }

    if(c->qid.type & QTDIR)
        error(Eisdir);

    n = arg[2];

    if(offp == nil){ /* use and maintain channel's offset */
        lock(c);
        off = c->offset;
        c->offset += n;
        unlock(c);
    }else
        off = *offp;

    if(off < 0)
        error(Enegoff);
}
```

```

    m = devtab[c->type]->write(c, (void*)arg[1], n, off);

    if(offp == nil && m < n){
        lock(c);
        c->offset -= n - m;
        unlock(c);
    }

    poperror();
    cclose(c);

    return m;
}

```

12.8 sysseek()

There is no `seek` method in the `\hlinkDev` interface — `seek` is handled entirely by the kernel by updating `c->offset`. The device never needs to know about seeks because it receives the offset on every read and write call. For type-2 seeks (from end of file), the kernel calls `stat()` to learn the file length. Directories have restricted seek: you can only rewind to offset 0.

<syscall seek 277a> ≡ (654)

```

// vlong seek(int fd, vlong n, int type);
long
sysseek(ulong* arg)
{
    validaddr(arg[0], BY2V, true);
    arch_validalign(arg[0], sizeof(vlong));
    sseek(arg);
    return 0;
}

```

<function sseek 277b> ≡ (654)

```

union v_or_u2 {
    vlong v;
    ulong u[2];
};

// vlong seek(int fd, vlong n, int type);
static void
sseek(ulong *arg)
{
    Chan *c;
    byte buf[sizeof(DirEntry)+100];
    DirEntry dir;
    int n;
    vlong off;
    union v_or_u2 o;

    c = fdtochan(arg[1], -1, true, true);
    if(waserror()){
        cclose(c);
        nexterror();
    }
    <sseek() and pipes 402b>

    off = 0;
    o.u[0] = arg[2];
}

```

```

o.u[1] = arg[3];
switch(arg[4]){
case 0: // from the start
    off = o.v;
    <sseek() ensures off is 0 for directories 282c>
    if(off < 0)
        error(Enegoff);
    c->offset = off; // just write, no need for lock
    break;

case 1: // from the current location
    <sseek() disallows seek type 1 or 2 for directories 282d>
    lock(c); /* lock for read/write update */
    off = o.v + c->offset;
    if(off < 0){
        unlock(c);
        error(Enegoff);
    }
    c->offset = off;
    unlock(c);
    break;

case 2: // from the end of the file
    <sseek() disallows seek type 1 or 2 for directories 282d>
    n = devtab[c->type]->stat(c, buf, sizeof buf);
    if(convM2D(buf, n, &dir, nil) == 0)
        error("internal error: stat error in seek");
    off = dir.length + o.v;
    if(off < 0)
        error(Enegoff);
    c->offset = off;
    break;

default:
    error(Ebadarg);
}
*(vlong*)arg[0] = off;
c->uri = 0;
c->dri = 0;
cclose(c);
poperror();
}

```

12.9 Collecting unclosed files

When a process exits, `\hlinkclosefgrp` iterates over all open file descriptors and calls `\hlinkcclose` on each. But closing a channel that refers to a remote file server (via the mount device) can block or even deadlock — for instance, if the file server is the dying process itself. If a kill signal arrives during this teardown, `\hlinkforceclosefgrp` hands the remaining unclosed channels to a background “clunk queue” served by a dedicated kernel process, breaking the deadlock.

```

<Proc files fields 278a>+≡ (49f) <261a 303a>
    Fgrp *closingfgrp; /* used during teardown */

<sleep() forceclosefgrp 278b>≡ (395g)
    if(up->procctl == Proc_exitme && up->closingfgrp)
        forceclosefgrp();

```

<function forceclosefgrp 279a>≡ (673b)

```
/*
 * Called from sleep because up is in the middle
 * of closefgrp and just got a kill ctl message.
 * This usually means that up has wedged because
 * of some kind of deadly embrace with mntclose
 * trying to talk to itself. To break free, hand the
 * unclosed channels to the close queue. Once they
 * are finished, the blocked cclose that we've
 * interrupted will finish by itself.
 */
void
forceclosefgrp(void)
{
    int i;
    Chan *c;
    Fgrp *f;

    if(up->procctl != Proc_exitme || up->closingfgrp == nil){
        print("bad forceclosefgrp call");
        return;
    }

    f = up->closingfgrp;
    for(i = 0; i <= f->maxfd; i++){
        if(c = f->fd[i]){
            f->fd[i] = nil;
            ccloseq(c);
        }
    }
}
```

<struct Clunkq 279b>≡ (647)

```
/*
 * Queue a chan to be closed by one of the clunk procs.
 */
struct Clunkq {
    Chan *head;
    Chan *tail;
    int nqueued;
    int nclosed;
    Lock l;
    QLock q;
    Rendez r;
};
```

<global clunkq 279c>≡ (647)

```
struct Clunkq clunkq;
```

<function ccloseq 279d>≡ (647)

```
void
ccloseq(Chan *c)
{
    if(c->flag&CFREE)
        panic("cclose %#p", getcallerpc(&c));

    DBG("ccloseq %p name=%s ref=%ld\n", c, c->path->s, c->ref);

    if(deceref(c))
        return;
}
```



```

    lock(&clunkq.l);
    clunkq.nqueued++;
    c->next = nil;
    if(clunkq.head)
        clunkq.tail->next = c;
    else
        clunkq.head = c;
    clunkq.tail = c;
    unlock(&clunkq.l);

    if(!wakeup(&clunkq.r))
        kproc("kcloseproc", closeproc, nil);
}

⟨function clunkwork 280a⟩≡ (647)
static int
clunkwork(void*)
{
    return clunkq.head != nil;
}

⟨function closeproc 280b⟩≡ (647)
void
closeproc(void*)
{
    Chan *c;

    for(;;){
        qlock(&clunkq.q);
        if(clunkq.head == nil){
            if(!waserror()){
                tsleep(&clunkq.r, clunkwork, nil, 5000);
                poperror();
            }
            if(clunkq.head == nil){
                qunlock(&clunkq.q);
                pexit("no work", /*freemem*/true);
            }
        }
        lock(&clunkq.l);
        c = clunkq.head;
        clunkq.head = c->next;
        clunkq.nclosed++;
        unlock(&clunkq.l);
        qunlock(&clunkq.q);
        if(!waserror()){
            devtab[c->type]->close(c);
            poperror();
        }
        chanfree(c);
    }
}

```

12.10 Special opening modes

Two optional flags can be OR'd into the open mode: `OCEXEC` causes the fd to be automatically closed when the process calls `exec()` (preventing fd leaks into child programs), and `ORCLOSE` causes the file to be removed when

the fd is closed (useful for temporary files). These flags are stored in the channel's `flag` field as `CCEXEC` and `CRCLOSE` respectively, since `c->mode` only holds the base access mode.

12.10.1 Close on exec

```

<OpenFlags cases 281a>+≡ (264a) <267b 281g>
    OCEXEC = 32, /* or'ed in, close on exec */

<ChannelFlag cases 281b>≡ (257b) 281h>
    CCEXEC = 0x0008, /* close on exec */

<namec() set channel flag after open 281c>≡ (267g) 281i>
    if(omode & OCEXEC)
        c->flag |= CCEXEC;

<namec() set channel flag after create 281d>≡ (291d) 281j>
    if(omode & OCEXEC)
        cnew->flag |= CCEXEC;

<sysexec() locals 281e>+≡ (119d) <170c 448h>
    Fgrp *f;

<sysexec() close files marked as opened with close on exec 281f>≡ (119d)
    /*
     * Close on exec
     */
    f = up->fgrp;
    for(i=0; i<=f->maxfd; i++)
        fdclose(i, CCEXEC);

```

12.10.2 Remove on close

```

<OpenFlags cases 281g>+≡ (264a) <281a 293a>
    ORCLOSE = 64, /* or'ed in, remove on close */

<ChannelFlag cases 281h>+≡ (257b) <281b 520a>
    CRCLOSE = 0x0020, /* remove on close */

<namec() set channel flag after open 281i>+≡ (267g) <281c
    if(omode & ORCLOSE)
        c->flag |= CRCLOSE;

<namec() set channel flag after create 281j>+≡ (291d) <281d
    if(omode & ORCLOSE)
        cnew->flag |= CRCLOSE;

```

Chapter 13

Directories

```
⟨systab directory syscalls 282a⟩≡ (41f)
[CREATE]      syscreate,
[REMOVE]      sysremove,

[CHDIR]       syschdir,
[FD2PATH]     sysfd2path, // =~ pwd

[STAT]        sysstat,
[FSTAT]       sysfstat,
[WSTAT]       syswstat,
[FWSTAT]      sysfwstat,
```

13.1 Overview

This chapter covers the directory-related syscalls: `create`, `remove`, `chdir`, `fd2path`, `stat/fstat`, and `wstat/fwstat`. Directories in Plan 9 are not fundamentally different from files — they are channels whose `qid.type` has the `QTDIR` bit set. The kernel enforces a few directory-specific rules (no writing, no exec, restricted seek), but most directory operations are delegated to device drivers via the same `\hlinkDev` method table. This chapter also presents the `\hlinkPath` structure for tracking human-readable pathnames, the `\hlinkwalk` function that resolves path elements one by one through device drivers, and the `\hlinkDirEntry` structure (Plan 9’s equivalent of UNIX’s `struct stat`).

```
⟨function error_if_not_dir 282b⟩≡ (647)
void
error_if_not_dir(Chan *c)
{
    if(c->qid.type & QTDIR)
        return;
    error(Enotdir);
}
```

```
⟨sseek() ensures off is 0 for directories 282c⟩≡ (277b)
if((c->qid.type & QTDIR) && off != 0)
    error(Eisdir);
```

```
⟨sseek() disallows seek type 1 or 2 for directories 282d⟩≡ (277)
if(c->qid.type & QTDIR)
    error(Eisdir);
```

```

⟨namec() error if not a directory or cannot exec directory 283a⟩≡ (267g)
    if(e.mustbedir && !(c->qid.type & QTDIR))
        error(Enotdir);

    if((amode == Aopen) && (omode & OEXEC) && (c->qid.type & QTDIR))
        error("cannot exec directory");

```

13.2 Path

Every `\hlinkChan` carries a `\hlinkPath` that records the human-readable filename used to reach it (e.g., `“/usr/pad/foo.c”`). The path is needed for `fd2path()` (the Plan 9 equivalent of “`pwd`” for arbitrary fds) and for good error messages. One might think the kernel could reconstruct the path by walking “`..`” back to the root, but mount points make that impossible — the “`..`” of a mounted directory leads to a different place than the “`..`” of the underlying device. So the path must be tracked explicitly as the name is resolved. `\hlinkPath` is reference-counted and uses copy-on-write via `\hlinkuniquepath`.

```

⟨struct Path 283b⟩≡ (645c)
    struct Path
    {
        char *s;
        int len;    /* strlen(s) */
        int alen;   /* allocated length of s */

        ⟨Path mtpt fields 311b⟩

        // extra
        Ref;
    };

```

```

⟨constructor newpath 283c⟩≡ (647)
    Path*
    newpath(char *s)
    {
        int i;
        Path *p;

        p = smalloc(sizeof(Path));
        i = strlen(s);
        p->len = i;
        p->alen = i+PATHSLOP;
        p->s = smalloc(p->alen);
        memmove(p->s, s, i+1);
        p->ref = 1;
        incref(&npath);

        ⟨newpath() mtpt handling 312a⟩
        return p;
    }

```

```

⟨constant PATHSLOP 283d⟩≡ (647)
    PATHSLOP    = 20,

```

```

⟨function copypath 283e⟩≡ (647)
    static Path*
    copypath(Path *p)
    {
        int i;

```

```

Path *pp;

pp = smalloc(sizeof(Path));
pp->ref = 1;
incref(&npath);
DBG("copypath %s %p => %p\n", p->s, p, pp);

pp->len = p->len;
pp->alen = p->alen;
pp->s = smalloc(p->alen);
memmove(pp->s, p->s, p->len+1);

⟨copypath() mtpt handling 312b⟩
return pp;
}

⟨destructor pathclose 284a⟩≡ (647)
void
pathclose(Path *p)
{
    int i;

    if(p == nil)
        return;
    ⟨pathclose() debugging ??⟩
    if(decref(p))
        return;

    decref(&npath);
    free(p->s);

    ⟨pathclose() mtpt handling 312c⟩
    free(p);
}

⟨function uniquepath 284b⟩≡ (647)
static Path*
uniquepath(Path *p)
{
    Path *new;

    if(p->ref > 1){
        /* copy on write */
        new = copypath(p);
        pathclose(p);
        p = new;
    }
    return p;
}

```

`\hlinkaddelem` appends a path element to a `\hlinkPath`, growing the string buffer if needed. For “..” it calls `\hlinkfixdotdotname` to clean the path in place. The `from` parameter records the mount point channel at this position (or nil), which is stored in the `mtpt` array for later “..” handling.

```

⟨function addelem 284c⟩≡ (647)
static Path*
addelem(Path *p, char *s, Chan *from)
{
    char *t;
    int a, i;

```

```

Chan *c, **tt;

if(s[0]=='.' && s[1]=='\0')
    return p;

p = uniquepath(p);

i = strlen(s);
// +1 for the '\0'
if(p->len+1+i+1 > p->alen){
    a = p->len+1+i+1 + PATHSLOP;
    t = smalloc(a);
    memmove(t, p->s, p->len+1);
    free(p->s);
    p->s = t;
    p->alen = a;
}
/* don't insert extra slash if one is present */
if(p->len>0 && p->s[p->len-1]!='/' && s[0]!='/')
    p->s[p->len++] = '/';
memmove(p->s+p->len, s, i+1);
p->len += i;

if(isdotdot(s)){
    fixdotdotname(p);
}
⟨addelem() mtpt handling 312d⟩

return p;
}

```

13.3 ..

⟨function isdotdot 285a⟩≡ (647)

```

int
isdotdot(char *p)
{
    return p[0]=='.' && p[1]=='.' && p[2]=='\0';
}

```

⟨function fixdotdotname 285b⟩≡ (647)

```

/*
 * In place, rewrite name to compress multiple /, eliminate ., and process ..
 * (Really only called to remove a trailing .. that has been added.
 * Otherwise would need to update n->mtpt as well.)
 */
static void
fixdotdotname(Path *p)
{
    char *r;

    if(p->s[0] == '#'){
        r = strchr(p->s, '/');
        if(r == nil)
            return;
        cleannname(r);

        /*

```

```

    * The correct name is #i rather than #i/,
    * but the correct name of #/ is #/.
    */
    if(strcmp(r, "/")==0 && p->s[1] != '/')
        *r = '\\0';
}
    }else
        cleannname(p->s);
    p->len = strlen(p->s);
}

```

13.4 walk()

The `walk \hlinkDev` method takes an array of path element names and walks through them as far as possible within one device. It returns a `\hlinkWalkqid` containing the cloned channel and an array of `Qids` for each element successfully walked. Taking multiple names in a single call avoids a 9P round-trip per path element.

```

<Dev methods 286a>+≡ (54a) <276b 291c>
    Walkqid*(*walk)(Chan*, Chan*, char**, int);

```

```

<struct Walkqid 286b>≡ (645c)
    struct Walkqid
    {
        Chan *clone;
        int nqid;
        // variable array length, size = nqid
        Qid qid[1];
    };

```

```

<walk() locals 286c>≡ (287) 286d>
    Chan *c;
    Path *path;
    Walkqid *wq;
    Chan *nc;

```

```

<walk() locals 286d>+≡ (287) <286c 286e>
    Mhead *mh;
    Mhead *nmh;
    int type, dev;

```

```

<walk() locals 286e>+≡ (287) <286d 286f>
    Chan *mtpt;
    bool didmount;

```

```

<walk() locals 286f>+≡ (287) <286e 286g>
    bool dotdot;

```

```

<walk() locals 286g>+≡ (287) <286f ??>
    int nhave, ntry;

```

```

<namec() locals 286h>+≡ (267g) <273b 291b>
    bool sharppath;

```

The kernel-level `\hlinkwalk` is the mount-point-aware layer above the device walk method. Its loop processes batches of path elements: it calls the device's walk for a run of names (stopping before any “.” element), then checks each intermediate Qid for mount points via `\hlinkfindmount`. When it hits a mount point mid-path, it crosses to the mounted channel and continues walking in the new device. The “.” element is handled specially: it may need to uncross a mount point via `\hlinkundomount`, returning to the channel that was originally mounted upon. The `didmount` flag tracks which side of a mount point the current channel is on.

```

⟨function walk 287⟩≡ (647)
/*
 * Either walks all the way or not at all.  No partial results in *cp.
 * *nerror is the number of names to display in an error message.
 */
int
walk(Chan **cp, char **names, int nnames, bool sharppath, int *nerror)
{
    ⟨walk() locals 286c⟩

    c = *cp;
    incref(c);
    path = c->path;
    incref(path);

    mh = nil;

    /*
     * While we haven't gotten all the way down the path:
     *   1. step through a mount point, if any
     *   2. send a walk request for initial dotdot or initial prefix without dotdot
     *   3. move to the first mountpoint along the way.
     *   4. repeat.
     *
     * Each time through the loop:
     *
     *   If didmount==false, c is on the undomount side of the mount point.
     *   If didmount==true, c is on the domount side of the mount point.
     *   Either way, c's full path is path.
     */
    didmount = false;
    for(nhave=0; nhave<nnames; nhave+=n){

        ⟨walk() return error if channel c is not a dir 289c⟩

        ⟨walk() set dotdot and ntry to index of next '..' or end 289b⟩

        if(!sharppath && !dotdot && !didmount)
            domount(&c, &mh, &path);

        type = c->type;
        dev = c->dev;

        if((wq = ewalk(c, names+nhave, ntry)) == nil){
            // didn't find the element, look in union mount
            ⟨walk() if c is a union mount, find and adjust type and dev 313b⟩
            ⟨walk() if wq is still nil, close and return 289d⟩
        }

        didmount = false;
        if(dotdot){
            assert(wq->nqid == 1);

```



```

    assert(wq->clone != nil);
    path = addelem(path, "..", nil);
    nc = undomount(wq->clone, path);
    nmh = nil;
    n = 1;
}else{
    nc = nil;
    nmh = nil;
    if(!sharppath){
        // mount points along path?
        for(i=0; i<wq->nqid && i<ntry-1; i++){
            if(findmount(&nc, &nmh, type, dev, wq->qid[i])){
                didmount = true;
                break;
            }
        }
    }

    if(!didmount){ /* no mount points along path */
        <walk() if nc == nil and wq->clone == nil return error 290a>
        n = wq->nqid;
        nc = wq->clone;
    }else{ /* stopped early, at a mount point */
        if(wq->clone != nil){
            cclose(wq->clone);
            wq->clone = nil;
        }
        n = i+1;
    }

    for(i=0; i<n; i++){
        mtpt = nil;
        if(i==n-1 && nmh)
            mtpt = nmh->from;
        path = addelem(path, names[nhave+i], mtpt);
    }
}
cclose(c);
c = nc;
putmhead(mh);
mh = nmh;
free(wq);
}

putmhead(mh);

c = uniquechan(c); // late? why now? we've modified in place c already

<walk() print error if c->umh != nil 314d>

pathclose(c->path);
c->path = path;

cclose(*cp);
*cp = c;
if(nerror)
    *nerror = nhave;
return 0;
}

```

<function ewalk 289a>≡ (647)

```
/*
 * Call dev walk but catch errors.
 */
static Walkqid*
ewalk(Chan *c, char **names, int nnames)
{
    Walkqid *wq;

    if(waserror())
        return nil;
    wq = devtab[c->type]->walk(c, nil, names, nnames);
    poperror();
    return wq;
}
```

<walk() set dotdot and ntry to index of next '..' or end 289b>≡ (287)

```
dotdot = false;
// remaining names
ntry = nnames - nhave;
if(ntry > MAXWELEM)
    ntry = MAXWELEM;

for(i=0; i<ntry; i++){
    if(isdotdot(names[nhave+i])){
        if(i==0){
            dotdot = true;
            ntry = 1;
        }else
            ntry = i;
        break;
    }
}
// ntry is now the index of the next '..' or the remaining names to walk
```

<walk() return error if channel c is not a dir 289c>≡ (287)

```
if(!(c->qid.type & QTDIR)){
    pathclose(path);
    cclose(c);
    if(nerror)
        *nerror = nhave;
    if(mh != nil)
        putmhead(mh);
    strcpy(up->errstr, Enotdir);
    return -1;
}
```

<walk() if wq is still nil, close and return 289d>≡ (287)

```
if(wq == nil){
    pathclose(path);
    cclose(c);
    if(nerror)
        *nerror = nhave+1;
    if(mh != nil)
        putmhead(mh);
    return -1;
}
```

`<walk() if nc == nil and wq->clone == nil return error 290a>≡` (287)

```

    if(wq->clone == nil){
        pathclose(path);
        cclose(c);
        if(wq->nqid==0 || (wq->qid[wq->nqid-1].type&QTDIR)){
            if(nerror)
                *nerror = nhave+wq->nqid+1;
            strcpy(up->errstr, Edoesnotexist);
        }else{
            if(nerror)
                *nerror = nhave+wq->nqid;
            strcpy(up->errstr, Enotdir);
        }
        free(wq);
        if(mh != nil)
            putmhead(mh);
        return -1;
    }

```

`<function cclone 290b>≡` (647)

```

Chan*
ccclone(Chan *c)
{
    Chan *nc;
    Walkqid *wq;

    wq = devtab[c->type]->walk(c, nil, nil, 0);

    if(wq == nil)
        error("clone failed");
    nc = wq->clone;
    free(wq);
    nc->path = c->path;
    if(c->path)
        incref(c->path);
    return nc;
}

```

13.5 syscreate()

The `create` syscall is structurally similar to `open`, but calls `namec()` with `Acreate` mode. The interesting complexity is in the `namec()` `Acreate` case, which must handle the “create/create race”: if two processes try to create the same file simultaneously, one `create(5)` 9P message will succeed and the other will fail. The losing process retries with `open+OTRUNC` instead, providing the expected semantics where `create` on an existing file truncates it. The `OEXCL` flag lets applications that care about atomicity get the raw “fail if exists” behavior.

`<syscall create 290c>≡` (654)

```

// int create(char *file, int omode, ulong perm);
long
syscreate(ulong* arg)
{
    int fd;
    Chan *c;

    openmode(arg[1]&~OEXCL); /* error check only; OEXCL okay here */
    validaddr(arg[0], 1, false);
    c = namec((char*)arg[0], Acreate, arg[1], arg[2]);
    if(waserror()){

```

```

        cclose(c);
        nexterror();
    }
    fd = newfd(c);
    if(fd < 0)
        error(Enofd);
    poperror();
    return fd;
}

⟨namec() adjust Elemlist e if Acreate 291a⟩≡ (267g)
    if(amode == Acreate){
        /* perm must have DMDIR if last element is / or /. */
        if(e.mustbedir && !(perm&DMDIR)){
            e.nerror = e.nelems;
            error("create without DMDIR");
        }

        /* don't try to walk the last path element just yet. */
        if(e.nelems == 0)
            error(Eexist);
        e.nelems--;
    }

⟨namec() locals 291b⟩+≡ (267g) <286h 304a>
    Chan *cnew;
    char *createerr;

⟨Dev methods 291c⟩+≡ (54a) <286a 293c>
    void (*create)(Chan*, char*, int, ulong);

⟨namec() other cases 291d⟩≡ (267g) 294b>
    case Acreate:
        /*
         * We've already walked all but the last element.
         * If the last exists, try to open it OTRUNC.
         * If omode&OEXCL is set, just give up.
         */
        e.nelems++;
        e.nerror++;
        if(walk(&c, e.elems+e.nelems-1, 1, sharppath, nil) == 0){
            if(omode&OEXCL)
                error(Eexist);
            omode |= OTRUNC;
            goto Open;
        }

        /*
         * The semantics of the create(2) system call are that if the
         * file exists and can be written, it is to be opened with truncation.
         * On the other hand, the create(5) message fails if the file exists.
         * If we get two create(2) calls happening simultaneously,
         * they might both get here and send create(5) messages, but only
         * one of the messages will succeed. To provide the expected create(2)
         * semantics, the call with the failed message needs to try the above
         * walk again, opening for truncation. This correctly solves the
         * create/create race, in the sense that any observable outcome can
         * be explained as one happening before the other.
         * The create/create race is quite common. For example, it happens
         * when two rc subshells simultaneously update the same

```

```

* environment variable.
*
* The implementation still admits a create/create/remove race:
* (A) walk to file, fails
* (B) walk to file, fails
* (A) create file, succeeds, returns
* (B) create file, fails
* (A) remove file, succeeds, returns
* (B) walk to file, return failure.
*
* This is hardly as common as the create/create race, and is really
* not too much worse than what might happen if (B) got a hold of a
* file descriptor and then the file was removed -- either way (B) can't do
* anything with the result of the create call. So we don't care about this race.
*
* Applications that care about more fine-grained decision of the races
* can use the OEXCL flag to get at the underlying create(5) semantics;
* by default we provide the common case.
*
* We need to stay behind the mount point in case we
* need to do the first walk again (should the create fail).
*
* We also need to cross the mount point and find the directory
* in the union in which we should be creating.
*
* The channel staying behind is c, the one moving forward is cnew.
*/
m = nil;
cnew = nil; /* is this assignment necessary? */
if(!waserror()){ /* try create */
    <namec() Acreate case, if cnew is a mountpoint 316b>
    else{
        cnew = c;
        incref(cnew);
    }

    /*
     * We need our own copy of the Chan because we're
     * about to send a create, which will move it. Once we have
     * our own copy, we can fix the name, which might be wrong
     * if findmount gave us a new Chan.
     */
    cnew = uniquechan(cnew);
    pathclose(cnew->path);
    cnew->path = c->path;
    incref(cnew->path);

    devtab[cnew->type]->create(cnew, e.elems[e.nelems-1], omode&~(OEXCL|OCEXEC), perm);
    poperror();
    <namec() set channel flag after create 281d>
    if(m)
        putmhead(m);
    cclose(c);
    c = cnew;
    c->path = addelem(c->path, e.elems[e.nelems-1], nil);
    break; // switch
}

/* create failed */
cclose(cnew);

```

```

if(m)
    putmhead(m);
if(omode & OEXCL)
    nexterror();

/* save error */
createerr = up->errstr;
up->errstr = tmperrbuf;
/* note: we depend that walk does not error */
if(walk(&c, e.elems+e.nelems-1, 1, sharppath, nil) < 0){
    up->errstr = createerr;
    error(createerr); /* report true error */
}
up->errstr = createerr;
omode |= OTRUNC;
goto Open;

```

<OpenFlags cases 293a>+≡ (264a) <281g 293b>
 OTRUNC = 16, /* or'ed in (except for exec), truncate file first */

<OpenFlags cases 293b>+≡ (264a) <293a
 OEXCL = 0x1000, /* or'ed in, exclusive create */

13.6 sysremove()

Remove takes a filename, resolves it via `namec()` with `Aremove`, and calls the device's `remove()` method. Removing a mount point is explicitly disallowed — the ambiguity of whether to remove the mount point or the mounted channel is resolved by simply forbidding it. After `remove()`, the channel's type is reset to 0 (root device) because the 9P remove clunks the fid, so the channel must not call the original device's `close()` again.

```

<Dev methods 293c>+≡ (54a) <291c 296b>
void (*remove)(Chan*);

```

<syscall remove 293d>≡ (654)

```

// int remove(char *file);
long
sysremove(ulong* arg)
{
    Chan *c;

    validaddr(arg[0], 1, false);
    c = namec((char*)arg[0], Aremove, 0, 0);
    /*
     * Removing mount points is disallowed to avoid surprises
     * (which should be removed: the mount point or the mounted Chan?).
     */
    if(c->ismtpt){
        cclose(c);
        error(Eismtpt);
    }
    if(waserror()){
        c->type = 0; /* see below */
        cclose(c);
        nexterror();
    }
    devtab[c->type]->remove(c);
    /*
     * Remove clunks the fid, but we need to recover the Chan

```

```

    * so fake it up.  rootclose() is known to be a nop.
    */
    c->type = 0;
    poperror();
    cclose(c);
    return 0;
}

```

13.7 syschdir()

Changing directory is simple: resolve the path via `namec()` with `Atodir` mode (which verifies it is a directory), then replace `up->dot`.

```

⟨syscall chdir 294a⟩≡ (654)
// int chdir(char *dirname);
long
syschdir(ulong* arg)
{
    Chan *c;

    validaddr(arg[0], 1, false);
    c = namec((char*)arg[0], Atodir, 0, 0);
    cclose(up->dot);
    up->dot = c;
    return 0;
}

```

```

⟨namec() other cases 294b⟩+≡ (267g) <291d 305d>
case Atodir:
    /*
     * Directories (e.g. for cd) are left before the mount point,
     * so one may mount on / or . and see the effect.
     */
    if(!(c->qid.type & QTDIR))
        error(Enotdir);
    break;

```

13.8 sysfd2path() (a.k.a pwd())

`fd2path` returns the full pathname associated with an open file descriptor. This is why the kernel must track `\hlinkPath` objects on every channel — without them, there would be no way to answer “what file is fd 3 pointing to?”. The implementation is trivial: look up the fd, return `c->path->s`.

```

⟨syscall fd2path 294c⟩≡ (654)
// int fd2path(int fd, char *buf, int nbuf);
long
sysfd2path(ulong* arg)
{
    Chan *c;

    validaddr(arg[1], arg[2], true);
    c = fdtochan(arg[0], -1, false, true);
    snprintf((char*)arg[1], arg[2], "%s",
             chanpath(c));
    cclose(c);
    return 0;
}

```

```

⟨function chanpath 295a⟩≡ (647)
char*
chanpath(Chan *c)
{
    if(c == nil)
        return "<nil chan>";
    if(c->path == nil)
        return "<nil path>";
    if(c->path->s == nil)
        return "<nil path.s>";
    return c->path->s;
}

```

13.9 DirEntry

Where UNIX uses numeric uid/gid, Plan 9 uses string usernames directly in `\hlinkDirEntry`, eliminating the need for `/etc/passwd`. The structure is serialized into a platform-independent wire format matching the 9P stat message.

```

⟨struct DirEntry 295b⟩≡ (656e)
struct DirEntry {
    /* system-modified data */
    ushort type; /* server type */
    uint dev; /* server subtype */

    /* file data */
    Qid qid; /* unique id from server */
    // bitset<enum<dirmode>>
    ulong mode; /* permissions */

    ulong atime; /* last read time */
    ulong mtime; /* last write time */

    vlong length; /* file length: see <u.h> */
    char *name; /* last element of path */

    char *uid; /* owner name */
    char *gid; /* group name */
    char *muid; /* last modifier name */
};

```

```

⟨enum dirmode 295c⟩≡ (656e)
/* bits in DirEntry.mode */
enum Dirmode {
    DMDIR = 0x80000000, /* mode bit for directories */

    DMREAD = 0x4, /* mode bit for read permission */
    DMWRITE = 0x2, /* mode bit for write permission */
    DMEXEC = 0x1, /* mode bit for execute permission */
    ⟨Dirmode cases 379a⟩
};

```

```

⟨function direntry_setname 295d⟩≡ (654)
static long
direntry_setname(char *name, int len, byte *p, long n, long maxn)
{
    char *oname;
    int olen;
}

```



```

long nn;

if(n == BIT16SZ)
    return BIT16SZ;

oname = offsetof_name_dirent(p, &olen);

nn = n+len-olen;
PBIT16(p, nn-BIT16SZ);
if(nn > maxn)
    return BIT16SZ;

if(len != olen)
    memmove(oname+len, oname+olen, p+n-(byte*)(oname+olen));
PBIT16((byte*)(oname-2), len);
memmove(oname, name, len);
return nn;
}

```

```

⟨function offsetof_name_dirent 296a⟩≡ (654)
static char*
offsetof_name_dirent(byte *p, int *n)
{
    p += BIT16SZ+BIT16SZ+BIT32SZ +
        BIT8SZ+BIT32SZ+BIT64SZ +
        BIT32SZ+BIT32SZ+BIT32SZ +
        BIT64SZ;
    *n = GBIT16(p);
    return (char*)p+BIT16SZ;
}

```

13.10 sysstat()

The `stat/fstat` syscalls call the device's `stat()` method to fill a buffer with a serialized `\hlinkDirEntry`. After getting the raw `stat`, `sysstat()` rewrites the name field to match the last component of the path the user supplied, since the device may return a different name after crossing mount points.

```

⟨Dev methods 296b⟩+≡ (54a) <293c 297c>
int (*stat)(Chan*, byte*, int);

⟨syscall stat 296c⟩≡ (654)
// int stat(char *name, byte *edir, int nedit);
long
sysstat(ulong* arg)
{
    char *name;
    Chan *c;
    uint l;

    l = arg[2];
    validaddr(arg[1], l, true);
    validaddr(arg[0], 1, false);
    c = namec((char*)arg[0], Aaccess, 0, 0);
    if(waserror()){
        cclose(c);
        nexterror();
    }
    l = devtab[c->type]->stat(c, (byte*)arg[1], 1);
}

```

```

    name = pathlast(c->path);
    if(name)
        l = direntry_setname(name, strlen(name), (byte*)arg[1], 1, arg[2]);

    poperror();
    cclose(c);
    return l;
}

```

⟨*syscall fstat* 297a⟩≡ (654)

```

// int fstat(int fd, byte *edir, int nedit);
long
sysfstat(ulong* arg)
{
    Chan *c;
    uint l;

    l = arg[2];
    validaddr(arg[1], l, true);
    c = fdtochan(arg[0], -1, false, true);
    if(waserror()) {
        cclose(c);
        nexterror();
    }
    l = devtab[c->type]->stat(c, (byte*)arg[1], l);
    poperror();
    cclose(c);
    return l;
}

```

⟨*function pathlast* 297b⟩≡ (654)

```

static char*
pathlast(Path *p)
{
    char *s;

    if(p == nil)
        return nil;
    if(p->len == 0)
        return nil;
    s = strrchr(p->s, '/');
    if(s)
        return s+1;
    return p->s;
}

```

⟨*Dev methods* 297c⟩+≡ (54a) <296b 304c>

```

int (*wstat)(Chan*, byte*, int);

```

The `wstat` syscalls modify file metadata. The kernel validates the `stat` buffer and checks that mount points cannot be renamed (to avoid ambiguity about what is being renamed — the mount point or the mounted channel).

⟨*syscall wstat* 297d⟩≡ (654)

```

// int wstat(char *name, byte *edir, int nedit);
long
syswstat(ulong* arg)
{
    Chan *c;
    uint l;

```

```

    l = arg[2];
    validaddr(arg[1], l, false);
    validstat((byte*)arg[1], l);
    validaddr(arg[0], 1, false);
    c = namec((char*)arg[0], Aaccess, 0, 0);
    return wstat(c, (byte*)arg[1], l);
}

```

<syscall fstat 298a>≡ (654)

```

// int fstat(int fd, byte *edir, int n edir);
long
sysfstat(ulong* arg)
{
    Chan *c;
    uint l;

    l = arg[2];
    validaddr(arg[1], l, false);
    validstat((byte*)arg[1], l);
    c = fdtochan(arg[0], -1, true, true);
    return wstat(c, (byte*)arg[1], l);
}

```

<function validstat 298b>≡ (651b)

```

void
validstat(byte *s, int n)
{
    int m;
    char buf[64];

    if(statcheck(s, n) < 0)
        error(Ebadstat);
    /* verify that name entry is acceptable */
    s += STATFIXLEN - 4*BIT16SZ;    /* location of first string */
    /*
     * s now points at count for first string.
     * if it's too long, let the server decide; this is
     * only for his protection anyway. otherwise
     * we'd have to allocate and waserror.
     */
    m = GBIT16(s);
    s += BIT16SZ;
    if(m+1 > sizeof buf)
        return;
    memmove(buf, s, m);
    buf[m] = '\0';
    /* name could be '/' */
    if(strcmp(buf, "/") != 0)
        validname(buf, false);
}

```

<function wstat 298c>≡ (654)

```

static long
wstat(Chan *c, byte *d, int nd)
{
    long l;
    int namelen;

    if(waserror()){

```

```

        cclose(c);
        nexterror();
    }
    if(c->ismtpt){
        /*
         * Renaming mount points is disallowed to avoid surprises
         * (which should be renamed? the mount point or the mounted Chan?).
         */
        offsetof_name_dirent(d, &namelen);
        if(namelen)
            nameerror(chanpath(c), Eismtpt);
    }
    l = devtab[c->type]->wstat(c, d, nd);
    poperror();
    cclose(c);
    return l;
}

```

Chapter 14

Namespaces

```
<systab namespace syscalls 300a>≡ (41f)
[BIND]      sysbind,
[MOUNT]     sysmount,
[UNMOUNT]   sysunmount,
```

14.1 Overview

Namespaces are Plan 9’s most distinctive kernel feature. Each process has its own namespace — a private view of the file hierarchy, defined by a mount table (`\hlinkPgrp`). The `bind` and `mount` syscalls modify this table, and `\hlinknamec` consults it when resolving paths. A mount point pairs a “from” channel (`\hlinkMhead`) with one or more “to” channels (`\hlinkMount` list). Union mounts allow multiple directories to appear merged at a single path, ordered by `MBEFORE`, `MAFTER`, or replaced entirely with `MREPL`.

```
<enum mount 300b>≡ (656e)
enum MountFlags {
    MREPL = 0x0000, /* mount replaces object */

    MBEFORE = 0x0001, /* mount goes before others in union directory */
    MAFTER = 0x0002, /* mount goes after others in union directory */

    MCREATE = 0x0004, /* permit creation in mounted directory */
    <MountFlags cases 319c>

    MORDERMASK = 0x0003, /* mask for bits defining order of mounting */
    MMASK = 0x0017, /* all bits on */
};
```

An `\hlinkMhead` stores the “from” channel (the directory being mounted upon) and a linked list of `\hlinkMount` structures representing what is mounted there. For a simple bind or mount, there is one `\hlinkMount` entry. For a union mount, there are multiple entries in the list, and the order matters: when walking a name through a union directory, the kernel tries each mount’s channel in list order.

14.2 Mount point

```
<struct Mhead 300c>≡ (645c)
struct Mhead
{
    // ref<Chan>
    Chan* from; /* channel mounted upon */
    // list<ref<Mount>> list because of union mount (ordered via MBEFORE|MAFTER)
    Mount* mount; /* what’s mounted upon it */
};
```

```

    Ref;
    RWlock lock;
    ⟨Mhead extra fields 303c⟩
};

⟨constructor newmhead 301a⟩≡ (647)
Mhead*
newmhead(Chan *from)
{
    Mhead *mh;

    mh = smalloc(sizeof(Mhead));
    mh->ref = 1;
    mh->from = from;
    incref(from);
    return mh;
}

⟨destructor putmhead 301b⟩≡ (647)
/*
 * This is necessary because there are many
 * pointers to the top of a given mount list:
 *
 * - the mhead in the namespace hash table
 * - the mhead in chans returned from findmount:
 *   used in namec and then by unionread.
 * - the mhead in chans returned from createdir:
 *   used in the open/create race protect, which is gone.
 *
 * The RWlock in the Mhead protects the mount list it contains.
 * The mount list is deleted when we cunmount.
 * The RWlock ensures that nothing is using the mount list at that time.
 *
 * It is okay to replace c->mh with whatever you want as
 * long as you are sure you have a unique reference to it.
 *
 * This comment might belong somewhere else.
 */
void
putmhead(Mhead *m)
{
    if(m && decref(m) == 0){
        m->mount = (Mount*)0xCafeBeef;
        free(m);
    }
}

```

A `\hlinkMount` holds the “to” channel and flags. The `MBEFORE/MAFTER` ordering is implicit in list position.

```

⟨struct Mount 301c⟩≡ (645c)
struct Mount
{
    // ref<Chan>
    Chan* to;    /* channel replacing channel */

    // enum<mount>, but mainly for MCREATE, the MBEFORE|MAFTER is encoded via list
    int mflag;

    // option<string>
    char *spec; // for mount(), nil for bind()
}

```

```

    <Mount other fields 321a>

    <Mount extra fields 302b>
};

<constructor newmount 302a>≡ (673b)
Mount*
newmount(Chan *to, int flag, char *spec)
{
    Mount *m;

    m = smalloc(sizeof(Mount));
    m->to = to;
    incref(to);
    m->mountid = incref(&mountid);
    m->mflag = flag;
    if(spec != nil)
        kstrdup(&m->spec, spec);

    return m;
}

<Mount extra fields 302b>≡ (301c) 322a▷
Mount* next;

<destructor mountfree 302c>≡ (673b)
void
mountfree(Mount *m)
{
    Mount *f;

    while(m) {
        f = m->next;
        cclose(m->to);
        m->mountid = 0;
        free(m->spec);
        free(m);
        m = f;
    }
}

```

14.3 Mount table

The mount table `\hlinkPgrp` (“process group”) is the per-process namespace. It is a hash table of `\hlinkMhead` entries, keyed by `qid.path`. When `\hlinknamec` encounters a directory, it checks this table for mount points. The `Pgrp` is reference-counted; processes created with `RFNAMEG` get their own copy.

```

<struct Pgrp 302d>≡ (668c)
// Mount table, aka Namespace, aka process group
struct Pgrp
{
    // hash<qid.path, list<ref<Mhead> (next = Mhead.next)>
    Mhead *mnthash[MNTHASH];

    <Pgrp other fields 303d>

    // extra
    Ref; /* also used as a lock when mounting */
}

```

```

// ORDER OF LOCK: first debug and then ns
QLock debug;      /* single access via devproc.c */
RWlock ns;        /* Namespace n read/one write lock */
};

```

```

<Proc files fields 303a>+≡ (49f) <278a
// ref_counted<pgrp>
Pgrp *pgrp; /* Process group for namespace */

```

The mount table hash has only 32 buckets (MNTHASH), indexed by the low bits of `qid.path`. Collisions are handled by chaining `\hlinkMhead` entries via `Mhead.hash`. The small size is adequate because mount tables are typically small (a few dozen entries at most).

```

<function MOUNT 303b>≡ (668c)
enum
{
    MNTLOG = 5,
    MNTHASH = 1<<MNTLOG, /* Hash to walk mount table */
};
#define MOUNT(p,qid) ((p)->mnthash[(qid).path&((1<<MNTLOG)-1)])

```

```

<Mhead extra fields 303c>≡ (300c)
// hash<qid.path, list<ref<Mhead>> of Pgrp.mnthash
Mhead* hash; /* Hash chain */

```

```

<Pgrp other fields 303d>≡ (302d) 317e▷
ulong pgrpid;

```

```

<global pgrpid 303e>≡ (673b)
static Counter pgrpid;

```

```

<constructor newpgrp 303f>≡ (673b)
Pgrp*
newpgrp(void)
{
    Pgrp *p;

    p = smalloc(sizeof(Pgrp));
    p->ref = 1;
    p->pgrpid = incref(&pgrpid);
    return p;
}

```

```

<destructor closepgrp 303g>≡ (673b)
void
closepgrp(Pgrp *p)
{
    Mhead **h, **e, *f, *next;

    if(decref(p) != 0)
        return;

    qlock(&p->debug);
    wlock(&p->ns);
    p->pgrpid = -1;

    e = &p->mnthash[MNTHASH];
    for(h = p->mnthash; h < e; h++) {
        for(f = *h; f; f = next) {
            wlock(&f->lock);

```



```

        cclose(f->from);
        mountfree(f->mount);
        f->mount = nil;
        next = f->hash;
        wunlock(&f->lock);
        putmhead(f);
    }
}
wunlock(&p->ns);
qunlock(&p->debug);
free(p);
}

```

14.4 Device path

The “#” prefix in path names is how the kernel bootstraps the namespace. A path like “#c” directly attaches to the console device driver, bypassing the mount table entirely. During boot, `init0()` uses `bind("\#c", "/dev", MREPL)` to make device drivers accessible at conventional paths. This is Plan 9’s alternative to UNIX’s major/minor device numbers and static `/dev` entries — here the namespace is constructed dynamically by `bind` commands.

```

⟨namec() locals 304a⟩+≡ (267g) <291b 310a>
    Rune r;

⟨namec() if name[0] is a sharp 304b⟩≡ (267g)
    case '#':
        up->genbuf[0] = '\0';
        n = 0;
        while(*name != '\0' && (*name != '/' || n < 2)){ // "#/" is ok hence n < 2
            if(n >= sizeof(up->genbuf)-1)
                error(Efilename);
            up->genbuf[n++] = *name++;
        }
        up->genbuf[n] = '\0';

        n = chartorune(&r, up->genbuf+1)+1; // +1 for the leading '#'

⟨namec() noattach checking 318b⟩

    t = devno(r, true);
    if(t == -1)
        error(Ebadsharp);

    c = devtab[t]->attach(up->genbuf+n);
    break;

⟨Dev methods 304c⟩+≡ (54a) <297c 381a>
    Chan* (*attach)(char*);

```

14.5 sysbind()

Both `bind` and `mount` go through `\hlinkbindmount`, which resolves two paths and calls `\hlinkcmount` to insert a mount point.

```

⟨syscall bind 304d⟩≡ (654)
    // int bind(char *to, char *from, int flag);

```

```

long
sysbind(ulong* arg)
{
    return bindmount(false, -1, -1, (char*)arg[0], (char*)arg[1], arg[2], nil);
}

⟨bindmount() locals 305a⟩≡ (305b) ??▷
    Chan *c0;
    Chan *c1;

⟨function bindmount 305b⟩≡ (654)

long
bindmount(bool ismount, int fd, int afd, char* arg0, char* arg1, ulong flag, char* spec)
{
    ⟨bindmount() locals 305a⟩

    if((flag&~MMASK) || (flag&MORDERMASK)==(MBEFORE|MAFTER))
        error(Ebadarg);

    ⟨bindmount() if ismount 319a⟩
    else{
        spec = nil;
        validaddr((ulong)arg0, 1, false);
        c0 = namec(arg0, Abind, 0, 0);
    }

    if(waserror()){
        cclose(c0);
        nexterror();
    }

    validaddr((ulong)arg1, 1, false);
    c1 = namec(arg1, Amount, 0, 0);

    if(waserror()){
        cclose(c1);
        nexterror();
    }

    ret = cmount(c0, c1, flag, spec);

    poperror();
    cclose(c1);
    poperror();
    cclose(c0);
    ⟨bindmount() if ismount free 319b⟩
    return ret;
}

⟨AccessFlags cases 305c⟩≡ (267c)
    Abind,          /* for left-hand-side of bind */
    Amount,         /* to be mounted or mounted upon */

⟨namec() other cases 305d⟩+≡ (267g) ◁294b 306a▷
    case Abind:
        m = nil;
        if(!sharppath)
            /* no need to maintain path - cannot dotdot an Abind */ // so pass nil
            domount(&c, &m, nil);
        ⟨namec() set c-umh in Abind if mounted point 317a⟩
        break;

```

<namec() other cases 306a>+≡ (267g) <305d

```
case Amount:
/*
 * When mounting on an already mounted upon directory,
 * one wants subsequent mounts to be attached to the
 * original directory, not the replacement. Don't domount.
 */
break;
```

<cmount() locals 306b>≡ (306c) 317b▷

```
Pgrp *pg;
Mhead *mh;
Mount *m;
int order;
```

`\hlinkcmount` looks up `old`'s `qid` in the mount table hash; if no `\hlinkMhead` exists, it creates one. Then it inserts a new `\hlinkMount` according to the flag: `MREPL` replaces the existing list, `MBEFORE` prepends, `MAFTER` appends. For union mounts on a fresh `\hlinkMhead`, the original channel is also added to the list so the union includes the original directory contents.

<function cmount 306c>≡ (647)

```
int
cmount(Chan *new, Chan *old, int flag, char *spec)
{
    <cmount() locals 306b>

    if((old->qid.type ^ new->qid.type) & QTDIR)
        error(Emount);
    <cmount() print error if old->umh != nil 314e>

    order = flag&MORDERMASK;
    if(!(old->qid.type & QTDIR) && order != MREPL)
        error(Emount);

    <cmount() if new is itself a mount point, error if cant create there 317c>

    pg = up->pgrp;
    wlock(&pg->ns);

    // mh = lookup(old->gid, pg.mnthash)
    l = &MOUNTH(pg, old->qid);
    for(mh = *l; mh; mh = mh->hash){
        if(eqchan(mh->from, old, true))
            break;
        l = &mh->hash;
    }

    if(mh == nil){
        /*
         * nothing mounted here yet. create a mount
         * head and add to the hash table.
         */
        mh = newmhead(old);
        *l = mh;

        /*
         * if this is a union mount, add the old
         * node to the mount chain.
         */
        if(order != MREPL)
```

```

        mh->mount = newmount(old, 0, nil);
    }

    wlock(&mh->lock);
    if(waserror()){
        wunlock(&mh->lock);
        nexterror();
    }
    wunlock(&pg->ns);

    m = newmount(new, flag, spec);

    <cmount() if new is itself a mount point, copy mounts 317d>

    if(mh->mount && order == MREPL){
        mountfree(mh->mount);
        mh->mount = nil;
    }
    if(flag & MCREATE)
        m->mflag |= MCREATE;

    if(mh->mount && order == MAFTER){
        for(f = mh->mount; f->next; f = f->next)
            ;
        f->next = m;
    }else{ // MBEFORE or MREPL
        // when new was itself a mount point, can have union of mount again
        for(f = m; f->next; f = f->next)
            ;
        f->next = mh->mount; // =~ m->next = mh->mount for the most common case
        mh->mount = m;
    }

    wunlock(&mh->lock);
    poperror();
    return m->mountid;
}

```

14.6 /proc/#/ns

A process's namespace can be inspected by reading `/proc/\#/ns`, which produces a sequence of `bind` and `mount` commands that reconstruct the namespace. The `\hlinkmntscan` function walks the mount table hash in `mountid` order (not hash order), so the output is deterministic and reflects the temporal ordering of mounts.

```

<struct Mntwalk 307a>≡ (643)
    struct Mntwalk          /* state for /proc/#/ns */
    {
        bool cddone;
        Mhead*  mh;
        Mount*  cm;
    };

<procread() locals 307b>+≡ (540c) <101e 540b>
    Mntwalk *mw;
    char flag[10];

```

⟨procopen() cases 308a⟩≡ (545b)

```
case Qns:
    if(omode != OREAD)
        error(Eperm);
    c->aux = malloc(sizeof(Mntwalk));
    break;
```

⟨procclose() hooks 308b⟩≡ (551)

```
if(QID(c->qid) == Qns && c->aux != nil)
    free(c->aux);
```

⟨procread() cases 308c⟩+≡ (540c) <101f 429g>

```
case Qns:
    qlock(&p->debug);
    if(waserror()){
        qunlock(&p->debug);
        nexterror();
    }
    if(p->pgrp == nil || p->pid != PID(c->qid))
        error(Eprocdied);

    mw = c->aux;
    if(mw == nil)
        error(Enomem);
    if(mw->cddone){
        qunlock(&p->debug);
        poperror();
        return 0;
    }

    mntscan(mw, p);

    if(mw->mh == nil){
        mw->cddone = true;
        i = snprintf(a, n, "cd %s\n", p->dot->path->s);
        qunlock(&p->debug);
        poperror();
        return i;
    }

    int2flag(mw->cm->mflag, flag);
    if(strcmp(mw->cm->to->path->s, "#M") == 0){
        srv = srvname(mw->cm->to->mchan);
        i = snprintf(a, n, "mount %s %s %s %s\n", flag,
            srv==nil? mw->cm->to->mchan->path->s : srv,
            mw->mh->from->path->s, mw->cm->spec? mw->cm->spec : "");
        free(srv);
    }else
        i = snprintf(a, n, "bind %s %s %s\n", flag,
            mw->cm->to->path->s, mw->mh->from->path->s);
    qunlock(&p->debug);
    poperror();
    return i;
```

⟨function int2flag 308d⟩≡ (643)

```
static void
int2flag(int flag, char *s)
{
    if(flag == 0){
        *s = '\0';
```

```

        return;
    }
    *s++ = '-';
    if(flag & MAFTER)
        *s++ = 'a';
    if(flag & MBEFORE)
        *s++ = 'b';
    if(flag & MCREATE)
        *s++ = 'c';
    if(flag & MCACHE)
        *s++ = 'C';
    *s = '\\0';
}

```

<function mntscan 309>≡ (643)

```

void
mntscan(Mntwalk *mw, Proc *p)
{
    Pgrp *pg;
    Mount *t;
    Mhead *f;
    int i;
    bool nxt;
    ulong last, bestmid;

    pg = p->pgrp;
    rlock(&pg->ns);

    nxt = false;
    bestmid = ~0;

    last = 0;
    if(mw->mh)
        last = mw->cm->mountid;

    for(i = 0; i < MNTHASH; i++) {
        for(f = pg->mnthash[i]; f; f = f->hash) {
            for(t = f->mount; t; t = t->next) {
                if(mw->mh == nil ||
                    (t->mountid > last && t->mountid < bestmid)) {
                    mw->cm = t;
                    mw->mh = f;
                    bestmid = mw->cm->mountid;
                    nxt = true;
                }
            }
        }
    }
    if(nxt == false)
        mw->mh = nil;

    runlock(&pg->ns);
}

```

14.7 findmount(), domount()

`\hlinkfindmount` looks up a channel's `qid` in the `\hlinkPgrp` hash and returns the first mount's "to" channel. `\hlinkdomount` wraps it and also updates the `\hlinkPath` mount point array for ".." handling.

```

⟨namec() locals 310a⟩+≡ (267g) <304a
    Mhead *m;
    Path *path;

⟨namec() case Aopen, Acreate, Aremove, Aaccess, handle mountpoint part1 310b⟩≡ (267g)
    /* save&update the name; domount might change c */
    path = c->path;
    incref(path);

    m = nil;
    if(!sharppath)
        domount(&c, &m, &path);

    /* our own copy to open or remove */
    c = uniquechan(c);

    /* now it's our copy anyway, we can put the name back */
    pathclose(c->path);
    c->path = path;

    /* record whether c is on a mount point */
    c->ismtpt = (m!=nil);

⟨namec() case Aremove, Aaccess, handle mountpoint part2 310c⟩≡ (267g)
    putmhead(m);

⟨function findmount 310d⟩≡ (647)
    /* also used by sysfile.c:/^mountfix */
    bool
    findmount(Chan **cp, Mhead **mp, int type, int dev, Qid qid)
    {
        Pgrp *pg;
        Mhead *m;

        pg = up->pgrp;
        rlock(&pg->ns);

        // *mp = lookup(gqid, pg.mnthash); *cp = mp->mount->to
        for(m = MOUNTH(pg, qid); m; m = m->hash){
            rlock(&m->lock);
            if(m->from == nil){
                print("m %p m->from 0\n", m);
                runlock(&m->lock);
                continue;
            }
            if(eqchantdqid(m->from, type, dev, qid, true)){
                runlock(&pg->ns);
                if(mp != nil){
                    if(*mp != nil)
                        putmhead(*mp);
                    *mp = m;
                    incref(m);
                }
                if(*cp != nil)
                    cclose(*cp);
                *cp = m->mount->to;
                incref(m->mount->to);
                runlock(&m->lock);
                return true;
            }
        }
    }

```

```

        runlock(&m->lock);
    }

    runlock(&pg->ns);
    return false;
}

⟨function domount 311a⟩≡ (647)
/*
 * Calls findmount but also updates path.
 */
static void
domount(Chan **cp, Mhead **mp, Path **path)
{
    Chan **lc;
    Path *p;

    if(!findmount(cp, mp, (*cp)->type, (*cp)->dev, (*cp)->qid))
        return;

    if(path){
        p = *path;
        p = uniquepath(p);
        if(p->mten <= 0)
            print("domount: path %s has mten==%d\n", p->s, p->mten);
        else{
            lc = &p->mtpt[p->mten-1];
            DBG("domount %p %s => add %p (was %p)\n", p, p->s, (*mp)->from, p->mtpt[p->mten-1]);
            if(*lc)
                cclose(*lc);
            *lc = (*mp)->from;
            incref((*mp)->from);
        }
        *path = p;
    }
    return;
}

```

14.8 Path mount points and ..

The trickiest part of namespace handling is “..” (dot-dot). If /usr is a mount point, then “cd /usr/pad; cd ..” should return to /, not to the root of whatever device is mounted on /usr. Each \hlinkPath carries an mtpt array that records, for each path element, the “from” channel of any mount point crossed. When \hlinkwalk processes “..”, \hlinkundomount uses this array to return to the channel that was mounted upon.

```

⟨Path mtpt fields 311b⟩≡ (283b)
// array<option<ref_counted<Chan>>, nil for elements which are not mount point
Chan **mtpt; /* mtpt history */
int mten; /* number of path elements */
int malen; /* allocated length of mtpt */

```

```

⟨constant PATHMSLOP 311c⟩≡ (647)
PATHMSLOP = 20,

```



```

<newpath() mtpt handling 312a>≡ (283c)
/*
 * Cannot use newpath for arbitrary names because the mtpt
 * array will not be populated correctly. The names #/ and / are
 * allowed, but other names with / in them draw warnings.
 */
if(strchr(s, '/') && strcmp(s, "#/") != 0 && strcmp(s, "/") != 0)
    print("newpath: %s from %p\n", s, getcallerpc(&s));

p->mten = 1;
p->malen = PATHMSLOP;
p->mtpt = smalloc(p->malen*sizeof p->mtpt[0]); // smalloc set p->mtpt[0] = nil

```

```

<copypath() mtpt handling 312b>≡ (283e)
pp->mten = p->mten;
pp->malen = p->malen;
pp->mtpt = smalloc(p->malen*sizeof pp->mtpt[0]);
for(i=0; i<pp->mten; i++){
    pp->mtpt[i] = p->mtpt[i];
    if(pp->mtpt[i])
        incref(pp->mtpt[i]);
}

```

```

<pathclose() mtpt handling 312c>≡ (284a)
for(i=0; i<p->mten; i++)
    if(p->mtpt[i])
        cclose(p->mtpt[i]);
free(p->mtpt);

```

```

<addelem() mtpt handling 312d>≡ (284c)
if(isdotdot(s)){
    DBG("addelem %s .. => rm %p\n", p->s, p->mtpt[p->mten-1]);
    if(p->mten>1 && (c = p->mtpt[--p->mten])){
        p->mtpt[p->mten] = nil;
        cclose(c);
    }
}else{
    if(p->mten >= p->malen){
        // realloc
        p->malen = p->mten+1+PATHMSLOP;
        tt = smalloc(p->malen*sizeof tt[0]);
        memmove(tt, p->mtpt, p->mten*sizeof tt[0]);
        free(p->mtpt);
        p->mtpt = tt;
    }
    DBG("addelem %s %s => add %p\n", p->s, s, from);
    p->mtpt[p->mten++] = from;
    if(from)
        incref(from);
}

```

```

<function undomount 312e>≡ (647)
/*
 * If c is the right-hand-side of a mount point, returns the left hand side.
 * Changes name to reflect the fact that we've uncrossed the mountpoint,
 * so name had better be ours to change!
 */
static Chan*
undomount(Chan *c, Path *path)
{

```

```

Chan *nc;

if(path->ref != 1 || path->mten == 0)
    print("undomount: path %s ref %ld mten %d caller %p\n",
        path->s, path->ref, path->mten, getcallerpc(&c));

if(path->mten>0 && (nc=path->mtpt[path->mten-1]) != nil){
    DBG("undomount %p %s => remove %p\n", path, path->s, nc);
    cclose(c);
    path->mtpt[path->mten-1] = nil;
    // c = nc = mh->from
    c = nc;
}
return c;
}

```

14.9 Union mount

Union mounts allow multiple directories to appear merged at a single mount point. When walking a name through a union directory, if the first mount's device walk fails (the name is not found there), `\hlinkwalk` tries the next mount in the list, and so on. Reading a union directory is handled by `\hlinkunionread`, which reads from each mount's channel in sequence, advancing to the next when one is exhausted.

14.9.1 Union walk

```

⟨walk() locals 313a⟩+≡ (287) <??
    Mount *f;

⟨walk() if c is a union mount, find and adjust type and dev 313b⟩≡ (287)
    /* try a union mount, if any */
    if(mh && !sharppath){
        /*
         * mh->mount->to == c, so start at mh->mount->next
         */
        rlock(&mh->lock);
        f = mh->mount;
        for(f = (f? f->next: f); f; f = f->next)
            if((wq = ewalk(f->to, names+nhave, ntry)) != nil)
                break;
        runlock(&mh->lock);
        if(f != nil){
            type = f->to->type;
            dev = f->to->dev;
        }
    }
}

```

14.9.2 ->umh

```

⟨namec() case Aopen, Acreate, handle mountpoint part2 313c⟩≡ (267g)
    /* only save the mount head if it's a multiple element union */
    if(m && m->mount && m->mount->next)
        c->umh = m;
    else
        putmhead(m);

```

```

<Chan other fields 314a>+≡ (51e) <257c 314f>
Mhead* umh; /* mount point that derived Chan; used in unionread */

<chanfree() optional free 314b>≡ (259a) 314i>
if(c->umh != nil){
    putmhead(c->umh);
    c->umh = nil;
}

<namec() print error if c->umh != nil 314c>≡ (267g)
if(c->umh != nil){
    print("uniquechan umh Open\n");
    putmhead(c->umh);
    c->umh = nil;
}

<walk() print error if c->umh != nil 314d>≡ (287)
if(c->umh != nil){ //BUG
    print("walk umh\n");
    putmhead(c->umh);
    c->umh = nil;
}

<cmount() print error if old->umh != nil 314e>≡ (306c)
if(old->umh)
    print("cmount: unexpected umh, caller %#p\n", getcallerpc(&new));

```

14.9.3 syspread() for directories

Reading a directory on a mount point requires extra bookkeeping. The channel has two offsets: `c->offset` (user-visible) and `c->devoffset` (what the device sees), because `\hlinkmountfix` rewrites entries to reflect the current namespace, changing sizes. Overflow entries are saved in `c->dirrock` for the next read.

```

<Chan other fields 314f>+≡ (51e) <314a 314h>
vlong devoffset; /* in underlying device; see read */

<read() if c is a QTDIR 314g>≡ (275b)
if(c->qid.type & QTDIR){
    if(mountrockread(c, p, n, &nn)){
        /* do nothing: mountrockread filled buffer */
    }else if(c->umh){
        nn = unionread(c, p, n);
    }else{
        if(off != c->offset)
            error(Edirseek);
        nn = devtab[c->type]->read(c, p, n, c->devoffset);
    }
    nnn = mountfix(c, p, nn, n);
}

<Chan other fields 314h>+≡ (51e) <314f 320a>
Chan* umc; /* channel in union; held for union read */
QLock umqlock; /* serialize unionreads */
int uri; /* union read index */

<chanfree() optional free 314i>+≡ (259a) <314b 323a>
if(c->umc != nil){
    cclose(c->umc);
    c->umc = nil;
}

```

<function unionread 315a>≡ (654)

```

long
unionread(Chan *c, virt_vp va, long n)
{
    int i;
    long nr;
    Mhead *m;
    Mount *mount;

    qlock(&c->umqlock);
    m = c->umh;
    rlock(&m->lock);
    mount = m->mount;

    /* bring mount in sync with c->uri and c->umc */
    for(i = 0; mount != nil && i < c->uri; i++)
        mount = mount->next;

    nr = 0;
    while(mount != nil){
        /* Error causes component of union to be skipped */
        if(mount->to && !waserror()){
            if(c->umc == nil){
                c->umc = cclone(mount->to);
                c->umc = devtab[c->umc->type]->open(c->umc, OREAD);
            }

            nr = devtab[c->umc->type]->read(c->umc, va, n, c->umc->offset);
            c->umc->offset += nr;
            poperror();
        }
        if(nr > 0)
            break;

        /* Advance to next element */
        c->uri++;
        if(c->umc){
            cclose(c->umc);
            c->umc = nil;
        }
        mount = mount->next;
    }
    runlock(&m->lock);
    qunlock(&c->umqlock);
    return nr;
}

```

<read() rewind when off == 0 315b>≡ (275b)

```

/*
 * The offset is passed through on directories, normally.
 * Sysseek complains, but pread is used by servers like exportfs,
 * that shouldn't need to worry about this issue.
 *
 * Notice that c->devoffset is the offset that c's dev is seeing.
 * The number of bytes read on this fd (c->offset) may be different
 * due to rewritings in rockfix.
 */
mountrewind(c);
unionrewind(c);

```

```

⟨function unionrewind 316a⟩≡ (654)
static void
unionrewind(Chan *c)
{
    qlock(&c->umqlock);
    c->uri = 0;
    if(c->umc){
        cclose(c->umc);
        c->umc = nil;
    }
    qunlock(&c->umqlock);
}

```

14.9.4 MCREATE

In a union directory, only the mount entry with the MCREATE flag can receive new files. `\hlinkcreatedir` walks the union mount list to find it. This is why `bind -c` exists: without the `-c` flag, you cannot create files in the mounted directory.

```

⟨namec() Acreate case, if cnew is a mountpoint 316b⟩≡ (291d)
    if(!sharppath && findmount(&cnew, &m, c->type, c->dev, c->qid))
        cnew = createdir(cnew, m);

```

```

⟨function createdir 316c⟩≡ (647)
/*
 * c is a mounted non-creatable directory. find a creatable one.
 */
Chan*
createdir(Chan *c, Mhead *m)
{
    Chan *nc;
    Mount *f;

    rlock(&m->lock);
    if(waserror()){
        runlock(&m->lock);
        nexterror();
    }
    for(f = m->mount; f; f = f->next){
        if(f->mflag&MCREATE){
            nc = cclone(f->to);
            runlock(&m->lock);
            poperror();
            cclose(c);
            return nc;
        }
    }
    error(Enocreate);
    panic("createdir: should not reach this point");
    return nil; // unreachable
}

```

14.9.5 Union of union

When binding a union directory onto another location, `\hlinkcmount` must copy the entire union mount list from the source, so the new location inherits all the union members. The `umh` field on a channel records the

\hlinkMhead that produced this channel, so \hlinkcmount can walk the source union’s mount chain.

<namec() set c->umh in Abind if mounted point 317a>≡ (305d)

```
if(c->umh != nil)
    putmhead(c->umh);
c->umh = m;
```

<cmount() locals 317b>+≡ (306c) <306b

```
int flg;
Mhead *umh, **l;
Mount *f, *um, **h;
```

<cmount() if new is itself a mount point, error if cant create there 317c>≡ (306c)

```
umh = new->umh;
/*
 * Not allowed to bind when the old directory is itself a union.
 * (Maybe it should be allowed, but I don't see what the semantics
 * would be.)
 *
 * We need to check umh->mount->next to tell unions apart from
 * simple mount points, so that things like
 * mount -c fd /root
 * bind -c /root /
 * work.
 *
 * The check of mount->mflag allows things like
 * mount fd /root
 * bind -c /root /
 *
 * This is far more complicated than it should be, but I don't
 * see an easier way at the moment.
 */
if((flag&MCREATE) && umh && umh->mount
    && (umh->mount->next || !(umh->mount->mflag&MCREATE)))
    error(Emount);
```

<cmount() if new is itself a mount point, copy mounts 317d>≡ (306c)

```
if(umh != nil && umh->mount != nil){
    /*
     * copy a union when binding it onto a directory
     */
    flg = order;
    if(order == MREPL)
        flg = MAFTER;
    h = &m->next;
    um = umh->mount;
    for(um = um->next; um; um = um->next){
        f = newmount(um->to, flg, um->spec);
        *h = f;
        h = &f->next;
    }
}
```

14.10 Sandboxing

The `noattach` flag provides namespace sandboxing: when set via `RFNOMNT`, the process cannot mount new file servers or access most “#” device paths. Only safe devices (pipes, fds, environment, console, proc) are allowed through.

<Pgrp other fields 317e>+≡ (302d) <303d

```
bool noattach;
```

`<bindmount() error if noattach 318a>≡ (319a)`

```
if(up->pgrp->noattach)
    error(Enoattach);
```

`<namec() noattach checking 318b>≡ (304b)`

```
/*
 * noattach is sandboxing.
 *
 * the OK exceptions are:
 * | it only gives access to pipes you create
 * d this process's file descriptors
 * e this process's environment
 * the iffy exceptions are:
 * c time and pid, but also cons and consctl
 * p control of your own processes (and unfortunately
 * any others left unprotected)
 */
/* actually / is caught by parsing earlier */
if(utfrune("M", r))
    error(Enoattach);
if(up->pgrp->noattach && utfrune("|decp", r)==nil)
    error(Enoattach);
```

`<sysrfork() inherit noattach, RFPROC==0 case 318c>≡ (118a)`

```
/* inherit noattach */
up->pgrp->noattach = opg->noattach;
```

`<sysrfork() inherit noattach, RFPROC==1 case 318d>≡ (116b)`

```
/* inherit noattach */
p->pgrp->noattach = up->pgrp->noattach;
```

`<sysrfork() set noattach to true when RFNOMNT, RFPROC==0 case 318e>≡ (118a)`

```
if(flag & RFNOMNT)
    up->pgrp->noattach = true;
```

`<sysrfork() set noattach to true when RFNOMNT, RFPROC==1 case 318f>≡ (116b)`

```
if(flag & RFNOMNT)
    p->pgrp->noattach = true;
```

14.11 sysmount()

While `bind` connects two local paths, `mount` connects a file descriptor (typically a pipe to a file server) to a path, using the mount device (`devmnt`) to speak 9P over the fd.

`<syscall mount 318g>≡ (654)`

```
// int mount(int fd, int afd, char *old, int flag, char *aname);
long
sysmount(ulong* arg)
{
    return bindmount(true, arg[0], arg[1], nil, (char*)arg[2], arg[3], (char*)arg[4]);
}
```

`<bindmount() locals 318h>+≡ (305b) <??`

```
Chan *ac, *bc;
```

```
struct Bogus bogus;
```

```

⟨bindmount() if ismount 319a⟩≡ (305b)
    if(ismount){
        validaddr((ulong)spec, 1, false);
        spec = validnamedup(spec, true);
        if(waserror()){
            free(spec);
            nexterror();
        }
        ⟨bindmount() error if noattach 318a⟩

        ac = nil;
        bc = fdtochan(fd, ORDWR, false, true);
        if(waserror()) {
            if(ac)
                cclose(ac);
            cclose(bc);
            nexterror();
        }

        if(afd >= 0)
            ac = fdtochan(afd, ORDWR, false, true);

        bogus.flags = flag & MCACHE;
        bogus.chan = bc;
        bogus.authchan = ac;
        bogus.spec = spec;
        ret = devno('M', false);
        c0 = devtab[ret]->attach((char*)&bogus);
        poperror(); /* ac bc */
        if(ac)
            cclose(ac);
        cclose(bc);
    }

⟨bindmount() if ismount free 319b⟩≡ (305b)
    if(ismount){
        fdclose(fd, 0);
        poperror();
        free(spec);
    }

⟨MountFlags cases 319c⟩≡ (300b)
    MCACHE = 0x0010, /* cache some data */

```

14.12 sysumount()

`\hlinkcunmount` searches the `\hlinkPgrp` hash and removes either the entire mount point or a specific entry from the union list.

```

⟨syscall unmount 319d⟩≡ (654)
    // int unmount(char *name, char *old);
    long
    sysumount(ulong* arg)
    {
        Chan *cmount, *cmounted;

        cmounted = nil;
    }

```



```

validaddr(arg[1], 1, false);
cmount = namec((char *)arg[1], Amount, 0, 0);
if(waserror()) {
    cclose(cmount);
    if(cmounted)
        cclose(cmounted);
    nexterror();
}

if(arg[0]) {
    /*
     * This has to be namec(..., Aopen, ...) because
     * if arg[0] is something like /srv/cs or /fd/0,
     * opening it is the only way to get at the real
     * Chan underneath.
     */
    validaddr(arg[0], 1, false);
    cmounted = namec((char*)arg[0], Aopen, OREAD, 0);
}
cunmount(cmount, cmounted);
poperror();
cclose(cmount);
if(cmounted)
    cclose(cmounted);
return 0;
}

```

⟨Chan other fields 320a⟩+≡ (51e) <314h 323c>
 Chan* mchan; /* channel to mounted server */

⟨cunmount() print error if mnt-¿umh 320b⟩≡ (320c)
 if(mnt->umh) /* should not happen */
 print("cunmount newp extra umh %p has %p\n", mnt, mnt->umh);

⟨function cunmount 320c⟩≡ (647)
 void
 cunmount(Chan *mnt, Chan *mounted)
 {
 Pgrp *pg;
 Mhead *m, **l;
 Mount *f, **p;

 ⟨cunmount() print error if mnt-¿umh 320b⟩

 /*
 * It _can_ happen that mounted->umh is non-nil,
 * because mounted is the result of namec(Aopen)
 * (see sysfile.c:~sysunmount).
 * If we open a union directory, it will have a umh.
 * Although surprising, this is okay, since the
 * cclose will take care of freeing the umh.
 */

 pg = up->pgrp;
 wlock(&pg->ns);

 l = &MOUNTH(pg, mnt->qid);
 for(m = *l; m; m = m->hash){
 if(eqchan(m->from, mnt, true))
 break;

```

    l = &m->hash;
}

if(m == nil){
    wunlock(&pg->ns);
    error(Eunmount);
}

wlock(&m->lock);
if(mounted == nil){
    *l = m->hash;
    wunlock(&pg->ns);
    mountfree(m->mount);
    m->mount = nil;
    cclose(m->from);
    wunlock(&m->lock);
    putmhead(m);
    return;
}

p = &m->mount;
for(f = *p; f; f = f->next){
    /* BUG: Needs to be 2 pass */
    if(eqchan(f->to, mounted, true) ||
        (f->to->mchan && eqchan(f->to->mchan, mounted, true))){
        *p = f->next;
        f->next = nil;
        mountfree(f);
        if(m->mount == nil){
            *l = m->hash;
            cclose(m->from);
            wunlock(&m->lock);
            wunlock(&pg->ns);
            putmhead(m);
            return;
        }
        wunlock(&m->lock);
        wunlock(&pg->ns);
        return;
    }
    p = &f->next;
}
wunlock(&m->lock);
wunlock(&pg->ns);
error(Eunion);
}

```

14.13 XTODD

⟨Mount other fields 321a⟩≡ (301c) 321c▷
 ulong mountid;

⟨global mountid 321b⟩≡ (673b)
 static Counter mountid;

⟨Mount other fields 321c⟩+≡ (301c) ◁321a
 // list⟨ref⟨Mount⟩⟩ sorted list by mount->mountid
 Mount* order;

⟨Mount *extra fields* 322a⟩+≡ (301c) ◁302b

```
Mount* copy;
```

⟨function pgrpinsert 322b⟩≡ (673b)

```
void
pgrpinsert(Mount **order, Mount *m)
{
    Mount *f;

    m->order = nil;
    if(*order == nil) {
        *order = m;
        return;
    }
    for(f = *order; f; f = f->order) {
        if(m->mountid < f->mountid) {
            m->order = f;
            *order = m;
            return;
        }
        order = &f->order;
    }
    *order = m;
}
```

\hlinkpgrpcpy duplicates a namespace for RFNAMEG. It walks the entire mount table hash, cloning every \hlinkMhead and \hlinkMount. The tricky part is preserving the mountid order: mount ids are allocated globally and must increase monotonically, so the copy collects all old mounts in sorted order via \hlinkpgrpinsert and then assigns new ids in the same sequence.

⟨function pgrpcpy 322c⟩≡ (673b)

```
/*
 * pgrpcpy MUST preserve the mountid allocation order of the parent group
 */
void
pgrpcpy(Pgrp *to, Pgrp *from)
{
    int i;
    Mount *n, *m, **link, *order;
    Mhead *f, **tom, **l, *mh;

    wlock(&from->ns);
    order = nil;
    tom = to->mnthash;
    for(i = 0; i < MNTHASH; i++) {
        l = tom++;
        for(f = from->mnthash[i]; f; f = f->hash) {
            rlock(&f->lock);
            mh = newmhead(f->from);
            *l = mh;
            l = &mh->hash;
            link = &mh->mount;
            for(m = f->mount; m; m = m->next) {
                n = newmount(m->to, m->mflag, m->spec);

                m->copy = n;
                pgrpinsert(&order, m);

                *link = n;
                link = &n->next;
            }
        }
    }
}
```

```

        }
        runlock(&f->lock);
    }
}
/*
 * Allocate mount ids in the same sequence as the parent group
 */
lock(&mountid);
for(m = order; m; m = m->order)
    m->copy->mountid = mountid.ref++;
unlock(&mountid);
wunlock(&from->ns);
}

```

14.14 XTODO

⟨chanfree() optional free 323a⟩+≡ (259a) ◁314i 323b▷

```

if(c->mux != nil){
    muxclose(c->mux);
    c->mux = nil;
}
if(c->mchan != nil){
    cclose(c->mchan);
    c->mchan = nil;
}

```

⟨chanfree() optional free 323b⟩+≡ (259a) ◁323a

```

if(c->dirrock != nil){
    free(c->dirrock);
    c->dirrock = nil;
    c->nrock = 0;
    c->mrock = 0;
}

```

⟨Chan other fields 323c⟩+≡ (51e) ◁320a 379b▷

```

byte* dirrock; /* directory entry rock for translations */
int nrock;
int mrock;
QLock rockqlock;

```

⟨function mountrock 323d⟩≡ (654)

```

/*
 * Mountfix might have caused the fixed results of the directory read
 * to overflow the buffer. Catch the overflow in c->dirrock.
 */
static void
mountrock(Chan *c, byte *p, byte **pe)
{
    byte *e, *r;
    int len, n;

    e = *pe;

    /* find last directory entry */
    for(;;){
        len = BIT16SZ+GBIT16(p);
        if(p+len >= e)
            break;
    }
}

```

```

        p += len;
    }

    /* save it away */
    qlock(&c->rockqlock);
    if(c->nrock+len > c->mrock){
        n = ROUND(c->nrock+len, 1024);
        r = smalloc(n);
        memmove(r, c->dirrock, c->nrock);
        free(c->dirrock);
        c->dirrock = r;
        c->mrock = n;
    }
    memmove(c->dirrock+c->nrock, p, len);
    c->nrock += len;
    qunlock(&c->rockqlock);

    /* drop it */
    *pe = p;
}

```

⟨function mountrockread 324⟩≡ (654)

```

/*
 * Satisfy a directory read with the results saved in c->dirrock.
 */
static bool
mountrockread(Chan *c, byte *op, long n, long *nn)
{
    long dirlen;
    byte *rp, *erp, *ep, *p;

    /* common case */
    if(c->nrock == 0)
        return false;

    /* copy out what we can */
    qlock(&c->rockqlock);
    rp = c->dirrock;
    erp = rp+c->nrock;
    p = op;
    ep = p+n;
    while(rp+BIT16SZ <= erp){
        dirlen = BIT16SZ+GBIT16(rp);
        if(p+dirlen > ep)
            break;
        memmove(p, rp, dirlen);
        p += dirlen;
        rp += dirlen;
    }

    if(p == op){
        qunlock(&c->rockqlock);
        return false;
    }

    /* shift the rest */
    if(rp != erp)
        memmove(c->dirrock, rp, erp-rp);
    c->nrock = erp - rp;
}

```

```

    *nn = p - op;
    qunlock(&c->rockqlock);
    return true;
}

```

```

⟨function mountrewind 325a⟩≡ (654)
    static void
    mountrewind(Chan *c)
    {
        c->nrock = 0;
    }

```

```

⟨function mountfix 325b⟩≡ (654)
/*
 * Rewrite the results of a directory read to reflect current
 * name space bindings and mounts. Specifically, replace
 * directory entries for bind and mount points with the results
 * of statting what is mounted there. Except leave the old names.
 */
static long
mountfix(Chan *c, byte *op, long n, long maxn)
{
    char *name;
    int nbuf, nname;
    Chan *nc;
    Mhead *mh;
    Mount *m;
    byte *p;
    int dirlen, rest;
    long l;
    byte *buf, *e;
    DirEntry d;

    p = op;
    buf = nil;
    nbuf = 0;
    for(e=&p[n]; p+BIT16SZ<e; p+=dirlen){
        dirlen = dirfixed(p, e, &d);
        if(dirlen < 0)
            break;
        nc = nil;
        mh = nil;
        if(findmount(&nc, &mh, d.type, d.dev, d.qid)){
            /*
             * If it's a union directory and the original is
             * in the union, don't rewrite anything.
             */
            for(m=mh->mount; m; m=m->next)
                if(eqchantdqid(m->to, d.type, d.dev, d.qid, true))
                    goto Norewrite;

            name = offsetof_name_dirent(p, &nname);
            /*
             * Do the stat but fix the name. If it fails, leave old entry.
             * BUG: If it fails because there isn't room for the entry,
             * what can we do? Nothing, really. Might as well skip it.
             */
            if(buf == nil){
                buf = smalloc(4096);
                nbuf = 4096;
            }

```

```

    }
    if(waserror())
        goto Norewrite;
    l = devtab[nc->type]->stat(nc, buf, nbuf);
    l = direntry_setname(name, nname, buf, l, nbuf);
    if(l == BIT16SZ)
        error("direntry_setname");
    poperror();

    /*
     * Shift data in buffer to accomodate new entry,
     * possibly overflowing into rock.
     */
    rest = e - (p+dirlen);
    if(l > dirlen){
        while(p+l+rest > op+maxn){
            mountrock(c, p, &e);
            if(e == p){
                dirlen = 0;
                goto Norewrite;
            }
            rest = e - (p+dirlen);
        }
    }
    if(l != dirlen){
        memmove(p+l, p+dirlen, rest);
        dirlen = l;
        e = p+dirlen+rest;
    }

    /*
     * Rewrite directory entry.
     */
    memmove(p, buf, l);

    Norewrite:
    cclose(nc);
    putmhead(mh);
}
}
if(buf)
    free(buf);

if(p != e)
    error("oops in rockfix");

return e-op;
}

```

```

⟨function dirfixed 326⟩≡ (654)
static int
dirfixed(byte *p, byte *e, DirEntry *d)
{
    int len;

    len = GBIT16(p)+BIT16SZ;
    if(p + len > e)
        return -1;

    p += BIT16SZ;    /* ignore size */

```

```

d->type = devno(GBIT16(p), true);
p += BIT16SZ;
d->dev = GBIT32(p);
p += BIT32SZ;
d->qid.type = GBIT8(p);
p += BIT8SZ;
d->qid.vers = GBIT32(p);
p += BIT32SZ;
d->qid.path = GBIT64(p);
p += BIT64SZ;
d->mode = GBIT32(p);
p += BIT32SZ;
d->atime = GBIT32(p);
p += BIT32SZ;
d->mtime = GBIT32(p);
p += BIT32SZ;
d->length = GBIT64(p);

return len;
}

```

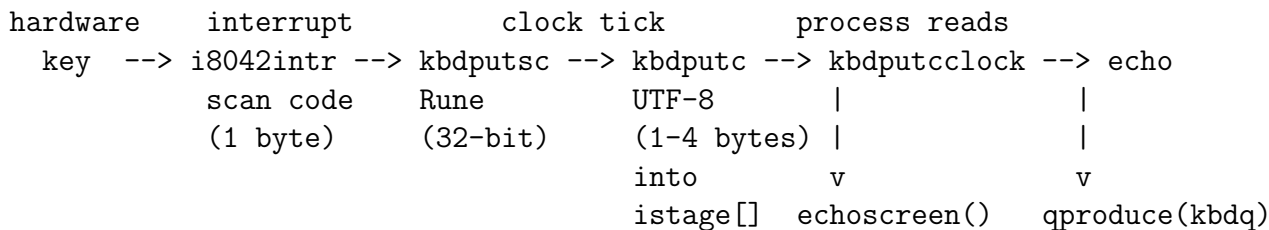

Chapter 15

Devices

15.1 The keyboard

15.1.1 Overview

Keyboard input passes through a multi-stage pipeline before reaching a user process. When you press a key, the hardware generates a scan code (a raw byte identifying the physical key). The keyboard interrupt handler calls `[kbdputc]` which translates that scan code into a Rune (a Unicode code point) by looking it up in one of several tables depending on the modifier state (Shift, Ctrl, Alt, Altgr). The Rune is then passed to `[kbdputc]`, which converts it to UTF-8 bytes and stores them in a circular buffer (`kbd.istage`). A periodic clock callback (`[kbdputclock]`) drains that buffer, calling `[echo]` which both echoes characters to the screen and enqueues them into `kbdq` via `[qproduce]`. The pipeline looks like this:



The reason for the intermediate `istage` circular buffer and the clock-driven drain is to amortize the cost of queue operations: calling `[qproduce]` on every single keystroke interrupt would be wasteful, so instead characters accumulate at interrupt time and get flushed in batch every 22 ms by the clock.

15.1.2 Scan code

The scan code tables map raw hardware bytes to Unicode code points (Runes). Special keys like Shift, Ctrl, and arrow keys cannot be represented as normal characters, so they are assigned values in Unicode's Private Use Area (starting at `0xF000`). This is a clever trick: modifier keys and function keys live in the same Rune space as regular characters, so the entire translation can be done with a single array lookup rather than complex conditional logic. The `Nscan` constant is 128 because the 8th bit of a scan code encodes key-up vs. key-down, leaving 7 bits for the key identity.

```
<enum specialkey 328>≡ (612)
enum {
    KF=          0xF000,    /* function key (begin Unicode private space) */
    Spec=        0xF800,    /* Unicode private space */

    PF=          Spec|0x20, /* num pad function key */
    View=        Spec|0x00, /* view (shift window up) */
```

```

Shift=      Spec|0x60,
Break=      Spec|0x61,
Ctrl=       Spec|0x62,
Alt=        Spec|0x63,
Caps=       Spec|0x64,
Num=        Spec|0x65,
Middle=     Spec|0x66,
Altgr=      Spec|0x67,

Kmouse=     Spec|0x100,

No=         0x00,      /* peter */

/* KF|1, KF|2, ..., KF|0xC is F1, F2, ..., F12 */

Home=       KF|13,
Up=         KF|14,
Pgup=       KF|15,
Print=      KF|16,
Left=       KF|17,
Right=      KF|18,
End=        KF|24,
Down=       View,
Pgdown=     KF|19,
Ins=        KF|20,
Del=        0x7F,
Scroll=     KF|21,

Nscan= 128,
};

```

<global kbtabs 329a>≡ (612)

```

Rune kbtabs[Nscan] =
{
[0x00] No, 0x1b, '1', '2', '3', '4', '5', '6',
[0x08] '7', '8', '9', '0', '-', '=', '\b', '\t',
[0x10] 'q', 'w', 'e', 'r', 't', 'y', 'u', 'i',
[0x18] 'o', 'p', '[', ']', '\n', Ctrl, 'a', 's',
[0x20] 'd', 'f', 'g', 'h', 'j', 'k', 'l', ';',
[0x28] '\'', ',', Shift, '\\', 'z', 'x', 'c', 'v',
[0x30] 'b', 'n', 'm', ',', '.', '/', Shift, '*',
[0x38] Alt, ' ', Ctrl, KF|1, KF|2, KF|3, KF|4, KF|5,
[0x40] KF|6, KF|7, KF|8, KF|9, KF|10, Num, Scroll, '7',
[0x48] '8', '9', '-', '4', '5', '6', '+', '1',
[0x50] '2', '3', '0', '.', No, No, No, KF|11,
[0x58] KF|12, No, No, No, No, No, No, No,
[0x60] No, No, No, No, No, No, No, No,
[0x68] No, No, No, No, No, No, No, No,
[0x70] No, No, No, No, No, No, No, No,
[0x78] No, No, No, No, No, No, No, No,
};

```

<global kbtabsshift 329b>≡ (612)

```

Rune kbtabsshift[Nscan] =
{
[0x00] No, 0x1b, '!', '@', '#', '$', '%', '^',
[0x08] '&', '*', '(', ')', '_', '+', '\b', '\t',
[0x10] 'Q', 'W', 'E', 'R', 'T', 'Y', 'U', 'I',
[0x18] 'O', 'P', '{', '}', '\n', Ctrl, 'A', 'S',

```

```

[0x20] 'D', 'F', 'G', 'H', 'J', 'K', 'L', ':',
[0x28] '"', '~', Shift, '|', 'Z', 'X', 'C', 'V',
[0x30] 'B', 'N', 'M', '<', '>', '?', Shift, '*',
[0x38] Alt, ' ', Ctrl, KF|1, KF|2, KF|3, KF|4, KF|5,
[0x40] KF|6, KF|7, KF|8, KF|9, KF|10, Num, Scroll, '7',
[0x48] '8', '9', '-', '4', '5', '6', '+', '1',
[0x50] '2', '3', '0', '.', No, No, No, KF|11,
[0x58] KF|12, No, No, No, No, No, No, No,
[0x60] No, No, No, No, No, No, No, No,
[0x68] No, No, No, No, No, No, No, No,
[0x70] No, No, No, No, No, No, No, No,
[0x78] No, No, No, No, No, No, No, No,
};

```

```

⟨global kbtctrl decl 330a⟩≡ (612)
extern Rune kbtctrl[];

```

```

⟨global kbtaltgr 330b⟩≡ (612)
Rune kbtaltgr[Nscan] =
{
[0x00] No, No, No, No, No, No, No, No,
[0x08] No, No, No, No, No, No, No, No,
[0x10] No, No, No, No, No, No, No, No,
[0x18] No, No, No, No, '\n', Ctrl, No, No,
[0x20] No, No, No, No, No, No, No, No,
[0x28] No, No, Shift, No, No, No, No, No,
[0x30] No, No, No, No, No, '/', No, Print,
[0x38] Altgr, No, No, No, No, No, No, No,
[0x40] No, No, No, No, No, No, Break, Home,
[0x48] Up, Pgup, No, Left, No, Right, No, End,
[0x50] Down, Pgdown, Ins, Del, No, No, No, No,
[0x58] No, No, No, No, No, No, No, No,
[0x60] No, No, No, No, No, No, No, No,
[0x68] No, No, No, No, No, No, No, No,
[0x70] No, No, No, No, No, No, No, No,
[0x78] No, No, No, No, No, No, No, No,
};

```

```

⟨struct Kbscan 330c⟩≡ (611d)
struct Kbscan {
    bool ctl;
    bool shift;
    bool caps;
    bool alt;
    bool altgr;
    bool num;
    ⟨Kbscan other fields 333c⟩
};

```

The `Kbscan` structure tracks the current modifier key state for the scan code state machine. When a Shift key-down event arrives, `kbscan->shift` is set to true; when the key-up event arrives, it is cleared. The next regular key press then consults these flags to pick the right translation table (`kbtctrl`, `kbtctrlshift`, `kbtctrl`, or `kbtaltgr`). The `kbscans` array has two entries: one for the built-in keyboard (`KbInt`) and one for an external USB keyboard (`KbExt`), so each can track its modifier state independently.

```

⟨enum kbscan 330d⟩≡ (611d)
/* kbscans indices */
enum KbscanKind {
    KbInt= 0,
    KbExt,

```

```

    KbNscans,
};

```

```

⟨global kbscans 331a⟩≡ (612)
// hash<enum<KbscanKind>, Kbscan>
Kbscan kbscans[KbNscans]; /* kernel and external scan code state */

```

[kbdputc] is the heart of the scan code state machine. It takes a raw byte from the hardware interrupt and converts it to a Rune. The high bit of the byte indicates key-up (release) vs. key-down (press). For key-up events, only modifier keys matter – releasing a regular key has no effect, since we already processed it on key-down. For key-down events of modifier keys (Ctrl, Shift, Alt, Caps), the function updates the **Kbscan** state without producing any character. Only non-modifier key-down events result in a call to [kbdputc] to deposit the character.

```

⟨function kbdputc 331b⟩≡ (612)
/*
 * Scan code processing
 */
void
kbdputc(byte k, int external)
{
    bool keyup;
    Kbscan *kbscan;
    // Rune is (uint) but actually 'c' can get the result of functions
    // like latin1() which can return negative numbers.
    long c = k;

    if(external)
        kbscan = &kbscans[KbExt];
    else
        kbscan = &kbscans[KbInt];

    ⟨kbdputc() debugging 573c⟩

    ⟨kbdputc() esc key handling part1 and possible return 333d⟩

    keyup = c & 0x80; // key released (1 bit)
    c &= 0x7f; // 128

    ⟨kbdputc() ensures c is in boundary ??⟩

    ⟨kbdputc() esc key handling part2 and possible return 334a⟩
    else if(kbscan->shift)
        c = kbtabs[shift][c];
    else if(kbscan->altgr)
        c = kbtabs[altgr][c];
    else if(kbscan->ctl)
        c = kbtabs[ctl][c];
    else
        c = kbtabs[c];

    if(kbscan->caps && c<='z' && c>='a')
        c += 'A' - 'a';

    /*
     * keyup only important for shifts
     */
    if(keyup){
        switch(c){

```

```

case Ctrl:
    kbscan->ctl = false;
    break;
case Alt:
    kbscan->alt = false;
    break;
case Altgr:
    kbscan->altgr = false;
    break;
case Shift:
    kbscan->shift = false;
    <kbdputc() reset mouseshift 348b>
    <kbdputc() debugging up shift 573d>
    break;
    <kbdputc() mouse keyup cases 348d>
}
return;
}

/*
 * normal character
 */
if(!(c & (Spec|KF))){
    <kbdputc() reboot if ctl-alt-del 561d>
    <kbdputc() if collecting 603c>
    else {
        kbdputc(c); //!! adding the character in kbd staging area
    }
}else{
    switch(c){
    case Ctrl:
        kbscan->ctl = true;
        break;
    case Alt:
        kbscan->alt = true;
        <kbdputc() start collecting 603d>
        break;
    case Altgr:
        kbscan->altgr = true;
        break;
    case Shift:
        kbscan->shift = true;
        <kbdputc() set mouseshift 348c>
        <kbdputc() debugging down shift 573e>
        break;
    case Caps:
        kbscan->caps ^= true;
        break;
    case Num:
        kbscan->num ^= true;
        if(!external)
            arch_setleds(kbscan);
        break;
        <kbdputc() mouse keydown cases 348e>
        <kbdputc() special keyboard debug keys cases 573b>
        // e.g. Left, Right or other special key
    default:
        kbdputc(c);
        break;
    }
}

```

```

    }
}

```

```

⟨function arch_setleds(arm) 333a⟩≡ (698a)
// empty callback for portkbd.c
void arch_setleds(Kbscan *kbscan) { USED(kbscan); }

```

15.1.3 Esc

Despite the name, this subsection has nothing to do with the Esc key (which is just scan code 0x01). The “esc” here refers to escape sequences in the PS/2 scan code protocol: certain keys (arrow keys, Home, End, etc.) generate multi-byte sequences prefixed by 0xe0 (2-byte) or 0xe1 (3-byte). The state machine in [kbdputc] handles this by setting `esc1` or `esc2` flags when it sees the prefix byte and returning immediately, then on the next interrupt it uses `kbtabc1` instead of `kbtabc` for the lookup. This is why arrow keys like Left and Right appear in `kbtabc1` rather than in the main `kbtabc`.

```

⟨global kbtabc1 333b⟩≡ (612)
Rune kbtabc1[Nscan] =
{
[0x00] No, No, No, No, No, No, No, No,
[0x08] No, No, No, No, No, No, No, No,
[0x10] No, No, No, No, No, No, No, No,
[0x18] No, No, No, No, '\n', Ctrl, No, No,
[0x20] No, No, No, No, No, No, No, No,
[0x28] No, No, Shift, No, No, No, No, No,
[0x30] No, No, No, No, No, '/', No, Print,
[0x38] Altgr, No, No, No, No, No, No, No,
[0x40] No, No, No, No, No, No, Break, Home,
[0x48] Up, Pgup, No, Left, No, Right, No, End,
[0x50] Down, Pgdown, Ins, Del, No, No, No, No,
[0x58] No, No, No, No, No, No, No, No,
[0x60] No, No, No, No, No, No, No, No,
[0x68] No, No, No, No, No, No, No, No,
[0x70] No, No, No, No, No, No, No, No,
[0x78] No, No, No, No, No, No, No, No,
};

```

```

⟨Kbscan other fields 333c⟩≡ (330c) 348a▷
bool esc1;
int esc2;

```

```

⟨kbdputc() esc key handling part1 and possible return 333d⟩≡ (331b)
/*
 * e0's is the first of a 2 character sequence, e1 the first
 * of a 3 character sequence (on the safari)
 */
if(c == 0xe0){
    kbscan->esc1 = true;
    return;
} else if(c == 0xe1){
    kbscan->esc2 = 2;
    return;
}

```

```

⟨kbdputc() esc key handling part2 and possible return 334a⟩≡ (331b)
    if(kbscan->esc1){
        c = kbtabsesc1[c];
        kbscan->esc1 = false;
    } else if(kbscan->esc2){
        kbscan->esc2--;
        return;
    }

```

15.1.4 kbd

The ConsKbd structure is the staging area between the keyboard interrupt and the queue system. Its `istage` circular buffer accumulates UTF-8 bytes deposited by `[kbdputc]` at interrupt time. The circular buffer uses three pointers: `iw` (write position), `ir` (read position), and `ie` (end sentinel). This is a classic lock-free single-producer single-consumer ring buffer – the interrupt handler writes at `iw` and the clock callback reads from `ir`.

```

⟨struct ConsKbd 334b⟩≡ (607)
    struct ConsKbd
    {
        /* a place to save up characters at interrupt time before dumping them in the queue */
        char    istage[1024];
        // pointers into istage to implement a circular buffer
        char    *iw; // write
        char    *ir; // read
        char    *ie; // end
        ILock    lockputc;

        ⟨ConsKbd other fields 340f⟩
        // extra
        QLock;
    };

⟨global kbd 334c⟩≡ (607)
    static struct ConsKbd kbd =
    {
        .iw = kbd.istage,
        .ir = kbd.istage,
        .ie = kbd.istage + sizeof(kbd.istage),
    };

```

`[kbdputc]` bridges the Rune world and the byte world. It takes the Rune produced by `[kbdputc]` and converts it to UTF-8 via `runetochar`, then writes those bytes into the `istage` circular buffer. This is where the `/dev/cons` interface becomes UTF-8 encoded – even though the keyboard hardware deals in scan codes and the internal processing uses 32-bit Runes, what reaches user processes through the queue is a stream of UTF-8 bytes. Note the use of `ilock` rather than `qlock`: this function runs at interrupt time, so it must use a spin lock rather than a sleeping mutex.

```

⟨function kbdputc 334d⟩≡ (607)
    /*
     * Put character, possibly a rune, into kbd at interrupt time.
     * Called at interrupt time to process a character.
     */
    void
    kbdputc(Rune ch)
    {
        int i, n;
        char buf[UTFmax]; // pad's fourth bugfix :)
        Rune r;

```

```

char *next;

⟨kbdputc() debugging 573f⟩

if(kbd.ir == nil)
    return;          /* in case we're not init'd yet */

ilock(&kbd.lockputc);    /* just a mutex */
r = ch;
n = runetochar(buf, &r);
for(i = 0; i < n; i++){
    next = kbd.iw+1;
    // circular buffer
    if(next >= kbd.ie)
        next = kbd.istage;
    if(next == kbd.ir) // full
        break;
    *kbd.iw = buf[i];
    kbd.iw = next;
}
iunlock(&kbd.lockputc);
return;
}

```

[kbdputclock] is called every 22 ms from the clock interrupt. It drains the `istage` circular buffer by calling [echo] on whatever bytes have accumulated. The wraparound case (when `iw < ir`) requires two calls to [echo]: one for the tail of the buffer (from `ir` to `ie`) and one for the head (from `istage` to `iw`). There is no explicit lock here because the clock callback and the keyboard interrupt are carefully ordered: the clock reads `iw` once into a local variable, creating a consistent snapshot.

```

⟨clock callback kbdputclock 335a⟩≡ (607)
/*
 * we save up input characters till clock time to reduce
 * per character interrupt overhead.
 */
static void
kbdputclock(void)
{
    char *iw;

    /* this amortizes cost of qproduce */
    if(kbd.iw != kbd.ir){
        iw = kbd.iw;
        if(iw < kbd.ir){
            echo(kbd.ir, kbd.ie-kbd.ir);
            kbd.ir = kbd.istage;
        }
        if(kbd.ir != iw){
            echo(kbd.ir, iw-kbd.ir);
            kbd.ir = iw;
        }
    }
}

```

```

⟨consinit() initializing things 335b⟩≡ (344c) 567c▷
/*
 * at 115200 baud, the 1024 char buffer takes 56 ms to process,
 * processing it every 22 ms should be fine
 */
addclock0link(kbdputclock, 22);

```


15.1.5 echo()

[echo] serves a dual purpose: it enqueues the keyboard bytes into `kbdq` via [qproduce] so that processes reading `/dev/cons` can consume them, and it echoes them back to the screen via the `screenputs` hook so the user can see what they typed. The `screenputs` function pointer is initially `nil` and gets set later when the graphics subsystem initializes (e.g., to `swconsole_screenputs` for framebuffer output or `cgascreenputs` on x86). In raw mode, echoing is suppressed – the characters still go into `kbdq`, but nothing is displayed, which is exactly what programs like `rio` need when they handle their own display.

```
<function echo 336a>≡ (607)
static void
echo(char *buf, int n)
{
    <echo() locals 574a>

    if(n == 0)
        return;

    <echo() special keys handler 574b>

    qproduce(kbdq, buf, n); //!! add in kbd queue to be read from /dev/cons

    <echo() return before any echoscreen if raw mode 347c>

    <echo() hooks 336b>
}

<echo() hooks 336b>≡ (336a) 581d▷
    if(screenputs != nil)
        echoscreen(buf, n);

<hook screenputs 336c>≡ (607)
void    (*screenputs)(char*, int) = nil;

<function echoscreen 336d>≡ (607)
static void
echoscreen(char *buf, int n)
{
    char *e, *p;
    char ebuf[128];
    int x;

    p = ebuf;
    e = ebuf + sizeof(ebuf) - 4;
    while(n-- > 0){
        if(p >= e){
            screenputs(ebuf, p - ebuf);
            p = ebuf;
        }
        x = *buf++;
        if(x == 0x15){ // ??
            *p++ = '^';
            *p++ = 'U';
            *p++ = '\n';
        } else
            *p++ = x;
    }
    if(p != ebuf)
        screenputs(ebuf, p - ebuf);
}
```

15.1.6 PC Keyboard (x86)

15.1.7 Raspberry Pi USB Keyboard (ARM)

15.1.8 Changing keyboard: /dev/kbmap

<function kbdgetmap 337a>≡ (612)

```
int
kbdgetmap(uint offset, int *t, int *sc, Rune *r)
{
    if ((int)offset < 0)
        error(Ebadarg);
    *t = offset/Nscan;
    *sc = offset%Nscan;
    switch(*t) {
    default:
        return 0;
    case 0:
        *r = kbtabs[*sc];
        return 1;
    case 1:
        *r = kbtabsshift[*sc];
        return 1;
    case 2:
        *r = kbtabsesc1[*sc];
        return 1;
    case 3:
        *r = kbtabsaltgr[*sc];
        return 1;
    case 4:
        *r = kbtabsctrl[*sc];
        return 1;
    }
}
```

<function kbdputmap 337b>≡ (612)

```
void
kbdputmap(ushort m, ushort scanc, Rune r)
{
    if(scanc >= Nscan)
        error(Ebadarg);
    switch(m) {
    default:
        error(Ebadarg);
    case 0:
        kbtabs[scanc] = r;
        break;
    case 1:
        kbtabsshift[scanc] = r;
        break;
    case 2:
        kbtabsesc1[scanc] = r;
        break;
    case 3:
        kbtabsaltgr[scanc] = r;
        break;
    case 4:
        kbtabsctrl[scanc] = r;
        break;
    }
```

```
}
```

15.1.9 External keyboard input: /dev/kbin

15.2 The screen

15.2.1 Overview

15.2.2 Text output on a PC (x86)

15.2.3 Text output on a Raspberry Pi (ARM)

```
<constant VCBUFFER(arm) 338a>≡ (709b)
#define VCBUFFER      (KZERO+0x3400)      /* videcore mailbox buffer */

<function fbinit(arm) 338b>≡ (691d)
void*
fbinit(bool set, int *width, int *height, int *depth)
{
    Fbinfo *fi = (Fbinfo*)(VCBUFFER);
    uintptr va;

    if(!set)
        fbdefault(width, height, depth);
    /* Screen width must be a multiple of 16 */
    *width &= ~0xF;

    memset(fi, 0, sizeof(Fbinfo));
    fi->xres = fi->xresvirtual = *width;
    fi->yres = fi->yresvirtual = *height;
    fi->bpp = *depth;
    <fbinit() write back cache after modified fb info(arm) 470k>
    vcwrite(ChanFb, dmaaddr(fi));

    if(vcread(ChanFb) != 0)
        return nil;
    //TODO: fi->base is in virtual space?? how vcore knows about that? PADDR?
    va = mmukmap(FRAMEBUFFER, PADDR(fi->base), fi->screensize);
    // make it a blue screen
    if(va)
        memset((char*)va, 0x7F, fi->screensize);
    return (void*)va;
}

<function mmukmap(arm) 338c>≡ (711a)
uintptr
mmukmap(uintptr va, uintptr pa, usize size)
{
    int o;
    usize n;
    PTE *pte, *pte0;

    assert((va & (MiB-1)) == 0);
    o = pa & (MiB-1);
    pa -= o;
    size += o;
    pte = pte0 = &cpu->mmul1[L1X(va)];
    for(n = 0; n < size; n += MiB)
```

```

        if(*pte++ != Fault)
            return 0;
    pte = pte0;
    for(n = 0; n < size; n += MiB){
        *pte++ = (pa+n)|Dom0|L1AP(Krw)|Section;
        mmuinvalidateaddr(va+n);
    }
    <mmukmap() write back cache after changing page tables(arm) 470d>
    return va + o;
}

<function fbdefault(arm) 339a>≡ (691d)
/*
 * Framebuffer
 */
static errorneg1
fbdefault(int *width, int *height, int *depth)
{
    u32int buf[3];

    if(vcreq(TagGetres, &buf[0], 0, 2*4) != 2*4 ||
        vcreq(TagGetdepth, &buf[2], 0, 4) != 4)
        return ERROR_NEG1;
    *width = buf[0];
    *height = buf[1];
    *depth = buf[2];
    return OK_0;
}

<function fbblank(arm) 339b>≡ (691d)
int
fbblank(bool blank)
{
    u32int buf[1];

    buf[0] = blank;
    if(vcreq(TagFbblank, buf, sizeof buf, sizeof buf) != sizeof buf)
        return ERROR_NEG1;
    return buf[0] & 1;
}

```

15.3 The console device /dev/cons

The console device exposes keyboard input and screen output through the file system as `/dev/cons` and `/dev/consctl`. Reading `/dev/cons` in cooked mode gives line-edited input: the kernel buffers characters until a newline or Control-D, handling backspace and Control-U (kill line) internally. This is much cleaner than Unix's layered `tty/pty/termios` system – in Plan 9, all line discipline lives in this single device driver.

15.3.1 Overview

```

<devcons.c enum Qxxx 339c>≡ (607)
enum{
    Qdir,

    Qcons,
    Qconsctl,

```

```

    <devcons.c enum Qxxx cases 567a>
};

<global consdir 340a>≡ (607)
static Dirtab consdir[]={
    ".",    {Qdir, 0, QTDIR},    0,    DMDIR|0555,

    "cons",    {Qcons},    0,    0660,
    "consctl", {Qconsctl}, 0,    0220,

    <consdir fields 567b>
};

```

15.3.2 Buffered input

The console uses a two-queue design. Raw keystrokes arrive in `kbdq` (the “unprocessed” queue, fed by `[echo]` via `[qproduce]`). When a process reads `/dev/cons` in cooked mode, the `[consread]` code pulls bytes one at a time from `kbdq`, applies line editing (backspace, Control-U, newline detection), and when a complete line is ready, writes it into `lineq` (the “processed” queue). The process then reads from `lineq`. In raw mode, this two-stage pipeline is short-circuited: bytes go from `kbdq` directly to the reader without line editing.

```

<global kbdq 340b>≡ (607)
Queue* kbdq;          /* unprocessed console input */

```

```

<function kbdqinit 340c>≡ (607)
void
kbdqinit(void)
{
    kbdq = qopen(4*1024, 0, 0, 0);
    if(kbdq == nil)
        panic("kbdinit");
    qnoblock(kbdq, true);
}

```

```

<global lineq 340d>≡ (607)
Queue* lineq;         /* processed console input */

```

```

<function lineqinit 340e>≡ (607)
void
lineqinit(void)
{
    lineq = qopen(2*1024, 0, nil, nil);
    if(lineq == nil)
        panic("lineqinit");
    qnoblock(lineq, true);
}

```

```

<ConsKbd other fields 340f>≡ (334b) 347a▷
char    line[1024]; /* current input line */
int x;    /* index into line */

```

This is the core of console input. The outer `while` loop spins until `lineq` has data. Inside, it reads one byte at a time from `kbdq` and implements a simple line editor: backspace decrements the index, Control-U resets it to zero, newline marks the line as complete (sets `send`), and Control-D sends whatever has been accumulated (even if empty, which the reader sees as EOF). When `send` is true or the line buffer is full, the accumulated

bytes are written to `lineq` in one shot. This batching means a `read` on `/dev/cons` always returns a complete line in cooked mode.

```

<consread() Qcons case 341a>≡ (345c)
case Qcons:
    qlock(&kbd);
    if(waserror()) {
        qunlock(&kbd);
        nexterror();
    }
    while(!qcanread(lineq)){
        if(qread(kbdq, &ch, 1) == 0)
            continue;
        send = false;
        if(ch == 0){
            /* flush output on rawoff -> rawon */
            if(kbd.x > 0)
                send = !qcanread(kbdq);

<consread() else if raw mode 347b>
        }else{
            switch(ch){
            case '\b':
                if(kbd.x > 0)
                    kbd.x--;
                break;
            case 0x15: /* ^U */
                kbd.x = 0;
                break;
            case '\n':
                send = true;
                kbd.line[kbd.x++] = ch;
                break;
            case 0x04: /* ^D */
                send = true;
                break;
            default:
                kbd.line[kbd.x++] = ch;
                break;
            }
        }
        if(send || kbd.x == sizeof kbd.line){
            qwrite(lineq, kbd.line, kbd.x);
            kbd.x = 0;
        }
    }
    n = qread(lineq, buf, n);
    qunlock(&kbd);
    poperror();
    return n;

```

15.3.3 Output

Writing to `/dev/cons` calls `[putstrn0]`, which dispatches to whatever output backend is active: the `screenputs` hook for display output, `kmesg` for the kernel message buffer, or the serial port. The data is copied to a local buffer first to avoid page faults while holding locks inside the output path.

```

<conswrite() Qcons case 341b>≡ (346a)
case Qcons:

```

```

/*
 * Can't page fault in putstrn, so copy the data locally.
 */
l = n;
while(l > 0){
    bp = 1;
    if(bp > sizeof buf)
        bp = sizeof buf;
    memmove(buf, a, bp);
    putstrn0(buf, bp, 1);
    a += bp;
    l -= bp;
}
break;

⟨function putstrn0 342a⟩≡ (607)
/*
 * Print a string on the console.
 */
static void
putstrn0(char *str, int n, bool usewrite)
{
    int m;
    char *t;

    if(!arch_islo())
        usewrite = false;

    ⟨putstrn0() kmesg handling 581e⟩
    ⟨putstrn0() if kprint 583d⟩
    else if(screenputs != nil)
        screenputs(str, n);

    ⟨putstrn0() serialog handling 585a⟩
}

```

15.3.4 print(), iprint(), pprint()

print()

```

⟨constant PRINTSIZE 342b⟩≡ (667b)
    PRINTSIZE = 256,

⟨function print 342c⟩≡ (607)
    bool noprint; // to debug?

    int
    devcons_print(char *fmt, ...)
    {
        int n;
        va_list arg;
        char buf[PRINTSIZE];

        if(noprint)
            return -1;

        va_start(arg, fmt);
        n = vseprint(buf, buf+sizeof(buf), fmt, arg) - buf;
        va_end(arg);
    }

```

```

    putstrn(buf, n);

    return n;
}

⟨function putstrn 343a⟩≡ (607)
void
putstrn(char *str, int n)
{
    putstrn0(str, n, false);
}

iprint()

⟨global iprintscreenputs 343b⟩≡ (607)
bool iprintscreenputs = true;

⟨function iprint 343c⟩≡ (607)
int
devcons_iprint(char *fmt, ...)
{
    int n, s, locked;
    va_list arg;
    char buf[PRINTSIZE];

    s = arch_splhi();

    va_start(arg, fmt);
    n = vseprint(buf, buf+sizeof(buf), fmt, arg) - buf;
    va_end(arg);

    locked = iprintcanlock(&iprintlock);

    kmesgputs(buf, n); // addon
    if(screenputs != nil && iprintscreenputs)
        screenputs(buf, n);
    uartputs(buf, n);

    if(locked)
        unlock(&iprintlock);

    arch_splx(s);

    return n;
}

⟨global iprintlock 343d⟩≡ (607)
/*
 * Want to interlock iprints to avoid interlaced output on
 * multiprocessor, but don't want to deadlock if one processor
 * dies during print and another has something important to say.
 * Make a good faith effort.
 */
static Lock iprintlock;

```



```

⟨function iprintcanlock 344a⟩≡ (607)
static bool
iprintcanlock(Lock *l)
{
    int i;

    for(i=0; i<1000; i++){
        if(canlock(l))
            return true;
        if(l->cpu == CPUS(cpu->cpuno))
            return false;
        arch_microdelay(100);
    }
    return false;
}

```

pprint()

```

⟨function pprint 344b⟩≡ (607)
int
devcons_pprint(char *fmt, ...)
{
    int n;
    Chan *c;
    va_list arg;
    char buf[2*PRINTSIZE];

    if(up == nil || up->fgrp == nil)
        return 0;

    c = up->fgrp->fd[2]; // stderr
    if(c==nil || (c->mode!=OWRITE && c->mode!=ORDWR))
        return 0;

    n = snprintf(buf, sizeof buf, "%s %lud: ", up->text, up->pid);
    va_start(arg, fmt);
    n = vseprint(buf+n, buf+sizeof(buf), fmt, arg) - buf;
    va_end(arg);

    if(waserror())
        return 0;
    devtab[c->type]->write(c, buf, n, c->offset);
    poperror();

    lock(c);
    c->offset += n;
    unlock(c);

    return n;
}

```

15.3.5 /dev/cons

```

⟨method consinit 344c⟩≡ (607)
static void
consinit(void)
{
    ⟨consinit() initializing things 335b⟩
}

```

<method consopen 345a>≡ (607)

```
static Chan*
consopen(Chan *c, int omode)
{
    c->aux = nil;
    c = devopen(c, omode, consdir, nelem(consdir), devgen);

    switch((ulong)c->qid.path){
        <consopen() cases 347d>
    }
    return c;
}
```

<method consclose 345b>≡ (607)

```
static void
consclose(Chan *c)
{
    switch((ulong)c->qid.path){
        <consclose() cases 347e>
    }
}
```

<method consread 345c>≡ (607)

```
static long
consread(Chan *c, void *buf, long n, vlong off)
{
    ulong l;
    char ch;
    char tmp[256];          /* must be >= 18*NUMSIZE (Qswap) */
    int i, k, send;
    vlong offset = off;

    if(n <= 0)
        return n;

    switch((ulong)c->qid.path){
        case Qdir:
            return devdirread(c, buf, n, consdir, nelem(consdir), devgen);

        <consread() Qcons case 341a>

        <consread() cases 179b>

        case Quser:
            return readstr((ulong)offset, buf, n, up->user);

        case Qpid:
            return readnum((ulong)offset, buf, n, up->pid, NUMSIZE);

        case Qppid:
            return readnum((ulong)offset, buf, n, up->parentpid, NUMSIZE);

        case Qpgrp:
            return readnum((ulong)offset, buf, n, up->pgrp->pgrp, NUMSIZE);

        case Qnull:
            return 0;

        case Qzero:

```

```

    memset(buf, 0, n);
    return n;

default:
    print("consread %#llx\n", c->qid.path);
    error(Egreg);
}
panic("consread: should not reach this point");
return -1;
}

```

<method conswrite 346a>≡ (607)

```

static long
conswrite(Chan *c, void *va, long n, vlong off)
{
    char *a;

    <conswrite locals 346b>

    a = va;
    USED(off);

    switch((ulong)c->qid.path){

    <conswrite() Qcons case 341b>

    <conswrite() cases 188c>

    case Quser:
        return userwrite(a, n);

    // > /dev/null :)
    case Qnull:
        break;

    default:
        print("conswrite: %#llx\n", c->qid.path);
        error(Egreg);
    }
    return n;
}

```

<conswrite locals 346b>≡ (346a)

```

char buf[256], ch;
long l, bp;
int fd;
Chan *swc;

```

<global consdevtab 346c>≡ (607)

```

Dev consdevtab = {
    .dc      =    'c',
    .name     =    "cons",

    .reset    =    devreset,
    .init     =    consinit,
    .shutdown =    devshutdown,
    .attach   =    consattach,
    .walk     =    conswalk,
    .stat     =    consstat,
}

```

```

.open      =   consopen,
.create    =   devcreate,
.close     =   consclose,
.read      =   consread,
.bread     =   devbread,
.write     =   conswrite,
.bwrite    =   devbwrite,
.remove    =   devremove,
.wstat     =   devwstat,
};

```

15.3.6 /dev/consctl and raw input

Raw mode is controlled by writing “rawon” or “rawoff” to /dev/consctl. When raw mode is active, [consread] bypasses line editing entirely: each byte from kbdq is passed straight through without waiting for a newline, and [echo] skips the screen echo. The ctl reference count tracks how many processes have /dev/consctl open; when the last one closes it, raw mode is automatically turned off. This is important because programs like rio that need raw keyboard input must not leave the console in a broken state if they crash.

⟨ConsKbd other fields 347a⟩+≡ (334b) ◁340f 561e▷

```

bool raw;          /* true if we shouldn't process input */
Ref ctl;           /* number of opens to the control file */

```

⟨consread() else if raw mode 347b⟩≡ (341a)

```

}else if(kbd.raw){
    kbd.line[kbd.x++] = ch;
    send = !qcanread(kbdq);
}

```

⟨echo() return before any echoscreen if raw mode 347c⟩≡ (336a)

```

if(kbd.raw)
    return;

```

⟨consopen() cases 347d⟩≡ (345a) 583a▷

```

case Qconsctl:
    incref(&kbd.ctl);
    break;

```

⟨consclose() cases 347e⟩≡ (345b) 583b▷

```

/* last close of control file turns off raw */
case Qconsctl:
    if(c->flag&COPEN){
        if(decreef(&kbd.ctl) == 0)
            kbd.raw = false;
    }
    break;

```

⟨conswrite() cases 347f⟩+≡ (346a) ◁232c

```

case Qconsctl:
    if(n >= sizeof(buf))
        n = sizeof(buf)-1;
    strncpy(buf, a, n);
    buf[n] = 0;
    for(a = buf; a;){
        if(strncmp(a, "rawon", 5) == 0){
            kbd.raw = true;
            /* clumsy hack - wake up reader */
            ch = 0;
            qwrite(kbdq, &ch, 1);
        } else if(strncmp(a, "rawoff", 6) == 0){

```

```

        kbd.raw = false;
    }
    <conswrite() Qconscctl other ifs 561f>
    if(a = strchr(a, ' '))
        a++;
}
break;

```

15.4 The mouse

15.4.1 Mouse and keyboard

The keyboard and mouse are tightly coupled in Plan 9. The `Kbscan` structure tracks mouse button state alongside modifier keys because some keyboards generate mouse button events (e.g., via USB HID). When the scan code falls in the `Kmouse` range, `[kbdputc]` updates the `buttons` bitmask and calls the `kbdmouse` hook, which injects the event into the mouse device driver.

```

<Kbscan other fields 348a>+≡ (330c) <333c 603b>
    int buttons;

```

```

<kbdputc() reset mouseshift 348b>≡ (331b)
    mouseshifted = false;

```

```

<kbdputc() set mouseshift 348c>≡ (331b)
    mouseshifted = true;

```

```

<kbdputc() mouse keyup cases 348d>≡ (331b)
    case Kmouse|1:
    case Kmouse|2:
    case Kmouse|3:
    case Kmouse|4:
    case Kmouse|5:
        kbscan->buttons &= ~(1<<(c-Kmouse-1));
        if(kbdmouse)
            kbdmouse(kbscan->buttons);
        break;

```

```

<kbdputc() mouse keydown cases 348e>≡ (331b)
    case Kmouse|1:
    case Kmouse|2:
    case Kmouse|3:
    case Kmouse|4:
    case Kmouse|5:
        kbscan->buttons |= 1<<(c-Kmouse-1);
        if(kbdmouse)
            kbdmouse(kbscan->buttons);
        break;

```

15.4.2 PS2 mouse (x86)

15.4.3 USB mouse (ARM)

15.5 Other devices

Chapter 16

Queue

Queues are the kernel's general-purpose buffering mechanism for I/O data flow. Unlike simple byte ring buffers, these queues are block-based: data is stored in linked lists of **Block** structures, each containing a contiguous byte range. This design is optimized for networking where data arrives in packets that should not be split unnecessarily. Queues provide flow control (back-pressure when a producer is faster than a consumer), support both message mode (preserving packet boundaries) and stream mode (treating data as a byte stream), and offer two APIs: a process-context API ([qread]/[qwrite]) that can sleep, and an interrupt-context API ([qproduce]/[qconsume]) that never blocks. The state bits encode the queue's current condition. The two most important for flow control are **Qstarve** and **Qflow**: **Qstarve** is set by the consumer when it finds an empty queue, signaling the producer to wake it up when data arrives; **Qflow** is set by the producer when the queue exceeds its limit, signaling it to stop and wait. **Qmsg** changes the read semantics: in stream mode, partial blocks can be returned; in message mode, each read returns exactly one block (or discards excess data), preserving packet boundaries for protocols like UDP.

```
<enum queuestate 349a>≡ (645c)
/* queue state bits, Qmsg, Qcoalesce, and Qkick can be set in qopen */
enum QueueState
{
    /* Queue.state */
    Qstarve  = (1<<0), /* consumer starved */
    Qmsg     = (1<<1), /* message stream */
    Qclosed  = (1<<2), /* queue has been closed/hungup */
    Qflow    = (1<<3), /* producer flow controlled */
    Qcoalesce = (1<<4), /* coalesce packets on read */
    Qkick    = (1<<5), /* always call the kick routine after qwrite */
};
```

The **Queue** structure uses separate locks for different concerns. The embedded **Lock** (a spin lock) protects the block list and state bits – it is used with **ilock**/**iunlock** so it can be held at interrupt level. The **rlock** and **wlock** **QLocks** (sleeping mutexes) serialize multiple reader or writer processes respectively. The **Rendez** variables **rr** and **wr** are the sleep/wakeup rendezvous points for blocked readers and writers. The **kick** callback is called when the queue drains below the flow control threshold, which is used by network drivers to restart hardware transmission.

```
<struct Queue 349b>≡ (645c)
struct Queue
{
    Lock;

    Block* bfirst; /* buffer */
    Block* blast;

    int len; /* bytes allocated to queue */
    int dlen; /* data bytes in queue */
};
```

```

int limit;    /* max bytes in queue */
int inilim;   /* initial limit */
int state;
int noblock;  /* true if writes return immediately when q full */
int eof;      /* number of eofs read by user */

void (*kick)(void*); /* restart output */
void (*bypass)(void*, Block*); /* bypass queue altogether */
void* arg;      /* argument to kick */

QLock rlock;   /* mutex for reading processes */
Rendez rr;     /* process waiting to read */
QLock wlock;   /* mutex for writing processes */
Rendez wr;     /* process waiting to write */

char err[ERRMAX];
};

⟨function pullupqueue 350a⟩≡ (652b)
/*
 * make sure the first block has at least n bytes
 */
Block*
pullupqueue(Queue *q, int n)
{
    Block *b;

    if(BLEN(q->bfirst) >= n)
        return q->bfirst;
    q->bfirst = pullupblock(q->bfirst, n);
    for(b = q->bfirst; b != nil && b->next != nil; b = b->next)
        ;
    q->blast = b;
    return q->bfirst;
}

```

[qwrite] is the process-context write API. It breaks the data into `Maxatomic`-sized chunks, wraps each in a `Block`, and calls [qbwrit] which handles flow control and sleeping. In message mode (`Qmsg`), only the first chunk is sent – the loop exits after one iteration – preserving the message as a single block. In stream mode, all chunks are sent sequentially.

```

⟨function qwrite 350b⟩≡ (652b)
/*
 * write to a queue. only Maxatomic bytes at a time is atomic.
 */
int
qwrite(Queue *q, void *vp, int len)
{
    int n, sofar;
    Block *b;
    byte *p = vp;

    // QDEBUG if(!arch_islo()) print("qwrite hi %#p\n", getcallerpc(&q));

    sofar = 0;
    do {
        n = len-sofar;
        if(n > Maxatomic)
            n = Maxatomic;

```

```

    b = allocb(n);
    setmalloctag(b, (up->text[0]<<24)|(up->text[1]<<16)|(up->text[2]<<8)|up->text[3]);
    if(waserror()){
        freeb(b);
        nexterror();
    }
    memmove(b->wp, p+sofar, n);
    poperror();
    b->wp += n;

    qbwrite(q, b);

    sofar += n;
} while(sofar < len && (q->state & Qmsg) == 0);

return len;
}

```

⟨function qget 351a⟩≡ (652b)

```

/*
 * get next block from a queue, return null if nothing there
 */
Block*
qget(Queue *q)
{
    int dowakeup;
    Block *b;

    /* sync with qwrite */
    ilock(q);

    b = q->bfirst;
    if(b == nil){
        q->state |= Qstarve;
        iunlock(q);
        return nil;
    }
    q->bfirst = b->next;
    b->next = 0;
    q->len -= BALLOC(b);
    q->dlen -= BLEN(b);
    // QDEBUG checkb(b, "qget");

    /* if writer flow controlled, restart */
    if((q->state & Qflow) && q->len < q->limit/2){
        q->state &= ~Qflow;
        dowakeup = 1;
    } else
        dowakeup = 0;

    iunlock(q);

    if(dowakeup)
        wakeup(&q->wr);

    return b;
}

```

⟨function qdiscard 351b⟩≡ (652b)

```

/*

```



```

    * throw away the next 'len' bytes in the queue
    */
int
qdiscard(Queue *q, int len)
{
    Block *b;
    int dowakeup, n, sofar;

    ilock(q);
    for(sofar = 0; sofar < len; sofar += n){
        b = q->bfirst;
        if(b == nil)
            break;
        // QDEBUG checkb(b, "qdiscard");
        n = BLEN(b);
        if(n <= len - sofar){
            q->bfirst = b->next;
            b->next = 0;
            q->len -= BALLOC(b);
            q->dlen -= BLEN(b);
            freeb(b);
        } else {
            n = len - sofar;
            b->rp += n;
            q->dlen -= n;
        }
    }

    /*
     * if writer flow controlled, restart
     *
     * This used to be
     * q->len < q->limit/2
     * but it slows down tcp too much for certain write sizes.
     * I really don't understand it completely. It may be
     * due to the queue draining so fast that the transmission
     * stalls waiting for the app to produce more data. - presotto
     */
    if((q->state & Qflow) && q->len < q->limit){
        q->state &= ~Qflow;
        dowakeup = 1;
    } else
        dowakeup = 0;

    iunlock(q);

    if(dowakeup)
        wakeup(&q->wr);

    return sofar;
}

```

[qproduce] and [qconsume] are the interrupt-safe queue API. They use `ilock` (spin lock with interrupts disabled) instead of `qlock` (sleeping mutex), so they can be called from interrupt handlers. [qconsume] copies data out of the first block. If the queue is empty it returns `-1` and sets `Qstarve` rather than blocking – an interrupt handler cannot sleep. These are the functions used by the keyboard driver ([echo] calls [qproduce] to feed `kbdq`) and by network device drivers that process packets at interrupt time.

⟨function qconsume 352⟩≡
/*

(652b)

```

* Interrupt level copy out of a queue, return # bytes copied.
*/
int
qconsume(Queue *q, void *vp, int len)
{
    Block *b;
    int n, dowakeup;
    byte *p = vp;
    Block *tofree = nil;

    /* sync with qwrite */
    ilock(q);

    for(;;) {
        b = q->bfirst;
        if(b == 0){
            q->state |= Qstarve;
            iunlock(q);
            return -1;
        }
        // QDEBUG checkb(b, "qconsume 1");

        n = BLEN(b);
        if(n > 0)
            break;
        q->bfirst = b->next;
        q->len -= BALLOC(b);

        /* remember to free this */
        b->next = tofree;
        tofree = b;
    };

    if(n < len)
        len = n;
    memmove(p, b->rp, len);
    consumeent += n;
    b->rp += len;
    q->dlen -= len;

    /* discard the block if we're done with it */
    if((q->state & Qmsg) || len == n){
        q->bfirst = b->next;
        b->next = 0;
        q->len -= BALLOC(b);
        q->dlen -= BLEN(b);

        /* remember to free this */
        b->next = tofree;
        tofree = b;
    }

    /* if writer flow controlled, restart */
    if((q->state & Qflow) && q->len < q->limit/2){
        q->state &= ~Qflow;
        dowakeup = 1;
    } else
        dowakeup = 0;

    iunlock(q);

```

```

    if(dowakeup)
        wakeup(&q->wr);

    if(tofree != nil)
        freeblist(tofree);

    return len;
}

```

<function qpass 354>≡ (652b)

```

int
qpass(Queue *q, Block *b)
{
    int dlen, len, dowakeup;

    /* sync with qread */
    dowakeup = 0;
    ilock(q);
    if(q->len >= q->limit){
        freeblist(b);
        iunlock(q);
        return -1;
    }
    if(q->state & Qclosed){
        len = BALLOC(b);
        freeblist(b);
        iunlock(q);
        return len;
    }

    /* add buffer to queue */
    if(q->bfirst)
        q->blast->next = b;
    else
        q->bfirst = b;
    len = BALLOC(b);
    dlen = BLEN(b);
    //    QDEBUG checkb(b, "qpass");
    while(b->next){
        b = b->next;
    //    QDEBUG checkb(b, "qpass");
        len += BALLOC(b);
        dlen += BLEN(b);
    }
    q->blast = b;
    q->len += len;
    q->dlen += dlen;

    if(q->len >= q->limit/2)
        q->state |= Qflow;

    if(q->state & Qstarve){
        q->state &= ~Qstarve;
        dowakeup = 1;
    }
    iunlock(q);

    if(dowakeup)
        wakeup(&q->rr);
}

```

```

        return len;
    }

⟨function qpassnolim 355a⟩≡ (652b)
    int
    qpassnolim(Queue *q, Block *b)
    {
        int dlen, len, dowakeup;

        /* sync with qread */
        dowakeup = 0;
        ilock(q);

        if(q->state & Qclosed){
            freeblist(b);
            iunlock(q);
            return BALLOC(b);
        }

        /* add buffer to queue */
        if(q->bfirst)
            q->blast->next = b;
        else
            q->bfirst = b;
        len = BALLOC(b);
        dlen = BLEN(b);
        // QDEBUG checkb(b, "qpass");
        while(b->next){
            b = b->next;
        // QDEBUG checkb(b, "qpass");
            len += BALLOC(b);
            dlen += BLEN(b);
        }
        q->blast = b;
        q->len += len;
        q->dlen += dlen;

        if(q->len >= q->limit/2)
            q->state |= Qflow;

        if(q->state & Qstarve){
            q->state &= ~Qstarve;
            dowakeup = 1;
        }
        iunlock(q);

        if(dowakeup)
            wakeup(&q->rr);

        return len;
    }

```

[qproduce] is the interrupt-safe counterpart of [qwrite]. It allocates a block with `iallocb` (the interrupt-safe allocator that does not sleep), copies the data in, and appends it to the queue. If the queue is full, it returns `-1` and sets `Qflow` rather than blocking. If a consumer was starved (waiting for data), it clears `Qstarve` and wakes it up.

```

⟨function qproduce 355b⟩≡ (652b)
    int

```

```

qproduce(Queue *q, void *vp, int len)
{
    Block *b;
    int dowakeup;
    byte *p = vp;

    /* sync with qread */
    dowakeup = 0;
    ilock(q);

    /* no waiting receivers, room in buffer? */
    if(q->len >= q->limit){
        q->state |= Qflow;
        iunlock(q);
        return -1;
    }

    /* save in buffer */
    b = iallocb(len);
    if(b == 0){
        iunlock(q);
        return 0;
    }
    memmove(b->wp, p, len);
    producecnt += len;
    b->wp += len;
    if(q->bfirst)
        q->blast->next = b;
    else
        q->bfirst = b;
    q->blast = b;
    /* b->next = 0; done by iallocb() */
    q->len += BALLOC(b);
    q->dlen += BLEN(b);
    //    QDEBUG checkb(b, "qproduce");

    if(q->state & Qstarve){
        q->state &= ~Qstarve;
        dowakeup = 1;
    }

    if(q->len >= q->limit)
        q->state |= Qflow;
    iunlock(q);

    if(dowakeup)
        wakeup(&q->rr);

    return len;
}

```

⟨function qcopy 356⟩≡

(652b)

```

/*
 * copy from offset in the queue
 */
Block*
qcopy(Queue *q, int len, ulong offset)
{
    intsofar;
    int n;

```

```

Block *b, *nb;
byte *p;

nb = allocb(len);

ilock(q);

/* go to offset */
b = q->bfirst;
for(sofar = 0; ;sofar += n){
    if(b == nil){
        iunlock(q);
        return nb;
    }
    n = BLEN(b);
    if(sofar + n > offset){
        p = b->rp + offset - sofar;
        n -= offset - sofar;
        break;
    }
    //      QDEBUG checkb(b, "qcopy");
    b = b->next;
}

/* copy bytes from there */
for(sofar = 0; sofar < len;){
    if(n > len - sofar)
        n = len - sofar;
    memmove(nb->wp, p, n);
    qcopycnt += n;
    sofar += n;
    nb->wp += n;
    b = b->next;
    if(b == nil)
        break;
    n = BLEN(b);
    p = b->rp;
}
iunlock(q);

return nb;
}

```

[qopen] creates a new queue. The **limit** parameter sets the flow control threshold: when the total allocated bytes exceed this limit, producers get back-pressured. The **msg** parameter is actually the initial state bits – passing **Qmsg** here creates a message-mode queue. The **kick** callback, if provided, is called whenever the queue drains below the flow control mark, giving network drivers a chance to restart hardware transmission. The queue starts in **Qstarve** state because there is no data yet.

```

⟨function qopen 357⟩≡ (652b)
/*
 * called by non-interrupt code
 */
Queue*
qopen(int limit, int msg, void (*kick)(void*), void *arg)
{
    Queue *q;

    q = malloc(sizeof(Queue));
    if(q == nil)

```

```

        return nil;

    q->limit = q->inilim = limit;
    q->kick = kick;
    q->arg = arg;
    q->state = msg;

    q->state |= Qstarve;
    q->eof = 0;
    q->noblock = 0;

    return q;
}

```

```

⟨function qbypass 358a⟩≡ (652b)
/* open a queue to be bypassed */
Queue*
qbypass(void (*bypass)(void*, Block*), void *arg)
{
    Queue *q;

    q = malloc(sizeof(Queue));
    if(q == nil)
        return nil;

    q->limit = 0;
    q->arg = arg;
    q->bypass = bypass;
    q->state = 0;

    return q;
}

```

```

⟨function notempty 358b⟩≡ (652b)
static int
notempty(void *a)
{
    Queue *q = a;

    return (q->state & Qclosed) || q->bfirst != 0;
}

```

[qwait] implements the sleep/wakeup protocol for readers. It is called with the queue's spin lock held. If the queue is empty, it sets `Qstarve`, drops the lock, and sleeps on the `rr` rendezvous until [notempty] returns true. When it wakes up, it re-acquires the lock and checks again (the loop handles spurious wakeups). For closed queues, the `eof` counter allows up to 3 reads to return 0 (EOF) before returning an error, matching the Unix convention that EOF can be read multiple times.

```

⟨function qwait 358c⟩≡ (652b)
/*
 * wait for the queue to be non-empty or closed.
 * called with q ilocked.
 */
static int
qwait(Queue *q)
{
    /* wait for data */
    for(;;){
        if(q->bfirst != nil)
            break;
    }
}

```

```

    if(q->state & Qclosed){
        if(++q->eof > 3)
            return -1;
        if(*q->err && strcmp(q->err, Ehungup) != 0)
            return -1;
        return 0;
    }

    q->state |= Qstarve;    /* flag requesting producer to wake me */
    iunlock(q);
    sleep(&q->rr, notempty, q);
    ilock(q);
}
return 1;
}

```

⟨function qaddlist 359a⟩≡ (652b)

```

/*
 * add a block list to a queue
 */
void
qaddlist(Queue *q, Block *b)
{
    /* queue the block */
    if(q->bfirst)
        q->blast->next = b;
    else
        q->bfirst = b;
    q->len += blockalloclen(b);
    q->dlen += blocklen(b);
    while(b->next)
        b = b->next;
    q->blast = b;
}

```

⟨function qremove 359b⟩≡ (652b)

```

/*
 * called with q ilocked
 */
Block*
qremove(Queue *q)
{
    Block *b;

    b = q->bfirst;
    if(b == nil)
        return nil;
    q->bfirst = b->next;
    b->next = nil;
    q->dlen -= BLEN(b);
    q->len -= BALLOC(b);
    // QDEBUG checkb(b, "qremove");
    return b;
}

```

⟨function qputback 359c⟩≡ (652b)

```

/*
 * put a block back to the front of the queue
 * called with q ilocked

```



```

    */
void
qputback(Queue *q, Block *b)
{
    b->next = q->bfirst;
    if(q->bfirst == nil)
        q->blast = b;
    q->bfirst = b;
    q->len += BALLOC(b);
    q->dlen += BLEN(b);
}

⟨function qwakeuper_iunlock 360a⟩≡ (652b)
/*
 * flow control, get producer going again
 * called with q ilocked
 */
static void
qwakeup_iunlock(Queue *q)
{
    int dowakeup = 0;

    /* if writer flow controlled, restart */
    if((q->state & Qflow) && q->len < q->limit/2){
        q->state &= ~Qflow;
        dowakeup = 1;
    }

    iunlock(q);

    /* wakeup flow controlled writers */
    if(dowakeup){
        if(q->kick)
            q->kick(q->arg);
        wakeup(&q->wr);
    }
}

```

[qbread] and [qread] are the process-context read API. They acquire `rlock` to serialize readers, then call [qwait] which sleeps on `rr` until data arrives or the queue is closed. A key difference between message and stream mode appears here: in stream mode, if the first block is larger than requested, it is split and the remainder is put back with [qputback]; in message mode, excess data is silently discarded, preserving packet boundaries.

```

⟨function qbread 360b⟩≡ (652b)
/*
 * get next block from a queue (up to a limit)
 */
Block*
qbread(Queue *q, int len)
{
    Block *b, *nb;
    int n;

    qlock(&q->rlock);
    if(waserror()){
        qunlock(&q->rlock);
        nexterror();
    }
}

```

```

ilock(q);
switch(qwait(q)){
case 0:
    /* queue closed */
    iunlock(q);
    qunlock(&q->rlock);
    poperror();
    return nil;
case -1:
    /* multiple reads on a closed queue */
    iunlock(q);
    error(q->err);
}

/* if we get here, there's at least one block in the queue */
b = qremove(q);
n = BLEN(b);

/* split block if it's too big and this is not a message queue */
nb = b;
if(n > len){
    if((q->state&Qmsg) == 0){
        n -= len;
        b = allocb(n);
        memmove(b->wp, nb->rp+len, n);
        b->wp += n;
        qputback(q, b);
    }
    nb->wp = nb->rp + len;
}

/* restart producer */
qwakep_iunlock(q);

poperror();
qunlock(&q->rlock);
return nb;
}

```

<function qread 361>≡ (652b)

```

/*
 * read a queue.  if no data is queued, post a Block
 * and wait on its Rendez.
 */
long
qread(Queue *q, void *vp, int len)
{
    Block *b, *first, **l;
    int m, n;

    qlock(&q->rlock);
    if(waserror()){
        qunlock(&q->rlock);
        nexterror();
    }

    ilock(q);
again:
    switch(qwait(q)){
case 0:

```

```

    /* queue closed */
    iunlock(q);
    qunlock(&q->rlock);
    poperror();
    return 0;
case -1:
    /* multiple reads on a closed queue */
    iunlock(q);
    error(q->err);
}

/* if we get here, there's at least one block in the queue */
if(q->state & Qcoalesce){
    /* when coalescing, 0 length blocks just go away */
    b = q->bfirst;
    if(BLEN(b) <= 0){
        freeb(qremove(q));
        goto again;
    }

    /* grab the first block plus as many
     * following blocks as will completely
     * fit in the read.
     */
    n = 0;
    l = &first;
    m = BLEN(b);
    for(;;) {
        *l = qremove(q);
        l = &b->next;
        n += m;

        b = q->bfirst;
        if(b == nil)
            break;
        m = BLEN(b);
        if(n+m > len)
            break;
    }
} else {
    first = qremove(q);
    n = BLEN(first);
}

/* copy to user space outside of the ilock */
iunlock(q);
b = bl2mem(vp, first, len);
ilock(q);

/* take care of any left over partial block */
if(b != nil){
    n -= BLEN(b);
    if(q->state & Qmsg)
        freeb(b);
    else
        qputback(q, b);
}

/* restart producer */
qwakeuper_iunlock(q);

```

```

    poperror();
    qunlock(&q->rlock);
    return n;
}

```

```

⟨function qnotfull 363a⟩≡ (652b)
    static int
    qnotfull(void *a)
    {
        Queue *q = a;

        return q->len < q->limit || (q->state & Qclosed);
    }

```

[qbwrite] is the workhorse behind [qwrite]. It handles the full lifecycle: check for closed/bypass queues, append the block, wake up starved readers, call the `kick` callback for network drivers, and then – critically – block the writer if the queue exceeds its limit. The flow control sleep happens *after* the data is queued, not before. This is deliberate: it ensures that 9P flush messages and SSL records are fully queued before the process can be interrupted, preventing partial writes from corrupting protocol state.

```

⟨function qbwrite 363b⟩≡ (652b)
/*
 * add a block to a queue obeying flow control
 */
long
qbwrite(Queue *q, Block *b)
{
    int n, dowakeup;
    Proc *p;

    n = BLEN(b);

    if(q->bypass){
        (*q->bypass)(q->arg, b);
        return n;
    }

    dowakeup = 0;
    qlock(&q->wlock);
    if(waserror()){
        if(b != nil)
            freeb(b);
        qunlock(&q->wlock);
        nexterror();
    }

    ilock(q);

    /* give up if the queue is closed */
    if(q->state & Qclosed){
        iunlock(q);
        error(q->err);
    }

    /* if nonblocking, don't queue over the limit */
    if(q->len >= q->limit){
        if(q->noblock){
            iunlock(q);
            freeb(b);

```

```

        noblockcnt += n;
        qunlock(&q->wlock);
        perror();
        return n;
    }
}

/* queue the block */
if(q->bfirst)
    q->blast->next = b;
else
    q->bfirst = b;
q->blast = b;
b->next = 0;
q->len += BALLOC(b);
q->dlen += n;
// QDEBUG checkb(b, "qbwrite");
b = nil;

/* make sure other end gets awakened */
if(q->state & Qstarve){
    q->state &= ~Qstarve;
    dowakeup = 1;
}
iunlock(q);

/* get output going again */
if(q->kick && (dowakeup || (q->state&Qkick)))
    q->kick(q->arg);

/* wakeup anyone consuming at the other end */
if(dowakeup){
    p = wakeup(&q->rr);

    /* if we just wokeup a higher priority process, let it run */
    if(p != nil && p->priority > up->priority)
        sched();
}

/*
 * flow control, wait for queue to get below the limit
 * before allowing the process to continue and queue
 * more. We do this here so that postnote can only
 * interrupt us after the data has been queued. This
 * means that things like 9p flushes and ssl messages
 * will not be disrupted by software interrupts.
 *
 * Note - this is moderately dangerous since a process
 * that keeps getting interrupted and rewriting will
 * queue infinite crud.
 */
for(;;){
    if(q->noblock || qnotfull(q))
        break;

    ilock(q);
    q->state |= Qflow;
    iunlock(q);
    sleep(&q->wr, qnotfull, q);
}

```

```

USED(b);

qunlock(&q->wlock);
poperror();
return n;
}

```

[qiwrite] is a special variant used by `print()` and other kernel output functions. Unlike [qbwrite], it does not use `qlock` because it may be called from interrupt context or from contexts where no process is running (e.g., during boot). It also uses `iallocb` instead of `allocb` and has a hardcoded 16KB limit to prevent runaway kernel messages from consuming all memory.

```

⟨function qiwrite 365⟩≡ (652b)
/*
 * used by print() to write to a queue. Since we may be arch_splhi or not in
 * a process, don't qlock.
 *
 * this routine merges adjacent blocks if block n+1 will fit into
 * the free space of block n.
 */
int
qiwrite(Queue *q, void *vp, int len)
{
    int n, sofar, dowakeup;
    Block *b;
    byte *p = vp;

    dowakeup = 0;

    sofar = 0;
    do {
        n = len-sofar;
        if(n > Maxatomic)
            n = Maxatomic;

        b = iallocb(n);
        if(b == nil)
            break;
        memmove(b->wp, p+sofar, n);
        b->wp += n;

        ilock(q);

        /* we use an artificially high limit for kernel prints since anything
         * over the limit gets dropped
         */
        if(q->dlen >= 16*1024){
            iunlock(q);
            freeb(b);
            break;
        }

        // QDEBUG checkb(b, "qiwrite");
        if(q->bfirst)
            q->blast->next = b;
        else
            q->bfirst = b;
        q->blast = b;
        q->len += BALLOC(b);
        q->dlen += n;
    }
}

```

```

    if(q->state & Qstarve){
        q->state &= ~Qstarve;
        dowakeup = 1;
    }

    iunlock(q);

    if(dowakeup){
        if(q->kick)
            q->kick(q->arg);
        wakeup(&q->rr);
    }

   sofar += n;
} while(sofar < len && (q->state & Qmsg) == 0);

return sofar;
}

```

<function qfree 366a>≡ (652b)

```

/*
 * be extremely careful when calling this,
 * as there is no reference accounting
 */
void
qfree(Queue *q)
{
    qclose(q);
    free(q);
}

```

[qclose] and [qhangup] both mark a queue as **Qclosed**, but they differ in an important way: [qclose] immediately frees all queued blocks (hard close), while [qhangup] leaves the data in place so the consumer can drain remaining bytes before seeing the hangup (graceful shutdown). The **eof** counter in [qwait] allows a closed queue to be read a few times before returning an error, giving the reader a chance to process the final data. [greopen] reverses a close, restoring the queue to its initial state – this is used when a network connection is recycled.

<function qclose 366b>≡ (652b)

```

/*
 * Mark a queue as closed. No further IO is permitted.
 * All blocks are released.
 */
void
qclose(Queue *q)
{
    Block *bfirst;

    if(q == nil)
        return;

    /* mark it */
    ilock(q);
    q->state |= Qclosed;
    q->state &= ~(Qflow|Qstarve);
    strcpy(q->err, Ehungup);
    bfirst = q->bfirst;
    q->bfirst = 0;
    q->len = 0;
    q->dlen = 0;
}

```

```

q->noblock = 0;
iunlock(q);

/* free queued blocks */
freeblist(bfirst);

/* wake up readers/writers */
wakeup(&q->rr);
wakeup(&q->wr);
}

```

⟨function qhangup 367a⟩≡ (652b)

```

/*
 * Mark a queue as closed. Wakeup any readers. Don't remove queued
 * blocks.
 */
void
qhangup(Queue *q, char *msg)
{
    /* mark it */
    ilock(q);
    q->state |= Qclosed;
    if(msg == 0 || *msg == 0)
        strcpy(q->err, Ehungup);
    else
        strncpy(q->err, msg, ERRMAX-1);
    iunlock(q);

    /* wake up readers/writers */
    wakeup(&q->rr);
    wakeup(&q->wr);
}

```

⟨function qisclosed 367b⟩≡ (652b)

```

/*
 * return non-zero if the q is hungup
 */
int
qisclosed(Queue *q)
{
    return q->state & Qclosed;
}

```

⟨function qreopen 367c⟩≡ (652b)

```

/*
 * mark a queue as no longer hung up
 */
void
qreopen(Queue *q)
{
    ilock(q);
    q->state &= ~Qclosed;
    q->state |= Qstarve;
    q->eof = 0;
    q->limit = q->inilim;
    iunlock(q);
}

```


<function qlen 368a>≡ (652b)

```
/*
 * return bytes queued
 */
int
qlen(Queue *q)
{
    return q->dlen;
}
```

<function qwindow 368b>≡ (652b)

```
/*
 * return space remaining before flow control
 */
int
qwindow(Queue *q)
{
    int l;

    l = q->limit - q->len;
    if(l < 0)
        l = 0;
    return l;
}
```

<function qcanread 368c>≡ (652b)

```
/*
 * return true if we can read without blocking
 */
int
qcanread(Queue *q)
{
    return q->bfirst!=0;
}
```

<function qsetlimit 368d>≡ (652b)

```
/*
 * change queue limit
 */
void
qsetlimit(Queue *q, int limit)
{
    q->limit = limit;
}
```

<function qnoblock 368e>≡ (652b)

```
/*
 * set blocking/nonblocking
 */
void
qnoblock(Queue *q, bool onoff)
{
    q->noblock = onoff;
}
```

<function qflush 368f>≡ (652b)

```
/*
 * flush the output queue
 */
void
```

```

qflush(Queue *q)
{
    Block *bfirst;

    /* mark it */
    ilock(q);
    bfirst = q->bfirst;
    q->bfirst = 0;
    q->len = 0;
    q->dlen = 0;
    iunlock(q);

    /* free queued blocks */
    freeblist(bfirst);

    /* wake up readers/writers */
    wakeup(&q->wr);
}

```

```

⟨function qfull 369⟩≡ (652b)
int
qfull(Queue *q)
{
    return q->state & Qflow;
}

```

Chapter 17

Filesystems

In Plan 9, every device driver implements the **Dev** interface, which is essentially a file server interface. This chapter covers two in-kernel “filesystems” — the root device (**#/**) and the boot device — plus the generic helper functions ([**devdir**], [**devgen**], [**devwalk**], [**devstat**], [**devdirread**]) that factor out the boilerplate every driver needs to present its files as a directory tree. The chapter ends with **Block**, the I/O buffer structure used by queues and networking.

Dirtab is a compact in-memory directory entry used by device drivers to describe their synthetic files. Unlike a full **DirEntry** (which is the on-the-wire 9P format), **Dirtab** contains just the fields a driver needs to enumerate its files: a name, a **Qid**, a length, and permissions. The [**devgen**] family of helpers iterate over arrays of **Dirtab** entries to implement walk, stat, and directory reads.

```
<struct Dirtab 370a>≡ (645c)
struct Dirtab
{
    char    name[KNAMELEN];
    Qid qid;
    vlong length;
    long  perm;
};
```

17.1 The root filesystem

The root device (**#/**) is the in-kernel filesystem that provides the top-level directory structure: **/dev**, **/proc**, **/bin**, **/env**, **/net**, **/srv**, and so on. These are not real directories on disk — they are just entries in a static array that serve as mount points. When the system boots, other devices and file servers get bound onto these directories (e.g., **#c** onto **/dev**, **#p** onto **/proc**). Notice how many of the **Dev** methods delegate to default implementations (**devinit**, **devshutdown**, **devcreate**, **devbread**, **devbwrite**, **devremove**, **devwstat**). The root device only needs custom implementations for **rootreset** (to populate the directory entries), **rootread**/**rootwrite** (to serve file content), and the **walk**/**stat**/**open** methods that use its specific **Dirtab** arrays.

```
<global rootdevtab 370b>≡ (634)
Dev rootdevtab = {
    .dc      = '/',
    .name    = "root",

    .reset   = rootreset,
    .init    = devinit,
    .shutdown = devshutdown,
    .attach  = rootattach,
    .walk    = rootwalk,
    .stat    = rootstat,
    .open    = rootopen,
```

```

    .create    = devcreate,
    .close     = rootclose,
    .read      = rootread,
    .bread     = devbread,
    .write     = rootwrite,
    .bwrite    = devbwrite,
    .remove    = devremove,
    .wstat     = devwstat,
};

```

<devroot enum Qxxx 371a>≡ (634)

```

enum
{
    Qdir = 0,
    Qboot = 0x1000,

    Nrootfiles = 32,
    Nbootfiles = 32,
};

```

The root device stores its files in two parallel arrays: **dir** holds the **Dirtab** entries (names, Qids, permissions) and **data** holds pointers to file contents. The **Dirlist** groups these arrays together with counters. There are two instances: **rootlist** for the root directory and **bootlist** for **/boot/**.

<struct Dirlist 371b>≡ (634)

```

struct Dirlist
{
    uint base; // for unique qids
    Dirtab *dir;
    byte **data;
    int ndir; // number of dir used
    int mdir; // max dir entries
};

```

<globals rootdir, rootdata, rootlist 371c>≡ (634)

```

static Dirtab rootdir[Nrootfiles] = {
    {
        .name = "#/",
        .qid = {Qdir, 0, QTDIR},
        .length = 0,
        .perm = DMDIR|0555,
    },
    {
        .name = "boot",
        .qid = {Qboot, 0, QTDIR},
        .length = 0,
        .perm = DMDIR|0555,
    }
};
static byte *rootdata[Nrootfiles];
static Dirlist rootlist =
{
    .base = 0,
    .dir = rootdir,
    .data = rootdata,
    .ndir = 2,
    .mdir = Nrootfiles
};

```

<function addlist 372a>≡ (634)

```
/*
 * add a file to the list
 */
static void
addlist(Dirlist *l, char *name, byte *contents, ulong len, int perm)
{
    Dirtab *d;

    if(l->ndir >= l->mdir)
        panic("too many root files");
    l->data[l->ndir] = contents;
    d = &l->dir[l->ndir];
    strcpy(d->name, name);
    d->length = len;
    d->perm = perm;
    d->qid.type = 0;
    d->qid.vers = 0;
    d->qid.path = ++l->ndir + l->base;
    if(perm & DMDIR)
        d->qid.type |= QTDIR;
}
```

<function addrootdir 372b>≡ (634)

```
/*
 * add a root directory
 */
static void
addrootdir(char *name)
{
    addlist(&rootlist, name, nil, 0, DMDIR|0555);
}
```

[rootreset] is called once at boot to populate the root directory with the standard Plan 9 top-level directories. These are all empty directories that will later be populated via **bind** and **mount** operations. For example, **/dev** will have **#c** (the console device) bound onto it, **/proc** will get **#p**, and so on.

<method rootreset 372c>≡ (634)

```
static void
rootreset(void)
{
    addrootdir("bin");
    addrootdir("dev");
    addrootdir("env");
    addrootdir("fd");
    addrootdir("mnt");
    addrootdir("net");
    addrootdir("proc");
    addrootdir("root");
    addrootdir("srv");
    addrootdir("sys");
}
```

[rootgen] is the **Devgen** callback for the root device. It dispatches on the **Qid** path to decide whether to list root directories or boot files. When called with **Qdir** it delegates to [devgen] with the **rootlist** table; when called with **Qboot** it uses **bootlist**. For individual files (the default case), it computes an index by subtracting the base **Qid** and looks up the corresponding **Dirtab** entry directly.

<function rootgen 372d>≡ (634)

```
static int
rootgen(Chan *c, char *name, Dirtab*, int, int s, DirEntry *dp)
```

```

{
    int t;
    Dirtab *d;
    Dirlist *l;

    switch((int)c->qid.path){
    case Qdir:
        if(s == DEVDOTDOT){
            devdir(c, (Qid){Qdir, 0, QTDIR}, "#/", 0, eve, 0555, dp);
            return 1;
        }
        return devgen(c, name, rootlist.dir, rootlist.ndir, s, dp);
    case Qboot:
        if(s == DEVDOTDOT){
            devdir(c, (Qid){Qdir, 0, QTDIR}, "#/", 0, eve, 0555, dp);
            return 1;
        }
        return devgen(c, name, bootlist.dir, bootlist.ndir, s, dp);
    default:
        if(s == DEVDOTDOT){
            if((int)c->qid.path < Qboot)
                devdir(c, (Qid){Qdir, 0, QTDIR}, "#/", 0, eve, 0555, dp);
            else
                devdir(c, (Qid){Qboot, 0, QTDIR}, "#/", 0, eve, 0555, dp);
            return 1;
        }
        if(s != 0)
            return -1;
        if((int)c->qid.path < Qboot){
            t = c->qid.path-1;
            l = &rootlist;
        }else{
            t = c->qid.path - Qboot - 1;
            l = &bootlist;
        }
        if(t >= l->ndir)
            return -1;
        d = &l->dir[t];
        devdir(c, d->qid, d->name, d->length, eve, d->perm, dp);
        return 1;
    }
}

```

⟨method rootread 373⟩≡ (634)

```

static long
rootread(Chan *c, void *buf, long n, vlong off)
{
    ulong t;
    Dirtab *d;
    Dirlist *l;
    byte *data;
    ulong offset = off;

    t = c->qid.path;
    switch(t){
    case Qdir:
    case Qboot:
        return devdirread(c, buf, n, nil, 0, rootgen);
    }
}

```

```

    if(t<Qboot)
        l = &rootlist;
    else{
        t -= Qboot;
        l = &bootlist;
    }

    t--;
    if(t >= l->ndir)
        error(Egreg);

    d = &l->dir[t];
    data = l->data[t];
    if(offset >= d->length)
        return 0;
    if(offset+n > d->length)
        n = d->length - offset;
    memmove(buf, data+offset, n);
    return n;
}

```

17.2 The /boot/ filesystem

The /boot/ filesystem is a second in-kernel file store, separate from the root directory. At link time, essential binaries (the `boot` program, `/bin/rc`, `/bin/ls`, etc.) are converted to C data arrays by `data2s` and registered via `[addbootfile]`. This gives the kernel a minimal set of executables it needs to bootstrap without any external storage — enough to run the `boot` script, which then mounts the real filesystem and binds everything into place. Note how `bootlist.base` is set to `Qboot` (0x1000), ensuring that boot file Qids live in a different range from root directory Qids. This lets `[rootgen]` distinguish which `Dirlist` to consult based solely on the Qid path.

<globals bootdir, bootdata, bootlist 374a> ≡ (634)

```

static Dirtab bootdir[Nbootfiles] = {
    {
        .name = "boot",
        .qid = {Qboot, 0, QTDIR},
        .length = 0,
        .perm = DMDIR|0555,
    }
};

static byte *bootdata[Nbootfiles];
static Dirlist bootlist =
{
    .base = Qboot,
    .dir = bootdir,
    .data = bootdata,
    .ndir = 1,
    .mdir = Nbootfiles
};

```

<function addbootfile 374b> ≡ (634)

```

/*
 * add a boot file
 */
void
addbootfile(char *name, byte *contents, ulong len)
{

```

```

    addlist(&bootlist, name, contents, len, 0555);
}

```

17.3 Helpers

The `dev.c` helper functions — `[devdir]`, `[devgen]`, `[devwalk]`, `[devstat]`, `[devdirread]`, `[devopen]` — implement the generic walk/stat/read-directory logic once, parameterized by a `Dirtab` array and a `Devgen` callback. A typical device driver just declares its file table as a static `Dirtab` array and passes it to these helpers.

```

⟨Qidtype cases 375a⟩≡ (52c) 392▷
    QTMOUNT = 0x10, /* type bit for mounted channel */

⟨function devdir 375b⟩≡ (649)
    void
    devdir(Chan *c, Qid qid, char *n, vlong length, char *user, long perm, DirEntry *db)
    {
        db->name = n;
        if(c->flag&CMMSG)
            qid.type |= QTMOUNT;
        db->qid = qid;
        db->type = devtab[c->type]->dc;
        db->dev = c->dev;
        db->mode = perm;
        db->mode |= qid.type << 24;
        db->atime = seconds();
        db->mtime = kerndate;
        db->length = length;
        db->uid = user;
        db->gid = eve;
        db->muid = user;
    }

```

The long comment before `[devgen]` documents a subtle tension in the `Devgen` interface. Rule (i) says a generator should list a directory's children; rule (ii) says it should list siblings. `[devgen]` follows rule (i) only when `c` is the root directory; otherwise it follows (ii). When `name` is non-nil, `[devgen]` does a linear search by name (for walks); when `name` is nil and `s` is a numeric index, it returns the `s`-th entry (for directory reads).

```

⟨function devgen 375c⟩≡ (649)
/*
 * (here, Devgen is the prototype; devgen is the function in dev.c.)
 *
 *
 * a Devgen is expected to return the directory entry for ".."
 * if you pass it s==DEVDOTDOT (-1). otherwise...
 *
 * there are two contradictory rules.
 *
 * (i) if c is a directory, a Devgen is expected to list its children
 * as you iterate s.
 *
 * (ii) whether or not c is a directory, a Devgen is expected to list
 * its siblings as you iterate s.
 *
 * devgen always returns the list of children in the root
 * directory. thus it follows (i) when c is the root and (ii) otherwise.
 * many other Devgens follow (i) when c is a directory and (ii) otherwise.
 *
 * devwalk assumes (i). it knows that devgen breaks (i)

```



```

* for children that are themselves directories, and explicitly catches them.
*
* devstat assumes (ii). if the Devgen in question follows (i)
* for this particular c, devstat will not find the necessary info.
* with our particular Devgen functions, this happens only for
* directories, so devstat makes something up, assuming
* c->name, c->qid, eve, DMDIR|0555.
*
* devdirread assumes (i). the callers have to make sure
* that the Devgen satisfies (i) for the chan being read.
*/
/*
* the zeroth element of the table MUST be the directory itself for ..
*/
int
devgen(Chan *c, char *name, Dirtab *tab, int ntab, int i, DirEntry *dp)
{
    if(tab == 0)
        return -1;
    if(i == DEVDOTDOT){
        /* nothing */
    }else if(name){
        for(i=1; i<ntab; i++)
            if(strcmp(tab[i].name, name) == 0)
                break;
        if(i==ntab)
            return -1;
        tab += i;
    }else{
        /* skip over the first element, that for . itself */
        i++;
        if(i >= ntab)
            return -1;
        tab += i;
    }
    devdir(c, tab->qid, tab->name, tab->length, eve, tab->perm, dp);
    return 1;
}

```

⟨function devclone 376⟩≡ (649)

```

Chan*
devclone(Chan *c)
{
    Chan *nc;

    if(c->flag & COPEN)
        panic("clone of open file type %C\n", devtab[c->type]->dc);

    nc = newchan();

    nc->type = c->type;
    nc->dev = c->dev;
    nc->mode = c->mode;
    nc->qid = c->qid;
    nc->offset = c->offset;
    nc->umh = nil;
    nc->aux = c->aux;
    nc->mqid = c->mqid;
    nc->mcp = c->mcp;
    return nc;
}

```

```
}
```

[devwalk] is the generic walk implementation. Given a channel `c` and path components `name`, it clones `c` into `nc`, then walks each component by calling the `Devgen` callback. On partial success (some but not all names walked), it drops the clone and returns just the Qids reached so far — the 9P layer uses this to decide whether to retry with a different mount point.

```
<function devwalk 377>≡ (649)
Walkqid*
devwalk(Chan *c, Chan *nc, char **name, int nname, Dirtab *tab, int ntab, Devgen *gen)
{
    int i, j;
    bool alloc;
    Walkqid *wq;
    char *n;
    DirEntry dir;

    if(nname > 0)
        error_if_not_dir(c);

    alloc = false;
    wq = smalloc(sizeof(Walkqid)+(nname-1)*sizeof(Qid));
    if(waserror()){
        if(alloc && wq->clone!=nil)
            cclose(wq->clone);
        free(wq);
        return nil;
    }
    if(nc == nil){
        nc = devclone(c);
        nc->type = 0; /* device doesn't know about this channel yet */
        alloc = true;
    }
    wq->clone = nc;

    for(j=0; j<nname; j++){
        if(!(nc->qid.type&QTDIR)){
            if(j==0)
                error(Enotdir);
            goto Done;
        }
        n = name[j];
        if(strcmp(n, ".") == 0){
Accept:
            wq->qid[wq->nqid++] = nc->qid;
            continue;
        }
        if(strcmp(n, "..") == 0){
            if((*gen)(nc, nil, tab, ntab, DEVDOTDOT, &dir) != 1){
                print("devgen walk .. in dev%s %llx broken\n",
                    devtab[nc->type]->name, nc->qid.path);
                error("broken devgen");
            }
            nc->qid = dir.qid;
            goto Accept;
        }
        /*
        * Ugly problem: If we're using devgen, make sure we're
        * walking the directory itself, represented by the first
        * entry in the table, and not trying to step into a sub-
```

```

    * directory of the table, e.g. /net/net. Devgen itself
    * should take care of the problem, but it doesn't have
    * the necessary information (that we're doing a walk).
    */
if(gen==devgen && nc->qid.path!=tab[0].qid.path)
    goto Notfound;
for(i=0;; i++) {
    switch((*gen)(nc, n, tab, ntab, i, &dir)){
    case -1:
        Notfound:
            if(j == 0)
                error(Enonexist);
            kstrcpy(up->errstr, Enonexist, ERRMAX);
            goto Done;
    case 0:
        continue;
    case 1:
        if(strcmp(n, dir.name) == 0){
            nc->qid = dir.qid;
            goto Accept;
        }
        continue;
    }
}
}
/*
 * We processed at least one name, so will return some data.
 * If we didn't process all nname entries succesfully, we drop
 * the cloned channel and return just the Qids of the walks.
 */
Done:
    poperror();
    if(wq->nqid < nname){
        if(alloc)
            cclose(wq->clone);
        wq->clone = nil;
    }else if(wq->clone){
        /* attach cloned channel to same device */
        wq->clone->type = c->type;
    }
    return wq;
}

```

[devstat] iterates the Devgen to find the entry matching *c*'s Qid path, then serializes it with convD2M. If the generator never finds a match and *c* is a directory, it fabricates a minimal entry from *c->qid* and *c->path* — this handles the case where the generator follows rule (i) but [devstat] needs rule (ii).

```

⟨function devstat 378⟩≡ (649)
int
devstat(Chan *c, byte *db, int n, Dirtab *tab, int ntab, Devgen *gen)
{
    int i;
    DirEntry dir;
    char *p, *elem;

    for(i=0;; i++){
        switch((*gen)(c, nil, tab, ntab, i, &dir)){
        case -1:
            if(c->qid.type & QTDIR){
                if(c->path == nil)

```

```

        elem = "???";
    else if(strcmp(c->path->s, "/") == 0)
        elem = "/";
    else
        for(elem=p=c->path->s; *p; p++)
            if(*p == '/')
                elem = p+1;
    devdir(c, c->qid, elem, 0, eve, DMDIR|0555, &dir);
    n = convD2M(&dir, db, n);
    if(n == 0)
        error(Ebadarg);
    return n;
}

    error(Enonexist);
case 0:
    break;
case 1:
    if(c->qid.path == dir.qid.path) {
        if(c->flag&CMMSG)
            dir.mode |= DMMOUNT;
        n = convD2M(&dir, db, n);
        if(n == 0)
            error(Ebadarg);
        return n;
    }
    break;
}
}
return -1; // unreachable
}

```

⟨Dirmode cases 379a⟩≡ (295c) 582f▷
 DMMOUNT = 0x10000000, /* mode bit for mounted channel */

⟨Chan other fields 379b⟩+≡ (51e) <323c ??>
 int dri; /* devdirread index */

[devdirread] is the generic read implementation for directories. It iterates the Devgen starting from c->dri (the saved read index), serializing entries with convD2M until the output buffer is full or the generator returns -1. The dri field in Chan remembers the position across successive read calls, so the user-space dirread library function can consume directory entries incrementally.

⟨function devdirread 379c⟩≡ (649)
 long
 devdirread(Chan *c, char *d, long n, Dirtab *tab, int ntab, Devgen *gen)
 {
 long m, dsz;
 DirEntry dir;

 for(m=0; m<n; c->dri++) {
 switch((*gen)(c, nil, tab, ntab, c->dri, &dir)){
 case -1:
 return m;

 case 0:
 break;

 case 1:
 dsz = convD2M(&dir, (byte*)d, n-m);

```

        if(dsz <= BIT16SZ){ /* <= not < because this isn't stat; read is stuck */
            if(m == 0)
                error(Eshort);
            return m;
        }
        m += dsz;
        d += dsz;
        break;
    }
}

return m;
}

```

⟨function devpermcheck 380a⟩≡ (649)

```

/*
 * error(Eperm) if open permission not granted for up->user.
 */
void
devpermcheck(char *fileuid, ulong perm, int omode)
{
    ulong t;
    static int access[] = { 0400, 0200, 0600, 0100 };

    if(strcmp(up->user, fileuid) == 0)
        perm <<= 0;
    else
        if(strcmp(up->user, eve) == 0)
            perm <<= 3;
        else
            perm <<= 6;

    t = access[omode&3];
    if((t&perm) != t)
        error(Eperm);
}

```

⟨function devopen 380b⟩≡ (649)

```

Chan*
devopen(Chan *c, int omode, Dirtab *tab, int ntab, Devgen *gen)
{
    int i;
    DirEntry dir;

    for(i=0;; i++) {
        switch((*gen)(c, nil, tab, ntab, i, &dir)){
            case -1:
                goto Return;
            case 0:
                break;
            case 1:
                if(c->qid.path == dir.qid.path) {
                    devpermcheck(dir.uid, dir.mode, omode);
                    goto Return;
                }
                break;
        }
    }
}

Return:
    c->offset = 0;

```

```

    if((c->qid.type&QTDIR) && omode!=OREAD)
        error(Eperm);
    c->mode = openmode(omode);
    c->flag |= COPEN;
    return c;
}

```

<Dev methods 381a>+≡ (54a) <304c 562d>
 Block* (*bread)(Chan*, long, ulong);
 long (*bwrite)(Chan*, Block*, ulong);

[devbread] and [devbwrite] are default Dev methods that bridge between the block-oriented interface (used by devmnt for 9P communication) and the byte-oriented read/write methods that most devices implement. They allocate or consume a Block, calling the device's read or write method on its data region.

<function devbread 381b>≡ (649)
 Block*
 devbread(Chan *c, long n, ulong offset)
 {
 Block *bp;

 bp = allocb(n);
 if(bp == 0)
 error(Enomem);
 if(waserror()) {
 freeb(bp);
 nexterror();
 }
 bp->wp += devtab[c->type]->read(c, bp->wp, n, offset);
 poperror();
 return bp;
 }

<function devbrwrite 381c>≡ (649)
 long
 devbwrite(Chan *c, Block *bp, ulong offset)
 {
 long n;

 if(waserror()) {
 freeb(bp);
 nexterror();
 }
 n = devtab[c->type]->write(c, bp->rp, BLEN(bp), offset);
 poperror();
 freeb(bp);

 return n;
 }

17.4 SimpleFS

17.5 Block

A Block is the fundamental I/O buffer in Plan 9, used by queues, pipes, and the networking stack. Data lives between **rp** (read pointer) and **wp** (write pointer), with header space before **rp** that lets protocol layers prepend

headers without copying. Blocks chain via `next` for scatter/gather I/O.

```
<enum blockflag 382a>≡ (645c)
/* flag values */
enum BlockFlags
{
    BINTR = (1<<0),
    BFREE = (1<<1),
    Bipck = (1<<2), /* ip checksum */
    Budpck = (1<<3), /* udp checksum */
    Btcpck = (1<<4), /* tcp checksum */
    Bpktck = (1<<5), /* packet checksum */
};
```

```
<struct Block 382b>≡ (645c)
struct Block
{
    long ref;
    Block* next;
    Block* list;
    byte* rp; /* first unconsumed byte */
    byte* wp; /* first empty byte */
    byte* lim; /* 1 past the end of the buffer */
    byte* base; /* start of the buffer */
    void (*free)(Block*);
    ushort flag;
    ushort checksum; /* IP checksum of complete packet (minus media header) */
};
```

```
<function BLEN 382c>≡ (645c)
#define BLEN(s) ((s)->wp - (s)->rp)
```

```
<function BALLOC 382d>≡ (645c)
#define BALLOC(s) ((s)->lim - (s)->base)
```

```
<constant BLOCKALIGN(arm) 382e>≡ (709b)
// possible alternative fix: git show 2342e706e42d1e6653c51cc1433f44816cf53fe6
#define BLOCKALIGN 32 /* only used in allocb.c */
```

`_allocb` allocates a `Block` and its data buffer in a single `mallocz` call. The data region is aligned to `BLOCKALIGN` (32 bytes for DMA compatibility). The key subtlety is that `rp` starts near the end of the buffer, leaving `Hdrspc` at the front so protocol layers can prepend headers by decrementing `rp` without copying.

```
<function _allocb 382f>≡ (650)
static Block*
_allocb(int size)
{
    Block *b;
    ulong addr;

    if((b = mallocz(sizeof(Block)+size+Hdrspc, false)) == nil)
        return nil;

    b->next = nil;
    b->list = nil;
    b->free = 0;
    b->flag = 0;
    b->ref = 0;
    arch_xinc(&b->ref); // why not incref?? or at least incCNT?

    /* align start of data portion by rounding up */
```

```

addr = (ulong)b;
addr = ROUND(addr + sizeof(Block), BLOCKALIGN);
b->base = (byte*)addr;

/* align end of data portion by rounding down */
b->lim = ((byte*)b) + msize(b);
addr = (ulong)(b->lim);
addr = addr & ~(BLOCKALIGN-1);
b->lim = (byte*)addr;

/* leave sluff at beginning for added headers */
b->rp = b->lim - ROUND(size, BLOCKALIGN);
if(b->rp < b->base)
    panic("_allocb");
b->wp = b->rp;

return b;
}

```

```

⟨function msize 383a⟩≡ (661c)
    ulong
    msize(void *v)
    {
        return poolmsize(mainmem, (ulong*)v-Npadlong)-Npadlong*sizeof(ulong);
    }

```

[allocb] is for process context: it panics if memory is exhausted. [iallocb] below is for interrupt context: it returns nil on failure and tracks memory against a quota (`conf.ialloc`) to prevent interrupt handlers from consuming all memory. Blocks from [iallocb] are flagged BINTR so [freeb] can decrement the interrupt allocation counter.

```

⟨function allocb 383b⟩≡ (650)
    Block*
    allocb(int size)
    {
        Block *b;

        /*
         * Check in a process and wait until successful.
         * Can still error out of here, though.
         */
        if(up == nil)
            panic("allocb without up: %#p", getcallerpc(&size));
        if((b = _allocb(size)) == nil){
            arch_splhi();
            xsummary();
            mallocsummary();
            arch_delay(500);
            panic("allocb: no memory for %d bytes; caller %#p", size,
                getcallerpc(&size));
        }
        setmalloctag(b, getcallerpc(&size));

        return b;
    }
}

```

```

⟨struct Ialloc 383c⟩≡ (650)
    struct Ialloc
    {
        Lock;
    }

```



```

    ulong    bytes;
};

⟨global ialloc 384a⟩≡ (650)
    struct Ialloc ialloc;

⟨function iallocb 384b⟩≡ (650)
    Block*
    iallocb(int size)
    {
        Block *b;
        static int m1, m2, mp;

        if(ialloc.bytes > conf.ialloc){
            if((m1++%10000)==0){
                if(mp++ > 1000){
                    active.exiting = true;
                    arch_exit(0);
                }
                iprint("iallocb: limited %lud/%lud\n",
                    ialloc.bytes, conf.ialloc);
            }
            return nil;
        }

        if((b = _allocb(size)) == nil){
            if((m2++%10000)==0){
                if(mp++ > 1000){
                    active.exiting = true;
                    arch_exit(0);
                }
                iprint("iallocb: no memory %lud/%lud\n",
                    ialloc.bytes, conf.ialloc);
            }
            return nil;
        }
        setmalloctag(b, getcallerpc(&size));
        b->flag = BINTR;

        ilock(&ialloc);
        ialloc.bytes += b->lim - b->base;
        iunlock(&ialloc);

        return b;
    }

```

[freeb] uses atomic reference counting: `arch_xdec` returns the new count, and only the caller who decrements to zero actually frees. Before freeing, it poisons all pointers with `Bdead` — a debugging aid that makes use-after-free crash immediately with a recognizable address rather than silently corrupting data.

```

⟨function freeb 384c⟩≡ (650)
    void
    freeb(Block *b)
    {
        void *dead = (void*)Bdead;
        long ref;

        if(b == nil || (ref = arch_xdec(&b->ref)) > 0)
            return;
    }

```

```

if(ref < 0){
    arch_dumpstack();
    panic("freeb: ref %ld; caller pc %#p", ref, getcallerpc(&b));
}

/*
 * drivers which perform non cache coherent DMA manage their own buffer
 * pool of uncached buffers and provide their own free routine.
 */
if(b->free) {
    b->free(b);
    return;
}
if(b->flag & BINTR) {
    ilock(&ialloc);
    ialloc.bytes -= b->lim - b->base;
    iunlock(&ialloc);
}

/* poison the block in case someone is still holding onto it */
b->next = dead;
b->rp = dead;
b->wp = dead;
b->lim = dead;
b->base = dead;

free(b);
}

⟨function checkb 385a⟩≡ (650)
void
checkb(Block *b, char *msg)
{
    void *dead = (void*)Bdead;

    if(b == dead)
        panic("checkb b %s %#p", msg, b);
    if(b->base == dead || b->lim == dead || b->next == dead
        || b->rp == dead || b->wp == dead){
        print("checkb: base %#p lim %#p next %#p\n",
            b->base, b->lim, b->next);
        print("checkb: rp %#p wp %#p\n", b->rp, b->wp);
        panic("checkb dead: %s", msg);
    }

    if(b->base > b->lim)
        panic("checkb 0 %s %#p %#p", msg, b->base, b->lim);
    if(b->rp < b->base)
        panic("checkb 1 %s %#p %#p", msg, b->base, b->rp);
    if(b->wp < b->base)
        panic("checkb 2 %s %#p %#p", msg, b->base, b->wp);
    if(b->rp > b->lim)
        panic("checkb 3 %s %#p %#p", msg, b->rp, b->lim);
    if(b->wp > b->lim)
        panic("checkb 4 %s %#p %#p", msg, b->wp, b->lim);
}

⟨function freeblist 385b⟩≡ (652b)
/*
 * free a list of blocks

```

```

*/
void
freeblist(Block *b)
{
    Block *next;

    for(; b != 0; b = next){
        next = b->next;
        if(b->ref == 1)
            b->next = nil;
        freeb(b);
    }
}

```

[padblock] ensures there is enough space to prepend (positive **size**) or append (negative **size**) data to a block. If the existing header or trailer space is sufficient, it simply adjusts **rp** or does nothing. Otherwise it allocates a new block, copies the data, and frees the old one. This is the function that makes the “prepend headers without copying” design work in practice — when the initial header space runs out, [padblock] transparently reallocates.

```

⟨function padblock 386⟩≡ (652b)
/*
 * pad a block to the front (or the back if size is negative)
 */
Block*
padblock(Block *bp, int size)
{
    int n;
    Block *nbp;

//    QDEBUG checkb(bp, "padblock 1");
    if(size >= 0){
        if(bp->rp - bp->base >= size){
            bp->rp -= size;
            return bp;
        }

        if(bp->next)
            panic("padblock %#p", getcallerpc(&bp));
        n = BLEN(bp);
        padblockcnt++;
        nbp = allocb(size+n);
        nbp->rp += size;
        nbp->wp = nbp->rp;
        memmove(nbp->wp, bp->rp, n);
        nbp->wp += n;
        freeb(bp);
        nbp->rp -= size;
    } else {
        size = -size;

        if(bp->next)
            panic("padblock %#p", getcallerpc(&bp));

        if(bp->lim - bp->wp >= size)
            return bp;

        n = BLEN(bp);
        padblockcnt++;
        nbp = allocb(size+n);

```

```

        memmove(nbp->wp, bp->rp, n);
        nbp->wp += n;
        freeb(bp);
    }
//    QDEBUG checkb(nbp, "padblock 1");
return nbp;
}

```

<function blocklen 387a>≡ (652b)

```

/*
 * return count of bytes in a string of blocks
 */
int
blocklen(Block *bp)
{
    int len;

    len = 0;
    while(bp) {
        len += BLEN(bp);
        bp = bp->next;
    }
    return len;
}

```

<function blockalloclen 387b>≡ (652b)

```

/*
 * return count of space in blocks
 */
int
blockalloclen(Block *bp)
{
    int len;

    len = 0;
    while(bp) {
        len += BALLOC(bp);
        bp = bp->next;
    }
    return len;
}

```

<function concatblock 387c>≡ (652b)

```

/*
 * copy the string of blocks into
 * a single block and free the string
 */
Block*
concatblock(Block *bp)
{
    int len;
    Block *nb, *f;

    if(bp->next == 0)
        return bp;

    nb = allocb(blocklen(bp));
    for(f = bp; f; f = f->next) {
        len = BLEN(f);
        memmove(nb->wp, f->rp, len);
    }
}

```

```

        nb->wp += len;
    }
    concatblockcnt += BLEN(nb);
    freeblist(bp);
//    QDEBUG checkb(nb, "concatblock 1");
    return nb;
}

```

[pullupblock] consolidates the first *n* bytes of a block chain into a single contiguous block. Protocol processing often needs to examine a fixed-size header (e.g., an IP header) without it being split across blocks.

(*function pullupblock 388*)≡ (652b)

```

/*
 * make sure the first block has at least n bytes
 */
Block*
pullupblock(Block *bp, int n)
{
    int i;
    Block *nbp;

    /*
     * this should almost always be true, it's
     * just to avoid every caller checking.
     */
    if(BLEN(bp) >= n)
        return bp;

    /*
     * if not enough room in the first block,
     * add another to the front of the list.
     */
    if(bp->lim - bp->rp < n){
        nbp = allocb(n);
        nbp->next = bp;
        bp = nbp;
    }

    /*
     * copy bytes from the trailing blocks into the first
     */
    n -= BLEN(bp);
    while(nbp = bp->next){
        i = BLEN(nbp);
        if(i > n) {
            memmove(bp->wp, nbp->rp, n);
            pullupblockcnt++;
            bp->wp += n;
            nbp->rp += n;
//            QDEBUG checkb(bp, "pullupblock 1");
            return bp;
        } else {
            /* shouldn't happen but why crash if it does */
            if(i < 0){
                print("pullup negative length packet, called from %p\n",
                    getcallerpc(&bp));
                i = 0;
            }
            memmove(bp->wp, nbp->rp, i);
            pullupblockcnt++;
            bp->wp += i;

```

```

        bp->next = nbp->next;
        nbp->next = 0;
        freeb(nbp);
        n -= i;
        if(n == 0){
//            QDEBUG checkb(bp, "pullupblock 2");
            return bp;
        }
    }
    freeb(bp);
    return nil;
}

⟨function packblock 389a⟩≡ (652b)
/*
 * if the allocated space is way out of line with the used
 * space, reallocate to a smaller block
 */
Block*
packblock(Block *bp)
{
    Block **l, *nbp;
    int n;

    for(l = &bp; *l; l = &(*l)->next){
        nbp = *l;
        n = BLEN(nbp);
        if((n<<2) < BALLOC(nbp)){
            *l = allocb(n);
            memmove((*l)->wp, nbp->rp, n);
            (*l)->wp += n;
            (*l)->next = nbp->next;
            freeb(nbp);
        }
    }

    return bp;
}

⟨function trimblock 389b⟩≡ (652b)
/*
 * trim to len bytes starting at offset
 */
Block *
trimblock(Block *bp, int offset, int len)
{
    ulong l;
    Block *nb, *startb;

//    QDEBUG checkb(bp, "trimblock 1");
    if(blocklen(bp) < offset+len) {
        freeblist(bp);
        return nil;
    }

    while((l = BLEN(bp)) < offset) {
        offset -= l;
        nb = bp->next;
        bp->next = nil;
    }

```

```

        freeb(bp);
        bp = nb;
    }

    startb = bp;
    bp->rp += offset;

    while((l = BLEN(bp)) < len) {
        len -= l;
        bp = bp->next;
    }

    bp->wp -= (BLEN(bp) - len);

    if(bp->next) {
        freeblist(bp->next);
        bp->next = nil;
    }

    return startb;
}

```

<function copyblock 390a>≡ (652b)

```

/*
 * copy 'count' bytes into a new block
 */
Block*
copyblock(Block *bp, int count)
{
    int l;
    Block *nbp;

    //    QDEBUG checkb(bp, "copyblock 0");
    nbp = allocb(count);
    for(; count > 0 && bp != 0; bp = bp->next){
        l = BLEN(bp);
        if(l > count)
            l = count;
        memmove(nbp->wp, bp->rp, l);
        nbp->wp += l;
        count -= l;
    }
    if(count > 0){
        memset(nbp->wp, 0, count);
        nbp->wp += count;
    }
    copyblockcnt++;
    //    QDEBUG checkb(nbp, "copyblock 1");

    return nbp;
}

```

<function adjustblock 390b>≡ (652b)

```

Block*
adjustblock(Block* bp, int len)
{
    int n;
    Block *nbp;

    if(len < 0){

```

```

        freeb(bp);
        return nil;
    }

    if(bp->rp+len > bp->lim){
        nbp = copyblock(bp, len);
        freeblist(bp);
//        QDEBUG checkb(nbp, "adjustblock 1");

        return nbp;
    }

    n = BLEN(bp);
    if(len > n)
        memset(bp->wp, 0, len-n);
    bp->wp = bp->rp+len;
//    QDEBUG checkb(bp, "adjustblock 2");

    return bp;
}

```

⟨function pullblock 391a⟩≡ (652b)

```

/*
 * throw away up to count bytes from a
 * list of blocks. Return count of bytes
 * thrown away.
 */
int
pullblock(Block **bph, int count)
{
    Block *bp;
    int n, bytes;

    bytes = 0;
    if(bph == nil)
        return 0;

    while(*bph != nil && count != 0) {
        bp = *bph;
        n = BLEN(bp);
        if(count < n)
            n = count;
        bytes += n;
        count -= n;
        bp->rp += n;
//        QDEBUG checkb(bp, "pullblock ");
        if(BLEN(bp) == 0) {
            *bph = bp->next;
            bp->next = nil;
            freeb(bp);
        }
    }
    return bytes;
}

```

⟨function bl2mem 391b⟩≡ (652b)

```

/*
 * copy the contents of a string of blocks into
 * memory. emptied blocks are freed. return
 * pointer to first unconsumed block.

```



```

*/
Block*
bl2mem(byte *p, Block *b, int n)
{
    int i;
    Block *next;

    for(; b != nil; b = next){
        i = BLEN(b);
        if(i > n){
            memmove(p, b->rp, n);
            b->rp += n;
            return b;
        }
        memmove(p, b->rp, i);
        n -= i;
        p += i;
        b->rp += i;
        next = b->next;
        freeb(b);
    }
    return nil;
}

```

17.6 Fileservers

17.7 Fcall

17.8 XTODO

⟨Qidtype *cases* 392⟩+≡

```

QTAUTH = 0x08,    /* type bit for authentication file */
QTAPPEND = 0x40,  /* type bit for append only files */
QTEXCL = 0x20,    /* type bit for exclusive use files */

```

(52c) <375a

Part III

Beyond the Basics

The previous two parts covered the kernel's core: process virtualization and the file abstraction. This part covers the remaining subsystems that build on top of that foundation: inter-process communication (pipes, notes, shared memory), the debugging and profiling infrastructure, special file servers, and advanced topics that crosscut multiple subsystems.

Chapter 18

IPC

Plan 9 provides several IPC mechanisms beyond the filesystem (which is itself an IPC mechanism via `read/write` on shared channels). This chapter covers notes (the Plan 9 equivalent of Unix signals, but string-based), pipes (bidirectional, unlike Unix), shared memory segments, user-level semaphores with kernel backing, and rendezvous (a lightweight synchronization primitive for exchanging values between processes).

```
<sysstab ipc syscalls 394a>≡ (41f)
[NOTIFY]    sysnotify,
[NOTED]     sysnoted,

[PIPE]      syspipe,

[SEGATTACH] syssegattach,
[SEGDETACH] syssegdetach,
[SEGFREE]   syssegfree,
[SEGFLUSH]  syssegflush,
[SEGBRK]    syssegbrk,
```

```
<sysstab concurrency syscalls 394b>≡ (41f)
[RENDEZVOUS] sysrendezvous,

[SEMACQUIRE] syssemacquire,
[SEMRELEASE]  syssemrelease,
[TSEMACQUIRE] systsemacquire,
```

18.1 Notes (signals)

Notes are Plan 9's replacement for Unix signals. Instead of numbered signals (`SIGSEGV`, `SIGPIPE`), notes are arbitrary strings like `"sys: trap: fault read addr=0x0"`. A process registers a handler via `notify(handler)`; when a note arrives, the kernel builds a "note frame" on the user stack containing saved registers and the note string, then redirects execution to the handler. The handler calls `noted(NCONT)` to resume or `noted(NDFLT)` to terminate.

```
<enum notekind 394c>≡ (668c)
enum NoteKind
{
    NUser,        /* note provided externally */
    NExit,        /* deliver note quietly */
    NDebug,       /* print debug message */
};
```

```

⟨enum note 395a⟩≡ (656e)
enum NoteHook {
    NCONT = 0, /* continue after note */
    NDFLT = 1, /* terminate after note */
    NSAVE = 2, /* clear note but hold state */
    NRSTR = 3, /* restore saved state */
};

```

```

⟨global noteidalloc 395b⟩≡ (675c)
// also used by sysrfork()
Counter noteidalloc;

```

Each process has a circular buffer of up to NNOTE (5) pending notes. The `flag` field distinguishes user-posted notes (NUser) from system notes (NExit, NDebug). System notes with no handler cause immediate termination; user notes without a handler also terminate, but a handler can intercept them.

```

⟨struct Note 395c⟩≡ (668c)
// a kind of Unix signal
struct Note
{
    char msg[ERRMAX];
    // enum<NoteKind>
    int flag; /* whether system posted it */
};

```

```

⟨newproc() set fields of p 395d⟩+≡ (108c) <209e 395e>
p->noteid = incref(&noteidalloc);
if(p->noteid==0)
    panic("pidalloc");

```

```

⟨newproc() set fields of p 395e⟩+≡ (108c) <395d 396a>
p->notepending = false;
p->notified = false;

```

```

⟨sysrfork() when RFPROC propagate fields from up to p 395f⟩+≡ (112d) <197c 396b>
memmove(p->note, up->note, sizeof(p->note));
p->nnote = up->nnote;
p->lastnote = up->lastnote;
p->notify = up->notify;

```

```

⟨sleep() if notepending 395g⟩≡ (204)
if(up->notepending) {
    up->notepending = false;
    arch_splx(s);
    ⟨sleep() forceclosegrp 278b⟩
    error(Eintr);
}

```

The note machinery in `Proc` tracks several states: `notify` is the user handler, `notified` prevents recursive delivery, `notepending` wakes sleeping processes, and `ureg` saves registers for `noted()`.

```

⟨Proc notes fields 395h⟩≡ (49f)
Note note[NNOTE];
short nnote;

int (*notify)(void*, char*);
bool_ushort notified; /* sysnoted is due */

ulong noteid; /* Equivalent of note group */

bool notepending; /* note issued but not acted on */

```

```

Note lastnote;

void *ureg;    /* User registers for notes */

⟨newproc() set fields of p 396a⟩+≡ (108c) <395e 453c>
p->ureg = nil;

⟨sysrfork() when RFPROC propagate fields from up to p 396b⟩+≡ (112d) <395f 429d>
p->ureg = up->ureg; // will be adjusted for R0??

⟨sysexec() when hold debug lock, reset some fields 396c⟩≡ (119d) 536i>
up->nnote = 0;
up->notify = nil;
up->notified = false;

⟨constant NNOTE 396d⟩≡ (668c)
NNOTE = 5,

⟨trap() call possibly notify(arm) 396e⟩≡ (146)
if(up->procctl || up->nnote)
    arch__notify(ureg);

```

18.1.1 sysnotify() (arm)

```

⟨syscall notify 396f⟩≡ (679a)
// int notify(void (*f)(void*, char*));
long
sysnotify(ulong* arg)
{
    if(arg[0] != 0)
        validaddr(arg[0], sizeof(ulong), false);
    up->notify = (int (*)(void*, char*))(arg[0]);
    return 0;
}

⟨syscall() call notify(arm) 396g⟩≡ (141e)
arch_splhi();
if(scallnr != RFORK && (up->procctl || up->nnote))
    arch__notify(ureg);

```

The NFrame is the activation record that `arch__notify` builds on the user stack. It contains a copy of the Ureg (so the handler can inspect and modify registers), the note message string, and a pointer to the previous NFrame (old) for nested notes. The `ip` field is set to zero, acting as a return address that will cause a fault if the handler tries to return normally instead of calling `noted()`.

```

⟨struct NFrame(arm) 396h⟩≡ (719c)
struct NFrame {
    uintptr ip;
    Ureg*   arg0;
    char*   arg1;
    char    msg[ERRMAX];
    Ureg*   old;
    Ureg    ureg;
};

```

`arch__notify` is the heart of note delivery. It builds an `NFrame` on the user stack containing the saved `Ureg`, the note message, and the handler address, then overwrites the trap frame's PC and SP so that returning to user mode jumps to the handler. The handler receives (`ureg*`, `msg`) as arguments and calls `noted()` to return control. If the process is already notified, has no handler, or the handler address is invalid, the process is killed.

```

(function arch__notify(arm) 397)≡ (719c)
/*
 * Call user, if necessary, with note.
 * Pass user the Ureg struct and the note on his stack.
 */
int
arch__notify(Ureg* ureg)
{
    int l;
    Note *n;
    u32int s;
    uintptr sp;
    NFrame *nf;

    if(up->procctl)
        procctl(up);
    if(up->nnote == 0)
        return 0;

    fpunotify(ureg);

    s = arch_spllo();
    qlock(&up->debug);

    up->notepending = 0;
    n = &up->note[0];
    if(strncmp(n->msg, "sys:", 4) == 0){
        l = strlen(n->msg);
        if(l > ERRMAX-23) /* " pc=0x0123456789abcdef\0" */
            l = ERRMAX-23;
        snprintf(n->msg + 1, sizeof n->msg - 1, " pc=%#lux", ureg->pc);
    }

    if(n->flag != NUser && (up->notified || up->notify == 0)){
        if(n->flag == NDebug)
            pprint("suicide: %s\n", n->msg);
        qunlock(&up->debug);
        pexit(n->msg, n->flag != NDebug);
    }

    if(up->notified){
        qunlock(&up->debug);
        arch_splhi();
        return 0;
    }

    if(up->notify == nil){
        qunlock(&up->debug);
        pexit(n->msg, n->flag != NDebug);
    }
    if(!okaddr(PTR2UINT(up->notify), 1, 0)){
        pprint("suicide: notify function address %#p\n", up->notify);
        qunlock(&up->debug);
        pexit("Suicide", 0);
    }
}

```

```

sp = ureg->sp - sizeof(NFrame);
if(!okaddr(sp, sizeof(NFrame), 1)){
    qunlock(&up->debug);
    pprint("suicide: notify stack address %#p\n", sp);
    pexit("Suicide", 0);
}

nf = UINT2PTR(sp);
memmove(&nf->ureg, ureg, sizeof(Ureg));
nf->old = up->ureg;
up->ureg = nf;
memmove(nf->msg, up->note[0].msg, ERRMAX);
nf->arg1 = nf->msg;
nf->arg0 = &nf->ureg;
ureg->r0 = PTR2UINT(nf->arg0);
nf->ip = 0;

ureg->sp = sp;
ureg->pc = PTR2UINT(up->notify);
up->notified = 1;
up->nnote--;
memmove(&up->lastnote, &up->note[0], sizeof(Note));
memmove(&up->note[0], &up->note[1], up->nnote*sizeof(Note));

qunlock(&up->debug);
arch_splx(s);

return 1;
}

```

18.1.2 sysnoted() (arm)

<syscall noted 398a>≡ (679a)

```

// int noted(int v);
long
sysnoted(ulong* arg)
{
    if(arg[0] != NRSTR && !up->notified)
        error(Egreg);
    return 0;
}

```

<syscall() call noted() 398b>≡ (141e)

```

if(scallnr == NOTED)
    arch__noted(ureg, *(ulong*)(sp+BY2WD));

```

`arch__noted` restores process state after the note handler finishes. `NCONT`/`NRSTR` restore saved registers and pop the note frame. `NSAVE` re-enters the handler (for coroutine-like handling). `NDFLT` terminates the process. The PSR is masked to prevent user code from escalating to kernel mode by modifying the saved status register.

<function arch__noted(arm) 398c>≡ (719c)

```

/*
 * Return user to state before notify()
 */
static void
arch__noted(Ureg* cur, uintptr arg0)
{
    NFrame *nf;

```

```

Ureg *nur;

qlock(&up->debug);
if(arg0 != NRSTR && !up->notified){
    qunlock(&up->debug);
    pprint("call to noted() when not notified\n");
    pexit("Suicide", 0);
}
up->notified = 0;
fpunoted();

nf = up->ureg;

/* sanity clause */
if(!okaddr(PTR2UINT(nf), sizeof(NFrame), 0)){
    qunlock(&up->debug);
    pprint("bad ureg in noted %#p\n", nf);
    pexit("Suicide", 0);
}

/* don't let user change system flags */
nur = &nf->ureg;
nur->psr &= PsrMask|PsrDfiq|PsrDirq;
nur->psr |= (cur->psr & ~(PsrMask|PsrDfiq|PsrDirq));

memmove(cur, nur, sizeof(Ureg));

switch((int)arg0){
case NCONT:
case NRSTR:
    if(!okaddr(nur->pc, BY2WD, 0) || !okaddr(nur->sp, BY2WD, 0)){
        qunlock(&up->debug);
        pprint("suicide: trap in noted\n");
        pexit("Suicide", 0);
    }
    up->ureg = nf->old;
    qunlock(&up->debug);
    break;
case NSAVE:
    if(!okaddr(nur->pc, BY2WD, 0) || !okaddr(nur->sp, BY2WD, 0)){
        qunlock(&up->debug);
        pprint("suicide: trap in noted\n");
        pexit("Suicide", 0);
    }
    qunlock(&up->debug);

    arch_splhi();
    nf->arg1 = nf->msg;
    nf->arg0 = &nf->ureg;
    nf->ip = 0;
    cur->sp = PTR2UINT(nf);
    cur->r0 = PTR2UINT(nf->arg0);
    break;
default:
    pprint("unknown noted arg %#p\n", arg0);
    up->lastnote.flag = NDebug;
    /*FALLTHROUGH*/
case NDFLT:
    if(up->lastnote.flag == NDebug){
        qunlock(&up->debug);
    }
}

```



```

        pprint("suicide: %s\n", up->lastnote.msg);
    }
    else
        qunlock(&up->debug);
    pexit(up->lastnote.msg, up->lastnote.flag != NDebug);
}
}

```

18.1.3 postnote()

[postnote] is one of the trickiest functions in the kernel due to lock ordering. To wake a sleeping process, it needs both `p->rlock` and the Rendez lock, but [sleep] acquires them in the opposite order. The solution is a retry loop: acquire `p->rlock`, try `canlock(r)`, and if that fails, drop `p->rlock`, call `sched()`, and retry. The second half handles rendezvous: if the target is blocked in [sysrendezvous], it gets pulled out of the hash table and made ready.

```

⟨function postnote 400⟩≡ (675c)
/*
 * if waking a sleeping process, this routine must hold both
 * p->rlock and r->lock. However, it can't know them in
 * the same order as wakeup causing a possible lock ordering
 * deadlock. We break the deadlock by giving up the p->rlock
 * lock if we can't get the r->lock and retrying.
 */
int
proc_postnote(Proc *p, int dolock, char *n, int flag)
{
    int s, ret;
    Rendez *r;
    Proc *d, **l;

    if(dolock)
        qlock(&p->debug);

    if(flag != NUser && (p->notify == 0 || p->notified))
        p->nnote = 0;

    ret = 0;
    if(p->nnote < NNOTE) {
        strcpy(p->note[p->nnote].msg, n);
        p->note[p->nnote++].flag = flag;
        ret = 1;
    }
    p->notepending = true;
    if(dolock)
        qunlock(&p->debug);

    /* this loop is to avoid lock ordering problems. */
    for(;;){
        s = arch_splhi();
        lock(&p->rlock);
        r = p->r;

        /* waiting for a wakeup? */
        if(r == nil)
            break; /* no */

        /* try for the second lock */

```

```

    if(canlock(r)){
        if(p->state != Wakeme || r->p != p)
            panic("postnote: state %d %d %d", r->p != p, p->r != r, p->state);
        p->r = nil;
        r->p = nil;
        ready(p);
        unlock(r);
        break;
    }

    /* give other process time to get out of critical section and try again */
    unlock(&p->rlock);
    arch_splx(s);
    sched();
}
unlock(&p->rlock);
arch_splx(s);

if(p->state != Rendezvous)
    return ret;

/* Try and pull out of a rendezvous */
lock(p->rgrp);
if(p->state == Rendezvous) {
    p->rendval = ~0;
    l = &REND(p->rgrp, p->rendtag);
    for(d = *l; d; d = d->rendhash) {
        if(d == p) {
            *l = p->rendhash;
            break;
        }
        l = &d->rendhash;
    }
    ready(p);
}
unlock(p->rgrp);
return ret;
}

```

18.2 Pipes

18.2.1 syspipe()

Plan 9 pipes are bidirectional: both ends can read and write. A pipe is a pair of Queues: writing to `fd[0]` enqueues on `q[1]` (which `fd[1]` reads from), and vice versa. A single pipe suffices for two-way communication, whereas Unix requires two. `[syspipe]` creates a pipe by attaching to the pipe device (`#l`), cloning the channel, then walking each clone to `data` and `data1`.

```

⟨syscall pipe 401⟩≡ (654)
// int pipe(int fd[2]);
long
syspipe(ulong* arg)
{
    int fd[2];
    Chan *c[2];
    Dev *d;
    static char *datastr[] = {"data", "data1"};

```

```

validaddr(arg[0], 2*BY2WD, true);
arch_validalign(arg[0], sizeof(int));
d = devtab[devno('|', false)];
c[0] = namec("#|", Atodir, 0, 0);
c[1] = nil;
fd[0] = -1;
fd[1] = -1;

if(waserror()){
    cclose(c[0]);
    if(c[1])
        cclose(c[1]);
    nexterror();
}
c[1] = cclone(c[0]);
if(walk(&c[0], datastr+0, 1, 1, nil) < 0)
    error(Egreg);
if(walk(&c[1], datastr+1, 1, 1, nil) < 0)
    error(Egreg);
c[0] = d->open(c[0], ORDWR);
c[1] = d->open(c[1], ORDWR);
if(newfd2(fd, c) < 0)
    error(Enofd);
poperror();

((long*)arg[0])[0] = fd[0];
((long*)arg[0])[1] = fd[1];
return 0;
}

```

<function newfd2 402a>≡ (654)

```

int
newfd2(int fd[2], Chan *c[2])
{
    Fgrp *f;

    f = up->fgrp;
    lock(f);
    fd[0] = findfreefd(f, 0);
    if(fd[0] < 0){
        unlockfgrp(f);
        return -1;
    }
    fd[1] = findfreefd(f, fd[0]+1);
    if(fd[1] < 0){
        unlockfgrp(f);
        return -1;
    }
    if(fd[1] > f->maxfd)
        f->maxfd = fd[1];
    f->fd[fd[0]] = c[0];
    f->fd[fd[1]] = c[1];
    unlockfgrp(f);

    return 0;
}

```

<sseek() and pipes 402b>≡ (277b)

```

if(devtab[c->type]->dc == '|')
    error(Eisstream);

```

18.2.2 /dev/pipe

The Pipe structure holds two Queues and reference counts. The `qref` array tracks how many open channels reference each side; when a side's reference count drops to zero, the opposite queue is hung up (sending EOF to readers) and the local queue is closed. This cross-wiring is key to bidirectional pipes: closing one end signals the other.

<struct Pipe 403a>≡ (642)

```
struct Pipe
{
    int ref;
    ulong path;
    long perm;
    Queue *q[2];
    int qref[2];

    // extra
    QLock;
    Pipe *next;
};
```

<struct Pipealloc 403b>≡ (642)

```
struct Pipealloc
{
    ulong path;

    // extra
    Lock;
};
```

<global pipealloc 403c>≡ (642)

```
struct Pipealloc pipealloc;
```

<devpipe.c enum Qxxx 403d>≡ (642)

```
enum
{
    Qdir,
    Qdata0,
    Qdata1,
};
```

<global pipedir 403e>≡ (642)

```
Dirtab pipedir[] =
{
    ".",      {Qdir,0,QTDIR}, 0,      DMDIR|0500,
    "data",   {Qdata0}, 0,      0600,
    "data1",  {Qdata1}, 0,      0600,
};
```

<Conf other fields 403f>+≡ (46a) <241d

```
ulong pipeqsize; /* size in bytes of pipe queues */
```

<method pipeinit 403g>≡ (642)

```
static void
pipeinit(void)
{
    if(conf.pipeqsize == 0){
        if(conf.ncpu > 1)
            conf.pipeqsize = 256*1024;
        else
            conf.pipeqsize = 32*1024;
    }
}
```

⟨Chan union other fields 404a⟩≡ (51e) 539c▷

```
/*Pipe*/void* chanpipe; // for pipes
```

⟨method pipeattach 404b⟩≡ (642)

```
/*
 * create a pipe, no streams are created until an open
 */
static Chan*
pipeattach(char *spec)
{
    Pipe *p;
    Chan *c;

    c = devattach('|', spec);
    p = malloc(sizeof(Pipe));
    if(p == 0)
        exhausted("memory");
    p->ref = 1;

    p->q[0] = qopen(conf.pipeqsize, 0, 0, 0);
    if(p->q[0] == 0){
        free(p);
        exhausted("memory");
    }
    p->q[1] = qopen(conf.pipeqsize, 0, 0, 0);
    if(p->q[1] == 0){
        free(p->q[0]);
        free(p);
        exhausted("memory");
    }

    lock(&pipealloc);
    p->path = ++pipealloc.path;
    unlock(&pipealloc);
    p->perm = pipedir[Qdata0].perm;

    mkqid(&c->qid, NETQID(2*p->path, Qdir), 0, QTDIR);
    c->aux = p;
    c->dev = 0;
    return c;
}
```

⟨function pipegen 404c⟩≡ (642)

```
static int
pipegen(Chan *c, char*, Dirtab *tab, int ntab, int i, DirEntry *dp)
{
    Qid q;
    int len;
    Pipe *p;

    if(i == DEVDOTDOT){
        devdir(c, c->qid, "#|", 0, eve, DMDIR|0555, dp);
        return 1;
    }
    i++; /* skip . */
    if(tab==0 || i>=ntab)
        return -1;

    tab += i;
    p = c->aux;
```

```

switch((ulong)tab->qid.path){
case Qdata0:
    len = qlen(p->q[0]);
    break;
case Qdata1:
    len = qlen(p->q[1]);
    break;
default:
    len = tab->length;
    break;
}
mkqid(&q, NETQID(NETID(c->qid.path), tab->qid.path), 0, QTFILE);
devdir(c, q, tab->name, len, eve, p->perm, dp);
return 1;
}

```

⟨method *pipewalk* 405a⟩≡ (642)

```

static Walkqid*
pipewalk(Chan *c, Chan *nc, char **name, int nname)
{
    Walkqid *wq;
    Pipe *p;

    wq = devwalk(c, nc, name, nname, pipedir, NPIPEDIR, pipegen);
    if(wq != nil && wq->clone != nil && wq->clone != c){
        p = c->aux;
        qlock(p);
        p->ref++;
        if(c->flag & COPEN){
            print("channel open in pipewalk\n");
            switch(NETTYPE(c->qid.path)){
            case Qdata0:
                p->qref[0]++;
                break;
            case Qdata1:
                p->qref[1]++;
                break;
            }
        }
        qunlock(p);
    }
    return wq;
}

```

⟨method *pipestat* 405b⟩≡ (642)

```

static int
pipestat(Chan *c, byte *db, int n)
{
    Pipe *p;
    DirEntry dir;

    p = c->aux;

    switch(NETTYPE(c->qid.path)){
    case Qdir:
        devdir(c, c->qid, ".", 0, eve, DMDIR|0555, &dir);
        break;
    case Qdata0:
        devdir(c, c->qid, "data", qlen(p->q[0]), eve, p->perm, &dir);
        break;
    }
}

```

```

    case Qdata1:
        devdir(c, c->qid, "data1", qlen(p->q[1]), eve, p->perm, &dir);
        break;
    default:
        panic("pipestat");
}
n = convD2M(&dir, db, n);
if(n < BIT16SZ)
    error(Eshortstat);
return n;
}

```

<method pipewstat 406a>≡ (642)

```

static int
pipewstat(Chan* c, byte* db, int n)
{
    int m;
    DirEntry *dir;
    Pipe *p;

    p = c->aux;
    if(strcmp(up->user, eve) != 0)
        error(Eperm);
    if(NETTYPE(c->qid.path) == Qdir)
        error(Eisdir);

    dir = smalloc(sizeof(DirEntry)+n);
    if(waserror()){
        free(dir);
        nexterror();
    }
    m = convM2D(db, n, &dir[0], (char*)&dir[1]);
    if(m == 0)
        error(Eshortstat);
    if(!emptystr(dir[0].uid))
        error("can't change owner");
    if(dir[0].mode != ~0UL)
        p->perm = dir[0].mode;
    poperror();
    free(dir);
    return m;
}

```

<method pipeopen 406b>≡ (642)

```

/*
 * if the stream doesn't exist, create it
 */
static Chan*
pipeopen(Chan *c, int omode)
{
    Pipe *p;

    if(c->qid.type & QTDIR){
        if(omode != OREAD)
            error(Ebadarg);
        c->mode = omode;
        c->flag |= COPEN;
        c->offset = 0;
        return c;
    }
}

```

```

p = c->aux;
qlock(p);
switch(NETTYPE(c->qid.path)){
case Qdata0:
    p->qref[0]++;
    break;
case Qdata1:
    p->qref[1]++;
    break;
}
qunlock(p);

c->mode = openmode(omode);
c->flag |= COPEN;
c->offset = 0;
c->iounit = qiomaxatomic;
return c;
}

```

[pipeclose] implements the cross-hangup semantics: when the last reference to **data** closes, it hangs up **q[1]** (sending EOF to readers of **data1**) and closes **q[0]**. When both sides reach zero references, the queues are reopened for potential reuse.

<method pipeclose 407> (642)

```

static void
pipeclose(Chan *c)
{
    Pipe *p;

    p = c->aux;
    qlock(p);

    if(c->flag & COPEN){
        /*
         * closing either side hangs up the stream
         */
        switch(NETTYPE(c->qid.path)){
        case Qdata0:
            p->qref[0]--;
            if(p->qref[0] == 0){
                qhangup(p->q[1], 0);
                qclose(p->q[0]);
            }
            break;
        case Qdata1:
            p->qref[1]--;
            if(p->qref[1] == 0){
                qhangup(p->q[0], 0);
                qclose(p->q[1]);
            }
            break;
        }
    }

    /*
     * if both sides are closed, they are reusable
     */
    if(p->qref[0] == 0 && p->qref[1] == 0){

```



```

        qreopen(p->q[0]);
        qreopen(p->q[1]);
    }

    /*
     * free the structure on last close
     */
    p->ref--;
    if(p->ref == 0){
        qunlock(p);
        free(p->q[0]);
        free(p->q[1]);
        free(p);
    } else
        qunlock(p);
}

⟨method piperead 408a⟩≡ (642)
static long
piperead(Chan *c, void *va, long n, vlong)
{
    Pipe *p;

    p = c->aux;

    switch(NETTYPE(c->qid.path)){
    case Qdir:
        return devdirread(c, va, n, pipedir, NPIPEDIR, pipegen);
    case Qdata0:
        return qread(p->q[0], va, n);
    case Qdata1:
        return qread(p->q[1], va, n);
    default:
        panic("piperead");
    }
    panic("piperead: should not reach this point");
    return -1; // unreachable
}

⟨method pipebread 408b⟩≡ (642)
static Block*
pipebread(Chan *c, long n, ulong offset)
{
    Pipe *p;

    p = c->aux;

    switch(NETTYPE(c->qid.path)){
    case Qdata0:
        return qbread(p->q[0], n);
    case Qdata1:
        return qbread(p->q[1], n);
    }

    return devbread(c, n, offset);
}

```

Note the cross-wiring in [pipewrite]: writing to Qdata0 enqueues on q[1], and writing to Qdata1 enqueues on q[0]. If the write fails (the other end is closed), a "sys: write on closed pipe" note is posted — the

Plan 9 equivalent of Unix's SIGPIPE.

<method pipewrite 409a>≡ (642)

```
/*
 * a write to a closed pipe causes a note to be sent to
 * the process.
 */
static long
pipewrite(Chan *c, void *va, long n, vlong)
{
    Pipe *p;

    if(!arch_islo())
        print("pipewrite hi %#p\n", getcallerpc(&c));
    if(waserror()) {
        /* avoid notes when pipe is a mounted queue */
        if((c->flag & CMSG) == 0)
            postnote(up, 1, "sys: write on closed pipe", NUser);
        nexterror();
    }

    p = c->aux;

    switch(NETTYPE(c->qid.path)){
    case Qdata0:
        n = qwrite(p->q[1], va, n);
        break;

    case Qdata1:
        n = qwrite(p->q[0], va, n);
        break;

    default:
        panic("pipewrite");
    }

    poperror();
    return n;
}
```

<method pipebwrite 409b>≡ (642)

```
static long
pipebwrite(Chan *c, Block *bp, ulong)
{
    long n;
    Pipe *p;

    if(waserror()) {
        /* avoid notes when pipe is a mounted queue */
        if((c->flag & CMSG) == 0)
            postnote(up, 1, "sys: write on closed pipe", NUser);
        nexterror();
    }

    p = c->aux;
    switch(NETTYPE(c->qid.path)){
    case Qdata0:
        n = qbwrite(p->q[1], bp);
        break;

    case Qdata1:
```

```

        n = qbwrite(p->q[0], bp);
        break;

default:
    n = 0;
    panic("pipebwrite");
}

poperror();
return n;
}

⟨global pipedevtab 410a⟩≡ (642)
Dev pipedevtab = {
    .dc      =    '|',
    .name     =    "pipe",

    .reset    =    devreset,
    .init     =    pipeinit,
    .shutdown =    devshutdown,
    .attach   =    pipeattach,
    .walk     =    pipewalk,
    .stat     =    pipestat,
    .open     =    pipeopen,
    .create   =    devcreate,
    .close    =    pipeclose,
    .read     =    piperead,
    .bread    =    pipebread,
    .write    =    pipewrite,
    .bwrite   =    pipebwrite,
    .remove   =    devremove,
    .wstat    =    pipewstat,
};

```

18.3 Shared segment and semaphore

Shared memory segments are created via `segattach()`, which maps a named physical memory region (`Physseg`) into the process's address space. Multiple processes can attach the same segment to share memory. The semaphore syscalls (`semacquire`, `semrelease`) operate on addresses within shared segments, providing the kernel-side backing for user-level locks.

18.3.1 Segment

```

⟨Segtype other flags 410b⟩+≡ (92a) <99a
    SG_CEXEC = 0100,    /* Detach at exec */

⟨sysexec() free special segments if close on exec 410c⟩≡ (121d)
    for(i = BSEG+1; i < NSEG; i++) {
        s = up->seg[i];
        if(s != nil && (s->type&SG_CEXEC)) { // close on exec
            putseg(s);
            up->seg[i] = nil;
        }
    }
}

```

<syscall segbrk 411a>≡ (679a)

```
// void* segbrk(void *saddr, void *addr);
long
syssegbrk(ulong* arg)
{
    int i;
    ulong addr;
    Segment *s;

    addr = arg[0];
    for(i = 0; i < NSEG; i++) {
        s = up->seg[i];
        if(s == nil || addr < s->base || addr >= s->top)
            continue;
        switch(s->type&SG_TYPE) {
        case SG_TEXT:
        case SG_DATA:
        case SG_STACK:
            error(Ebadarg);
        default:
            return ibrk(arg[1], i);
        }
    }

    error(Ebadarg);
    panic("syssegbrk: should not reach this point");
    return -1; // unreachable
}
```

<syscall segattach 411b>≡ (679a)

```
// void* segattach(int attr, char *class, void *va, ulong len);
long
syssegattach(ulong* arg)
{
    return segattach(up, arg[0], (char*)arg[1], arg[2], arg[3]);
}
```

<hook _globalsegattach 411c>≡ (665)

```
Segment* (*_globalsegattach)(Proc*, char*);
```

[segattach] first tries to find a globally registered segment with the matching name (via the `_globalsegattach` hook). If none exists, it searches the `physseg` table for a matching `Physseg`, finds a free slot in the process's segment array, and uses [isoverlap] to locate a non-conflicting virtual address range (growing downward from the stack). This is how device drivers expose memory-mapped hardware to user space.

<function segattach 411d>≡ (665)

```
ulong
segattach(Proc *p, ulong attr, char *name, ulong va, ulong len)
{
    int sno;
    Segment *s, *os;
    Physseg *ps;

    if(va != 0 && va >= USTKTOP)
        error(Ebadarg);

    validaddr((ulong)name, 1, false);
    vmemchr(name, 0, ~0);

    for(sno = 0; sno < NSEG; sno++)
```

```

        if(p->seg[sno] == nil && sno != ESEG)
            break;

if(sno == NSEG)
    error(Enovmem);

/*
 * first look for a global segment with the
 * same name
 */
if(_globalsegattach != nil){
    s = (*_globalsegattach)(p, name);
    if(s != nil){
        p->seg[sno] = s;
        return s->base;
    }
}

len = PGROUND(len);
if(len == 0)
    error(Ebadarg);

/*
 * Find a hole in the address space.
 * Starting at the lowest possible stack address - len,
 * check for an overlapping segment, and repeat at the
 * base of that segment - len until either a hole is found
 * or the address space is exhausted. Ensure that we don't
 * map the zero page.
 */
if(va == 0) {
    for (os = p->seg[SSEG]; os != nil; os = isoverlap(p, va, len)) {
        va = os->base;
        if(len >= va)
            error(Enovmem);
        va -= len;
    }
    va &= ~(BY2PG-1);
} else {
    va &= ~(BY2PG-1);
    if(va == 0 || va >= USTKTOP)
        error(Ebadarg);
}

if(isoverlap(p, va, len) != nil)
    error(Esoverlap);

for(ps = physseg; ps->name; ps++)
    if(strcmp(name, ps->name) == 0)
        goto found;

error(Ebadarg);
found:
if(len > ps->size)
    error(Enovmem);

attr &= ~SG_TYPE;        /* Turn off what is not allowed */
attr |= ps->attr;         /* Copy in defaults */

```

```

    s = newseg(attr, va, len/BY2PG);
    s->pseg = ps;
    p->seg[sno] = s;

    return va;
}

```

<function isoverlap 413a>≡ (665)

```

Segment*
isoverlap(Proc *p, ulong va, int len)
{
    int i;
    Segment *ns;
    ulong newtop;

    newtop = va+len;
    for(i = 0; i < NSEG; i++) {
        ns = p->seg[i];
        if(ns == 0)
            continue;
        if((newtop > ns->base && newtop <= ns->top) ||
            (va >= ns->base && va < ns->top))
            return ns;
    }
    return nil;
}

```

<syscall segdetach 413b>≡ (679a)

```

// int segdetach(void *addr);
long
syssegdetach(ulong* arg)
{
    int i;
    ulong addr;
    Segment *s;

    qlock(&up->seglock);
    if(waserror()){
        qunlock(&up->seglock);
        nexterror();
    }

    s = 0;
    addr = arg[0];
    for(i = 0; i < NSEG; i++)
        if(s = up->seg[i]) {
            qlock(&s->lk);
            if((addr >= s->base && addr < s->top) ||
                (s->top == s->base && addr == s->base))
                goto found;
            qunlock(&s->lk);
        }

    error(Ebadarg);

found:
    /*
     * Check we are not detaching the initial stack segment.
     */
    if(s == up->seg[SSEG]){

```

```

        qunlock(&s->lk);
        error(Ebadarg);
    }
    up->seg[i] = 0;
    qunlock(&s->lk);
    putseg(s);
    qunlock(&up->seglock);
    poperror();

    /* Ensure we flush any entries from the lost segment */
    arch_flushmmu();
    return 0;
}

```

<syscall segfree 414a> ≡ (679a)

```

// int segfree(void *va, ulong len);
long
syssegfree(ulong* arg)
{
    Segment *s;
    ulong from, to;

    from = arg[0];
    s = seg(up, from, 1);
    if(s == nil)
        error(Ebadarg);
    to = (from + arg[1]) & ~(BY2PG-1);
    from = PGROUND(from);

    if(to > s->top) {
        qunlock(&s->lk);
        error(Ebadarg);
    }

    mfreeseg(s, from, (to - from) / BY2PG);
    qunlock(&s->lk);
    arch_flushmmu();

    return 0;
}

```

<dupseg() if original segment was shared, flush it 414b> ≡ (101b)

```

if(s->ref > 1)
    procflushseg(s); // ??

```

<mfreeseg() if segment was shared, flush it 414c> ≡ (104b)

```

/* flush this seg in all other processes */
if(s->ref > 1)
    procflushseg(s);

```

When a shared segment is modified (e.g., pages freed via `mfreeseg`), all processes sharing it must flush their TLB entries. `[procflushseg]` iterates over all processes and CPUs, setting `newtlb` and `flushmmu` flags. It then busy-waits for each CPU to take a clock interrupt and perform the flush — an expensive but necessary operation for multiprocessor correctness.

<function procflushseg 414d> ≡ (675c)

```

/*
 * wait till all processes have flushed their mmu
 * state about segment s
 */

```

```

void
procflushseg(Segment *s)
{
    int i, ns, nm;
    bool wait;
    Proc *p;

    /*
     * tell all processes with this
     * segment to flush their mmu's
     */
    wait = false;
    for(i=0; i<conf.nproc; i++) {
        p = &procalloc.arena[i];
        if(p->state == Dead)
            continue;
        for(ns = 0; ns < NSEG; ns++)
            if(p->seg[ns] == s){
                p->newtlb = true;
                for(nm = 0; nm < conf.ncpu; nm++){
                    if(CPUS(nm)->proc == p){
                        CPUS(nm)->flushmmu = true;
                        wait = true;
                    }
                }
                break;
            }
    }
    if(!wait)
        return;

    /*
     * wait for all processors to take a clock interrupt
     * and flush their mmu's
     */
    for(nm = 0; nm < conf.ncpu; nm++)
        if(CPUS(nm) != cpu)
            while(CPUS(nm)->flushmmu)
                sched();
}

⟨hzclock() if flushmmu 415a⟩≡ (207a)
    if(cpu->flushmmu){
        if(up)
            arch_flushmmu();
        cpu->flushmmu = false;
    }

⟨Cpu other fields 415b⟩+≡ (48a) <206c 448a>
    bool flushmmu; /* make current proc flush it's mmu state */

⟨syscall segflush 415c⟩≡ (667a)
    long
    syssegflush(ulong* arg)
    {
        Segment *s;
        ulong addr, l;
        Pagetable *pt;
        int chunk, ps, pe, len;

```



```

addr = arg[0];
len = arg[1];

while(len > 0) {
    s = seg(up, addr, 1);
    if(s == 0)
        error(Ebadarg);

    <syssegflush() set flushme 494e>
more:
    l = len;
    if(addr+l > s->top)
        l = s->top - addr;

    ps = addr-s->base;
    pt = s->pagedir[ps/PAGETABMAPMEM];
    ps &= PAGETABMAPMEM-1;
    pe = PAGETABMAPMEM;
    if(pe-ps > 1){
        pe = ps + 1;
        pe = ROUND(pe, BY2PG);
    }
    if(pe == ps) {
        qunlock(&s->lk);
        error(Ebadarg);
    }

    // when have cachectl
    if(pt)
        ptflush(pt, ps/BY2PG, pe/BY2PG);

    chunk = pe-ps;
    len -= chunk;
    addr += chunk;

    if(len > 0 && addr < s->top)
        goto more;

    qunlock(&s->lk);
}
arch_flushmmu();
return 0;
}

```

18.3.2 Semaphore

Plan 9's semaphores are a hybrid user/kernel design. The semaphore value lives in user memory (shared segment), and the fast path uses compare-and-swap entirely in user space. Only when a process must block does it enter the kernel via `semacquire()`, which enqueues a `Sema` on the segment's wait list and calls `[sleep]`. This gives spin-lock performance in the uncontended case with proper blocking when needed.

```

<struct Sema 416>≡ (604)
// user level semaphores, used to implement user-level lock,
// see libc/port/lock.c
struct Sema
{
    long *addr; // value stored in user space!
    bool waiting;

```

```

    //list<Sema> of Segment.sema
    Sema *next;
    Sema *prev;

    Rendez;
};

⟨Segment other fields 417a⟩+≡ (92c) <170b 430d>
    Sema sema;

⟨newseg() sema initialization 417b⟩≡ (95)
    // no list, just one sema
    s->sema.prev = &s->sema;
    s->sema.next = &s->sema;

cmpswap() (arm)

⟨function arch_cmpswap(raspberry pi1)(arm) 417c⟩≡ (694a)
    bool
    arch_cmpswap(long *addr, long old, long new)
    {
        return cas32(addr, old, new);
    }

⟨function cas32(arm) 417d⟩≡ (694a)
    bool
    cas32(void* addr, u32int old, u32int new)
    {
        bool r;
        int s;

        s = arch_splhi();
        r = (*(u32int*)addr == old);
        if(r)
            *(u32int*)addr = new;
        arch_splx(s);
        ⟨cas32() if address contained old value, call coherence(arm) 473d⟩
        return r;
    }

semxxx()

⟨function semqueue 417e⟩≡ (679b)
    /* Add semaphore p with addr a to list in seg. */
    static void
    semqueue(Segment *s, long *a, Sema *p)
    {
        memset(p, 0, sizeof *p);
        p->addr = a;
        lock(&s->sema); /* uses s->sema.Rendez.Lock, but no one else is */
        p->next = &s->sema;
        p->prev = s->sema.prev;
        p->next->prev = p;
        p->prev->next = p;
        unlock(&s->sema);
    }

```

<function semdequeue 418a>≡ (679b)

```
/* Remove semaphore p from list in seg. */
static void
semdequeue(Segment *s, Sema *p)
{
    lock(&s->sema);
    p->next->prev = p->prev;
    p->prev->next = p->next;
    unlock(&s->sema);
}
```

<function semwakeup 418b>≡ (679b)

```
/* Wake up n waiters with addr a on list in seg. */
static void
semwakeup(Segment *s, long *a, long n)
{
    Sema *p;

    lock(&s->sema);
    for(p=s->sema.next; p!=&s->sema && n>0; p=p->next){
        if(p->addr == a && p->waiting){
            p->waiting = false;
            arch_coherence();
            wakeup(p);
            n--;
        }
    }
    unlock(&s->sema);
}
```

<function semrelease 418c>≡ (679b)

```
/* Add delta to semaphore and wake up waiters as appropriate. */
static long
semrelease(Segment *s, long *addr, long delta)
{
    long value;

    do
        value = *addr;
    while(!arch_cmpswap(addr, value, value+delta));
    semwakeup(s, addr, delta);
    return value+delta;
}
```

<function canacquire 418d>≡ (679b)

```
/* Try to acquire semaphore using compare-and-swap */
static bool
canacquire(long *addr)
{
    long value;

    while((value=*addr) > 0)
        if(arch_cmpswap(addr, value, value-1))
            return true;
    return false;
}
```

<function semawoke 418e>≡ (679b)

```
/* Should we wake up? */
static bool
```

```

semawoke(void *p)
{
    arch_coherence();
    return !((Sema*)p)->waiting;
}

```

The [semacquire] loop is subtle: after enqueueing itself on the wait list, the process sets `phore.waiting = true`, calls `coherence()` to ensure the write is visible, then tries `canacquire` one more time. This recheck prevents a lost-wakeup race where `semrelease` could run between the failed `canacquire` and the [sleep]. If a note interrupts the sleep, the function cleans up and re-raises the error via [nexterror].

```

⟨function semacquire 419a⟩≡ (679b)
/* Acquire semaphore (subtract 1). */
static bool
semacquire(Segment *s, long *addr, bool block)
{
    int acquired;
    Sema phore;

    if(canacquire(addr))
        return true;
    if(!block)
        return false;

    acquired = false;
    semqueue(s, addr, &phore);
    for(;;){
        phore.waiting = true;
        arch_coherence();
        if(canacquire(addr)){
            acquired = true;
            break;
        }
        if(waserror())
            break;
        sleep(&phore, semawoke, &phore);
        poperror();
    }
    semdequeue(s, &phore);
    arch_coherence(); /* not strictly necessary due to lock in semdequeue */
    if(!phore.waiting)
        semwakeup(s, addr, 1);
    if(!acquired)
        nexterror();
    return false;
}

```

```

⟨function tsemacquire 419b⟩≡ (679b)
/* Acquire semaphore or time-out */
static bool
tsemacquire(Segment *s, long *addr, ulong ms)
{
    int acquired, timedout;
    ulong t, elms;
    Sema phore;

    if(canacquire(addr))
        return true;
    if(ms == 0)
        return false;

```

```

acquired = timedout = false;
semqueue(s, addr, &phore);
for(;;){
    phore.waiting = true;
    arch_coherence();
    if(canacquire(addr)){
        acquired = true;
        break;
    }
    if(waserror())
        break;
    t = cpu->ticks;
    tsleep(&phore, semawoke, &phore, ms);
    elms = TK2MS(cpu->ticks - t);
    poperror();
    if(elms >= ms){
        timedout = true;
        break;
    }
    ms -= elms;
}
semdequeue(s, &phore);
arch_coherence(); /* not strictly necessary due to lock in semdequeue */
if(!phore.waiting)
    semwakeup(s, addr, 1);
if(timedout)
    return false;
if(!acquired)
    nexterror();
return false;
}

```

<syscall semacquire 420a>≡ (679b)

```

// int semacquire(long *addr, int block);
long
syssemacquire(ulong* arg)
{
    int block;
    long *addr;
    Segment *s;

    validaddr(arg[0], sizeof(long), true);
    arch_validalign(arg[0], sizeof(long));
    addr = (long*)arg[0];
    block = arg[1];

    if((s = seg(up, (ulong)addr, 0)) == nil)
        error(Ebadarg);
    if(*addr < 0)
        error(Ebadarg);
    return semacquire(s, addr, block);
}

```

<syscall tsemacquire 420b>≡ (679b)

```

// int tsemacquire(long *addr, ulong ms);
long
systsemacquire(ulong* arg)
{
    long *addr;
    ulong ms;

```

```

Segment *s;

validaddr(arg[0], sizeof(long), true);
arch_validalign(arg[0], sizeof(long));
addr = (long*)arg[0];
ms = arg[1];

if((s = seg(up, (ulong)addr, 0)) == nil)
    error(Ebadarg);
if(*addr < 0)
    error(Ebadarg);
return tsemacquire(s, addr, ms);
}

⟨syscall semrelease 421a⟩≡ (679b)
// long semrelease(long *addr, long count);
long
syssemrelease(ulong* arg)
{
    long *addr, delta;
    Segment *s;

    validaddr(arg[0], sizeof(long), true);
    arch_validalign(arg[0], sizeof(long));
    addr = (long*)arg[0];
    delta = arg[1];

    if((s = seg(up, (ulong)addr, 0)) == nil)
        error(Ebadarg);
    /* delta == 0 is a no-op, not a release */
    if(delta < 0 || *addr < 0)
        error(Ebadarg);
    return semrelease(s, addr, delta);
}

```

18.4 sysrendezvous()

`rendezvous()` is a lightweight synchronization primitive that also exchanges a value. Two processes call `rendezvous(tag, value)` with the same tag: the first to arrive sleeps in a hash table; when the second arrives, they swap values and both continue. The `Rgrp` (rendezvous group) scopes which processes can rendezvous with each other.

```

⟨function REND 421b⟩≡ (668c)
enum
{
    RENDLOG = 5,
    RENDHASH = 1<<RENDLOG, /* Hash to lookup rendezvous tags */
};
#define REND(p,s) ((p)->rendhash[(s)&((1<<RENDLOG)-1)])

⟨struct Rgrp 421c⟩≡ (668c)
struct Rgrp
{
    // hash<??, list<ref<Proc>>>
    Proc *rendhash[RENDHASH]; /* Rendezvous tag hash */

    // extra
    Ref; /* the Ref's lock is also the Rgrp's lock */
};

```

⟨Proc *synchronization fields* 422a⟩+≡ (49f) <222f

```
Rgrp *rgrp;    /* Rendez group */

uintptr rendtag; /* Tag for rendezvous */
uintptr rendval; /* Value for rendezvous */
//??
Proc *rendhash; /* Hash list for tag values */
```

⟨*constructor newrgrp* 422b⟩≡ (673b)

```
Rgrp*
newrgrp(void)
{
    Rgrp *r;

    r = smalloc(sizeof(Rgrp));
    r->ref = 1;
    return r;
}
```

⟨*destructor closergrp* 422c⟩≡ (673b)

```
void
closergrp(Rgrp *r)
{
    if(decref(r) == 0)
        free(r);
}
```

⟨Procstate *cases* 422d⟩+≡ (51b) <203g 425a>

```
Rendezvous,
```

[sysrendezvous] searches the hash table for a process with the same tag. If found, it swaps values and wakes the sleeper. If not found, the caller inserts itself into the hash chain, sets state to **Rendezvous**, and calls **sched()** to sleep until a partner arrives. A return value of 0 indicates interruption (e.g., by a note).

⟨*syscall rendezvous* 422e⟩≡ (679a)

```
// void* rendezvous(void* tag, void* value);
long
sysrendezvous(ulong* arg)
{
    uintptr tag, val;
    Proc *p, **l;

    tag = arg[0];
    l = &REND(up->rgrp, tag);
    up->rendval = ~(uintptr)0;

    lock(up->rgrp);
    for(p = *l; p; p = p->rendhash) {
        if(p->rendtag == tag) {
            *l = p->rendhash;
            val = p->rendval;
            p->rendval = arg[1];

            while(p->cpu != 0) // ???
                ;

            ready(p);
            unlock(up->rgrp);
            return val;
        }
    }
    l = &p->rendhash;
```

```
}

/* Going to sleep here */
up->rendtag = tag;
up->rendval = arg[1];
up->rendhash = *l;
*l = up;
up->state = Rendezvous;
unlock(up->rgrp);

sched();

return up->rendval;
}
```

18.5 Network

Chapter 19

Debugging Support

In this chapter, I will present the kernel infrastructure that makes debuggers and system call tracers possible. Building a tool like `/bin/db` (the Plan 9 debugger) or `/bin/strace` requires three capabilities from the kernel: the ability to *stop* another process, the ability to *inspect* its memory and registers, and the ability to *resume* it. In Plan 9, all of these are provided through the `/proc` filesystem—there is no `ptrace()` system call as in UNIX. The debugger simply reads and writes files like `/proc/\emphpid/mem`, `/proc/\emphpid/regs`, and `/proc/\emphpid/ctl`.

Breakpoints are implemented using the ARM BKPT instruction. When the processor hits this instruction, it triggers a prefetch fault, and the trap handler recognizes it as a debug event. For user-mode breakpoints, the kernel posts a “sys: breakpoint” note to the process, which the debugger can intercept. A kernel-mode breakpoint is a fatal error.

```
<trap() when undefined instruction, if breakpoint(arm) 424a>≡ (148b)
    if(seg(up, ureg->pc, false) != nil &&
        *(u32int*)ureg->pc == 0xD1200070)
        postnote(up, 1, "sys: breakpoint", NDebug);
```

```
<trap() when prefetch fault, switch fst cases(arm) 424b>≡ (148e)
    case 0x02: /* instruction debug event (BKPT) */
        if(user){
            snprint(buf, sizeof buf, "sys: breakpoint");
            postnote(up, 1, buf, NDebug);
        }else{
            iprint("kernel bkpt: pc %#lux inst %#ux\n",
                ureg->pc, *(u32int*)ureg->pc);
            panic("kernel bkpt");
        }
        break;
```

```
<function arch_setregisters(arm) 424c>≡ (717a)
/* This routine must save the values of registers the user is not permitted
 * to write from devproc and then restore the saved values before returning.
 */
void
arch_setregisters(Ureg* ureg, char* pureg, char* uva, int n)
{
    USED(ureg, pureg, uva, n);
}
```

19.1 Core dumps and broken processes

Instead of writing core dumps to disk like UNIX, Plan 9 keeps a crashed process alive in the **Broken** state. The process retains its address space and registers intact, so a debugger can attach to it with `db \emphpid` and do

post-mortem analysis without any intermediate file. This is both more convenient (no multi-megabyte core files) and more faithful (the entire process state is preserved, not just what the kernel chose to dump). The kernel limits broken processes to NBROKEN (4) at a time; when a new one arrives and the table is full, the oldest is evicted and allowed to finish dying.

```
<Procstate cases 425a>+= (51b) <422d 426d>
    Broken,
```

```
<struct Broken 425b>= (675c)
/*
 * weird thing: keep at most NBROKEN around
 */
#define NBROKEN 4
struct Broken
{
    // array<ref<Proc>>
    Proc    *p[NBROKEN];
    // number of entries used in p
    int n;

    // extra
    QLock;
};
```

```
<global broken 425c>= (675c)
    struct Broken broken;
```

```
<pexit() if not freemem, add process to broken processes list 425d>= (130b)
    // instead of a core dump we just keep the process around
    if(!freemem)
        addbroken(up); // will call sched()
```

When `addbroken()` is called (from `pexit()` with `freemem` false), the process is suspended in the **Broken** state and added to the fixed-size **broken** array. If the array is full, the oldest broken process is evicted by calling `ready()` on it, which resumes its `pexit()` and lets it finally free its resources and die. Killing a broken process (via “kill” in its `ctl` file) calls `unbreak()`, which also readies it to resume dying. The `freebroken()` function is a last-resort call from the pager when memory is critically low.

```
<function addbroken 425e>= (675c)
void
addbroken(Proc *p)
{
    qlock(&broken);
    if(broken.n == NBROKEN) {
        ready(broken.p[0]);
        memmove(&broken.p[0], &broken.p[1], sizeof(Proc)*(NBROKEN-1));
        --broken.n;
    }
    broken.p[broken.n++] = p;
    qunlock(&broken);

    edfstop(up);
    p->state = Broken;
    p->psstate = nil;
    sched();
}
```

```
<function unbreak 426a>≡ (675c)
```

```
void
unbreak(Proc *p)
{
    int b;

    qlock(&broken);
    for(b=0; b < broken.n; b++)
        if(broken.p[b] == p) {
            broken.n--;
            memmove(&broken.p[b], &broken.p[b+1],
                    sizeof(Proc*)*(NBROKEN-(b+1)));
            ready(p);
            break;
        }
    qunlock(&broken);
}
```

```
<function freebroken 426b>≡ (675c)
```

```
int
freebroken(void)
{
    int i, n;

    qlock(&broken);
    n = broken.n;
    for(i=0; i<n; i++) {
        ready(broken.p[i]);
        broken.p[i] = nil;
    }
    broken.n = 0;
    qunlock(&broken);
    return n;
}
```

```
<procctlreq() CMkill case, Broken case 426c>≡ (137a)
```

```
case Broken:
    // will resume the broken process to finally die and free its mem
    unbreak(p);
    break;
```

19.2 Stopping a process: /proc/x/ctl

The Stopped state is the foundation for all debugging. A process enters this state when `procctl()` is called with `Proc_stopme`. But a process can only stop *itself*—the kernel cannot arbitrarily suspend a running process on another CPU. Instead, the debugger writes “stop” to `/proc/\emphpid/ctl`, which sets the target’s `procctl` field to `Proc_stopme`. This field is checked at the next trap or system call return (via `notify()`), at which point the process stops itself. This is similar to how notes (signals) are delivered—they are deferred until a safe point.

```
<Procstate cases 426d>+≡ (51b) <425a 505b>
```

```
Stopped,
```

```
<enum procctl 426e>≡ (668c)
```

```
enum Procctl
{
    Proc_nothing = 0,
    Proc_stopme,
    <Procctl cases 136c>
};
```

<Proc debugger fields 427a>+≡ (49f) <143b 427e>

```
// enum<procctl>
int procctl; /* Control for /proc debugging */
```

The `procctl()` function is where a process actually stops itself. It is called from `notify()` on return from a trap or system call, so it runs in the context of the process being controlled (always up). For `Proc_stopme`, the process sets its state to `Stopped`, wakes up any waiting debugger via `pdbg->sleepr`, and calls `sched()` to give up the CPU. It will remain in the scheduler's stopped list until the debugger writes "start" to its ctl file.

<function procctl 427b>≡ (675c)

```
/*
 * called arch_splhi() by notify(). See comment in notify for the
 * reasoning.
 */
void
procctl(Proc *p)
{
    char *oldstate;
    ulong s;

    switch(p->procctl) {
        <procctl() Proc_traceme case (and fallthrough Proc_stopme) 443b>
        case Proc_stopme:
            p->procctl = Proc_nothing;
            oldstate = p->psstate;
            p->psstate = "Stopped";
            s = arch_spllo();
            <procctl() wakeup waiting debugger 428d>
            arch_splhi();
            p->state = Stopped;
            sched();
            // here when something (debugger, strace, user) ready(p) back
            p->psstate = oldstate;
            arch_splx(s);
            return;
        <procctl() Proc_exitbig case 674b>
        <procctl() Proc_exitme case 136d>
    }
}
```

<procctlreq() CMstop case 427c>≡ (554b)

```
case CMstop:
    procstopwait(p, Proc_stopme);
    break;
```

<procctlreq() CMstart case 427d>≡ (554b)

```
case CMstart:
    if(p->state != Stopped)
        error(Ebadctl);
    ready(p);
    break;
```

<Proc debugger fields 427e>+≡ (49f) <427a 428a>

```
Proc *pdbg; /* the debugging process */
```

<pexit() release debuggers 427f>≡ (130b)

```
/* release debuggers */
qlock(&up->debug);
if(up->pdbg) {
    wakeup(&up->pdbg->sleepr);
}
```

```

    up->pdbg = nil;
}
qunlock(&up->debug);
⟨Proc debugger fields 428a⟩+≡ (49f) <427e 429a>
    QLock debug; /* to access debugging elements */ // used for many things

```

The `procstopwait()` function is the debugger's half of the stop/resume handshake. The debugger (caller) sets the target's `procctl` field, then sleeps on its own `sleepr` rendez until the target actually reaches the **Stopped** state. The `pdbg` field records who is debugging whom, so only one debugger can control a process at a time. When the target stops (in `procctl()`), it wakes up the sleeping debugger.

```

⟨function procstopwait 428b⟩≡ (643)
// assumes p->debug is held
void
procstopwait(Proc *p, int ctl)
{
    int pid;

    if(p->pdbg)
        error(Einuse);
    if(procstopped(p) || p->state == Broken)
        return;

    if(ctl != Proc_nothing)
        p->procctl = ctl;

    p->pdbg = up;
    pid = p->pid;

    qunlock(&p->debug);
    up->psstate = "Stopwait";
    if(waserror()) {
        p->pdbg = nil;
        qlock(&p->debug);
        nexterror();
    }

    sleep(&up->sleepr, procstopped, p);

    poperror();
    qlock(&p->debug);
    if(p->pid != pid) // p was reallocated to a new process
        error(Eprocdied);
}

```

```

⟨function procstopped 428c⟩≡ (643)
bool
procstopped(void *a)
{
    Proc *p = a;
    return p->state == Stopped;
}

```

```

⟨procctl() wakeup waiting debugger 428d⟩≡ (427b)
/* free a waiting debugger */
qlock(&p->debug);
if(p->pdbg) {
    wakeup(&p->pdbg->sleepr);
    p->pdbg = nil;
}
qunlock(&p->debug);

```

To debug a program from the start, the debugger uses the “hang” mechanism. The sequence is: fork a child, write “hang” to the child’s ctl file, then let the child exec the target program. During `sysexec()`, the kernel sees `up->hang` and sets `procctl` to `Proc_stopme`, so the child stops before executing its first instruction. The debugger can then set breakpoints, inspect the initial state, and resume with “start”. The hang flag is inherited across `rfork`, so `strace` can trace children of the traced process as well.

```

⟨Proc debugger fields 429a⟩+≡ (49f) <428a 433d>
    bool hang; /* hang at next exec for debug */

⟨procctlreq() CMhang case 429b⟩≡ (554b)
    case CMhang:
        p->hang = true;
        break;

⟨procctlreq() CMnohang case 429c⟩≡ (554b)
    case CMnohang:
        p->hang = false;
        break;

⟨sysrfork() when RFPROC propagate fields from up to p 429d⟩+≡ (112d) <396b 454b>
    p->hang = up->hang;

⟨sysexec() if hang 429e⟩≡ (119d)
    if(up->hang)
        up->procctl = Proc_stopme;

⟨procctlreq() CMwaitstop case 429f⟩≡ (554b)
    case CMwaitstop:
        procstopwait(p, Proc_nothing);
        break;

```

19.3 Inspecting process memory: `/proc/x/mem`

Reading `/proc/emphid/mem` lets a debugger inspect another process’s address space. Reads at offsets below `KZERO` go through `procctlmemio()`, which walks the target’s page tables and temporarily maps the physical page into kernel space to copy the data. Writes require the process to be stopped first. If the debugger writes to a text segment, the kernel first converts it to a data segment via `text2data()`—this is copy-on-write for code, needed because text segments may be shared across processes.

```

⟨procread() cases 429g⟩+≡ (540c) <308c 433f>
    case Qmem:
        if(offset < KZERO)
            return procctlmemio(p, offset, n, va, true);

        if(!iseve())
            error(Eperm);

        // why allowing read access to kernel code or kernel memory?
        // who is using that?

        /* validate kernel addresses */
        if(offset < (kern_addr)end) {
            if(offset+n > (kern_addr)end)
                n = (kern_addr)end - offset;
            memmove(a, (char*)offset, n);
            return n;
        }
        for(i=0; i<nelem(conf.mem); i++){

```

```

    cm = &conf.mem[i];
    /* klimit-1 because klimit might be zero! */
    if(cm->kbase <= offset && offset <= cm->klimit-1){
        if(offset+n >= cm->klimit-1)
            n = cm->klimit - offset;
        memmove(a, (char*)offset, n);
        return n;
    }
}
error(Ebadarg);

⟨xinit() nkpages kernel memory in m 430a⟩≡ (246b)
    m->kbase = (ulong)KADDR(m->base);
    m->klimit = (ulong)KADDR(m->base+nkpages*BY2PG);

⟨Confmem other fields 430b⟩≡ (46c)
    kern_addr kbase; // KADDR(Confmem.base)
    kern_addr klimit; // KADDR(base+ x*BY2PG) where x <= Confmem.npage

⟨procwrite() cases 430c⟩≡ (543)
    case Qmem:
        if(p->state != Stopped)
            error(Ebadctl);

        n = procctlmemio(p, offset, n, va, false);
        break;

⟨Segment other fields 430d⟩+≡ (92c) <417a 446a>
    ushort steal; /* Page stealer lock */

⟨function procctlmemio 430e⟩≡ (643)
    int
    procctlmemio(Proc *p, ulong offset, int n, virt_vp va, bool read)
    {
        Arch_KMap *k;
        Pagetable *pt;
        Page *pg;
        Segment *s;
        ulong soff, l;
        char *a = va, *b;

        for(;;) {
            s = seg(p, offset, 1);
            if(s == nil)
                error(Ebadarg);

            if(offset+n >= s->top)
                n = s->top-offset;

            if(!read && (s->type&SG_TYPE) == SG_TEXT)
                s = text2data(p, s);

            s->steal++;
            soff = offset-s->base;
            if(waserror()) {
                s->steal--;
                nexterror();
            }
            if(fixfault(s, offset, read, /*putmmu*/false) == 0)
                break;
        }
    }

```

```

        poperror();
        s->steal--;
    }
    poperror();
    pt = s->pagedir[soff/PAGETABMAPMEM];
    if(pt == nil)
        panic("procctlmemio");
    pg = pt->pagetab[(soff&(PAGETABMAPMEM-1))/BY2PG];
    if(pagedout(pg))
        panic("procctlmemio1");

    l = BY2PG - (offset&(BY2PG-1));
    if(n > l)
        n = l;

    k = arch_kmap(pg);
    if(waserror()) {
        s->steal--;
        arch_kunmap(k);
        nexterror();
    }
    b = (char*)VA(k);
    b += offset&(BY2PG-1);
    if(read == 1)
        memmove(a, b, n);    /* This can fault */
    else
        memmove(b, a, n);
    arch_kunmap(k);
    poperror();

    <procctlmemio() set cachectl if flushme segment 494g>

    s->steal--;

    if(read == 0)
        p->newtlb = false;

    return n;
}

```

<function text2data 431>≡ (643)

```

Segment*
text2data(Proc *p, Segment *s)
{
    int i;
    Segment *ps;

    ps = newseg(SG_DATA, s->base, s->size);
    ps->image = s->image;
    incref(ps->image);
    ps->fstart = s->fstart;
    ps->flen = s->flen;
    <text2data() initialize other fields 494c>

    qlock(&p->seglock);
    for(i = 0; i < NSEG; i++)
        if(p->seg[i] == s)
            break;
    if(i == NSEG)
        panic("segment gone");
}

```



```

    qunlock(&s->lk);
    putseg(s);
    qlock(&ps->lk);
    p->seg[i] = ps;
    qunlock(&p->seglock);

    return ps;
}

```

19.4 /bin/strace

System call tracing uses a ping-pong protocol between the tracer and the traced process. The tracer writes “startsyscall” to `/proc/\emphpid/ctl`, which readies the traced process and immediately sleeps waiting for it to stop again. The traced process runs until its next syscall entry, where it formats the syscall name and arguments into `syscalltrace`, sets itself to `Proc_stopme`, and stops. The tracer wakes up, reads `/proc/\emphpid/syscall`, then writes “startsyscall” again to let the traced process execute the actual syscall. The process stops a second time after the syscall completes, allowing the tracer to read the return value. Here is the sequence:

tracer	traced (ls)
-- "startsyscall" --> ready	
(sleep on sleep)	
	run until syscall
	format args
	stop (wakeup tracer)
read /proc/pid/syscall <---	
-- "startsyscall" --> ready	
(sleep on sleep)	
	execute syscall
	format return value
	stop (wakeup tracer)
read /proc/pid/syscall <---	
... repeat ...	

`<Procctl cases 432a>+≡` (426e) <136c 443a>
`Proc_tracesyscall,`

`<procctlreq() CMstartsyscall case 432b>≡` (554b)
`case CMstartsyscall:`
`if(p->state != Stopped)`
`error(Ebadctl);`
`p->procctl = Proc_tracesyscall;`
`ready(p);`
`procstopwait(p, Proc_tracesyscall); // will sleep`
`break;`

`<syscall() Proc_tracesyscall if, syscall entry(arm) 432c>≡` (141e)
`if(up->procctl == Proc_tracesyscall){`
`/*`
`* Redundant validaddr. Do we care?`
`* Tracing syscalls is not exactly a fast path...`
`* Beware, validaddr currently does a pexit rather`
`* than an error if there's a problem; that might`

```

    * change in the future.
    */
if(sp < (USTKTOP-BY2PG) || sp > (USTKTOP-sizeof(Sargs)-BY2WD))
    validaddr(sp, sizeof(Sargs)+BY2WD, false);

syscallfmt(scallnr, ureg->pc, (va_list)(sp+BY2WD));
up->procctl = Proc_stopme;
// this will call sched() and wakeup the tracer process
procctl(up);
// back here when the tracer process readied us back and
// should have set procctl back to Proc_tracesyscall
if(up->syscalltrace)
    free(up->syscalltrace);
up->syscalltrace = nil;
}

⟨syscall() other locals(arm) 433a⟩+≡ (141e) <142a
    vlong startns, stopns;

⟨syscall() Proc_tracesyscall if, syscall exit(arm) 433b⟩≡ (141e)
    if(up->procctl == Proc_tracesyscall){
        stopns = todget(nil);
        up->procctl = Proc_stopme;
        sysretfmt(scallnr, (va_list)(sp+BY2WD), ret, startns, stopns);
        s = arch_splhi();
        procctl(up); // again, will call sched() and wakeup tracer process
        arch_splx(s);
        if(up->syscalltrace)
            free(up->syscalltrace);
        up->syscalltrace = nil;
    }

⟨sysrfork() inherit Proc_tracesyscall 433c⟩≡
    if(up && up->procctl == Proc_tracesyscall)
        p->procctl = Proc_tracesyscall;

⟨Proc debugger fields 433d⟩+≡ (49f) <429a 440a>
    char *syscalltrace; /* syscall trace */

⟨pexit() at the start, free resources 433e⟩+≡ (130b) <225b
    if(up->syscalltrace)
        free(up->syscalltrace);

⟨procread() cases 433f⟩+≡ (540c) <429g 447b>
    case Qsyscall:
        if(!p->syscalltrace)
            return 0;
        n = readstr(offset, a, n, p->syscalltrace);
        return n;

```

The `syscallfmt()` and `sysretfmt()` functions format system call entries and exits as human-readable strings for `strace`. Each syscall number has a custom case that knows how to decode its specific argument types—strings are read from user space and quoted, pointers are shown as hex, file descriptors as integers. The result is stored in `up->syscalltrace`, which the tracer reads via `/proc/emphpid/syscall`.

```

⟨function syscallfmt 433g⟩≡ (682a)
void
syscallfmt(int syscallno, ulong pc, va_list list)
{
    long l;

```

```

Fmt fmt;
void *v;
vlong vl;
uintptr p;
int i[2], len;
char *a, **argv;

fmtstrinit(&fmt);
fmtprint(&fmt, "%uld %s ", up->pid, up->text);

if(syscallno > nsyscall)
    fmtprint(&fmt, " %d ", syscallno);
else
    fmtprint(&fmt, "%s ", sysctab[syscallno]?
        sysctab[syscallno]: "huh?");

fmtprint(&fmt, "%ulx ", pc);
if(up->syscalltrace != nil)
    free(up->syscalltrace);

switch(syscallno){
case NOP:
    p = va_arg(list, uintptr);
    fmtprint(&fmt, "%#p", p);
    break;
case CHDIR:
case EXITS:
case REMOVE:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, "");
    break;
case BIND:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    i[0] = va_arg(list, int);
    fmtprint(&fmt, "%#ux", i[0]);
    break;
case CLOSE:
case NOTED:
    i[0] = va_arg(list, int);
    fmtprint(&fmt, "%d", i[0]);
    break;
case DUP:
    i[0] = va_arg(list, int);
    i[1] = va_arg(list, int);
    fmtprint(&fmt, "%d %d", i[0], i[1]);
    break;
case ALARM:
    l = va_arg(list, unsigned long);
    fmtprint(&fmt, "%#lud ", l);
    break;
case EXEC:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, "");
    argv = va_arg(list, char**);
    //validalign(PTR2UINT(argv), sizeof(char*)); ??
    for(;;){
        validaddr((ulong)argv, sizeof(char**), false);

```

```

        a = *(char **)argv;
        if(a == nil)
            break;
        fprintf(&fmt, " ");
        fmtuserstring(&fmt, a, "");
        argv++;
    }
    break;
case FAUTH:
    i[0] = va_arg(list, int);
    a = va_arg(list, char*);
    fprintf(&fmt, "%d", i[0]);
    fmtuserstring(&fmt, a, "");
    break;
case SEGBRK:
case RENDEZVOUS:
    v = va_arg(list, void*);
    fprintf(&fmt, "%#p ", v);
    v = va_arg(list, void*);
    fprintf(&fmt, "%#p", v);
    break;
case OPEN:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    i[0] = va_arg(list, int);
    fprintf(&fmt, "%#ux", i[0]);
    break;
case SLEEP:
    l = va_arg(list, long);
    fprintf(&fmt, "%ld", l);
    break;
case RFORK:
    i[0] = va_arg(list, int);
    fprintf(&fmt, "%#ux", i[0]);
    break;
case PIPE:
case BRK:
    v = va_arg(list, int*);
    fprintf(&fmt, "%#p", v);
    break;
case CREATE:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    i[0] = va_arg(list, int);
    i[1] = va_arg(list, int);
    fprintf(&fmt, "%#ux %#ux", i[0], i[1]);
    break;
case FD2PATH:
case FSTAT:
case FWSTAT:
    i[0] = va_arg(list, int);
    a = va_arg(list, char*);
    l = va_arg(list, unsigned long);
    fprintf(&fmt, "%d %#p %lud", i[0], a, l);
    break;
case NOTIFY:
case SEGDETACH:
    v = va_arg(list, void*);
    fprintf(&fmt, "%#p", v);
    break;

```

```

case SEGATTACH:
    i[0] = va_arg(list, int);
    fprintf(&fmt, "%d ", i[0]);
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    /*FALLTHROUGH*/
case SEGFREE:
case SEGFLUSH:
    v = va_arg(list, void*);
    l = va_arg(list, unsigned long);
    fprintf(&fmt, "%#p %lud", v, l);
    break;
case UNMOUNT:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, "");
    break;
case SEMACQUIRE:
case SEMRELEASE:
    v = va_arg(list, int*);
    i[0] = va_arg(list, int);
    fprintf(&fmt, "%#p %d", v, i[0]);
    break;
case TSEMACQUIRE:
    v = va_arg(list, long*);
    l = va_arg(list, ulong);
    fprintf(&fmt, "%#p %ld", v, l);
    break;
case SEEK:
    v = va_arg(list, vlong*);
    i[0] = va_arg(list, int);
    vl = va_arg(list, vlong);
    i[1] = va_arg(list, int);
    fprintf(&fmt, "%#p %d %#llx %d", v, i[0], vl, i[1]);
    break;
case FVERSION:
    i[0] = va_arg(list, int);
    i[1] = va_arg(list, int);
    fprintf(&fmt, "%d %d ", i[0], i[1]);
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    l = va_arg(list, unsigned long);
    fprintf(&fmt, "%lud", l);
    break;
case WSTAT:
case STAT:
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    /*FALLTHROUGH*/
case ERRSTR:
case AWAIT:
    a = va_arg(list, char*);
    l = va_arg(list, unsigned long);
    fprintf(&fmt, "%#p %lud", a, l);
    break;
case MOUNT:
    i[0] = va_arg(list, int);
    i[1] = va_arg(list, int);
    fprintf(&fmt, "%d %d ", i[0], i[1]);

```

```

    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, " ");
    i[0] = va_arg(list, int);
    fmtprint(&fmt, "%#ux ", i[0]);
    a = va_arg(list, char*);
    fmtuserstring(&fmt, a, "");
    break;
case PREAD:
    i[0] = va_arg(list, int);
    v = va_arg(list, void*);
    l = va_arg(list, long);
    fmtprint(&fmt, "%d %#p %ld", i[0], v, l);
    if(syscallno == PREAD){
        vl = va_arg(list, vlong);
        fmtprint(&fmt, " %lld", vl);
    }
    break;
case PWRITE:
    i[0] = va_arg(list, int);
    v = va_arg(list, void*);
    l = va_arg(list, long);
    fmtprint(&fmt, "%d ", i[0]);
    len = MIN(l, 64);
    fmtrwdata(&fmt, v, len, " ");
    fmtprint(&fmt, "%ld", l);
    if(syscallno == PWRITE){
        vl = va_arg(list, vlong);
        fmtprint(&fmt, " %lld", vl);
    }
    break;
}

up->syscalltrace = fmtstrflush(&fmt);
}

```

$\langle$ function sysretfmt 437 $\rangle \equiv$ (682a)

```

void
sysretfmt(int syscallno, va_list list, long ret, uulong start, uulong stop)
{
    long l;
    void* v;
    Fmt fmt;
    vlong vl;
    int i, len;
    char *a, *errstr;

    fmtstrinit(&fmt);

    if(up->syscalltrace)
        free(up->syscalltrace);

    errstr = "\"\\";
    switch(syscallno){
    default:
    case ALARM:
    case PWRITE:
        if(ret == -1)
            errstr = up->syserrstr;
        fmtprint(&fmt, " = %ld", ret);
        break;
    }
}

```

```

case EXEC:
case SEGBRK:
case SEGATTACH:
case RENDEZVOUS:
    if((void *)ret == (void*)-1)
        errstr = up->syserrstr;
    fprintf(&fmt, " = %#p", (void *)ret);
    break;
case AWAIT:
    a = va_arg(list, char*);
    l = va_arg(list, unsigned long);
    if(ret > 0){
        fmtuserstring(&fmt, a, " ");
        fprintf(&fmt, "%lud = %ld", l, ret);
    }
    else{
        fprintf(&fmt, "%#p/\" \" %lud = %ld", a, l, ret);
        errstr = up->syserrstr;
    }
    break;
case ERRSTR:
    a = va_arg(list, char*);
    l = va_arg(list, unsigned long);
    if(ret > 0){
        fmtuserstring(&fmt, a, " ");
        fprintf(&fmt, "%lud = %ld", l, ret);
    }
    else{
        fprintf(&fmt, "\" \" %lud = %ld", l, ret);
        errstr = up->syserrstr;
    }
    break;
case FD2PATH:
    i = va_arg(list, int);
    USED(i);
    a = va_arg(list, char*);
    l = va_arg(list, unsigned long);
    if(ret > 0){
        fmtuserstring(&fmt, a, " ");
        fprintf(&fmt, "%lud = %ld", l, ret);
    }
    else{
        fprintf(&fmt, "\" \" %lud = %ld", l, ret);
        errstr = up->syserrstr;
    }
    break;
case PREAD:
    i = va_arg(list, int);
    USED(i);
    v = va_arg(list, void*);
    l = va_arg(list, long);
    if(ret > 0){
        len = MIN(ret, 64);
        fmtrwdata(&fmt, v, len, "");
    }
    else{
        fprintf(&fmt, "/\" \"");
        errstr = up->syserrstr;
    }
    fprintf(&fmt, " %ld", l);

```

```

        if(syscallno == PREAD){
            vl = va_arg(list, vlong);
            fprintf(&fmt, " %lld", vl);
        }
        fprintf(&fmt, " = %ld", ret);
        break;
    }
    fprintf(&fmt, " %s %lldud %lldud\n", errstr, start, stop);
    up->syscalltrace = fmtstrflush(&fmt);
}

⟨function fmtrwdata 439a⟩≡ (682a)
// WE ARE OVERRUNNING SOMEHOW
static void
fmtrwdata(Fmt* f, char* a, int n, char* suffix)
{
    int i;
    char *t;

    if(a == nil){
        fprintf(f, "0x0%s", suffix);
        return;
    }
    validaddr((ulong)a, n, false);
    t = smalloc(n+1);
    for(i = 0; i < n; i++)
        if(a[i] > 0x20 && a[i] < 0x7f) /* printable ascii? */
            t[i] = a[i];
        else
            t[i] = '.';

    fprintf(f, " %#p/\\"%s\\"%s", a, t, suffix);
    free(t);
}

⟨function fmtuserstring 439b⟩≡ (682a)
static void
fmtuserstring(Fmt* f, char* a, char* suffix)
{
    int n;
    char *t;

    if(a == nil){
        fprintf(f, "0/\\"%s\"", suffix);
        return;
    }
    validaddr((ulong)a, 1, false);
    n = ((char*)vmemchr(a, 0, 0x7fffffff) - a) + 1;
    t = smalloc(n+1);
    memmove(t, a, n);
    t[n] = 0;
    fprintf(f, " %#p/\\"%s\\"%s", a, t, suffix);
    free(t);
}

```

19.5 Scheduler events: /proc/trace and /bin/trace

The /proc/trace file provides a stream of scheduler events for processes that have tracing enabled. Each event records a process ID, an event type (SReady, SRun, SDead, SSleep, or SUser), and a timestamp. The events

are stored in a circular buffer of `Nevents` (16384) entries. The tracing hooks are scattered throughout the scheduler: in `ready()`, `runproc()`, `sleep()`, and `pexit()`. To avoid races, each hook copies the `proctrace` function pointer to a local variable before calling it—if the pointer were read twice, it could become nil between the check and the call.

```

<Proc debugger fields 440a>+≡ (49f) <433d 552a>
    bool trace; /* process being traced? */

<procctlreq() CMtrace case 440b>≡ (554b)
    case CMtrace:
        switch(cb->nf){
            case 1:
                p->trace ^= true;
                break;
            case 2:
                p->trace = (atoi(cb->f[1]) != 0);
                break;
            default:
                error("args");
        }
        break;

<hook proctrace 440c>≡ (671a)
    void (*proctrace)(Proc*, /*enum<tevent>*/int, vlong) = 0; // was in devproc.c

<enum Tevent 440d>≡ (? 0—1)
    enum Tevent {
        SReady = 0, /* runnable but not running */
        SRun, /* running best effort */
        SDead, /* proc dies */
        SSleep, /* blocked */
        SUser, /* user event */

        /* real-time extensions */
        <enum Tevent real-time scheduling events 517a>

        Nevent, // must be last
    };

<struct Traceevent 440e>≡ (? 0—1)
    struct Traceevent {
        ulong pid;
        // enum<tevent>
        ulong etype; /* Event type */
        vlong time; /* time stamp */ // dimension?
    };

<ready() other locals 440f>≡ (202c)
    void (*pt)(Proc*, int, vlong);

<ready() hook proctrace 440g>≡ (202c)
    pt = proctrace;
    if(pt)
        pt(p, SReady, 0);

<runproc() other locals 440h>≡ (196) 587c>
    void (*pt)(Proc*, int, vlong);

<runproc() hook proctrace 440i>≡ (196)
    pt = proctrace;
    if(pt)
        pt(p, SRun, 0);

```

```

⟨pexit() other locals 441a⟩≡ (130b) <133f 445d>
    void (*pt)(Proc*, int, vlong);

⟨pexit() hook proctrace 441b⟩≡ (130b)
    pt = proctrace;
    if(pt)
        pt(up, SDead, 0);

⟨sleep() other locals 441c⟩≡ (204)
    void (*pt)(Proc*, int, vlong);

⟨sleep() hook proctrace 441d⟩≡ (204)
    pt = proctrace;
    if(pt)
        pt(up, SSleep, 0);

⟨procctlreq() CMevent case 441e⟩≡ (554b)
    case CMevent:
        pt = proctrace;
        if(up->trace && pt)
            pt(up, SUser, 0);
        break;

⟨global trace txxx 441f⟩≡ (643)
    // array<Traceevent>
    static Traceevent *tevents;
    static int topens;
    static int tproduced, tconsumed;
    static Lock tlock;

⟨function eventsavailable 441g⟩≡ (643)
    static int
    eventsavailable(void *)
    {
        return tproduced > tconsumed;
    }

⟨function _proctrace 441h⟩≡ (643)
    static void
    _proctrace(Proc* p, Tevent etype, vlong ts)
    {
        Traceevent *te;

        if (p->trace == false || topens == 0 ||
            tproduced - tconsumed >= Nevents)
            return;

        te = &tevents[tproduced&Emask];
        te->pid = p->pid;
        te->etype = etype;
        if (ts == 0)
            te->time = todget(nil);
        else
            te->time = ts;
        tproduced++;
    }

⟨constant Nevents 441i⟩≡ (643)
    Nevents = 0x4000,

```

`<constant Emask 442a>≡` (643)

```
Emask = Nevents - 1,
```

`<procopen() Qtrace if 442b>≡` (545b)

```
if(QID(c->qid) == Qtrace){
    if (omode != OREAD)
        error(Eperm);
    lock(&tlock);
    if (waserror()){
        unlock(&tlock);
        nexterror();
    }
    if (topens > 0)
        error("already open");
    topens++;
    if (tevents == nil){
        tevents = (Traceevent*)malloc(Nevents * sizeof(Traceevent));
        if(tevents == nil)
            error(Enomem);
        tproduced = tconsumed = 0;
    }
    proctrace = _proctrace;
    unlock(&tlock);
    poperror();

    c->mode = openmode(omode);
    c->flag |= COPEN;
    c->offset = 0;
    return c;
}
```

`<procclose() Qtrace if 442c>≡` (551)

```
if(QID(c->qid) == Qtrace){
    lock(&tlock);
    if(topens > 0)
        topens--;
    if(topens == 0)
        proctrace = nil;
    unlock(&tlock);
}
```

`<procread() Qtrace if 442d>≡` (540c)

```
if(QID(c->qid) == Qtrace){
    if(!eventsavailable(nil))
        return 0;

    rptr = (byte*)va;
    navail = tproduced - tconsumed;
    if(navail > n / sizeof(Traceevent))
        navail = n / sizeof(Traceevent);
    while(navail > 0) {
        ne = ((tconsumed & Emask) + navail > Nevents)?
            Nevents - (tconsumed & Emask): navail;
        memmove(rptr, &tevents[tconsumed & Emask],
            ne * sizeof(Traceevent));

        tconsumed += ne;
        rptr += ne * sizeof(Traceevent);
        navail -= ne;
    }
}
```

```

    return rptr - (byte*)va;
}

```

19.6 /bin/db

The debugger `db` sets breakpoints by writing a BKPT instruction into the target’s text segment via `/proc/\emphpid/m` (after stopping the process and converting text to data with `text2data()`). When the process resumes and hits that address, the hardware faults, the trap handler recognizes the BKPT opcode, and a “sys: breakpoint” note is posted. Since the debugger is waiting on the target’s ctl file, it wakes up and can inspect the state. This is efficient because the breakpoint check happens in hardware, not on every instruction.

19.7 Other

```

<Procctl cases 443a>+≡ (426e) <432a 674a>
    Proc_traceme,

```

```

<procctl() Proc_traceme case (and fallthrough Proc_stopme) 443b>≡ (427b)
    case Proc_traceme:
        if(p->nnote == 0)
            return;
        /* No break */

```

```

<procctlreq() CMstartstop case 443c>≡ (554b)
    case CMstartstop:
        if(p->state != Stopped)
            error(Ebadctl);
        p->procctl = Proc_traceme;
        ready(p);
        procstopwait(p, Proc_traceme);
        break;

```

Chapter 20

Profiling Support

In this chapter, I will present two forms of profiling support in the kernel: timing and sampling. The timing infrastructure tracks how much CPU time each process spends in user mode vs kernel mode, reported via `/dev/cputime` and `/proc/\emphpid/status`. The sampling infrastructure (`/bin/tprof`) uses a periodic timer to record which instruction the program is executing, building a coverage histogram over the text segment. A third mechanism, `/bin/prof`, requires cooperation between the compiler, linker, and kernel through the `Tos` structure at the top of the user stack.

20.1 `/dev/cputime` and `/proc/#/status`

Each process tracks five time counters: user time, system time, real (wall-clock) time, and cumulative user/system time of children. The counters are incremented by `accounttime()`, which is called on every clock tick and checks `p->insyscall` to decide whether to attribute the tick to user or system time. The real time is stored as the tick count at process creation, so the actual elapsed time is computed on-the-fly as `CPUS(0)->ticks - p->time[TReal]`.

```
<enum proctimer 444a>≡ (668c)
enum Proctime
{
    TUser = 0,    /* Proc.time */
    TSys,
    TReal,

    // to accumulate also the time of the children of the process
    TCUser,
    TCSys,
};

<Proc stats and profiling fields 444b>≡ (49f) 448b▷
// hash<enum<proctime>, ulong>
ulong time[5]; /* User, Sys, Real; child U, S */

<sysrfork() setting time field 444c>≡ (112d)
memset(p->time, 0, sizeof(p->time));
p->time[TReal] = CPUS(0)->ticks;

<kproc() setting time field 444d>≡ (135b)
memset(p->time, 0, sizeof(p->time));
p->time[TReal] = CPUS(0)->ticks;

<accounttime() update time of current process 444e>≡ (588b)
p->time[p->insyscall ? TSys : TUser]++;
```

```

<consread() cases 445a>+≡ (345c) <232b 570b>
case Qcputime:
    k = offset;
    if(k >= 5*NUMSIZE)
        return 0;
    if(k+n > 5*NUMSIZE)
        n = 5*NUMSIZE - k;
    /* easiest to format in a separate buffer and copy out */

    for(i=0; i<5 && NUMSIZE*i<k+n; i++){
        l = up->time[i];
        if(i == TReal)
            l = CPUS(0)->ticks - l;
        l = TK2MS(l);
        readnum(0, tmp+NUMSIZE*i, NUMSIZE, l, NUMSIZE);
    }
    memmove(buf, tmp+k, n);
    return n;

```

```

<procread() Qstatus case, time part 445b>≡ (540c)
for(i = 0; i < 5; i++) {
    l = p->time[i];
    if(i == TReal)
        l = CPUS(0)->ticks - l;
    l = TK2MS(l);
    readnum(0, statbuf+j+NUMSIZE*i, NUMSIZE, l, NUMSIZE);
}

```

20.2 /bin/time

The `/bin/time` command is trivially simple because the kernel already includes timing information in the wait message that a parent receives when a child exits. The `Waitmsg` carries three values—user, system, and real time—which `pexit()` computes from the process’s counters. The `TCUser` and `TCSys` fields accumulate the children’s times, so the reported values include all descendants, just like UNIX’s `/usr/bin/time`.

```

<Waitmsg time field 445c>≡ (127f)
    ulong time[3]; /* of loved one and descendants */

```

```

<pexit() other locals 445d>+≡ (130b) <441a
    long utime, stime;

```

```

<pexit() set wait msg time field 445e>≡ (132b)
    utime = up->time[TUser] + up->time[TCUser];
    stime = up->time[TSys] + up->time[TCSys];

    wq->w.time[TUser] = tk2ms(utime);
    wq->w.time[TSys] = tk2ms(stime);
    wq->w.time[TReal] = tk2ms(CPUS(0)->ticks - up->time[TReal]);

```

```

<pexit() update TC time of parent 445f>≡ (132c)
    p->time[TCUser] += utime;
    p->time[TCSys] += stime;

```

```

<sysawait() snprint time field arguments 445g>≡ (128a)
    w.time[TUser], w.time[TSys], w.time[TReal],

```

```

<procread() Qwait case, snprint time field arguments 445h>≡ (535b)
    wq->w.time[TUser], wq->w.time[TSys], wq->w.time[TReal],

```

20.3 /bin/tprof

Tick-based profiling (`tprof`) works by sampling: a periodic timer fires at a frequency deliberately not aligned with HZ (to avoid aliasing), checks what PC the user process is executing, and increments a counter in the `profile` array attached to the text segment. The array covers the entire text segment with a resolution of $2^3 = 8$ bytes per bucket (LRESPROF). The profile is stored in the segment rather than the process so that multi-threaded programs sharing the same text accumulate a single combined profile. The first element `profile[0]` holds total execution time in milliseconds; subsequent elements hold per-bucket times. The `/bin/tprof` tool reads `/proc/\emphpid/profile` and correlates the buckets with symbol information from the binary.

```
<Segment other fields 446a>+≡ (92c) <430d 493i>
    kern_wp  profile; /* Tick profile area */ // for TSEG only
```

```
<putseg() free profile 446b>≡ (96b)
    if(s->profile != nil)
        free(s->profile);
```

```
<constant LRESPROF 446c>≡ (658)
    LRESPROF = 3,
```

```
<proc_init() other init 446d>≡ (539d)
    addclock0link((void (*)(void))profclock, 113); /* Relative prime to HZ */
```

```
<clock callback profclock 446e>≡ (643)
    static void
    profclock(Ureg *ur, Timer *)
    {
        if(up == nil || up->state != Running)
            return;

        /* user profiling clock */
        if(arch_userureg(ur)){
            <profclock tos adjustments when user 449b>
            segclock(ur->pc);
        }
    }
```

```
<clock callback segclock 446f>≡ (665)
    // called via profclock
    void
    segclock(ulong pc)
    {
        Segment *s;

        s = up->seg[TSEG];
        if(s == nil || s->profile == nil)
            return;

        s->profile[0] += TK2MS(1);
        if(pc >= s->base && pc < s->top) {
            pc -= s->base;
            s->profile[pc>>LRESPROF] += TK2MS(1);
        }
    }
```

`<procctlreq() CMprofile case 447a>≡` (554b)

```
case CMprofile:
    s = p->seg[TSEG];
    if(s == nil || (s->type&SG_TYPE) != SG_TEXT)
        error(Ebadctl);
    if(s->profile != nil)
        free(s->profile);
    npc = (s->top-s->base)>>LRESPROF;
    s->profile = malloc(npc*sizeof(*s->profile));
    if(s->profile == nil)
        error(Enomem);
    break;
```

`<procread() cases 447b>+≡` (540c) <433f 535b>

```
case Qprofile:
    s = p->seg[TSEG];
    if(s == nil || s->profile == nil)
        error("profile is off");
    i = (s->top-s->base)>>LRESPROF;
    i *= sizeof(*s->profile);
    if(offset >= i)
        return 0;
    if(offset+n > i)
        n = i - offset;
    memmove(a, ((char*)s->profile)+offset, n);
    return n;
```

`<procgen() Qprofile case 447c>≡` (549)

```
case Qprofile:
    q = p->seg[TSEG];
    if(q && q->profile) {
        len = (q->top-q->base)>>LRESPROF;
        len *= sizeof(*q->profile);
    }
    break;
```

20.4 /bin/prof

Function-level profiling (**prof**) requires cooperation between the compiler, linker, and kernel. The compiler instruments each function entry with profiling code, and the linker allocates profiling data structures. But making a system call on every function call would be far too slow, so the kernel provides the **Tos** (Top of Stack) structure mapped at the top of every user stack. This structure gives user code direct read access to cycle counts and a millisecond clock without entering the kernel. The kernel updates `tos->kcycles` and `tos->pcycles` on every kernel entry/exit, and `tos->clock` on every profiling tick, so the profiling library can compute accurate per-function times entirely in user space.

`<struct Tos 447d>≡` (? 0—1)

```
struct Tos {
    <Tos prof field ??>
    uulong cyclefreq; /* cycle clock frequency if there is one, 0 otherwise */

    vlong kcycles; /* cycles spent in kernel */
    vlong pcycles; /* cycles spent in process (kernel + user) */
    ulong clock; // in ms

    ulong pid; /* might as well put the pid here */
    <Tos other fields ??>
```



```

    /* top of stack is here */
};

<Cpu other fields 448a>+≡ (48a) <415b
    // cyclefreq == cpuhz if havetsc, 0 otherwise
    uulong cyclefreq; /* Frequency of user readable cycle counter */

<Proc stats and profiling fields 448b>+≡ (49f) <444b
    uulong kentry; /* Kernel entry time stamp (for profiling) */
    /*
     * pcycles: cycles spent in this process (updated on procsave/restore)
     * when this is the current proc and we're in the kernel
     * (procrestores outnumber procsaves by one)
     * the number of cycles spent in the proc is pcycles + cycles()
     * when this is not the current process or we're in user mode
     * (procrestores and procsaves balance), it is pcycles.
     */
    vlong pcycles;

<macro arch_cycles(arm) 448c>≡ (695d)
    #define arch_cycles(ip) *(ip) = arch_lcycles()

<type CpSPM secondary registers(arm) 448d>≡ (683c)
    /*
     * CpSPM Secondary (CRm) registers and opcode2 fields (armv6)
     */
    #define CpSPMperf 12 /* various counters */

<type CpSPM opcode2(arm) 448e>≡ (683c)
    #define CpSPMctl 0 /* performance monitor control */
    #define CpSPMcy 1 /* cycle counter register */

<_startpg() enable cycle counter(raspberry pi1)(arm) 448f>≡ (238a)
    /*
     * enable cycle counter
     */
    MOVW $1, R1
    MCR CpSC, 0, R1, C(CpSPM), C(CpSPMperf), CpSPMctl

<function arch_lcycles(raspberry pi1)(arm) 448g>≡ (716a)
    TEXT arch_lcycles(SB), 1, $-4
    MRC CpSC, 0, R0, C(CpSPM), C(CpSPMperf), CpSPMcy
    RET

<sysexec() locals 448h>+≡ (119d) <281e 450a>
    Tos *tos;

<sysexec() tos settings 448i>≡ (124c)
    tos = (Tos*)(TSTKTOP - sizeof(Tos));

    tos->cyclefreq = cpu->cyclefreq;
    arch_cycles((uulong*)&tos->pcycles);
    tos->pcycles = -tos->pcycles; // see comment above on Proc->pcycle
    tos->kcycles = tos->pcycles;
    tos->clock = 0;
    // what about other fields? like pid? will be set in kexit! but could be
    // done here? what about sysrfork? call kexit?

```

```

⟨arch__kexit() tos adjustments 449a⟩≡ (147a)
    uulong t;
    Tos *tos;

    /* precise time accounting, kernel exit */
    tos = (Tos*)(USTKTOP-offsetof(Tos));
    arch_cycles(&t);
    tos->kcycles += t - up->kentry;
    tos->pcycles = up->pcycles;
    tos->pid = up->pid;

⟨profclock tos adjustments when user 449b⟩≡ (446e)
    Tos *tos;
    tos = (Tos*)(USTKTOP-offsetof(Tos));
    tos->clock += TK2MS(1);

⟨syscall() adjust kentry 449c⟩≡ (141e)
    arch_cycles(&up->kentry);

⟨trap() adjust kentry when interrupt user 449d⟩≡ (146)
    arch_cycles(&up->kentry);

⟨arch_procsave() cycles adjustments 449e⟩≡ (201a)
    uulong t;

    arch_cycles(&t);
    p->pcycles += t;

⟨arch_procrestore cycles adjustments 449f⟩≡ (201b)
    arch_cycles(&t);
    p->pcycles -= t;

⟨sysexec() nbytes tos adjustments 449g⟩≡ (123c)
    nbytes += sizeof(Tos); /* hole for profiling clock at top of stack (and more) */

```

Chapter 21

Userspace System Programs

This short chapter covers the handful of programs that the kernel needs to get from bare hardware to a running shell. These programs are compiled into the kernel image itself (via the root filesystem) so they are available before any external storage is mounted. The most important mechanism here is the “sharpbang” (`#!`) support in `sysexec()`, which is needed because the boot script `boot.rc` is an `rc` script, and `sysexec()` must recognize the `#!` header to invoke the interpreter.

21.1 `/boot/boot`

21.2 `/root/init`

21.3 The shell

21.3.1 `#!`

When `sysexec()` reads the file header and finds `#!` instead of a valid binary magic number, it parses the interpreter path and arguments from the first line with `shargs()`, then restarts the exec with the interpreter as the target and the original script as an argument. The `indir` flag prevents recursive sharpbang processing—you cannot have a script whose interpreter is itself a script.

```
<sysexec() locals 450a>+≡ (119d) <448h
    char line[sizeof(Exec)];
    char *progarg[sizeof(Exec)/2+1];
    char progelem[64];

    bool indir = false;

<sysexec() process sharpbang 450b>≡ (119d)
    /*
     * Process #! /bin/sh args ...
     */
    memmove(line, &exec, sizeof(Exec));
    if(indir || line[0]!='#' || line[1]!='!')
        error(Ebadexec);
    n = shargs(line, n, progarg);
    if(n == 0)
        error(Ebadexec);
    indir = true;
    /*
     * First arg becomes complete file name
     */
```

```

progarg[n++] = file;
progarg[n] = 0;
validaddr(arg[1], BY2WD, true);
arg[1] += BY2WD;
file = progarg[0];
if(strlen(elem) >= sizeof progelem)
    error(Ebadexec);
strcpy(progelem, elem);
progarg[0] = progelem;
poperror();
cclose(tc);

```

⟨function shargs 451a⟩≡ (679a)

```

int
shargs(char *s, int n, char **ap)
{
    int i;

    s += 2;
    n -= 2;    /* skip #! */
    for(i=0; s[i]!='\n'; i++)
        if(i == n-1)
            return 0;
    s[i] = 0;
    *ap = 0;
    i = 0;
    for(;;) {
        while(*s==' ' || *s=='\t')
            s++;
        if(*s == 0)
            break;
        i++;
        *ap++ = s;
        *ap = 0;
        while(*s && *s!=' ' && *s!='\t')
            s++;
        if(*s == 0)
            break;
        else
            *s++ = 0;
    }
    return i;
}

```

⟨sysexec() if indir arg adjustments 451b⟩≡ (123c)

```

if(indir){
    argp = progarg;
    while(*argp){
        a = *argp++;
        nbytes += strlen(a) + 1;
        nargs++;
    }
}

```

⟨sysexec() if indir argp adjustments 451c⟩≡ (124c)

```

if(indir)
    argp = progarg;

```

$\langle \text{sysexec()} \text{ if indir argp adjustments again } 452 \rangle \equiv (124c)$

```
if(indir && *argp==nil) {  
    indir = false;  
    argp = (char**)arg[1];  
}
```

Chapter 22

Advanced Topics

In this chapter, I cover the remaining advanced aspects of the kernel: floating point management, CPU caches, the memory model, SMP support for the Raspberry Pi 2, real-time scheduling, and the 9P fileservers protocol. These topics were deferred from earlier chapters because they add complexity on top of the core mechanisms already presented.

22.1 Floating point

22.1.1 FPSaveStatus

The kernel uses lazy floating point context switching: the FPU is kept disabled by default, and only enabled when a process actually executes a floating point instruction. This avoids the cost of saving and restoring 32 double-precision registers on every context switch for processes that never use floating point. The `FPSaveStatus` enum tracks where each process stands in this lifecycle: `FPinit` means the process has never used the FPU, `FPactive` means the FPU is currently on for this process, and `FPinactive` means the FPU state has been saved and the hardware disabled. The `FPillegal` bit is or'd in during note (signal) handling to prevent note handlers from corrupting the interrupted code's FPU state.

```
<enum fpsavestatus 453a>≡ (668c)
/*
 * FPSave.status
 */
enum FPSaveStatus
{
    /* this is a state */
    FPinit= 0,

    FPactive= 1,
    FPinactive= 2,

    /* the following is a bit that can be or'd into the state */
    FPillegal= 0x100,
};
```

Each process carries an `fpstate` field that the scheduler hooks consult to decide whether FPU registers need saving or restoring. New processes start in `FPinit`, and a forked child inherits the parent's state minus the `FPillegal` bit so it is not penalized for something the parent did in a note handler.

```
<Proc other fields 453b>+≡ (49f) <269c 454d>
// enum<FPSaveStatus>
int fpstate;

<newproc() set fields of p 453c>+≡ (108c) <396a 502d>
<newproc() fp init 454a>
```

```

⟨newproc() fp init 454a⟩≡ (453c)
    p->fpstate = FPinit;

⟨sysrfork() when RFPROC propagate fields from up to p 454b⟩+≡ (112d) <429d 454e>
    ⟨sysrfork() propagate fpstate 454c⟩

⟨sysrfork() propagate fpstate 454c⟩≡ (454b)
    /* don't penalize the child, it hasn't done FP in a note handler. */
    p->fpstate = up->fpstate & ~FPillegal;

```

22.1.2 Arch_FPsave (ARM)

```

⟨Proc other fields 454d⟩+≡ (49f) <453b 533b>
    Arch_FPsave fpstate; /* address of this is known by db */

⟨sysrfork() when RFPROC propagate fields from up to p 454e⟩+≡ (112d) <454b 536e>
    ⟨sysrfork() propagate fpstate 454f⟩

⟨sysrfork() propagate fpstate 454f⟩≡ (454e)
    p->fpstate = up->fpstate;

⟨struct Arch_FPsave(arm) 454g⟩≡ (716d)
    /*
     * emulated or vfp3 floating point
     */
    struct Arch_FPsave
    {
        /*
         * vfp3 with ieee fp regs; uulong is sufficient for hardware but
         * each must be able to hold an Internal from fpi.h for sw emulation.
         */
        uulong regs[Maxfpregs][3];
        ⟨Proc Arch_FPsave other fields 454h⟩
    };

⟨Proc Arch_FPsave other fields 454h⟩≡ (454g)
    int fpstate; // vs Proc.fpstate??

    uulong status;
    uulong control;

    uintptr pc; /* of failed fp instr. */

⟨enum _anon_((arch/arm/dat_processes.h)(arm)) 454i⟩≡ (716d)
    enum {
        Maxfpregs = 32, /* could be 16 or 32, see Mach.fpnregs */
        Nfpctlregs = 16,
    };

```

22.1.3 Floating point hooks

The lazy FPU scheme plugs into the scheduler through a set of hooks called at key process lifecycle points. The flow is:

procsave()	--> fpupprocsave()	save FPU if FPactive, mark FPinactive
procrestore()	--> fpuprocrestore()	(no-op: restored lazily on fault)
sysrfork()	--> fpusysrfork()	save parent's FPU before fork
forkchild()	--> fpusysrforkchild()	copy state to child
sysexec()	--> fpusysprocsetup()	reset to FPinit, disable FPU

The key insight is that [fpuprocrestore] is intentionally empty. Rather than eagerly restoring FPU state when a process is rescheduled, the kernel leaves the FPU disabled. If the process tries to use floating point, it triggers an “undefined instruction” trap, and [fpuemu] restores the state at that point. This means processes that were using FPU before being descheduled but happen not to use it in their next quantum pay zero cost.

```
<arch_forkchild() floating point(arm) 455a>≡ (114b)
    fpusysrforkchild(p, cureg, up);
```

```
<function fpusysrforkchild(arm) 455b>≡ (717b)
/*
 * Called later in sysrfork() via the machine-dependent
 * sysrforkchild() routine.
 * Copy the parent FPU state to the child.
 */
void
fpusysrforkchild(Proc *p, Ureg *, Proc *up)
{
    /* don't penalize the child, it hasn't done FP in a note handler. */
    p->fpstate = up->fpstate & ~FPillegal;
}
```

```
<arch_procsetup() floating point(arm) 455c>≡ (121e)
    fpusysprocsetup(p);
```

```
<function fpusysprocsetup(arm) 455d>≡ (717b)
/*
 * Disable the FPU.
 * Called from sysexec() via sysprocsetup() to
 * set the FPU for the new process.
 */
void
fpusysprocsetup(Proc *p)
{
    p->fpstate = FPinit;
    fpoff();
}
```

```
<arch_procsave() floating point(arm) 455e>≡ (201a)
/* TODO: save and restore VFPv3 FP state once 5[cal] know the new registers.*/
fpuprocsave(p);
```

When a process is about to be descheduled, [fpuprocsave] saves the FPU registers only if the process was actively using them. A dying process (Moribund) gets its FPU cleared outright since its state will never be needed again. The subtle point in the comment is that [fpsave] cannot handle pending unmasked exceptions here because [postnote] requires the process lock, which [sleep] may already hold. Instead, exception handling is deferred until the process runs again and triggers a new FPU fault.

```
<function fpuprocsave(arm) 455f>≡ (717b)
/*
 * Called from sched() and sleep() via the machine-dependent
 * procsave() routine.
 * About to go in to the scheduler.
 * If the process wasn't using the FPU
 * there's nothing to do.
 */
void
fpuprocsave(Proc *p)
{
    if(p->fpstate == FPactive){
        if(p->state == Moribund)
```



```

        fpclear();
    else{
        /*
         * Fpsave() stores without handling pending
         * unmasked exeptions. Postnote() can't be called
         * here as sleep() already has up->rlock, so
         * the handling of pending exceptions is delayed
         * until the process runs again and generates an
         * emulation fault to activate the FPU.
         */
        fpsave(&p->fpsave);
    }
    p->fpstate = FPinactive;
}
}

```

<arch_procrestore() floating point(arm) 456a>≡ (201b)

```

    fpuprocrestore(p);

```

<function fpuprocrestore(arm) 456b>≡ (717b)

```

/*
 * The process has been rescheduled and is about to run.
 * Nothing to do here right now. If the process tries to use
 * the FPU again it will cause a Device Not Available
 * exception and the state will then be restored.
 */
void
fpuprocrestore(Proc *)
{
}

```

<syscall() if RFORK, handle floating point(arm) 456c>≡ (141e)

```

    if(scallnr == RFORK)
        fpusysrfork(ureg);

```

<function fpusysrfork(arm) 456d>≡ (717b)

```

/*
 * Called early in the non-interruptible path of
 * sysrfork() via the machine-dependent syscall() routine.
 * Save the state so that it can be easily copied
 * to the child process later.
 */
void
fpusysrfork(Ureg*)
{
    if(up->fpstate == FPactive){
        fpsave(&up->fpsave);
        up->fpstate = FPinactive;
    }
}

```

The actual register save loop iterates over all FPU registers (16 for VFPv2, 32 for VFPv3), reading each via [fpsavereg] which dynamically constructs a VSTR instruction. The FPSCR (floating point status and control register) is saved as both **status** and **control** so that pending exception flags and configuration bits are preserved separately.

<function fpsave(arm) 456e>≡ (717b)

```

/* should only be called if p->fpstate == FPactive */
void
fpsave(Arch_FPsave *fps)

```

```

{
    int n;

    fpon();
    fps->control = fps->status = fprd(Fpscr);
    assert(cpu->fpnregs);
    for (n = 0; n < cpu->fpnregs; n++)
        fpsavereg(n, (uvlong *)fps->regs[n]);
    fpoff();
}

```

Note handlers are forbidden from using the FPU because their execution is interposed between the interrupted code and its eventual resumption. If a note handler used floating point, it would silently corrupt the interrupted code's FPU registers. The `FPillegal` bit enforces this: `[fpunotify]` sets it before delivering the note, and if the handler tries to execute an FP instruction, `[fpuemu]` will see the bit and post an error note. When the handler finishes, `[fpunoted]` clears the bit.

`<function fpunotify(arm) 457a>`≡ (717b)

```

/*
 * Called when a note is about to be delivered to a
 * user process, usually at the end of a system call.
 * Note handlers are not allowed to use the FPU so
 * the state is marked (after saving if necessary) and
 * checked in the Device Not Available handler.
 */
void
fpunotify(Ureg*)
{
    if(up->fpstate == FPactive){
        fpsave(&up->fpsave);
        up->fpstate = FPinactive;
    }
    up->fpstate |= FPillegal;
}

```

`<function fpunoted(arm) 457b>`≡ (717b)

```

/*
 * Called from sysnoted() via the machine-dependent
 * noted() routine.
 * Clear the flag set above in fpunotify().
 */
void
fpunoted(void)
{
    up->fpstate &= ~FPillegal;
}

```

22.1.4 ARM floating point coprocessors (ARM)

On ARM, floating point is accessed through coprocessors 10 (single precision) and 11 (double precision). The kernel detects whether the hardware implements VFPv2 (16 registers) or VFPv3 (32 registers) and records this in `cpu->fpnregs`. This fork of Plan 9 relies on the compiler (51 -f) generating native VFP3 instructions, so the old software FPU emulation code (`fpiarm.c`) has been removed.

`<Coprocessors other cases(arm) 457c>`≡ (29a)

```

#define CpFP      10      /* float FP, VFP cfg. */
#define CpDFP     11      /* double FP */

```

```

⟨enum _anon_((processes/arm/vfp3.c)(arm)) 458a)≡ (717b)
/* subarchitecture code in cpu->havefp */
enum VFPKind {
    VFPv2    = 2,
    VFPv3    = 3,
};

⟨Cpu Arch float fields(arm) 458b)≡ (48e) 458c>
// option<VFPKind>, None = 0 /* vfp2 or vfp3 fpu */
int havefp;
bool havefpvalid;
// 16 for VFPv2, 32 for VFPv3
int fpnregs;

⟨Cpu Arch float fields(arm) 458c)+≡ (48e) <458b 458d>
bool fpon;

⟨Cpu Arch float fields(arm) 458d)+≡ (48e) <458c 463b>
ulong fpscr; /* sw copy */

```

22.1.5 coproc.c (ARM)

The ARM architecture does not provide general-purpose instructions to read and write individual VFP registers by index. Instead, the kernel uses a remarkable trick: it constructs a VMRS or VMSR instruction at runtime by writing the machine code into a local array, then calls that array as a function pointer. The `setupfpctlop` function patches the register number into the instruction encoding, flushes the data cache so the new instruction is visible to the instruction fetcher, and invalidates the instruction cache. This self-modifying-code technique avoids needing 32 separate save/restore functions.

```

⟨function setupfpctlop(arm) 458e)≡ (687b)
/* fp coproc control */
static void
setupfpctlop(ulong instr[2], int opcode, int fpctlreg)
{
    ulong instrsz[2];

    fpctlreg &= Nfpctlregs - 1;
    instr[0] = opcode | fpctlreg << 16 | 0 << 12 | CpFP << 8;
    instr[1] = RETinst;

    ⟨setupfpctlop() write back cache for modified instr(arm) 470m)
    ⟨setupfpctlop() invalidate instruction cache(arm) 470o)
}

⟨function fprd(arm) 458f)≡ (687b)
ulong
fprd(int fpreg)
{
    int s, r;
    volatile ulong instr[2];
    Pufv fp;

    if (!cpu->fpon) {
        arch_dumpstack();
        panic("fprd: cpu%d fpu off", cpu->cpuno);
    }
    s = arch_splhi();
    /*

```

```

    * VMRS.  return value will be in R0, which is convenient.
    * Rt will be R0.
    */
    setupfpctlop(instr, 0xeef00010, fpreg);
    fp = (Pufv)instr;
    r = fp();
    arch_splx(s);
    return r;
}

⟨function fpwr(arm) 459a⟩≡ (687b)
void
fpwr(int fpreg, ulong val)
{
    int s;
    volatile ulong instr[2];
    Pvfufp fp;

    /* fpu might be off and this VMSR might enable it */
    s = arch_splhi();
    setupfpctlop(instr, 0xeeee00010, fpreg);    /* VMSR, Rt is R0 */
    fp = (Pvfufp)instr;
    fp(val);
    arch_coherence();
    arch_splx(s);
}

⟨function setupfpop(arm) 459b⟩≡ (687b)
/* fp register access; don't bother with single precision */
static void
setupfpop(ulong instr[2], int opcode, int fpreg)
{
    ulong instrsz[2];

    instr[0] = opcode | 0 << 16 | (fpreg & (16 - 1)) << 12;
    if (fpreg >= 16)
        instr[0] |= 1 << 22;    /* high bit of dfp reg # */
    instr[1] = RETinst;

    ⟨setupfpop() write back cache for modified instr(arm) 470n⟩
    ⟨setupfpop() invalidate instruction cache(arm) 470p⟩
}

⟨function fprestreg(arm) 459c⟩≡ (687b)
void
fprestreg(int fpreg, uulong val)
{
    int s;
    volatile ulong instr[2];
    void (*fp)(uulong *);

    if (!cpu->fpon)
        panic("fprestreg: cpu%d fpu off", cpu->cpuno);
    s = arch_splhi();
    setupfpop(instr, 0xed100000 | CpDFP << 8, fpreg); /* VLDR, Rt is R0 */
    fp = (void (*)(uulong *))instr;
    fp(&val);
    arch_coherence();
    arch_splx(s);
}

```

```

⟨function fpsavereg(arm) 460a)≡ (687b)
    ulong
    fpsavereg(int fpreg, uulong *fpp)
    {
        int s, r;
        volatile ulong instr[2];
        ulong (*fp)(uulong *);

        if (!cpu->fpon)
            panic("fpsavereg: cpu%d fpu off", cpu->cpuno);
        s = arch_splhi();
        /*
         * VSTR.  pointer will be in R0, which is convenient.
         * Rt will be R0.
         */
        setupfpop(instr, 0xed000000 | CpDFP << 8, fpreg);
        fp = (ulong (*)(uulong *))instr;
        r = fp(fpp);
        arch_splx(s);
        arch_coherence();
        return r;          /* not too meaningful */
    }

```

22.1.6 vfp.c (ARM)

This subsection contains the VFP3 initialization and configuration code. The key function is [havefp], which probes the coprocessor access control register to determine what floating point hardware is present, then reads the FPSID register to distinguish VFPv2 from VFPv3. The [fpon]/[fpoff] functions enable and disable the FPU by writing the Fpexc (exception) register, while [fpcfg] sets the default FPSCR configuration: denormals-are-zero (Dn), flush-to-zero (Fz), and round-to-nearest (FPRNR).

```

⟨enum _anon_((processes/arm/vfp3.c)2(arm)) 460b)≡ (717b)
    /* fp control regs.  most are read-only */
    enum {
        Fpsid = 0,
        Fpscr = 1,          /* rw */
        Mvfr1 = 6,
        Mvfr0 = 7,
        Fpexc = 8,          /* rw */
        Fpinst = 9,         /* optional, for exceptions */
        Fpinst2 = 10,
    };

```

```

⟨enum _anon_((processes/arm/vfp3.c)3(arm)) 460c)≡ (717b)
    enum {
        /* Fpexc bits */
        Fpex = 1u << 31,
        Fpenabled = 1 << 30,
        Fpdex = 1 << 29,    /* defined synch exception */
        /* FSR bits appear here */
        Fpmbc = Fpdex,      /* bits exception handler must clear */

        /* Fpscr bits; see u.h for more */
        Stride = MASK(2) << 20,
        Len = MASK(3) << 16,
        Dn = 1 << 25,
        Fz = 1 << 24,
        /* trap exception enables (not allowed in vfp3) */
    };

```

```

    FPIDNRM = 1 << 15, /* input denormal */
    Alltraps = FPIDNRM | FPINEX | FPUNFL | FPOVFL | FPZDIV | FPINVAL,
    /* pending exceptions */
    FPAIDNRM = 1 << 7, /* input denormal */
    Allexc = FPAIDNRM | FPAINEX | FPAUNFL | FPAOVFL | FPAZDIV | FPAINVAL,
    /* condition codes */
    Allcc = MASK(4) << 28,
};

⟨enum _anon_((processes/arm/vfp3.c)4(arm)) 461a)≡ (717b)
enum {
    /* CpCPaccess bits */
    Cpaccnosimd = 1u << 31,
    Cpaccd16 = 1 << 30,
};

⟨function subarch(arm) 461b)≡ (717b)
static char *
subarch(int impl, uint sa)
{
    static char *armarchs[] = {
        "VFPv1 (unsupported)",
        "VFPv2",
        "VFPv3+ with common VFP subarch v2",
        "VFPv3+ with null subarch",
        "VFPv3+ with common VFP subarch v3",
    };

    if (impl != 'A' || sa >= nelem(armarchs))
        return "GOK";
    else
        return armarchs[sa];
}

⟨function implement(arm) 461c)≡ (717b)
static char *
implement(uchar impl)
{
    if (impl == 'A')
        return "arm";
    else
        return "unknown";
}

⟨CpCONTROL.opcode2 other cases(arm) 461d)≡ (29c) 478b▷
#define CpCPaccess 2

⟨function havefp(arm) 461e)≡ (717b)
static int
havefp(void)
{
    int gotfp;
    ulong acc, sid;

    if (cpu->havefpvalid)
        return cpu->havefp;

    cpu->havefp = 0;
    gotfp = 1 << CpFP | 1 << CpDFP;
    cpwsrc(0, CpCONTROL, 0, CpCPaccess, MASK(28));

```

```

acc = cprdsc(0, CpCONTROL, 0, CpCPaccess);
if ((acc & (MASK(2) << (2*CpFP))) == 0) {
    gotfp &= ~(1 << CpFP);
    print("fpon: no single FP coprocessor\n");
}
if ((acc & (MASK(2) << (2*CpDFP))) == 0) {
    gotfp &= ~(1 << CpDFP);
    print("fpon: no double FP coprocessor\n");
}
if (!gotfp) {
    print("fpon: no FP coprocessors\n");
    cpu->havefpvalid = true;
    return 0;
}
cpu->fpon = true;          /* don't panic */
sid = fprd(Fpsid);
cpu->fpon = false;

switch((sid >> 16) & MASK(7)){
case 0:                    /* VFPv1 */
    break;
case 1:                    /* VFPv2 */
    cpu->havefp = VFPv2;
    cpu->fpnregs = 16;
    break;
default:                   /* VFPv3 or later */
    cpu->havefp = VFPv3;
    cpu->fpnregs = (acc & Cpaccd16) ? 16 : 32;
    break;
}
if (cpu->cpuno == 0)
    print("fp: %d registers, %s simd\n", cpu->fpnregs,
        (acc & Cpaccnosimd? " no": ""));
cpu->havefpvalid = true;
return 1;
}

⟨function fpoff(arm) 462a⟩≡ (717b)
/*
 * these can be called to turn the fpu on or off for user procs,
 * not just at system start up or shutdown.
 */
void
fpoff(void)
{
    if (cpu->fpon) {
        fpwr(Fpexc, 0);
        cpu->fpon = false;
    }
}

⟨function fpononly(arm) 462b⟩≡ (717b)
void
fpononly(void)
{
    if (!cpu->fpon && havefp()) {
        /* enable fp. must be first operation on the FPUs. */
        fpwr(Fpexc, Fpenabled);
        cpu->fpon = true;
    }
}

```

⟨function fpcfg(*arm*) 463a⟩≡ (717b)

```
static void
fpcfg(void)
{
    int impl;
    ulong sid;
    static int printed;

    /* clear pending exceptions; no traps in vfp3; all v7 ops are scalar */
    cpu->fpscr = Dn | Fz | FPRNR | (FPINVAL | FPZDIV | FPOVFL) & ~Alltraps;
    fpwr(Fpscr, cpu->fpscr);
    cpu->fpconfiged = true;

    if (printed)
        return;
    sid = fprd(Fpsid);
    impl = sid >> 24;
    print("fp: %s arch %s; rev %ld\n", implement(impl),
        subarch(impl, (sid >> 16) & MASK(7)), sid & MASK(4));
    printed = 1;
}
```

⟨Cpu Arch float fields(*arm*) 463b⟩+≡ (48e) ◁458d 464b▷

```
bool fpconfiged;
```

⟨function fpinit(*arm*) 463c⟩≡ (717b)

```
void
fpinit(void)
{
    if (havefp()) {
        fpononly();
        fpcfg();
    }
}
```

⟨function fpon(*arm*) 463d⟩≡ (717b)

```
void
fpon(void)
{
    if (havefp()) {
        fpononly();
        if (cpu->fpconfiged)
            fpwr(Fpscr, (fprd(Fpscr) & Allcc) | cpu->fpscr);
        else
            fpcfg(); /* 1st time on this fpu; configure it */
    }
}
```

⟨function fpclear(*arm*) 463e⟩≡ (717b)

```
void
fpclear(void)
{
    // ulong scr;

    fpon();
    // scr = fprd(Fpscr);
    // cpu->fpscr = scr & ~Allexc;
    // fpwr(Fpscr, cpu->fpscr);

    fpwr(Fpexc, fprd(Fpexc) & ~Fpmbc);
}
```


<function fpstuck(arm) 464a>≡ (717b)

```
void
fpstuck(uintptr pc)
{
    if (cpu->fppc == pc && cpu->fppid == up->pid) {
        cpu->fpcnt++;
        if (cpu->fpcnt > 4)
            panic("fpuemu: cpu%d stuck at pid %ld %s pc %#p "
                  "instr %#8lux", cpu->cpuno, up->pid, up->text,
                  pc, *(ulong *)pc);
    } else {
        cpu->fppid = up->pid;
        cpu->fppc = pc;
        cpu->fpcnt = 0;
    }
}
```

<Cpu Arch float fields(arm) 464b>+≡ (48e) <463b

```
int fppid;          /* pid of last fault */
uintptr fppc;       /* addr of last fault */
int fpcnt;          /* how many consecutive at that addr */
```

<enum _anon_((processes/arm/vfp3.c)5(arm)) 464c>≡ (717b)

```
enum {
    N = 1<<31,
    Z = 1<<30,
    C = 1<<29,
    V = 1<<28,
    REGPC = 15,
};
```

<function condok(arm) 464d>≡ (717b)

```
static int
condok(int cc, int c)
{
    switch(c){
    case 0: /* Z set */
        return cc&Z;
    case 1: /* Z clear */
        return (cc&Z) == 0;
    case 2: /* C set */
        return cc&C;
    case 3: /* C clear */
        return (cc&C) == 0;
    case 4: /* N set */
        return cc&N;
    case 5: /* N clear */
        return (cc&N) == 0;
    case 6: /* V set */
        return cc&V;
    case 7: /* V clear */
        return (cc&V) == 0;
    case 8: /* C set and Z clear */
        return cc&C && (cc&Z) == 0;
    case 9: /* C clear or Z set */
        return (cc&C) == 0 || cc&Z;
    case 10: /* N set and V set, or N clear and V clear */
        return (~cc&(N|V))==0 || (cc&(N|V)) == 0;
    case 11: /* N set and V clear, or N clear and V set */
        return (cc&(N|V))==N || (cc&(N|V))==V;
    }
```

```

case 12:    /* Z clear, and either N set and V set or N clear and V clear */
    return (cc&Z) == 0 && ((~cc&(N|V))==0 || (cc&(N|V))==0);
case 13:    /* Z set, or N set and V clear or N clear and V set */
    return (cc&Z) || (cc&(N|V))==N || (cc&(N|V))==V;
case 14:    /* always */
    return 1;
case 15:    /* never (reserved) */
    return 0;
}
return 0;    /* not reached */
}

```

```

⟨macro ISCPPOP(arm) 465a)≡ (717b)
#define ISCPPOP(op) ((op) == 0xE || ((op) & ~1) == 0xC)

```

```

⟨macro ISVFPOP(arm) 465b)≡ (717b)
#define ISVFPOP(cp, op) (((cp) == CpDFP || (cp) == CpFP) && ISCPPOP(op))

```

When a user process executes a floating point instruction with the FPU disabled, the CPU raises an “undefined instruction” exception. The trap handler calls [fpuemu], which is the entry point for the lazy FPU scheme. It checks whether the faulting instruction is actually a VFP operation (using ISVFPOP), and if so calls [mathemu] to enable the FPU and let the instruction be retried. The [fpstuck] function is a debugging safeguard: if the same process faults repeatedly at the same PC, something is wrong and the kernel panics rather than looping forever.

```

⟨function fpuemu(arm) 465c)≡ (717b)
/* only called to deal with user-mode instruction faults */
int
fpuemu(Ureg* ureg)
{
    int s, nfp, cop, op;
    uintptr pc;

    if(waserror()){
        postnote(up, 1, up->errstr, NDebug);
        return 1;
    }

    if(up->fpstate & FPillegal)
        error("floating point in note handler");

    nfp = 0;
    pc = ureg->pc;
    validaddr(pc, 4, 0);
    if(!condok(ureg->psr, *(ulong*)pc >> 28))
        iprint("fpuemu: conditional instr shouldn't have got here\n");
    op = (*(ulong*)pc >> 24) & MASK(4);
    cop = (*(ulong*)pc >> 8) & MASK(4);
    if(cpu->fpon)
        fpstuck(pc);    /* debugging; could move down 1 line */
    if (ISVFPOP(cop, op)) {    /* if vfp, fpu must be off */
        mathemu(ureg);    /* enable fpu & retry */
        nfp = 1;
    }

    poperror();
    return nfp;
}

```

```

⟨trap() other locals(arm) 465d)+≡ (146) <149c
int rv;

```

```

⟨trap() when undefined instruction, if float instr(arm) 466a)≡ (148b)
/* look for floating point instructions to interpret */
rv = fpuemu(ureg);

```

22.1.7 mathxxx() (ARM)

The [mathemu] function implements the FPU state machine. On FPinit (first-ever use), it calls [fpinit] to configure the hardware from scratch. On FPinactive (previously saved state), it checks for pending exceptions before restoring—if there are exceptions, it delivers a note to the process instead of restoring. On FPactive (FPU already on), the fault is a genuine illegal instruction. After handling any state transition, [fpclear] clears pending exceptions so the retried instruction starts clean.

```

⟨function mathnote(arm) 466b)≡ (717b)
static void
mathnote(void)
{
    ulong status;
    char *msg, note[ERRMAX];

    status = up->fpsave.status;

    /*
     * Some attention should probably be paid here to the
     * exception masks and error summary.
     */
    if (status & FPAINEX)
        msg = "inexact";
    else if (status & FPAOVFL)
        msg = "overflow";
    else if (status & FPAUNFL)
        msg = "underflow";
    else if (status & FPAZDIV)
        msg = "divide by zero";
    else if (status & FPAINVAL)
        msg = "bad operation";
    else
        msg = "spurious";
    snprintf(note, sizeof note, "sys: fp: %s fppc=%#p status=%#lux",
        msg, up->fpsave.pc, status);
    postnote(up, 1, note, NDebug);
}

```

```

⟨function mathemu(arm) 466c)≡ (717b)
static void
mathemu(Ureg *)
{
    switch(up->fpstate){
    case FPinit:
        fpinit();
        up->fpstate = FPactive;
        break;
    case FPinactive:
        /*
         * Before restoring the state, check for any pending
         * exceptions. There's no way to restore the state without
         * generating an unmasked exception.
         * More attention should probably be paid here to the
         * exception masks and error summary.

```

```

    */
    if(up->fpsave.status & (FPAINEX|FPAUNFL|FPAOVFL|FPAZDIV|FPAINVAL)){
        mathnote();
        break;
    }
    fprestore(up);
    up->fpstate = FPActive;
    break;
case FPActive:
    error("illegal instruction: bad vfp fpu opcode");
    break;
}
fpclear();
}

```

```

⟨function fprestore(arm) 467a⟩≡ (717b)
static void
fprestore(Proc *p)
{
    int n;

    fpon();
    fpwr(Fpscr, p->fpsave.control);
    cpu->fpscr = fprd(Fpscr) & ~Allcc;
    assert(cpu->fpnregs);
    for (n = 0; n < cpu->fpnregs; n++)
        fprestreg(n, *(uvlong *)p->fpsave.regs[n]);
}

```

22.1.8 /proc/x/fpregs

```

⟨procread() Qfpregs case 467b⟩≡ (540c)
case Qfpregs:
    rptra = (uchar*)&p->fpsave;
    rsize = sizeof(Arch_FPsave);
    goto regread;

⟨procwrite Qfpregs case 467c⟩≡ (543)
case Qfpregs:
    if(offset >= sizeof(Arch_FPsave))
        n = 0;
    else if(offset+n > sizeof(Arch_FPsave))
        n = sizeof(Arch_FPsave) - offset;
    memmove((uchar*)&p->fpsave+offset, va, n);
    break;

```

22.2 CPU cache

CPU caches are small fast memories that sit between the processor and main RAM. The ARM has separate L1 caches for instructions (I-cache) and data (D-cache), which means code modifications (like the self-modifying FPU trick above, or writing exception vectors) require explicit cache maintenance: the D-cache must be written back so the new data reaches memory, and the I-cache must be invalidated so the processor fetches the updated instructions. The kernel must also manage caches around MMU operations, because changing page tables can make cached virtual-to-physical translations stale.

22.2.1 Instruction and data cache (ARM)

<CpSC primary registers other cases(arm) 468a>≡ (29b) 473g▷
#define CpCACHE 7 /* cache/write buffer control */

<CpCONTROL.CpMainctl other cases(arm) 468b>≡ (29d) 477a▷
#define CpCdcache 0x00000004 /* C: data cache on */
#define CpCicache 0x00001000 /* I: instruction cache on */

<armstart() disable features(arm) 468c>≡ (236)
BIC \$(CpCdcache|CpCicache), R1

<armstart() invalidate caches(arm) 468d>≡ (236)
MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvu), CpCACHEall

<armstart() reenale features(arm) 468e>≡ (236)
ORR \$(CpCdcache|CpCicache), R0

22.2.2 Cache policy

The cache can operate in write-through mode (every store goes immediately to memory) or write-back mode (stores stay in the cache and are written to memory later). Write-back is more efficient but requires explicit “writeback” operations before other bus masters (DMA, other CPUs) need to see the data. The **Buffered** and **Cached** page table entry attributes select write-back caching for normal DRAM pages. Memory-mapped I/O regions must not be cached, since reading a device register should always go to the hardware, not return a stale cached value.

<type CpCACHERANGE opcode2(arm) 468f>≡ (683c)
/*
 * CpCACHERANGE opcode2 fields for MCRR instruction (armv6)
 */
#define CpCACHERANGEinvd 6 /* invalidate data */
#define CpCACHERANGEDwb 12 /* writeback */
#define CpCACHERANGEDwbi 14 /* writeback+invalidate */

<instruction MCRR(arm) 468g>≡ (684)
#define MCRR(coproc, op, rd, rn, crm) \
WORD \$(0xec400000|(rn)<<16|(rd)<<12|(coproc)<<8|(op)<<4|(crm))

22.2.3 Paging and caching (ARM)

<type PageTableEntryAttribute1(arm) 468h>≡ (683c)
#define Buffered 0x00000004 /* L[12]: write-back not -thru */
#define Cached 0x00000008 /* L[12] */

<Soc mmu fields 468i>≡ (54c)
u32int l1ptedramattr;
u32int l2ptedramattr;

<soc(raspberry pi1) other fields(arm) 468j>≡ (54d) 475e▷
.l1ptedramattr = Cached | Buffered,
.l2ptedramattr = Cached | Buffered,

<constant L1ptedramattr(arm) 468k>≡ (711a)
#define L1ptedramattr soc.l1ptedramattr

<constant L2ptedramattr(arm) 468l>≡ (711a)
#define L2ptedramattr soc.l2ptedramattr

22.2.4 Memory barriers (ARM)

On ARMv6, ISB and DSB are implemented via MCR instructions to coprocessor 7, not as dedicated instructions. ISB invalidates the instruction prefetch buffer, DSB drains the write buffer. They are combined into the **BARRIERS** macro used in boot code.

```
<instruction ISB(arm) 469a>≡ (684)
#define ISB \
    MOVW    $0, R0; \
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvi), CpCACHWait
```

```
<instruction DSB(arm) 469b>≡ (684)
#define DSB \
    MOVW    $0, R0; \
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHewb), CpCACHWait
```

22.2.5 Cache operations (ARM)

The ARM cache operations are accessed through MCR/MRC instructions to coprocessor 7 (CpCACHE). The terminology is confusing: ARM’s “flush” means invalidate (discard cached data without writing back), and “clean” means writeback (push dirty data to memory). The operations come in three granularities: entire cache (CpCACHEall), single cache line by virtual address (CpCACHEse), and by set/way index (CpCACHesi). The kernel provides C-callable wrappers: **cachedwb** (writeback), **cachedwbinv** (writeback + invalidate), **cachedwbse** (writeback a range), **cacheiinv** (invalidate I-cache), and **cacheuwbinv** (unified: writeback D-cache + invalidate I-cache).

```
<type CpCACHE secondary registers(arm) 469c>≡ (683c)
/*
 * CpCACHE Secondary (CRm) registers and opcode2 fields. op1==0.
 * In ARM-speak, 'flush' means invalidate and 'clean' means writeback.
 */
#define CpCACHEintr 0      /* interrupt (op2==4) */
#define CpCACHEinvi 5      /* instruction, branch table */
#define CpCACHEinvd 6      /* data or unified */
#define CpCACHEinvu 7      /* unified (not on cortex) */
#define CpCACHewb 10       /* writeback to PoC */
#define CpCACHewbi 14      /* writeback+invalidate (to PoC) */
```

```
<type CpCACHE opcode2(arm) 469d>≡ (683c)
#define CpCACHEall 0      /* entire (not for invd nor wb(i) on cortex) */
#define CpCACHEse 1       /* single entry */
#define CpCACHesi 2       /* set/index (set/way) */
#define CpCACHewait 4      /* wait (prefetch flush on cortex) */
```

```
<constant CACHELINESZ(raspberry pi1)(ram) 469e>≡ (711c)
#define CACHELINESZ 32
```

22.2.6 cachexxx() (ARM)

The cache operations are called throughout the kernel at points where coherency between caches, memory, and the MMU must be maintained. The most common pattern is after modifying page table entries: **cachedwbse** writes back the modified PTEs so the MMU’s page table walker sees them. Before an MMU switch (**arch_mmuswitch**), **cacheuwbinv** flushes everything because virtual addresses are about to change meaning. The self-modifying FPU code calls **cachedwbse** followed by **cacheiinv** to ensure the dynamically constructed instructions reach the I-cache.

```
<arch_mmuswitch() write back and invalidate cache before switch(arm) 469f>≡ (200g)
/* write back dirty and invalidate l1 caches */
cacheuwbinv();
```

<code><arch_mmurelease() write back cache after adjusting page tables(<i>arm</i>) 470a>≡</code>	<code>(157f)</code>
<code>/* make sure map is in memory */</code>	
<code>/* could be smarter about how much? */</code>	
<code>cachedwbse(&cpu->mmul1[L1X(UZERO)], (L1hi - L1lo)*sizeof(PTE));</code>	
<code><arch_mmuswitch() write back cache after adjusting page tables(<i>arm</i>) 470b>≡</code>	<code>(200g)</code>
<code>/* make sure map is in memory */</code>	
<code>/* could be smarter about how much? */</code>	
<code>cachedwbse(&l1[L1X(UZERO)], (L1hi - L1lo)*sizeof(PTE));</code>	
<code><mmuinit1() write back cache after adjusting page tables(<i>arm</i>) 470c>≡</code>	<code>(246a)</code>
<code>cachedwbse(&l1[L1X(0)], sizeof(PTE));</code>	
<code><mmukmap() write back cache after changing page tables(<i>arm</i>) 470d>≡</code>	<code>(338c)</code>
<code>cachedwbse(pte0, (uintptr)pte - (uintptr)pte0);</code>	
<code><arch_putmmu() write back cache after changing pde(<i>arm</i>) 470e>≡</code>	<code>(162)</code>
<code>cachedwbse(l1, sizeof *l1);</code>	
<code><arch_putmmu() write back cache after changing pte(<i>arm</i>) 470f>≡</code>	<code>(162)</code>
<code>cachedwbse(&pte[L2X(va)], sizeof pte[0]);</code>	
<code><arch_putmmu() write back cache before fixing va fault(<i>arm</i>) 470g>≡</code>	<code>(162)</code>
<code>/* force l2 page to memory */</code>	
<code>cachedwbse((void *)pg->va, BY2PG);</code>	
<code><arch_mmurelease() invalidate cache before adjusting page tables(<i>arm</i>) 470h>≡</code>	<code>(157f)</code>
<code>/* write back dirty and invalidate l1 caches */</code>	
<code>cacheuwbinv();</code>	
<code><arch_kunmap() write back cache for page(<i>arm</i>) 470i>≡</code>	<code>(164d)</code>
<code>cachedwbinvse(k, BY2PG);</code>	
<code><arch__trapinit() invalidate cache after adjust vectors(<i>arm</i>) 470j>≡</code>	<code>(243f)</code>
<code>cacheuwbinv();</code>	
<code>l2cacheuwbinv();</code>	
<code><fbinit() write back cache after modified fb info(<i>arm</i>) 470k>≡</code>	<code>(338b)</code>
<code>cachedwbinvse(fi, sizeof(Fbinfo));</code>	
<code><arch__kexit() write back cache(<i>arm</i>) 470l>≡</code>	<code>(147a)</code>
<code>/* make visible immediately to user proc */</code>	
<code>cachedwbinvse(tos, sizeof *tos);</code>	
<code><setupfpctlop() write back cache for modified instr(<i>arm</i>) 470m>≡</code>	<code>(458e)</code>
<code>cachedwbse(instr, sizeof instrsz);</code>	
<code><setupfpop() write back cache for modified instr(<i>arm</i>) 470n>≡</code>	<code>(459b)</code>
<code>cachedwbse(instr, sizeof instrsz);</code>	
<code><setupfpctlop() invalidate instruction cache(<i>arm</i>) 470o>≡</code>	<code>(458e)</code>
<code>cacheiinv();</code>	
<code><setupfpop() invalidate instruction cache(<i>arm</i>) 470p>≡</code>	<code>(459b)</code>
<code>cacheiinv();</code>	

<function cachexxx(raspberry pi1)(ram) 471>≡ (711c)

```

/*
 * drain write buffer
 * writeback data cache
 */
TEXT cachedwb(SB), 1, $-4
    DSB
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEall
    RET

/*
 * drain write buffer
 * writeback and invalidate data cache
 */
TEXT cachedwbinv(SB), 1, $-4
    DSB
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwbi), CpCACHEall
    RET

/*
 * cachedwbinvse(va, n)
 *   drain write buffer
 *   writeback and invalidate data cache range [va, va+n)
 */
TEXT cachedwbinvse(SB), 1, $-4
    MOVW    R0, R1      /* DSB clears R0 */
    DSB
    MOVW    n+4(FP), R2
    ADD R1, R2
    SUB $1, R2
    BIC $(CACHELINESZ-1), R1
    BIC $(CACHELINESZ-1), R2
    MCRR(CpSC, 0, 2, 1, CpCACHERANGEDwbi)
    RET

/*
 * cachedwbse(va, n)
 *   drain write buffer
 *   writeback data cache range [va, va+n)
 */
TEXT cachedwbse(SB), 1, $-4
    MOVW    R0, R1      /* DSB clears R0 */
    DSB
    MOVW    n+4(FP), R2
    ADD R1, R2
    BIC $(CACHELINESZ-1), R1
    BIC $(CACHELINESZ-1), R2
    MCRR(CpSC, 0, 2, 1, CpCACHERANGEDwb)
    RET

/*
 * cachedinvse(va, n)
 *   drain write buffer
 *   invalidate data cache range [va, va+n)
 */
TEXT cachedinvse(SB), 1, $-4
    MOVW    R0, R1      /* DSB clears R0 */
    DSB

```



```

    MOVW    n+4(FP), R2
    ADD R1, R2
    SUB $1, R2
    BIC $(CACHELINESZ-1), R1
    BIC $(CACHELINESZ-1), R2
    MCRR(CpSC, 0, 2, 1, CpCACHERANGEinvd)
    RET

/*
 * drain write buffer and prefetch buffer
 * writeback and invalidate data cache
 * invalidate instruction cache
 */
TEXT cacheuwbinv(SB), 1, $-4
    BARRIERS
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwbi), CpCACHEall
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvi), CpCACHEall
    RET

/*
 * L2 cache is not enabled
 */
TEXT l2cacheuwbinv(SB), 1, $-4
    RET

/*
 * invalidate instruction cache
 */
TEXT cacheiinv(SB), 1, $-4
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvi), CpCACHEall
    RET

```

22.3 Memory model

Modern processors do not necessarily perform memory operations in program order. The CPU may reorder loads and stores for performance, and with SMP the situation is worse because each processor has its own cache. Memory barriers force ordering: a Data Synchronization Barrier (DSB) ensures all preceding stores are visible before any following ones, and an Instruction Synchronization Barrier (ISB) flushes the instruction pipeline. The kernel's `arch_coherence` function issues both barriers and is called at critical points: after releasing a lock (so other processors see the unlocked state), around interrupt dispatch, and after compare-and-swap operations.

22.3.1 `arch_coherence()`

```

<unlock() after modified key, call coherence 472a>≡ (60c)
    // for processor caches, to ensure the lock value is seen by other
    // processors so that if they were doing while(l->key) { ... } they
    // can finally exit the while loop.
    arch_coherence();

<iunlock() after modified key, call coherence 472b>≡ (62a)
    arch_coherence();

<irq() before dispatch(arm) 472c>≡ (154c)
    arch_coherence();

```

```

⟨irq() after dispatch(arm) 473a⟩≡ (154c)
    arch_coherence();

⟨arch__trapinit() coherence(arm) 473b⟩≡ (243f)
    arch_coherence();

⟨init0() coherence(arm) 473c⟩≡ (252)
    arch_coherence();

⟨cas32() if address contained old value, call coherence(arm) 473d⟩≡ (417d)
    if (r)
        arch_coherence();

```

22.3.2 Memory barriers (ARM)

```

⟨instruction BARRIERS(arm) 473e⟩≡ (684)
    #define BARRIERS    ISB; DSB

```

22.4 The Raspberry Pi 2 machine

The Raspberry Pi 2 uses a BCM2836 SoC with four Cortex-A7 cores, 1 GB of RAM, and peripheral I/O remapped to 0x3F000000 (versus 0x20000000 on the Pi 1). Most of the kernel is portable between Pi 1 and Pi 2, but SMP support, the different cache line size (64 bytes vs 32), the generic ARM timer, per-CPU local registers, and the LDREX/STREX atomic instructions are Pi 2-specific. The Soc structure captures these hardware differences so the same kernel source can target both machines.

```

⟨global soc(raspberry pi2)(arm) 473f⟩≡ (707b)
    Soc soc = {
        .dramsize    = 1024*MiB,
        .physio      = 0x3F000000,
        ⟨soc(raspberry pi2) other fields(arm) 474c⟩
    };

```

22.4.1 CPU ID

```

⟨CpSC primary registers other cases(arm) 473g⟩+≡ (29b) <468a 475h>
    #define CpID      0          /* ID and cache type */

```

```

⟨type CpID secondary registers(arm) 473h⟩≡ (683c)
    /*
     * CpID Secondary (CRm) registers.
     */
    #define CpIDidct    0
    #define CpIDfeat    1

```

```

⟨type CpID opcode2(arm) 473i⟩≡ (683c)
    /*
     * CpID op1==0 opcode2 fields.
     * the cortex has more op1 codes for cache size, etc.
     */
    #define CpIDid      0          /* main ID */
    #define CpIDmpid    5          /* multiprocessor id (cortex) */

```

```

<instruction CPUID(arm) 474a>≡ (684)
/*
 * get cpu id, or zero if armv6
 */
#define CPUID(r) \
    MRC CpSC, 0, r, C(CpID), C(CpIDfeat), 7; \
    CMP $0, r; \
    B.EQ    2(PC); \
    MRC CpSC, 0, r, C(CpID), C(CpIDidct), CpIDmpid; \
    AND.S   $(MAXCPUS-1), r

```

22.4.2 MMU

On the Pi 2, page table entries for DRAM include additional attributes: write-allocate (so cache lines are allocated on writes, not just reads) and sharable (so the cache coherency protocol keeps the entry consistent across CPUs). These are critical for SMP correctness—without the sharable bit, one CPU’s cache could hold stale data that another CPU has modified.

```

<type PageTableEntryAttribute2(arm) 474b>≡ (683c)
#define L1wralloc    (1<<12)        /* L1 TEX */
#define L1sharable   (1<<16)
#define L2wralloc    (1<<6)          /* L2 TEX (small pages) */
#define L2sharable   (1<<10)

```

```

<soc(raspberry pi2) other fields(arm) 474c>≡ (473f) 475f>
.l1ptedramattrs = Cached | Buffered | L1wralloc | L1sharable,
.l2ptedramattrs = Cached | Buffered | L2wralloc | L2sharable,

```

```

<function l2ap(raspberrypi2)(arm) 474d>≡ (707b)
int
l2ap(int ap)
{
    return (AP(0, (ap)));
}

```

22.4.3 TTB

```

<type CpTTB(arm) 474e>≡ (683c)
/*
 * CpTTB op1==0, Crm==0 opcode2 values.
 */
#define CpTTB1      1                /* cortex */

```

```

<type CpTTB cache control bits(arm) 474f>≡ (683c)
/*
 * CpTTB cache control bits
 */
#define CpTTBs      (1<<1) /* page table in shareable memory */

```

22.4.4 Local registers

The BCM2836 adds per-CPU “local” registers at physical address 0x40000000, above the main 1 GB RAM. These include per-CPU mailboxes used to start secondary processors during boot: CPU 0 writes the entry

point address into a secondary CPU's `startcpu` mailbox, and that CPU begins executing at that address. The mailboxes are also used for inter-processor interrupts.

```
<struct Mbox(arm) 475a>≡ (707b)
/*
 * Arm local regs for SMP
 */
struct Mbox {
    u32int  doorbell;
    u32int  mbox1;
    u32int  mbox2;
    u32int  startcpu;
};
```

```
<struct Mboxes(arm) 475b>≡ (707b)
struct Mboxes {
    Mbox    set[4];
    Mbox    clr[4];
};
```

```
<constant ARMLocal(arm) 475c>≡ (715b 707b)
#define ARMLocal    (VIRTIO+IOSIZE)
```

```
<Soc other fields 475d>≡ (54c)
uintptr armlocal;
```

```
<soc(raspberry pi1) other fields(arm) 475e>+≡ (54d) <468j 790f>
.armlocal    = 0,
```

```
<soc(raspberry pi2) other fields(arm) 475f>+≡ (473f) <474c 790g>
.armlocal    = 0x40000000,
```

```
<mmuinit() set armlocal virtual memory mapping(arm) 475g>≡ (237c)
// for raspi2
pa = soc.armlocal;
if(pa)
    l1[L1X(va)] = pa|Dom0|L1AP(Krw)|Section;
```

22.4.5 Timer

```
<CpSC primary registers other cases(arm) 475h>+≡ (29b) <473g 475j>
#define CpTIMER    14          /* Generic timer (cortex-a7) */
```

```
<type CpTIMER(arm) 475i>≡ (683c)
/*
 * CpTIMER op1==0 Crm and opcode2 registers (cortex-a7)
 */
#define CpTIMERphys    2

#define CpTIMERphysval  0
#define CpTIMERphysctl  1
```

22.4.6 Performance monitor

```
<CpSC primary registers other cases(arm) 475j>+≡ (29b) <475h>
#define CpCLD        9          /* Performance monitor
```

```

<type CpCLD secondary registers(arm) 476a>≡ (683c)
/*
 * CpCLD Secondary (CRm) registers and opcode2 fields for op1==0. (cortex)
 */
#define CpCLDena    12          /* enables */
#define CpCLDcyc    13          /* cycle counter */

<type CpCLD opcode2(arm) 476b>≡ (683c)
#define CpCLDenapmnc    0
#define CpCLDenacyc    1

```

22.5 SMP (ARM)

Adding SMP support to the kernel requires changes at many levels: the boot sequence must start secondary processors, each processor needs its own `Cpu` structure and page tables, locks must use atomic instructions (LDREX/STREX instead of SWP), memory barriers must be inserted around lock operations, and the timer/interrupt subsystems must handle per-CPU local interrupts. The overall structure remains the same—each CPU ends up calling `[schedinit]` and picking up processes from the shared run queue.

22.5.1 Initialization adjustments

Basic pagination `cpu0`

The Pi 2 boot sequence for CPU 0 is almost identical to the Pi 1: disable MMU and caches, clear the page table area, call `[mmuinit]`, then enable the MMU and jump to virtual addresses. The main difference is that the SMP coherency bit (`CpACsmp`) must be set in the auxiliary control register before enabling caches, so that L1 caches participate in the hardware coherency protocol. The very first thing `armstart` does is read the CPU ID: if this is not CPU 0, it branches to `reset` for secondary CPU startup.

```

<function armstart(raspberry pi2)(arm) 476c>≡ (703c)
TEXT armstart(SB), 1, $-4

/*
 * if not cpu0, go to secondary startup
 */
CPUID(R1)
BNE reset

/*
 * disable the mmu and caches
 * invalidate tlb
 */
MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
BIC $(CpCdcache|CpCicache|CpCmmu), R1
ORR $(CpCsbo|CpCsw), R1
BIC $CpCsbz, R1
MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinu
ISB

/*
 * clear mach and page tables
 */
MOVW    $PADDR(CPUADDR), R1
MOVW    $PADDR(KTZERO), R2
_ramZ:

```

```

MOVW    R0, (R1)
ADD $4, R1
CMP R1, R2
BNE _ramZ

/*
 * start stack at top of mach (physical addr)
 * set up page tables for kernel
 */
MOVW    $PADDR(CPUADDR+CPUSIZE-4), R13
MOVW    $PADDR(L1), R0
BL      mmuinit(SB)

/*
 * set up domain access control and page table base
 */
MOVW    $Client, R1
MCR CpSC, 0, R1, C(CpDAC), C(0)
MOVW    $PADDR(L1), R1
ORR     $(CpTTBs/*|CpTTBowba|CpTTBiwba*/), R1
MCR CpSC, 0, R1, C(CpTTB), C(0)
MCR CpSC, 0, R1, C(CpTTB), C(0), CpTTB1 /* cortex has two */

/*
 * invalidate my caches before enabling
 */
BL      cachedinv(SB)
BL      cacheiinv(SB)
BL      l2cacheuinv(SB)
BARRIERS

/*
 * enable caches, mmu, and high vectors
 */
MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpAuxctl
ORR $CpACsmp, R1 /* turn SMP on */
MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpAuxctl
BARRIERS

MRC CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
ORR $(CpChv|CpCdcache|CpCicache|CpCmmu), R0
MCR CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
BARRIERS

/*
 * switch SB, SP, and PC into KZERO space
 */
MOVW    $setR12(SB), R12
MOVW    $(CPUADDR+CPUSIZE-4), R13
MOVW    $_startpg(SB), R15

```

$\langle$ CpCONTROL.CpMainctl other cases(*arm*) 477a $\rangle$ +≡ (29d) $\triangleleft$ 468b 477b $\triangleright$
 #define CpCsw (1<<10) /* SW: SWP(B) enable (deprecated in v7) */
 #define CpCpredict 0x00000800 /* Z: branch prediction (armv7) */

$\langle$ CpCONTROL.CpMainctl other cases(*arm*) 477b $\rangle$ +≡ (29d) $\triangleleft$ 477a 478a $\triangleright$
 #define CpCve (1<<24) /* VE: intr vectors enable */
 #define CpCtre (1<<28) /* TRE: TEX remap enable */

```

<CpCONTROL.CpMainctl other cases(arm) 478a>+≡ (29d) <477b
#define CpCsbo (3<<22|1<<18|1<<16|017<<3) /* must be 1 (armv7) */
#define CpCsbz (CpCtre|1<<26|CpCve|1<<15|7<<7) /* must be 0 (armv7) */

<CpCONTROL.opcode2 other cases(arm) 478b>+≡ (29c) <461d
#define CpAuxctl 1

<type CpCONTROL CpAuxctl(arm) 478c>≡ (683c)
/*
 * CpCONTROL: op1==0, CRm==0, op2==CpAuxctl.
 * Auxiliary control register on cortex at least.
 */
/* cortex-a7 and cortex-a9 */
#define CpACsmp (1<<6) /* SMP l1 caches coherence; needed for ldrex/strex */

```

Jumping to C main() cpu0

```

<function _startpg(raspberry pi2)(arm) 478d>≡ (703c)
TEXT _startpg(SB), 1, $-4

<_startpg() enable cycle counter(raspberry pi2)(arm) 492a>
/*
 * call main and loop forever if it returns
 */
BL main(SB)
B 0(PC)

BL _div(SB) /* hack to load _div, etc. */

```

Starting the other processors

After CPU 0 finishes its initialization and starts the first user process, it calls [launchinit] to bring up the secondary CPUs. For each secondary CPU, [launchinit] allocates a Cpu structure and a fresh L1 page table (cloned from CPU 0's), then calls [startcpus]. The startup handshake uses the BCM2836 mailbox mechanism: CPU 0 writes the physical address of [cpureset] into the target CPU's mailbox, which wakes it up. A per-CPU lock synchronizes the handshake—CPU 0 locks it before starting the CPU and waits for the secondary to unlock it once running.

```

<main() start the other processors(arm) 478e>≡ (238b)
launchinit(getncpus());

<function launchinit(arm) 478f>≡ (704c)
void
launchinit(int ncpus)
{
    int xcpu;
    Cpu *mm;
    PTE *l1;

    <launchinit() sanity check ncpus(arm) 479b>
    for(xcpu = 1; xcpu < ncpus; xcpu++){
        cpus[xcpu] = mm = mallocalign(CPUSIZE, CPUSIZE, 0, 0);
        l1 = mallocalign(L1SIZE, L1SIZE, 0, 0);
        if(mm == nil || l1 == nil)
            panic("launchinit");
        memset(mm, 0, CPUSIZE);
        mm->cpuno = xcpu;
    }
}

```

```

        memmove(l1, cpu->mmul1, L1SIZE); /* clone cpu0's l1 table */
        cachedwbse(l1, L1SIZE);
        mm->mmul1 = l1;
        cachedwbse(mm, CPUSIZE);

    }
    cachedwbse(cpus, sizeof cpus);
    xcpu = startcpus(ncpus);
    if(xcpu < ncpus)
        panic("only %d cpu%s started", xcpu, xcpu == 1? "" : "s");
}

⟨constant L1SIZE(arm) 479a⟩≡ (709b)
#define L1SIZE (4 * BY2PG)

⟨launchinit() sanity check ncpus(arm) 479b⟩≡ (478f)
if(ncpus > MAXCPUS)
    ncpus = MAXCPUS;

⟨function startcpus(raspberry pi1)(arm) 479c⟩≡ (707a)
int
startcpus(uint)
{
    return 1;
}

⟨function getncpus(raspberry pi1)(arm) 479d⟩≡ (707a)
int
getncpus(void)
{
    return 1;
}

⟨function getncpus(raspberry pi2)(arm) 479e⟩≡ (707b)
int
getncpus(void)
{
    int n, max;
    char *p;

    n = 4;
    if(n > MAXCPUS)
        n = MAXCPUS;
    ⟨getncpus() adjust n if ncpu config parameter(arm) 479f⟩
    return n;
}

⟨getncpus() adjust n if ncpu config parameter(arm) 479f⟩≡ (479e)
p = getconf("ncpu");
if(p && (max = atoi(p)) > 0 && n > max)
    n = max;

⟨function startcpus(raspberry pi2)(arm) 479g⟩≡ (707b)
int
startcpus(uint ncpu)
{
    int i;

    for(i = 0; i < ncpu; i++)
        lock(&startlock[i]);
}

```



```

    cachedwbse(startlock, sizeof startlock);
    for(i = 1; i < ncpu; i++){
        if(startcpu(i) < 0)
            return i;
        lock(&startlock[i]);
        unlock(&startlock[i]);
    }
    return ncpu;
}

⟨function startcpu(raspberry pi2)(arm) 480a⟩≡ (707b)
static int
startcpu(uint xcpu)
{
    Mboxes *mb;
    int i;
    void cpureset();

    mb = (Mboxes*)(ARMLocal + Mboxregs);
    if(mb->clr[xcpu].startcpu)
        return -1;
    mb->set[xcpu].startcpu = PADDR(cpureset);
    for(i = 0; i < 1000; i++)
        if(mb->clr[xcpu].startcpu == 0)
            return 0;
    mb->clr[xcpu].startcpu = PADDR(cpureset);
    mb->set[xcpu].doorbell = 1;
    return 0;
}

```

Basic pagination other cpus

Secondary CPUs wake up executing at the physical address written to their mailbox. The [cpureset] code is similar to CPU 0's `armstart` but must find its own `Cpu` structure: it reads its CPU ID, indexes into the `cpus` array (set up by [launchinit]), and uses that CPU's pre-allocated L1 page table. After enabling the MMU, it adds KZERO to all registers to switch into virtual address space, then jumps to `_startpg2`.

```

⟨function cpureset(raspberry pi2)(arm) 480b⟩≡ (703c)
/*
 * startup entry for cpu(s) other than 0
 */
TEXT cpureset(SB), 1, $-4
reset:
/*
 * load physical base for SB addressing while mmu is off
 * keep a handy zero in R0 until first function call
 */
MOVW    $setR12(SB), R12
SUB $KZERO, R12
MOVW    $0, R0

/*
 * SVC mode, interrupts disabled
 */
MOVW    $(PsrDirq|PsrDfiq|PsrMsvc), R1
MOVW    R1, CPSR

/*
 * disable the mmu and caches

```

```

* invalidate tlb
*/
MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
BIC $(CpCdcache|CpCicache|CpCmmu), R1
ORR $(CpCsbo|CpCsw), R1
BIC $CpCsbz, R1
MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinu
ISB

/*
* find Mach for this cpu
*/
MRC CpSC, 0, R2, C(CpID), C(CpIDidct), CpIDmpid
AND $(MAXCPUS-1), R2 /* mask out non-cpu-id bits */
SLL $2, R2 /* convert to word index */
MOVW $cpus(SB), R0
ADD R2, R0 /* R0 = &cpus[cpuid] */
MOVW (R0), R0 /* R0 = cpus[cpuid] */
CMP $0, R0
MOVW.EQ $CPUADDR, R0 /* paranoia: use CPUADDR if 0 */
SUB $KZERO, R0 /* phys addr */
MOVW R0, R(CPU) /* m = PADDR(cpus[cpuid]) */

/*
* start stack at top of local Mach
*/
MOVW R(CPU), R13
ADD $(CPUSIZE-4), R13

/*
* set up page tables for kernel
*/
MOVW 12(R(CPU)), R0 /* m->mmul1 */
SUB $KZERO, R0 /* phys addr */
BL mmuinit(SB)

/*
* set up domain access control and page table base
*/
MOVW $Client, R1
MCR CpSC, 0, R1, C(CpDAC), C(0)
MOVW 12(R(CPU)), R1 /* m->mmul1 */
SUB $KZERO, R1 /* phys addr */
ORR $(CpTTBs/*|CpTTBowba|CpTTBiwba*/), R1
MCR CpSC, 0, R1, C(CpTTB), C(0)
MCR CpSC, 0, R1, C(CpTTB), C(0), CpTTB1 /* cortex has two */

/*
* invalidate my caches before enabling
*/
BL cachedinv(SB)
BL cacheiinv(SB)
BARRIERS

/*
* enable caches, mmu, and high vectors
*/
MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpAuxctl
ORR $CpACsmp, R1 /* turn SMP on */

```

```

MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpAuxctl
BARRIERS

MRC CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
ORR $(CpChv|CpCdcache|CpCicache|CpCmmu), R0
MCR CpSC, 0, R0, C(CpCONTROL), C(0), CpMainctl
BARRIERS

/*
 * switch CPU, SB, SP, and PC into KZERO space
 */
ADD $KZERO, R(CPU)
MOVW    $setR12(SB), R12
ADD $KZERO, R13
MOVW    $_startpg2(SB), R15

```

Jumping to C cpustart()

```

<function _startpg2(raspberry pi2)(arm) 482a>≡ (703c)
TEXT _startpg2(SB), 1, $-4

<_startpg2() enable cycle counter(raspberry pi2)(arm) 492b>
/*
 * call cpustart and loop forever if it returns
 */
MRC CpSC, 0, R0, C(CpID), C(CpIDidct), CpIDmpid
AND $(MAXCPUS-1), R0          /* mask out non-cpu-id bits */
BL    ,cpustart(SB)
B     ,0(PC)

```

main() other processors (cpustart())

The C entry point for secondary CPUs is much simpler than `main`: there is no need to initialize memory pools, the device table, or the first user process—all of that was done by CPU 0. Each secondary CPU just needs to finalize its own MMU, set up its trap vectors and local timer, then call `[schedinit]` to enter the scheduler loop and start running processes. The `unlock` of `startlock` signals to CPU 0 that this CPU is ready.

```

<function cpustart(raspberry pi2)(arm) 482b>≡ (707b)
void
cpustart(int xcpu)
{
    Mboxes *mb;

    up = nil;
    arch__cpuinit();
    mmuinit1((void*)cpu->mmu1);

    mb = (Mboxes*)(ARMLOCAL + Mboxregs);
    mb->clr[xcpu].doorbell = 1;

    arch__trapinit();
    clockinit();
    timersinit();

    arch_cpuidprint();
    archreset();

    machon(cpu->cpuno);
}

```

```

unlock(&startlock[xcpu]);

// schedule a ready process (not necessarily the first one now)
schedinit();

panic("schedinit returned");
}

⟨arch__cpuinit() it not cpu0 483a⟩≡ (240b)
if (cpu->cpuno != 0) {
    /* synchronise with cpu 0 */
    cpu->ticks = cpu0->ticks;
}

```

Displaying processor information

```

⟨function arch_cpuidprint(raspberry pi2)(arm) 483b⟩≡ (707b)
void
arch_cpuidprint(void)
{
    char name[64];

    cputype2name(name, sizeof name);
    arch_delay(50);          /* let uart catch up */
    print("cpu%d: %dMHz ARM %s\n", cpu->cpuno, cpu->cpumhz, name);
}

⟨function cputype2name(raspberry pi2)(arm) 483c⟩≡ (707b)
char*
cputype2name(char *buf, int size)
{
    ulong r;

    r = cpidget();          /* main id register */
    assert((r >> 24) == 'A');
    seprint(buf, buf + size, "Cortex-A7 r%ldp%ld",
        (r >> 20) & MASK(4), r & MASK(4));
    return buf;
}

```

```

⟨function cpidget(raspberry pi2)(arm) 483d⟩≡ (703c)
TEXT cpidget(SB), 1, $-4      /* main ID */
    MRC CpSC, 0, R0, C(CpID), C(0), CpIDid
    RET

```

Halting

When a CPU has nothing to run, `arch_idlehands` could use WFI (Wait For Interrupt) to save power. However, WFI is disabled on the Pi 2 because local timer interrupts can be lost, causing the CPU to hang.

```

⟨function arch_idlehands(raspberry pi2)(arm) 483e⟩≡ (703c)
TEXT arch_idlehands(SB), $-4
    MOVW    CPSR, R3
    ORR $(PsrDirq|PsrDfiq), R3, R1    /* splfhi */
    MOVW    R1, CPSR

    DSB
    MOVW    nrdy(SB), R0

```

```

    CMP $0, R0
    /** with WFI, local timer interrupts can be lost and dispatching stops
        WFI_EQ
    ***/
    DSB

    MOVW    R3, CPSR          /* splx */
    RET

```

<instruction WFI_EQ(arm) 484a>≡ (703c)

```

#define WFI_EQ    WORD    $0x0320f003 /* wait for interrupt if eq */

```

22.5.2 Setting cpu

On a uniprocessor, the `cpu` pointer lives at a fixed address (`CPUADDR`) and never changes. With SMP, each exception handler must figure out which CPU it is running on and load the correct `Cpu` pointer. The `CPUID` macro reads the multiprocessor ID register, and the result indexes into the `cpus` array. This code appears in both `_vsvc` (syscall entry) and `_vswitch` (trap from user mode).

<_vsvc() set cpu register(arm) 484b>≡ (140c)

```

CPUID(R1)
SLL $2, R1          /* convert to word index */
MOVW    $cpus(SB), R2
ADD R1, R2
MOVW    (R2), R(CPU)    /* m = cpus[cpuid] */
CMP $0, R(CPU)
MOVW.EQ $CPUADDR, R0    /* paranoia: use CPUADDR if 0 */

```

<_vswitch() set cpu register(arm) 484c>≡ (145a)

```

CPUID(R1)
SLL $2, R1          /* convert to word index */
MOVW    $cpus(SB), R2
ADD R1, R2
MOVW    (R2), R(CPU)    /* m = cpus[cpuid] */
CMP $0, R(CPU)
MOVW.EQ $CPUADDR, R0    /* paranoia: use CPUADDR if 0 */

```

22.5.3 Concurrency adjustments

The Pi 1's `arch_tas` uses the `SWP` instruction, which is deprecated on ARMv7 and does not work correctly on multiprocessors. The Pi 2 replaces it with the `LDREX`/`STREX` (load/store exclusive) pair, which provides true atomic compare-and-swap via the cache coherency protocol. `LDREX` loads a value and marks the cache line as “exclusive”; `STREX` stores a new value only if the line is still exclusive, returning success or failure. If another CPU touched the same cache line in between, `STREX` fails and the code retries. `DMB` (data memory barrier) instructions bracket the sequence to ensure proper ordering.

<instruction LDREX(arm) 484d>≡ (684)

```

#define LDREX(fp,t)    WORD $(0xe<<28|0x01900f9f | (fp)<<16 | (t)<<12)

```

<instruction STREX(arm) 484e>≡ (684)

```

/* 'The order of operands is from left to right in dataflow order' - asm man */
#define STREX(f,tp,r)  WORD $(0xe<<28|0x01800f90 | (tp)<<16 | (r)<<12 | (f)<<0)

```

<instruction CLREX(arm) 484f>≡ (684)

```

#define CLREX    WORD    $0xf57ff01f

```

<instruction DMB(arm) 484g>≡ (693b)

```

#define DMB WORD    $0xf57ff05f /* data mem. barrier; last f = SY */

```

<function arch_tas(raspberry pi2)(arm) 485a>≡ (693b)

```
TEXT    arch_tas(SB), $-4
        /* returns old (R0) after modifying (R0) */
        MOVW    R0,R5
        DMB

        MOVW    $1,R2          /* new value of (R0) */
        MOVW    $MAXSC, R8

tas1:
        LDREX(5,7)             /* LDREX 0(R5),R7 */
        CMP.S    $0, R7        /* old value non-zero (lock taken)? */
        BNE lockbusy          /* we lose */
        SUB.S    $1, R8
        BEQ lockloop2
        STREX(2,5,4)           /* STREX R2,(R5),R4 */
        CMP.S    $0, R4
        BNE tas1              /* strex failed? try again */
        DMB
        B    tas0
lockloop2:
        BL    abort(SB)
lockbusy:
        CLREX
tas0:
        MOVW    R7, R0          /* return old value */
        RET
```

22.5.4 Timer adjustments

On the Pi 1, there is a single system timer shared by the one CPU. On the Pi 2, CPU 0 still uses the system timer, but the secondary CPUs use the ARM generic timer (a per-CPU countdown timer accessed via coprocessor 14). CPU 0 configures the prescaler to produce a 1 MHz tick, and each secondary CPU enables its own local timer interrupt via `arch_intrenable` with the `IRQcntpns` vector.

<constant Localctl(arm) 485b>≡ (714c)

```
Localctl    = 0x00,
```

<constant Prescaler(arm) 485c>≡ (714c)

```
Prescaler   = 0x08,
```

<clockinit() if many processors 485d>≡ (245a)

```
if(((cprdsc(0, CpID, CpIDfeat, 1) >> 16) & 0xF) != 0) {
    /* generic timer supported */
    if(cpu->cpuno == 0){
        *(ulong*)(ARMLLOCAL + Localctl) = 0;          /* magic */
        *(ulong*)(ARMLLOCAL + Prescaler) = 0x06aaaaab; /* magic for 1 Mhz */
    }
    cpwrsc(0, CpTIMER, CpTIMERphys, CpTIMERphysctl, Imask);
}
```

<clockinit() if not cpu0 485e>≡ (245a)

```
else
    arch_intrenable(IRQcntpns, localclockintr, nil, 0, "clock");
```

```

⟨function localclockintr(arm) 486a⟩≡ (715b)
static void
localclockintr(Ureg *ureg, void *)
{
    if(cpu->cpuno == 0)
        panic("cpu0: Unexpected local generic timer interrupt");
    cpwrrsc(0, CpTIMER, CpTIMERphys, CpTIMERphysctl, Imask|Enable);
    timerintr(ureg, 0);
}

```

```

⟨function arch_lcycles(raspberrypi2)(arm) 486b⟩≡ (716c)
TEXT arch_lcycles(SB), 1, $-4
    MRC CpSC, 0, R0, C(CpCLD), C(CpCLDcyc), 0
    RET

```

```

⟨arch_timerset() if not cpu0(arm) 486c⟩≡ (219d)
if(cpu->cpuno > 0){
    cpwrrsc(0, CpTIMER, CpTIMERphys, CpTIMERphysval, period);
    cpwrrsc(0, CpTIMER, CpTIMERphys, CpTIMERphysctl, Enable);
}

```

22.5.5 arch_coherence() (ARM)

```

⟨function arm_arch_coherence(arm) 486d⟩≡ (692)
TEXT arm_arch_coherence(SB), $-4
    BARRIERS
    RET

```

22.5.6 Cache adjustments

The Cortex-A7’s cache line is 64 bytes (vs 32 on the ARM1176 in Pi 1), and the cache operations use the ARMv7 set/way interface rather than the simpler whole-cache MCR instructions. The [wholecache] routine iterates over all sets and ways at a given cache level, calling a function pointer (writeback, invalidate, or both) for each set/way combination. This is necessary because the ARMv7 architecture removed the “invalidate entire data cache” operation to prevent accidental data loss in multi-core systems.

```

⟨constant CACHELINESZ(raspberrypi2)(ram) 486e⟩≡ (712a)
#define CACHELINESZ 64

```

```

⟨instruction CPSID(arm) 486f⟩≡ (684)
#define CPSID    WORD    $0xf10c0080 /* intr disable: sets I bit */

```

```

⟨type CpID CpIDid op1(arm) 486g⟩≡ (683c)
/* CpIDid op1 values */
#define CpIDcsize    1          /* cache size (cortex) */
#define CpIDcssel    2          /* cache size select (cortex) */

```

```

⟨function cachexxx(raspberrypi2)(ram) 486h⟩≡ (712a)
/*
 * ‘single-element’ cache operations.
 * in arm arch v7, they operate on all cache levels, so separate
 * l2 functions are unnecessary.
 */

TEXT cachedwbse(SB), $-4          /* D writeback SE */
    MOVW    R0, R2

    MOVW    CPSR, R3

```

```

CPSID                /* splhi */

BARRIERS             /* force outstanding stores to cache */
MOVW    R2, R0
MOVW    4(FP), R1
ADD R0, R1           /* R1 is end address */
BIC $(CACHELINESZ-1), R0      /* cache line start */
_dwbse:
MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEse
/* can't have a BARRIER here since it zeroes R0 */
ADD $CACHELINESZ, R0
CMP.S   R0, R1
BGT _dwbse
B       _wait

TEXT cachedwbinvse(SB), $-4      /* D writeback+invalidate SE */
MOVW    R0, R2

MOVW    CPSR, R3
CPSID                /* splhi */

BARRIERS             /* force outstanding stores to cache */
MOVW    R2, R0
MOVW    4(FP), R1
ADD R0, R1           /* R1 is end address */
BIC $(CACHELINESZ-1), R0      /* cache line start */
_dwbinvse:
MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEse
/* can't have a BARRIER here since it zeroes R0 */
ADD $CACHELINESZ, R0
CMP.S   R0, R1
BGT _dwbinvse
_wait:                /* drain write buffer */
BARRIERS

MOVW    R3, CPSR      /* splx */
RET

TEXT cachedinvse(SB), $-4        /* D invalidate SE */
MOVW    R0, R2

MOVW    CPSR, R3
CPSID                /* splhi */

BARRIERS             /* force outstanding stores to cache */
MOVW    R2, R0
MOVW    4(FP), R1
ADD R0, R1           /* R1 is end address */
BIC $(CACHELINESZ-1), R0      /* cache line start */
_dinvse:
MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvd), CpCACHEse
/* can't have a BARRIER here since it zeroes R0 */
ADD $CACHELINESZ, R0
CMP.S   R0, R1
BGT _dinvse
B       _wait

/*
* cortex arm arch v7 cache flushing and invalidation
* shared by l.s and rebootcode.s

```



```

*/

TEXT cacheiinv(SB), $-4          /* I invalidate */
    MOVW    $0, R0
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvi), CpCACHEall /* ok on cortex */
    ISB
    RET

/*
 * set/way operators, passed a suitable set/way value in R0.
 */
TEXT cachedwb_sw(SB), $-4
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEsi
    RET

TEXT cachedwbinv_sw(SB), $-4
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwbi), CpCACHEsi
    RET

TEXT cachedinv_sw(SB), $-4
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvd), CpCACHEsi
    RET

    /* set cache size select */
TEXT setcachelv1(SB), $-4
    MCR CpSC, CpIDcssel, R0, C(CpID), C(CpIDidct), 0
    ISB
    RET

    /* return cache sizes */
TEXT getwayssets(SB), $-4
    MRC CpSC, CpIDcsize, R0, C(CpID), C(CpIDidct), 0
    RET

/*
 * l1 cache operations.
 * l1 and l2 ops are intended to be called from C, thus need save no
 * caller's regs, only those we need to preserve across calls.
 */

TEXT cachedwb(SB), $-4
    MOVW.W   R14, -8(R13)
    MOVW     $cachedwb_sw(SB), R0
    MOVW     $1, R8
    BL      wholecache(SB)
    MOVW.P   8(R13), R15

TEXT cachedwbinv(SB), $-4
    MOVW.W   R14, -8(R13)
    MOVW     $cachedwbinv_sw(SB), R0
    MOVW     $1, R8
    BL      wholecache(SB)
    MOVW.P   8(R13), R15

TEXT cachedinv(SB), $-4
    MOVW.W   R14, -8(R13)
    MOVW     $cachedinv_sw(SB), R0
    MOVW     $1, R8
    BL      wholecache(SB)
    MOVW.P   8(R13), R15

```

```

TEXT cacheuwbinv(SB), $-4
    MOVW.DB.W [R14], (R13) /* save lr on stack */
    MOVW      CPSR, R1
    CPSID     /* splhi */

    MOVW.DB.W [R1], (R13) /* save R1 on stack */

    BL  cachedwbinv(SB)
    BL  cacheiinv(SB)

    MOVW.IA.W (R13), [R1] /* restore R1 (saved CPSR) */
    MOVW      R1, CPSR
    MOVW.IA.W (R13), [R14] /* restore lr */
    RET

/*
 * l2 cache operations
 */

TEXT l2cacheuwb(SB), $-4
    MOVW.W R14, -8(R13)
    MOVW    $cachedwb_sw(SB), R0
    MOVW    $2, R8
    BL  wholecache(SB)
    MOVW.P 8(R13), R15

TEXT l2cacheuwbinv(SB), $-4
    MOVW.W R14, -8(R13)
    MOVW    CPSR, R1
    CPSID     /* splhi */

    MOVW.DB.W [R1], (R13) /* save R1 on stack */

    MOVW    $cachedwbinv_sw(SB), R0
    MOVW    $2, R8
    BL  wholecache(SB)
    BL  l2cacheuinv(SB)

    MOVW.IA.W (R13), [R1] /* restore R1 (saved CPSR) */
    MOVW      R1, CPSR
    MOVW.P 8(R13), R15

TEXT l2cacheuinv(SB), $-4
    MOVW.W R14, -8(R13)
    MOVW    $cachedinv_sw(SB), R0
    MOVW    $2, R8
    BL  wholecache(SB)
    MOVW.P 8(R13), R15

/*
 * these shift values are for the Cortex-A8 L1 cache (A=2, L=6) and
 * the Cortex-A8 L2 cache (A=3, L=6).
 * A = log2(# of ways), L = log2(bytes per cache line).
 * see armv7 arch ref p. 1403.
 */
#define L1WAYSH 30
#define L1SETSH 6
#define L2WAYSH 29
#define L2SETSH 6

```

```

/*
 * callers are assumed to be the above l1 and l2 ops.
 * R0 is the function to call in the innermost loop.
 * R8 is the cache level (one-origin: 1 or 2).
 *
 * initial translation by 5c, then massaged by hand.
 */
TEXT wholecache+0(SB), $-4
    MOVW    R0, R1      /* save argument for inner loop in R1 */
    SUB $1, R8          /* convert cache level to zero origin */

    /* we may not have the MMU on yet, so map R1 to PC's space */
    BIC $KSEGM, R1      /* strip segment from address */
    MOVW    PC, R2      /* get PC's segment ... */
    AND $KSEGM, R2
    ORR R2, R1          /* combine them */

    /* drain write buffers */
    BARRIERS
    MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEwait
    ISB

    MOVW    CPSR, R2
    MOVW.DB.W [R2,R14], (SP) /* save regs on stack */
    CPSID   /* splhi to make entire op atomic */

    /* get cache sizes */
    SLL $1, R8, R0      /* R0 = (cache - 1) << 1 */
    MCR CpSC, CpIDcssel, R0, C(CpID), C(CpIDidct), 0 /* set cache size select */
    ISB
    MRC CpSC, CpIDcsize, R0, C(CpID), C(CpIDidct), 0 /* get cache sizes */

    /* compute # of ways and sets for this cache level */
    SRA $3, R0, R5      /* R5 (ways) = R0 >> 3 */
    AND $1023, R5       /* R5 = (R0 >> 3) & MASK(10) */
    ADD $1, R5          /* R5 (ways) = ((R0 >> 3) & MASK(10)) + 1 */

    SRA $13, R0, R2     /* R2 = R0 >> 13 */
    AND $32767, R2      /* R2 = (R0 >> 13) & MASK(15) */
    ADD $1, R2          /* R2 (sets) = ((R0 >> 13) & MASK(15)) + 1 */

    /* precompute set/way shifts for inner loop */
    CMP $0, R8          /* cache == 1? */
    MOVW.EQ $L1WAYSH, R3 /* yes */
    MOVW.EQ $L1SETSH, R4
    MOVW.NE $L2WAYSH, R3 /* no */
    MOVW.NE $L2SETSH, R4

    /* iterate over ways */
    MOVW    $0, R7      /* R7: way */
outer:
    /* iterate over sets */
    MOVW    $0, R6      /* R6: set */
inner:
    /* compute set/way register contents */
    SLL R3, R7, R0      /* R0 = way << R3 (L?WAYSH) */
    ORR R8<<1, R0       /* R0 = way << L?WAYSH | (cache - 1) << 1 */
    ORR R6<<R4, R0      /* R0 = way<<L?WAYSH | (cache-1)<<1 |set<<R4 */

```

```

BL   (R1)          /* call set/way operation with R0 */

ADD $1, R6          /* set++ */
CMP R2, R6          /* set >= sets? */
BLT inner           /* no, do next set */

ADD $1, R7          /* way++ */
CMP R5, R7          /* way >= ways? */
BLT outer           /* no, do next way */

MOVM.IA.W (SP), [R2,R14] /* restore regs */
MOVW   R2, CPSR      /* splx */

/* drain write buffers */
MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEwb), CpCACHEwait
ISB
RET

```

22.5.7 IRQ adjustments

The Pi 2 adds per-CPU local interrupts (timer, mailbox) that are separate from the shared peripheral interrupts. These local IRQs start at number 96 (`IRQlocal`) and are routed through per-CPU pending/enable registers in the ARM local register block, rather than the shared `Intregs` used for peripheral interrupts. Each `Vctl` gains a `cpu` field to record which processor the interrupt handler was registered on.

$\langle \text{IRQ local cases}(\text{arm}) \text{ 491a} \rangle \equiv$ (151d)

```

IRQlocal    = 96,
IRQcntps    = IRQlocal + 0,
IRQcntpns   = IRQlocal + 1,

```

$\langle \text{Vctl other fields}(\text{arm}) \text{ 491b} \rangle \equiv$ (152a)

```

int cpu;

```

$\langle \text{irqenable}() \text{ if } \text{IRQlocal}(\text{arm}) \text{ 491c} \rangle \equiv$ (153b)

```

if(irq >= IRQlocal){
    enable = (u32int*)(LOCALREGS + Localtimerint) + cpu->cpuno;
    v->reg = (u32int*)(LOCALREGS + Localintpending) + cpu->cpuno;
    v->mask = 1 << (irq - IRQlocal);
    v->cpu = cpu->cpuno;
}

```

$\langle \text{irqenable}() \text{ if } \text{IRQlocal adjust enable}(\text{arm}) \text{ 491d} \rangle \equiv$ (153b)

```

if(irq >= IRQlocal)
    *enable |= 1 << (irq - IRQlocal);

```

$\langle \text{constant Localtimerint}(\text{arm}) \text{ 491e} \rangle \equiv$ (712b)

```

Localtimerint = 0x40,

```

$\langle \text{constant Localintpending}(\text{arm}) \text{ 491f} \rangle \equiv$ (712b)

```

Localintpending = 0x60,

```

$\langle \text{constant Localmbboxint}(\text{arm}) \text{ 491g} \rangle \equiv$ (712b)

```

Localmbboxint = 0x50,

```

$\langle \text{constant LOCALREGS}(\text{arm}) \text{ 491h} \rangle \equiv$ (712c)

```

#define LOCALREGS (VIRTIO+IOSIZE)

```

22.5.8 Cycle counter adjustments

```
<_startpg() enable cycle counter(raspberry pi2)(arm) 492a>≡ (478d)
/*
 * enable cycle counter
 */
MOVW    $(1<<31), R1
MCR CpSC, 0, R1, C(CpCLD), C(CpCLDena), CpCLDenacyc
MOVW    $1, R1
MCR CpSC, 0, R1, C(CpCLD), C(CpCLDena), CpCLDenapmnc

<_startpg2() enable cycle counter(raspberry pi2)(arm) 492b>≡ (482a)
/*
 * enable cycle counter
 */
MOVW    $(1<<31), R1
MCR CpSC, 0, R1, C(CpCLD), C(CpCLDena), CpCLDenacyc
MOVW    $1, R1
MCR CpSC, 0, R1, C(CpCLD), C(CpCLDena), CpCLDenapmnc
```

22.6 Advanced virtual memory

This section covers virtual memory features beyond the basic page fault handler presented in Chapter ??: page coloring (a cache optimization), physical memory segments (for memory-mapped I/O), and the `flushme` mechanism for maintaining instruction cache coherency when text pages are modified.

22.6.1 Page coloring

Page coloring is a technique to reduce cache conflicts in virtually-indexed caches. By assigning pages to “colors” based on their physical address, the kernel ensures that pages mapped at different virtual addresses do not compete for the same cache lines. On the ARM, the data cache is set-associative rather than direct-mapped, so coloring is less important—`NCOLOR` is set to 1, effectively disabling it. The machinery remains in the code because other architectures (like MIPS) benefit from it.

```
<Page other fields 492c>+≡ (87) <187c 493b>
// int [0..NCOLOR[
char color; /* Cache coloring */

<pageinit() other locals 492d>≡ (248)
int color = 0;

<pageinit() set page color 492e>≡ (248)
p->color = color;
color = (color+1)%NCOLOR;

<macro getpgcolor(arm) 492f>≡ (695d)
#define getpgcolor(a) 0

<constant NCOLOR(arm) 492g>≡ (709a)
#define NCOLOR 1 /* 1 level cache, don't worry about VCE's */
```

Each page carries a per-CPU `cachectl` array that tracks whether the instruction cache needs to be invalidated when the page is mapped. `PG_TXTFLUSH` is set by `pio` whenever a text segment page is written (e.g., during `sysexec`). When `arch_putmmu` maps this page, it checks `cachectl` and calls `cacheiinv` if needed. The `flushme` flag on a `Segment` controls which segments participate in this mechanism.

```
<enum Cachectl 493a>≡ (658)
enum Cachectl {
    PG_NOFLUSH = 0,
    PG_TXTFLUSH = 1, /* flush dcache and invalidate icache */

    PG_NEWCOL = 3, /* page has been recolored */
};
```

```
<Page other fields 493b>+≡ (87) <492c
// array<enum<Cachectl>>
char cachectl[MAXCPUS]; /* Cache flushing control for putmmu */
```

```
<newpage() other locals 493c>+≡ (89c) <183a 493e>
int color;
// enum<Cachectl>
uchar ct;
```

```
<newpage() find free page p, set color and ct 493d>≡ (89c)
color = getpgcolor(va);
```

```
/* First try for our colour */
for(p = palloc.head; p; p = p->next)
    if(p->color == color)
        break;
```

```
ct = PG_NOFLUSH;
if(p == nil) {
    p = palloc.head;
    p->color = color;
    ct = PG_NEWCOL;
}
```

```
<newpage() other locals 493e>+≡ (89c) <493c
int i;
```

```
<newpage() set cachectl using ct 493f>≡ (89c)
for(i = 0; i < MAXCPUS; i++)
    p->cachectl[i] = ct;
```

```
<arch_putmmu() if PG_TXTFLUSH, invalidate cache(arm) 493g>≡ (162)
if(page->cachectl[cpu->cpuno] == PG_TXTFLUSH){
    /* pio() sets PG_TXTFLUSH whenever a text pg has been written */
    cacheiinv();
    page->cachectl[cpu->cpuno] = PG_NOFLUSH;
}
```

```
<arch__userinit() set cachectl(arm) 493h>≡ (251d)
memset(pg->cachectl, PG_TXTFLUSH, sizeof(pg->cachectl));
```

```
<Segment other fields 493i>+≡ (92c) <446a 494l>
bool flushme; /* maintain icache for this segment */
```

```
<sysexec() initialize other fields 493j>≡ (170d)
ts->flushme = true;
```

`<arch__userinit() set flushme for stack segment 494a>≡ (251d)`
`s->flushme = true;`

`<data2txt() initialize other segment fields 494b>≡ (171c)`
`ps->flushme = true;`

`<text2data() initialize other fields 494c>≡ (431)`
`ps->flushme = true;`

`<dupseg() copy other fields 494d>≡ (101b)`
`n->flushme = s->flushme;`

`<syssegflush() set flushme 494e>≡ (415c)`
`s->flushme = true;`

`<pio() set cachectl if flushme segment 494f>≡ (171g)`
`if(s->flushme)`
`memset((*p)->cachectl, PG_TXTFLUSH, sizeof((*p)->cachectl));`

`<procctlmemio() set cachectl if flushme segment 494g>≡ (430e)`
`/* Ensure the process sees text page changes */`
`if(s->flushme)`
`memset(pg->cachectl, PG_TXTFLUSH, sizeof(pg->cachectl));`

22.6.2 Extra segment types

Beyond the standard text/data/bss/stack segments, Plan 9 supports `SG_SHARED` (memory shared between processes via `[syssegattach]`) and `SG_PHYSICAL` (direct mapping of physical memory, used for memory-mapped I/O like the VGA framebuffer). Both are handled specially during `[dupseg]`: they share the same `Segment` object rather than copying, since the whole point is that multiple processes see the same memory. For `SG_PHYSICAL`, the page fault handler (`[fixfault]`) creates pages whose physical addresses map directly to the device's physical memory, and marks them uncached (`PTEUNCACHED`) so reads and writes go straight to hardware.

`<Segtype other cases 494h>≡ (92a) 494i▷`
`SG_SHARED = 04,`

`<Segtype other cases 494i>+≡ (92a) ◁494h`
`SG_PHYSICAL = 05,`

`<dupseg() switch segment type cases 494j>+≡ (100b) ◁101d`
`case SG_SHARED:`
`case SG_PHYSICAL:`
`goto sameseg;`

`<struct Physseg 494k>≡ (658)`
`struct Physseg`
`{`
`ulong attr;    /* Segment attributes */`
`char *name;    /* Attach name */`
`phys_addr pa;  /* Physical address */`
`ulong size;    /* Maximum segment size in pages */`
`};`

`<Segment other fields 494l>+≡ (92c) ◁493i 497b▷`
`Physseg *pseg;`

<global physseg 495a>≡ (665)

```
/*
 * Attachable segment types
 */
static Physseg physseg[10] = {
    { .attr= SG_SHARED,
      .name= "shared",
      .pa= 0,
      .size= SEGMAXSIZE,
    },
    { .attr= SG_BSS,
      .name= "memory",
      .pa = 0,
      .size = SEGMAXSIZE,
    },
    { 0, 0, 0, 0},
};
```

<global physseglock 495b>≡ (665)

```
static Lock physseglock;
```

<function addphysseg 495c>≡ (665)

```
int
addphysseg(Physseg* new)
{
    Physseg *ps;

    /*
     * Check not already entered and there is room
     * for a new entry and the terminating null entry.
     */
    lock(&physseglock);
    for(ps = physseg; ps->name; ps++){
        if(strcmp(ps->name, new->name) == 0){
            unlock(&physseglock);
            return -1;
        }
    }
    if(ps-physseg >= nelem(physseg)-2){
        unlock(&physseglock);
        return -1;
    }

    *ps = *new;
    unlock(&physseglock);

    return 0;
}
```

<freept() other locals 495d>≡ (94c)

```
int ref;
Page *pt, **ptop;
```

<freept() SG_PHYSICAL case 495e>≡ (94c)

```
case SG_PHYSICAL:
    ptop = &p->pagetab[PAGETABSIZE];
    for(pte = p->pagetab; pte < ptop; pte++) {
        pt = *pte;
        if(pt == nil)
            continue;
```



```

    lock(pt);
    ref = --pt->ref;
    unlock(pt);
    if(ref == 0)
        free(pt); // because was smalloc'ed in fixfault
}
break;

```

For `SG_PHYSICAL` segments, the page fault handler does not allocate real memory. Instead, it creates a `Page` whose physical address maps directly to the device's physical address (computed from the segment's base PA plus the offset). The page is marked `PTEUNCACHED` so that reads and writes bypass the CPU cache and go directly to the hardware.

`<fixfault() SG_PHYSICAL case 496a>≡ (161)`

```

case SG_PHYSICAL:
    if(*pte == nil) {
        new = smalloc(sizeof(Page));
        new->va = addr;
        new->pa = s->pseg->pa+(addr-s->base);
        new->ref = 1;
        *pte = new;
    }

    if (checkaddr && addr == addr2check)
        (*checkaddr)(addr, s, *pte);
    mmuppte = PPN((*pte)->pa) |PTEWRITE|PTEUNCACHED|PTEVALID;
    (*pte)->modref = PG_MOD|PG_REF;
    break;

```

`<PTExxx other cases 496b>≡ (160b)`

```

#define PTEUNCACHED (1<<4)

```

`<arch_putmmu() if PTEUNCACHED, do not change x(arm) 496c>≡ (162)`

```

if(pa & PTEUNCACHED) {
}

```

22.6.3 arch_kmap()

22.6.4 Extra checks

`<function checkpagerefs 496d>≡ (664)`

```

void
checkpagerefs(void)
{
    int s;
    ulong i, np, nwrong;
    ulong *ref;

    np = palloc.user;
    ref = malloc(np*sizeof ref[0]);
    if(ref == nil){
        print("checkpagerefs: out of memory\n");
        return;
    }

    /*
     * This may not be exact if there are other processes
     * holding refs to pages on their stacks. The hope is
     * that if you run it on a quiescent system it will still

```

```

    * be useful.
    */
    s = arch_splhi();
    lock(&palloc);
    arch_countpagerefs(ref, 0);
    portcountpagerefs(ref, 0);
    nwrong = 0;
    for(i=0; i<np; i++){
        if(palloc.pages[i].ref != ref[i]){
            iprint("page %#.8lux ref %d actual %lud\n",
                palloc.pages[i].pa, palloc.pages[i].ref, ref[i]);
            ref[i] = 1;
            nwrong++;
        }else
            ref[i] = 0;
    }
    arch_countpagerefs(ref, 1);
    portcountpagerefs(ref, 1);
    iprint("%lud mistakes found\n", nwrong);
    unlock(&palloc);
    arch_splx(s);
}

```

$\langle$ macro arch_countpagerefs(*arm*) 497a $\rangle \equiv$ (695d)
 // same signatures but optimized away
 #define arch_countpagerefs(a, b)

$\langle$ Segment other fields 497b $\rangle + \equiv$ (92c) $\triangleleft$ 494l
 ulong mark; /* portcountrefs */

$\langle$ function portcountpagerefs 497c $\rangle \equiv$ (664)
 void
 portcountpagerefs(ulong *ref, int print)
 {
 ulong i, j, k, ns, n;
 Page **pg, *entry;
 Proc *p;
 Pagetable *pt;
 Segment *s;

 /*
 * Pages in segments. s->mark avoids double-counting.
 */
 n = 0;
 ns = 0;
 for(i=0; i<conf.nproc; i++){
 p = proctab(i);
 for(j=0; j<NSEG; j++){
 s = p->seg[j];
 if(s)
 s->mark = 0;
 }
 }
 for(i=0; i<conf.nproc; i++){
 p = proctab(i);
 for(j=0; j<NSEG; j++){
 s = p->seg[j];
 if(s == nil || s->mark++)
 continue;
 ns++;
 }
 }
 }

```

        for(k=0; k<s->pagedirsize; k++){
            pt = s->pagedir[k];
            if(pt == nil)
                continue;
            for(pg = pt->first; pg <= pt->last; pg++){
                entry = *pg;
                if(pagedout(entry))
                    continue;
                if(print){
                    if(ref[pagenumner(entry)])
                        iprint("page %#.8lux in segment %#p\n", entry->pa, s);
                    continue;
                }
                if(ref[pagenumner(entry)]++ == 0)
                    n++;
            }
        }
    }
}

if(!print){
    iprint("%lud pages in %lud segments\n", n, ns);
    for(i=0; i<conf.nproc; i++){
        p = proctab(i);
        for(j=0; j<NSEG; j++){
            s = p->seg[j];
            if(s == nil)
                continue;
            if(s->ref != s->mark){
                iprint("segment %#p (used by proc %lud pid %lud) has bad ref count %lud actual %lud\n",
                    s, i, p->pid, s->ref, s->mark);
            }
        }
    }
}

}

}

}

function pagenumner 498a)≡ (664)
ulong
pagenumner(Page *p)
{
    return p - palloc.pages;
}

```

```

⟨trapnames() other entries(arm) 499a⟩≡ (139b)
    [ PsrMfiq ] "fiq interrupt",

⟨PsrDisable other cases(arm) 499b⟩≡ (28b)
    #define PsrDfiq      0x00000040      /* disable FIQ interrupts */

⟨constant FIQSTKTOP(arm) 499c⟩≡ (709b)
    #define FIQSTKTOP    (KZERO+0x4000)    /* FIQ stack */

⟨arch__trapinit() set stack for other exception/processor-modes 499d⟩+≡ (243f) <244b
    setr13(PsrMfiq, (u32int*)(FIQSTKTOP));

⟨function _vfiq(arm) 499e⟩≡ (714a)
    TEXT _vfiq(SB), 1, $-4      /* FIQ */
        CLREX
        MOVW    $PsrMfiq, R8      /* trap type */
        MOVW    SPSR, R9          /* interrupted psr */
        MOVW    R14, R10         /* interrupted pc */
        MOVM.DB.W [R8-R10], (R13) /* save in ureg */
        MOVM.DB.S [R0-R14], (R13) /* save interrupted regs */
        SUB $(15*4), R13
        MOVW    $setR12(SB), R12 /* Make sure we've got the kernel's SB loaded */

        /* get R(CPU) for this cpu */
        CPUID(R1)
        SLL $2, R1      /* convert to word index */
        MOVW    $cpus(SB), R2
        ADD R1, R2
        MOVW    (R2), R(CPU) /* m = cpus[cpuid] */
        CMP $0, R(CPU)
        MOVW.EQ $CPUADDR, R(CPU) /* paranoia: use CPUADDR if 0 */

        MOVW    8(R(CPU)), R(UP) /* up */
        MOVW    R13, R0          /* first arg is pointer to ureg */
        SUB $(4*2), R13 /* space for argument+link (for debugger) */

        BL    fiq(SB)

        ADD $(8+4*15), R13 /* make r13 point to ureg->type */
        MOVW    8(R13), R14 /* restore link */
        MOVW    4(R13), R0 /* restore SPSR */
        MOVW    R0, SPSR /* ... */
        MOVM.DB.S (R13), [R0-R14] /* restore registers */
        ADD $8, R13 /* pop past ureg->{type+psr} */
        RFE /* MOVM.IA.S.W (R13), [R15] */

```

22.7.2 IRQfiq and vfiq

```

⟨IRQ other cases(arm) 499f⟩≡ (151d)
    IRQfiq      = IRQusb, /* only one source can be FIQ */

⟨irqenable() if IRQfiq(arm) 499g⟩≡ (153b)
    if(irq == IRQfiq){
        ⟨irqenable() sanity check no previous FIQ(arm) 500b⟩
        vfiq = v;
        ip->FIQctl = Fiqenable | irq;
    }

⟨global vfiq(arm) 499h⟩≡ (712c)
    static Vctl *vfiq;

```

```
⟨constant Fiqenable(arm) 500a)≡ (712b)
    Fiqenable = 1<<7,
```

```
⟨irqenable() sanity check no previous FIQ(arm) 500b)≡ (499g)
    assert((ip->FIQctl & Fiqenable) == 0);
    assert((*enable & v->mask) == 0);
```

22.7.3 fiq()

Unlike IRQ dispatch which walks a linked list of handlers, FIQ dispatch is trivial: there is at most one FIQ source, stored in the global `vfiq`. The `ureg->pc -= 4` adjustment is the standard ARM convention for interrupt return address correction.

```
⟨function fiq(arm) 500c)≡ (712c)
/*
 * called direct from lexception.s to handle fiq interrupt.
 */
void
fiq(Ureg *ureg)
{
    Vctl *v;

    cpu->perf.intrts = arch_perfticks();
    v = vfiq;
    if(v == nil)
        panic("cpu%d: unexpected item in bagging area", cpu->cpuno);
    cpu->intr++;
    ureg->pc -= 4;
    arch_coherence();

    // Dispatch!
    v->f(ureg, v->a);

    arch_coherence();
    intrtime();
}
```

```
⟨function splfhi(arm) 500d)≡ (692)
TEXT splfhi(SB), 1, $-4
    MOVW    $(CPUADDR+4), R2        /* save caller pc in Mach */
    MOVW    R14, 0(R2)

    MOVW    CPSR, R0                /* turn off irqs and fiqs */
    ORR     $(PsrDirq|PsrDfiq), R0, R1
    MOVW    R1, CPSR
    RET
```

22.8 Power management (ARM)

Power management on the Raspberry Pi is handled through the VideoCore mailbox interface (see [vcreq]). The kernel can turn individual peripherals on or off by sending a `TagSetpower` request. Currently only `PowerUsb` is used in practice.

```
⟨enum Power(arm) 500e)≡ (685a)
enum {
    PowerSd      = 0,
    PowerUart0,
```

```

    PowerUart1,
    PowerUsb,
    PowerI2c0,
    PowerI2c1,
    PowerI2c2,
    PowerSpi,
    PowerCcp2tx,
};

```

```

⟨function setpower(arm) 501a)≡ (691d)
/*
 * Power management
 */
void
setpower(int dev, bool on)
{
    u32int buf[2];

    buf[0] = dev;
    buf[1] = Powerwait | on;
    vcreq(TagSetpower, buf, sizeof buf, sizeof buf);
}

```

```

⟨function getpower(arm) 501b)≡ (691d)
int
getpower(int dev)
{
    u32int buf[2];

    buf[0] = dev;
    buf[1] = 0;
    if(vcreq(TagGetpower, buf, sizeof buf[0], sizeof buf) != sizeof buf)
        return ERROR_NEG1;
    return buf[0] & 1;
}

```

22.9 Alignments (ARM)

```

⟨function arch_validalign(arm) 501c)≡ (717a)
/*
 * called in syscallfmt.c, sysfile.c, sysproc.c
 */
void
arch_validalign(uintptr addr, unsigned align)
{
    /*
     * Plan 9 is a 32-bit O/S, and the hardware it runs on
     * does not usually have instructions which move 64-bit
     * quantities directly, synthesizing the operations
     * with 32-bit move instructions. Therefore, the compiler
     * (and hardware) usually only enforce 32-bit alignment,
     * if at all.
     *
     * Take this out if the architecture warrants it.
     */
    if(align == sizeof(vlong))
        align = sizeof(long);
}

```

```

/*
 * Check align is a power of 2, then addr alignment.
 */
if((align != 0 && !(align & (align-1))) && !(addr & (align-1)))
    return;
postnote(up, 1, "sys: odd address", NDebug);
error(Ebadarg);
/*NOTREACHED*/
}

```

22.10 Real-time scheduling

Plan 9 includes an Earliest Deadline First (EDF) real-time scheduler, configurable through `/proc/x/ctl` commands (`period`, `deadline`, `cost`, `admit`, `expel`). An admitted EDF process is guaranteed a CPU time budget (C) within each period (T), with a deadline (D) by which the budget must be consumed. EDF processes run at priorities above all best-effort processes (`PriEdf` and `PriRelease` are above `Npriq`). The scheduler hooks into the normal scheduling path: `[runproc]` checks if the selected process is EDF and calls `[edfrun]` to start a deadline timer, `[sched]` skips priority adjustment for EDF processes, and `[ready]` calls `[edfready]` to handle release timing.

<Priority cases for real-time priority 502a>≡ (190b)

```

PriRelease = Npriq, /* released edf processes */
PriEdf     = Npriq+1, /* active edf processes */
PriExtra   = Npriq-1, /* edf processes at high best-effort pri */

```

<runproc() test for empty real-time scheduling queue 502b>≡ (203b)

```

runq[Nrq-1].head == nil && runq[Nrq-2].head == nil

```

<runproc() test if p is a real-time process 502c>≡ (196)

```

if(edflock(p)){
    edfrun(p, rq == &runq[PriEdf]); /* start deadline timer and do admin */
    edfunlock();
}

```

<newproc() set fields of p 502d>+≡ (108c) <453c 536g>

```

p->edf = nil;

```

<updatecpu() return if real-time process 502e>≡ (209i)

```

if(p->edf)
    return;

```

<schedinit() locals 502f>≡ (195e)

```

Edf *e;

```

<schedinit() optional real-time edfrecord() 502g>≡ (199a)

```

if((e = up->edf) && (e->flags & Admitted))
    edfrecord(up);

```

<schedinit() optional real-time edfstop() 502h>≡ (131c)

```

edfstop(up);
if (up->edf)
    free(up->edf);
up->edf = nil;

```

<ready() optional edfready() for real-time scheduling 502i>≡ (202c)

```

if(edfready(p)){
    arch_splx(s);
    return;
}

```

`<sys.sleep() optional edfyield() for real-time scheduling 503a>≡ (223a)`

```
if (up->edf && (up->edf->flags & Admitted))
    edfyield();
else
```

EDF parameters are set by writing commands to `/proc/x/ctl`: “period 10ms”, “cost 2ms”, “deadline 8ms”, then “admit” to activate the schedule. The kernel checks $C \neq D \neq T$ before running the schedulability test.

`<procctlreq() optional real-time commands 503b>≡ (554b)`

```
/* real time */
case CMperiod:
    if(p->edf == nil)
        edfinit(p);
    if(e=parsetime(&time, cb->f[1])) /* time in ns */
        error(e);
    edfstop(p);
    p->edf->T = time/1000; /* Edf times are in microseconds */
    break;
case CMdeadline:
    if(p->edf == nil)
        edfinit(p);
    if(e=parsetime(&time, cb->f[1]))
        error(e);
    edfstop(p);
    p->edf->D = time/1000;
    break;
case CMcost:
    if(p->edf == nil)
        edfinit(p);
    if(e=parsetime(&time, cb->f[1]))
        error(e);
    edfstop(p);
    p->edf->C = time/1000;
    break;
case CMsporadic:
    if(p->edf == nil)
        edfinit(p);
    p->edf->flags |= Sporadic;
    break;
case CMdeadlinenotes:
    if(p->edf == nil)
        edfinit(p);
    p->edf->flags |= Sendnotes;
    break;
case CMadmit:
    if(p->edf == 0)
        error("edf params");
    if(e = edfadm(p))
        error(e);
    break;
case CMextra:
    if(p->edf == nil)
        edfinit(p);
    p->edf->flags |= Extratime;
    break;
case CMexpel:
    if(p->edf)
        edfstop(p);
    break;
```


<sched() if real-time process then do nothing, do not reprioritize 504a>≡ (209a)

```
if(p->edf) {
}
```

<pexit() optional edfstop() for real-time scheduling 504b>≡ (130b)

```
edfstop(up);
```

<Proc scheduling fields 504c>+≡ (49f) <209d 504d>

```
<Proc optional edf field for real-time scheduling 504e>
```

<Proc scheduling fields 504d>+≡ (49f) <504c 675a>

```
ulong readytime; /* time process came ready */
```

<Proc optional edf field for real-time scheduling 504e>≡ (504c)

```
// option<ref_own?<edf>>
```

```
Edf *edf; /* if non-null, real-time proc, edf contains scheduling params */
```

On a uniprocessor, if a real-time process spins on a lock held by a lower-priority process, the lower-priority process will never get to run and release the lock—classic priority inversion. The fix is to set the EDF deadline to “now”, which causes the real-time process to be descheduled at the next scheduling point, giving the lock holder a chance to run. On a multiprocessor, this is not needed because the lock holder may be running on another CPU.

<lock() optional priority-inversion for real-time process 504f>≡ (58d)

```
if(conf.ncpu < 2 && up && up->edf && (up->edf->flags & Admitted)){
    /*
     * Priority inversion, yield on a uniprocessor; on a
     * multiprocessor, the other processor will unlock
     */
    print("inversion %p pc %#lux proc %lud held by pc %#lux proc %lud\n",
        l, pc, up ? up->pid : 0, l->pc, l->p ? l->p->pid : 0);
    up->edf->d = todget(nil); /* yield to process with lock */
}
```

<enum edfflags 504g>≡ (668c)

```
enum EdfFlags
{
    /* Edf.flags field */
    Admitted    = 0x01,
    Sporadic    = 0x02,
    Yieldonblock = 0x04,
    Sendnotes   = 0x08,
    Deadline    = 0x10,
    Yield       = 0x20,
    Extratime   = 0x40,
};
```

The Edf structure stores both the task parameters (period T, deadline D, cost C) and the per-instance state (current release time r, current deadline d, remaining slice S). It also embeds a **Timer** (via anonymous inclusion) so the scheduler can set deadline and release timers directly. The **testDelta/testtime** fields are used only during the admission test, which simulates the schedule to verify that adding a new real-time task will not cause any existing task to miss its deadline.

<struct Edf 504h>≡ (668c)

```
struct Edf {
    /* All times in µs */
    /* time intervals */
    long    D; /* Deadline */
    long    Delta; /* Inherited deadline */
    long    T; /* period */
};
```

```

long    C;    /* Cost */
long    S;    /* Slice: time remaining in this period */
/* times (only low-order bits of absolute time) */
long    r;    /* (this) release time */
long    d;    /* (this) deadline */
long    t;    /* Start of next period, t += T at release */
long    s;    /* Time at which this proc was last scheduled */

/* for schedulability testing */
long    testDelta;
int     testtype; /* Release or Deadline */
long    testtime;
Proc    *testnext;

/* other */
// set<enum<edfflags>>
ushort   flags;

Timer;

/* Stats */
long     edfused;
long     extraused;
long     aged;
ulong    periods;
ulong    missed;
};

```

processes/edf.c

The EDF implementation centers on three interrupt-driven callbacks: [deadlineintr] fires when a process has consumed its time slice and forces a reschedule, [releaseintr] fires at the start of a new period to make the process eligible again, and [release] resets the per-period bookkeeping (remaining slice S, new deadline d). The [edfadm] function performs an offline schedulability test before admitting a new real-time task: it simulates the combined schedule of all admitted tasks to verify that no deadlines will be missed. Sporadic tasks (those with the `Sporadic` flag) are released at most once per period, while periodic tasks are released at fixed intervals regardless of when they last ran.

```

<edf.c statistics 505a>≡ (505c)
/* Statistics stuff */
ulong    edfnrun;
//ulong   nilcount;
//ulong   scheds;
//int     misseddeadlines;

<Procstate cases 505b>+≡ (51b) <426d
    Waitrelease, // for real-time scheduling

<edf.c 505c>≡
/* EDF scheduling */
<kernel basic includes 35a>

#include    <trace.h>

// in proc.c
extern Schedq  runq[Nrq];
extern ulong   delayed_schedulers;
extern int     nrdy;

```

```

extern ulong    runvec;
// in devcons.c
extern int panicking;
// used to be in edf.h
//unused: extern Lock edftestlock; /* for atomic admitting/expelling */

<edf.c debugging macro ??>

enum
{
    Maxsteps = 200 * 100 * 2, /* 100 periods of 200 procs */
};

static long now;    /* Low order 32 bits of time in µs */

<edf.c statistics 505a>

int    edfinited;

/* Edfschedlock protects modification of admission params */
QLock    edfschedlock;
static Lock thelock;

enum{
    Dl, /* Invariant for schedulability test: Dl < Rl */
    Rl,
};

static char *testschedulability(Proc*);
static Proc *qschedulability;

enum {
    Onemicrosecond =    1,
    Onemillisecond =   1000,
    Onesecond =        1000000,
    OneRound =         Onemillisecond/2,
};

static int
timeconv(Fmt *f)
{
    char buf[128], *sign;
    vlong t;

    buf[0] = 0;
    switch(f->r) {
    case 'U':
        t = va_arg(f->args, uvlong);
        break;
    case 't':
        /* vlong in nanoseconds */
        t = va_arg(f->args, long);
        break;
    default:
        return fmtstrcpy(f, "(timeconv)");
    }
    if (t < 0) {
        sign = "-";
        t = -t;
    }

```

```

}
else
    sign = "";
if (t > Onesecond){
    t += OneRound;
    snprintf(buf, sizeof buf, "%s%d.%.3ds", sign,
        (int)(t / Onesecond),
        (int)(t % Onesecond)/Onemillisecond);
}else if (t > Onemillisecond)
    snprintf(buf, sizeof buf, "%s%d.%.3dms", sign,
        (int)(t / Onemillisecond), (int)(t % Onemillisecond));
else
    snprintf(buf, sizeof buf, "%s%dµs", sign, (int)t);
return fmtstrcpy(f, buf);
}

#ifdef EDFCYCLES
long edfcycles;
#endif

Edf*
edflock(Proc *p)
{
    Edf *e;

    if (p->edf == nil)
        return nil;
    ilock(&thelock);
    if((e = p->edf) && (e->flags & Admitted)){
        thelock.pc = getcallerpc(&p);
#ifdef EDFCYCLES
        edfcycles -= lcycles();
#endif
        now = arch_us();
        return e;
    }
    iunlock(&thelock);
    return nil;
}

void
edfunlock(void)
{
#ifdef EDFCYCLES
    edfcycles += lcycles();
#endif
    edfnrun++;
    iunlock(&thelock);
}

void
edfinit(Proc* p)
{
    if(!edfinitd){
        fmtinstall('t', timeconv);
        edfinitd++;
    }
    now = arch_us();
    DPRINT("%lud edfinit %lud[%s]\n", now, p->pid, statename[p->state]);
}

```

```

    p->edf = malloc(sizeof(Edf));
    if(p->edf == nil)
        error(Enomem);
    return;
}

static void
deadlineintr(Ureg*, Timer *t)
{
    /* Proc reached deadline */
    Proc *p;
    void (*pt)(Proc*, int, vlong);

    if(panicking || active.exiting)
        return;

    p = t->ta;
    now = arch_us();
    DPRINT("%lud deadlineintr %lud[%s]\n", now, p->pid, statename[p->state]);
    /* If we're interrupting something other than the proc pointed to by t->a,
     * we've already achieved recheduling, so we need not do anything
     * Otherwise, we must cause a reschedule, but if we call sched()
     * here directly, the timer interrupt routine will not finish its business
     * Instead, we cause the resched to happen when the interrupted proc
     * returns to user space
     */
    if(p == up){
        if(up->trace && (pt = proctrace))
            pt(up, SInts, 0);
        up->delaysched++;
        delayed_schds++;
    }
}

static void
release(Proc *p)
{
    /* Called with edflock held */
    Edf *e;
    void (*pt)(Proc*, int, vlong);
    long n;
    vlong nowns;

    e = p->edf;
    e->flags &= ~Yield;
    if(e->d - now < 0){
        e->periods++;
        e->r = now;
        if((e->flags & Sporadic) == 0){
            /*
             * Non sporadic processes stay true to their period;
             * calculate next release time.
             * Second test limits duration of while loop.
             */
            if((n = now - e->t) > 0){
                if(n < e->T)
                    e->t += e->T;
                else
                    e->t = now + e->T - (n % e->T);
            }
        }
    }
}

```

```

    }elseif
        /* Sporadic processes may not be released earlier than
         * one period after this release
         */
        e->t = e->r + e->T;
    }
    e->d = e->r + e->D;
    e->S = e->C;
    DPRINT("%lud release %lud[%s], r=%lud, d=%lud, t=%lud, S=%lud\n",
        now, p->pid, statename[p->state], e->r, e->d, e->t, e->S);
    if(pt = proctrace){
        nowns = todget(nil);
        pt(p, SRelease, nowns);
        pt(p, SDeadline, nowns + 1000LL*e->D);
    }
}
}else{
    DPRINT("%lud release %lud[%s], too late t=%lud, called from %#p\n",
        now, p->pid, statename[p->state], e->t, getcallerpc(&p));
}
}

static void
releaseintr(Ureg*, Timer *t)
{
    Proc *p;
    Schedq *rq;

    if(panicking || active.exiting)
        return;

    p = t->ta;
    if((edflock(p)) == nil)
        return;
    DPRINT("%lud releaseintr %lud[%s]\n", now, p->pid, statename[p->state]);
    switch(p->state){
    default:
        edfunlock();
        return;
    case Ready:
        /* remove proc from current runq */
        rq = &runq[p->priority];
        if(dequeueproc(rq, p) != p){
            DPRINT("releaseintr: can't find proc or lock race\n");
            release(p); /* It'll start best effort */
            edfunlock();
            return;
        }
        p->state = Waitrelease;
        /* fall through */
    case Waitrelease:
        release(p);
        edfunlock();
        if(p->state == Wakeme){
            iprint("releaseintr: wakeme\n");
        }
        ready(p);
        if(up){
            up->delaysched++;
            delayededsched++;
        }
    }
}

```

```

        return;
case Running:
    release(p);
    edfrun(p, 1);
    break;
case Wakeme:
    release(p);
    edfunlock();
    if(p->trend)
        wakeup(p->trend);
    p->trend = nil;
    if(up){
        up->delaysched++;
        delayedscheds++;
    }
    return;
}
edfunlock();
}

void
edfrecord(Proc *p)
{
    long used;
    Edf *e;
    void (*pt)(Proc*, int, vlong);

    if((e = edflock(p)) == nil)
        return;
    used = now - e->s;
    if(e->d - now <= 0)
        e->edfused += used;
    else
        e->extraused += used;
    if(e->S > 0){
        if(e->S <= used){
            if(pt = proctrace)
                pt(p, SSlice, 0);
            DPRINT("%lud edfrecord slice used up\n", now);
            e->d = now;
            e->S = 0;
        }else
            e->S -= used;
    }
    e->s = now;
    edfunlock();
}

void
edfrun(Proc *p, int edfpri)
{
    Edf *e;
    void (*pt)(Proc*, int, vlong);
    long tns;

    e = p->edf;
    /* Called with edflock held */
    if(edfpri){
        tns = e->d - now;
        if(tns <= 0 || e->S == 0){

```

```

        /* Deadline reached or resources exhausted,
        * deschedule forthwith
        */
        p->delaysched++;
        delayedscheds++;
        e->s = now;
        return;
    }
    if(e->S < tns)
        tns = e->S;
    if(tns < 20)
        tns = 20;
    e->tns = 1000LL * tns; /* µs to ns */
    if(e->tt == nil || e->tf != deadlineintr){
        DPRINT("%lud edfrun, deadline=%lud\n", now, tns);
    }else{
        DPRINT("v");
    }
    if(p->trace && (pt = proctrace))
        pt(p, SInte, todget(nil) + e->tns);
    e->tmode = Trelative;
    e->tf = deadlineintr;
    e->ta = p;
    timeradd(e);
}
else{
    DPRINT("<");
}
e->s = now;
}

```

```

char *
edfadmit(Proc *p)
{
    char *err;
    Edf *e;
    int i;
    Proc *r;
    void (*pt)(Proc*, int, vlong);
    long tns;

    e = p->edf;
    if (e->flags & Admitted)
        return "task state";    /* should never happen */

    /* simple sanity checks */
    if (e->T == 0)
        return "T not set";
    if (e->C == 0)
        return "C not set";
    if (e->D > e->T)
        return "D > T";
    if (e->D == 0) /* if D is not set, set it to T */
        e->D = e->T;
    if (e->C > e->D)
        return "C > D";

    qlock(&edfschedlock);
    if (err = testschedulability(p)){
        qunlock(&edfschedlock);
        return err;
    }
}

```



```

}
e->flags |= Admitted;

edflock(p);

if(p->trace && (pt = proctrace))
    pt(p, SAdmit, 0);

/* Look for another proc with the same period to synchronize to */
SET(r);
for(i=0; i<conf.nproc; i++) {
    r = proctab(i);
    if(r->state == Dead || r == p)
        continue;
    if (r->edf == nil || (r->edf->flags & Admitted) == 0)
        continue;
    if (r->edf->T == e->T)
        break;
}
if (i == conf.nproc){
    /* Can't synchronize to another proc, release now */
    e->t = now;
    e->d = 0;
    release(p);
    if (p == up){
        DPRINT("%lud edfadmit self %lud[%s], release now: r=%lud d=%lud t=%lud\n",
            now, p->pid, statename[p->state], e->r, e->d, e->t);
        /* We're already running */
        edfrun(p, 1);
    }else{
        /* We're releasing another proc */
        DPRINT("%lud edfadmit other %lud[%s], release now: r=%lud d=%lud t=%lud\n",
            now, p->pid, statename[p->state], e->r, e->d, e->t);
        p->ta = p;
        edfunlock();
        qunlock(&edfschedlock);
        releaseintr(nil, p);
        return nil;
    }
}
}else{
    /* Release in synch to something else */
    e->t = r->edf->t;
    if (p == up){
        DPRINT("%lud edfadmit self %lud[%s], release at %lud\n",
            now, p->pid, statename[p->state], e->t);
        edfunlock();
        qunlock(&edfschedlock);
        return nil;
    }else{
        DPRINT("%lud edfadmit other %lud[%s], release at %lud\n",
            now, p->pid, statename[p->state], e->t);
        if(e->tt == nil){
            e->tf = releaseintr;
            e->ta = p;
            tns = e->t - now;
            if(tns < 20)
                tns = 20;
            e->tns = 1000LL * tns;
            e->tmode = Trelative;
            timeradd(e);
        }
    }
}

```

```

        }
    }
}
edfunlock();
qunlock(&edfschedlock);
return nil;
}

void
edfstop(Proc *p)
{
    Edf *e;
    void (*pt)(Proc*, int, vlong);

    if(e = edflock(p)){
        DPRINT("%lud edfstop %lud[%s]\n", now, p->pid, statename[p->state]);
        if(p->trace && (pt = proctrace))
            pt(p, SExpel, 0);
        e->flags &= ~Admitted;
        if(e->tt)
            timerdel(e);
        edfunlock();
    }
}

static int
yfn(void *)
{
    now = arch_us();
    return up->trend == nil || now - up->edf->r >= 0;
}

void
edfyield(void)
{
    /* sleep until next release */
    Edf *e;
    void (*pt)(Proc*, int, vlong);
    long n;

    if((e = edflock(up)) == nil)
        return;
    if(up->trace && (pt = proctrace))
        pt(up, SYield, 0);
    if((n = now - e->t) > 0){
        if(n < e->T)
            e->t += e->T;
        else
            e->t = now + e->T - (n % e->T);
    }
    e->r = e->t;
    e->flags |= Yield;
    e->d = now;
    if (up->tt == nil){
        n = e->t - now;
        if(n < 20)
            n = 20;
        up->tns = 1000LL * n;
        up->tfr = releaseintr;
        up->tmode = Trelative;
    }
}

```

```

        up->ta = up;
        up->trend = &up->sleepr;
        timeradd(up);
    }else if(up->tf != releaseintr)
        print("edfyield: surprise! %#p\n", up->tf);
    edfunlock();
    sleep(&up->sleepr, yfn, nil);
}

int
edfready(Proc *p)
{
    Edf *e;
    Schedq *rq;
    Proc *l, *pp;
    void (*pt)(Proc*, int, vlong);
    long n;

    if((e = edflock(p)) == nil)
        return 0;

    if(p->state == Wakeme && p->r){
        iprint("edfready: wakeme\n");
    }
    if(e->d - now <= 0){
        /* past deadline, arrange for next release */
        if((e->flags & Sporadic) == 0){
            /*
             * Non sporadic processes stay true to their period;
             * calculate next release time.
             */
            if((n = now - e->t) > 0){
                if(n < e->T)
                    e->t += e->T;
                else
                    e->t = now + e->T - (n % e->T);
            }
        }
        if(now - e->t < 0){
            /* Next release is in the future, schedule it */
            if(e->tt == nil || e->tf != releaseintr){
                n = e->t - now;
                if(n < 20)
                    n = 20;
                e->tns = 1000LL * n;
                e->tmode = Trelative;
                e->tf = releaseintr;
                e->ta = p;
                timeradd(e);
                DPRINT("%lud edfready %lud[%s], release=%lud\n",
                    now, p->pid, statename[p->state], e->t);
            }
            if(p->state == Running && (e->flags & (Yield|Yieldonblock)) == 0 && (e->flags & Extratime)){
                /* If we were running, we've overrun our CPU allocation
                 * or missed the deadline, continue running best-effort at low priority
                 * Otherwise we were blocked. If we don't yield on block, we continue
                 * best effort
                 */
                DPRINT(">");
                p->basepri = PriExtra;
            }
        }
    }
}

```

```

        p->fixedpri = 1;
        edfunlock();
        return 0; /* Stick on runq[PriExtra] */
    }
    DPRINT("%lud edfready %lud[%s] wait release at %lud\n",
        now, p->pid, statename[p->state], e->t);
    p->state = Waitrelease;
    edfunlock();
    return 1; /* Make runnable later */
}
DPRINT("%lud edfready %lud %s release now\n", now, p->pid, statename[p->state]);
/* release now */
release(p);
}
edfunlock();
DPRINT("^");
rq = &runq[PriEdf];
/* insert in queue in earliest deadline order */
lock(runq);
l = nil;
for(pp = rq->head; pp; pp = pp->rnext){
    if(pp->edf->d > e->d)
        break;
    l = pp;
}
p->rnext = pp;
if (l == nil)
    rq->head = p;
else
    l->rnext = p;
if(pp == nil)
    rq->tail = p;
rq->n++;
nrdy++;
runvec |= 1 << PriEdf;
p->priority = PriEdf;
p->readytime = cpu->ticks;
p->state = Ready;
unlock(runq);
if(p->trace && (pt = proctrace))
    pt(p, SReady, 0);
return 1;
}

```

```

static void
testenq(Proc *p)
{
    Proc *xp, **xpp;
    Edf *e;

    e = p->edf;
    e->testnext = nil;
    if (qschedulability == nil) {
        qschedulability = p;
        return;
    }
    SET(xp);
    for (xpp = &qschedulability; *xpp; xpp = &xp->edf->testnext) {
        xp = *xpp;

```

```

        if (e->testtime - xp->edf->testtime < 0
            || (e->testtime == xp->edf->testtime && e->testtype < xp->edf->testtype)){
            e->testnext = xp;
            *xpp = p;
            return;
        }
    }
    assert(xp->edf->testnext == nil);
    xp->edf->testnext = p;
}

static char *
testschedulability(Proc *theproc)
{
    Proc *p;
    long H, G, Cb, ticks;
    int steps, i;

    /* initialize */
    DPRINT("schedulability test %lud\n", theproc->pid);
    qschedulability = nil;
    for(i=0; i<conf.nproc; i++) {
        p = proctab(i);
        if(p->state == Dead)
            continue;
        if ((p->edf == nil || (p->edf->flags & Admitted) == 0) && p != theproc)
            continue;
        p->edf->testtype = Rl;
        p->edf->testtime = 0;
        DPRINT("\tInit: edfenqueue %lud\n", p->pid);
        testenq(p);
    }
    H=0;
    G=0;
    for(steps = 0; steps < Maxsteps; steps++){
        p = qschedulability;
        qschedulability = p->edf->testnext;
        ticks = p->edf->testtime;
        switch (p->edf->testtype){
        case Dl:
            H += p->edf->C;
            Cb = 0;
            DPRINT("\tStep %3d, Ticks %lud, pid %lud, deadline, H += %lud → %lud, Cb = %lud\n",
                steps, ticks, p->pid, p->edf->C, H, Cb);
            if (H+Cb>ticks){
                DPRINT("not schedulable\n");
                return "not schedulable";
            }
            p->edf->testtime += p->edf->T - p->edf->D;
            p->edf->testtype = Rl;
            testenq(p);
            break;
        case Rl:
            DPRINT("\tStep %3d, Ticks %lud, pid %lud, release, G %lud, C%lud\n",
                steps, ticks, p->pid, p->edf->C, G);
            if(ticks && G <= ticks){
                DPRINT("schedulable\n");
                return nil;
            }
            G += p->edf->C;

```

```

        p->edf->testtime += p->edf->D;
        p->edf->testtype = D1;
        testenq(p);
        break;
    default:
        assert(0);
}
}
DPRINT("probably not schedulable\n");
return "probably not schedulable";
}

```

Tracing scheduling events

```

⟨enum Tevent real-time scheduling events 517a⟩≡ (440d)
    SAdmit, /* Edf admit */
    SRelease, /* Edf release, waiting to be scheduled */
    SYield, /* blocked waiting for release */
    SSlice, /* slice exhausted */
    SDeadline, /* proc's deadline */
    SExpel, /* Edf expel */
    SInts, /* Interrupt start */
    SInte, /* Interrupt end */

```

22.11 Fileservers

When a file server is mounted on a directory, the kernel forwards file operations (open, read, write, walk) to the server via the 9P protocol. The `Mnt` structure represents a connection to a file server: it holds the transport channel (`c`), the negotiated message size, and a queue of pending RPCs. The key design is multiplexing: multiple processes can have files open on the same server, but 9P messages must be serialized onto a single connection. The `rip` (reader in progress) field gates access—only one process reads replies at a time, dispatching replies to other waiting processes via `[mountmux]`.

22.11.1 Mnt

```

⟨struct Mnt 517b⟩≡ (645c)
    struct Mnt
    {
        Lock;
        /* references are counted using c->ref; channels on this mount point incref(c->mchan) == Mnt.c */
        Chan *c; /* Channel to file service */
        Proc *rip; /* Reader in progress */
        Mntrpc *queue; /* Queue of pending requests on this channel */
        ulong id; /* Multiplexer id for channel check */
        Mnt *list; /* Free list */
        int flags; /* cache */
        int msize; /* data + IOHDRSZ */
        char *version; /* 9P version */
        Queue *q; /* input queue */
    };

```

```

⟨struct Mntalloc 517c⟩≡ (645c)
    /*
    * References are managed as follows:
    * The channel to the server - a network connection or pipe - has one

```

```

* reference for every Chan open on the server. The server channel has
* c->mux set to the Mnt used for muxing control to that server. Mnts
* have no reference count; they go away when c goes away.
* Each channel derived from the mount point has mchan set to c,
* and increfs/decrefs mchan to manage references on the server
* connection.
*/
struct Mntalloc
{
    Mnt*    list;        /* Mount devices in use */
    Mnt*    mntfree;     /* Free list */
    Mntrpc* rpcfree;
    int nrpcfree;
    int nrpcused;
    ulong   id;
    ulong   tagmask[NMASK];

    // extra
    Lock;

};

⟨global mntalloc 518a⟩≡                                     (651c)
    struct Mntalloc mntalloc;

⟨constants tags 518b⟩≡                                       (645c)
    enum
    {
        TAGSHIFT = 5,          /* ulong has to be 32 bits */
        TAGMASK = (1<<TAGSHIFT)-1,
        NMASK = (64*1024)>>TAGSHIFT,
    };

⟨function freetag 518c⟩≡                                       (651c)
    void
    freetag(int t)
    {
        mntalloc.tagmask[t>>TAGSHIFT] &= ~(1<<(t&TAGMASK));
    }

⟨function mntfree 518d⟩≡                                       (651c)
    void
    mntfree(Mntrpc *r)
    {
        if(r->b != nil)
            freeblist(r->b);
        lock(&mntalloc);
        if(mntalloc.nrpcfree >= 10){
            free(r->rpc);
            freetag(r->request.tag);
            free(r);
        }
        else{
            r->list = mntalloc.rpcfree;
            mntalloc.rpcfree = r;
            mntalloc.nrpcfree++;
        }
        mntalloc.nrpcused--;
        unlock(&mntalloc);
    }

```

```

⟨function mntpntfree 519a⟩≡ (651c)
void
mntpntfree(Mnt *m)
{
    Mnt *f, **l;
    Queue *q;

    lock(&mntalloc);
    l = &mntalloc.list;
    for(f = *l; f; f = f->list) {
        if(f == m) {
            *l = m->list;
            break;
        }
        l = &f->list;
    }
    m->list = mntalloc.mntfree;
    mntalloc.mntfree = m;
    q = m->q;
    unlock(&mntalloc);

    qfree(q);
}

⟨function muxclose 519b⟩≡ (651c)
void
muxclose(Mnt *m)
{
    Mntrpc *q, *r;

    for(q = m->queue; q; q = r) {
        r = q->list;
        mntfree(q);
    }
    m->id = 0;
    free(m->version);
    m->version = nil;
    mntpntfree(m);
}

```

22.11.2 MntRpc

Each in-flight 9P request is represented by an **Mntrpc**. It pairs the outgoing **Fcall** (request) with the space for the incoming reply, plus a **Rendez** for the requesting process to sleep on while waiting. The **flushed** pointer supports the 9P flush protocol: if a process is interrupted, the kernel sends a **Tflush** to cancel the original request.

```

⟨struct Mntrpc 519c⟩≡ (645c)
struct Mntrpc
{
    Chan* c;      /* Channel for whom we are working */
    Mntrpc* list; /* Free/pending list */
    Fcall request; /* Outgoing file system protocol message */
    Fcall reply;   /* Incoming reply */
    Mnt* m;        /* Mount device during rpc */
    Rendez r;      /* Place to hang out */
    byte* rpc;     /* I/O Data buffer */
    uint rpclen;   /* len of buffer */
    Block *b;     /* reply blocks */
}

```



```

char done; /* Rpc completed */
uvlong stime; /* start time for mnt statistics */
ulong reqlen; /* request length for mnt statistics */
ulong replen; /* reply length for mnt statistics */
MntRpc* flushed; /* message this one flushes */
};

```

22.11.3 Fcall

22.11.4 mntversion()

⟨ChannelFlag cases 520a⟩+≡ (257b) ◁281h ??▷
 CMSG = 0x0002, /* the message channel for a mount */

Before any 9P operations can proceed, the client and server must agree on a protocol version and maximum message size through the Tversion/Rversion exchange. This is the only 9P message that is not multiplexed—it must be the first message on a new connection. After the handshake, [mntversion] allocates the Mnt structure and attaches it to the channel via c->mux.

⟨function mntversion 520b⟩≡ (651c)

```

/*
 * Version is not multiplexed: message sent only once per connection.
 */
long
mntversion(Chan *c, char *version, int msize, int returnlen)
{
    Fcall f;
    uchar *msg;
    Mnt *m;
    char *v;
    long k, l;
    uvlong oo;
    char buf[128];

    qlock(&c->umqlock); /* make sure no one else does this until we've established ourselves */
    if(waserror()){
        qunlock(&c->umqlock);
        nexterror();
    }

    /* defaults */
    if(msize == 0)
        msize = MAXRPC;
    if(msize > c->iounit && c->iounit != 0)
        msize = c->iounit;
    v = version;
    if(v == nil || v[0] == '\0')
        v = VERSION9P;

    /* validity */
    if(msize < 0)
        error("bad iounit in version call");
    if(strncmp(v, VERSION9P, strlen(VERSION9P)) != 0)
        error("bad 9P version specification");

    m = c->mux;

    if(m != nil){
        qunlock(&c->umqlock);

```

```

    poperror();

    strencpy(buf, buf+sizeof buf, m->version);
    k = strlen(buf);
    if(strncmp(buf, v, k) != 0){
        snprintf(buf, sizeof buf, "incompatible 9P versions %s %s", m->version, v);
        error(buf);
    }
    if(returnlen > 0){
        if(returnlen < k)
            error(Eshort);
        memmove(version, buf, k);
    }
    return k;
}

f.type = Tversion;
f.tag = NOTAG;
f.msize = msize;
f.version = v;
msg = malloc(8192+IOHDRSZ);
if(msg == nil)
    exhausted("version memory");
if(waserror()){
    free(msg);
    nexterror();
}
k = convS2M(&f, msg, 8192+IOHDRSZ);
if(k == 0)
    error("bad fversion conversion on send");

lock(c);
oo = c->offset;
c->offset += k;
unlock(c);

l = devtab[c->type]->write(c, msg, k, oo);

if(l < k){
    lock(c);
    c->offset -= k - l;
    unlock(c);
    error("short write in fversion");
}

/* message sent; receive and decode reply */
k = devtab[c->type]->read(c, msg, 8192+IOHDRSZ, c->offset);
if(k <= 0)
    error("EOF receiving fversion reply");

lock(c);
c->offset += k;
unlock(c);

l = convM2S(msg, k, &f);
if(l != k)
    error("bad fversion conversion on reply");
if(f.type != Rversion){
    if(f.type == Rerror)
        error(f.ename);
}

```

```

        error("unexpected reply type in fversion");
    }
    if(f.msize > msize)
        error("server tries to increase msize in fversion");
    if(f.msize<256 || f.msize>1024*1024)
        error("nonsense value of msize in fversion");
    k = strlen(f.version);
    if(strncmp(f.version, v, k) != 0)
        error("bad 9P version returned from server");

    /* now build Mnt associated with this connection */
    lock(&mntalloc);
    m = mntalloc.mntfree;
    if(m != 0)
        mntalloc.mntfree = m->list;
    else {
        m = malloc(sizeof(Mnt));
        if(m == 0) {
            unlock(&mntalloc);
            exhausted("mount devices");
        }
    }
    m->list = mntalloc.list;
    mntalloc.list = m;
    m->version = nil;
    kstrdup(&m->version, f.version);
    m->id = mntalloc.id++;
    m->q = qopen(10*MAXRPC, 0, nil, nil);
    m->msize = f.msize;
    unlock(&mntalloc);

    if(returnlen > 0){
        if(returnlen < k)
            error(Eshort);
        memmove(version, f.version, k);
    }

    poperror(); /* msg */
    free(msg);

    lock(m);
    m->queue = 0;
    m->rip = 0;

    c->flag |= CMSG;
    c->mux = m;
    m->c = c;
    unlock(m);

    poperror(); /* c */
    qunlock(&c->umqlock);

    return k;
}

```

22.11.5 mountrpc()

The [mountrpc]/[mountio] pair implements the core RPC mechanism. [mountio] serializes the **Fcall**, writes it to the server channel, then enters the reader gate: processes take turns being the “reader in progress” (**rip**).

The reader calls [mnttrpcread] to receive a reply, then [mountmux] to match the reply's tag to the correct waiting Mnttrpc and wake up that process.

```

⟨function mountrpc 523a⟩≡                                     (? 0—1)
void
mountrpc(Mnt *m, Mnttrpc *r)
{
    char *sn, *cn;
    int t;

    r->reply.tag = 0;
    r->reply.type = Tmax;    /* can't ever be a valid message type */

    mountio(m, r);

    t = r->reply.type;
    switch(t) {
    case Rerror:
        error(r->reply.ename);
    case Rflush:
        error(Eintr);
    default:
        if(t == r->request.type+1)
            break;
        sn = "?";
        if(m->c->path != nil)
            sn = m->c->path->s;
        cn = "?";
        if(r->c != nil && r->c->path != nil)
            cn = r->c->path->s;
        print("mnt: proc %s %lud: mismatch from %s %s rep %#p tag %d fid %d T%d R%d rp %d\n",
            up->text, up->pid, sn, cn,
            r, r->request.tag, r->request.fid, r->request.type,
            r->reply.type, r->reply.tag);
        error(Emountrpc);
    }
}

```

```

⟨function mountio 523b⟩≡                                     (? 0—1)
void
mountio(Mnt *m, Mnttrpc *r)
{
    int n;

    while(waserror()) {
        if(m->rip == up)
            mntgate(m);
        if(strcmp(up->errstr, Eintr) != 0){
            mntflushfree(m, r);
            nexterror();
        }
        r = mntflushalloc(r, m->msize);
    }

    lock(m);
    r->m = m;
    r->list = m->queue;
    m->queue = r;
    unlock(m);
}

```

```

/* Transmit a file system rpc */
if(m->msize == 0)
    panic("msize");
n = convS2M(&r->request, r->rpc, m->msize);
if(n < 0)
    panic("bad message type in mountio");
if(devtab[m->c->type]->write(m->c, r->rpc, n, 0) != n)
    error(Emountrpc);
r->stime = arch_fastticks(nil);
r->reqlen = n;

/* Gate readers onto the mount point one at a time */
for(;;) {
    lock(m);
    if(m->rip == 0)
        break;
    unlock(m);
    sleep(&r->r, rpcattn, r);
    if(r->done){
        poperror();
        mntflushfree(m, r);
        return;
    }
}
m->rip = up;
unlock(m);
while(r->done == 0) {
    if(mntrpcread(m, r) < 0)
        error(Emountrpc);
    mountmux(m, r);
}
mntgate(m);
poperror();
mntflushfree(m, r);
}

```

22.11.6 devmnt

The mount device (`devmnt`) implements the kernel's 9P client. Its methods translate **Chan** operations into 9P messages, send them via `[mountrpc]`, and convert the replies back. `[mntattach]` sends **Tattach** to establish a session, and `[mntwalk]` sends **Twalk** to navigate the remote file tree.

`mntattach()`

```

⟨function mntattach 524⟩≡                                     (? 0—1)
static Chan*
mntattach(char *muxattach)
{
    Mnt *m;
    Chan *c;
    Mntrpc *r;
    struct Bogus bogus;

    bogus = *((struct Bogus *)muxattach);
    c = bogus.chan;

    m = c->mux;

```

```

if(m == nil){
    mntversion(c, nil, 0, 0);
    m = c->mux;
    if(m == nil)
        error(Enoversion);
}

c = mntchan();
if(waserror()) {
    /* Close must not be called since it will
       * call mnt recursively
       */
    chanfree(c);
    nexterror();
}

r = mntralloc(0, m->msize);

if(waserror()) {
    mntfree(r);
    nexterror();
}

r->request.type = Tattach;
r->request.fid = c->fid;
if(bogus.authchan == nil)
    r->request.afid = NOFID;
else
    r->request.afid = bogus.authchan->fid;
r->request.username = up->user;
r->request.aname = bogus.spec;

mountrpc(m, r);

c->qid = r->reply.qid;
c->mchan = m->c;
incref(m->c);
c->mqid = c->qid;

poperror(); /* r */
mntfree(r);

poperror(); /* c */

if(bogus.flags&MCACHE)
    c->flag |= CCACHE;
return c;
}

```

mntwalk()

```

⟨function mntwalk 525⟩≡                                     (? 0—1)
static Walkqid*
mntwalk(Chan *c, Chan *nc, char **name, int nname)
{
    int i, alloc;
    Mnt *m;
    Mntrpc *r;
    Walkqid *wq;

```

```

if(nc != nil)
    print("mntwalk: nc != nil\n");
if(nname > MAXWELEM)
    error("devmnt: too many name elements");
alloc = 0;
wq = smalloc(sizeof(Walkqid)+(nname-1)*sizeof(Qid));
if(waserror()){
    if(alloc && wq->clone!=nil)
        cclose(wq->clone);
    free(wq);
    return nil;
}

alloc = 0;
m = mntchk(c);
r = mntralloc(c, m->msize);
if(nc == nil){
    nc = devclone(c);
    /*
     * Until the other side accepts this fid, we can't mntclose it.
     * Therefore set type to 0 for now; rootclose is known to be safe.
     */
    nc->type = 0;
    alloc = 1;
}
wq->clone = nc;
nc->flag |= c->flag&CCACHE;

if(waserror()) {
    mntfree(r);
    nexterror();
}
r->request.type = Twalk;
r->request.fid = c->fid;
r->request.newfid = nc->fid;
r->request.nwnname = nname;
memmove(r->request.wname, name, nname*sizeof(char*));

mountrpc(m, r);

if(r->reply.nwqid > nname)
    error("too many QIDs returned by walk");
if(r->reply.nwqid < nname){
    if(alloc)
        cclose(nc);
    wq->clone = nil;
    if(r->reply.nwqid == 0){
        free(wq);
        wq = nil;
        goto Return;
    }
}

/* move new fid onto mnt device and update its qid */
if(wq->clone != nil){
    if(wq->clone != c){
        wq->clone->type = c->type;
        wq->clone->mchan = c->mchan;
        incref(c->mchan);
    }
}

```

```

    }
    if(r->reply.nwqid > 0)
        wq->clone->qid = r->reply.wqid[r->reply.nwqid-1];
}
wq->nqid = r->reply.nwqid;
for(i=0; i<wq->nqid; i++)
    wq->qid[i] = r->reply.wqid[i];

Return:
poperror();
mntfree(r);
poperror();
return wq;
}

```

22.12 Network

22.13 Security

Plan 9 security is based on a per-process user identity string (`up->user`) rather than numeric UIDs. The “host owner” (`eve`) is the privileged user, analogous to `root` on Unix. The `[iseve]` function checks whether the current process is running as the host owner. Authentication is handled outside the kernel via the `fauth/fversion` system calls, which set up an authenticated channel to a file server—the kernel itself does not implement any authentication protocol but provides the plumbing for user-space auth servers. A notable security feature is that any process can become “none” (the unprivileged identity) by writing to `/dev/user`, but no process can become anyone else. This one-way transition provides a simple sandboxing mechanism.

<global eve 527a>≡ (610b)
 char *eve;

<function iseve 527b>≡ (610b)
 bool iseve(void) {
 return strcmp(eve, up->user) == 0;
 }

<systab security syscalls 527c>≡ (41f)
 [FAUTH] sysfauth,
 [FVERSION] sysfversion,

<global hostdomain 527d>≡ (681)
 char hostdomain[DOMLEN];

<syscall fversion 527e>≡ (681)
 long
 sysfversion(ulong *arg)
 {
 char *vers;
 uint arglen, m, msize;
 Chan *c;

 msize = arg[1];
 vers = (char*)arg[2];
 arglen = arg[3];
 validaddr(arg[2], arglen, true);
 /* check there's a NUL in the version string */
 if(arglen==0 || memchr(vers, 0, arglen)==0)


```

        error(Ebadarg);
c = fdtochan(arg[0], ORDWR, false, true);
if(waserror()){
    cclose(c);
    nexterror();
}

m = mntversion(c, vers, msize, arglen);

cclose(c);
poperror();
return m;
}

```

<syscall fauth 528a>≡ (681)

```

long
sysfauth(ulong *arg)
{
    Chan *c, *ac;
    char *aname;
    int fd;

    validaddr(arg[1], 1, false);
    aname = validnamedup((char*)arg[1], true);
    if(waserror()){
        free(aname);
        nexterror();
    }
    c = fdtochan(arg[0], ORDWR, false, true);
    if(waserror()){
        cclose(c);
        nexterror();
    }

    ac = mntauth(c, aname);
    /* at this point ac is responsible for keeping c alive */
    poperror(); /* c */
    cclose(c);
    poperror(); /* aname */
    free(aname);

    if(waserror()){
        cclose(ac);
        nexterror();
    }

    fd = newfd(ac);
    if(fd < 0)
        error(Enofd);
    poperror(); /* ac */

    /* always mark it close on exec */
    ac->flag |= CCEXEC;

    return fd;
}

```

<function userwrite 528b>≡ (681)

```

/*
 * called by devcons() for user device

```

```

*
* anyone can become none
*/
long
userwrite(char *a, int n)
{
    if(n!=4 || strcmp(a, "none", 4)!=0)
        error(Eperm);
    kstrdup(&up->user, "none");
    up->basepri = PriNormal;
    return n;
}

⟨function hostdomainwrite 529⟩≡ (681)
long
hostdomainwrite(char *a, int n)
{
    char buf[DOMLEN];

    if(!iseve())
        error(Eperm);
    if(n >= DOMLEN)
        error(Ebadarg);
    memset(buf, 0, DOMLEN);
    strncpy(buf, a, n);
    if(buf[0] == 0)
        error(Ebadarg);
    memmove(hostdomain, buf, DOMLEN);
    return n;
}

```

22.14 Graphics

Chapter 23

Special Fileservers

23.1 Cmdtab

In this chapter, I will present the various special-purpose filesystems that the kernel provides: `/env/` for environment variables, `/proc/` for process inspection and control, `/fd/` for file descriptor access, `/sys/` for system configuration, and `/dev/` for miscellaneous devices. Each is implemented as a device driver following the `Dev` interface described in the `Devices` chapter. But first, the `Cmddtab` infrastructure—Plan 9’s replacement for `ioctl()`. Instead of cryptic numeric `ioctl` codes, device control files accept human-readable text commands like “stop” or “hang”. The `parsecmd()` function tokenizes the command string into a `Cmdbuf`, and `lookupcmd()` matches the first word against a table of `Cmddtab` entries, validating the argument count.

`<struct Cmddtab 530a>≡` (667b)

```
struct Cmddtab
{
    int index; /* used by client to switch on result */
    char *cmd; /* command name */
    int nargs; /* expected #args; 0 ==> variadic */
};
```

`<struct Cmdbuf 530b>≡` (667b)

```
struct Cmdbuf
{
    char *buf;
    char **f;
    int nf;
};
```

`<function parsecmd 530c>≡` (667c)

```
/*
 * parse a command written to a device
 */
Cmdbuf*
parsecmd(char *p, int n)
{
    Cmdbuf *volatile cb;
    int nf;
    char *sp;

    nf = ncmdfield(p, n);

    /* allocate Cmdbuf plus string pointers plus copy of string including \0 */
    sp = smalloc(sizeof(*cb) + nf * sizeof(char*) + n + 1);
    cb = (Cmdbuf*)sp;
    cb->f = (char**>(&cb[1]));
```

```

cb->buf = (char*)&cb->f[nf]);

if(up!=nil && waserror()){
    free(cb);
    nexterror();
}
memmove(cb->buf, p, n);
if(up != nil)
    poperror();

/* dump new line and null terminate */
if(n > 0 && cb->buf[n-1] == '\n')
    n--;
cb->buf[n] = '\0';

cb->nf = tokenize(cb->buf, cb->f, nf-1);
cb->f[cb->nf] = nil;

return cb;
}

```

⟨function cmderror 531a⟩≡ (667c)

```

/*
 * Reconstruct original message, for error diagnostic
 */
void
cmderror(Cmdbuf *cb, char *s)
{
    int i;
    char *p, *e;

    p = up->genbuf;
    e = p+ERRMAX-10;
    p = seprint(p, e, "%s \"", s);
    for(i=0; i<cb->nf; i++){
        if(i > 0)
            p = seprint(p, e, " ");
        p = seprint(p, e, "%q", cb->f[i]);
    }
    strcpy(p, "\"");
    error(up->genbuf);
}

```

⟨function lookupcmd 531b⟩≡ (667c)

```

/*
 * Look up entry in table
 */
Cmdtab*
lookupcmd(Cmdbuf *cb, Cmdtab *ctab, int nctab)
{
    int i;
    Cmdtab *ct;

    if(cb->nf == 0)
        error("empty control message");

    for(ct = ctab, i=0; i<nctab; i++, ct++){
        if(strcmp(ct->cmd, "*") !=0) /* wildcard always matches */
            if(strcmp(ct->cmd, cb->f[0]) != 0)
                continue;
    }
}

```

```

        if(ct->narg != 0 && ct->narg != cb->nf)
            cmderror(cb, Ecmdargs);
        return ct;
    }

    cmderror(cb, "unknown control message");
    return nil;
}

```

```

⟨ncmdfield 532a⟩≡ (667c)
/*
 * Generous estimate of number of fields, including terminal nil pointer
 */
static int
ncmdfield(char *p, int n)
{
    int white, nwhite;
    char *ep;
    int nf;

    if(p == nil)
        return 1;

    nf = 0;
    ep = p+n;
    white = 1; /* first text will start field */
    while(p < ep){
        nwhite = (strchr(" \t\r\n", *p++ & 0xFF) != 0); /* UTF is irrelevant */
        if(white && !nwhite) /* beginning of field */
            nf++;
        white = nwhite;
    }
    return nf+1; /* +1 for nil */
}

```

23.2 /env/

In Plan 9, environment variables are not an in-process data structure as in UNIX—they are files in the `/env/` filesystem. Each variable is a separate file; reading it gives the value, writing it sets it. This means any program in any language can access environment variables through the standard file API, and tools like `cat` and `echo` can inspect and modify them. The variables are grouped per-process in an **Egrp** (environment group), which can be shared or copied across `rfork`.

23.2.1 Egrp

An **Evalue** pairs a name with a byte array value. The value is typically a string but can be arbitrary bytes. The **Egrp** holds a dynamically-sized array of **Evalue** pointers, reference-counted so that processes sharing an environment group (via `RFENVG`) see the same variables. The `envcpy()` function performs a deep copy when a process gets its own environment group.

```

⟨struct Evalue 532b⟩≡ (645c)
struct Evalue
{
    // string
    char *name;
    // option<array<byte>, usually a string but can be something else

```

```

    byte *value;
    // nelem(Evalue.value),
    int len;

    Qid qid;
};

⟨struct Egrp 533a⟩≡ (645c)
struct Egrp
{
    // array<ref_counted<Evalue>>
    Evalue **ent;
    // used entries, <= ment
    int nent;
    // nelem(Egrp.ent), malloc'ed entries
    int ment;

    ulong path; /* qid.path of next Evalue to be allocated */
    ulong vers; /* of Egrp */

    // extra
    Ref;
    RWlock;
};

⟨Proc other fields 533b⟩+≡ (49f) <454d 536d>
// ref_counted<Egrp>
Egrp *egrp; /* Environment group */

⟨function closeegrp 533c⟩≡ (651a)
void
closeegrp(Egrp *eg)
{
    int i;
    Evalue *e;

    if(decref(eg) == 0){
        for(i=0; i<eg->nent; i++){
            e = eg->ent[i];
            free(e->name);
            if(e->value)
                free(e->value);
            free(e);
        }
        free(eg->ent);
        free(eg);
    }
}

⟨function envcpy 533d⟩≡ (637)
void
envcpy(Egrp *to, Egrp *from)
{
    int i;
    Evalue *ne, *e;

    rlock(from);
    to->ment = (from->nent+31)&~31;
    to->ent = smalloc(to->ment*sizeof(to->ent[0]));
    for(i=0; i<from->nent; i++){

```

```

    e = from->ent[i];
    ne = smalloc(sizeof(Evalue));
    ne->name = smalloc(strlen(e->name)+1);
    strcpy(ne->name, e->name);
    if(e->value){
        ne->value = smalloc(e->len);
        memmove(ne->value, e->value, e->len);
        ne->len = e->len;
    }
    ne->qid.path = ++to->path;
    to->ent[i] = ne;
}
to->nent = from->nent;
runlock(from);
}

```

23.2.2 /env/

The `ksetenv()` function lets the kernel set environment variables during initialization. It works by opening a file through `namec()` using the device shorthand `#e` (or `#ec` for configuration variables) and writing the value to it—the kernel uses its own filesystem interface to talk to the environment device, treating itself as just another client.

<function ksetenv 534a>≡ (637)

```

/*
 * to let the kernel set environment variables
 */
void
ksetenv(char *ename, char *eval, bool conf)
{
    Chan *c;
    char buf[2*KNAMELEN];

    snprintf(buf, sizeof(buf), "#e%s/%s", conf?"c":"", ename);
    c = namec(buf, Acreate, OWRITE, 0600);
    devtab[c->type]->write(c, eval, strlen(eval), 0);
    cclose(c);
}

```

<global envdevtab 534b>≡ (637)

```

Dev envdevtab = {
    .dc      =    'e',
    .name    =    "env",

    .reset   =    devreset,
    .init    =    devinit,
    .shutdown = devshutdown,
    .attach  =    envattach,
    .walk    =    envwalk,
    .stat    =    envstat,
    .open    =    envopen,
    .create  =    envcreate,
    .close   =    envclose,
    .read    =    envread,
    .bread   =    devbread,
    .write   =    envwrite,
    .bwrite  =    devbwrite,
    .remove  =    envremove,
    .wstat   =    devwstat,
}

```

```
};
```

23.3 /proc/

The /proc filesystem (device letter **#p**) is the most complex special filesystem in the kernel. Each running process appears as a directory `/proc/\emphpid/` containing files for inspection and control: **ctl** for sending commands, **mem** for reading/writing the address space, **status** for state information, **note** for posting notifications, **regs/fpregs** for register access, **wait** for collecting child exit statuses, **syscall** for tracing output, and **profile** for tick profiling data. The qid encoding packs both the file type (5 bits) and the process slot number (26 bits) into the path, with the pid in the version field for consistency checking.

23.3.1 /proc/x/ctl

23.3.2 /proc/x/mem

23.3.3 /proc/x/status

23.3.4 /proc/x/args

23.3.5 /proc/x/wait

Reading `/proc/\emphpid/wait` is an alternative to the `await()` system call for collecting child exit statuses. The `qwaitr` lock ensures only one reader can block waiting for children at a time—if a second reader tries, it gets an “in use” error.

```
<Proc hierarchy fields 535a>+≡ (49f) <129b
    QLock qwaitr;
```

```
<procread() cases 535b>+≡ (540c) <447b 542a>
    case Qwait:
        if(!canqlock(&p->qwaitr))
            error(Einuse);

        if(waserror()) {
            qunlock(&p->qwaitr);
            nexterror();
        }
```

```
<procread() when Qwait case, sanity check process has children 536a>
```

```
pid = p->pid;
while(p->waitq == nil) {
    unlock(&p->exl);
    sleep(&p->waitr, haswaitq, p);
    if(p->pid != pid)
        error(Eprocdied);
    lock(&p->exl);
}
wq = p->waitq;
p->waitq = wq->next;
p->nwait--;
unlock(&p->exl);

qunlock(&p->qwaitr);
poperror();

n = snprintf(a, n, "%d %lud %lud %lud %q",
```



```

    wq->w.pid,
    ⟨procread() Qwait case, snprintf time field arguments 445h⟩
    wq->w.msg);
free(wq);
return n;

⟨procread() when Qwait case, sanity check process has children 536a⟩≡ (535b)
// same in sysawait
lock(&p->exl);
if(up == p && p->nchild == 0 && p->waitq == 0) {
    unlock(&p->exl);
    error(Enochild);
}

⟨pwait() lock qwaitr 536b⟩≡ (128c)
if(!canqlock(&up->qwaitr))
    error(Einuse); // someone is reading /proc/pid/wait?
if(waserror()) {
    qunlock(&up->qwaitr);
    nexterror();
}

⟨pwait() unlock qwaitr 536c⟩≡ (128c)
qunlock(&up->qwaitr);
poperror();

```

23.3.6 Other /proc/x/xxx

23.3.7 XXX

The `procmode` field controls the default permissions for files in a process's `/proc` directory. When `procmode` is 0, the per-file defaults from `procdir` apply. A process can also set `privatemem` to prevent anyone from reading its memory or posting notes—useful for processes handling sensitive data.

```

⟨Proc other fields 536d⟩+≡ (49f) <533b 536f>
    ulong procmode; /* proc device default file mode */

⟨sysrfork() when RFPROC propagate fields from up to p 536e⟩+≡ (112d) <454e 536h>
    p->procmode = up->procmode;

⟨Proc other fields 536f⟩+≡ (49f) <536d
    bool privatemem; /* proc does not let anyone read mem */

⟨newproc() set fields of p 536g⟩+≡ (108c) <502d 552b>
    p->privatemem = false;

⟨sysrfork() when RFPROC propagate fields from up to p 536h⟩+≡ (112d) <536e
    p->privatemem = up->privatemem;

⟨sysexec() when hold debug lock, reset some fields 536i⟩+≡ (119d) <396c
    up->privatemem = false;

⟨procctlreq() CMprivate case 536j⟩≡ (554b)
    case CMprivate:
        p->privatemem = true;
        break;

```

<devproc.c enum Qxxx 537a>≡

(643)

```
enum
{
    Qdir,
    Qtrace,

    Qargs,
    Qctl,
    Qfd,
    Qfpregs,
    Qkregs,
    Qmem,
    Qnote,
    Qnoteid,
    Qnotepg,
    Qns,
    Qproc,
    Qregs,
    Qsegment,
    Qstatus,
    Qtext,
    Qwait,
    Qprofile,
    Qsyscall,
};
```

<devproc enum CMxxx 537b>≡

(643)

```
enum
{
    CMclose,
    CMclosefiles,
    CMhang,
    CMkill,
    CMnohang,
    CMnoswap,
    CMprivate,
    CMprofile,

    CMpri,
    CMfixedpri,
    CMwired,

    CMstart,
    CMstartstop,
    CMstartsyscall,
    CMstop,
    CMwaitstop,
    CMtrace,

    /* real time */
    CMperiod,
    CMdeadline,
    CMcost,
    CMsporadic,
    CMdeadlinenotes,
    CMadmit,
    CMextra,
    CMexpel,
    CMevent,
};
```

The `procd` table defines the files visible in each `/proc/\emphpid/` directory, and the `proccmd` table below defines the text commands accepted by the `ctl` file. Files with permission 0000 use the process's `procmode` as their default; `status`, `fd`, and `ns` are world-readable (0444) because tools like `ps` need to read them for any process.

```
(global procd 538a)≡ (643)
/*
 * Status, fd, and ns are left fully readable (0444) because of their use in debugging,
 * particularly on shared servers.
 * Arguably, ns and fd shouldn't be readable; if you'd prefer, change them to 0000
 */
Dirtab procd[] =
{
    "args",      {Qargs},    0,          0660,
    "ctl",       {Qctl},     0,          0000,
    "fd",        {Qfd},      0,          0444,
    "fpregs",    {Qfpregs},  sizeof(Arch_FPsave), 0000,
    "kregs",     {Qkregs},   sizeof(Ureg),  0400,
    "mem",       {Qmem},     0,          0000,
    "note",      {Qnote},    0,          0000,
    "noteid",    {Qnoteid},  0,          0664,
    "notepg",    {Qnotepg},  0,          0000,
    "ns",        {Qns},      0,          0444,
    "proc",      {Qproc},    0,          0400,
    "regs",      {Qregs},    sizeof(Ureg),  0000,
    "segment",   {Qsegment}, 0,          0444,
    "status",    {Qstatus},  STATSIZE,  0444,
    "text",      {Qtext},    0,          0000,
    "wait",      {Qwait},    0,          0400,
    "profile",   {Qprofile}, 0,          0400,
    "syscall",   {Qsyscall}, 0,          0400,
};
```

```
(global proccmd 538b)≡ (643)
static
Cmdtab proccmd[] = {
    CMclose,      "close",      2,
    CMclosefiles, "closefiles", 1,
    CMfixedpri,   "fixedpri",   2,
    CMhang,       "hang",       1,
    CMnohang,     "nohang",     1,
    CMnoswap,     "noswap",     1,
    CMkill,       "kill",       1,
    CMpri,        "pri",        2,
    CMprivate,    "private",    1,
    CMprofile,    "profile",    1,
    CMstart,      "start",      1,
    CMstartstop,  "startstop",   1,
    CMstartsyscall, "startsyscall", 1,
    CMstop,       "stop",       1,
    CMwaitstop,   "waitstop",   1,
    CMwired,      "wired",      2,
    CMtrace,      "trace",      0,
    CMperiod,     "period",     2,
    CMdeadline,   "deadline",   2,
    CMcost,       "cost",       2,
    CMsporadic,   "sporadic",   1,
    CMdeadlinenotes, "deadlinenotes", 1,
    CMadmit,      "admit",      1,
    CMextra,      "extra",      1,
```

```

    CMexpel,      "expel",      1,
    CMevent,      "event",      1,
};

```

The qid encoding for `/proc` is cleverly packed: the low 5 bits of the path identify which file within the process directory (e.g., `Qctl`, `Qmem`, `Qstatus`), while the upper bits encode the process slot number. The version field carries the pid, so stale channels to dead-and-reallocated process slots can be detected. The `QID()`, `SLOT()`, and `PID()` macros extract these components.

<devproc QXXX macros 539a>≡ (643)

```

/*
 * Qids are, in path:
 *   5 bits of file type (qids above)
 * 26 bits of process slot number + 1
 *   in vers,
 * 32 bits of pid, for consistency checking
 * If notepg, c->pgrp.pid is pgrp slot, .vers is noteid.
 */
#define QSHIFT 5 /* location in qid of proc slot # */

#define QID(q)      (((ulong)(q).path) & ((1<<QSHIFT)-1)) >> 0)
#define SLOT(q)     (((ulong)(q).path) & ~(1UL<<31)) >> QSHIFT) - 1)
#define PID(q)      ((q).vers)
#define NOTEID(q)   ((q).vers)

```

<function proctab 539b>≡ (675c)

```

Proc*
proc_proctab(int i)
{
    return &proccalloc.arena[i];
}

```

<Chan union other fields 539c>+≡ (51e) <404a 540a>

```

    Qid pgrp; /* for #p/notepg */

```

<method procinit 539d>≡ (643)

```

static void
proc_init(void)
{
    if(conf.nproc >= (1<<(31-QSHIFT))-1)
        print("warning: too many procs for devproc\n");

    <proc_init() other init 446d>
}

```

<global procdevtab 539e>≡ (643)

```

Dev procdevtab = {
    .dc      = 'p',
    .name    = "proc",

    .reset   = devreset,
    .init    = proc_init,
    .shutdown = devshutdown,
    .attach  = procattach,
    .walk    = procwalk,
    .stat    = procstat,
    .open    = procopen,
    .create  = devcreate,
    .close   = procclose,
    .read    = procread,

```

```

        .bread    =    devbread,
        .write    =    procwrite,
        .bwrite   =    devbwrite,
        .remove   =    devremove,
        .wstat    =    procwstat,
};

```

⟨Chan union other fields 540a⟩+≡ (51e) <539c

```

    ulong mid;    /* for ns in devproc */

```

⟨procread() locals 540b⟩+≡ (540c) <307b

```

/* NSEG*32 was too small for worst cases */
char *a, *sps, *srv, statbuf[NSEG*64];
int i, j, m, navail, ne, pid, rsize;
long l;
byte *rptr;
ulong offset;
Confmem *cm;
Proc *p;
Segment *s;
Ureg kur;
Waitq *wq;

```

The `procread()` function is a large switch on the file type extracted from the channel's `qid`. Most cases follow a similar pattern: validate that the target process still exists (comparing `p->pid` against `PID(c->qid)`), lock `p->debug` if needed, format the requested data into a buffer, and copy it to the caller. The `Qstatus` case assembles a fixed-format string with the process name, user, state, time counters, memory usage, and priority—this is what `/bin/ps` reads.

⟨method procread 540c⟩≡ (643)

```

static long
procread(Chan *c, void *va, long n, vlong off)
{
    ⟨procread() locals 101e⟩

    a = va;
    offset = off;

    if(c->qid.type & QTDIR)
        return devdirread(c, a, n, 0, 0, procgen);

    ⟨procread() Qtrace if 442d⟩

    p = proctab(SLOT(c->qid));
    if(p->pid != PID(c->qid))
        error(Eprocdied);

    switch(QID(c->qid)){

    ⟨procread() cases 101f⟩

    case Qargs:
        qlock(&p->debug);
        j = procargs(p, up->genbuf, sizeof up->genbuf);
        qunlock(&p->debug);
        if(offset >= j)
            return 0;
        if(offset+n > j)
            n = j-offset;
        memmove(a, &up->genbuf[offset], n);

```

```

return n;

case Qnote:
    qlock(&p->debug);
    if(waserror()){
        qunlock(&p->debug);
        nexterror();
    }
    if(p->pid != PID(c->qid))
        error(Eprocdied);
    if(n < 1) /* must accept at least the '\0' */
        error(Etoosmall);
    if(p->nnote == 0)
        n = 0;
    else {
        m = strlen(p->note[0].msg) + 1;
        if(m > n)
            m = n;
        memmove(va, p->note[0].msg, m);
        ((char*)va)[m-1] = '\0';
        p->nnote--;
        memmove(p->note, p->note+1, p->nnote*sizeof(Note));
        n = m;
    }
    if(p->nnote == 0)
        p->notepending = false;
    poperror();
    qunlock(&p->debug);
    return n;

case Qproc:
    if(offset >= sizeof(Proc))
        return 0;
    if(offset+n > sizeof(Proc))
        n = sizeof(Proc) - offset;
    memmove(a, ((char*)p)+offset, n);
    return n;

case Qregs:
    rptra = (byte*)p->dbgreg;
    rsize = sizeof(Ureg);
    goto regread;

⟨procread() Qfpregs case 467b⟩

regread:
    if(rptra == nil)
        error(Enoreg);
    if(offset >= rsize)
        return 0;
    if(offset+n > rsize)
        n = rsize - offset;
    memmove(a, rptra+offset, n);
    return n;

// coupling with ps.c

```

```

/*
 * 0  text
 * 1  user
 * 2  state
 * 3  cputime[5]
 * 8  memory
 * 9  basepri
 * 10 pri
 */
case Qstatus:
    if(offset >= STATSIZE)
        return 0;
    if(offset+n > STATSIZE)
        n = STATSIZE - offset;

    memset(statbuf, ' ', sizeof statbuf);
    readstr(0, statbuf+0*KNAMELEN, KNAMELEN-1, p->text);
    readstr(0, statbuf+1*KNAMELEN, KNAMELEN-1, p->user);
    sps = p->psstate;
    if(sps == nil)
        sps = statename[p->state];
    readstr(0, statbuf+2*KNAMELEN, 11, sps);

    j = 2*KNAMELEN + 12;
    <procread() Qstatus case, time part 445b>

    /* ignore stack, which is mostly non-existent */
    l = 0;
    for(i=1; i<NSEG; i++){
        s = p->seg[i];
        if(s)
            l += s->top - s->base;
    }
    readnum(0, statbuf+j+NUMSIZE*5, NUMSIZE, l>>10, NUMSIZE);
    readnum(0, statbuf+j+NUMSIZE*6, NUMSIZE, p->basepri, NUMSIZE);
    readnum(0, statbuf+j+NUMSIZE*7, NUMSIZE, p->priority, NUMSIZE);
    memmove(a, statbuf+offset, n);
    return n;

case Qnoteid:
    return readnum(offset, va, n, p->noteid, NUMSIZE);
case Qfd:
    return procfds(p, va, n, offset);
}
error(Egreg);
panic("procread: should not reach this point");
return -1; // unreachable
}

<procread() cases 542a>+≡ (540c) <535b

case Qkregs:
    memset(&kur, 0, sizeof(Ureg));
    arch_setkernur(&kur, p);
    rptr = (byte*)&kur;
    rsize = sizeof(Ureg);
    goto regread;

<function arch_setkernur(arm) 542b>≡ (717a)

```

```

/* Give enough context in the ureg to produce a kernel stack for
 * a sleeping process
 */
void
arch_setkernur(Ureg* ureg, Proc* p)
{
    ureg->pc = p->sched.pc;
    ureg->sp = p->sched.sp+4;
    ureg->r14 = PTR2UINT(sched);
}

```

The `procwrite()` method handles writes to `/proc/\emphpid/ctl` (dispatched to `procctlreq()`), `/proc/\emphpid/notify` (posting notifications to another process), `/proc/\emphpid/noteid` (changing the note group for signal broadcasting), `/proc/\emphpid/regs` (modifying registers of a stopped process), and `/proc/\emphpid/mem` (writing into the process's address space). The `Qnotepg` case is handled specially—it uses a remembered note group ID stored in the channel, not the target process's current group.

<method procwrite 543>≡ (643)

```

static long
procwrite(Chan *c, void *va, long n, vlong off)
{
    int id, m;
    Proc *p, *t, *et;
    char *a, *arg, buf[ERRMAX];
    ulong offset = off;

    a = va;
    if(c->qid.type & QTDIR)
        error(Eisdir);

    p = proctab(SLOT(c->qid));

    /* Use the remembered noteid in the channel rather
     * than the process pgrpid
     */
    if(QID(c->qid) == Qnotepg) {
        pgrpnote(NOTEID(c->pgrpid), va, n, NUser);
        return n;
    }

    qlock(&p->debug);
    if(waserror()){
        qunlock(&p->debug);
        nexterror();
    }
    if(p->pid != PID(c->qid))
        error(Eprocdied);

    switch(QID(c->qid)){

    <procwrite() cases 430c>

    case Qctl:
        procctlreq(p, va, n);
        break;

    case Qargs:
        if(n == 0)
            error(Eshort);
        if(n >= ERRMAX)

```



```

        error(Etoobig);
    arg = malloc(n+1);
    if(arg == nil)
        error(Enomem);
    memmove(arg, va, n);
    m = n;
    if(arg[m-1] != 0)
        arg[m++] = 0;
    free(p->args);
    p->nargs = m;
    p->args = arg;
    p->setargs = true;
    break;

case Qregs:
    if(offset >= sizeof(Ureg))
        n = 0;
    else if(offset+n > sizeof(Ureg))
        n = sizeof(Ureg) - offset;
    if(p->dbgreg == nil)
        error(Enoreg);
    arch_setregisters(p->dbgreg, (char*)(p->dbgreg)+offset, va, n);
    break;

⟨procwrite Qfpregs case 467c⟩

case Qnote:
    if(p->kp)
        error(Eperm);
    if(n >= ERRMAX-1)
        error(Etoobig);
    memmove(buf, va, n);
    buf[n] = 0;
    if(!postnote(p, 0, buf, NUser))
        error("note not posted");
    break;
case Qnoteid:
    id = atoi(a);
    if(id == p->pid) {
        p->noteid = id;
        break;
    }
    t = proctab(0);
    for(et = t+conf.nproc; t < et; t++) {
        if(t->state == Dead)
            continue;
        if(id == t->noteid) {
            if(strcmp(p->user, t->user) != 0)
                error(Eperm);
            p->noteid = id;
            break;
        }
    }
    if(p->noteid != id)
        error(Ebadarg);
    break;

default:
    pprint("unknown qid in procwrite\n");

```

```

        error(Egreg);
    }
    poperror();
    qunlock(&p->debug);
    return n;
}

```

The `pgrpnote()` function broadcasts a note (notification) to all processes sharing a note group—Plan 9’s equivalent of sending a signal to a process group. Writing to `/proc/\emphpid/notepg` triggers this, delivering the message to every non-kernel process with the matching `noteid`.

```

⟨function pgrpnote 545a⟩≡ (673b)
void
pgrpnote(ulong noteid, char *a, long n, int flag)
{
    Proc *p, *ep;
    char buf[ERRMAX];

    if(n >= ERRMAX-1)
        error(Etoobig);

    memmove(buf, a, n);
    buf[n] = 0;
    p = proctab(0);
    ep = p+conf.nproc;
    for(; p < ep; p++) {
        if(p->state == Dead)
            continue;
        if(up != p && p->noteid == noteid && p->kp == 0) {
            qlock(&p->debug);
            if(p->pid == 0 || p->noteid != noteid){
                qunlock(&p->debug);
                continue;
            }
            if(!waserror()) {
                postnote(p, 0, buf, flag);
                poperror();
            }
            qunlock(&p->debug);
        }
    }
}

```

The `procopen()` method enforces per-file access policies: `Qtext` and `Qprofile` are read-only, `Qnote` and `Qmem` are forbidden if the process has set `privatemem`, and the “none” user is blocked from most files via `nonone()`. The `Qtext` case is special—it returns a channel to the executable’s binary file, not a synthetic proc file, allowing debuggers to read symbol tables.

```

⟨method procopen 545b⟩≡ (643)
static Chan*
procopen(Chan *c, int omode)
{
    Proc *p;
    Pgrp *pg;
    Chan *tc;
    int pid;

    if(c->qid.type & QTDIR)
        return devopen(c, omode, 0, 0, procgen);

    ⟨procopen() Qtrace if 442b⟩

```

```

p = proctab(SLOT(c->qid));
qlock(&p->debug);
if(waserror()){
    qunlock(&p->debug);
    nexterror();
}
pid = PID(c->qid);
if(p->pid != pid)
    error(Eprocdied);

omode = openmode(omode);

switch(QID(c->qid)){

<procopen() cases 308a>
case Qtext:
    if(omode != OREAD)
        error(Eperm);
    tc = proctext(c, p);
    tc->offset = 0;
    qunlock(&p->debug);
    poperror();
    cclose(c);
    return tc;

case Qproc:
case Qkregs:
case Qsegment:
case Qprofile:
case Qfd:
    if(omode != OREAD)
        error(Eperm);
    break;

case Qnote:
    if(p->privatemem)
        error(Eperm);
    break;

case Qmem:
case Qctl:
    if(p->privatemem)
        error(Eperm);
    nonone(p);
    break;

case Qargs:
case Qnoteid:
case Qstatus:
case Qwait:
case Qregs:
case Qfpregs:
case Qsyscall:
    nonone(p);
    break;

case Qnotepg:
    nonone(p);

```

```

    pg = p->pgrp;
    if(pg == nil)
        error(Eprocdied);
    if(omode!=OWRITE || pg->pgrpid == 1)
        error(Eperm);
    c->pgrpid.path = pg->pgrpid+1;
    c->pgrpid.vers = p->noteid;
    break;

default:
    pprint("procopen %#lux\n", QID(c->qid));
    error(Egreg);
}

/* Affix pid to qid */
if(p->state != Dead)
    c->qid.vers = p->pid;

/* make sure the process slot didn't get reallocated while we were playing */
arch_coherence();
if(p->pid != pid)
    error(Eprocdied);

tc = devopen(c, omode, 0, 0, procgen);
qunlock(&p->debug);
poperror();

return tc;
}

```

The `procwstat()` method allows changing the owner and default permissions of proc files. This is how `chmod +w /proc/96/mem` works to allow a debugger to write to a process's memory. Only the process owner or the host owner (`eve`) can change these settings.

```

⟨method procwstat 547⟩≡ (643)
static int
procwstat(Chan *c, byte *db, int n)
{
    Proc *p;
    DirEntry *d;

    if(c->qid.type&QTDIR)
        error(Eperm);

    if(QID(c->qid) == Qtrace)
        return devwstat(c, db, n);

    p = proctab(SLOT(c->qid));
    nonone(p);
    d = nil;
    if(waserror()){
        free(d);
        qunlock(&p->debug);
        nexterror();
    }
    qlock(&p->debug);

    if(p->pid != PID(c->qid))
        error(Eprocdied);
}

```

```

if(strcmp(up->user, p->user) != 0 && strcmp(up->user, eve) != 0)
    error(Eperm);

d = smalloc(sizeof(DirEntry)+n);
n = convM2D(db, n, &d[0], (char*)&d[1]);
if(n == 0)
    error(Eshortstat);
if(!emptystr(d->uid) && strcmp(d->uid, p->user) != 0){
    if(strcmp(up->user, eve) != 0)
        error(Eperm);
    else
        kstrdup(&p->user, d->uid);
}
/* p->procmode determines default mode for files in /proc */
if(d->mode != ~OUL)
    p->procmode = d->mode&0777;

poperror();
free(d);
qunlock(&p->debug);
return n;
}

```

```

⟨function procoffset 548a⟩≡ (643)
static long
procoffset(long offset, char *va, int *np)
{
    if(offset > 0) {
        offset -= *np;
        if(offset < 0) {
            memmove(va, va+*np+offset, -offset);
            *np = -offset;
        }
        else
            *np = 0;
    }
    return offset;
}

```

```

⟨function procqidwidth 548b⟩≡ (643)
static int
procqidwidth(Chan *c)
{
    char buf[32];

    return snprintf(buf, sizeof buf, "%1ud", c->qid.vers);
}

```

```

⟨function procfidprint 548c⟩≡ (643)
int
procfidprint(Chan *c, int fd, int w, char *s, int ns)
{
    int n;

    if(w == 0)
        w = procqidwidth(c);
    n = snprintf(s, ns, "%3d %.2s %C %4ld (%.16llx %*1ud %.2ux) %5ld %8lld %s\n",
        fd,
        &"r w rw"[(c->mode&3)<<1],
        devtab[c->type]->dc, c->dev,

```

```

        c->qid.path, w, c->qid.vers, c->qid.type,
        c->iounit, c->offset, c->path->s);
    return n;
}

```

The `procgen()` function generates directory entries for the `/proc` filesystem. At the top level (`/proc/`), it generates one entry per process (named by pid). Within each process directory, it generates entries from the `procdirent` table. The permissions are computed from both the table defaults and the process's `procmode`, so a process can restrict access to its own proc files.

```

⟨function procgen 549⟩≡ (643)
static int
procgen(Chan *c, char *name, Dirtab *tab, int, int s, DirEntry *dp)
{
    Qid qid;
    Proc *p;
    char *ename;
    Segment *q;
    ulong pid, path, perm, len;

    if(s == DEVDOTDOT){
        mkqid(&qid, Qdir, 0, QTDIR);
        devdir(c, qid, "#p", 0, eve, 0555, dp);
        return 1;
    }

    if(c->qid.path == Qdir){
        if(s == 0){
            strcpy(up->genbuf, "trace");
            mkqid(&qid, Qtrace, -1, QTFILE);
            devdir(c, qid, up->genbuf, 0, eve, 0444, dp);
            return 1;
        }

        if(name != nil){
            /* ignore s and use name to find pid */
            pid = strtol(name, &ename, 10);
            if(pid==0 || ename[0]!='\0')
                return -1;
            s = procindex(pid);
            if(s < 0)
                return -1;
        }
        else if(--s >= conf.nproc)
            return -1;

        p = proctab(s);
        pid = p->pid;
        if(pid == 0)
            return 0;
        snprintf(up->genbuf, sizeof up->genbuf, "%lud", pid);
        /*
         * String comparison is done in devwalk so name must match its formatted pid
         */
        if(name != nil && strcmp(name, up->genbuf) != 0)
            return -1;
        mkqid(&qid, (s+1)<<QSHIFT, pid, QTDIR);
        devdir(c, qid, up->genbuf, 0, p->user, DMDIR|0555, dp);
        return 1;
    }
}

```

```

if(c->qid.path == Qtrace){
    strcpy(up->genbuf, "trace");
    mkqid(&qid, Qtrace, -1, QTFILE);
    devdir(c, qid, up->genbuf, 0, eve, 0444, dp);
    return 1;
}
if(s >= nelem(procdir))
    return -1;
if(tab)
    panic("procgen");

tab = &procdir[s];
path = c->qid.path&~(((1<<QSHIFT)-1)); /* slot component */

/* p->procmode determines default mode for files in /proc */
p = proctab(SLOT(c->qid));
perm = tab->perm;
if(perm == 0)
    perm = p->procmode;
else /* just copy read bits */
    perm |= p->procmode & 0444;

len = tab->length;
switch(QID(c->qid)) {
case Qwait:
    len = p->nwait; /* incorrect size, but >0 means there's something to read */
    break;

⟨procgen() Qprofile case 447c⟩
}

mkqid(&qid, path|tab->qid.path, c->qid.vers, QTFILE);
devdir(c, qid, tab->name, len, p->user, perm, dp);
return 1;
}

```

The `nonone()` function implements a security policy: the “none” user (the Plan 9 equivalent of “nobody”) cannot inspect or control other processes. This prevents a compromised network server running as none from using `/proc` to attack other processes on the system.

⟨function `nonone` 550a⟩≡ (643)

```

/*
 * none can't read or write state on other
 * processes. This is to contain access of
 * servers running as none should they be
 * subverted by, for example, a stack attack.
 */
static void
nonone(Proc *p)
{
    if(p == up)
        return;
    if(strcmp(up->user, "none") != 0)
        return;
    if(iseve())
        return;
    error(Eperm);
}

```

⟨function `procfds` 550b⟩≡ (643)

```

static int
procfds(Proc *p, char *va, int count, long offset)
{
    Fgrp *f;
    Chan *c;
    char buf[256];
    int n, i, w, ww;
    char *a;

    /* print to buf to avoid holding fgrp lock while writing to user space */
    if(count > sizeof buf)
        count = sizeof buf;
    a = buf;

    qlock(&p->debug);
    f = p->fgrp;
    if(f == nil){
        qunlock(&p->debug);
        return 0;
    }
    lock(f);
    if(waserror()){
        unlock(f);
        qunlock(&p->debug);
        nexterror();
    }

    n = readstr(0, a, count, p->dot->path->s);
    n += snprintf(a+n, count-n, "\n");
    offset = procoffset(offset, a, &n);
    /* compute width of qid.path */
    w = 0;
    for(i = 0; i <= f->maxfd; i++) {
        c = f->fd[i];
        if(c == nil)
            continue;
        ww = procqidwidth(c);
        if(ww > w)
            w = ww;
    }
    for(i = 0; i <= f->maxfd; i++) {
        c = f->fd[i];
        if(c == nil)
            continue;
        n += procfidprint(c, i, w, a+n, count-n);
        offset = procoffset(offset, a, &n);
    }
    unlock(f);
    qunlock(&p->debug);
    poperror();

    /* copy result to user space, now that locks are released */
    memmove(va, buf, n);

    return n;
}

```

<method procclose 551>≡
 static void
 procclose(Chan* c)

(643)


```

{
    <procclose() Qtrace if 442c>
    <procclose() hooks 308b>
}

<Proc debugger fields 552a>+≡ (49f) <440a
    bool setargs;

<newproc() set fields of p 552b>+≡ (108c) <536g 596c>
    p->setargs = false;

<function procargs 552c>≡ (643)
    static int
    procargs(Proc *p, char *buf, int nbuf)
    {
        int j, k, m;
        char *a;
        int n;

        a = p->args;
        if(p->setargs){
            snprintf(buf, nbuf, "%s [%s]", p->text, p->args);
            return strlen(buf);
        }
        n = p->nargs;
        for(j = 0; j < nbuf - 1; j += m){
            if(n <= 0)
                break;
            if(j != 0)
                buf[j++] = ' ';
            m = snprintf(buf+j, nbuf-j, "%q", a);
            k = strlen(a) + 1;
            a += k;
            n -= k;
        }
        return j;
    }

<function proctext 552d>≡ (643)
    Chan*
    proctext(Chan *c, Proc *p)
    {
        Chan *tc;
        KImage *i;
        Segment *s;

        s = p->seg[TSEG];
        if(s == 0)
            error(Enonexist);
        if(p->state==Dead)
            error(Eprocdied);

        lock(s);
        i = s->image;
        if(i == 0) {
            unlock(s);
            error(Eprocdied);
        }
        unlock(s);
    }

```

```

lock(i);
if(waserror()) {
    unlock(i);
    nexterror();
}

tc = i->c;
if(tc == 0)
    error(Eprocdied);

if(incrcf(tc) == 1 || (tc->flag&COPEN) == 0 || tc->mode!=OREAD) {
    cclose(tc);
    error(Eprocdied);
}

if(p->pid != PID(c->qid)){
    cclose(tc);
    error(Eprocdied);
}

unlock(i);
poperror();

return tc;
}

```

<function procctlcloseone 553a>≡ (643)

```

static void
procctlcloseone(Proc *p, Fgrp *f, int fd)
{
    Chan *c;

    c = f->fd[fd];
    if(c == nil)
        return;
    f->fd[fd] = nil;
    unlock(f);
    qunlock(&p->debug);
    cclose(c);
    qlock(&p->debug);
    lock(f);
}

```

<function procctlclosefiles 553b>≡ (643)

```

void
procctlclosefiles(Proc *p, bool all, int fd)
{
    int i;
    Fgrp *f;

    f = p->fgrp;
    if(f == nil)
        error(Eprocdied);

    lock(f);
    f->ref++;
    if(all)
        for(i = 0; i < f->maxfd; i++)
            procctlcloseone(p, f, i);
    else

```

```

        procctlcloseone(p, f, fd);
unlock(f);
closefgrp(f);
}

```

```

⟨function parsetime 554a⟩≡ (643)
static char *
parsetime(vlong *rt, char *s)
{
    uulong ticks;
    ulong l;
    char *e, *p;
    static int p10[] = {1000000000, 100000000, 10000000, 100000, 10000, 1000, 100, 10, 1};

    if (s == nil)
        return "missing value";
    ticks=strtoul(s, &e, 10);
    if (*e == '.') {
        p = e+1;
        l = strtoul(p, &e, 10);
        if(e-p > nelem(p10))
            return "too many digits after decimal point";
        if(e-p == 0)
            return "ill-formed number";
        l *= p10[e-p-1];
    } else
        l = 0;
    if (*e == '\\0' || strcmp(e, "s") == 0) {
        ticks = 1000000000 * ticks + l;
    } else if (strcmp(e, "ms") == 0) {
        ticks = 1000000 * ticks + l/1000;
    } else if (strcmp(e, "µs") == 0 || strcmp(e, "us") == 0) {
        ticks = 1000 * ticks + l/1000000;
    } else if (strcmp(e, "ns") != 0)
        return "unrecognized unit";
    *rt = ticks;
    return nil;
}

```

The `procctlreq()` function is the central dispatch for all `/proc/\\emphpid/ctl` commands. It parses the text command using `parsecmd()/lookupcmd()` against the `proccmd` table, then switches on the command index. The function assumes the caller holds `p->debug`, which serializes all control operations on a given process.

```

⟨function procctlreq 554b⟩≡ (643)
// assumes p->debug is held
void
procctlreq(Proc *p, char *va, int n)
{
    Segment *s;
    int npc, pri;
    Cmdbuf *cb;
    Cmdtab *ct;
    vlong time;
    char *e;
    void (*pt)(Proc*, int, vlong);

    if(p->kp) /* no ctl requests to kprocs */
        error(Eperm);

    cb = parsecmd(va, n);
}

```

```

if(waserror()){
    free(cb);
    nexterror();
}

ct = lookupcmd(cb, proccmd, nelem(proccmd));

switch(ct->index){

    <procctlreq() CMpri case 191e>
    <procctlreq() CMfixedpri case 191f>
    <procctlreq() CMwired case 207g>
    <procctlreq() CMnoswap case 186d>

    <procctlreq() CMprivate case 536j>

    <procctlreq() CMhang case 429b>
    <procctlreq() CMnohang case 429c>

    <procctlreq() CMstop case 427c>
    <procctlreq() CMstart case 427d>
    <procctlreq() CMwaitstop case 429f>

    <procctlreq() CMstartsyscall case 432b>
    <procctlreq() CMstartstop case 443c>

    <procctlreq() CMtrace case 440b>
    <procctlreq() CMevent case 441e>

    <procctlreq() CMprofile case 447a>

    <procctlreq() CMkill case 137a>

    case CMclose:
        procctlclosefiles(p, false, atoi(cb->f[1]));
        break;
    case CMclosefiles:
        procctlclosefiles(p, true, 0);
        break;

    <procctlreq() optional real-time commands 503b>

}

poperror();
free(cb);
}

```

23.4 /fd/

The `/fd/` device (also known as `devdup`) exposes the current process's file descriptors as files. Opening `/fd/0` gives you a new reference to file descriptor 0. This is used during boot to set up standard I/O: the boot script opens the console device and then uses `/fd/0`, `/fd/1`, `/fd/2` to wire up `stdin`, `stdout`, and `stderr`. The `dup()` system call below provides programmatic access to the same functionality.

23.4.1 sysdup()

`<sysstab special file syscalls 556a>≡ (41f)`
`[DUP] sysdup,`

The `dup()` system call duplicates a file descriptor. If `newfd` is -1, a new descriptor is allocated. Otherwise, the specified slot is used, and any existing channel there is closed *after* the dup—this ordering matters for idioms like `date > #d/1` where `stdout` is being replaced.

`<syscall dup 556b>≡ (654)`

```
// int dup(int oldfd, int newfd);
long
sysdup(ulong* arg)
{
    int fd;
    Chan *c, *oc;
    Fgrp *f = up->fgrp;

    /*
     * Close after dup'ing, so date > #d/1 works
     */
    c = fdtochan(arg[0], -1, false, true);
    fd = arg[1];
    if(fd != -1){
        lock(f);
        if(fd<0 || growfd(f, fd)<0) {
            unlockfgrp(f);
            cclose(c);
            error(Ebadfd);
        }
        if(fd > f->maxfd)
            f->maxfd = fd;

        oc = f->fd[fd];
        f->fd[fd] = c;
        unlockfgrp(f);
        if(oc)
            cclose(oc);
    }else{
        if(waserror()) {
            cclose(c);
            nexterror();
        }
        fd = newfd(c);
        if(fd < 0)
            error(Enofd);
        poperror();
    }

    return fd;
}
```

23.4.2 /fd/

23.5 /sys/

The `/sys/` device (device letter `#k`) provides system-wide configuration files that are independent of any particular process: the OS version string, the system name, the host owner, the list of registered device drivers, and

a reboot/halt interface. Unlike `/dev/` files which may return per-process information (e.g., `/dev/pid`), `/sys/` files represent global system state. The distinction is: if the value depends on up, it goes in `/dev`; if it is a machine-wide setting, it goes in `/sys`.

`<devsys.c enum Qxxx 557a>≡` (636a)

```
enum{
    Qdir,

    <devsys.c enum Qxxx cases 558d>
};
```

`<global sysdir 557b>≡` (636a)

```
static Dirtab sysdir[]={
    ".",    {Qdir, 0, QTDIR},    0,    DMDIR|0555,

    <sysdir fields 558e>
};
```

`<method sysinit 557c>≡` (636a)

```
static void
sysinit(void)
{
}
```

`<method sysattach 557d>≡` (636a)

```
static Chan*
sysattach(char *spec)
{
    return devattach('k', spec);
}
```

`<method sysread 557e>≡` (636a)

```
static long
sysread(Chan *c, void *buf, long n, vlong off)
{
    vlong offset = off;
    char tmp[256];
    <sysread() locals 557f>

    if(n <= 0)
        return n;

    switch((ulong)c->qid.path){
    case Qdir:
        return devdirread(c, buf, n, sysdir, nelem(sysdir), devgen);

    <sysread() cases 559a>

    default:
        print("sysread %#llx\n", c->qid.path);
        error(Egreg);
    }
    return -1;    /* never reached */
}
```

`<sysread() locals 557f>≡` (557e)

```
char *b, *bp;
int i, k;
Cpu *mp;
int id;
```

⟨method syswrite 558a⟩≡ (636a)

```
static long
syswrite(Chan *c, void *va, long n, vlong off)
{
    ulong offset;
    char *a;
    ⟨syswrite() locals 558b⟩

    offset = off;
    a = va;

    switch((ulong)c->qid.path){

    ⟨syswrite() cases 559i⟩

    default:
        print("syswrite: %#llx\n", c->qid.path);
        error(Egreg);
    }
    return n;
}
```

⟨syswrite() locals 558b⟩≡ (558a) 561b▷

```
char buf[256];
Cpu *mp;
int id;
```

⟨global sysdevtab 558c⟩≡ (636a)

```
Dev sysdevtab = {
    .dc      =    'k',
    .name    =    "sys",

    .reset   =    devreset,
    .init    =    sysinit,
    .shutdown =    devshutdown,
    .attach  =    sysattach,
    .walk    =    syswalk,
    .stat    =    sysstat,
    .open    =    sysopen,
    .create  =    devcreate,
    .close   =    sysclose,
    .read    =    sysread,
    .bread   =    devbread,
    .write   =    syswrite,
    .bwrite  =    devbwrite,
    .remove  =    devremove,
    .wstat   =    devwstat,
};
```

23.5.1 /sys/osversion

⟨devsys.c enum Qxxx cases 558d⟩≡ (557a) 559c▷

```
Qosversion,
```

⟨sysdir fields 558e⟩≡ (557b) 559d▷

```
"osversion",    {Qosversion},    0,    0444,
```

```

<sysread() cases 559a>≡ (557e) 559e▷
    case Qosversion:
        snprintf(tmp, sizeof tmp, "pad's version");
        n = readstr((ulong)offset, buf, n, tmp);
        return n;

```

23.5.2 /sys/config

```

<devsys.c decls and globals 559b>≡ (636a) 561a▷
    extern byte configfile[]; // in $CONF.c

<devsys.c enum Qxxx cases 559c>+≡ (557a) <558d 559f▷
    Qconfig,

<sysdir fields 559d>+≡ (557b) <558e 559g▷
    "config", {Qconfig}, 0, 0444,

<sysread() cases 559e>+≡ (557e) <559a 559h▷
    case Qconfig:
        return readstr((ulong)offset, buf, n, (char*) configfile);

```

23.5.3 /ys/hostowner, /sys/hostdomain

```

<devsys.c enum Qxxx cases 559f>+≡ (557a) <559c 559k▷
    Qhostowner,
    Qhostdomain,

<sysdir fields 559g>+≡ (557b) <559d 559l▷
    "hostowner", {Qhostowner}, 0, 0664,
    "hostdomain", {Qhostdomain}, DOMLEN, 0664,

<sysread() cases 559h>+≡ (557e) <559e 560a▷
    case Qhostowner:
        return readstr((ulong)offset, buf, n, eve);

    case Qhostdomain:
        return readstr((ulong)offset, buf, n, hostdomain);

<syswrite() cases 559i>≡ (558a) 560b▷
    case Qhostowner:
        return hostownerwrite(a, n);

    case Qhostdomain:
        return hostdomainwrite(a, n);

```

23.5.4 /sys/sysname

```

<global sysname 559j>≡ (636a)
    char *sysname;

<devsys.c enum Qxxx cases 559k>+≡ (557a) <559f 560c▷
    Qsysname,

<sysdir fields 559l>+≡ (557b) <559g 560d▷
    "sysname", {Qsysname}, 0, 0664,

```



```

<sysread() cases 560a>+≡ (557e) <559h 560e>
    case Qsysname:
        if(sysname == nil)
            return 0;
        return readstr((ulong)offset, buf, n, sysname);

```

```

<syswrite() cases 560b>+≡ (558a) <559i 561c>
    case Qsysname:
        if(offset != 0)
            error(Ebadarg);
        if(n <= 0 || n >= sizeof buf)
            error(Ebadarg);
        strncpy(buf, a, n);
        buf[n] = 0;
        if(buf[n-1] == '\n')
            buf[n-1] = 0;
        kstrdup(&sysname, buf);
        break;

```

23.5.5 /sys/drivers

```

<devsys.c enum Qxxx cases 560c>+≡ (557a) <559k 560f>
    Qdrivers,

```

```

<sysdir fields 560d>+≡ (557b) <559l 560g>
    "drivers", {Qdrivers}, 0,      0444,

```

```

<sysread() cases 560e>+≡ (557e) <560a 589f>
    case Qdrivers:
        b = malloc(READSTR);
        if(b == nil)
            error(Enomem);
        k = 0;
        for(i = 0; devtab[i] != nil; i++)
            k += snprintf(b+k, READSTR-k, "%C %s\n",
                devtab[i]->dc, devtab[i]->name);
        if(waserror()){
            free(b);
            nexterror();
        }
        n = readstr((ulong)offset, buf, n, b);
        free(b);
        poperror();
        return n;

```

23.5.6 /sys/reboot

Writing “halt”, “reboot”, or “panic” to `/sys/reboot` triggers a system shutdown. The reboot path is particularly interesting on multi-core systems: the kernel must migrate to CPU 0 (the boot processor), shut down all other CPUs, disable interrupts and caches, then copy a small trampoline (`rebootcode.s`) to a fixed physical address. This trampoline turns off the MMU and copies the new kernel image to its load address before jumping to it.

```

<devsys.c enum Qxxx cases 560f>+≡ (557a) <560c 589d>
    Qreboot,

```

```

<sysdir fields 560g>+≡ (557b) <560d 589e>
    "reboot", {Qreboot}, 0,      0660,

```

```

<devsys.c decls and globals 561a>+≡ (636a) <559b
enum
{
    CMhalt,
    CMreboot,
    CMpanic,
};

Cmdtab rebootmsg[] =
{
    CMhalt,      "halt",      1,
    CMreboot,    "reboot",    0,
    CMpanic,     "panic",     0,
};

<syswrite() locals 561b>+≡ (558a) <558b
    Cmdbuf *cb;
    Cmdtab *ct;

<syswrite() cases 561c>+≡ (558a) <560b 590a>
    case Qreboot:
        if(!iseve())
            error(Eperm);
        cb = parsecmd(a, n);

        if(waserror()) {
            free(cb);
            nexterror();
        }
        ct = lookupcmd(cb, rebootmsg, nelem(rebootmsg));
        switch(ct->index) {
            case CMhalt:
                arch_reboot(nil, 0, 0);
                break;
            case CMreboot:
                rebootcmd(cb->nf-1, cb->f+1);
                break;
            case CMpanic:
                *(ulong*)0=0;
                panic("/dev/reboot");
        }
        poperror();
        free(cb);
        break;

<kbdputc() reboot if ctl-alt-del 561d>≡ (331b)
    if(kbscan->ctl)
        if(kbscan->alt && c == Del) // Ctl-Alt-Del
            arch_exit(0);

<ConsKbd other fields 561e>+≡ (334b) <347a
    bool ctlpoff; // ^P will not reboot if true

<conswrite() Qconscctl other ifs 561f>≡ (347f)
    else if(strncmp(a, "ctlpon", 6) == 0){
        kbd.ctlpoff = false;
    } else if(strncmp(a, "ctlpoff", 7) == 0){
        kbd.ctlpoff = true;
    }

```

```

⟨echo() special key C-p 562a⟩≡ (574b)
case 0x10: /* ^P */
    if(cpuserver && !kbd.ctlpoff){
        active.exiting = true;
        return;
    }
    break;

```

```

⟨Active other fields 562b⟩≡ (49d) 563b▷
bool ispanic; /* shutdown in response to a panic */

```

The shutdown sequence calls each device driver's `shutdown()` method in reverse order of initialization. This gives devices a chance to flush buffers, complete pending I/O, and put hardware into a safe state before the system halts or reboots.

```

⟨function chandevshutdown 562c⟩≡ (647)
void
chandevshutdown(void)
{
    int i;

    /* shutdown in reverse order */
    for(i=0; devtab[i] != nil; i++)
        ;
    for(i--; i >= 0; i--)
        devtab[i]->shutdown();
}

```

```

⟨Dev methods 562d⟩+≡ (54a) ◁381a
void (*shutdown)(void);

```

arch_exit()(ARM)

```

⟨function main_arch_exit(arm) 562e⟩≡ (704c)
/*
 * exit kernel either on a panic or user request
 */
void
main_arch_exit(int code)
{
    void (*f)(ulong, ulong, ulong);

    shutdown(code);
    splfhi();
    if(cpu->cpuno == 0)
        archreboot();
    else{
        f = (void*)REBOOTADDR;
        intrcpushutdown();
        cacheuwbinv();
        l2cacheuwbinv();
        (*f)(0, 0, 0);
        for(;;){}
    }
}

```

```

⟨constant REBOOTADDR(arm) 562f⟩≡ (709b)
/* address at which to copy and execute rebootcode */
#define REBOOTADDR (KZERO+0x1800)

```

⟨function shutdown((init/arm/main.c)(arm) 563a)≡ (704c)

```
static void
shutdown(int ispanic)
{
    int ms, once;

    lock(&active);
    if(ispanic)
        active.ispanic = ispanic;
    else if(cpu->cpuno == 0 && (active.cpus & (1 << cpu->cpuno)) == 0)
        active.ispanic = 0;
    once = active.cpus & (1 << cpu->cpuno);
    active.cpus &= ~(1 << cpu->cpuno);
    active.exiting = 1;
    unlock(&active);

    if(once)
        iprint("cpu%d: exiting\n", cpu->cpuno);
    arch_spllo();
    for(ms = 5*1000; ms > 0; ms -= TK2MS(2)){
        arch_delay(TK2MS(2));
        if(active.cpus == 0 && consactive() == 0)
            break;
    }
    arch_delay(100*cpu->cpuno);
}
```

⟨Active other fields 563b)+≡ (49d) <562b

```
bool rebooting;    /* just idle cpus > 0 */
```

The reboot function must carefully dismantle the running kernel: first it migrates to CPU 0, copies the reboot trampoline to a fixed physical address, shuts down all other CPUs, devices, and interrupts, then calls the trampoline which turns off the MMU and copies the new kernel image to its final location in physical memory before jumping to the entry point. The trampoline itself must be position-independent and work without virtual memory, since it disables the MMU before doing the copy.

⟨function arch_reboot(arm) 563c)≡ (704c)

```
/*
 * the new kernel is already loaded at address 'code'
 * of size 'size' and entry point 'entry'.
 */
void
arch_reboot(void *entry, void *code, ulong size)
{
    void (*f)(ulong, ulong, ulong);

    //writeconf();

    /*
     * the boot processor is cpu0.  execute this function on it
     * so that the new kernel has the same cpu0.
     */
    if (cpu->cpuno != 0) {
        procwired(up, 0);
        sched();
    }
    if (cpu->cpuno != 0)
        print("on cpu%d (not 0)!\n", cpu->cpuno);

    /* setup reboot trampoline function */
```

```

f = (void*)REBOOTADDR;
memmove(f, rebootcode, sizeof(rebootcode));
cachedwbse(f, sizeof(rebootcode));

shutdown(0);

/*
 * should be the only processor running now
 */

arch_delay(5000);
print("active.machs = %x\n", active.cpus);
print("reboot entry %#lux code %#lux size %ld\n",
      PADDR(entry), PADDR(code), size);
arch_delay(100);

/* turn off buffered serial console */
serialoq = nil;
kprintoq = nil;
screenputs = nil;

/* shutdown devices */
if(!waserror()){
    chandevshutdown();
    poperror();
}

/* stop the clock (and watchdog if any) */
clockshutdown();

splfhi();
intrshutdown();

/* off we go - never to return */
cacheuwbinv();
l2cacheuwbinv();
(*f)(PADDR(entry), PADDR(code), size);

iprint("loaded kernel returned!\n");
arch_delay(1000);
archreboot();
}

```

*<function intrshutdown(*arm*) 564a>* ≡ (712c)

```

/* called from cpu0 after other cpus are shutdown */
void
intrshutdown(void)
{
    intrsoff();
    intrcpushutdown();
}

```

*<function clockshutdown(*arm*) 564b>* ≡ (715b)

```

void
clockshutdown(void)
{
    Armtimer *tm;

    tm = (Armtimer*)ARMTIMER;
    tm->ctl = 0;
}

```

```

        //wdogoff();
    }

⟨function intrcpushutdown(arm) 565a⟩≡ (712c)
void
intrcpushutdown(void)
{
    u32int *enable;

    if(soc.armlocal == 0)
        return;
    enable = (u32int*)(LOCALREGS + Localtimerint) + cpu->cpuno;
    *enable = 0;
    if(cpu->cpuno){
        enable = (u32int*)(LOCALREGS + Localmboxint) + cpu->cpuno;
        *enable = 1;
    }
}

```

rebootcode.s (arm)

```

⟨constant PTEDRAM(arm) 565b⟩≡ (708)
#define PTEDRAM      (Dom0|L1AP(Krw)|Section)

⟨instruction WFI(arm) 565c⟩≡ (708)
#define WFI WORD     $0xe320f003 /* wait for interrupt */

⟨instruction WFE(arm) 565d⟩≡ (708)
#define WFE WORD     $0xe320f002 /* wait for event */

⟨function main(rebootcode.s)(arm) 565e⟩≡ (708)
/*
 * Turn off MMU, then copy the new kernel to its correct location
 * in physical memory. Then jump to the start of the kernel.
 */

/* main(PADDR(entry), PADDR(code), size); */
TEXT    main(SB), 1, $-4
    MOVW    $setR12(SB), R12

    /* copy in arguments before stack gets unmapped */
    MOVW    R0, R8          /* entry point */
    MOVW    p2+4(FP), R9     /* source */
    MOVW    n+8(FP), R6      /* byte count */

    /* SVC mode, interrupts disabled */
    MOVW    $(PsrDirq|PsrDfiq|PsrMsvc), R1
    MOVW    R1, CPSR

    /* prepare to turn off mmu */
    BL      cachesoff(SB)

    /* turn off mmu */
    MRC     CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
    BIC     $CpCmmu, R1
    MCR     CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl

    /* continue with reboot only on cpu0 */
    CPUID(R2)

```

```

BEQ bootcpu

/* other cpus wait for inter processor interrupt from cpu0 */
/* turn icache back on */
MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
ORR $(CpCicache), R1
MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
BARRIERS
dowfi:
WFI
MOVW    $0x40000060, R1
ADD     R2<<2, R1
MOVW    0(R1), R0
AND     $0x10, R0
BEQ     dowfi
MOVW    $0x8000, R1
BL      (R1)
B       dowfi

bootcpu:
/* set up a tiny stack for local vars and memmove args */
MOVW    R8, SP          /* stack top just before kernel dest */
SUB $20, SP             /* allocate stack frame */

/* copy the kernel to final destination */
MOVW    R8, 16(SP)       /* save dest (entry point) */
MOVW    R8, R0           /* first arg is dest */
MOVW    R9, 8(SP)        /* push src */
MOVW    R6, 12(SP)       /* push size */
BL      memmove(SB)
MOVW    16(SP), R8       /* restore entry point */

/* jump to kernel physical entry point */
ORR R8,R8
B      (R8)
B      0(PC)

/*
* turn the caches off, double map PHYSDRAM & KZERO, invalidate TLBs, revert
* to tiny addresses. upon return, it will be safe to turn off the mmu.
* clobbers R0-R2, and returns with SP invalid.
*/
TEXT cachesoff(SB), 1, $-4
MOVW.DB.W [R14,R1-R10], (R13)      /* save regs on stack */

/* turn caches off, invalidate icache */
MRC CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
BIC $(CpCdcache|CpCicache|CpCpredict), R1
MCR CpSC, 0, R1, C(CpCONTROL), C(0), CpMainctl
MOVW    $0, R0
MCR CpSC, 0, R0, C(CpCACHE), C(CpCACHEinvi), CpCACHEall

/* invalidate stale TLBs before changing them */
BARRIERS
MOVW    $0, R0
MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinvu), CpTLBinu
BARRIERS

/* redo double map of first MiB PHYSDRAM = KZERO */
MOVW    12(R(CPU)), R2      /* m->mmul1 (virtual addr) */

```

```

MOVW    $PTEDRAM, R1          /* PTE bits */
MOVW    R1, (R2)

/* invalidate stale TLBs again */
BARRIERS
MOVW    $0, R0
MCR CpSC, 0, R0, C(CpTLB), C(CpTLBinv), CpTLBinv
BARRIERS

/* relocate SB and return address to PHYSDRAM addressing */
MOVW    $KSEGM, R1           /* clear segment bits */
BIC R1, R12                  /* adjust SB */
MOVM.IA.W (R13), [R14,R1-R10] /* restore regs from stack */

MOVW    $KSEGM, R1           /* clear segment bits */
BIC R1, R14                  /* adjust return address */

RET

```

23.6 /dev/

The /dev/ device (device letter #c, for “cons”) is a grab-bag of per-process and miscellaneous files. Some files return per-process values (/dev/pid, /dev/ppid, /dev/user, /dev/cputime), while others provide system services (/dev/random, /dev/null, /dev/zero, /dev/time). The console (/dev/cons) was covered in the Devices chapter; here we see the remaining files.

<devcons.c enum Qxxx cases 567a> ≡ (339c) 582a▷

```

Qbintime,
Qcputime,
Qnull,
Qpgrp,
Qpid,
Qppid,
Qrandom,
Qswap,
Qtime,
Quser,
Qzero,

```

<conssdir fields 567b> ≡ (340a) 582b▷

```

"bintime", {Qbintime}, 24,      0664,
"cputime", {Qcputime}, 6*NUMSIZE, 0444,
"null",    {Qnull},    0,      0666,
"pgrp",    {Qpgrp},    NUMSIZE, 0444,
"pid",     {Qpid},     NUMSIZE, 0444,
"ppid",    {Qppid},    NUMSIZE, 0444,
"random",  {Qrandom},  0,      0444,
"swap",    {Qswap},    0,      0664,
"time",    {Qtime},    NUMSIZE+3*VLNUMSIZE, 0664,
"user",    {Quser},    0,      0666,
"zero",    {Qzero},    0,      0444,

```

<consinit() initializing things 567c> +≡ (344c) ◁335b

```

todinit();
randominit();

```


23.6.1 /dev/random

The random number generator uses two layers. The outer layer (`nrand()`) is a simple linear congruential generator seeded from the inner layer. The inner layer (`/dev/random`) collects entropy from timing jitter: a dedicated kernel process (`kgenrandom`) busy-loops incrementing a counter, and a periodic clock callback (`randomclock`) samples that counter to extract unpredictable bits. The idea is that the exact count depends on cache behavior, interrupts, and scheduling—all sources of genuine randomness. The entropy is accumulated in a circular buffer (`rb`) using a producer/consumer pattern.

```
<global randn 568a>≡ (607)
static ulong randn;
```

```
<function seedrand 568b>≡ (607)
static void
seedrand(void)
{
    if(!waserror()){
        randomread((void*)&randn, sizeof(randn));
        poperror();
    }
}
```

```
<function nrand 568c>≡ (607)
int
nrand(int n)
{
    if(randn == 0)
        seedrand();
    randn = randn*1103515245 + 12345 + CPUS(0)->ticks;
    return (randn>>16) % n;
}
```

```
<struct Rb 568d>≡ (668a)
struct Rb
{
    QLock;
    Rendez producer;
    Rendez consumer;
    ulong randomcount;
    byte buf[1024];
    byte *ep;
    byte *rp;
    byte *wp;
    byte next;
    byte wakeme;
    ushort bits;
    ulong randn;
};
```

```
<global rb 568e>≡ (668a)
struct Rb rb;
```

```
<struct rbnotfull 568f>≡ (668a)
static int
rbnotfull(void*)
{
    int i;

    i = rb.rp - rb.wp;
    return i != 1 && i != (1 - sizeof(rb.buf));
}
```

```

⟨struct rbnotempty 569a⟩≡ (668a)
    static int
    rbnotempty(void*)
    {
        return rb.wp != rb.rp;
    }

⟨function genrandom 569b⟩≡ (668a)
    static void
    genrandom(void*)
    {
        up->basepri = PriNormal;
        up->priority = up->basepri;

        for(;;){
            for(;;)
                if(++rb.randomcount > 100000)
                    break;
            if(anyhigher())
                sched();
            if(!rbnotfull(0))
                sleep(&rb.producer, rbnotfull, 0);
        }
    }

⟨clock callback randomclock 569c⟩≡ (668a)
    /*
     * produce random bits in a circular buffer
     */
    static void
    randomclock(void)
    {
        if(rb.randomcount == 0 || !rbnotfull(0))
            return;

        rb.bits = (rb.bits<<2) ^ rb.randomcount;
        rb.randomcount = 0;

        rb.next++;
        if(rb.next != 8/2)
            return;
        rb.next = 0;

        *rb.wp ^= rb.bits;
        if(rb.wp+1 == rb.ep)
            rb.wp = rb.buf;
        else
            rb.wp = rb.wp+1;

        if(rb.wakeme)
            wakeup(&rb.consumer);
    }

⟨function randominit 569d⟩≡ (668a)
    void
    randominit(void)
    {
        /* Frequency close but not equal to HZ */
        addclock0link(randomclock, 13);
        rb.ep = rb.buf + sizeof(rb.buf);
    }

```

```

    rb.rp = rb.wp = rb.buf;
    kproc("kgenrandom", genrandom, 0);
}

```

⟨function randomread 570a⟩≡ (668a)

```

/*
 * consume random bytes from a circular buffer
 */
ulong
randomread(void *xp, ulong n)
{
    byte *e, *p;
    ulong x;

    p = xp;

    if(waserror()){
        qunlock(&rb);
        nexterror();
    }

    qlock(&rb);
    for(e = p + n; p < e; ){
        if(rb.wp == rb.rp){
            rb.wakeme = 1;
            wakeup(&rb.producer);
            sleep(&rb.consumer, rb.notempty, 0);
            rb.wakeme = 0;
            continue;
        }

        /*
         * beating clocks will be predictable if
         * they are synchronized. Use a cheap pseudo-
         * random number generator to obscure any cycles.
         */
        x = rb.randn*1103515245 ^ *rb.rp;
        *p++ = rb.randn = x;

        if(rb.rp+1 == rb.ep)
            rb.rp = rb.buf;
        else
            rb.rp = rb.rp+1;
    }
    qunlock(&rb);
    poperror();

    wakeup(&rb.producer);

    return n;
}

```

⟨consread() cases 570b⟩+≡ (345c) <445a 582c>

```

case Qrandom:
    return randomread(buf, n);

```

Chapter 24

Conclusion

You now know how the Plan 9 kernel works, to the smallest details: from the first instruction at `_start` to the scheduler loop, from page faults to pipes, from system calls to the 9P protocol.

Despite being a complete operating system kernel—with virtual memory, preemptive scheduling, multiprocessor support, a file-oriented device framework, and a network-transparent namespace—the entire kernel fits in roughly 30 000 lines of C and a few hundred lines of assembly. This is remarkably small. The key to this compactness is the “everything is a file server” philosophy: instead of adding features to the kernel, Plan 9 pushes them to user-space file servers communicating over 9P. The kernel provides just enough mechanism (processes, memory, files, namespaces) and leaves policy to user space.

Along the way, you have seen many recurring patterns: reference counting for shared resources (`Ref`), sleep/wakeup for blocking (`Rendez`), two-level tables for sparse mappings (`Segment/Pagetable`, L1/L2 page tables), dispatch tables for polymorphism (`systab`, `devtab`), and the `waserror/poperror` mechanism for exception handling in C. You have also seen how a well-chosen set of abstractions —`Chan`, `Dev`, `Segment`, `Proc`, `Page`—can compose to build a complete system where each piece is understandable in isolation.

24.1 Connections to other books

The kernel is the foundation on which every other program in Principia Softwarica runs. The books are deeply interconnected:

- The SHELL book [Pad18] explains `rc`, the first program most users interact with. Every command you type in `rc` exercises `rfork()`, `exec()`, `wait()`, pipes, and file descriptors—all of which you now understand from the kernel side.
- The COMPILER book [Pad16b] and ASSEMBLER book [Pad15a] describe the tools that produce the kernel binary itself, including the 5c/5a compilers whose calling conventions and `extern register` extension you have seen throughout the ARM-specific code.
- The LINKER book [Pad15c] explains how 5l produces the `a.out` binary that `sysexec()` loads, and how the static base register R12 is set up.
- The BUILDER book [Pad16a] explains `mk`, which orchestrates the kernel build and whose `mkfile` rules drive the compilation of all the source files discussed in this book.
- The GRAPHICS book [Pad16d] covers `devdraw` and the framebuffer infrastructure that this book only touched briefly (the screen output and `screenputs` hook).
- The LIBCORE book [Pad16c] explains the C library that sits between user programs and the kernel: the `pwrite()/brk()/rfork()` wrappers, the `Pool` allocator shared with the kernel’s `malloc`, and the `print()` formatting engine.

24.2 What this book does not cover

This book focused on the core kernel mechanisms. Several important subsystems were left for other books or future work:

- *Networking:* The IP stack (`devip`), the Ethernet driver (`devether`), and the network file servers are substantial enough for their own treatment.
- *Graphics:* The draw device (`devdraw`), the window system `rio`, and the graphics libraries deserve a dedicated GRAPHICS book [Pad16d].
- *On-disk file systems:* The file servers `fossil`, `cwfs`, and `hjfs` implement the actual storage layer that the kernel's `devmnt` connects to over 9P.
- *Authentication:* The `factotum` authentication agent and the security architecture (capabilities, `srvauth`, tickets) are covered only superficially here.

Despite these omissions, the kernel described in this book is complete and self-contained: it can boot on a Raspberry Pi, run user programs, manage memory, handle interrupts, and serve files. Everything else is built on top of the mechanisms you now understand.

Appendix A

Debugging

This appendix collects the kernel's debugging infrastructure: special keyboard shortcuts that dump internal state, the `kmesg` and `kprint` log devices, the serial console (essential when the screen is not yet working), and the remote debugger interface. These are the tools a kernel developer reaches for when something goes wrong during boot or at runtime.

A.1 `sysnop()`

A.2 Special keys and dumpers

Pressing F11 enables keyboard debug mode, which prints every scan code as it arrives. F12 turns it off. Other special key combinations dump internal state (memory allocator, process table, etc.) to the console—invaluable when the system is hung and you cannot type commands:

```
<global kdebug 573a>≡ (612)
    bool kdebug;
```

```
<kbdputc() special keyboard debug keys cases 573b>≡ (331b)
    case KF|11:
        print("kbd debug on, F12 turns it off\n");
        kdebug = true;
        kbdputc(c);
        break;
    case KF|12:
        kdebug = false;
        kbdputc(c);
        break;
```

```
<kbdputc() debugging 573c>≡ (331b)
    if(kdebug)
        print("sc %x (ms %d)\n", k, mouseshifted);
```

```
<kbdputc() debugging up shift 573d>≡ (331b)
    if(kdebug)
        print("shiftclr\n");
```

```
<kbdputc() debugging down shift 573e>≡ (331b)
    if(kdebug)
        print("shift\n");
```

```
<kbdputc() debugging 573f>≡ (334d)
    extern bool kdebug;

    if(kdebug)
        print("kbdputc(0x%x)\n", ch);
```

`<echo() locals 574a>≡` (336a)

```
static int ctrlt;  
void* tmp;  
int x;  
char *e, *p;
```

`<echo() special keys handler 574b>≡` (336a)

```
e = buf+n;  
for(p = buf; p < e; p++){  
    switch(*p){
```

`<echo() special key C-p 562a>`

```
case 0x14: /* ^T */  
    ctrlt++;  
    if(ctrlt > 2)  
        ctrlt = 2;  
    continue;  
}  
if(ctrlt != 2)  
    continue;  
  
/* ^T escapes */  
ctrlt = 0;  
switch(*p){  
case 's':  
    arch_dumpstack();  
    return;  
case 'S':  
    x = arch_splhi();  
    arch_dumpstack();  
    procdump();  
    arch_splx(x);  
    return;  
case 'x':  
    xsummary();  
    tmp = xalloc(1000); // PAD: just to debug  
    xalloc(1000);  
    xfree(tmp);  
    xsummary();  
    return;  
case 'X':  
    xsummary();  
    ixsummary();  
    mallocsummary();  
    arch_memorysummary();  
    pagerssummary();  
    return;  
case 'm':  
    arch_memorysummary();  
    return;  
case 'p':  
    x = arch_spllo();  
    procdump();  
    arch_splx(x);  
    return;  
case 'q':  
    scheddump();  
    return;  
case 'k':
```

```

        killbig("^t ^t k");
        return;
    case 'r':
        arch_exit(0);
        return;
    <echo() C-t C-t special keys handler other cases 584a>
}

}

<function xsummary 575a>≡ (660b)
void
xsummary(void)
{
    int i;
    Hole *h;

    for(i = 0; i < Nhole && xlists.hole[i].top != 0; i++) {
        print("|i| = %d (0x%luX), addr 0x%luX, top = 0x%luX, size = %ld, link = 0x%luX\n",
            i, &xlists.hole[i],
            xlists.hole[i].addr, xlists.hole[i].top, xlists.hole[i].size,
            xlists.hole[i].next
        );
    }
    print("flists = 0x%luX, sorted_holes = 0x%luX\n", xlists.unused_slots, xlists.sorted_holes);
    i = 0;
    for(h = xlists.unused_slots; h; h = h->next)
        i++;

    print("%d holes free", i);
    i = 0;
    for(h = xlists.sorted_holes; h; h = h->next) {
        if (0) {
            print("addr %#.8lux top %#.8lux size %lud\n",
                h->addr, h->top, h->size);
            arch_delay(10);
        }
        i += h->size;
        if (h == h->next) {
            print("xsummary: infinite loop broken\n");
            break;
        }
    }
    print(" %d bytes free\n", i);
}

<function mallosummary 575b>≡ (661b)
void
mallosummary(void)
{
    poolsummary(mainmem);
    poolsummary(imagemem);
}

<function poolsummary 575c>≡ (661b)
void
poolsummary(Pool *p)
{
    print("%s max %lud cur %lud free %lud alloc %lud\n", p->name,
        p->maxsize, p->cursize, p->curfree, p->curalloc);
}

```



```

⟨function arch_memorysummary(arm) 576a)≡ (704c)
// called from devcons.c
void
arch_memorysummary(void) {
}

⟨function pagerssummary 576b)≡ (666)
void
pagerssummary(void)
{
    print("%lud/%lud memory %lud/%lud swap %d iolist\n",
        palloc.user-palloc.freecount,
        palloc.user, conf.nswap-swapalloc.free, conf.nswap,
        ioptr);
}

⟨function ixsummary 576c)≡ (652b)
void
ixsummary(void)
{
    debugging ^= 1;
    iallocsummary();
    print("pad %lud, concat %lud, pullup %lud, copy %lud\n",
        padblockcnt, concatblockcnt, pullupblockcnt, copyblockcnt);
    print("consume %lud, produce %lud, qcopy %lud\n",
        consumeent, produceent, qcopyent);
}

⟨function iallocsummary 576d)≡ (650)
void
iallocsummary(void)
{
    print("ialloc %lud/%lud\n", ialloc.bytes, conf.ialloc);
}

⟨function scheddump 576e)≡ (675c)
void
scheddump(void)
{
    Proc *p;
    Schedq *rq;

    for(rq = &runq[Nrq-1]; rq >= runq; rq--){
        if(rq->head == 0)
            continue;
        print("rq%ld:", rq-runq);
        for(p = rq->head; p; p = p->rnext)
            print(" %lud(%lud)", p->pid, cpu->ticks - p->readytime);
        print("\n");
        arch_delay(150);
    }
    print("nrty %d\n", nrty);
}

⟨function procdump 576f)≡ (675c)
void
procdump(void)
{
    int i;
    Proc *p;

```

```

    if(up)
        print("up %lud\n", up->pid);
    else
        print("no current process\n");
    for(i=0; i<conf.nproc; i++) {
        p = &procalloc.arena[i];
        if(p->state == Dead)
            continue;

        dumpaproc(p);
    }
}

⟨function dumpaproc 577a⟩≡ (675c)
void
proc_dumpaproc(Proc *p)
{
    ulong bss;
    char *s;

    if(p == nil)
        return;

    bss = 0;
    if(p->seg[BSEG])
        bss = p->seg[BSEG]->top;

    s = p->psstate;
    if(s == nil)
        s = statename[p->state];
    print("%3lud:%10s pc %8lux dbgpc %8lux %8s (%s) ut %ld st %ld bss %lux qpc %lux nl %lud nd %lud lpc %lux p
        p->pid, p->text, p->pc, arch_dbgpc(p), s, statename[p->state],
        p->time[TUser], p->time[TSys], bss, p->qpc, p->nlocks.ref, p->delaysched, p->lastlock ? p->lastlock->pc
}

⟨function arch_dbgpc(arm) 577b⟩≡ (717a)
/*
 * pc output by dumpaproc
 */
uintptr
arch_dbgpc(Proc* p)
{
    Ureg *ureg;

    ureg = p->dbgreg;
    if(ureg == 0)
        return 0;

    return ureg->pc;
}

⟨function lockloop 577c⟩≡ (605b)
void
lockloop(Lock *l, ulong pc)
{
    Proc *p;

    p = l->p;
    print("lock %#p loop key %#lux pc %#lux held by pc %#lux proc %lud\n",

```

```

    l, l->key, pc, l->pc, p ? p->pid : 0);
dumpaproc(up);
if(p != nil)
    dumpaproc(p);
}

```

⟨function dumpregs(*arm*) 578a)≡ (712c)

```

void
dumpregs(Ureg* ureg)
{
    int s;

    if (ureg == nil) {
        iprint("trap: no user process\n");
        return;
    }
    s = arch_splhi();
    iprint("trap: %s", trapname(ureg->type));
    if(ureg != nil && (ureg->psr & PsrMask) != PsrMsvc)
        iprint(" in %s", trapname(ureg->psr));
    iprint("\n");
    iprint("psr %8.8lux type %2.2lux pc %8.8lux link %8.8lux\n",
        ureg->psr, ureg->type, ureg->pc, ureg->link);
    iprint("R14 %8.8lux R13 %8.8lux R12 %8.8lux R11 %8.8lux R10 %8.8lux\n",
        ureg->r14, ureg->r13, ureg->r12, ureg->r11, ureg->r10);
    iprint("R9 %8.8lux R8 %8.8lux R7 %8.8lux R6 %8.8lux R5 %8.8lux\n",
        ureg->r9, ureg->r8, ureg->r7, ureg->r6, ureg->r5);
    iprint("R4 %8.8lux R3 %8.8lux R2 %8.8lux R1 %8.8lux R0 %8.8lux\n",
        ureg->r4, ureg->r3, ureg->r2, ureg->r1, ureg->r0);
    iprint("stack is at %#p\n", ureg);
    iprint("pc %#lux link %#lux\n", ureg->pc, ureg->link);

    if(up)
        iprint("user stack: %#p-%#p\n", up->kstack, up->kstack+KSTACK-4);
    else
        iprint("kernel stack: %8.8lux-%8.8lux\n",
            (ulong)(cpu+1), (ulong)cpu+BY2PG-4);
    dumplongs("stack", (ulong*)(ureg + 1), 16);
    arch_delay(2000);
    arch_dumpstack();
    arch_splx(s);
}

```

⟨function dumplongs(*arm*) 578b)≡ (712c)

```

static void
dumplongs(char *msg, ulong *v, int n)
{
    int i, l;

    l = 0;
    iprint("%s at %.8p: ", msg, v);
    for(i=0; i<n; i++){
        if(l >= 4){
            iprint("\n    %.8p: ", v);
            l = 0;
        }
        if(isvalidaddr(v)){
            iprint(" %8.8lux", *v++);
            l++;
        }else{

```

```

        iprint(" invalid");
        break;
    }
}
iprint("\n");
}

⟨function isvalidaddr(arm) 579a)≡ (712c)
int
isvalidaddr(void *v)
{
    return (uintptr)v >= KZERO;
}

⟨function trapname(arm) 579b)≡ (712c)
static char *
trapname(int psr)
{
    char *s;

    s = trapnames[psr & PsrMask];
    if(s == nil)
        s = "unknown trap number in psr";
    return s;
}

⟨faultarm() if debug 579c)≡ (159)
if (Debug)
    ckfaultstuck(va);

⟨function ckfaultstuck(arm) 579d)≡ (712c)
/* this is quite helpful during mmu and cache debugging */
static void
ckfaultstuck(uintptr va)
{
    static int cnt, lastpid;
    static uintptr lastva;

    if (va == lastva && up->pid == lastpid) {
        ++cnt;
        if (cnt >= 2)
            /* fault() isn't fixing the underlying cause */
            panic("fault: %d consecutive faults for va %#p",
                cnt+1, va);
    } else {
        cnt = 0;
        lastva = va;
        lastpid = up->pid;
    }
}

```

A.3 /bin/ktrace (ARM)

```

⟨function trap_arch_dumpstack(arm) 579e)≡ (712c)
void
trap_arch_dumpstack(void)
{
    arch_callwithureg(dumpstackwithureg);
}

```

⟨function arch_callwithureg(*arm*) 580a)≡ (712c)

```
void
arch_callwithureg(void (*fn)(Ureg*))
{
    Ureg ureg;

    getpcsp((ulong*)&ureg.pc, (ulong*)&ureg.sp);
    ureg.r14 = getcallerpc(&fn);
    fn(&ureg);
}
```

⟨function getpcsp(*arm*) 580b)≡ (712c)

```
/*
 * Fill in enough of Ureg to get a stack trace, and call a function.
 * Used by debugging interface rdb.
 */
static void
getpcsp(ulong *pc, ulong *sp)
{
    *pc = getcallerpc(&pc);
    *sp = (ulong)&pc-4;
}
```

⟨function dumpstackwithureg(*arm*) 580c)≡ (712c)

```
static void
dumpstackwithureg(Ureg *ureg)
{
    uintptr l, i, v, estack;
    u32int *p;
    char *s;

    if((s = getconf("nodumpstack")) != nil && strcmp(s, "0") != 0){
        iprint("dumpstack disabled\n");
        return;
    }
    iprint("ktrace /kernel/path %#.8lux %#.8lux %#.8lux # pc, sp, link\n",
        ureg->pc, ureg->sp, ureg->r14);
    arch_delay(2000);
    i = 0;
    if(up != nil && (uintptr)&l <= (uintptr)up->kstack+KSTACK)
        estack = (uintptr)up->kstack+KSTACK;
    else if((uintptr)&l >= (uintptr)cpu->stack
        && (uintptr)&l <= (uintptr)cpu + CPUSIZE)
        estack = (uintptr)cpu + CPUSIZE;
    else{
        if(up != nil)
            iprint("&up->kstack %#p &l %#p\n", up->kstack, &l);
        else
            iprint("&m %#p &l %#p\n", cpu, &l);
        return;
    }
    for(l = (uintptr)&l; l < estack; l += sizeof(uintptr)){
        v = *(uintptr*)l;
        if(KTZERO < v && v < (uintptr)etext && !(v & 3)){
            v -= sizeof(u32int);          /* back up an instr */
            p = (u32int*)v;
            if((*p & 0x0f000000) == 0xb000000){ /* BL instr? */
                iprint("%#8.8lux=%#8.8lux ", l, v);
                i++;
            }
        }
    }
```

```

    }
    if(i == 4){
        i = 0;
        iprint("\n");
    }
}
if(i)
    iprint("\n");
}

```

A.4 /dev/kmesg

<struct KMesg 581a>≡ (607)

```

/*
 * Log console output so it can be retrieved via /dev/kmesg.
 * This is good for catching boot-time messages after the fact.
 */
struct KMesg {
    Lock lk;
    char buf[KMESGSIZE];
    uint n;
};

```

<global kmesg 581b>≡ (607)

```

struct KMesg kmesg;

```

<constant KMESGSIZE(arm) 581c>≡ (695a)

```

#define KMESGSIZE (16*1024) // for /dev/kmesg

```

<echo() hooks 581d>+≡ (336a) <336b 584d>

```

kmesgputs(buf, n);

```

<putstrn0() kmesg handling 581e>≡ (342a)

```

/*
 * how many different output devices do we need?
 */
kmesgputs(str, n);

```

<function kmesgputs 581f>≡ (607)

```

static void
kmesgputs(char *str, int n)
{
    uint nn, d;

    ilock(&kmesg.lk);
    /* take the tail of huge writes */
    if(n > sizeof kmesg.buf){
        d = n - sizeof kmesg.buf;
        str += d;
        n -= d;
    }

    /* slide the buffer down to make room */
    nn = kmesg.n;
    if(nn + n >= sizeof kmesg.buf){
        d = nn + n - sizeof kmesg.buf;
        if(d)
            memmove(kmesg.buf, kmesg.buf+d, sizeof kmesg.buf-d);
    }
}

```

```

        nn -= d;
    }

    /* copy the data in */
    memmove(kmesg.buf+nn, str, n);
    nn += n;
    kmesg.n = nn;
    iunlock(&kmesg.lk);
}

⟨devcons.c enum Qxxx cases 582a⟩+≡ (339c) ◁567a 582d▷
    Qkmesg,

⟨conmdir fields 582b⟩+≡ (340a) ◁567b 582e▷
    "kmesg",    {Qkmesg},    0,    0440,

⟨consread() cases 582c⟩+≡ (345c) ◁570b 583c▷
    case Qkmesg:
        /*
         * This is unlocked to avoid tying up a process
         * that's writing to the buffer.  kmesg.n never
         * gets smaller, so worst case the reader will
         * see a slurred buffer.
         */
        if(off >= kmesg.n)
            n = 0;
        else{
            if(off+n > kmesg.n)
                n = kmesg.n - off;
            memmove(buf, kmesg.buf+off, n);
        }
        return n;

```

A.5 /dev/kprint

```

⟨devcons.c enum Qxxx cases 582d⟩+≡ (339c) ◁582a
    Qkprint,

⟨conmdir fields 582e⟩+≡ (340a) ◁582b
    "kprint",    {Qkprint, 0, QTEXCL},    0,    DMEXCL|0440,

⟨Dirmode cases 582f⟩+≡ (295c) ◁379a
    DMEXCL = 0x20000000, /* mode bit for exclusive use files */

⟨global kprntoq 582g⟩≡ (607)
    Queue* kprntoq; /* console output, for /dev/kprint */

⟨global kprntinuse 582h⟩≡ (607)
    ulong kprntinuse; /* test and set whether /dev/kprint is open */

```

```

⟨consopen() cases 583a⟩+≡ (345a) <347d
    case Qkprint:
        if(arch_tas(&kprintinuse) != 0){
            c->flag &= ~COPEN;
            error(Einuse);
        }
        if(kprintoq == nil){
            kprintoq = qopen(8*1024, Qcoalesce, 0, 0);
            if(kprintoq == nil){
                c->flag &= ~COPEN;
                error(Enomem);
            }
            qnoblock(kprintoq, true);
        }else
            qreopen(kprintoq);
        c->iounit = qiomaxatomic;
        break;

⟨consclose() cases 583b⟩+≡ (345b) <347e
    /* close of kprint allows other opens */
    case Qkprint:
        if(c->flag & COPEN){
            kprintinuse = 0;
            qhangup(kprintoq, nil);
        }
        break;

⟨consread() cases 583c⟩+≡ (345c) <582c
    case Qkprint:
        return qread(kprintoq, buf, n);

⟨putstrn0() if kprint 583d⟩≡ (342a)
    /*
     * if someone is reading /dev/kprint,
     * put the message there.
     * if not and there's an attached bit mapped display,
     * put the message there.
     *
     * if there's a serial line being used as a console,
     * put the message there.
     */
    if(kprintoq != nil && !qisclosed(kprintoq)){
        if(usewrite)
            qwrite(kprintoq, str, n);
        else
            qiwrite(kprintoq, str, n);
    }

⟨panic() reset kprintoq 583e⟩≡ (594b)
    kprintoq = nil; /* don't try to write to /dev/kprint */

```

A.6 Remote debugger

```

⟨hook consdebug 583f⟩≡ (607)
    void (*consdebug)(void) = nil; // for rdb

⟨panic() run consdebug hook 583g⟩≡ (594b)
    if(consdebug)
        (*consdebug)();

```



```

⟨echo() C-t C-t special keys handler other cases 584a⟩≡ (574b)
case 'd':
    if(consdebug == nil)
        consdebug = rdb;
    else
        consdebug = nil;
    print("consdebug now %#p\n", consdebug);
    return;
case 'D':
    if(consdebug == nil)
        consdebug = rdb;
    consdebug();
    return;

```

A.7 Serial line

On the Raspberry Pi, printing to the screen requires a working framebuffer, font rendering, and drawing primitives—none of which are available during early boot. The serial UART is the escape hatch: it works with a single register write, so `uartputs()` can output text from the very first instruction. This is how kernel developers debug boot failures.

A.7.1 serialoq

```

⟨global serialoq 584b⟩≡ (607)
// option<Queue>
Queue* serialoq; /* serial console output */

```

```

⟨function consactive 584c⟩≡ (607)
int
consactive(void)
{
    if(serialoq)
        return qlen(serialoq) > 0;
    return 0;
}

```

```

⟨echo() hooks 584d⟩+≡ (336a) <581d
if(serialoq)
    echoserialoq(buf, n);

```

```

⟨function echoserialoq 584e⟩≡ (607)
static void
echoserialoq(char *buf, int n)
{
    char *e, *p;
    char ebuf[128];
    int x;

    p = ebuf;
    e = ebuf + sizeof(ebuf) - 4;
    while(n-- > 0){
        if(p >= e){
            qiwrite(serialoq, ebuf, p - ebuf);
            p = ebuf;
        }
        x = *buf++;
        if(x == '\n'){

```

```

        *p++ = '\r';
        *p++ = '\n';
    } else if(x == 0x15){
        *p++ = '^';
        *p++ = 'U';
        *p++ = '\n';
    } else
        *p++ = x;
}
if(p != ebuf)
    qiwrite(serialoq, ebuf, p - ebuf);
}

⟨putstrn0() serialoq handling 585a⟩≡ (342a)
/*
 * Convert \n to \r\n for serial
 * line consoles. Locking of the queues is left up to the screen
 * or uart code. Multi-line messages to serial consoles may get
 * interspersed with other messages.
 */
⟨putstrn0() if serialoq nil 585b⟩
// else

while(n > 0) {
    t = memchr(str, '\n', n);
    if(t && !kbd.raw) {
        m = t-str;
        if(usewrite){
            qwrite(serialoq, str, m);
            qwrite(serialoq, "\r\n", 2);
        } else {
            qiwrite(serialoq, str, m);
            qiwrite(serialoq, "\r\n", 2);
        }
        n -= m+1;
        str = t+1;
    } else {
        if(usewrite)
            qwrite(serialoq, str, n);
        else
            qiwrite(serialoq, str, n);
        break;
    }
}
}

```

A.7.2 uartconsinit()

A.7.3 uartputs()

```

⟨putstrn0() if serialoq nil 585b⟩≡ (585a)
    if(serialoq == nil){
        uartputs(str, n);
        return;
    }

```

A.7.4 consuart

```

⟨global consuart 585c⟩≡ (? 0—1)
Uart* consuart;

```

A.7.5 Uart, PhysUart

Appendix B

Profiling

This appendix collects the kernel’s performance measurement code: per-CPU statistics (idle time, interrupt time), the `/dev/kprof` sampling profiler, and various counters for locks, qlocks, and scheduling. These are the “gauges on the dashboard” that let developers find bottlenecks.

B.1 `/sys/sysstat` (ARM)

```
<Cpu stat fields 587a>≡ (48a) 587f▷
    Perf perf; /* performance counters */

<struct Perf 587b>≡ (609)
/*
 * performance timers, all units in perfticks
 */
struct Perf
{
    // intr-ts? interrupt time stamp?
    ulong intrts; /* time of last interrupt */

    ulong inintr; /* time since last clock tick in interrupt handlers */
    ulong avg_inintr; /* avg time per clock tick in interrupt handlers */

    ulong inidle; /* time since last clock tick in idle loop */
    ulong avg_inidle; /* avg time per clock tick in idle loop */

    ulong last; /* value of arch_perfticks() at last clock tick */
    ulong period; /* arch_perfticks() per clock tick */
};

<runproc() other locals 587c>+≡ (196) <440h
    ulong start, now;

<runproc() start 587d>≡ (196)
    start = arch_perfticks();

<runproc() after arch_idlehands, before restarting loop 587e>≡ (196)
    /* remember how much time we're here */
    now = arch_perfticks();
    cpu->perf.inidle += now-start;
    start = now;

<Cpu stat fields 587f>+≡ (48a) <587a 589c>
    int load;
```

```
<hzclock() accounttime 588a>≡ (207a)
```

```
accounttime();
```

```
<function accounttime 588b>≡ (675c)
```

```
/*
 * time accounting called by clock() splhi'd
 */
void
accounttime(void)
{
    Proc *p;
    ulong n, per;
    static ulong nrun;

    p = cpu->proc; // why not p = up?
    if(p) {
        nrun++;
        <accounttime() update time of current process 444e>
    }

    /* calculate decaying duty cycles */
    n = arch_perfticks();
    per = n - cpu->perf.last;
    cpu->perf.last = n;
    per = (cpu->perf.period*(Arch_HZ-1) + per)/Arch_HZ;
    if(per != 0)
        cpu->perf.period = per;

    cpu->perf.avg_inidle =
        (cpu->perf.avg_inidle*(Arch_HZ-1)+cpu->perf.inidle)/Arch_HZ;
    cpu->perf.inidle = 0;

    cpu->perf.avg_inintr =
        (cpu->perf.avg_inintr*(Arch_HZ-1)+cpu->perf.inintr)/Arch_HZ;
    cpu->perf.inintr = 0;

    /* only one processor gets to compute system load averages */
    if(cpu->cpuno != 0)
        return;

    /*
     * calculate decaying load average.
     * if we decay by (n-1)/n then it takes
     * n clock ticks to go from load L to .36 L once
     * things quiet down. it takes about 5 n clock
     * ticks to go to zero. so using HZ means this is
     * approximately the load over the last second,
     * with a tail lasting about 5 seconds.
     */
    n = nrun;
    nrun = 0;
    n = (nrndy+n)*1000;
    cpu->load = (cpu->load*(Arch_HZ-1)+n)/Arch_HZ;
}
```

```
<function intrtime(arm) 588c>≡ (712c)
```

```
static void
intrtime(void)
{
    ulong diff;
```

```

    ulong x;

    x = arch_perfticks();
    diff = x - cpu->perf.intrts;
    cpu->perf.intrts = x;

    cpu->perf.inintr += diff;
    if(up == nil && cpu->perf.inidle > diff)
        cpu->perf.inidle -= diff;
}

⟨function arch_perfticks(arm) 589a⟩≡ (715b)
    ulong
    arch_perfticks(void)
    {
        Armtimer *tm;

        tm = (Armtimer*)ARMTIMER;
        return tm->count;
    }

⟨struct Armtimer(arm) 589b⟩≡ (715b)
    struct Armtimer {
        u32int load;
        u32int val;
        u32int ctl;
        u32int irqack;
        u32int irq;
        u32int maskedirq;
        u32int reload;
        u32int predivider;
        u32int count;
    };

⟨Cpu stat fields 589c⟩+≡ (48a) <587f
    int cs; // context switch, sched() and sleep() call
    int intr;
    int syscall;
    int pfault;
    int tlbfault;
    int tlbpurge;
    ulong spuriousintr; // not really used

⟨devsys.c enum Qxxx cases 589d⟩+≡ (557a) <560f
    Qsysstat,

⟨sysdir fields 589e⟩+≡ (557b) <560g
    "sysstat", {Qsysstat}, 0, 0666,

⟨sysread() cases 589f⟩+≡ (557e) <560e
    case Qsysstat:
        b = smalloc(conf.ncpu*(NUMSIZE*11+1) + 1); /* +1 for NUL */
        bp = b;
        for(id = 0; id < MAXCPUS; id++) {
            if(active.cpus & (1<<id)) {
                mp = CPUS(id);
                readnum(0, bp, NUMSIZE, id, NUMSIZE);
                bp += NUMSIZE;
                readnum(0, bp, NUMSIZE, mp->cs, NUMSIZE);
                bp += NUMSIZE;
            }
        }
    }

```

```

        readnum(0, bp, NUMSIZE, mp->intr, NUMSIZE);
        bp += NUMSIZE;
        readnum(0, bp, NUMSIZE, mp->syscall, NUMSIZE);
        bp += NUMSIZE;
        readnum(0, bp, NUMSIZE, mp->pfault, NUMSIZE);
        bp += NUMSIZE;
        readnum(0, bp, NUMSIZE, mp->tlbfault, NUMSIZE);
        bp += NUMSIZE;
        readnum(0, bp, NUMSIZE, mp->tlbpurge, NUMSIZE);
        bp += NUMSIZE;
        readnum(0, bp, NUMSIZE, mp->load, NUMSIZE);
        bp += NUMSIZE;
        readnum(0, bp, NUMSIZE,
            (mp->perf.avg_inidle*100)/mp->perf.period,
            NUMSIZE);
        bp += NUMSIZE;
        readnum(0, bp, NUMSIZE,
            (mp->perf.avg_inintr*100)/mp->perf.period,
            NUMSIZE);
        bp += NUMSIZE;
        *bp++ = '\n';
    }
}
if(waserror()){
    free(b);
    nexterror();
}
n = readstr((ulong)offset, buf, n, b);
free(b);
poperror();
return n;

```

```

<syswrite() cases 590a>+≡ (558a) <561c
case Qsysstat:
    for(id = 0; id < 32; id++) {
        if(active.cpus & (1<<id)) {
            mp = CPUS(id);
            mp->cs = 0;
            mp->intr = 0;
            mp->syscall = 0;
            mp->pfault = 0;
            mp->tlbfault = 0;
            mp->tlbpurge = 0;
        }
    }
    break;

```

B.2 /dev/kprof

```

<hzclock() if kproftimer 590b>≡ (207a)
    if(kproftimer != nil)
        kproftimer(ur->pc);

<hook kproftimer 590c>≡ (671a)
    void (*kproftimer)(ulong);

```

B.3 Statistics

B.3.1 Lock.lockcycles

```
<lock ifdef LOCKCYCLES 591a>≡ (61d 60e 58)
    #ifdef LOCKCYCLES
        l->lockcycles = -arch_lcycles();
    #endif

<Lock debugging fields 591b>+≡ (58c) <69b
    // #ifdef LOCKCYCLES
    long lockcycles;
    // #endif

<unlock ifdef LOCKCYCLES 591c>≡ (60c)
    #ifdef LOCKCYCLES
        l->lockcycles += lcycles();
        cumlockcycles += l->lockcycles;
        if(l->lockcycles > maxlockcycles){
            maxlockcycles = l->lockcycles;
            maxlockpc = l->pc;
        }
    #endif

<iunlock ifdef LOCKCYCLES 591d>≡ (62a)
    #ifdef LOCKCYCLES
        l->lockcycles += lcycles();
        cumilockcycles += l->lockcycles;
        if(l->lockcycles > maxilockcycles){
            maxilockcycles = l->lockcycles;
            maxilockpc = l->pc;
        }
        if(l->lockcycles > 2400)
            ilockpcs[n++ & 0xff] = l->pc;
    #endif

<globals lockcycles 591e>≡ (605b)
    #ifdef LOCKCYCLES
        long maxlockcycles;
        long maxilockcycles;
        long cumlockcycles;
        long cumilockcycles;
        ulong maxlockpc;
        ulong maxilockpc;

        ulong ilockpcs[0x100] = { [0xff] = 1 };
        static int n;
    #endif
```

B.3.2 TaslockStats

```
<struct TaslockStats 591f>≡ (605b)
    struct TaslockStats
    {
        ulong locks;
        ulong glare;
        ulong inglare;
    };
```


<lock() start of lock, stat 592a>≡ (58d)

```
lockstats.locks++;
```

<lock() when lock already held, stat 592b>≡ (58d)

```
lockstats.glare++;
```

<lock() in loop when lock already held, stat 592c>≡ (58d)

```
lockstats.inglare++;
```

<global lockstats 592d>≡ (605b)

```
struct TaslockStats lockstats;
```

B.3.3 QlockStats

<struct QlockStats 592e>≡ (606a)

```
struct QlockStats {
    ulong rlock;
    ulong rlockq;
    ulong wlock;
    ulong wlockq;
    ulong qlock;
    ulong qlockq;
};
```

<global rwstats 592f>≡ (606a)

```
struct QlockStats rwstats;
```

<qlock() start of qlock, stat 592g>≡ (63e)

```
rwstats.qlock++;
```

<qlock() when qlock already held, stat 592h>≡ (63e)

```
rwstats.qlockq++;
```

B.3.4 Kimage statistics

<struct Irstats 592i>≡ (665)

```
struct Irstats {
    int calls;           /* times imagereclaim was called */
    int loops;          /* times the main loop was run */
    uvlong ticks;       /* total time in the main loop */
    uvlong maxt;        /* longest time in main loop */
};
```

<segment.c global irstats 592j>≡ (665)

```
static struct Irstats irstats;
```

B.3.5 Scheduling

<proc.c statistics 592k>≡ (675c)

```
long preempts;
long delayedscheds; /* statistics */
long skipscheds;
```

B.3.6 Clock

<clock.c statistics 592l>≡ (673a)

```
ulong intrcount[MAXCPUS];
ulong fcallcount[MAXCPUS];
```

$\langle \text{timerintr}() \text{ start, stat 593a} \rangle \equiv$ (220d)
intrcount[cpu->cpuno]++;

$\langle \text{timerintr}() \text{ in timers loop, stat 593b} \rangle \equiv$ (220d)
fcallcount[cpu->cpuno]++;

Appendix C

Error Management

This appendix covers the kernel's error handling mechanisms: `panic()` for unrecoverable errors that halt the system, `error()` for recoverable errors that unwind the call stack (using `setjmp/longjmp`-style labels), and `waserror()/poperror()` which together form a try/catch mechanism in C. The `errstr` syscall lets user programs retrieve error strings—Plan 9's replacement for UNIX's numeric `errno`.

C.1 `panic()`

<global panicking 594a>≡ (607)
`bool panicking;`

<function panic 594b>≡ (607)
`void
devcons_panic(char *fmt, ...)
{
 int n, s;
 va_list arg;
 char buf[PRINTSIZE];

 <panic() reset kprintoq 583e>

 if(panicking)
 for(;;);
 panicking = true;

 s = arch_splhi();
 strcpy(buf, "panic: ");
 va_start(arg, fmt);
 n = vseprint(buf+strlen(buf), buf+sizeof(buf), fmt, arg) - buf;
 va_end(arg);
 iprint("%s\n", buf);
 <panic() run consdebug hook 583g>
 arch_splx(s);
 prflush();
 buf[n] = '\n';
 putstrn(buf, n+1);
 //TODO: put in comment for now because already got some panic with
 // lapic, but it seems to work still :)
 //arch_dumpstack();
 //exit(1);
}`

<function prflush 594c>≡ (607)
`void`

```

prflush(void)
{
    ulong now;

    now = cpu->ticks;
    while(consactive())
        if(cpu->ticks - now >= Arch_HZ)
            break;
}

⟨function sysfatal 595a⟩≡ (607)
/* libmp at least contains a few calls to sysfatal; simulate with panic */
// note that this is not a system call, even though it's prefixed with sys
//@Scheck: no dead, override also sysfatal from libc/9sys/sysfatal.c
void sysfatal(char *fmt, ...)
{
    char err[256];
    va_list arg;

    va_start(arg, fmt);
    vseprint(err, err + sizeof err, fmt, arg);
    va_end(arg);
    panic("sysfatal: %s", err);
}

⟨function _assert 595b⟩≡ (607)
void
devcons__assert(char *fmt)
{
    panic("assert failed at %#p: %s", getcallerpc(&fmt), fmt);
}

```

C.2 error()

```

⟨global Exxx errors 595c⟩≡ (610a)
char Emount[] = "inconsistent mount";
char Eunmount[] = "not mounted";
char Eismtpt[] = "is a mount point";
char Eunion[] = "not in union";
char Emountrpc[] = "mount rpc error";
char Eshutdown[] = "device shut down";
char Enocreate[] = "mounted directory forbids creation";
char Enonexist[] = "file does not exist";
char Eexist[] = "file already exists";
char Ebadsharp[] = "unknown device in # filename";
char Enotdir[] = "not a directory";
char Eisdir[] = "file is a directory";
char Ebadchar[] = "bad character in file name";
char Efilename[] = "file name syntax";
char Eperm[] = "permission denied";
char Ebadusefd[] = "inappropriate use of fd";
char Ebadarg[] = "bad arg in system call";
char Einuse[] = "device or object already in use";
char Eio[] = "i/o error";
char Etoobig[] = "read or write too large";
char Etoosmall[] = "read or write too small";
char Ehungup[] = "i/o on hungup channel";
char Ebadctl[] = "bad process or channel control request";

```

```

char Enodev[] = "no free devices";
char Eprocdied[] = "process exited";
char Enochild[] = "no living children";
char Eioload[] = "i/o error in demand load";
char Enovmem[] = "virtual memory allocation failed";
char Ebadfd[] = "fd out of range or not open";
char Enofd[] = "no free file descriptors";
char Eisstream[] = "seek on a stream";
char Ebadexec[] = "exec header invalid";
char Etimeout[] = "connection timed out";
char Econrefused[] = "connection refused";
char Econinuse[] = "connection in use";
char Eintr[] = "interrupted";
char Enomem[] = "kernel allocate failed";
char Esoverlap[] = "segments overlap";
char Eshort[] = "i/o count too small";
char Egreg[] = "jmk added reentrancy for threads";
char Ebadspec[] = "bad attach specifier";
char Enoreg[] = "process has no saved registers";
char Enoattach[] = "mount/attach disallowed";
char Eshortstat[] = "stat buffer too small";
char Ebadstat[] = "malformed stat buffer";
char Enegoff[] = "negative i/o offset";
char Ecmdargs[] = "wrong #args in control message";
char Ebadip[] = "bad ip address syntax";
char Edirseek[] = "seek in directory";
char Echange[] = "media or partition has changed";
char Edetach[] = "device is detached";
char Enotconf[] = "endpoint not configured";
char Estalled[] = "endpoint stalled";
char Esbadstat[] = "invalid directory entry received from server";
char Enoversion[] = "version not established for mount channel";

```

```

⟨global Edoesnotexist 596a⟩≡ (647)
    static char Edoesnotexist[] = "does not exist";

```

```

⟨Proc error managment fields 596b⟩≡ (49f) 597e▷
    // array<Label>, error labels, poor's man exceptions in C
    Label errlab[NERR];
    // length(errlab) used.
    int nerrlab;

    // ref<string> point to errbuf0 or to syserrstr (which points to errbuf1)
    char *errstr; /* reason we're unwinding the error stack, errbuf1 or 0 */
    char errbuf0[ERRMAX];
    char errbuf1[ERRMAX];

```

```

⟨newproc() set fields of p 596c⟩+≡ (108c) ◁552b
    p->nerrlab = 0;
    p->errstr = p->errbuf0;
    p->syserrstr = p->errbuf1;
    p->errbuf0[0] = '\0';
    p->errbuf1[0] = '\0';

```

```

⟨constant NERR 596d⟩≡ (668c)
    NERR = 64,

```

```

⟨function error 597a⟩≡ (675c)
void
proc_error(char *err)
{
    arch_spllo();

    assert(up->nerrlab < NERR);
    kstrcpy(up->errstr, err, ERRMAX);
    arch_setlabel(&up->errlab[NERR-1]);
    nexterror();
}

```

```

⟨function exhausted 597b⟩≡ (675c)
void
exhausted(char *resource)
{
    char buf[ERRMAX];

    snprintf(buf, sizeof buf, "no free %s", resource);
    iprint("%s\n", buf);
    error(buf);
}

```

C.3 waserror(), nexterror(), poperror()

This is Plan 9’s exception mechanism in C, built on `arch_setlabel/arch_gotolabel` (the same primitives used by the scheduler). `waserror()` pushes a label onto a per-process stack and returns false. If `error()` or `nexterror()` is later called, it pops the stack and jumps back to the `waserror()` call site, which now returns true—entering the “catch” branch. `poperror()` simply decrements the stack when the protected code completes normally.

```

⟨macro waserror poperror 597c⟩≡ (668c)
// poor’s man exceptions in C
// - waserror() =~ try
//   * if (!waserror()) { } else { } <=> try { } catch { }
//   * if (waserror()) { } <=> finally { }
// - poperror() = nothing
// - error() =~ raise
// - nexterror() =~ re raise from exn handler
// note, arch_setlabel() return false, so the branch is never taken first
// but nexterror() is using arch_gotolabel() which returns true, see l_switch.s
#define waserror() (up->nerrlab++, arch_setlabel(&up->errlab[up->nerrlab-1]))
#define poperror() up->nerrlab--

```

```

⟨function nexterror 597d⟩≡ (675c)
// raise an exception
void
proc_nexterror(void)
{
    arch_gotolabel(&up->errlab[--up->nerrlab]);
}

```

C.4 syserrstr()

```

⟨Proc error managment fields 597e⟩+≡ (49f) <596b
char *syserrstr; /* last error from a system call, errbuf0 or 1 */

```

<syscall errstr 598a>≡ (679a)

```
// int errstr(char *err, uint nerr);
long
syserrstr(ulong* arg)
{
    return generrstr((char*)arg[0], arg[1]);
}
```

<function generrstr 598b>≡ (679a)

```
static long
generrstr(char *buf, uint nbuf)
{
    char tmp[ERRMAX];

    if(nbuf == 0)
        error(Ebadarg);
    validaddr((ulong)buf, nbuf, true);
    if(nbuf > sizeof tmp)
        nbuf = sizeof tmp;
    memmove(tmp, buf, nbuf);

    /* make sure it's NUL-terminated */
    tmp[nbuf-1] = '\0';
    memmove(buf, up->syserrstr, nbuf);
    buf[nbuf-1] = '\0';
    memmove(up->syserrstr, tmp, nbuf);
    return 0;
}
```

<function werrstr 598c>≡ (679a)

```
/*@Scheck: this is also defined in libc, so it's supposed to override it? TODO
void werrstr(char *fmt, ...)
{
    va_list va;

    if(up == nil)
        return;

    va_start(va, fmt);
    vseprint(up->syserrstr, up->syserrstr+ERRMAX, fmt, va);
    va_end(va);
}
```

Appendix D

Utilities

D.1 Memory areas operations

```
<lib.h mem functions decl 599a>≡ (656e)
extern void* memccpy(void*, void*, int, ulong);
extern void* memset(void*, int, ulong);
extern int memcmp(void*, void*, ulong);
extern void* memmove(void*, void*, ulong);
extern void* memchr(void*, int, ulong);
```

```
<function memrchr 599b>≡ (647)
void*
memrchr(void *va, int c, long n)
{
    byte *a, *e;

    a = va;
    for(e=a+n-1; e>a; e--)
        if(*e == c)
            return e;
    return nil;
}
```

D.2 String functions

```
<lib.h string functions decl 599c>≡ (656e)
extern char* strchr(char*, int);
extern char* strrchr(char*, int);
extern int strcmp(char*, char*);
extern char* strcpy(char*, char*);
extern char* strecpy(char*, char*, char*);
extern char* strncpy(char*, char*, long);
extern int strncmp(char*, char*, long);
extern long strlen(char*);
extern char* strstr(char*, char*);
extern int atoi(char*);
extern int fullrune(char*, int);
```

```
<function kstrcpy 599d>≡ (661c)
/*
 * Rather than strncpy, which zeros the rest of the buffer, kstrcpy
 * truncates if necessary, always zero terminates, does not zero fill,
 * and puts ... at the end of the string if it's too long. Usually used to
```



```

    * save a string in up->genbuf;
    */
void
kstrncpy(char *s, char *t, int ns)
{
    int nt;

    nt = strlen(t);
    if(nt+1 <= ns){
        memmove(s, t, nt+1);
        return;
    }
    /* too long */
    if(ns < 4){
        /* but very short! */
        strncpy(s, t, ns);
        return;
    }
    /* truncate with ... at character boundary (very rare case) */
    memmove(s, t, ns-4);
    ns -= 4;
    s[ns] = '\0';
    /* look for first byte of UTF-8 sequence by skipping continuation bytes */
    while(ns>0 && (s[--ns]&0xC0)==0x80)
        ;
    strcpy(s+ns, "...");
}

```

⟨function kstrdup 600a⟩≡ (661c)

```

/*
 * Atomically replace *p with copy of s
 */
void
kstrdup(char **p, char *s)
{
    int n;
    char *t, *prev;

    n = strlen(s)+1;
    /* if it's a user, we can wait for memory; if not, something's very wrong */
    if(up){
        t = smalloc(n);
        setmalloctag(t, getcallerpc(&p));
    }else{
        t = malloc(n);
        if(t == nil)
            panic("kstrdup: no memory");
    }
    memmove(t, s, n);
    prev = *p;
    *p = t;
    free(prev);
}

```

⟨function emptystr 600b⟩≡ (647)

```

bool
emptystr(char *s)
{
    if(s == nil)
        return true;
}

```

```

    if(s[0] == '\0')
        return true;
    return false;
}

```

D.3 Conversion functions

<lib.h strt functions decl 601a>*≡ (656e)

```

extern long  strtol(char*, char**, int);
extern ulong strtoul(char*, char**, int);
extern vlong strtoll(char*, char**, int);
extern uulong strtoull(char*, char**, int);

```

<function readnum 601b>≡ (611a)

```

int
readnum(ulong off, char *buf, ulong n, ulong val, int size)
{
    char tmp[64];

    snprintf(tmp, sizeof(tmp), "%*lud", size-1, val);
    tmp[size-1] = ' ';
    if(off >= size)
        return 0;
    if(off+n > size)
        n = size-off;
    memmove(buf, tmp+off, n);
    return n;
}

```

<function readstr 601c>≡ (611a)

```

int
readstr(ulong off, char *buf, ulong n, char *str)
{
    int size;

    size = strlen(str);
    if(off >= size)
        return 0;
    if(off+n > size)
        n = size-off;
    memmove(buf, str+off, n);
    return n;
}

```

D.4 Pool allocation

D.5 Printf and Fmt

<lib.h printf functions decl 601d>≡ (656e)

```

extern char* seprint(char*, char*, char*, ...);
extern char* vseprint(char*, char*, char*, va_list);
extern int snprintf(char*, int, char*, ...);
extern int sprintf(char*, char*, ...);

```

```

⟨struct Fmt 602a⟩≡ (656e)
struct Fmt{
    byte runes;        /* output buffer is runes or chars? */
    void *start;        /* of buffer */
    void *to;           /* current place in the buffer */
    void *stop;         /* end of the buffer; overwritten if flush fails */
    int (*flush)(Fmt *); /* called when to == stop */
    void *farg;         /* to make flush a closure */
    int nfmt;           /* num chars formatted so far */
    va_list args;       /* args passed to dofmt */
    int r;              /* % format Rune */
    int width;
    int prec;
    ulong flags;
};

```

```

⟨lib.h fmt functions decl 602b⟩≡ (656e)
extern int fmtstrinit(Fmt*);
extern int fmtinstall(int, int (*)(Fmt*));
extern void quotefmtinstall(void);
extern int fmtprint(Fmt*, char*, ...);
extern int fmtstrcpy(Fmt*, char*);
extern char* fmtstrflush(Fmt*);

```

```

⟨lib.h pragmas 602c⟩≡ (656e)
#pragma varargck argpos fmtprint 2
#pragma varargck argpos print 1
#pragma varargck argpos seprint 3
#pragma varargck argpos snprintf 3
#pragma varargck argpos sprint 2

#pragma varargck type "lld" vlong
#pragma varargck type "llx" vlong
#pragma varargck type "lld" uulong
#pragma varargck type "llx" uulong
#pragma varargck type "ld" long
#pragma varargck type "lx" long
#pragma varargck type "ld" ulong
#pragma varargck type "lx" ulong
#pragma varargck type "d" int
#pragma varargck type "x" int
#pragma varargck type "c" int
#pragma varargck type "C" int
#pragma varargck type "d" uint
#pragma varargck type "x" uint
#pragma varargck type "c" uint
#pragma varargck type "C" uint
#pragma varargck type "s" char*
#pragma varargck type "q" char*
#pragma varargck type "S" Rune*
#pragma varargck type "%" void
#pragma varargck type "p" uintptr
#pragma varargck type "p" void*
#pragma varargck flag ', '

```

D.6 Runes, a.k.a unicode

```

⟨enum utf 602d⟩≡ (656e)

```

```

enum
{
    UTFmax    = 4,    /* maximum bytes per rune */
    Runeself   = 0x80, /* rune and UTF sequences are the same (<) */
};

<lib.h rune functions decl 603a>≡ (656e)
extern int runetochar(char*, Rune*);
extern int chartorune(Rune*, char*);
extern char* utfrune(char*, long);
extern int utfnlen(char*, long);

<Kbscan other fields 603b>+≡ (330c) <348a
    bool collecting;
    int nk;
    Rune    kc[5];

<kbdputc() if collecting 603c>≡ (331b)
    if(kbscan->collecting){
        int i;
        // pad's additional overflow checking, just to make sure
        if(kbscan->nk >= nelem(kbscan->kc)) {
            kbscan->nk = 0;
            kbscan->collecting = false;
            print("collecting overflow, possible bug in latin1()\n");
        }
        kbscan->kc[kbscan->nk++] = c;
        c = latin1(kbscan->kc, kbscan->nk);
        if(c < -1) /* need more keystrokes */
            return;
        if(c != -1) /* valid sequence */
            kbdputc(c);
        else /* dump characters */
            for(i=0; i<kbscan->nk; i++)
                kbdputc(kbscan->kc[i]);
        kbscan->nk = 0;
        kbscan->collecting = false;
    }

<kbdputc() start collecting 603d>≡ (331b)
    /*
    * VMware and Qemu use Ctl-Alt as the key combination
    * to make the VM give up keyboard and mouse focus.
    * This has the unfortunate side effect that when you
    * come back into focus, Plan 9 thinks you want to type
    * a compose sequence (you just typed alt).
    *
    * As a clumsy hack around this, we look for ctl-alt
    * and don't treat it as the start of a compose sequence.
    */
    if(!kbscan->ctl){
        kbscan->collecting = true;
        kbscan->nk = 0;
    }

<latin1.h 603e>≡

```

Appendix E

Extra Code

E.1 arch/

E.2 buses/

E.3 concurrency/

E.3.1 concurrency/portdat_concurrency.h

`<portdat_concurrency.h 604>`≡

```
//*****  
// Mutual exclusion  
//*****  
  
// arch_tas() < Lock|ILock < QLock|RWLock  
  
// This used to be in <arch>/, but its fields are used from port/, so the  
// code must be portable!  
<struct Lock 58c>  
  
<struct ILock 61a>  
  
<struct QLock 62f>  
  
<struct RWlock 65a>  
  
//*****  
// Atomicity  
//*****  
  
<struct Ref 68b>  
  
<struct Counter 69a>  
  
//*****  
// Synchronization  
//*****  
  
// defined in this directory but no functions are operating on it in this dir  
<struct Rendez 203d>  
  
<struct Sema 416>
```

```
// see also Waitq in portdat_processes.h?
```

E.3.2 concurrency/portfns_concurrency.h

```
<portfns_concurrency.h 605a>≡
// taslock.c
int lock(Lock*);
void unlock(Lock*);
void ilock(Lock*);
void iunlock(Lock*);
int canlock(Lock*);

// ref.c
long incref(Ref*);
long decref(Ref*);

// qlock.c
void qlock(QLock*);
void qunlock(QLock*);
int canqlock(QLock*);

void rlock(RWlock*);
void runlock(RWlock*);
int canrlock(RWlock*);
void wlock(RWlock*);
void wunlock(RWlock*);

// in <arch>/concurrency.s (but used in port)
/*@Scheck: Assembly
int arch_splhi(void);
/*@Scheck: Assembly
int arch_spllo(void);
/*@Scheck: Assembly
void arch_splx(int);
/*@Scheck: Assembly
bool arch_islo(void);

//test-and-set
/*@Scheck: Assembly
int arch_tas(void*);
/*@Scheck: Assembly
void arch_xinc(long*);
/*@Scheck: Assembly
long arch_xdec(long*);
// <arch>/??? (called from port but signature not portable across <arch>)
//int arch_cmpswap(long*, long, long);

//void arch_coherence(void); // now in core/ for backward deps
```

E.3.3 concurrency/taslock.c

```
<taslock.c 605b>≡
// TAS: Test And Set
<kernel basic includes 35a>

<struct TaslockStats 591f>
```

<global lockstats 592d>
 <globals lockcycles 591e>

 <function incCNT 70c>
 <function decCNT 70d>
 <function lockloop 577c>
 <function lock 58d>
 <function ilock 61d>
 <function canlock 60e>
 <function unlock 60c>
 <function iunlock 62a>

E.3.4 concurrency/qlock.c

<qlock.c 606a>≡
 <kernel basic includes 35a>

 <struct QlockStats 592e>
 <global rwstats 592f>

 <function qlock 63e>
 <function canqlock 64e>
 <function qunlock 64c>
 <function rlock 65d>
 <function runlock 66b>
 <function wlock 66d>
 <function wunlock 67b>
 <function canrlock 68a>

E.3.5 concurrency/ref.c

<ref.c 606b>≡
 <kernel basic includes 35a>

 <function incref 68c>
 <function decref 68d>

E.4 console/

E.4.1 console/portdat_console.h

E.4.2 console/portfns_console.h

E.4.3 console/devcons.c

```
<devcons.c 607>≡  
  <kernel basic includes 35a>  
  
#include    <pool.h>  
  
// used also by edf.c  
<global panicking 594a>  
  
<hook screenputs 336c>  
<hook consdebug 583f>  
  
<global kbdq 340b>  
<global lineq 340d>  
<global serialoq 584b>  
<global kprintoq 582g>  
  
<global kprintinuse 582h>  
<global iprintscreenputs 343b>  
  
<struct ConsKbd 334b>  
  
<global kbd 334c>  
  
<global fasthz 231d>  
  
<devcons.c forward decl ??>  
  
<function kbdqinit 340c>  
  
<function lineqinit 340e>  
  
<function consactive 584c>  
  
<function prflush 594c>  
  
<struct KMsg 581a>  
  
<global kmsg 581b>  
  
<function kmsgputs 581f>  
  
<function putstrn0 342a>  
  
<function putstrn 343a>  
  
<function print 342c>  
  
<global iprintlock 343d>  
  
<function iprintcanlock 344a>
```



```

⟨function iprint 343c⟩

⟨function panic 594b⟩

⟨function sysfatal 595a⟩

⟨function _assert 595b⟩

⟨function pprint 344b⟩

⟨function echoscreen 336d⟩

⟨function echoserialoq 584e⟩

⟨function echo 336a⟩

⟨function kbdcr2nl ??⟩

⟨function kbdputc 334d⟩

⟨clock callback kbdputclock 335a⟩

⟨devcons.c enum Qxxx 339c⟩

enum
{
    ⟨constant VLNUMSIZE 228b⟩
};

⟨global consdir 340a⟩

⟨method consinit 344c⟩

static Chan*
consattach(char *spec)
{
    return devattach('c', spec);
}

static Walkqid*
conswalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, consdir, nelem(consdir), devgen);
}

static int
consstat(Chan *c, uchar *dp, int n)
{
    return devstat(c, dp, n, consdir, nelem(consdir), devgen);
}

⟨method consopen 345a⟩

⟨method consclose 345b⟩

⟨method consread 345c⟩

⟨method conswrite 346a⟩

⟨global consdevtab 346c⟩

```

<global randn 568a>
 <function seedrand 568b>
 <function nrnd 568c>
 <global uvorder 234a>
 <function le2vlong 234b>
 <function vlong2le 234c>
 <global order 234d>
 <function le2long 234e>
 <devcons.c Exxx errors 228a>
 <function readtime 231f>
 <function writetime 232a>
 <function readbintime 232d>
 <function writebintime 233>

E.5 core/

E.5.1 core/portdat_forward.h

E.5.2 core/portdat_core.h

```

<portdat_core.h 609>≡

// All those structs used to be in <arch>/dat.h, but many of their fields
// were used from port/ so I've moved them here (and put the arch-specific
// fields in dat_core.h)

//*****
// Conf
//*****

<struct Confmem 46c>

<struct Conf 46a>

extern Conf conf;

<constant MAXCONF 47b>
// hash<string, string>
extern char *confname[];
extern char *confval[];
// Hashtbl.length(confname)
extern int nconf;

extern bool cpuserver; // defined in $CONF.c
  
```

```

char* getconf(char *name);

//*****
// Cpu
//*****

<struct Label 197d>

<struct Perf 587b>

<struct Cpu 48a>

// array<ref<Cpu>>, MAXCPUS is defined in <arch>/mem.h
extern Cpu* cpus[MAXCPUS];
<macro CPUS 49c>

// 'up' and 'cpu' can have different implement in different arch
// so they are not defined here.

//*****
// Other
//*****

extern char* eve;
int iseve(void);
// accessed by /dev/hostdomain, defined in auth.c
extern char hostdomain[];

// defined in ???
extern ulong kerndate;

```

E.5.3 core/portfns_core.h

E.5.4 core/portdat.h

E.5.5 core/error.h

E.5.6 core/error.c

```

<error.c 610a>≡
<global Exxx errors 595c>

```

E.5.7 core/portdat.c

```

<portdat.c 610b>≡
<kernel basic includes 35a>

```

```

// was in init/main.c
<global conf 46b>
// bool cpuserver; // defined in $CONF.c

<globals confname and confvar 47a>
<global nconf 47c>

//TODO: to factorize code between pc and pi

```

```

//< <function [[getconf]]> >

// was in dat.h
<global cpus 49a>
// cpu is in init/main.c because it can differ in different archi

// was in security/auth.c
<global eve 527a>
// should be in portfns.c, but then backward deps
/*
 * return true if current user is eve
 */
<function iseve 527b>

```

E.5.8 core/portfns.c

```

<portfns.c 611a>≡
<kernel basic includes 35a>

<portfns.c backward deps breaker ??>

<function returnfalse 84b>

// was in devcons.c, could be in lib/misc.c
<function readnum 601b>

<function readstr 601c>

```

E.6 devices/

E.6.1 devices/portdat_devices.h

```

<struct Devport 611b>≡ (611d)
    struct Devport {
        ulong port;
        int size;
    };

<struct DevConf 611c>≡ (611d)
    struct DevConf
    {
        ulong intnum;    /* interrupt number */
        char *type;      /* card type, malloced */
        int nports;      /* Number of ports */
        Devport *ports;  /* The ports themselves */
    };

<portdat_devices.h 611d>≡
/*
 * hardware info about a device
 */
<struct Devport 611b>

<struct DevConf 611c>

// keyboard

```

```

<enum kbscan 330d>

<struct Kbscan 330c>

// keyboard/portkbd.c
extern Kbscan kbscans[KbNscans];

// mouse

/*
 * mouse types
 */
enum MouseType
{
    Mouseother= 0,
    Mouseserial= 1,
    MousePS2= 2,
};

// mouse/devmouse.c (used in portmouse.c, portkbd.c, ...)
extern int mouseshifted;

// mouse/portmouse.c (used in <arch>/mouse.c)
extern int mousetype;
extern int packetsize;
extern bool mousehwaccel;
extern bool intellimouse;

/* devdraw.c */ // used in some <arch>/screen.c
extern QLock drawlock;
// for gscreen, gscreendata, see portscreen.h

```

E.6.2 devices/portfns_devices.h

E.6.3 devices/keyboard/portkbd.c

```

<portkbd.c 612>≡
/*
 * keyboard input // the portable part
 */
<kernel basic includes 35a>

// The main entry point in this file is kbdputc() called
// from devkbin.c when a process (e.g., usbd) writes into /dev/kbin.

// The functions kbdputmap/kbdgetmap are also portable functions called
// from devkbmap.c

<global kdebug 573a>

<enum specialkey 328>

<global kbtabs 329a>

<global kbtabsshift 329b>

<global kbtabsesc1 333b>

```

```

<global kbtaltgr 330b>

// see kbtabs.c for the ctrl one, put in another file because
// of issues with TeX with the special characters it contain
// can't LPize it.
<global kbtabsctrl decl 330a>

<global kbescans 331a>

// called when write in /dev/kbin (see devkbin.c)
<function kbdputc 331b>

// called when interact with /dev/kbmap (see devkbmap.c)
<function kbdputmap 337b>

<function kbdgetmap 337a>

```

E.6.4 devices/keyboard/latin1.c

```

<latin1.c 613>≡
<kernel basic includes 35a>

/*
 * The code makes two assumptions: strlen(ld) is 1 or 2; latintab[i].ld can be a
 * prefix of latintab[j].ld only when j<i.
 */
struct cvlist
{
    char    *ld;        /* must be seen before using this conversion */
    char    *si;        /* options for last input characters */

    Rune    *so;        /* the corresponding Rune for each si entry */
};

struct cvlist latintab[] = {
#include "../port/latin1.h"
    0, 0, 0
};

/*
 * Given n characters k[0]..k[n-1], find the rune or return -1 for failure.
 */
long
unicode(Rune *k, int n)
{
    long c;
    Rune *r;

    c = 0;
    for(r = &k[1]; r<&k[n]; r++){          /* +1 to skip [Xx] */
        c <= 4;
        if('0'<=*r && *r<='9')
            c += *r-'0';
        else if('a'<=*r && *r<='f')
            c += 10 + *r-'a';
        else if('A'<=*r && *r<='F')
            c += 10 + *r-'A';
    }
}

```

```

        else
            return -1;
    }
    return c;
}

/*
 * Given n characters k[0]..k[n-1], find the corresponding rune or return -1 for
 * failure, or something < -1 if n is too small. In the latter case, the result
 * is minus the required n.
 */
long
latin1(Rune* k, int n)
{
    struct cvlist *l;
    char c;
    char* p;

    //    print("latin1: %d, %d\n", n, k[n-1]);

    if(k[0] == 'X')
        if(n>=5)
            return unicode(k, 5);
        else
            return -5;
    if(k[0] == 'x')
        if(n>=UTFmax*2+1)
            return unicode(k, UTFmax*2+1);
        else
            return -(UTFmax+1);

    for(l=latintab; l->ld!=0; l++)
        if(k[0] == l->ld[0]){
            //            print("found: %s, %s\n", l->ld, l->si);
            if(n == 1)
                return -2;
            if(l->ld[1] == 0)
                c = k[1];
            else if(l->ld[1] != k[1])
                continue;
            else if(n == 2)
                return -3;
            else
                c = k[2];
            for(p=l->si; *p!=0; p++)
                if(*p == c) {
                    print("found: 0x%x\n", l->so[p - l->si]);
                    return l->so[p - l->si];
                }
            return -1;
        }
    return -1;
}

```

E.6.5 devices/keyboard/latin1.h

E.6.6 TO LP SPLIT devices/keyboard/devkbin.c

⟨devkbin.c 614⟩≡

```

/*
 * keyboard scan code input from outside the kernel.
 * to avoid duplication of keyboard map processing for usb.
 */
<kernel basic includes 35a>

extern void kbdputc(byte, int);

enum {
    Qdir,
    Qkbd,
};

Dirtab kbintab[] = {
    ".", {Qdir, 0, QTDIR}, 0, 0555,
    "kbin", {Qkbd, 0}, 0, 0200,
};

Lock    kbinlck;
int kbinbusy;

static Chan *
kbinattach(char *spec)
{
    return devattach(L'I', spec);
}

static Walkqid*
kbinwalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, kbintab, nelem(kbintab), devgen);
}

static int
kbinstat(Chan *c, uchar *dp, int n)
{
    return devstat(c, dp, n, kbintab, nelem(kbintab), devgen);
}

static Chan*
kbinopen(Chan *c, int omode)
{
    if(!iseve())
        error(Eperm);
    if(c->qid.path == Qkbd){
        lock(&kbinlck);
        if(kbinbusy){
            unlock(&kbinlck);
            error(Einuse);
        }
        kbinbusy++;
        unlock(&kbinlck);
    }
    return devopen(c, omode, kbintab, nelem(kbintab), devgen);
}

static void
kbinclose(Chan *c)
{
    if(c->aux){

```



```

        free(c->aux);
        c->aux = nil;
    }
    if(c->qid.path == Qkbd)
        kbinbusy = 0;
}

static long
kbinread(Chan *c, void *a, long n, vlong )
{
    if(c->qid.type == QTDIR)
        return devdirread(c, a, n, kbintab, nelem(kbintab), devgen);
    return 0;
}

static long
kbinwrite(Chan *c, void *a, long n, vlong)
{
    int i;
    uchar *p = a;

    if(c->qid.type == QTDIR)
        error(Eisdir);
    switch((int)c->qid.path){
    case Qkbd:
        for(i = 0; i < n; i++)
            kbdputc(*p++, 1); /* external source */
        break;
    default:
        error(Egreg);
    }
    return n;
}

Dev kbindevtab = {
    .dc = L'I',
    .name = "kbin",

    .reset = devreset,
    .init = devinit,
    .shutdown = devshutdown,
    .attach = kbinattach,
    .walk = kbinwalk,
    .stat = kbinstat,
    .open = kbinopen,
    .create = devcreate,
    .close = kbinclose,
    .read = kbinread,
    .bread = devbread,
    .write = kbinwrite,
    .bwrite = devbwrite,
    .remove = devremove,
    .wstat = devwstat,
};

```

E.6.7 TO LPSPLIT devices/keyboard/devkbmap.c

⟨devkbmap.c 616⟩≡
/*

```

* keyboard map
*/
⟨kernel basic includes 35a⟩

enum{
    Qdir,
    Qdata,
};
Dirtab kbmaptab[]={
    ".",      {Qdir, 0, QTDIR},    0, 0555,
    "kbmap",   {Qdata, 0},         0, 0600,
};
#define NKBFIL  sizeof(kbmaptab)/sizeof(kbmaptab[0])

#define KBLINLEN (3*NUMSIZE+1) /* t code val\n */

static Chan *
kbmapattach(char *spec)
{
    return devattach(L'K', spec);
}

static Walkqid*
kbmapwalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, kbmaptab, NKBFIL, devgen);
}

static int
kbmapstat(Chan *c, uchar *dp, int n)
{
    return devstat(c, dp, n, kbmaptab, NKBFIL, devgen);
}

static Chan*
kbmapopen(Chan *c, int omode)
{
    if(!iseve())
        error(Eperm);
    return devopen(c, omode, kbmaptab, NKBFIL, devgen);
}

static void
kbmapclose(Chan *c)
{
    if(c->aux){
        free(c->aux);
        c->aux = nil;
    }
}

static long
kbmapread(Chan *c, void *a, long n, vlong offset)
{
    char *bp;
    char tmp[KBLINLEN+1];
    int t, sc;
    Rune r;

    if(c->qid.type == QTDIR)

```

```

    return devdirread(c, a, n, kbmaptab, NKBFIL, devgen);

switch((int)(c->qid.path)){
case Qdata:
    if(kbdgetmap(offset/KBLINLEN, &t, &sc, &r)) {
        bp = tmp;
        bp += readnum(0, bp, NUMSIZE, t, NUMSIZE);
        bp += readnum(0, bp, NUMSIZE, sc, NUMSIZE);
        bp += readnum(0, bp, NUMSIZE, r, NUMSIZE);
        *bp++ = '\n';
        *bp = 0;
        n = readstr(offset%KBLINLEN, a, n, tmp);
    } else
        n = 0;
    break;
default:
    n=0;
    break;
}
return n;
}

static long
kbmapwrite(Chan *c, void *a, long n, vlong)
{
    char line[100], *lp, *b;
    int key, m, l;
    Rune r;

    if(c->qid.type == QTDIR)
        error(Eperm);

    switch((int)(c->qid.path)){
case Qdata:
    b = a;
    l = n;
    lp = line;
    if(c->aux){
        strcpy(line, c->aux);
        lp = line+strlen(line);
        free(c->aux);
        c->aux = nil;
    }
    while(--l >= 0) {
        *lp++ = *b++;
        if(lp[-1] == '\n' || lp == &line[sizeof(line)-1]) {
            *lp = 0;
            if(*line == 0)
                error(Ebadarg);
            if(*line == '\n' || *line == '#'){
                lp = line;
                continue;
            }
            lp = line;
            while(*lp == ' ' || *lp == '\t')
                lp++;
            m = strtoul(line, &lp, 0);
            key = strtoul(lp, &lp, 0);
            while(*lp == ' ' || *lp == '\t')
                lp++;

```

```

        r = 0;
        if(*lp == '\\' && lp[1])
            chartorune(&r, lp+1);
        else if(*lp == '^' && lp[1]){
            chartorune(&r, lp+1);
            if(0x40 <= r && r < 0x60)
                r -= 0x40;
            else
                error(Ebadarg);
        }else if(*lp == 'M' && ('1' <= lp[1] && lp[1] <= '5'))
            r = 0xF900+lp[1]-'0';
        else if(*lp>='0' && *lp<='9') /* includes 0x... */
            r = strtoul(lp, &lp, 0);
        else
            error(Ebadarg);
        kbdputmap(m, key, r);
        lp = line;
    }
}
if(lp != line){
    l = lp-line;
    c->aux = lp = smalloc(l+1);
    memmove(lp, line, l);
    lp[l] = 0;
}
break;
default:
    error(Ebadusefd);
}
return n;
}

```

```

Dev kmapdevtab = {
    .dc = L'k',
    .name = "kmap",

    .reset = devreset,
    .init = devinit,
    .shutdown = devshutdown,
    .attach = kmapattach,
    .walk = kmapwalk,
    .stat = kmapstat,
    .open = kmapopen,
    .create = devcreate,
    .close = kmapclose,
    .read = kmapread,
    .bread = devbread,
    .write = kmapwrite,
    .bwrite = devbwrite,
    .remove = devremove,
    .wstat = devwstat,
};

```

E.6.8 TO LPSPLIT devices/mouse/portmouse.c

<portmouse.c 619>≡
 <kernel basic includes 35a>

```

// enum<MouseEvent>
int mousetype;

int packetsize;
static int resolution;
static int accelerated;

bool mousehwaccel;
bool intellimouse;

static QLock mousectlqlock;

// for serial mouse
//static char mouseport[5];

enum
{
    CMaccelerated,
    CMhwaccel,
    CMintellimouse,
    CMlinear,
    CMps2,
    CMps2intellimouse,
    CMres,
    CMreset,
    CMserial,
};

static Cmdtab mousectlmsg[] =
{
    CMaccelerated    , "accelerated"    , 0 ,
    CMhwaccel        , "hwaccel"        , 2 ,
    CMintellimouse   , "intellimouse"   , 1 ,
    CMlinear         , "linear"         , 1 ,
    CMps2            , "ps2"            , 1 ,
    CMps2intellimouse , "ps2intellimouse" , 1 ,
    CMres            , "res"            , 0 ,
    CMreset          , "reset"          , 1 ,
    CMserial         , "serial"         , 0 ,
};

/*
 * set up a ps2 mouse
 */
static void
ps2mouse(void)
{
    if(mousetype == MousePS2)
        return;
    arch_ps2mouse();

    mousetype = MousePS2;
    packetsize = 3;
    mousehwaccel = 1;
}

/*
 * The PS/2 Trackpoint multiplexor on the IBM Thinkpad T23 ignores
 * acceleration commands. It is supposed to pass them on

```

```

* to the attached device, but my Logitech mouse is simply
* not behaving any differently.  For such devices, we allow
* the user to use "hwaccel off" to tell us to back off to
* software acceleration even if we're using the PS/2 port.
* (Serial mice are always software accelerated.)
* For more information on the Thinkpad multiplexor, see
* http://www.cssrv.almaden.ibm.com/trackpoint/
*/
static void
setaccelerated(int x)
{
    accelerated = x;
    arch_setaccelerated();
    // call in devmouse.c
    mouseaccelerate(x);
}

static void
setlinear(void)
{
    accelerated = 0;
    arch_setlinear();
    mouseaccelerate(0);
}

static void
setres(int n)
{
    resolution = n;
    arch_setres(n);
}

static void
setintellimouse(void)
{
    intellimouse = 1;
    packetsize = 4;
    arch_setintellimouse();
}

static void
resetmouse(void)
{
    packetsize = 3;
    arch_resetmouse();
}

void
kmousectl(Cmdbuf *cb)
{
    Cmdtab *ct;

    qlock(&mousectlqlock);
    if(waserror()){
        qunlock(&mousectlqlock);
        nexterror();
    }

    ct = lookupcmd(cb, mousectlmsg, nelem(mousectlmsg));
    switch(ct->index){

```

```

case CMaccelerated:
    setaccelerated(cb->nf == 1 ? 1: atoi(cb->f[1]));
    break;
case CMlinear:
    setlinear();
    break;
case CMps2:
    intellimouse = 0;
    ps2mouse();
    break;
case CMres:
    if(cb->nf >= 2)
        setres(atoi(cb->f[1]));
    else
        setres(1);
    break;
case CMreset:
    resetmouse();
    if(accelerated)
        setaccelerated(accelerated);
    if(resolution)
        setres(resolution);
    if(intellimouse)
        setintellimouse();
    break;
case CMhwaccel:
    if(strcmp(cb->f[1], "on")==0)
        mousehwaccel = 1;
    else if(strcmp(cb->f[1], "off")==0)
        mousehwaccel = 0;
    else
        cmderror(cb, "bad mouse control message");
    break;
case CMintellimouse:
    setintellimouse();
    break;
case CMps2intellimouse:
    ps2mouse();
    setintellimouse();
    break;
case CMserial:
    //if(mousetype == Mouseserial)
    // error(Emouseset);
    //
    //if(cb->nf > 2){
    //    if(strcmp(cb->f[2], "M") == 0)
    //        i8250mouse(cb->f[1], m3mouseputc, 0);
    //    else if(strcmp(cb->f[2], "MI") == 0)
    //        i8250mouse(cb->f[1], m5mouseputc, 0);
    //    else
    //        i8250mouse(cb->f[1], mouseputc, cb->nf == 1);
    //} else
    //    i8250mouse(cb->f[1], mouseputc, cb->nf == 1);
    //
    //mousetype = Mouseserial;
    //strncpy(mouseport, cb->f[1], sizeof(mouseport)-1);
    //packetsize = 3;
    error("serial mice not supported anymore");
    break;
}

```

```

    gunlock(&mousectlqlock);
    poperror();
}

```

E.6.9 TO LPSPLIT devices/mouse/devmouse.c

⟨devmouse.c 623⟩≡

⟨kernel basic includes 35a⟩

```

#include    <draw.h>
#include    <draw_private.h> // for BPLONG
#include    <memdraw.h>
#include    <cursor.h>

#include    "../port/portscreen.h"

enum {
    ScrollUp = 0x08,
    ScrollDown = 0x10,
    ScrollLeft = 0x20,
    ScrollRight = 0x40,
};

typedef struct Mouseinfo    Mouseinfo;
typedef struct Mousestate    Mousestate;

struct Mousestate
{
    Point    xy;        /* mouse.xy */
    int buttons;        /* mouse.buttons */
    ulong    counter;    /* increments every update */
    ulong    msec;        /* time of last event */
};

struct Mouseinfo
{
    Lock;
    Mousestate;
    int dx;
    int dy;
    bool track;        /* dx & dy updated */
    bool redraw;        /* update cursor on screen */
    ulong    lastcounter;    /* value when /dev/mouse read */
    ulong    lastresize;
    ulong    resize;
    Rendez    r;
    Ref;
    QLock;
    bool open;
    int acceleration;
    int maxacc;
    Mousestate    queue[16]; /* circular buffer of click events */
    int ri;        /* read index into queue */
    int wi;        /* write index into queue */
    uchar    qfull;    /* queue is full */
};

Mouseinfo    mouse;

```



```

Cursorinfo  cursor;
Cursor      curs;

// set to mousefromkbd here in mouseinit(), but used outside devmouse.c
// to let know devmouse.c about some keyboard events related to the mouse
void (*kbdmouse)(int);
int   kbdbuttons;

// not used in this file
bool   mouseshifted;

enum{
    Qdir,
    Qcursor,
    Qmouse,
    Qmousein,
    Qmousectl,
};

static Dirtab mousedir[]={
    ".",      {Qdir, 0, QTDIR},    0,      DMDIR|0555,
    "cursor",  {Qcursor},    0,      0666,
    "mouse",   {Qmouse},     0,      0666,
    "mousein", {Qmousein},  0,      0220,
    "mousectl", {Qmousectl}, 0,      0220,
};

enum
{
    CMbuttonmap,
    CMscrollswap,
    CMswap,
    CMwildcard,
};

static Cmdtab mousectlmsg[] =
{
    CMbuttonmap,    "buttonmap",    0,
    CMscrollswap,   "scrollswap",   0,
    CMswap,         "swap",         1,
    CMwildcard,     "*",            0,
};

// for CMwildcard, devmouse.c relies on arch defined kmousectl()

static uchar buttonmap[8] = {
    0, 1, 2, 3, 4, 5, 6, 7,
};

static bool mouseswap;
static bool scrollswap;
static ulong mousetime;

void Cursortocursor(Cursor*);

```

```

int mousechanged(void*);
static void mouseclock(void);

static void
mouserestart(void)
{
    curs = arch_arrow;
    Cursortocursor(&arch_arrow);
    /* redraw cursor about 30 times per second */
    addclock0link(mouseclock, 33);
}

static void
mousefromkbd(int buttons)
{
    kbdbuttons = buttons;
    mousetrack(0, 0, 0, TK2MS(CPUS(0)->ticks));
}

static int
mousedevgen(Chan *c, char *name, Dirtab *tab, int ntab, int i, DirEntry *dp)
{
    int rc;

    rc = devgen(c, name, tab, ntab, i, dp);
    if(rc != -1)
        dp->atime = mousetime;
    return rc;
}

static void
mouseinit(void)
{
    curs = arch_arrow;
    Cursortocursor(&arch_arrow);
    arch_cursoron(true);
    kbdmouse = mousefromkbd;
    mousetime = seconds();
}

static Chan*
mouseattach(char *spec)
{
    return devattach('m', spec);
}

static Walkqid*
mousewalk(Chan *c, Chan *nc, char **name, int nname)
{
    Walkqid *wq;

    /*
     * We use devgen() and not mousedevgen() here
     * see "Ugly problem" in dev.c/devwalk()
     */
    wq = devwalk(c, nc, name, nname, mousedir, nelelem(mousedir), devgen);
    if(wq != nil && wq->clone != c && wq->clone != nil && (wq->clone->qid.type&QTDIR)==0)

```

```

        incref(&mouse);
    return wq;
}

static int
mousestat(Chan *c, uchar *db, int n)
{
    return devstat(c, db, n, mousedir, nelem(mousedir), mousedevgen);
}

static Chan*
mouseopen(Chan *c, int omode)
{
    switch((ulong)c->qid.path){
    case Qdir:
        if(omode != OREAD)
            error(Eperm);
        break;
    case Qmouse:
        lock(&mouse);
        if(mouse.open){
            unlock(&mouse);
            error(Einuse);
        }
        mouse.open = true;
        mouse.ref++;
        mouse.lastresize = mouse.resize;
        unlock(&mouse);
        break;
    case Qmousein:
        if(!iseve())
            error(Eperm);
        break;
    default:
        incref(&mouse);
    }
    c->mode = openmode(omode);
    c->flag |= COPEN;
    c->offset = 0;
    return c;
}

static void
mousecreate(Chan*, char*, int, ulong)
{
    error(Eperm);
}

static void
mouseclose(Chan *c)
{
    if((c->qid.type&QTDIR)==0 && (c->flag&COPEN)){
        if(c->qid.path == Qmousein)
            return;
        lock(&mouse);
        if(c->qid.path == Qmouse)
            mouse.open = false;
        if(--mouse.ref == 0){
            arch_cursoroff(true);
            curs = arch_arrow;
        }
    }
}

```

```

        Cursortocursor(&arch_arrow);
        arch_cursoron(true);
    }
    unlock(&mouse);
}
}

static long
mouseread(Chan *c, void *va, long n, vlong off)
{
    char buf[1+4*12+1];
    uchar *p;
    static int map[8] = {0, 4, 2, 6, 1, 5, 3, 7 };
    ulong offset = off;
    Mousestate m;
    int b;

    p = va;
    switch((ulong)c->qid.path){
    case Qdir:
        return devdirread(c, va, n, mousedir, nelem(mousedir), mousedevgen);

    case Qcursor:
        if(offset != 0)
            return 0;
        if(n < 2*4+2*2*16)
            error(Eshort);
        n = 2*4+2*2*16;
        lock(&cursor);
        BPLONG(p+0, curs.offset.x);
        BPLONG(p+4, curs.offset.y);
        memmove(p+8, curs.clr, 2*16);
        memmove(p+40, curs.set, 2*16);
        unlock(&cursor);
        return n;

    case Qmouse:
        while(mousechanged(nil) == 0)
            sleep(&mouse.r, mousechanged, nil);

        mouse.qfull = 0;
        mousetime = seconds();

        /*
         * No lock of the indices is necessary here, because ri is only
         * updated by us, and there is only one mouse reader
         * at a time. I suppose that more than one process
         * could try to read the fd at one time, but such behavior
         * is degenerate and already violates the calling
         * conventions for sleep above.
         */
        if(mouse.ri != mouse.wi) {
            m = mouse.queue[mouse.ri];
            if(++mouse.ri == nelem(mouse.queue))
                mouse.ri = 0;
        } else {
            while(!canlock(&cursor))
                tsleep(&up->sleepr, returnfalse, 0, TK2MS(1));

```

```

        m = mouse.Mousestate;
        unlock(&cursor);
    }

    b = buttonmap[m.buttons&7];
    /* put buttons 4 and 5 back in */
    b |= m.buttons & (3<<3);
    if (scrollswap)
        if (b == 8)
            b = 16;
        else if (b == 16)
            b = 8;
    snprintf(buf, sizeof buf, "%11d %11d %11d %11lud ",
        m.xy.x, m.xy.y,
        b,
        m.msec);
    mouse.lastcounter = m.counter;
    if(n > 1+4*12)
        n = 1+4*12;
    if(mouse.lastresize != mouse.resize){
        mouse.lastresize = mouse.resize;
        buf[0] = 'r';
    }
    memmove(va, buf, n);
    return n;
}
return 0;
}

```

```

static void
setbuttonmap(char* map)
{
    int i, x, one, two, three;

    one = two = three = 0;
    for(i = 0; i < 3; i++){
        if(map[i] == 0)
            error(Ebadarg);
        if(map[i] == '1'){
            if(one)
                error(Ebadarg);
            one = 1<<i;
        }
        else if(map[i] == '2'){
            if(two)
                error(Ebadarg);
            two = 1<<i;
        }
        else if(map[i] == '3'){
            if(three)
                error(Ebadarg);
            three = 1<<i;
        }
        else
            error(Ebadarg);
    }
    if(map[i])
        error(Ebadarg);

    memset(buttonmap, 0, 8);
}

```

```

    for(i = 0; i < 8; i++){
        x = 0;
        if(i & 1)
            x |= one;
        if(i & 2)
            x |= two;
        if(i & 4)
            x |= three;
        buttonmap[x] = i;
    }
}

static long
mousewrite(Chan *c, void *va, long n, vlong)
{
    char *p;
    Point pt;
    Cmdbuf *cb;
    Cmdtab *ct;
    char buf[64];
    int b, msec;

    p = va;
    switch((ulong)c->qid.path){
    case Qdir:
        error(Eisdir);

    case Qcursor:
        arch_cursoroff(true);
        if(n < 2*4+2*2*16){
            curs = arch_arrow;
            Cursortocursor(&arch_arrow);
        }else{
            n = 2*4+2*2*16;
            curs.offset.x = BGLONG(p+0);
            curs.offset.y = BGLONG(p+4);
            memmove(curs.clr, p+8, 2*16);
            memmove(curs.set, p+40, 2*16);
            Cursortocursor(&curs);
        }
        qlock(&mouse);
        mouse.redraw = true;
        mouseclock();
        qunlock(&mouse);
        arch_cursoron(true);
        return n;

    case Qmousectl:
        cb = parsecmd(va, n);
        if(waserror()){
            free(cb);
            nexterror();
        }

        ct = lookupcmd(cb, mousectlmsg, nelem(mousectlmsg));

        switch(ct->index){
        case CMswap:
            if(mouseswap)
                setbuttonmap("123");

```

```

        else
            setbuttonmap("321");
        mouseswap ^= true;
        break;

    case CMscrollswap:
        scrollswap ^= true;
        break;

    case CMbuttonmap:
        if(cb->nf == 1)
            setbuttonmap("123");
        else
            setbuttonmap(cb->f[1]);
        break;

    case CMwildcard:
        kmousectl(cb); // device specific hook
        break;
}

free(cb);
poperror();
return n;

case Qmousein:
    if(n > sizeof buf-1)
        n = sizeof buf -1;
    memmove(buf, va, n);
    buf[n] = 0;
    p = 0;
    pt.x = strtol(buf+1, &p, 0);
    if(p == 0)
        error(Eshort);
    pt.y = strtol(p, &p, 0);
    if(p == 0)
        error(Eshort);
    b = strtol(p, &p, 0);
    msec = strtol(p, &p, 0);
    if(msec == 0)
        msec = TK2MS(CPUS(0)->ticks);
    mousetrack(pt.x, pt.y, b, msec);
    return n;

case Qmouse:
    if(n > sizeof buf-1)
        n = sizeof buf -1;
    memmove(buf, va, n);
    buf[n] = 0;
    p = 0;
    pt.x = strtoul(buf+1, &p, 0);
    if(p == 0)
        error(Eshort);
    pt.y = strtoul(p, 0, 0);
    qlock(&mouse);
    if(ptinrect(pt, gscreen->r)){
        mouse.xy = pt;
        mouse.redraw = true;
        mouse.track = true;
        mouseclock();
    }
}

```

```

    }
    qunlock(&mouse);
    return n;
}

error(Egreg);
return -1;
}

Dev mousedevtab = {
    .dc      =    'm',
    .name     =    "mouse",

    .reset    =    mouserreset,
    .init     =    mouseinit,
    .shutdown =    devshutdown,
    .attach   =    mouseattach,
    .walk     =    mousewalk,
    .stat     =    mousestat,
    .open     =    mouseopen,
    .create   =    mousecreate,
    .close    =    mouseclose,
    .read     =    mouserread,
    .bread    =    devbread,
    .write    =    mousewrite,
    .bwrite   =    devbwrite,
    .remove   =    devremove,
    .wstat    =    devwstat,
};

void
Cursortocursor(Cursor *c)
{
    lock(&cursor);
    memmove(&cursor.Cursor, c, sizeof(Cursor));
    arch_ksetcursor(c);
    unlock(&cursor);
}

/*
 * called by the clock routine to redraw the cursor
 */
static void
mouseclock(void)
{
    if(mouse.track){
        mousetrack(mouse.dx, mouse.dy, mouse.buttons, TK2MS(CPUS(0)->ticks));
        mouse.track = false;
        mouse.dx = 0;
        mouse.dy = 0;
    }
    if(mouse.redraw && canlock(&cursor)){
        mouse.redraw = false;
        arch_cursoroff(false);
        mouse.redraw = arch_cursoron(false);
        unlock(&cursor);
    }
    drawactive(false);
}

```



```

static int
scale(int x)
{
    int sign = 1;

    if(x < 0){
        sign = -1;
        x = -x;
    }
    switch(x){
    case 0:
    case 1:
    case 2:
    case 3:
        break;
    case 4:
        x = 6 + (mouse.acceleration>>2);
        break;
    case 5:
        x = 9 + (mouse.acceleration>>1);
        break;
    default:
        x *= mouse.maxacc;
        break;
    }
    return sign*x;
}

/*
 * called at interrupt level to update the structure and
 * awaken any waiting procs.
 */
void
mousetrack(int dx, int dy, int b, int msec)
{
    int x, y, lastb;

    if(gscreen==nil)
        return;

    if(mouse.acceleration){
        dx = scale(dx);
        dy = scale(dy);
    }
    x = mouse.xy.x + dx;
    if(x < gscreen->clipr.min.x)
        x = gscreen->clipr.min.x;
    if(x >= gscreen->clipr.max.x)
        x = gscreen->clipr.max.x;
    y = mouse.xy.y + dy;
    if(y < gscreen->clipr.min.y)
        y = gscreen->clipr.min.y;
    if(y >= gscreen->clipr.max.y)
        y = gscreen->clipr.max.y;

    lastb = mouse.buttons;
    mouse.xy = Pt(x, y);
    mouse.buttons = b|kbdbuttons;
    mouse.redraw = true;
}

```

```

mouse.counter++;
mouse.msec = msec;

/*
 * if the queue fills, we discard the entire queue and don't
 * queue any more events until a reader polls the mouse.
 */
if(!mouse.qfull && lastb != b) {    /* add to ring */
    mouse.queue[mouse.wi] = mouse.Mousestate;
    if(++mouse.wi == nelem(mouse.queue))
        mouse.wi = 0;
    if(mouse.wi == mouse.ri)
        mouse.qfull = 1;
}
wakeup(&mouse.r);
drawactive(true);
}

int
mousechanged(void*)
{
    return mouse.lastcounter != mouse.counter ||
        mouse.lastresize != mouse.resize;
}

// called from outside devmouse.c
Point
mousexy(void)
{
    return mouse.xy;
}

// called from outside devmouse.c to let know devmouse.c of changes
void
mouseaccelerate(int x)
{
    mouse.acceleration = x;
    if(mouse.acceleration < 3)
        mouse.maxacc = 2;
    else
        mouse.maxacc = mouse.acceleration;
}

// called from outside devmouse.c to let know devmouse.c of changes
/*
 * notify reader that screen has been resized
 */
void
mousereresize(void)
{
    mouse.resize++;
    wakeup(&mouse.r);
}

```

E.6.10 devices/sys/devroot.c

```

<method rootclose 633>≡
/*

```

(634)

```

    * sysremove() knows this is a nop
    */
static void
rootclose(Chan*)
{
}

⟨devroot.c 634⟩≡
⟨kernel basic includes 35a⟩

//*****
// Constants and types
//*****

⟨devroot enum Qxxx 371a⟩

⟨devroot.c forward decl ??⟩
⟨struct Dirlist 371b⟩

//*****
// Root
//*****

// note: directories have by convention 0 length

⟨globals rootdir, rootdata, rootlist 371c⟩

//*****
// Boot
//*****

⟨globals bootdir, bootdata, bootlist 374a⟩

//*****
// Functions
//*****

⟨function addlist 372a⟩

⟨function addbootfile 374b⟩

⟨function addrootdir 372b⟩

⟨method rootreset 372c⟩

static Chan*
rootattach(char *spec)
{
    return devattach('/', spec);
}

⟨function rootgen 372d⟩

static Walkqid*
rootwalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, nil, 0, rootgen);
}

static int

```

```

rootstat(Chan *c, uchar *dp, int n)
{
    return devstat(c, dp, n, nil, 0, rootgen);
}

```

```

static Chan*
rootopen(Chan *c, int omode)
{
    return devopen(c, omode, nil, 0, devgen);
}

```

<method rootclose 633>

<method rootread 373>

```

static long
rootwrite(Chan*, void*, long, vlong)
{
    error(Egreg);
    return 0;
}

```

<global rootdevtab 370b>

E.6.11 devices/sys/devsys.c

```

<method syswalk 635a>≡ (636a)
static Walkqid*
syswalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, sysdir, nelem(sysdir), devgen);
}

```

```

<method sysstat 635b>≡ (636a)
static int
sysstat(Chan *c, uchar *dp, int n)
{
    return devstat(c, dp, n, sysdir, nelem(sysdir), devgen);
}

```

```

<method sysopen 635c>≡ (636a)
static Chan*
sysopen(Chan *c, int omode)
{
    c->aux = nil;
    c = devopen(c, omode, sysdir, nelem(sysdir), devgen);
    switch((ulong)c->qid.path){
    }
    return c;
}

```

```

<method sysclose 635d>≡ (636a)
static void
sysclose(Chan *c)
{
    switch((ulong)c->qid.path){
    }
}

```

```

⟨devsys.c 636a⟩≡
    ⟨kernel basic includes 35a⟩

#include    <authsrv.h>

⟨devsys.c enum Qxxx 557a⟩

⟨global sysdir 557b⟩

⟨devsys.c decls and globals 559b⟩
⟨global sysname 559j⟩

⟨method sysinit 557c⟩

⟨method sysattach 557d⟩

⟨method syswalk 635a⟩

⟨method sysstat 635b⟩

⟨method sysopen 635c⟩

⟨method sysclose 635d⟩

⟨method sysread 557e⟩

⟨method syswrite 558a⟩

⟨global sysdevtab 558c⟩

```

E.6.12 TO LPSPLIT devices/sys/devenv.c

```

⟨function envgrp 636b⟩≡ (637)
    static Egrp*
    envgrp(Chan *c)
    {
        if(c->aux == nil)
            return up->egrp;
        return c->aux;
    }

```

```

⟨function envwriteable 636c⟩≡ (637)
    static bool
    envwriteable(Chan *c)
    {
        return iseve() || c->aux == nil;
    }

```

```

⟨function getconfenv 636d⟩≡ (637)
    /*
     * Return a copy of configuration environment as a sequence of strings.
     * The strings alternate between name and value. A zero length name string
     * indicates the end of the list
     */
    char *
    getconfenv(void)
    {
        Egrp *eg = &confegrp;

```

```

    Evaluate *e;
    char *p, *q;
    int i, n;

    rlock(eg);
    if(waserror()) {
        runlock(eg);
        nexterror();
    }

    /* determine size */
    n = 0;
    for(i=0; i<eg->nent; i++){
        e = eg->ent[i];
        n += strlen(e->name) + e->len + 2;
    }
    p = malloc(n + 1);
    if(p == nil)
        error(Enomem);
    q = p;
    for(i=0; i<eg->nent; i++){
        e = eg->ent[i];
        strcpy(q, e->name);
        q += strlen(q) + 1;
        memmove(q, e->value, e->len);
        q[e->len] = 0;
        /* move up to the first null */
        q += strlen(q) + 1;
    }
    *q = 0;

    poperror();
    runlock(eg);
    return p;
}

```

⟨devenv.c 637⟩≡
 ⟨kernel basic includes 35a⟩

```

enum
{
    Maxenvsize = 16300,
};

```

⟨devenv.c forward decl ??⟩

```

static Egrp confegrp; /* global environment group containing the kernel configuration */

static Evaluate*
envlookup(Egrp *eg, char *name, ulong qidpath)
{
    Evaluate *e;
    int i;

    for(i=0; i<eg->nent; i++){
        e = eg->ent[i];
        if(e->qid.path == qidpath || (name && e->name[0]==name[0] && strcmp(e->name, name) == 0))
            return e;
    }
    return nil;
}

```

```

}

static int
envgen(Chan *c, char *name, Dirtab*, int, int s, DirEntry *dp)
{
    Egrp *eg;
    Evalue *e;

    if(s == DEVDOTDOT){
        devdir(c, c->qid, "#e", 0, eve, DMDIR|0775, dp);
        return 1;
    }

    eg = envgrp(c);
    rlock(eg);
    e = nil;
    if(name)
        e = envlookup(eg, name, -1);
    else if(s < eg->nent)
        e = eg->ent[s];

    if(e == nil) {
        runlock(eg);
        return -1;
    }

    /* make sure name string continues to exist after we release lock */
    kstrncpy(up->genbuf, e->name, sizeof up->genbuf);
    devdir(c, e->qid, up->genbuf, e->len, eve, 0666, dp);
    runlock(eg);
    return 1;
}

static Chan*
envattach(char *spec)
{
    Chan *c;
    Egrp *egrp = nil;

    if(spec && *spec) {
        if(strcmp(spec, "c") == 0)
            egrp = &confegrp;
        if(egrp == nil)
            error(Ebadarg);
    }

    c = devattach('e', spec);
    c->aux = egrp;
    return c;
}

static Walkqid*
envwalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, 0, 0, envgen);
}

static int
envstat(Chan *c, uchar *db, int n)
{

```

```

    if(c->qid.type & QTDIR)
        c->qid.vers = envgrp(c)->vers;
    return devstat(c, db, n, 0, 0, envgen);
}

static Chan*
envopen(Chan *c, int omode)
{
    Egrp *eg;
    Evalue *e;
    bool trunc;

    eg = envgrp(c);
    if(c->qid.type & QTDIR) {
        if(omode != OREAD)
            error(Eperm);
    }
    else {
        trunc = omode & OTRUNC;
        if(omode != OREAD && !envwriteable(c))
            error(Eperm);
        if(trunc)
            wlock(eg);
        else
            rlock(eg);
        e = envlookup(eg, nil, c->qid.path);
        if(e == nil) {
            if(trunc)
                wunlock(eg);
            else
                runlock(eg);
            error(Enonexist);
        }
        if(trunc && e->value) {
            e->qid.vers++;
            free(e->value);
            e->value = nil;
            e->len = 0;
        }
        if(trunc)
            wunlock(eg);
        else
            runlock(eg);
    }
    c->mode = openmode(omode);
    c->flag |= COPEN;
    c->offset = 0;
    return c;
}

static void
envcreate(Chan *c, char *name, int omode, ulong)
{
    Egrp *eg;
    Evalue *e;
    Evalue **ent;

    if(c->qid.type != QTDIR)
        error(Eperm);
    if(strlen(name) >= sizeof up->genbuf)

```



```

        error("name too long");          /* protect envgen */

omode = openmode(omode);
eg = envgrp(c);

wlock(eg);
if(waserror()) {
    wunlock(eg);
    nexterror();
}

if(envlookup(eg, name, -1))
    error(Eexist);

e = smalloc(sizeof(Evalue));
e->name = smalloc(strlen(name)+1);
strcpy(e->name, name);

// realloc
if(eg->nent == eg->ment){
    eg->ment += 32;
    ent = smalloc(sizeof(eg->ent[0])*eg->ment);
    if(eg->nent)
        memmove(ent, eg->ent, sizeof(eg->ent[0])*eg->nent);
    free(eg->ent);
    eg->ent = ent;
}
e->qid.path = ++eg->path;
e->qid.vers = 0;
eg->vers++;
eg->ent[eg->nent++] = e;
c->qid = e->qid;

wunlock(eg);
poperror();

c->offset = 0;
c->mode = omode;
c->flag |= COPEN;
}

static void
envremove(Chan *c)
{
    int i;
    Egrp *eg;
    Evalue *e;

    if(c->qid.type & QTDIR)
        error(Eperm);

    eg = envgrp(c);
    wlock(eg);
    e = nil;
    for(i=0; i<eg->nent; i++){
        if(eg->ent[i]->qid.path == c->qid.path){
            e = eg->ent[i];
            eg->nent--;
            eg->ent[i] = eg->ent[eg->nent];
            eg->vers++;
        }
    }
}

```

```

        break;
    }
}
wunlock(eg);
if(e == nil)
    error(Enonexist);
free(e->name);
if(e->value)
    free(e->value);
free(e);
}

static void
envclose(Chan *c)
{
    /*
     * cclose can't fail, so errors from remove will be ignored.
     * since permissions aren't checked,
     * envremove can't not remove it if its there.
     */
    if(c->flag & CRCLOSE)
        envremove(c);
}

static long
envread(Chan *c, void *a, long n, vlong off)
{
    Egrp *eg;
    Evalue *e;
    ulong offset = off;

    if(c->qid.type & QTDIR)
        return devdirread(c, a, n, 0, 0, envgen);

    eg = envgrp(c);
    rlock(eg);
    e = envlookup(eg, nil, c->qid.path);
    if(e == nil) {
        runlock(eg);
        error(Enonexist);
    }

    if(offset > e->len) /* protects against overflow converting vlong to ulong */
        n = 0;
    else if(offset + n > e->len)
        n = e->len - offset;
    if(n <= 0)
        n = 0;
    else
        memmove(a, e->value+offset, n);
    runlock(eg);
    return n;
}

static long
envwrite(Chan *c, void *a, long n, vlong off)
{
    byte *s;
    ulong len;
    Egrp *eg;

```

```

    Evaluate *e;
    ulong offset = off;

    if(n <= 0)
        return 0;
    if(offset > Maxenvsize || n > (Maxenvsize - offset))
        error(Etoobig);

    eg = envgrp(c);
    wlock(eg);
    e = envlookup(eg, nil, c->qid.path);
    if(e == nil) {
        wunlock(eg);
        error(Enonexist);
    }

    len = offset+n;
    if(len > e->len) {
        s = smalloc(len);
        if(e->value){
            memmove(s, e->value, e->len);
            free(e->value);
        }
        e->value = s;
        e->len = len;
    }
    memmove(e->value+offset, a, n);
    e->qid.vers++;
    eg->vers++;
    wunlock(eg);
    return n;
}

```

<global envdevtab 534b>

<function envcpy 533d>

<function envgrp 636b>

<function envwriteable 636c>

<function ksetenv 534a>

<function getconfenv 636d>

E.6.13 devices/sys/devpipe.c

*<devpipe.c 642>≡
 <kernel basic includes 35a>*

```

// need that??
#include    "netif.h"

<devpipe.c forward decl ??>

<struct Pipe 403a>

```

<struct Pipealloc 403b>
 <global pipealloc 403c>
 <devpipe.c enum Qxxx 403d>
 <global pipedir 403e>
 #define NPIPEDIR 3
 <method pipeinit 403g>
 <method pipeattach 404b>
 <function pipegen 404c>
 <method pipewalk 405a>
 <method pipestat 405b>
 <method pipewstat 406a>
 <method pipeopen 406b>
 <method pipeclose 407>
 <method piperead 408a>
 <method pipebread 408b>
 <method pipewrite 409a>
 <method pipebwrite 409b>
 <global pipedevtab 410a>

E.6.14 devices/sys/devproc.c

<devproc.c 643>≡
 <kernel basic includes 35a>

```

#include    <trace.h>
#include    <tos.h>
#include    <ureg.h>

<struct Mntwalk 307a>

<devproc.c enum Qxxx 537a>

<devproc enum CMxxx 537b>

enum{
  <constant Nevents 441i>
  <constant Emask 442a>
};

#define STATSIZE    (2*KNAMELEN+12+9*12)
<global procdirent 538a>
  
```

<global proccmd 538b>
 /* Segment type from portdat.h */
 <global sname 92b>
 <devproc QXXX macros 539a>
 <devproc.c forward decl ??>
 <global trace txxx 441f>
 <clock callback profclock 446e>
 <function procgen 549>
 <function _proctrace 441h>
 <method procinit 539d>
 static Chan*
 procattach(char *spec)
 {
 return devattach('p', spec);
 }
 static Walkqid*
 procwalk(Chan *c, Chan *nc, char **name, int nname)
 {
 return devwalk(c, nc, name, nname, 0, 0, procgen);
 }
 static int
 procstat(Chan *c, uchar *db, int n)
 {
 return devstat(c, db, n, 0, 0, procgen);
 }
 <function nonone 550a>
 <method procopen 545b>
 <method procwstat 547>
 <function procoffset 548a>
 <function procqidwidth 548b>
 <function procfidprint 548c>
 <function procfds 550b>
 <method procclose 551>
 <function int2flag 308d>
 <function procargs 552c>
 <function eventsavailable 441g>
 <method procread 540c>

<function mntscan 309>

<method procwrite 543>

<global procdevtab 539e>

<function proctext 552d>

<function procstopwait 428b>

<function procctlcloseone 553a>

<function procctlclosefiles 553b>

<function parsetime 554a>

<function procctlreq 554b>

<function procstopped 428c>

<function procctlmemio 430e>

<function text2data 431>

E.7 files/

E.7.1 files/portdat_files.h

```
<struct Extent 645a>≡ (645c)
struct Extent
{
    int bid;
    ulong start;
    int len;
    Page *cache;
    Extent *next;
};
```

```
<struct Mntcache 645b>≡ (645c)
struct Mntcache
{
    Qid qid;
    int dev;
    int type;
    QLock;
    Extent *list;
    Mntcache *hash;
    Mntcache *prev;
    Mntcache *next;
};
```

```
<portdat_files.h 645c>≡
```

```
<enum blockflag 382a>
```

```

<struct Block 382b>

<function BLEN 382c>
<function BALLOC 382d>

<enum queuestate 349a>

// defined in qio.c
extern uint qiomaxatomic;

/*
 * IO queues
 */
// was in qio.c
<struct Queue 349b>

// was in cache.c
<struct Extent 645a>

// was in cache.c
<struct Mntcache 645b>

<struct Mount 301c>

<struct Mhead 300c>

#include <fcall.h>

// was in devmnt.c
<struct Mntrpc 519c>

<struct Mnt 517b>

<struct Path 283b>

<enum accessnamec 267c>

<enum channelflag 257b>

<struct Chan 51e>

<struct Evalue 532b>

<struct Egrp 533a>

// internals

<struct Walkqid 286b>

<struct Dev 54a>

```

```

// array<Dev>, it looks like an allocated array<ref<dev>> but
// it is really a static array put here to avoid backward deps on conf_devtab,
// and it is not really a <ref<dev>> because it's pointers to static
// structures (e.g. mousedevtab, vgadevtab, etc).
extern Dev** devtab;

<struct Dirstab 370a>

// used by with devmnt.c and sysfile.c
struct Bogus {
    Chan    *chan;
    Chan    *authchan;
    char    *spec;
    int flags;
};

//*****
// Internal to files/
//*****

<constants tags 518b>

// actually internal to devmnt.c and mnt.c
<struct Mntalloc 517c>
extern struct Mntalloc mntalloc;

```

E.7.2 files/portfns_files.h

E.7.3 files/chan.c

```

<chan.c 647>≡
<kernel basic includes 35a>

<chan.c forward decl ??>

<chan.c debugging macro ??>

enum
{
    <constant PATHSLOP 283d>
    <constant PATHMSLOP 311c>
};

<struct Chanalloc 258a>
<global chanalloc 258c>

<struct Elemlist 271b>

<function chanpath 295a>

<function isdotdot 285a>

<function emptystr 600b>

<function chandevreset 250a>

<function chandevinit 253a>

```


⟨function chandevshutdown 562c⟩
⟨constructor newchan 258d⟩
⟨global npath ??⟩
⟨constructor newpath 283c⟩
⟨function copypath 283e⟩
⟨destructor pathclose 284a⟩
⟨function fixdotdotname 285b⟩
⟨function uniquepath 284b⟩
⟨function addelem 284c⟩
⟨destructor chanfree 259a⟩
⟨function cclose 266c⟩
⟨struct Clunkq 279b⟩
⟨global clunkq 279c⟩
⟨function ccloseq 279d⟩
⟨function clunkwork 280a⟩
⟨function closeproc 280b⟩
⟨function uniquechan 259b⟩
⟨function eqqid 52d⟩
⟨function eqchan 53b⟩
⟨function eqchantdqid 53c⟩
⟨constructor newmhead 301a⟩
⟨function cmount 306c⟩
⟨function cunmount 320c⟩
⟨function cclone 290b⟩
⟨function findmount 310d⟩
⟨function domount 311a⟩
⟨function undomount 312e⟩
⟨function ewalk 289a⟩
⟨global Edoesnotexist 596a⟩
⟨function walk 287⟩

<function createdir 316c>
 <function growparse 272c>
 <function parsename 272b>
 <function memrchr 599b>
 <function namelerror 273d>
 <function nameerror 274a>
 <function namec 267g>
 <function skipslash 273a>
 <global isfrog 271a>
 <function validname0 270b>
 <function validname 270a>
 <function validnamedup 269d>
 <function error_if_not_dir 282b>
 <destructor putmhead 301b>

E.7.4 files/dev.c

<dev.c 649>≡
 <kernel basic includes 35a>

// was in \$CONF.c
 <global devtab 54b>
 <function mkqid 53a>
 <function devno 260b>
 <function devdir 375b>
 <function devgen 375c>

```

void
devreset(void)
{
}

```

```

void
devinit(void)
{
}

```

```

void
devshutdown(void)
{
}

```

<function devattach 260a>

<function devclone 376>

<function devwalk 377>

<function devstat 378>

<function devdirread 379c>

<function devpermcheck 380a>

<function devopen 380b>

```
void
devcreate(Chan*, char*, int, ulong)
{
    error(Eperm);
}
```

<function devbread 381b>

<function devbrwrite 381c>

```
void
devremove(Chan*)
{
    error(Eperm);
}
```

```
int
devwstat(Chan*, uchar*, int)
{
    error(Eperm);
    return 0;
}
```

```
//void
//devpower(int)
//{
//    error(Eperm);
//}
//
//int
//devconfig(int, char *, DevConf *)
//{
//    error(Eperm);
//    return 0;
//}
```

E.7.5 files/allocb.c

<allocb.c 650>≡
<kernel basic includes 35a>

```
enum
{
    Hdrspc      = 64,          /* leave room for high-level headers */
```

```

    Bdead      = 0x51494F42, /* "QIOB" */
};

⟨struct Ialloc 383c⟩
⟨global ialloc 384a⟩

⟨function _alloca 382f⟩

⟨function allocb 383b⟩

⟨function iallocb 384b⟩

⟨function freeb 384c⟩

⟨function checkb 385a⟩

⟨function iallocsummary 576d⟩

```

E.7.6 files/env.c

```

⟨env.c 651a⟩≡
    ⟨kernel basic includes 35a⟩

    // this used to be in devenv.c, but to avoid backward deps I've splitted
    // this file in 2
    ⟨function closeegrp 533c⟩

```

E.7.7 files/file.c

```

⟨file.c 651b⟩≡
    ⟨kernel basic includes 35a⟩

    ⟨function fdtochan 265b⟩

    ⟨function openmode 264c⟩

    ⟨function fdclose 266a⟩

    ⟨function validstat 298b⟩

```

E.7.8 files/mnt.c

```

⟨mnt.c 651c⟩≡
    ⟨kernel basic includes 35a⟩

    // this used to be in devmnt.c, but to avoid backward deps I've splitted
    // this file in 2 (which forced to put more stuff in portdat_files.h though).

    ⟨global mntalloc 518a⟩

    #define MAXRPC (IOHDRSZ+8192)

    ⟨function freetag 518c⟩

    ⟨function mntfree 518d⟩

```

<function mntpntfree 519a>

<function muxclose 519b>

<function mntversion 520b>

E.7.9 files/qio.c

<global qiomaxatomic 652a>≡ (652b)

```
enum
{
    Maxatomic    = 64*1024,
};
```

```
uint    qiomaxatomic = Maxatomic;
```

<qio.c 652b>≡

<kernel basic includes 35a>

<qio.c globals cnt ??>

<qio.c global debugging ??>

<global qiomaxatomic 652a>

<function ixsummary 576c>

<function freeblist 385b>

<function padblock 386>

<function blocklen 387a>

<function blockalloclen 387b>

<function concatblock 387c>

<function pullupblock 388>

<function pullupqueue 350a>

<function trimblock 389b>

<function copyblock 390a>

<function adjustblock 390b>

<function pullblock 391a>

<function qget 351a>

<function qdiscard 351b>

<function qconsume 352>

<function qpass 354>

<function qpassnolim 355a>

`<function packblock 389a>`
`<function qproduce 355b>`
`<function qcopy 356>`
`<function qopen 357>`
`<function qbypass 358a>`
`<function notempty 358b>`
`<function qwait 358c>`
`<function qaddlist 359a>`
`<function qremove 359b>`
`<function bl2mem 391b>`
`<function qputback 359c>`
`<function qwakeup_iunlock 360a>`
`<function qbread 360b>`
`<function qread 361>`
`<function qnotfull 363a>`
`<function qbwrite 363b>`
`<function qwrite 350b>`
`<function qiwrite 365>`
`<function qfree 366a>`
`<function qclose 366b>`
`<function qhangup 367a>`
`<function qiclosed 367b>`
`<function qreopen 367c>`
`<function qlen 368a>`
`<function qwindow 368b>`
`<function qcanread 368c>`
`<function qsetlimit 368d>`
`<function qnblock 368e>`
`<function qflush 368f>`
`<function qfull 369>`

E.7.10 files/sysfile.c

```
<sysfile.c 654>≡  
  <kernel basic includes 35a>  
  
  /*  
   * The sys*() routines needn't poperror() as they return directly to syscall().  
   */  
  
  <function unlockfgrp 263c>  
  
  <function growfd 263a>  
  
  <function findfreefd 262d>  
  
  <function newfd 262c>  
  
  <function newfd2 402a>  
  
  
  <syscall fd2path 294c>  
  
  <syscall pipe 401>  
  
  <syscall dup 556b>  
  
  <syscall open 264b>  
  
  <syscall close 265a>  
  
  <function unionread 315a>  
  
  <function unionrewind 316a>  
  
  <function dirfixed 326>  
  
  <function offsetof_name_dirent 296a>  
  
  <function dirent_setname 295d>  
  
  <function mountrock 323d>  
  
  <function mountrockread 324>  
  
  <function mountrewind 325a>  
  
  <function mountfix 325b>  
  
  <function read 275b>  
  
  <syscall pread 274b>  
  
  <function write 276c>  
  
  <syscall pwrite 276a>  
  
  <function sseek 277b>
```

<syscall seek 277a>
 <function pathlast 297b>
 <syscall fstat 297a>
 <syscall stat 296c>
 <syscall chdir 294a>
 <function bindmount 305b>
 <syscall bind 304d>
 <syscall mount 318g>
 <syscall unmount 319d>
 <syscall create 290c>
 <syscall remove 293d>
 <function wstat 298c>
 <syscall wstat 297d>
 <syscall fstat 298a>

E.8 filesystems/

E.9 init/

E.9.1 init/portfns_init.h

E.9.2 init/user/preboot/initcode.c

E.9.3 init/user/boot/aux.c

E.9.4 init/user/boot/boot.c

E.9.5 init/user/boot/boot.h

E.9.6 init/user/boot/local.c

E.10 lib/

E.10.1 lib/lib.h

<constant ERRMAX 655a>≡ (? 0—1)
 ERRMAX = 128, /* max length of error string */

<constant KNAMELEN 655b>≡ (? 0—1)
 KNAMELEN = 28, /* max length of name held in kernel */


```

⟨function nelem 656a⟩≡ (656e)
    #define nelem(x) (sizeof(x)/sizeof((x)[0]))

⟨function offsetof 656b⟩≡ (656e)
    // @Scheck: not dead, actually exanded in macros.h
    #define offsetof(s, m) (ulong)&(((s*)0)->m))

⟨function assert 656c⟩≡ (656e)
    #define assert(x) do { if(x) {}else _assert("x");}while(0)

⟨lib.h print decl 656d⟩≡ (656e)
    // This used to be regular function, but to avoid backward deps in the kernel
    // I made it into a pointer function (a bit ugly, and maybe unsafe)
    extern int (*print)(char*, ...);

⟨lib.h 656e⟩≡
    // coupling: libc.h
    // This file mostly exports code from lib_core/libc/ (linked with the kernel).
    // Those functions are also exported in include/libc.h but they preferred to
    // not include it and instead to reexport here a subset specific to the kernel.
    // There are also the poolxxx() functions exported in include/pool.h
    // (also part of libc) that are used in memory/alloc.c.
    // Some functions in libc are also "overridden" (via linker abuse):
    // - print.c overrides _fmtlock(), _fmtunlock() that were in libc/fmt/
    // - devcons.s overrides the sysfatal() that was in libc/9sys/

    // pad specific, could be in u.h
    ⟨pad basic types ??⟩

    // #define nil (void*)0 in u.h
    #define nilptr 0

    ⟨lib.h forward decl ??⟩

    /*
     * functions (mostly) linked in from libc.
     */
    ⟨function nelem 656a⟩
    ⟨function offsetof 656b⟩
    ⟨function assert 656c⟩

    /*
     * mem routines
     */
    ⟨lib.h mem functions decl 599a⟩

    /*
     * string routines
     */
    ⟨lib.h string functions decl 599c⟩

    // redefined in the kernel
    extern int cistrncmp(char*, char*);
    extern int cistrncmp(char*, char*, int);

    ⟨enum utf 602d⟩

    /*
     * rune routines
     */

```

```

<lib.h rune functions decl 603a>

extern  int  abs(int);

/*
 * print routines
 */
typedef int  (*Fmts)(Fmt*);
<struct Fmt 602a>

<lib.h print decl 656d>

<lib.h printf functions decl 601d>

<lib.h pragmas 602c>

<lib.h fmt functions decl 602b>

/*
 * one-of-a-kind
 */
<lib.h strt functions decl 601a>

//todo: should return a (void*)
extern  ulong  getcallerpc(void*);
extern  char*  cleannname(char*);
extern  int  getfields(char*, char**, int, int, char*);
extern  int  tokenize(char*, char**, int);
extern  void  qsort(void*, long, long, int (*)(void*, void*));

<lib.h exxx decl 239b>

/*
 * Syscall data structures
 */
<enum mount 300b>

<enum open 264a>

<enum note 395a>

<enum miscsize ??>

<enum qidtype 52c>

<enum dirmode 295c>

<struct Qid 52b>

<struct DirEntry 295b>

<struct Waitmsg 127f>

```

E.11 memory/

E.11.1 memory/portdat_memory.h

⟨portdat_memory.h 658⟩≡

```
// see also Arch_KMap in <arch>/ (but used in port)

//*****
// Page < Pagetable (can be filled by KImage) < Segment
//*****

// All the ref<Page> here are references to Pages in the array<Page> of
// Palloc.pages (pool allocator)
// All the ref<KImage> here are references to KImage in the ?? of
// Imagealloc.free?

⟨enum modref 88a⟩

⟨enum Cachect1 493a⟩

⟨struct Page 87⟩

⟨type PageOrSwap 180a⟩

⟨struct Pagetable 93b⟩

// used to be arch-specific, but many constants were used in port/ code
⟨type PTExxx 160b⟩

⟨struct KImage 168a⟩

⟨enum segtype 92a⟩

⟨constant PG_ONSWAP 179c⟩

⟨function onswap 180b⟩
⟨function pagedout 171d⟩
⟨function swapaddr 188b⟩

#define SEGMAXSIZE (PAGEDIRSIZE*PAGETABMAPMEM)

⟨struct Physseg 494k⟩

enum
{
    ⟨constant LRESPROF 446c⟩
};

⟨struct Segment 92c⟩

//*****
// Internal to memory/
//*****

// See also RMap in 386/

// actually internal to xalloc.c, but important so here
⟨constants holes 74c⟩
```

<struct Hole 74d>

<struct Xhdr 76a>

<struct Xalloc 74a>

```
//IMPORTANT: static Xalloc xlists; // private to xalloc.c
```

```
// from pool.h
//struct Pool {
//  char* name;
//  ulong maxsize;
//
//  ulong cursize;
//  ulong curfree;
//  ulong curalloc;
//
//  ulong minarena; /* smallest size of new arena */
//  ulong quantum; /* allocated blocks should be multiple of */
//  ulong minblock; /* smallest newly allocated block */
//
//  void* freeroot; /* actually Free* */
//  void* arenalist; /* actually Arena* */
//
//  void* (*alloc)(ulong);
//  int (*merge)(void*, void*);
//  void (*move)(void* from, void* to);
//
//  int flags;
//  int nfree;
//  int lastcompact;
//
//  void (*lock)(Pool*);
//  void (*unlock)(Pool*);
//  void (*print)(Pool*, char*, ...);
//  void (*panic)(Pool*, char*, ...);
//  void (*logstack)(Pool*);
//
//  void* private;
//};

// exported by libc include/pool.h, used by malloc, defined in pool.c in this dir
// memory pools for malloc()/free() (using xalloc pools)
//IMPORTANT: extern Pool* mainmem;
// memory pools for ??
//IMPORTANT: extern Pool* imagmem;
```

<struct Pallocmem 89b>

<function pghash 165d>

<struct Palloc 88b>

```
extern Palloc palloc;
```

<constant NFRECHAN 249d>

<function ihash 168d>

<struct Imagealloc 168b>

```
//IMPORTANT: static struct Imagealloc imagealloc; (segment.c)
// so have conf.nimage + 1 Kimages
extern KImage swapimage;

<struct Swapalloc 178a>
extern struct Swapalloc swapalloc;
```

E.11.2 memory/portfns_memory.h

E.11.3 memory/portdat_memory.c

```
<portdat_memory.c 660a>≡
<kernel basic includes 35a>

<global palloc 88d>

<global swapalloc 178b>
<global swapimage 178d>
```

E.11.4 memory/xalloc.c

```
<xalloc.c 660b>≡
<kernel basic includes 35a>

<xalloc.c forward decl ??>

//*****
// The global
//*****

<global xlists 75>

//*****
// Initialization
//*****

<function xinit 246b>

//*****
// Functions
//*****

<function xspanalloc 79a>

<function xallocz 76d>

<function xalloc 76c>

<function xfree 77a>

<function xmerge 78a>

<function xhole 77c>

//*****
// Debugging
//*****
```

<function xsummary 575a>

E.11.5 memory/pool.c

```
<pool.c forward decl 661a>≡ (661b)
    typedef struct Private Private;

<pool.c 661b>≡
    <kernel basic includes 35a>

    #include    <pool.h>

    <pool.c forward decl 661a>

    //*****
    // Concurrency
    //*****

    // See Pool.private, for mutual exclusion on memory pools
    <pool.c struct Private 80a>

    //*****
    // Pool methods
    //*****

    <function poolprint 82c>

    <function ppanic 83a>

    <function plock 80b>

    <function punlock 81a>

    <function poolsummary 575c>

    //*****
    // The globals
    //*****

    <global pmainmem 81b>

    <global pimagemem 82a>

    <global mainmem and imagemem 82b>

    <function mallosummary 575b>
```

E.11.6 memory/alloc.c

```
<alloc.c 661c>≡
    <kernel basic includes 35a>

    #include    <pool.h>

    /* everything from here down should be the same in libc, libdebugmalloc, and the kernel */
    /* - except the code for malloc(), which alternately doesn't clear or does. */
```

```

/* - except the code for smalloc(), which lives only in the kernel. */

//*****
// Debugging support
//*****

/*
 * Npadlong is the number of 32-bit longs to leave at the beginning of
 * each allocated buffer for our own bookkeeping. We return to the callers
 * a pointer that points immediately after our bookkeeping area. Incoming pointers
 * must be decremented by that much, and outgoing pointers incremented.
 * The malloc tag is stored at MallocOffset from the beginning of the block,
 * and the realloc tag at ReallocOffset. The offsets are from the true beginning
 * of the block, not the beginning the caller sees.
 *
 * The extra if(Npadlong != 0) in various places is a hint for the compiler to
 * compile out function calls that would otherwise be no-ops.
 */

/* tracing */
enum {
    Npadlong      = 2,
    MallocOffset  = 0,
    ReallocOffset = 1
};

<alloc.c debugging macro ??>

<function setmalloctag 85c>

<function setrealloctag 85d>

//*****
// Pool wrappers
//*****

<function smalloc 84a>

<function malloc 83b>

<function mallocz 84c>

<function mallocalign 84d>

<function free 83c>

<function realloc 85a>

<function msize 383a>

//*****
// kstr functions
//*****

//pad: was in chan.c

<function kstrcpy 599d>

<function kstrdup 600a>

```

E.11.7 memory/fault.c

```
<function checkpages 663a>≡ (663b)
void
checkpages(void)
{
    int checked;
    ulong addr, off;
    Pagetable *p;
    Page *pg;
    Segment **sp, **ep, *s;

    if(up == nil)
        return;

    checked = 0;
    // foreach(up->seg)
    for(sp=up->seg, ep=&up->seg[NSEG]; sp<ep; sp++){
        s = *sp;
        if(s == nil)
            continue;
        qlock(&s->lk);
        for(addr=s->base; addr<s->top; addr+=BY2PG){
            off = addr - s->base;
            p = s->pagedir[off/PAGETABMAPMEM];
            if(p == nil)
                continue;
            pg = p->pagetab[(off&(PAGETABMAPMEM-1))/BY2PG];
            if(pg == nil || pagedout(pg))
                continue;
            arch_checkmmu(addr, pg->pa);
            checked++;
        }
        qunlock(&s->lk);
    }
    print("%ld %s: checked %d page table entries\n", up->pid, up->text, checked);
}

<fault.c 663b>≡
<kernel basic includes 35a>

<fault.c forward decl ??>

<function fault 160a>

<function faulterror 173>

// for debugging SG_PHYSICAL
void (*checkaddr)(ulong, Segment *, Page *);
ulong addr2check;

<function fixfault 161>

<function pio 171g>

<function okaddr 98c>

<function validaddr 99b>

<function vmemchr 99c>
```


⟨function seg 98b⟩

⟨function checkpages 663a⟩

E.11.8 memory/page.c

⟨page.c 664⟩≡

⟨kernel basic includes 35a⟩

⟨page.c forward decl ??⟩

```
//*****  
// Initialization  
//*****
```

⟨function pageinit 248⟩

```
//*****  
// Functions  
//*****
```

⟨function pageunchain 90c⟩

⟨function pagechaintail 91b⟩

⟨function pagechainhead 91a⟩

⟨constructor newpage 89c⟩

⟨function hasfreepages 184c⟩

⟨destructor putpage 90d⟩

⟨function auxpage ??⟩

⟨global dupretries 174b⟩

⟨function duppage 174c⟩

⟨function copypage 165b⟩

⟨function uncachepage 167c⟩

⟨function cachepage 166a⟩

⟨function cachedel 167d⟩

⟨function lookpage 166b⟩

⟨function ptcpy 94d⟩

⟨constructor ptalloc 94b⟩

⟨destructor freept 94c⟩

⟨function pagenumber 498a⟩

⟨function checkpagerefs 496d⟩

<function portcountpagerefs 497c>

E.11.9 memory/segment.c

```
<segment.c 665>≡
<kernel basic includes 35a>
#include "io.h"

//*****
// Global
//*****

<global imagealloc 168e>

//*****
// Misc
//*****

<global physseg 495a>
<global physseglock 495b>

<hook _globalsegattach 411c>

<segment.c forward decl ??>

//*****
// Initialization
//*****

<function imageinit 249e>

//*****
// Functions
//*****

<constructor newseg 95>

<destructor putseg 96b>

<function relocateseg 125b>

<function dupseg 100b>

<function segpage 97c>

<constructor attachimage 169c>

<struct Irstats 592i>
<segment.c global irstats 592j>

<function imagereclaim 177b>

<function imagechanreclaim 177a>

<destructor putimage 176b>

<function ibrk 102b>
```

<function mfreeseg 104b>
 <function isoverlap 413a>
 <function addphysseg 495c>
 <function segattach 411d>
 <clock callback segclock 446f>
 // was in another file before
 <function data2txt 171c>

E.11.10 memory/swap.c

<swap.c 666>≡
 <kernel basic includes 35a>
 <swap.c forward decl ??>
 <global iolist 182b>
 <global ioptr 182c>
 <global genxxx 187b>
 <function gentick 187a>
 <function swapinit 249b>
 <function newswap 179a>
 <function putswap 180c>
 <function dupswap 180f>
 <function swapcount 180h>
 <function kickpager 184d>
 <function pager 184e>
 <function pageout 182d>
 <function canflush 186e>
 <function pagepte 181>
 <function pagersummary 576b>
 <function pageiocomp 188a>
 <function executeio 187d>
 <function needpages 185>
 <function setswapchan 188d>

E.11.11 memory/sysmemory.c

<sysmemory.c 667a>≡
<kernel basic includes 35a>

```
// see also sysbrk in sysproc.c

// those functions used to be in segment.c

void
ptflush(Pagetable *pt, int s, int e)
{
    int i;
    Page *p;

    for(i = s; i < e; i++) {
        p = pt->pagetab[i];
        if(pagedout(p) == 0)
            memset(p->cachectl, PG_TXTFLUSH, sizeof(p->cachectl));
    }
}

<syscall segflush 415c>
```

E.12 misc/

E.12.1 misc/portdat_misc.h

<portdat_misc.h 667b>≡

```
enum misc_constants_portable {
    /* READSTR was 1000, which is way too small for usb's ctl file */
    READSTR = 4000,    /* temporary buffer size for device reads */

    KB = 1024,
    MB = (1024*1024),

    NUMSIZE = 12,    /* size of formatted number */

    <constant PRINTSIZE 342b>
};

<struct Cmdbuf 530b>

<struct Cmdtab 530a>
```

E.12.2 misc/portfns_misc.h

E.12.3 misc/parse.c

<parse.c 667c>≡
<kernel basic includes 35a>

<ncmdfield 532a>

⟨function parsecmd 530c⟩
 ⟨function cmderror 531a⟩
 ⟨function lookupcmd 531b⟩

E.12.4 misc/random.c

⟨random.c 668a⟩≡
 ⟨kernel basic includes 35a⟩
 ⟨struct Rb 568d⟩
 ⟨global rb 568e⟩
 ⟨struct rbnotfull 568f⟩
 ⟨struct rbnotempty 569a⟩
 ⟨function genrandom 569b⟩
 ⟨clock callback randomclock 569c⟩
 ⟨function randominit 569d⟩
 ⟨function randomread 570a⟩

E.13 interrupts/

E.13.1 interrupts/portfns_interrupts.h

⟨portfns_interrupts.h 668b⟩≡
 // <arch>/trap.c (used in arch specific)
 void arch_intrenable(int, void (*)(Ureg*, void*), void*, int, char*);
 // <arch>/trap.c (called from main)
 void arch__trapinit(void);

E.14 processes/

E.14.1 processes/portdat_processes.h

⟨portdat_processes.h 668c⟩≡
 // in lib.h: Waitmsg, ERRMAX
 //*****
 // Proc components
 //*****
 // All the ref<Proc> here are references to Proc in the array<Proc> of
 // Proccalloc.arena (pool allocator)
 //-----

```

// State
//-----

// TODO: state transition diagram
⟨enum procstate 51b⟩

// hash<enum<procstate>, string>, to debug
extern char* statename[];

//-----
// Memory
//-----
⟨enum procseg 97e⟩

//-----
// Files
//-----

enum
{
    ⟨constant DELTAFD 262a⟩
};

⟨struct Fgrp 260c⟩

⟨function MOUNTH 303b⟩

⟨struct Pgrp 302d⟩

//-----
// System call
//-----

⟨constant MAXSYSARG 141a⟩

⟨struct Sargs 141b⟩

//-----
// Notes
//-----

enum {
    ⟨constant NNOTE 396d⟩
};

⟨enum notekind 394c⟩

⟨struct Note 395c⟩
extern Counter noteidalloc;

//-----
// Process children waiting
//-----

⟨struct Waitq 127g⟩

//-----
// Synchronization (Rendez vous)
//-----

```

```

<function REND 421b>

<struct Rgrp 421c>

//-----
// Alarms, timers
//-----
// now in portdat_time.h

//-----
// Scheduling
//-----

enum {
    <constant Npriq 190a>
    <constant Nrq 192a>
};

<enum priority 190b>

<enum edfflags 504g>

<struct Edf 504h>

//-----
// Error management
//-----
enum {
    <constant NERR 596d>
};

//-----
// Stats, profiling
//-----
<enum proctimer 444a>

//-----
// Debugger
//-----

<enum procctl 426e>

//-----
// Misc
//-----

<enum fpsavestatus 453a>

//*****
// Proc, the big one
//*****

<struct Proc 49f>

<macro waserror poperror 597c>

//*****
// Internal to process/

```

```
//*****

// Proc allocator (singleton), was actually in proc.c, but important so here
<struct Procalloc 107a>
//IMPORTANT: static struct Procalloc procalloc; (in proc.c)

<struct Schedq 192c>
// hash<enum<priority>, Schedq>, Nrq is the number of priority level (20+2)
//IMPORTANT: Schedq runq[Nrq]; (in proc.c)

// was in alarm.c, but important so here
<struct Alarms 224d>
//IMPORTANT: static Alarms alarms; (in alarm.c)
//IMPORTANT: static Rendez alarmr; (in alarm.c)

<struct Active 49d>
extern struct Active active;
```

E.14.2 processes/portfns_processes.h

E.14.3 processes/portdat_processes.c

```
<portdat_processes.c 671a>≡
<kernel basic includes 35a>

<hook proctrace 440c>
<hook kproftimer 590c>

<global active 49e>
```

E.14.4 time/portfns_time.h

```
<portfns_time.h 671b>≡

// portclock.c
ulong tk2ms(ulong);
<macro TK2MS 214a>
<macro MS2NS 213c>
#define TK2SEC(t) ((t)/Arch_HZ) /* ticks to seconds */ // not used in port
// #define MS2HZ (1000/Arch_HZ) /* millisec per clock tick */
ulong ms2tk(ulong);
void timerdel(Timer*);
void timeradd(Timer*);
Timer* addclock0link(void (*)(void), int);
void timerintr(Ureg*, Tval);
void timersinit(void);

// alarm.c
ulong procalarm(ulong);
void alarmkproc(void*);
void checkalarms(void);

// tod.c
// initialize the hidden global tod.c#tod
void todinit(void);
void todsetfreq(vlong);
void todset(vlong, vlong, int);
```



```

vlong   todget(vlong*);
uvlong  fastticks2us(uvlong);
uvlong  ns2fastticks(uvlong);
long    seconds(void);

// <arch>/clock.c (called from port)
void    arch_timerset(Tval x); // called from portclock.c (e.g., addclock0link)
ulong   arch_us(void);        // called from edf.c
long    arch_lcycles(void); // called from edf.c, sysproc.c, taslock.c
ulong   arch_perfticks(void);
// now in core/ for backward deps
//uvlong arch_fastticks(uvlong *hz); // called from portclock.c
//void    arch_microdelay(int);
//void    arch_delay(int);
// <arch>/clock.c (called from port but signature not portable across <arch>)
//void arch_cycles(uvlong*);

```

E.14.5 time/portdat_time.h

```

<portdat_time.h 672a>≡

<enum timermode 216b>

<type Txxx 213b>

<struct Timer 216a>

// was in clock.c
<struct Timers 216c>

// used only in arm/ for now
enum {
    Mhz = 1000 * 1000,
};

// <arch>/dat_time.h should defined Arch_HZ

#pragma varargck type "t" long
#pragma varargck type "U" uvlong

```

E.14.6 time/alarm.c

```

<alarm.c 672b>≡
<kernel basic includes 35a>

<global alarms 224f>
<global alarmr 226c>

<kernel process alarmkproc 226d>

<function checkalarms 227b>

<function procalarm 225c>

```

E.14.7 time/portclock.c

```
<portclock.c 673a>≡  
  <kernel basic includes 35a>  
  
  #include "io.h"  
  
  #include <ureg.h>  
  
  enum {  
    <constant Maxtimerloops 221d>  
  };  
  
  <global timers 217b>  
  <global timersinit 245b>  
  
  <clock.c statistics 592l>  
  
  <function tadd 217c>  
  
  <function tdel 218b>  
  
  <function timeradd 218d>  
  
  <function timerdel 219c>  
  
  <clock callback hzclock 207a>  
  
  <interrupt callback timerintr 220d>  
  
  <function timersinit 245c>  
  
  <function addclock0link 221f>  
  
  <function tk2ms 214b>  
  
  <function ms2tk 214c>
```

E.14.8 processes/pgrp.c

```
<pgrp.c 673b>≡  
  // Process group, and Namespace.  
  
  <kernel basic includes 35a>  
  
  enum {  
    <constant Whinesecs 110c>  
  };  
  
  <global pgrpuid 303e>  
  <global mountid 321b>  
  
  <function pgrpnote 545a>  
  
  <constructor newpgrp 303f>  
  
  <constructor newrgrp 422b>  
  
  <destructor closergrp 422c>
```

<destructor closegrp 303g>
 <function pgrpinsert 322b>
 <function pgrpcpy 322c>
 <function dupfgrp 261b>
 <function closefgrp 262b>
 <function forceclosefgrp 279a>
 <constructor newmount 302a>
 <destructor mountfree 302c>
 <function resrcwait 110b>

E.14.9 processes/proc.c

```

<Procctl cases 674a>+= (426e) <443a
    Proc_exitbig,

<procctl() Proc_exitbig case 674b>= (427b)
    case Proc_exitbig:
        arch_spllo();
        pexit("Killed: Insufficient physical memory", true);
        // Fallthrough

<function killbig 674c>= (675c)
    void
    killbig(char *why)
    {
        int i;
        Segment *s;
        ulong l, max;
        Proc *p, *ep, *kp;

        max = 0;
        kp = 0;
        ep = procalloc.arena+conf.nproc;
        for(p = procalloc.arena; p < ep; p++) {
            if(p->state == Dead || p->kp)
                continue;
            l = 0;
            for(i=1; i<NSEG; i++) {
                s = p->seg[i];
                if(s != 0)
                    l += s->top - s->base;
            }
            if(l > max && ((p->procmode&0222) || strcmp(why, p->user)!=0)) {
                kp = p;
                max = l;
            }
        }

        print("%lud: %s killed: %s\n", kp->pid, kp->text, why);
        for(p = procalloc.arena; p < ep; p++) {

```

```

    if(p->state == Dead || p->kp)
        continue;
    if(p != kp && p->seg[BSEG] && p->seg[BSEG] == kp->seg[BSEG])
        p->procctl = Proc_exitbig;
}
kp->procctl = Proc_exitbig;
for(i = 0; i < NSEG; i++) {
    s = kp->seg[i];
    if(s != 0 && canqlock(&s->lk)) {
        mfreeseg(s, s->base, (s->top - s->base)/BY2PG);
        qunlock(&s->lk);
    }
}
}

```

⟨Proc scheduling fields 675a⟩+≡ (49f) <504d

```

    bool preempted; /* true if this process hasn't finished the interrupt
        * that last preempted it
        */

```

⟨function preempt 675b⟩≡ (675c)

```

/*
 * here at the end of non-clock interrupts to see if we should preempt the
 * current process.
 */
void
preempt(void)
{
    if(up && up->state == Running)
        if(up->preempted == false)
            if(anyhigher())
                if(!active.exiting){
                    cpu->readied = nil; /* avoid cooperative scheduling */
                    up->preempted = true;
                    sched();
                    arch_splhi(); // still in interrupt context
                    up->preempted = false;
                }
    return;
}

```

⟨proc.c 675c⟩≡

```

⟨kernel basic includes 35a⟩
#include    <trace.h>

//*****
// Globals
//*****

⟨global runq 192b⟩
⟨global runveq 192e⟩

⟨global procalloc 107b⟩

⟨global nrdy 192g⟩
⟨global noteidalloc 395b⟩

⟨proc.c statistics 592k⟩

⟨global pidalloc 106c⟩

```

```

enum
{
    <constant Schedagain 209g>
    <constant Scaling 209f>
};

<global statename 51c>

<proc.c forward decl ??>

//*****
// Error management
//*****

// see also waserror() and poperror() macro in portdat_processes.h

<function error 597a>

<function nexterror 597d>

<function exhausted 597b>

//*****
// Functions
//*****

<function schedinit 195e>

<function sched 194c>

<function anyready 192f>

<function anyhigher 206f>

<function hzsched 206e>

<function preempt 675b>

<function updatecpu 209i>

<function reprioritize 210>

<function queueproc 193a>

<function dequeueproc 193b>

<function ready 202c>

<function yield 202a>

<global balancetime 212a>

<function rebalance 211b>

<function runproc 196>

<function canpage 186f>

<function noprocpanic 110a>

```

<constructor newproc 108c>
<function procwired 208a>
<function procpriority 191a>
<function procinit 247a>
<function sleep 204>
<function tfn 224b>
<function twakeup 224c>
<function tsleep 223c>
<function wakeup 205c>
<function postnote 400>
<struct Broken 425b>
<global broken 425c>
<function addbroken 425e>
<function unbreak 426a>
<function freebroken 426b>
<function pexit 130b>
<function haswaitq 129d>
<function pwait 128c>
<function proctab 539b>
<function dumpaproc 577a>
<function procdump 576f>
<function procflushseg 414d>
<function scheddump 576e>
<function kproc 135b>
<function procctl 427b>
<function killbig 674c>
<function renameuser 111d>
<function accounttime 588b>
<function pidhash 107e>
<function pidunhash 107f>

<function procindex 108a>

E.14.10 time/tod.c

<tod.c 678>≡

// TOD: Time Of Day.

<kernel basic includes 35a>

<tod.c forward decl ??>

*/**

** Compute nanosecond epoch time from the fastest ticking clock*
** on the system. Converting the time to nanoseconds requires*
** the following formula*

**
* t = (((1000000000<<31)/f)*ticks)>>31*

** where*

** 'f' is the clock frequency*

** 'ticks' are clock ticks*

** to avoid too much calculation in todget(), we calculate*

** mult = (1000000000<<32)/f*

** each time f is set. f is normally set by a user level*
** program writing to /dev/fastclock. arch_mul64fract will then*
** take that fractional multiplier and a 64 bit integer and*
** return the resulting integer product.*

** We assume that the cpu's of a multiprocessor are synchronized.*

** This assumption needs to be questioned with each new architecture.*

**/*

/ frequency of the tod clock */*

#define TODFREQ 1000000000ULL

#define MicroFREQ 1000000ULL

<struct TOD 227c>

<global tod 227d>

<function todinit 228c>

<function todsetfreq 228d>

<function todset 228e>

<function todget 229>

<function todfix 230a>

<function seconds 230b>

<function fastticks2us 230d>

<function ns2fastticks 231a>

⟨function mk64fract 231b⟩

E.14.11 processes/sysproc.c

⟨sysproc.c 679a⟩≡
⟨kernel basic includes 35a⟩

```
#include    <tos.h>
#include    <a.out.h>
```

⟨enum rfork 112a⟩

⟨sysproc.c forward decl ??⟩

⟨syscall nop 42b⟩

⟨syscall rfork 112b⟩

⟨function l2be 121b⟩

⟨syscall exec 119d⟩

⟨function shargs 451a⟩

⟨syscall sleep 223a⟩

⟨syscall alarm 224g⟩

⟨syscall exits 129e⟩

⟨syscall await 128a⟩

⟨function werrstr 598c⟩

⟨function generrstr 598b⟩

⟨syscall errstr 598a⟩

⟨syscall notify 396f⟩

⟨syscall noted 398a⟩

⟨syscall segbrk 411a⟩

⟨syscall segattach 411b⟩

⟨syscall segdetach 413b⟩

⟨syscall segfree 414a⟩

⟨syscall brk 102a⟩

⟨syscall rendezvous 422e⟩

E.14.12 processes/syssema.c

⟨syssema.c 679b⟩≡

<kernel basic includes 35a>

```
/*
 * The implementation of semaphores is complicated by needing
 * to avoid rescheduling in syssemrelease, so that it is safe
 * to call from real-time processes. This means syssemrelease
 * cannot acquire any qlocks, only spin locks.
 *
 * Semacquire and semrelease must both manipulate the semaphore
 * wait list. Lock-free linked lists only exist in theory, not
 * in practice, so the wait list is protected by a spin lock.
 *
 * The semaphore value *addr is stored in user memory, so it
 * cannot be read or written while holding spin locks.
 *
 * Thus, we can access the list only when holding the lock, and
 * we can access the semaphore only when not holding the lock.
 * This makes things interesting. Note that sleep's condition function
 * is called while holding two locks - r and up->rlock - so it cannot
 * access the semaphore value either.
 *
 * An acquirer announces its intention to try for the semaphore
 * by putting a Sema structure onto the wait list and then
 * setting Sema.waiting. After one last check of semaphore,
 * the acquirer sleeps until Sema.waiting==0. A releaser of n
 * must wake up n acquirers who have Sema.waiting set. It does
 * this by clearing Sema.waiting and then calling wakeup.
 *
 * There are three interesting races here.
 *
 * The first is that in this particular sleep/wakeup usage, a single
 * wakeup can rouse a process from two consecutive sleeps!
 * The ordering is:
 *
 * (a) set Sema.waiting = 1
 * (a) call sleep
 * (b) set Sema.waiting = 0
 * (a) check Sema.waiting inside sleep, return w/o sleeping
 * (a) try for semaphore, fail
 * (a) set Sema.waiting = 1
 * (a) call sleep
 * (b) call wakeup(a)
 * (a) wake up again
 *
 * This is okay - semacquire will just go around the loop
 * again. It does mean that at the top of the for(;;) loop in
 * semacquire, phore.waiting might already be set to 1.
 *
 * The second is that a releaser might wake an acquirer who is
 * interrupted before he can acquire the lock. Since
 * release(n) issues only n wakeup calls -- only n can be used
 * anyway -- if the interrupted process is not going to use his
 * wakeup call he must pass it on to another acquirer.
 *
 * The third race is similar to the second but more subtle. An
 * acquirer sets waiting=1 and then does a final canacquire()
 * before going to sleep. The opposite order would result in
 * missing wakeups that happen between canacquire and
 * waiting=1. (In fact, the whole point of Sema.waiting is to
 * avoid missing wakeups between canacquire() and sleep().) But
```

```

* there can be spurious wakeups between a successful
* canacquire() and the following semdequeue(). This wakeup is
* not useful to the acquirer, since he has already acquired
* the semaphore. Like in the previous case, though, the
* acquirer must pass the wakeup call along.
*
* This is all rather subtle. The code below has been verified
* with the spin model /sys/src/9/port/semaphore.p. The
* original code anticipated the second race but not the first
* or third, which were caught only with spin. The first race
* is mentioned in /sys/doc/sleep.ps, but I'd forgotten about it.
* It was lucky that my abstract model of sleep/wakeup still managed
* to preserve that behavior.
*
* I remain slightly concerned about memory coherence
* outside of locks. The spin model does not take
* queued processor writes into account so we have to
* think hard. The only variables accessed outside locks
* are the semaphore value itself and the boolean flag
* Sema.waiting. The value is only accessed with cmpswap,
* whose job description includes doing the right thing as
* far as memory coherence across processors. That leaves
* Sema.waiting. To handle it, we call coherence() before each
* read and after each write.          - rsc
*/

```

<function semqueue 417e>

<function semdequeue 418a>

<function semwakeup 418b>

<function semrelease 418c>

<function canacquire 418d>

<function semawoke 418e>

<function semacquire 419a>

<function tsemacquire 419b>

<syscall semacquire 420a>

<syscall tsemacquire 420b>

<syscall semrelease 421a>

E.15 security/

E.15.1 security/portfns_security.h

E.15.2 security/auth.c

<auth.c 681>≡

<kernel basic includes 35a>

```

#include    <authsrv.h>

//now in core/
//char  *eve;
//int iseve(void) { }

<global hostdomain 527d>

<syscall fversion 527e>

<syscall fauth 528a>

<function userwrite 528b>

<function hostownerwrite 111c>

<function hostdomainwrite 529>

```

E.16 syscalls/

E.16.1 syscalls/systab.h

E.16.2 syscalls/syscallfmt.c

```

<syscallfmt.c 682a>≡
/*
 * Print functions for system call tracing.
 */
<kernel basic includes 35a>

#include "../port/systab.h"

<function fmtrwdata 439a>

<function fmtuserstring 439b>

<function syscallfmt 433g>

<function sysretfmt 437>

```

E.16.3 syscalls/systab.c

```

<systab.c 682b>≡
<kernel basic includes 35a>

#include "../port/systab.h"

<systab.c forward decl ??>

//coupling: debuggers/acid/conf/syscall
//coupling: debuggers/acid/conf/truss
<global systab 41f>
<global nsyscall 42a>

<global sysstab ??>

```

E.17 ../include/arch/arm/

E.17.1 ../include/arch/arm/ureg.h

```
<arm/ureg.h 683a>≡  
    typedef struct Ureg Ureg;  
    <struct Ureg(arm) 26c>
```

E.18 arch/arm/

E.18.1 arch/arm/dat_arch.h

```
<arch/arm/dat_arch.h 683b>≡  
  
    <struct Soc(arm) 54c>  
  
    extern Soc soc;
```

E.18.2 arch/arm/arm.h

```
<arch/arm/arm.h 683c>≡  
/*  
 * ARM-specific definitions for armv6 (arm11), armv7 (cortex-a7 and -a8)  
 * these are used in C and assembler  
 */  
  
/*  
 * Program Status Registers  
 */  
<type PsrMode(arm) 28a>  
  
<type PsrDisable(arm) 28b>  
  
/*  
 * Coprocessors  
 */  
<type Coprocessors(arm) 29a>  
  
/*  
 * Primary (CRn) CpSC registers.  
 */  
<type CpSC primary registers(arm) 29b>  
  
// CpSC secondary registers  
  
<type CpID secondary registers(arm) 473h>  
<type CpID opcode2(arm) 473i>  
<type CpID CpIDid op1(arm) 486g>  
  
<type CpCONTROL opcode2(arm) 29c>  
<type CpCONTROL CpMainctl(arm) 29d>  
<type CpCONTROL CpAuxctl(arm) 478c>  
  
<type CpTTB(arm) 474e>  
  
<type CpFSR(arm) 149b>
```

```

<type CpCACHE secondary registers(arm) 469c>
<type CpCACHE opcode2(arm) 469d>

<type CpTLB secondary registers(arm) 157h>
<type CpTLB opcode2(arm) 157i>

<type CpCLD secondary registers(arm) 476a>
<type CpCLD opcode2(arm) 476b>

<type CpTIMER(arm) 475i>

<type CpSPM secondary registers(arm) 448d>
<type CpSPM opcode2(arm) 448e>

// For MCRR

<type CpCACHERANGE opcode2(arm) 468f>

// Misc

<type CpTTB cache control bits(arm) 474f>

/*
 * MMU page table entries.
 */
<constant Mbz(arm) 155e>

<constant Fault(arm) 156c>

<type PageDirGranularity(arm) 155d>
<type PageTableGranularity(arm) 155f>

<type PageTableEntryAttribute1(arm) 468h>

<constant Dom0(arm) 156d>

<type PageTableEntryAttribute2(arm) 474b>

<type PageTableEntryAttribute3(arm) 156b>

<type DAC(arm) 156a>

<macro F(arm) 156g>

<macro AP(arm) 156f>
<macro L1AP(arm) 156e>
/* L2AP differs between armv6 and armv7 -- see l2ap in arch*.c */
<macro DAC(arm) 156h>

<constant HVECTORS(arm) 28c>

```

E.18.3 arch/arm/arminstr.ha

```

<arch/arm/arminstr.ha 684>≡
/*
 * armv6/v7 machine assist, definitions
 *
 * loader uses R11 as scratch.

```

```

*/

⟨macro PADDR bis(arm) 40e⟩
⟨macro L1X bis(arm) 155c⟩

/*
 * new instructions
 */
⟨instruction ISB(arm) 469a⟩
⟨instruction DSB(arm) 469b⟩
⟨instruction BARRIERS(arm) 473e⟩

⟨instruction MCRR(arm) 468g⟩
⟨instruction MRRC(arm) 716b⟩

⟨instruction LDREX(arm) 484d⟩
⟨instruction STREX(arm) 484e⟩
⟨instruction CLREX(arm) 484f⟩

⟨instruction CPSID(arm) 486f⟩

⟨instruction CPUID(arm) 474a⟩

```

E.18.4 arch/arm/io.h

```

⟨arch/arm/io.h 685a⟩≡
// could be in dat_interrupts.h?
⟨enum IRQ(arm) 151d⟩
⟨macro IRQDMA(arm) 789a⟩

⟨enum Dma(arm) 789b⟩

⟨enum Power(arm) 500e⟩

// could be in dat_time.h?
⟨enum Clock(arm) 690b⟩

```

E.18.5 arch/arm/coproc.c

```

⟨enum _anon_(arm) 685b⟩≡ (687b)
enum {
    /* alternates:
     * 0xe12fff1e BX (R14); last e is R14
     * 0xe28ef000 B 0(R14); second e is R14 (ken)
     */
    RETinst = 0xe1a0f00e, /* MOV R14, R15 */

    Opmask = MASK(3),
    Regmask = MASK(4),
};

⟨function setupcpop(arm) 685c⟩≡ (687b)
// setup coprocessor operation
static void
setupcpop(ulong instr[2], ulong opcode, int cp, int op1, int crn, int crm,
          int op2)
{
    ulong instrsz[2];

```

```

    op1 &= Opmask;
    op2 &= Opmask;
    crn &= Regmask;
    crm &= Regmask;
    cp  &= Regmask;
    instr[0] = opcode | op1 << 21 | crn << 16 | cp << 8 | op2 << 5 | crm;
    instr[1] = RETinst;

    cachedwbse(instr, sizeof instrsz);
    cacheiinv();
}

⟨function cprd(arm) 686a⟩≡ (687b)
    ulong
    cprd(int cp, int op1, int crn, int crm, int op2)
    {
        int s, r;
        volatile ulong instr[2];
        Pufv fp;

        s = arch_splhi();
        /*
         * MRC.  return value will be in R0, which is convenient.
         * Rt will be R0.
         */
        // MRC 0x e(Mxx) e(Always) 0 (MRC) .... 10 (part of Mxxx opcode)
        setupcpop(instr, 0xee100010, cp, op1, crn, crm, op2);
        fp = (Pufv)instr;
        r = fp();
        arch_splx(s);
        return r;
    }

⟨function cpwr(arm) 686b⟩≡ (687b)
    void
    cpwr(int cp, int op1, int crn, int crm, int op2, ulong val)
    {
        int s;
        volatile ulong instr[2];
        Pvfu fp;

        s = arch_splhi();
        // MCR 0x e(Mxx) e(Always) 0 (MCR) .... 10 (part of Mxxx opcode)
        setupcpop(instr, 0xee000010, cp, op1, crn, crm, op2); /* MCR, Rt is R0 */
        fp = (Pvfu)instr;
        fp(val);
        arch_coherence();
        arch_splx(s);
    }

⟨function cprdsc(arm) 686c⟩≡ (687b)
    ulong
    cprdsc(int op1, int crn, int crm, int op2)
    {
        return cprd(CpSC, op1, crn, crm, op2);
    }

```

```

⟨function cpwrsc(arm) 687a⟩≡ (687b)
void
cpwrsc(int op1, int crn, int crm, int op2, ulong val)
{
    cpwr(CpSC, op1, crn, crm, op2, val);
}

⟨arch/arm/coproc.c 687b⟩≡
/*
 * ARM co-processors
 * mainly to cope with ARM hard-wiring register numbers into instructions.
 *
 * CP15 (system control) is the one that gets used the most in practice.
 * these routines must be callable from KZERO space or the 0 segment.
 */
⟨kernel basic includes 35a⟩

#include "io.h"
#include "arm.h"

⟨enum _anon_(arm) 685b⟩

// pointer to ulong function void
typedef ulong (*Pufv)(void);
// pointer to void function ulong
typedef void (*Pvfu)(ulong);

⟨function setupcpop(arm) 685c⟩

⟨function cprd(arm) 686a⟩

⟨function cpwr(arm) 686b⟩

⟨function cprdsc(arm) 686c⟩

⟨function cpwrsc(arm) 687a⟩

/* floating point */

⟨function setupfpctlop(arm) 458e⟩

⟨function fprd(arm) 458f⟩

⟨function fpwr(arm) 459a⟩

⟨function setupfpop(arm) 459b⟩

⟨function fpsavereg(arm) 460a⟩

⟨function fprestreg(arm) 459c⟩

```

E.18.6 arch/arm/vcore.c

```

⟨enum _anon_((arch/arm/vcore.c)(arm)) 687c⟩≡ (691d)
enum {
    Read      = 0x00>>2,
    Write     = 0x00>>2,
    Peek      = 0x10>>2,
    Sender    = 0x14>>2,

```



```

    Status      = 0x18>>2,
    Full        = 1<<31,
    Empty       = 1<<30,
    Config      = 0x1C>>2,

    NRegs       = 0x20>>2,
};

⟨enum _anon_((arch/arm/vcore.c)2(arm)) 688a)≡ (691d)
enum {
    ChanFb      = 1,
    ChanProps   = 8,
    ChanMask    = 0xF,
};

⟨enum _anon_((arch/arm/vcore.c)3(arm)) 688b)≡ (691d)
enum {
    Req         = 0x0,
    RspOk       = 0x80000000,
    TagResp     = 1<<31,
};

⟨enum _anon_((arch/arm/vcore.c)4(arm)) 688c)≡ (691d)
enum {
    TagGetfwrev = 0x00000001,
    TagGetrev   = 0x00010002,

    TagGetmac   = 0x00010003,
    TagGetram   = 0x00010005,

    TagGetpower = 0x00020001,
    TagSetpower = 0x00028001,
    Powerwait   = 1<<1,

    TagGetclkspd= 0x00030002,
    TagGetclkmax= 0x00030004,
    TagSetclkspd= 0x00038002,

    TagGettemp  = 0x00030006,

    TagFballoc  = 0x00040001,
    TagFbfree   = 0x00048001,
    TagFbblank  = 0x00040002,

    TagGetres   = 0x00040003,
    TagSetres   = 0x00048003,
    TagGetvres  = 0x00040004,
    TagSetvres  = 0x00048004,
    TagGetdepth = 0x00040005,
    TagSetdepth = 0x00048005,
    TagGetrgb   = 0x00044006,
    TagSetrgb   = 0x00048006,
};

⟨struct Prophdr(arm) 688d)≡ (691d)
struct Prophdr {
    u32int len;
    u32int req;
    u32int tag;
    u32int tagbuflen;
};

```

```

    u32int taglen;
    u32int data[1];
};

⟨function vcwrite(arm) 689a⟩≡ (691d)
    static void
    vcwrite(uint chan, int val)
    {
        u32int *r;

        r = (u32int*)MAILBOX + NRegs;
        val &= ~ChanMask;
        while(r[Status] & Full)
            ;
        arch_coherence();
        r[Write] = val | chan;
    }

⟨function vcread(arm) 689b⟩≡ (691d)
    static int
    vcread(uint chan)
    {
        u32int *r;
        int x;

        r = (u32int*)MAILBOX;
        do{
            while(r[Status]&Empty)
                ;
            arch_coherence();
            x = r[Read];
        }while((x&ChanMask) != chan);
        return x & ~ChanMask;
    }

⟨function vcreq(arm) 689c⟩≡ (691d)
    /*
     * Property interface
     */

    static int
    vcreq(int tag, void *buf, int vallen, int rsplen)
    {
        uintptr r;
        int n;
        Prophdr *prop = (Prophdr*)(VCBUFFER);
        uintptr aprop;
        static bool busaddr = true;

        if(rsplen < vallen)
            rsplen = vallen;
        rsplen = (rsplen+3) & ~3;

        n = sizeof(Prophdr) + rsplen + 8;
        memset(prop, 0, n);
        prop->len = n;
        prop->req = Req;
        prop->tag = tag;
        prop->tagbuflen = rsplen;
        prop->taglen = vallen;
    }

```

```

if(vallen > 0)
    memmove(prop->data, buf, vallen);
cachedwbinvse(prop, prop->len);
for(;;){
    apropos = busaddr? dmaaddr(prop) : PTR2UINT(prop);
    vcwrite(ChanProps, apropos);
    r = vread(ChanProps);
    if(r == apropos)
        break;
    if(!busaddr)
        return -1;
    busaddr = false;
}
if(prop->req == RspOk &&
    prop->tag == tag &&
    (prop->taglen & TagResp)) {
    if((n = prop->taglen & ~TagResp) < rsplen)
        rsplen = n;
    memmove(buf, prop->data, rsplen);
}else
    rsplen = -1;

return rsplen;
}

```

*<function getethermac(*arm*) 690a>* ≡ (691d)

```

/*
 * Get ethernet address (as hex string)
 * [not reentrant]
 */
char *
getethermac(void)
{
    uchar ea[8];
    char *p;
    int i;
    static char buf[16];

    memset(ea, 0, sizeof ea);
    vcreq(TagGetmac, ea, 0, sizeof ea);
    p = buf;
    for(i = 0; i < 6; i++)
        p += sprintf(p, "%.2x", ea[i]);
    return buf;
}

```

*<enum Clock(*arm*) 690b>* ≡ (685a)

```

enum {
    ClkEmmc      = 1,
    ClkUart,
    ClkArm,
    ClkCore,
    ClkV3d,
    ClkH264,
    ClkIsp,
    ClkSdram,
    ClkPixel,
    ClkPwm,
};

```

*<function getclkrate(*arm*) 691a>*≡ (691d)

```
/*
 * Get clock rate
 */
ulong
getclkrate(int clkid)
{
    u32int buf[2];

    buf[0] = clkid;
    if(vcreq(TagGetclkspd, buf, sizeof(buf[0]), sizeof(buf)) != sizeof buf)
        return 0;
    return buf[1];
}
```

*<function setclkrate(*arm*) 691b>*≡ (691d)

```
/*
 * Set clock rate to hz (or max speed if hz == 0)
 */
void
setclkrate(int clkid, ulong hz)
{
    u32int buf[2];

    buf[0] = clkid;
    if(hz != 0)
        buf[1] = hz;
    else if(vcreq(TagGetclkmax, buf, sizeof(buf[0]), sizeof(buf)) != sizeof buf)
        return;
    vcreq(TagSetclkspd, buf, sizeof(buf), sizeof(buf));
}
```

*<function getcputemp(*arm*) 691c>*≡ (691d)

```
/*
 * Get cpu temperature
 */
uint
getcputemp(void)
{
    u32int buf[2];

    buf[0] = 0;
    if(vcreq(TagGettemp, buf, sizeof(buf[0]), sizeof(buf)) != sizeof buf)
        return 0;
    return buf[1];
}
```

<arch/arm/vcore.c 691d>≡

<kernel basic includes 35a>

```
/*
 * Mailbox interface with videocore gpu
 */
```

*<constant MAILBOX(*arm*) 32b>*

```
typedef struct Fbinfo Fbinfo;
typedef struct Prophdr Prophdr;
```

*<enum _anon_((arch/arm/vcore.c)(*arm*)) 687c>*

*<enum _anon_((arch/arm/vcore.c)2(*arm*)) 688a>*

<enum _anon_((arch/arm/vcore.c)3(arm)) 688b>

<enum _anon_((arch/arm/vcore.c)4(arm)) 688c>

<struct Fbinfo(arm) 32c>

<struct Prophdr(arm) 688d>

<function vcwrite(arm) 689a>

<function vcread(arm) 689b>

<function vcreq(arm) 689c>

<function fbdefault(arm) 339a>

<function fbinit(arm) 338b>

<function fbblank(arm) 339b>

<function setpower(arm) 501a>

<function getpower(arm) 501b>

<function getethermac(arm) 690a>

<function getboardrev(arm) 250g>

<function getfirmware(arm) 251a>

<function getramsize(arm) 241b>

<function getclkrate(arm) 691a>

<function setclkrate(arm) 691b>

<function getcputemp(arm) 691c>

E.19 concurrency/arm/

E.19.1 concurrency/arm/spl.s

<concurrency/arm/spl.s 692>≡

#include "mem.h"

#include "arm.h"

#include "arminstr.ha"

<function arch_splhi(arm) 56a>

<function arch_spllo(arm) 56b>

<function arch_spllx(arm) 56c>

TEXT spldone(SB), 1, \$0

RET

/ end marker for devkprof.c */*

```
<function arch_islo(arm) 56d>
```

```
<function arm_arch_coherence(arm) 486d>
```

```
// FIQ
```

```
<function splfhi(arm) 500d>
```

E.19.2 concurrency/arm/tas_raspi1.s

```
<concurrency/arm/tas_raspi1.s 693a>≡  
TEXT    _tas(SB), $-4  
<function arch_tas(raspberrypi1)(arm) 57>
```

E.19.3 concurrency/arm/tas_raspi2.s

```
<concurrency/arm/tas_raspi2.s 693b>≡  
#include "mem.h"  
#include "arm.h"  
#include "arminstr.ha"  
  
<instruction DMB(arm) 484g>  
  
/* tas/cas strex debugging limits; started at 10000 */  
#define MAXSC 100000  
  
TEXT    _tas(SB), $-4          /* _tas(ulong *) */  
<function arch_tas(raspberrypi2)(arm) 485a>
```

E.19.4 concurrency/arm/concurrency_raspi1.c

```
<function ainc(arm) 693c>≡ (694a)  
// overwrite libc!  
int  
ainc(int *p)  
{  
    int s, v;  
  
    s = arch_splhi();  
    v = ++*p;  
    arch_splx(s);  
    return v;  
}  
  
<function adec(arm) 693d>≡ (694a)  
int  
adec(int *p)  
{  
    int s, v;  
  
    s = arch_splhi();  
    v = --*p;  
    arch_splx(s);  
    return v;  
}
```

```

<concurrency/arm/concurrency_raspi1.c 694a>≡
/*
 * atomic ops
 * make sure that we don't drag in the C library versions
 */
<kernel basic includes 35a>

<function arch_xdec(raspberry pi1)(arm) 58b>

<function arch_xinc(raspberry pi1)(arm) 58a>

<function ainc(arm) 693c>

<function adec(arm) 693d>

<function cas32(arm) 417d>

<function arch_cmpswap(raspberry pi1)(arm) 417c>

```

E.19.5 concurrency/arm/concurrency_raspi2.c

```

<concurrency/arm/concurrency_raspi2.c 694b>≡
<kernel basic includes 35a>

// in libc/arm/atom.s (use LDREX, STREX special instructions)
extern long _xdec(long *p);
extern long _xinc(long *p);

long
arch_xdec(long *p)
{
    return _xdec(p);
}

void
arch_xinc(long *p)
{
    _xinc(p);
}

int
arch_cmpswap(long *addr, long old, long new)
{
    return cas((ulong*)addr, old, new);
}

```

E.20 core/arm/

E.20.1 core/arm/dat.h

```

<constant STAGESIZE(arm) 694c>≡ (695a)
// was a default in portdat.h
#define STAGESIZE 64 // for struct Uart

```

```

<core/arm/dat.h 695a>≡
// pad's memory pointers (used in Lock so must be early)
<pad memory pointer types(arm) 40a>
//TODO: need to update port/ code to use the new pointer types
typedef void* virt_addr3;
typedef void* kern_addr3;

#include "dat_forward.h"
#include "../port/portdat_forward.h"
<constant STAGESIZE(arm) 694c>

<dat.h includes(arm) 35b>

// DO NOT switch those declarations. 5c allocates R10
// for the first 'extern register' declaration seen in a file
// and R9 for the second one. See Compiler.nw.
// Then mem.h relies on this to define the UP and CPU macros used
// inside assembly code.
<global cpu(arm) 48f>
<global up(arm) 51d>

// in main.c (used in mmu.c)
extern ulong memsize;

<constant KMESSGSIZE(arm) 581c>

// TODO: put that in portdat.h or in netif.h?
#pragma varargck type "I" uchar*
#pragma varargck type "V" uchar*
#pragma varargck type "E" uchar*
#pragma varargck type "M" uchar*

```

E.20.2 core/arm/dat_core.h

```

<core/arm/dat_core.h 695b>≡

<type PTE(arm) 156l>

<struct Arch_Cpu(arm) 48e>

```

E.20.3 core/arm/dat_forward.h

```

<core/arm/dat_forward.h 695c>≡
// could be in portdat_forward.h
typedef struct Arch_FPsave Arch_FPsave;
typedef struct Ureg Ureg;
#pragma incomplete Ureg

typedef struct Soc Soc;

```

E.20.4 core/arm/fns.h

```

<core/arm/fns.h 695d>≡

<fns.h includes(arm) 35c>

// not classified yet

```



```

extern void arch_cpuidprint(void);

// different signatures in archs or macro so cant factorize in portdat_XXX.h
extern void arch_touser(uintptr);
extern int arch_cmpswap(long*, long, long);
<macro getpgcolor(arm) 492f>
<macro arch_cycles(arm) 448c>
<macro arch_countpagerefs(arm) 497a>
<macro arch_intrenable(arm) 153a>

extern int arch__notify(Ureg*);
extern void arch__kexit(Ureg*);

extern void dumpregs(Ureg*);


extern void mmuinit1(void*);
extern void mmuinvalidate(void);
extern void mmuinvalidateaddr(u32int);


extern void irgenable(int, void (*)(Ureg*, void*), void*);

extern void intrcpushutdown(void);
extern void intrshutdown(void);
extern void intrsoff(void);


extern void archreboot(void);
extern void archreset(void);


extern void clockinit(void);
extern void clockshutdown(void);


extern int cas32(void*, u32int, u32int);
extern int cas(ulong*, ulong, ulong);

extern char *cputype2name(char *buf, int size);

extern void armtimerset(int);


extern void cachedwb(void);
extern void cachedwbinv(void);
extern void cachedinvse(void*, int);
extern void cachedwbse(void*, int);
extern void cachedwbinvse(void*, int);
extern void cacheiinv(void);
extern void cacheuwbinv(void);


// coproc.c
extern ulong cprd(int cp, int op1, int crn, int crm, int op2);
extern ulong cprdsc(int op1, int crn, int crm, int op2);
extern void cpwr(int cp, int op1, int crn, int crm, int op2, ulong val);
extern void cpwrsc(int op1, int crn, int crm, int op2, ulong val);


// dma.c
extern uintptr dmaaddr(void *va);
extern void dmastart(int, int, int, void*, void*, int);

```

```

extern int dmawait(int);

// vcore.c
extern int fbblank(int);
extern void* fbinit(bool, int*, int*, int*);

extern void fpon(void);
extern ulong fprd(int fpreg);
extern void fprestreg(int fpreg, uulong val);
extern void fpsave(Arch_FPsave *);
extern ulong fpsavereg(int fpreg, uulong *fpp);
extern void fpwr(int fpreg, ulong val);

extern u32int fsrget(void);
extern u32int farget(void);
extern u32int ifsrget(void);

// vcore.c
extern uint getboardrev(void);
extern ulong getclkrate(int);
extern uint getcputemp(void);
extern char *getethermac(void);
extern uint getfirmware(void);
extern void getramsize(Confmem*);

extern int getncpus(void);
extern int getpower(int);

extern void gpiosel(uint, int);
extern void gpiopullup(uint);
extern void gpiopulloff(uint);
extern void gpiopulldown(uint);

extern void i2cread(uint, void*, int);
extern void i2cwrite(uint, void*, int);

extern void okay(int);

extern int l2ap(int);
extern void l2cacheuwbinv(void);

<macro sdfree(arm) ??>
<macro sdmalloc(arm) ??>

extern void setclkrate(int, ulong);
extern void setpower(int, int);
extern void setr13(int, u32int*);

extern void spirw(uint, void*, int);
extern int splfhi(void);
extern int splflo(void);

extern int startcpus(uint);
extern void stopcpu(uint);

extern void uartconsinit(void);

```

```

extern void vectors(void);
extern void vtable(void);

extern void wdogoff(void);

/*
 * floating point emulation
 */
extern int fpiarm(Ureg*);
extern int fpudevprocio(Proc*, void*, long, uintptr, int);
extern void fpuinit(void);
extern void fpunoted(void);
extern void fpunotify(Ureg*);
extern void fpuprocrestore(Proc*);
extern void fpuprocsave(Proc*);
extern void fpusysprocsetup(Proc*);
extern void fpusysrfork(Ureg*);
extern void fpusysrforkchild(Proc*, Ureg*, Proc*);
extern int fpuemu(Ureg*);

/*
 * Miscellaneous machine dependent stuff.
 */
extern char* getenv(char*, char*, int);

uintptr mmukmap(uintptr, uintptr, usize);
uintptr mmukunmap(uintptr, uintptr, usize);

⟨macro PTR2UINT(arm) 41a⟩
⟨macro UINT2PTR(arm) 41b⟩

⟨macro KADDR(arm) 40h⟩
⟨macro PADDR(arm) 40d⟩

```

E.21 buses/arm/

E.22 devices/

E.22.1 devices/keyboard/arm/kbd.c

```

⟨devices/keyboard/arm/kbd.c 698a⟩≡
  ⟨kernel basic includes 35a⟩

// In the Raspberry Pi, the keyboard is handled by usbd
// with help from /dev/kbin (which then calls functions from portkbd
// which calls arch_setleds below).

⟨function arch_setleds(arm) 333a⟩

```

E.22.2 devices/mouse/arm/mouse.c

```

⟨devices/mouse/arm/mouse.c 698b⟩≡
  ⟨kernel basic includes 35a⟩

```

```
// In the Raspberry Pi, the mouse is handled by usbd
// with help from /dev/mousein.

// empty callbacks for portmouse.c

void arch_ps2mouse(void) { }
void arch_setaccelerated(void) { }
void arch_setlinear(void) { }
void arch_setres(int n) { USED(n); }
void arch_setintellimouse(void) { }
void arch_resetmouse(void) { }
```

E.22.3 devices/sys/arm/devarch.c

```
<enum _anon_((devices/sys/arm/devarch.c)(arm)) 699a>≡ (702c)
enum {
    Qdir = 0,
    Qbase,

    Qmax = 16,
};

<global readfn(arm) 699b>≡ (702c)
static Rdwrfn *readfn[Qmax];

<global writefn(arm) 699c>≡ (702c)
static Rdwrfn *writefn[Qmax];

<global archdir(arm) 699d>≡ (702c)
static Dirtab archdir[Qmax] = {
    ".",      { Qdir, 0, QTDIR }, 0, 0555,
};

<global archwlock(arm) 699e>≡ (702c)
Lock archwlock; /* the lock is only for changing archdir */

<global narchdir(arm) 699f>≡ (702c)
int narchdir = Qbase;

<function addarchfile(arm) 699g>≡ (702c)
/*
 * Add a file to the #P listing. Once added, you can't delete it.
 * You can't add a file with the same name as one already there,
 * and you get a pointer to the Dirtab entry so you can do things
 * like change the Qid version. Changing the Qid path is disallowed.
 */
Dirtab*
addarchfile(char *name, int perm, Rdwrfn *rdfn, Rdwrfn *wrfn)
{
    int i;
    Dirtab d;
    Dirtab *dp;

    memset(&d, 0, sizeof d);
    strcpy(d.name, name);
    d.perm = perm;

    lock(&archwlock);
    if(narchdir >= Qmax){
```

```

        unlock(&archwlock);
        return nil;
    }

    for(i=0; i<narchdir; i++)
        if(strcmp(archdir[i].name, name) == 0){
            unlock(&archwlock);
            return nil;
        }

    d.qid.path = narchdir;
    archdir[narchdir] = d;
    readfn[narchdir] = rdfn;
    writefn[narchdir] = wrfn;
    dp = &archdir[narchdir++];
    unlock(&archwlock);

    return dp;
}

⟨function archattach(arm) 700a)≡ (702c)
    static Chan*
    archattach(char* spec)
    {
        return devattach('P', spec);
    }

⟨function archwalk(arm) 700b)≡ (702c)
    Walkqid*
    archwalk(Chan* c, Chan *nc, char** name, int nname)
    {
        return devwalk(c, nc, name, nname, archdir, narchdir, devgen);
    }

⟨function archstat(arm) 700c)≡ (702c)
    static int
    archstat(Chan* c, uchar* dp, int n)
    {
        return devstat(c, dp, n, archdir, narchdir, devgen);
    }

⟨function archopen(arm) 700d)≡ (702c)
    static Chan*
    archopen(Chan* c, int omode)
    {
        return devopen(c, omode, archdir, narchdir, devgen);
    }

⟨function archclose(arm) 700e)≡ (702c)
    static void
    archclose(Chan*)
    {
    }

⟨function archread(arm) 700f)≡ (702c)
    static long
    archread(Chan *c, void *a, long n, vlong offset)
    {
        Rdwrfn *fn;

```

```

switch((ulong)c->qid.path){
case Qdir:
    return devdirread(c, a, n, archdir, narchdir, devgen);

default:
    if(c->qid.path < narchdir && (fn = readfn[c->qid.path]))
        return fn(c, a, n, offset);
    error(Eperm);
    break;
}

return 0;
}

⟨function archwrite(arm) 701a)≡ (702c)
static long
archwrite(Chan *c, void *a, long n, vlong offset)
{
    Rdwrfn *fn;

    if(c->qid.path < narchdir && (fn = writefn[c->qid.path]))
        return fn(c, a, n, offset);
    error(Eperm);

    return 0;
}

⟨global archdevtab(arm) 701b)≡ (702c)
Dev archdevtab = {
    .dc = 'P',
    .name = "arch",

    .reset = devreset,
    .init = archinit,
    .shutdown = devshutdown,
    .attach = archattach,
    .walk = archwalk,
    .stat = archstat,
    .open = archopen,
    .create = devcreate,
    .close = archclose,
    .read = archread,
    .bread = devbread,
    .write = archwrite,
    .bwrite = devbwrite,
    .remove = devremove,
    .wstat = devwstat,
};

⟨function cputyperead(arm) 701c)≡ (702c)
static long
cputyperead(Chan*, void *a, long n, vlong offset)
{
    char name[64], str[128];

    cputype2name(name, sizeof name);
    snprintf(str, sizeof str, "ARM %s %d\n", name, cpu->cpumhz);
    return readstr(offset, a, n, str);
}

```

```

⟨function cputempread(arm) 702a⟩≡ (702c)
static long
cputempread(Chan*, void *a, long n, vlong offset)
{
    char str[16];

    snprintf(str, sizeof str, "%ud\n", (getcputemp()+500)/1000);
    return readstr(offset, a, n, str);
}

⟨function archinit(arm) 702b⟩≡ (702c)
void
archinit(void)
{
    addarchfile("cputype", 0444, cputyperead, nil);
    addarchfile("cputemp", 0444, cputempread, nil);
}

⟨devices/sys/arm/devarch.c 702c⟩≡
⟨kernel basic includes 35a⟩

#include "io.h"

⟨enum _anon_((devices/sys/arm/devarch.c)(arm)) 699a⟩

typedef long Rdwrfn(Chan*, void*, long, vlong);
void archinit(void);

⟨global readfn(arm) 699b⟩
⟨global writefn(arm) 699c⟩

⟨global archdir(arm) 699d⟩

⟨global archwlock(arm) 699e⟩
⟨global narchdir(arm) 699f⟩

⟨function addarchfile(arm) 699g⟩

⟨function archattach(arm) 700a⟩

⟨function archwalk(arm) 700b⟩

⟨function archstat(arm) 700c⟩

⟨function archopen(arm) 700d⟩

⟨function archclose(arm) 700e⟩

⟨function archread(arm) 700f⟩

⟨function archwrite(arm) 701a⟩

⟨global archdevtab(arm) 701b⟩

⟨function cputyperead(arm) 701c⟩

⟨function cputempread(arm) 702a⟩

⟨function archinit(arm) 702b⟩

```

E.23 init/arm/

E.23.1 init/arm/start.s

```
<init/arm/start.s 703a>≡
/*
 * Common startup for armv6 and armv7
 * The rest of l.s has been moved to armv[67].s
 */
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"

<function _start(arm) 235>
```

E.23.2 init/arm/startv6.s

```
<init/arm/startv6.s 703b>≡
/*
 * Broadcom bcm2835 SoC, as used in Raspberry Pi
 * arm1176jzf-s processor (armv6)
 */
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"

<function armstart(raspberry pi1)(arm) 236>

<function _startpg(raspberry pi1)(arm) 238a>

<function arch_idlehands(raspberry pi1)(arm) 212c>
```

E.23.3 init/arm/startv7.s

```
<init/arm/startv7.s 703c>≡
/*
 * Broadcom bcm2836 SoC, as used in Raspberry Pi 2
 * 4 x Cortex-A7 processor (armv7)
 */
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"

<instruction WFI_EQ(arm) 484a>

<function armstart(raspberry pi2)(arm) 476c>

<function _startpg(raspberry pi2)(arm) 478d>

<function cpureset(raspberry pi2)(arm) 480b>

<function _startpg2(raspberry pi2)(arm) 482a>

<function cpidget(raspberry pi2)(arm) 483d>
```


<function arch_idlehands(raspberry pi2)(arm) 483e>

E.23.4 init/arm/main.c

<function getconf(arm) 704a>≡ (704c)

```
char*
getconf(char *name)
{
    USED(name);
    //int i;
    //
    //i = findconf(name);
    //if(i >= 0)
    //    return confval[i];
    return nil;
}
```

<function main_arch_isaconfig(arm) 704b>≡ (704c)

```
/*
 * stub for ../omap/devether.c
 */
int
main_arch_isaconfig(char *class, int ctnrno, ISAConf *isa)
{
    char cc[32], *p;
    int i;

    if(strcmp(class, "ether") != 0)
        return 0;
    snprintf(cc, sizeof cc, "%s%d", class, ctnrno);
    p = getconf(cc);
    if(p == nil)
        return (ctnrno == 0);
    isa->type = "";
    isa->nopt = tokenize(p, isa->opt, NISAOPT);
    for(i = 0; i < isa->nopt; i++){
        p = isa->opt[i];
        if(cistrncmp(p, "type=", 5) == 0)
            isa->type = p + 5;
    }
    return 1;
}
```

<init/arm/main.c 704c>≡
<kernel basic includes 35a>

```
#include "io.h"
// for mainmem and imagmem
#include <pool.h>

// initcode binary
#include "init.h"

// rebootcode binary
#include "reboot.h"

// in portscreen.h
extern void swcursor_init(void);
```

```

⟨constant Minfirmrev(arm) 250e⟩
⟨constant Minfirmdate(arm) 250f⟩

// part of a trick to remove some backward dependencies
int devcons_print(char*, ...);
int devcons_iprint(char*, ...);
int devcons_pprint(char*, ...);
void devcons_panic(char*, ...);
void devcons__assert(char*);
void proc_dumpaproc(Proc *p);
void proc_error(char*);
void proc_nexterror(void);
void proc_sched(void);
void proc_ready(Proc*);
void proc_sleep(Rendez*, int(*) (void*), void*);
void proc_tsleep(Rendez *r, int (*fn)(void*), void *arg, ulong ms);
Proc* proc_wakeup(Rendez*);
void proc_pexit(char *exitstr, bool freemem);
Proc* proc_proctab(int i);
int proc_postnote(Proc *p, int dolock, char *n, int flag);
void chan_cclose(Chan *c);

uvlong clock_arch_fastticks(uvlong *hz);
void clock_arch_delay(int millisecs);
void clock_arch_microdelay(int microsecs);
void trap_arch_dumpstack(void);
void arm_arch_coherence(void);
void main_arch_exit(int ispanic);
int main_arch_isaconfig(char *class, int ctrlno, ISAConf *isa);

//*****
// Configuration
//*****
// See globals in portdat_globals.h

// <conf>.c
extern Dev* conf_devtab[];
extern char* conffile;

⟨function getconf(arm) 704a⟩

//*****
// Boot parameters, see bootconf.c (not used by pad)
//*****

//*****
// Cpu init
//*****
⟨function arch__cpuinit(arm) 240b⟩

⟨function arch__cpu0init(arm) 240a⟩

⟨function machon(arm) 240c⟩

//*****
// Conf init
//*****

```

```

⟨global memsize(arm) 241e⟩

⟨function arch__confinit(arm) 242⟩

//*****
// First process init
//*****

⟨global sp(arm) 251b⟩

// kernel space instructions executed by first process
⟨function init0(arm) 252⟩

// user space instructions executed by first process
// see initcode in init.h (comes from ../port/initcode.c and init9.s

extern uintptr bootargs(uintptr base);
⟨function arch__userinit(arm) 251d⟩

//*****
// Shutdown/reboot
//*****

⟨function shutdown((init/arm/main.c)(arm)) 563a⟩

⟨function main_arch_exit(arm) 562e⟩

⟨function arch_reboot(arm) 563c⟩

//*****
// Misc
//*****

⟨function main_arch_isaconfig(arm) 704b⟩

⟨function arch_memorysummary(arm) 576a⟩

//*****
// Main entry point!
//*****

⟨function launchinit(arm) 478f⟩

⟨function main(arm) 238b⟩

```

E.23.5 init/arm/raspi1.c

```

⟨function archether(raspberry pi1)(arm) 706⟩≡ (707a)
int
archether(unsigned ctrlrno, Ether *ether)
{
    ether->type = "usb";
    ether->ctrlrno = ctrlrno;
    ether->irq = -1;
    ether->nopt = 0;
    return 1;
}

```

```

<init/arm/raspi1.c 707a>≡
/*
 * bcm2835 (e.g. original raspberry pi) architecture-specific stuff
 */
<kernel basic includes 35a>

#include "io.h"
#include "arm.h"

#include "../port/netif.h"
#include "../port/etherif.h"

<global soc(raspberry pi1)(arm) 54d>

<function cputype2name(raspberry pi1)(arm) 241a>

<function arch_cpuidprint(raspberry pi1)(arm) 240d>

<function getncpus(raspberry pi1)(arm) 479d>

<function startcpus(raspberry pi1)(arm) 479c>

<function archether(raspberry pi1)(arm) 706>

<function l2ap(raspberry pi1)(arm) 156j>

```

E.23.6 init/arm/raspi2.c

```

<init/arm/raspi2.c 707b>≡
/*
 * bcm2836 (e.g. raspberry pi 2) architecture-specific stuff
 */
<kernel basic includes 35a>

#include "io.h"
#include "arm.h"

#include "../port/netif.h"
#include "../port/etherif.h"

typedef struct Mbox Mbox;
typedef struct Mboxes Mboxes;

<constant ARMLocal(arm) 475c>

<global soc(raspberry pi2)(arm) 473f>

<struct Mbox(arm) 475a>
<struct Mboxes(arm) 475b>

enum {
    Mboxregs    = 0x80
};

static Lock startlock[MAXCPUS + 1];

```

```

// in startv7.s
extern u32int cpidget(void);

⟨function cputype2name(raspberry pi2)(arm) 483c⟩

⟨function arch_cpuidprint(raspberry pi2)(arm) 483b⟩

⟨function getncpus(raspberry pi2)(arm) 479e⟩

⟨function startcpu(raspberry pi2)(arm) 480a⟩

⟨function startcpus(raspberry pi2)(arm) 479g⟩


int
archether(unsigned ctrlrno, Ether *ether)
{
    ether->type = "usb";
    ether->ctrlrno = ctrlrno;
    ether->irq = -1;
    ether->nopt = 0;
    return 1;
}

⟨function l2ap(raspberry pi2)(arm) 474d⟩


// in main.c
extern void arch__cpuinit(void);
extern void machon(uint);

⟨function cpustart(raspberry pi2)(arm) 482b⟩

```

E.23.7 init/user/preboot/arm/init9.s

E.23.8 init/arm/rebootcode.s

```

⟨init/arm/rebootcode.s 708⟩≡
/*
 * armv6/armv7 reboot code
 */
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"

⟨constant PTEDRAM(arm) 565b⟩

⟨instruction WFI(arm) 565c⟩
⟨instruction WFE(arm) 565d⟩

⟨function main(rebootcode.s)(arm) 565e⟩

```

E.24 memory/arm/

E.24.1 memory/arm/dat_memory.h

⟨memory/arm/dat_memory.h 709a⟩≡

```
/*
 * MMU stuff in proc
 */
⟨constant NCOLOR(arm) 492g⟩

⟨struct Arch_Proc(arm) 50⟩

/*
 * Fake kmap.
 */
typedef void          Arch_KMap;
⟨macro VA(arm) 157e⟩
```

E.24.2 memory/arm/mem.h

⟨memory/arm/mem.h 709b⟩≡

```
/*
 * Memory and machine-specific definitions.  Used in C and assembler.
 */

⟨constant KiB(arm) 34f⟩
⟨constant MiB(arm) 34g⟩
⟨constant GiB(arm) 34h⟩

/*
 * Sizes
 */
⟨constant BY2PG(arm) 31a⟩
⟨constant PGSHIFT(arm) 31b⟩

⟨constant MAXCPUS(arm) 49b⟩
⟨constant CPUSIZE(arm) 48b⟩
⟨constant L1SIZE(arm) 479a⟩

⟨macro STACKALIGN(arm) 251c⟩

/*
 * Magic registers
 */
⟨constant UP(arm) 140b⟩
⟨constant CPU(arm) 140a⟩

/*
 * Address spaces.
 * KTZERO is used by kprof and dumpstack (if any).
 *
 * KZERO is mapped to physical 0 (start of ram).
 *
 * vectors are at 0, plan9.ini is at KZERO+256 and is limited to 16K by
 * devenv.
 */
⟨constant KSEG0(arm) 36a⟩
```

```

⟨constant KSEGM(arm) 40g⟩

⟨constant KZERO(arm) 36b⟩

⟨constant CONFADDR(arm) 719e⟩

// L1 must be 16KiB aligned so this dictates the organization below
⟨constant CPUADDR(arm) 40c⟩
⟨constant L2(arm) 237b⟩
⟨constant VCBUFFER(arm) 338a⟩
⟨constant FIQSTKTOP(arm) 499c⟩
⟨constant L1(arm) 237a⟩

⟨constant KTZERO(arm) 36c⟩

⟨constant VIRTIO(arm) 31c⟩
⟨constant FRAMEBUFFER(arm) 32d⟩

⟨constant UZERO(arm) 37a⟩
⟨constant UTZERO(arm) 37b⟩
⟨macro UTROUND(arm) 121c⟩
⟨constant USTKTOP(arm) 37c⟩
⟨constant USTKSIZE(arm) 37d⟩
⟨constant TSTKTOP(arm) 123d⟩
⟨constant TSTKSIZ(arm) 123e⟩

⟨constant REBOOTADDR(arm) 562f⟩

/*
 * Legacy...
 */
//old: was 64 in bcm-latest, but then get panic in _allocb
⟨constant BLOCKALIGN(arm) 382e⟩
⟨constant KSTACK(arm) 197g⟩

/*
 * Sizes
 */
⟨constant BY2WD(arm) 26a⟩
⟨constant BY2V(arm) 26b⟩

⟨constant PAGETABMAPMEM(arm) 93c⟩
⟨constant PAGETABSIZE(arm) 94a⟩
⟨constant PAGEDIRSIZE(arm) 93a⟩
⟨constant SMALLPAGEDIRSIZE(arm) 96e⟩
⟨macro PPN(arm) 157d⟩

/*
 * Physical machine information from here on.
 * PHYS addresses as seen from the arm cpu.
 * BUS addresses as seen from the videocore gpu.
 */
⟨constant PHYSDRAM(arm) 40f⟩
⟨constant IOSIZE(arm) 31d⟩

```

E.24.3 memory/arm/mmu.c

```

⟨enum _anon_((memory/arm/mmu.c)(arm)) 710⟩≡ (711a)
enum {

```

```

    <constant L1lo(arm) 157a>
    <constant L1hi(arm) 157b>
};

<memory/arm/mmu.c 711a>≡
    <kernel basic includes 35a>
    #include "arm.h"

    <macro L1X(arm) 155a>
    <macro L2X(arm) 155b>
    <macro L2AP(arm) 156i>

    <constant L1ptedramattrs(arm) 468k>
    <constant L2ptedramattrs(arm) 468l>

    <enum _anon_((memory/arm/mmu.c)(arm)) 710>

    <function mmuinit(arm) 237c>

    <function mmuinit1(arm) 246a>

    <function mmul2empty(arm) 158a>

    <function mmul1empty(arm) 158e>

    <function arch_mmuswitch(arm) 200g>

    <function arch_flushmmu(arm) 158g>

    <function arch_mmurelease(arm) 157f>

    <function arch_putmmu(arm) 162>

    <function arch_cankaddr(arm) 41c>

    <function mmukmap(arm) 338c>

    <function arch_checkmmu(arm) 164b>

    <function arch_kmap(arm) 164c>

    <function arch_kunmap(arm) 164d>

```

E.24.4 memory/arm/mmuinvalidate.s

```

<memory/arm/mmuinvalidate.s 711b>≡
    #include "mem.h"
    #include "arm.h"
    #include "arminstr.ha"

    <function mmuinvalidate(arm) 157g>

    <function mmuinvalidateaddr(arm) 163b>

```

E.24.5 memory/arm/cache_raspi1.s

```

<memory/arm/cache_raspi1.s 711c>≡

```



```
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"

⟨constant CACHELINESZ(raspberry pi1)(ram) 469e⟩

⟨function cachexxx(raspberry pi1)(ram) 471⟩
```

E.24.6 memory/arm/cache_raspi2.s

```
⟨memory/arm/cache_raspi2.s 712a⟩≡
#include "mem.h"
#include "arm.h"
#include "arminstr.ha"

⟨constant CACHELINESZ(raspberry pi2)(ram) 486e⟩

⟨function cachexxx(raspberry pi2)(ram) 486h⟩
```

E.25 interrupts/arm/

E.25.1 interrupts/arm/trap.c

```
⟨enum _anon_((interrupts/arm/trap.c)(arm)) 712b⟩≡ (712c)
enum {
    Debug = 0,

    ⟨constant Nvec(arm) 243e⟩
    ⟨constant Fiqenable(arm) 500a⟩

    ⟨constant Localtimerint(arm) 491e⟩
    ⟨constant Localmboxint(arm) 491g⟩
    ⟨constant Localintpending(arm) 491f⟩
};

⟨interrupts/arm/trap.c 712c⟩≡
/*
 * traps, exceptions, interrupts, system calls.
 */
⟨kernel basic includes 35a⟩
#include "io.h"
#include "ureg.h"
#include "arm.h"

⟨constant INTREGS(arm) 31g⟩
⟨constant LOCALREGS(arm) 491h⟩

typedef struct Intregs Intregs;
typedef struct Vctl Vctl;
typedef struct Vpage0 Vpage0;

⟨enum _anon_((interrupts/arm/trap.c)(arm)) 712b⟩

⟨struct Vpage0(arm) 243d⟩
```

`<struct Intregs(arm) 152e>`
`<struct Vctl(arm) 152a>`
`<global vctllock(arm) 152d>`
`<global vctl(arm) 152b>`
`<global vfiq(arm) 499h>`
`<global trapnames(arm) 139b>`
`<function arch__trapinit(arm) 243f>`
`<function intrcpushutdown(arm) 565a>`
`<function intrsoff(arm) 244a>`
`<function intrshutdown(arm) 564a>`
`<function intrtime(arm) 588c>`

`<function irq(arm) 154c>`
`<function fiq(arm) 500c>`
`<function irqenable(arm) 153b>`
`<function trapname(arm) 579b>`
`<function ckfaultstuck(arm) 579d>`
`<function faultarm(arm) 159>`
`<function writetomem(arm) 150a>`
`<function arch__trap(arm) 146>`
`<function isvalidaddr(arm) 579a>`
`<function dumplongs(arm) 578b>`
`<function dumpstackwithureg(arm) 580c>`
`<function getpcsp(arm) 580b>`
`<function arch_callwithureg(arm) 580a>`
`<function trap_arch_dumpstack(arm) 579e>`
`<function dumpregs(arm) 578a>`

E.25.2 interrupts/arm/fault_helpers.s

```

<interrupts/arm/fault_helpers.s 713>≡
#include "mem.h"
#include "arm.h"

```

<function fstget(*arm*) 149f>
 <function ifsrget(*arm*) 149a>
 <function farget(*arm*) 149e>

E.25.3 interrupts/arm/lexception.s

<interrupts/arm/lexception.s 714a>≡
 /*
 * arm exception handlers
 */
 #include "mem.h"
 #include "arm.h"
 #include "arminstr.ha"

 <global vectors(*arm*) 139c>

 <global vtable(*arm*) 139d>

 <function _vsvc(*arm*) 140c>

 <function _vund(*arm*) 143f>

 <function _vpabt(*arm*) 144a>

 <function _vdabt(*arm*) 144b>

 <function _virq(*arm*) 144c>

 <label _vswitch(*arm*) 144d>

 <function _vfiq(*arm*) 499e>

 <function setr13(*arm*) 244d>

E.26 time/arm/

E.26.1 time/arm/dat_time.h

<time/arm/dat_time.h 714b>≡
 /*
 * Time.
 *
 * HZ should divide 1000 evenly, ideally.
 * 100, 125, 200, 250 and 333 are okay.
 */
 <constant Arch_HZ(*arm*) 206b>

 // was vlong in x86
 //typedef uulong Tval;

E.26.2 time/arm/clock.c

<enum _anon_((time/arm/clock.c)(*arm*)) 714c>≡ (715b)

```

enum {
    <constant Localctl(arm) 485b>
    <constant Prescaler(arm) 485c>

    <constant SystimerFreq(arm) 214e>

    <constant MaxPeriod(arm) 220b>
    <constant MinPeriod(arm) 220c>
};

<enum _anon_((time/arm/clock.c)2(arm)) 715a>≡ (715b)
enum {
    CntPrescaleShift= 16, /* freq is sys_clk/(prescale+1) */
    CntPrescaleMask = 0xFF,
    CntEnable      = 1<<9,
    TmrDbgHalt     = 1<<8,
    TmrEnable      = 1<<7,
    TmrIntEnable    = 1<<5,
    TmrPrescale1    = 0x00<<2,
    TmrPrescale16   = 0x01<<2,
    TmrPrescale256  = 0x02<<2,
    CntWidth16      = 0<<1,
    CntWidth32      = 1<<1,

    /* generic timer (cortex-a7) */
    Enable = 1<<0,
    Imask  = 1<<1,
    Istatus = 1<<2,
};

<time/arm/clock.c 715b>≡
/*
 * bcm283[56] timers
 * System timers run at 1MHz (timers 1 and 2 are used by GPU)
 * ARM timer usually runs at 250MHz (may be slower in low power modes)
 * Cycle counter runs at 700MHz (unless overclocked)
 * All are free-running up-counters
 * Cortex-a7 has local generic timers per cpu (which we run at 1MHz)
 *
 * Use system timer 3 (64 bits) for hzclock interrupts and fastticks
 * For smp on bcm2836, use local generic timer for interrupts on cpu1-3
 * Use ARM timer (32 bits) for perfticks
 * Use ARM timer to force immediate interrupt
 * Use cycle counter for cycles()
 */
<kernel basic includes 35a>
#include "io.h"
#include "ureg.h"
#include "arm.h"

<constant SYSTIMERS(arm) 31e>
<constant ARMTIMER(arm) 31f>
//TODO factorize in mem.h
<constant ARMLOCAL(arm) 475c>

<enum _anon_((time/arm/clock.c)(arm)) 714c>

typedef struct Systimers Systimers;
typedef struct Armtimer Armtimer;

```

$\langle struct\ Sys timers (arm) \textcolor{red}{215c} \rangle$
 $\langle struct\ Arm timer (arm) \textcolor{red}{589b} \rangle$
 $\langle enum\ _anon_((time/arm/clock.c)2 (arm)) \textcolor{red}{715a} \rangle$
 $\langle function\ clock intr (arm) \textcolor{red}{215d} \rangle$
 $\langle function\ local clock intr (arm) \textcolor{red}{486a} \rangle$
 $\langle function\ clock shutdown (arm) \textcolor{red}{564b} \rangle$
 $\langle function\ clock init (arm) \textcolor{red}{245a} \rangle$
 $\langle function\ arch_timer set (arm) \textcolor{red}{219d} \rangle$
 $\langle function\ clock_arch_fast ticks (arm) \textcolor{red}{215a} \rangle$
 $\langle function\ arch_per fticks (arm) \textcolor{red}{589a} \rangle$
 $\langle function\ arm timer set (arm) \textcolor{red}{781a} \rangle$
 $\langle function\ arch_us (arm) \textcolor{red}{214g} \rangle$
 $\langle function\ clock_arch_micro delay (arm) \textcolor{red}{222d} \rangle$
 $\langle function\ clock_arch_delay (arm) \textcolor{red}{222e} \rangle$

E.26.3 time/arm/time_raspi1.s

$\langle time/arm/time_raspi1.s \textcolor{red}{716a} \rangle \equiv$
`#include "mem.h"`
`#include "arm.h"`
 $\langle function\ arch_l cycles (raspberry\ pi1) (arm) \textcolor{red}{448g} \rangle$

E.26.4 time/arm/time_raspi2.s

$\langle instruction\ MRRC (arm) \textcolor{red}{716b} \rangle \equiv \textcolor{red}{(684)}$
`#define MRRC(coproc, op, rd, rn, crm) \`
`WORD $(0xec500000 | (rn) << 16 | (rd) << 12 | (coproc) << 8 | (op) << 4 | (crm))`
 $\langle time/arm/time_raspi2.s \textcolor{red}{716c} \rangle \equiv$
`#include "mem.h"`
`#include "arm.h"`
`#include "arminstr.ha"`
 $\langle function\ arch_l cycles (raspberry\ pi2) (arm) \textcolor{red}{486b} \rangle$

E.27 processes/arm/

E.27.1 processes/arm/dat_processes.h

$\langle processes/arm/dat_processes.h \textcolor{red}{716d} \rangle \equiv$

⟨constant AOUT_MAGIC(*arm*) 118e⟩

⟨enum _anon_((arch/arm/dat_processes.h)(*arm*)) 454i⟩

⟨struct Arch_FPsave(*arm*) 454g⟩

E.27.2 processes/arm/arch.c

⟨processes/arm/arch.c 717a⟩≡
 ⟨kernel basic includes 35a⟩

```
#include <tos.h>
#include "ureg.h"

#include "arm.h"

/*
 * A lot of this stuff doesn't belong here
 * but this is a convenient dumping ground for
 * later sorting into the appropriate buckets.
 */

⟨function arch_userpc(arm) 143c⟩

⟨function arch_dbgpc(arm) 577b⟩

⟨function arch_userureg(arm) 143e⟩

⟨function arch_setregisters(arm) 424c⟩

⟨function arch_setkernur(arm) 542b⟩

⟨function arch_validalign(arm) 501c⟩

⟨function arch__kexit(arm) 147a⟩

⟨function linkproc(arm) 136b⟩

⟨function arch_kprocchild(arm) 136a⟩
```

E.27.3 processes/arm/vfp3.c

⟨processes/arm/vfp3.c 717b⟩≡
 /*
 * VFPv2 or VFPv3 floating point unit
 */
 ⟨kernel basic includes 35a⟩
 #include "ureg.h"
 #include "arm.h"

⟨enum _anon_((processes/arm/vfp3.c)(*arm*)) 458a⟩

⟨enum _anon_((processes/arm/vfp3.c)2(*arm*)) 460b⟩
 ⟨enum _anon_((processes/arm/vfp3.c)3(*arm*)) 460c⟩
 ⟨enum _anon_((processes/arm/vfp3.c)4(*arm*)) 461a⟩

`<function subarch(arm) 461b>`
`<function implement(arm) 461c>`
`<function havefp(arm) 461e>`
`<function fpoff(arm) 462a>`
`<function fpononly(arm) 462b>`
`<function fpcfg(arm) 463a>`
`<function fpinit(arm) 463c>`
`<function fpon(arm) 463d>`
`<function fpclear(arm) 463e>`

`<function fpunotify(arm) 457a>`
`<function fpunoted(arm) 457b>`
`<function fpusysrfork(arm) 456d>`
`<function fpusysrforkchild(arm) 455b>`
`<function fpsave(arm) 456e>`
`<function fprestore(arm) 467a>`
`<function fpuprocsave(arm) 455f>`
`<function fpuprocrestore(arm) 456b>`
`<function fpusysprocsetup(arm) 455d>`

`<function arch_procsetup(arm) 121e>`
`<function arch_procsave(arm) 201a>`
`<function arch_procrestore(arm) 201b>`
`<function mathnote(arm) 466b>`
`<function mathemu(arm) 466c>`
`<function fpstuck(arm) 464a>`
`<enum _anon_((processes/arm/vfp3.c)5(arm)) 464c>`
`<function condok(arm) 464d>`

`/* instruction decoding */ // was in arm.h`
`<macro ISCPPOP(arm) 465a>`
`<macro ISVFPOP(arm) 465b>`

`<function fpuemu(arm) 465c>`

E.27.4 processes/arm/lproc.s

```
<processes/arm/lproc.s 719a>≡  
#include "mem.h"  
#include "arm.h"  
  
<function arch_touser(arm) 253c>  
  
<function arch__forkret(arm) 114d>
```

E.27.5 processes/arm/switch.s

```
<processes/arm/switch.s 719b>≡  
<function arch_setlabel(arm) 198a>  
  
<function arch_gotolabel(arm) 197h>  
  
<function getcallerpc(arm) 27>
```

E.28 syscalls/arm/

E.28.1 syscalls/arm/syscall.c

```
<syscalls/arm/syscall.c 719c>≡  
<kernel basic includes 35a>  
#include "../port/systab.h"  
#include <tos.h>  
#include "ureg.h"  
#include "arm.h"  
  
typedef struct NFrame NFrame;  
<struct NFrame(arm) 396h>  
  
<function arch__noted(arm) 398c>  
  
<function arch__notify(arm) 397>  
  
<function arch__syscall(arm) 141e>  
  
<function arch_execregs(arm) 126>  
  
<function arch_forkchild(arm) 114b>
```

E.29 Extra files

E.29.1 init/arm/bootconf.c

```
<enum _anon_((init/arm/bootconf.c)(arm)) 719d>≡ (723)  
enum {  
    /* space for syscall args, return PC, top-of-stack struct */  
    Ustkheadroom = sizeof(Sargs) + sizeof(uintptr) + sizeof(Tos),  
};  
  
<constant CONFADDR(arm) 719e>≡ (709b)  
#define CONFADDR (KZERO+0x100) /* unparsed plan9.ini */
```



```

⟨constant BOOTARGS(arm) 720a⟩≡ (723)
/*
 * Where configuration info is left for the loaded programme.
 */
#define BOOTARGS ((char*)CONFADDR)

⟨constant BOOTARGSLLEN(arm) 720b⟩≡ (723)
#define BOOTARGSLLEN (CPUADDR-CONFADDR)

⟨global oargc(arm) 720c⟩≡ (723)
/*
 * Option arguments from the command line.
 * oargv[0] is the boot file.
 */
static int oargc;

⟨global oargv(arm) 720d⟩≡ (723)
static char* oargv[20];

⟨global oargb(arm) 720e⟩≡ (723)
static char oargb[128];

⟨global oargblen(arm) 720f⟩≡ (723)
static int oargblen;

⟨struct Atag(arm) 720g⟩≡ (723)
struct Atag {
    u32int size; /* size of atag in words, including this header */
    u32int tag; /* atag type */
    union {
        u32int data[1]; /* actually [size-2] */
        /* AtagMem */
        struct {
            u32int size;
            u32int base;
        } mem;
        /* AtagCmdLine */
        char cmdline[1]; /* actually [4*(size-2)] */
    };
};

⟨enum _anon_((init/arm/bootconf.c)2(arm)) 720h⟩≡ (723)
enum {
    AtagNone = 0x00000000,
    AtagCore = 0x54410001,
    AtagMem = 0x54410002,
    AtagCmdline = 0x54410009,
};

⟨function findconf(arm) 720i⟩≡ (723)
static int
findconf(char *name)
{
    int i;

    for(i = 0; i < nconf; i++)
        if(cistrncmp(confname[i], name) == 0)
            return i;
    return -1;
}

```

<function addconf (arm) 721a>≡ (723)

```
void
addconf(char *name, char *val)
{
    int i;

    i = findconf(name);
    if(i < 0){
        if(val == nil || nconf >= MAXCONF)
            return;
        i = nconf++;
        strcpy(confname[i], confname[i]+sizeof(confname[i]), name);
    }
    strcpy(confval[i], confval[i]+sizeof(confval[i]), val);
}
```

<function writeconf (arm) 721b>≡ (723)

```
static void
writeconf(void)
{
    char *p, *q;
    int n;

    p = getconfenv();

    if(waserror()) {
        free(p);
        nexterror();
    }

    /* convert to name=value\n format */
    for(q=p; *q; q++) {
        q += strlen(q);
        *q = '=';
        q += strlen(q);
        *q = '\n';
    }
    n = q - p + 1;
    if(n >= BOOTARGSLEN)
        error("kernel configuration too large");
    memmove(BOOTARGS, p, n);
    memset(BOOTARGS + n, '\n', BOOTARGSLEN - n);
    poperror();
    free(p);
}
```

<function plan9iniinit (arm) 721c>≡ (723)

```
static void
plan9iniinit(char *s, int cmdline)
{
    char *toks[MAXCONF];
    int i, c, n;
    char *v;

    if((c = *s) < ' ' || c >= 0x80)
        return;
    if(cmdline)
        n = tokenize(s, toks, MAXCONF);
    else
        n = getfields(s, toks, MAXCONF, 1, "\n");
}
```

```

    for(i = 0; i < n; i++){
        if(toks[i][0] == '#')
            continue;
        v = strchr(toks[i], '=');
        if(v == nil)
            continue;
        *v++ = '\0';
        addconf(toks[i], v);
    }
}

```

*<function ataginit(*arm*) 722a>*≡ (723)

```

static void
ataginit(Atag *a)
{
    int n;

    if(a->tag != AtagCore){
        plan9iniinit((char*)a, 0);
        return;
    }
    while(a->tag != AtagNone){
        switch(a->tag){
            case AtagMem:
                /* use only first bank */
                if(conf.mem[0].limit == 0 && a->mem.size != 0){
                    memsize = a->mem.size;
                    conf.mem[0].base = a->mem.base;
                    conf.mem[0].limit = a->mem.base + memsize;
                }
                break;
            case AtagCmdline:
                n = (a->size * sizeof(u32int)) - offsetof(Atag, cmdline[0]);
                if(a->cmdline + n < BOOTARGS + BOOTARGSLEN)
                    a->cmdline[n] = 0;
                else
                    BOOTARGS[BOOTARGSLEN-1] = 0;
                plan9iniinit(a->cmdline, 1);
                break;
        }
        a = (Atag*)((u32int*)a + a->size);
    }
}

```

*<function optionsinit(*arm*) 722b>*≡ (723)

```

static void
optionsinit(char* s)
{
    strecpy(oargb, oargb+sizeof(oargb), s);

    oargblen = strlen(oargb);
    oargc = tokenize(oargb, oargv, nelem(oargv)-1);
    oargv[oargc] = nil;
}

```

*<function bootargs(*arm*) 722c>*≡ (723)

```

uintptr
bootargs(uintptr base)
{
    int i;

```

```

ulong ssize;
char **av, *p;

/*
 * Push the boot args onto the stack.
 * The initial value of the user stack must be such
 * that the total used is larger than the maximum size
 * of the argument list checked in syscall.
 */
i = oargblen+1;
p = UINT2PTR(STACKALIGN(base + BY2PG - Ustkheadroom - i));
memmove(p, oargb, i);

/*
 * Now push the argv pointers.
 * The code jumped to by touser in lproc.s expects arguments
 * main(char* argv0, ...)
 * and calls
 * startboot("/boot/boot", &argv0)
 * not the usual (int argc, char* argv[])
 */
av = (char**)(p - (oargc+1)*sizeof(char*));
ssize = base + BY2PG - PTR2UINT(av);
for(i = 0; i < oargc; i++)
    *av++ = (oargv[i] - oargb) + (p - base) + (USTKTOP - BY2PG);
*av = nil;
//sp = USTKTOP - ssize;
return USTKTOP - ssize;
}

<init/arm/bootconf.c 723>≡
<kernel basic includes 35a>

#include "tos.h"

<enum _anon_((init/arm/bootconf.c)(arm)) 719d>

<constant BOOTARGS(arm) 720a>
<constant BOOTARGSLLEN(arm) 720b>
// #define MAXCONF 64
// #define MAXCONFLINE 160

<global oargc(arm) 720c>
<global oargv(arm) 720d>
<global oargb(arm) 720e>
<global oargblen(arm) 720f>

/* store plan9.ini contents here at least until we stash them in #ec */
//static char confname[MAXCONF][KNAMELEN];
//static char confval[MAXCONF][MAXCONFLINE];
//static int nconf;

typedef struct Atag Atag;
<struct Atag(arm) 720g>

<enum _anon_((init/arm/bootconf.c)2(arm)) 720h>

<function findconf(arm) 720i>

```

<function addconf (arm) 721a>

<function writeconf (arm) 721b>

<function plan9iniinit (arm) 721c>

<function ataginit (arm) 722a>

<function optionsinit (arm) 722b>

<function bootargs (arm) 722c>

E.29.2 devices/storage/arm/emmc.c

<enum _anon_((devices/storage/arm/emmc.c)(arm)) 724a>≡ (731b)

```
enum {
    Extfreq      = 100*Mhz, /* guess external clock frequency if */
                      /* not available from vcore */
    Initfreq     = 400000, /* initialisation frequency for MMC */
    SDFreq       = 25*Mhz, /* standard SD frequency */
    DT0          = 14,     /* data timeout exponent (guesswork) */

    MMCSelect    = 7,      /* mmc/sd card select command */
    Setbuswidth  = 6,      /* mmc/sd set bus width command */
};
```

<enum _anon_((devices/storage/arm/emmc.c)2(arm)) 724b>≡ (731b)

```
enum {
    /* Controller registers */
    Arg2          = 0x00>>2,
    Blksizcnt     = 0x04>>2,
    Arg1          = 0x08>>2,
    Cmdtm        = 0x0c>>2,
    Resp0         = 0x10>>2,
    Resp1        = 0x14>>2,
    Resp2        = 0x18>>2,
    Resp3        = 0x1c>>2,
    Data         = 0x20>>2,
    Status        = 0x24>>2,
    Control0      = 0x28>>2,
    Control1      = 0x2c>>2,
    Interrupt     = 0x30>>2,
    Irptmask      = 0x34>>2,
    Irpten        = 0x38>>2,
    Control2      = 0x3c>>2,
    Forceirpt     = 0x50>>2,
    Boottimeout   = 0x70>>2,
    Dbgsel        = 0x74>>2,
    Exrdfifocfg   = 0x80>>2,
    Exrdfifoen    = 0x84>>2,
    Tunestep      = 0x88>>2,
    Tunestepsstd  = 0x8c>>2,
    Tunestepsddr  = 0x90>>2,
    Spiintspt     = 0xf0>>2,
    Slotisrver    = 0xfc>>2,

    /* Control0 */
    Dwidth4       = 1<<1,
    Dwidth1       = 0<<1,
```

```

/* Control1 */
Srstdata      = 1<<26,    /* reset data circuit */
Srstcmd       = 1<<25,    /* reset command circuit */
Srsthc        = 1<<24,    /* reset complete host controller */
Datatoshift   = 16,      /* data timeout unit exponent */
Datatomask    = 0xF0000,
Clkfreq8shift = 8,        /* SD clock base divider LSBs */
Clkfreq8mask  = 0xFF00,
Clkfreqms2shift = 6,      /* SD clock base divider MSBs */
Clkfreqms2mask = 0xC0,
Clkgendiv     = 0<<5,     /* SD clock divided */
Clkgenprog    = 1<<5,     /* SD clock programmable */
Clken         = 1<<2,     /* SD clock enable */
Clkstable     = 1<<1,
Clkintlen     = 1<<0,     /* enable internal EMMC clocks */

/* Cmdtm */
Indexshift    = 24,
Suspend       = 1<<22,
Resume        = 2<<22,
Abort         = 3<<22,
Isdata        = 1<<21,
Ixchken       = 1<<20,
Crcchken      = 1<<19,
Respmask      = 3<<16,
Resptime     = 0<<16,
Resp136       = 1<<16,
Resp48        = 2<<16,
Resp48busy    = 3<<16,
Multiblock    = 1<<5,
Host2card     = 0<<4,
Card2host     = 1<<4,
Autocmd12     = 1<<2,
Autocmd23     = 2<<2,
Blkcnten      = 1<<1,

/* Interrupt */
Acmderr       = 1<<24,
Denderr       = 1<<22,
Dcrrerr       = 1<<21,
Dtoerr        = 1<<20,
Cbaderr       = 1<<19,
Cenderr       = 1<<18,
Ccrrerr       = 1<<17,
Ctoerr        = 1<<16,
Err           = 1<<15,
Cardintr      = 1<<8,      /* not in Broadcom datasheet */
Cardinsert    = 1<<6,      /* not in Broadcom datasheet */
Readrdy       = 1<<5,
Writerdy      = 1<<4,
Datadone      = 1<<1,
Cmddone       = 1<<0,

/* Status */
Bufread       = 1<<11,     /* not in Broadcom datasheet */
Bufwrite      = 1<<10,     /* not in Broadcom datasheet */
Readtrans     = 1<<9,
Writetrans    = 1<<8,
Dataactive    = 1<<2,

```

```

        Datinhibit  = 1<<1,
        Cmdinhibit  = 1<<0,
};

⟨global cmdinfo(arm) 726a⟩≡ (731b)
int cmdinfo[64] = {
    [0]  Ixchken,
    [2]  Resp136,
    [3]  Resp48 | Ixchken | Crcchken,
    [6]  Resp48 | Ixchken | Crcchken,
    [7]  Resp48busy | Ixchken | Crcchken,
    [8]  Resp48 | Ixchken | Crcchken,
    [9]  Resp136,
    [12] Resp48busy | Ixchken | Crcchken,
    [13] Resp48 | Ixchken | Crcchken,
    [16] Resp48,
    [17] Resp48 | Isdata | Card2host | Ixchken | Crcchken,
    [18] Resp48 | Isdata | Card2host | Multiblock | Blkcnten | Ixchken | Crcchken,
    [24] Resp48 | Isdata | Host2card | Ixchken | Crcchken,
    [25] Resp48 | Isdata | Host2card | Multiblock | Blkcnten | Ixchken | Crcchken,
    [41] Resp48,
    [55] Resp48 | Ixchken | Crcchken,
};

⟨struct Ctlr((devices/storage/arm/emmc.c)(arm)) 726b⟩≡ (731b)
struct Ctlr {
    Rendez  r;
    int datadone;
    int fastclock;
    ulong  extclk;
};

⟨global emmc(arm) 726c⟩≡ (731b)
static Ctlr emmc;

⟨function WR(arm) 726d⟩≡ (731b)
static void
WR(int reg, u32int val)
{
    u32int *r = (u32int*)EMMCREGS;

    //if(0)print("WR %2.2ux %ux\n", reg<<2, val);
    arch_microdelay(emmc.fastclock? 2 : 20);
    r[reg] = val;
}

⟨function clkdiv(arm) 726e⟩≡ (731b)
static uint
clkdiv(uint d)
{
    uint v;

    assert(d < 1<<10);
    v = (d << Clkfreq8shift) & Clkfreq8mask;
    v |= ((d >> 8) << Clkfreqms2shift) & Clkfreqms2mask;
    return v;
}

```

```

⟨function datadone(arm) 727a)≡ (731b)
static int
datadone(void*)
{
    int i;

    u32int *r = (u32int*)EMMCREGS;
    i = r[Interrupt];
    return i & (Datadone|Err);
}

⟨function emmcinit(arm) 727b)≡ (731b)
static int
emmcinit(void)
{
    u32int *r;
    ulong clk;
    char *s;

    clk = getclkrate(ClkEmmc);
    s = "";
    if(clk == 0){
        s = "Assuming ";
        clk = Extfreq;
    }
    emmc.extclk = clk;
    print("%seMMC external clock %lud Mhz\n", s, clk / Mhz);
    r = (u32int*)EMMCREGS;
    //if(0)print("emmc control %8.8ux %8.8ux %8.8ux\n", r[Control0], r[Control1], r[Control2]);
    WR(Control1, Srsthc);
    arch_delay(10);
    while(r[Control1] & Srsthc)
        ;
    return 0;
}

⟨function emmcinquiry(arm) 727c)≡ (731b)
static int
emmcinquiry(char *inquiry, int inqlen)
{
    u32int *r;
    uint ver;

    r = (u32int*)EMMCREGS;
    ver = r[Slotisrver] >> 16;
    return snprintf(inquiry, inqlen,
        "Arasan eMMC SD Host Controller %2.2x Version %2.2x",
        ver&0xFF, ver>>8);
}

⟨function mmcinterrupt(arm) 727d)≡ (731b)
static void
mmcinterrupt(Ureg*, void*)
{
    u32int *r;
    r = (u32int*)EMMCREGS;
    if(r[Interrupt]&(Datadone|Err)){
        WR(Irpten, 0);
        wakeup(&emmc.r);
    }
}

```


<function emmcenable(arm) 728a>≡ (731b)

```
static void
emmcenable(void)
{
    u32int *r;
    int i;

    r = (u32int*)EMMCREGS;
    WR(Control1, clkdiv(emmc.extclk/Initfreq - 1) |
        DTO<<Datatoshift | Clkgendiv | Clken | Clkintlen);
    for(i = 0; i < 1000; i++){
        arch_delay(1);
        if(r[Control1] & Clkstable)
            break;
    }
    if(i == 1000)
        print("SD clock won't initialise!\n");
    WR(Irptmask, ~(Dtoerr|Cardintr));
    arch_intrenable(IRQmmc, mmcinterrupt, nil, 0, "mmc");
}
```

<function emmccmd(arm) 728b>≡ (731b)

```
static int
emmccmd(u32int cmd, u32int arg, u32int *resp)
{
    u32int *r;
    u32int c;
    int i;
    ulong now;

    r = (u32int*)EMMCREGS;
    assert(cmd < nelem(cmdinfo) && cmdinfo[cmd] != 0);
    c = (cmd << Indexshift) | cmdinfo[cmd];
    if(r[Status] & Cmdinhibit){
        print("emmccmd: need to reset Cmdinhibit intr %ux stat %ux\n",
            r[Interrupt], r[Status]);
        WR(Control1, r[Control1] | Srstcmd);
        while(r[Control1] & Srstcmd)
            ;
        while(r[Status] & Cmdinhibit)
            ;
    }
    if((r[Status] & Datinhibit) &&
        ((c & Isdata) || (c & Respmask) == Resp48busy)){
        print("emmccmd: need to reset Datinhibit intr %ux stat %ux\n",
            r[Interrupt], r[Status]);
        WR(Control1, r[Control1] | Srstdata);
        while(r[Control1] & Srstdata)
            ;
        while(r[Status] & Datinhibit)
            ;
    }
    WR(Arg1, arg);
    if((i = r[Interrupt]) != 0){
        if(i != Cardinsert)
            print("emmc: before command, intr was %ux\n", i);
        WR(Interrupt, i);
    }
    WR(Cmdtm, c);
    now = cpu->ticks;
```

```

while(((i=r[Interrupt])&(Cmddone|Err)) == 0)
    if(cpu->ticks - now > Arch_HZ)
        break;
if((i&(Cmddone|Err)) != Cmddone){
    if((i&~Err) != Ctoerr)
        print("emmc: cmd %ux error intr %ux stat %ux\n", c, i, r[Status]);
    WR(Interrupt, i);
    if(r[Status]&Cmdinhibit){
        WR(Control1, r[Control1]|Srstcmd);
        while(r[Control1]&Srstcmd)
            ;
    }
    error(Eio);
}
WR(Interrupt, i & ~(Datadone|Readrdy|Writerdy));
switch(c & Respmask){
case Resp136:
    resp[0] = r[Resp0]<<8;
    resp[1] = r[Resp0]>>24 | r[Resp1]<<8;
    resp[2] = r[Resp1]>>24 | r[Resp2]<<8;
    resp[3] = r[Resp2]>>24 | r[Resp3]<<8;
    break;
case Resp48:
case Resp48busy:
    resp[0] = r[Resp0];
    break;
case Respname:
    resp[0] = 0;
    break;
}
if((c & Respmask) == Resp48busy){
    WR(Irpten, Datadone|Err);
    tsleep(&emmc.r, datadone, 0, 3000);
    WR(Irpten, 0);
    emmc.datadone = 0;
    i = r[Interrupt];
    if((i & Datadone) == 0)
        print("emmcio: no Datadone after CMD%d\n", cmd);
    if(i & Err)
        print("emmcio: CMD%d error interrupt %ux\n",
            cmd, r[Interrupt]);
    WR(Interrupt, i);
}
/*
 * Once card is selected, use faster clock
 */
if(cmd == MMCSelect){
    arch_delay(10);
    WR(Control1, clkdiv(emmc.extclk/SDfreq - 1) |
        DT0<<Datatoshift | Clkgendiv | Clken | Clkintlen);
    for(i = 0; i < 1000; i++){
        arch_delay(1);
        if(r[Control1] & Clkstable)
            break;
    }
    arch_delay(10);
    emmc.fastclock = 1;
}
/*
 * If card bus width changes, change host bus width

```

```

    */
    if(cmd == Setbuswidth){
        switch(arg){
            case 0:
                WR(Control0, r[Control0] & ~Dwidth4);
                break;
            case 2:
                WR(Control0, r[Control0] | Dwidth4);
                break;
        }
    }
    return 0;
}

⟨function emmciosetup(arm) 730a)≡ (731b)
void
emmciosetup(int write, void *buf, int bsize, int bcount)
{
    USED(write);
    USED(buf);
    WR(Blksizecnt, bcount<<16 | bsize);
}

⟨function emmcio(arm) 730b)≡ (731b)
static void
emmcio(int write, uchar *buf, int len)
{
    u32int *r;
    int i;

    r = (u32int*)EMMCREGS;
    assert((len&3) == 0);
    okay(1);
    if(waserror()){
        okay(0);
        nexterror();
    }
    if(write)
        dmastart(DmaChanEmmc, DmaDevEmmc, DmaM2D,
            buf, &r[Data], len);
    else
        dmastart(DmaChanEmmc, DmaDevEmmc, DmaD2M,
            &r[Data], buf, len);
    if(dmawait(DmaChanEmmc) < 0)
        error(Eio);
    if(!write)
        cachedinvse(buf, len);
    WR(Irpten, Datadone|Err);
    tsleep(&emmc.r, datadone, 0, 3000);
    WR(Irpten, 0);
    emmc.datadone = 0;
    i = r[Interrupt];
    if((i & Datadone) == 0){
        print("emmcio: %d timeout intr %ux stat %ux\n",
            write, i, r[Status]);
        WR(Interrupt, i);
        error(Eio);
    }
    if(i & Err){
        print("emmcio: %d error intr %ux stat %ux\n",

```

```

        write, r[Interrupt], r[Status]);
    WR(Interrupt, i);
    error(Eio);
}
if(i)
    WR(Interrupt, i);
poperror();
okay(0);
}

⟨global sdio(arm) 731a⟩≡ (731b)
SDio sdio = {
    "emmc",
    emmcinit,
    emmcenable,
    emmcinquiry,
    emmccmd,
    emmciosetup,
    emmcio,
};

⟨devices/storage/arm/emmc.c 731b⟩≡
/*
 * bcm2835 external mass media controller (mmc / sd host interface)
 *
 * Copyright © 2012 Richard Miller <r.miller@acm.org>
 */
⟨kernel basic includes 35a⟩
#include "io.h"
#include "../port/sd.h"

⟨constant EMMCREGS(arm) 32f⟩

⟨enum _anon_((devices/storage/arm/emmc.c)(arm)) 724a⟩

⟨enum _anon_((devices/storage/arm/emmc.c)2(arm)) 724b⟩

⟨global cmdinfo(arm) 726a⟩

typedef struct Ctlr Ctlr;
⟨struct Ctlr((devices/storage/arm/emmc.c)(arm)) 726b⟩

⟨global emmc(arm) 726c⟩

⟨function WR(arm) 726d⟩

⟨function clkdiv(arm) 726e⟩

⟨function datadone(arm) 727a⟩

⟨function emmcinit(arm) 727b⟩

⟨function emmcinquiry(arm) 727c⟩

⟨function mmcinterrupt(arm) 727d⟩

⟨function emmcenable(arm) 728a⟩

⟨function emmccmd(arm) 728b⟩

```

<function emmciosetup(arm) 730a>

<function emmcio(arm) 730b>

<global sdio(arm) 731a>

E.29.3 init/arm/watchdog.c

<enum _anon_((init/arm/watchdog.c)(arm)) 732a>≡ (733c)
enum {
 Wdogfreq = 65536,
 Wdogtime = 5, /* seconds, <= 15 */
};

<enum _anon_((init/arm/watchdog.c)2(arm)) 732b>≡ (733c)
/*
 * Power management / watchdog registers
 */
enum {
 Rstc = 0x1c>>2,
 Password = 0x5A<<24,
 CfgMask = 0x03<<4,
 CfgReset = 0x02<<4,
 Rsts = 0x20>>2,
 Wdog = 0x24>>2,
};

<function archreset(arm) 732c>≡ (733c)
void
archreset(void)
{
 fpon();
}

<function archreboot(arm) 732d>≡ (733c)
void
archreboot(void)
{
 u32int *r;

 r = (u32int*)POWERREGS;
 r[Wdog] = Password | 1;
 r[Rstc] = Password | (r[Rstc] & ~CfgMask) | CfgReset;
 arch_coherence();
 for(;;)
 ;
}

<function wdogfeed(arm) 732e>≡ (733c)
static void
wdogfeed(void)
{
 u32int *r;

 r = (u32int*)POWERREGS;
 r[Wdog] = Password | (Wdogtime * Wdogfreq);
 r[Rstc] = Password | (r[Rstc] & ~CfgMask) | CfgReset;
}

```

⟨function wdogoff(arm) 733a⟩≡ (733c)
void
wdogoff(void)
{
    u32int *r;

    r = (u32int*)POWERREGS;
    r[Rstc] = Password | (r[Rstc] & ~CfgMask);
}

⟨function watchdoglink(arm) 733b⟩≡ (733c)
void
watchdoglink(void)
{
    addclock0link(wdogfeed, Arch_HZ);
}

⟨init/arm/watchdog.c 733c⟩≡
⟨kernel basic includes 35a⟩

⟨constant POWERREGS(arm) 31i⟩

⟨enum _anon_((init/arm/watchdog.c)(arm)) 732a⟩

⟨enum _anon_((init/arm/watchdog.c)2(arm)) 732b⟩

⟨function archreset(arm) 732c⟩

⟨function archreboot(arm) 732d⟩

⟨function wdogfeed(arm) 732e⟩

⟨function wdogoff(arm) 733a⟩

⟨function watchdoglink(arm) 733b⟩

```

E.29.4 buses/arm/dwcotg.h

```

⟨enum _anon_((buses/arm/dwcotg.h)(arm)) 733d⟩≡ (734)
enum {
    Maxchans    = 16,    /* actual number of channels in ghwcfg2 */
};

⟨struct Dwcregs(arm) 733e⟩≡ (734)
struct Dwcregs {
    /* Core global registers 0x000-0x140 */
    Reg gotgctl;    /* OTG Control and Status */
    Reg gotgint;    /* OTG Interrupt */
    Reg gahbcfg;    /* Core AHB Configuration */
    Reg gusbcfg;    /* Core USB Configuration */
    Reg grstctl;    /* Core Reset */
    Reg gintsts;    /* Core Interrupt */
    Reg gintmsk;    /* Core Interrupt Mask */
    Reg grxstsr;    /* Receive Status Queue Read (RO) */
    Reg grxstsp;    /* Receive Status Queue Read & POP (RO) */
    Reg grxfsize;    /* Receive FIFO Size */
    Reg gnptxfsize; /* Non Periodic Transmit FIFO Size */
    Reg gnptxsts;    /* Non Periodic Transmit FIFO/Queue Status (RO) */
    Reg gi2cctl;    /* I2C Access */
}

```

```

Reg gpvndctl; /* PHY Vendor Control */
Reg ggpio; /* General Purpose Input/Output */
Reg guid; /* User ID */
Reg gsnpsid; /* Synopsys ID (RO) */
Reg ghwcfig1; /* User HW Config1 (RO) (DEVICE) */
Reg ghwcfig2; /* User HW Config2 (RO) */
Reg ghwcfig3; /* User HW Config3 (RO) */
Reg ghwcfig4; /* User HW Config4 (RO)*/
Reg glpmcfg; /* Core LPM Configuration */
Reg gpwrn; /* Global PowerDn */
Reg gdfifocfg; /* Global DFIFO SW Config (DEVICE?) */
Reg adpctl; /* ADP Control */
Reg reserved0[39];
Reg hptxsiz; /* Host Periodic Transmit FIFO Size */
Reg dtxsiz[15]; /* Device Periodic Transmit FIFOs (DEVICE) */
char pad0[0x400-0x140];

/* Host global registers 0x400-0x420 */
Reg hcfig; /* Configuration */
Reg hfir; /* Frame Interval */
Reg hfnum; /* Frame Number / Frame Remaining (RO) */
Reg reserved1;
Reg hptxsts; /* Periodic Transmit FIFO / Queue Status */
Reg haint; /* All Channels Interrupt */
Reg haintmsk; /* All Channels Interrupt Mask */
Reg hflbaddr; /* Frame List Base Address */
char pad1[0x440-0x420];

/* Host port register 0x440 */
Reg hport0; /* Host Port 0 Control and Status */
char pad2[0x500-0x444];

/* Host channel specific registers 0x500-0x700 */
struct Hostchan {
    Reg hcchar; /* Characteristic */
    Reg hcsplt; /* Split Control */
    Reg hcint; /* Interrupt */
    Reg hcintmsk; /* Interrupt Mask */
    Reg hctsiz; /* Transfer Size */
    Reg hcdma; /* DMA Address */
    Reg reserved;
    Reg hcdmab; /* DMA Buffer Address */
} hchan[Maxchans];
char pad3[0xE00-0x700];

/* Power & clock gating control register 0xE00 */
Reg pcgctl;
};

```

<buses/arm/dwcotg.h 734>≡

```

/*
 * USB host driver for BCM2835
 * Synopsys DesignWare Core USB 2.0 OTG controller
 *
 * Device register definitions
 */

```

```

typedef unsigned int Reg;
typedef struct Dwcregs Dwcregs;
typedef struct Hostchan Hostchan;

```

`<enum _anon_((buses/arm/dwcotg.h)(arm)) 733d>`

`<struct Dwcregs(arm) 733e>`

```
enum {
    /* gotgctl */
    Sesreqscs    = 1<<0,
    Sesreq       = 1<<1,
    Vbvalidoven  = 1<<2,
    Vbvalidovval = 1<<3,
    Avalidoven   = 1<<4,
    Avalidovval  = 1<<5,
    Bvalidoven   = 1<<6,
    Bvalidovval  = 1<<7,
    Hstnegscs    = 1<<8,
    Hnpreq       = 1<<9,
    Hstsethnpn   = 1<<10,
    Devhnpn      = 1<<11,
    Conidsts     = 1<<16,
    Dbnctime     = 1<<17,
    Asesvld      = 1<<18,
    Bsesvld      = 1<<19,
    Otgver       = 1<<20,
    Multvalidbc  = 0x1F<<22,
    Chirpen      = 1<<27,

    /* gotgint */
    Sesenddet    = 1<<2,
    Sesreqsucstschng= 1<<8,
    Hstnegsucstschng= 1<<9,
    Hstnegdet    = 1<<17,
    Adevtoutchng = 1<<18,
    Debdone      = 1<<19,
    Mvic         = 1<<20,

    /* gahbcfg */
    Glblintrmsk = 1<<0,
    /* bits 1:4 redefined for BCM2835 */
    Axiburstlen  = 0x3<<1,
        BURST1      = 3<<1,
        BURST2      = 2<<1,
        BURST3      = 1<<1,
        BURST4      = 0<<1,
    Axiwaitwrites = 1<<4,
    Dmaenable    = 1<<5,
    Nptxfemplvl  = 1<<7,
        NPTX_HALFEMPTY = 0<<7,
        NPTX_EMPTY    = 1<<7,
    Ptxfemplvl   = 1<<8,
        PTX_HALFEMPTY  = 0<<8,
        PTX_EMPTY      = 1<<8,
    Remmемsupp   = 1<<21,
    Notialldmawrit = 1<<22,
    Ahbsingle    = 1<<23,

    /* gusbcfg */
    Toutcal      = 0x7<<0,
    Phyif        = 1<<3,
    Ulpι_utmi_sel = 1<<4,
```



```

Fsintf      = 1<<5,
    FsUnidir  = 0<<5,
    FsBidir   = 1<<5,
Physel      = 1<<6,
    PhyHighspeed = 0<<6,
    PhyFullspeed  = 1<<6,
Ddrsel      = 1<<7,
Srcap       = 1<<8,
Hnpcap      = 1<<9,
Usbtrdtim   = 0xf<<10,
    OUsbtrdtim   = 10,
Phylpwrclkssel = 1<<15,
Otgutmiifssel = 1<<16,
Ulpifsls    = 1<<17,
Ulpiauto_res = 1<<18,
Ulpiclk_sus_m = 1<<19,
Ulpie_ext_vbus_drv= 1<<20,
Ulpie_int_vbus_indicator= 1<<21,
Term_sel_dl_pulse= 1<<22,
Indicator_complement= 1<<23,
Indicator_pass_through= 1<<24,
Ulpie_int_prot_dis= 1<<25,
Ic_usb_cap   = 1<<26,
Ic_traffic_pull_remove= 1<<27,
Tx_end_delay = 1<<28,
Force_host_mode = 1<<29,
Force_dev_mode = 1<<30,

/* grstctl */
Csfrst      = 1<<0,
Hsfrst      = 1<<1,
Hstfrm      = 1<<2,
Intknqflsh  = 1<<3,
Rxfflsh     = 1<<4,
Txfflsh     = 1<<5,
Txffnum     = 0x1f<<6,
    TXF_ALL    = 0x10<<6,
Dmareq      = 1<<30,
Ahbidle     = 1<<31,

/* gintsts, gintmsk */
Curmode     = 1<<0,
    HOSTMODE    = 1<<0,
    DEVMODE     = 0<<0,
Modemismatch = 1<<1,
Otgintr      = 1<<2,
Sofintr      = 1<<3,
Rxstsqqlvl   = 1<<4,
Nptxfempty   = 1<<5,
Ginnakeff    = 1<<6,
Goutnakeff   = 1<<7,
Ulpickint    = 1<<8,
I2cintr      = 1<<9,
Erlysuspend  = 1<<10,
Usbsuspend   = 1<<11,
Usbreset     = 1<<12,
Enumdone     = 1<<13,
Isooutdrop   = 1<<14,
Eopframe     = 1<<15,
Restoredone  = 1<<16,

```

```

Epmismatch = 1<<17,
Inepintr   = 1<<18,
Outepintr  = 1<<19,
Incomplisoin = 1<<20,
Incomplisoout = 1<<21,
Fetsusp    = 1<<22,
Resetdet   = 1<<23,
Portintr   = 1<<24,
Hcintr     = 1<<25,
Ptxfempty  = 1<<26,
Lpmtranrcvd = 1<<27,
Conidstschnrg = 1<<28,
Disconnect = 1<<29,
Sessreqintr = 1<<30,
Wkupintr   = 1<<31,

/* grxsts[rp] */
Chnum      = 0xf<<0,
Bcnt       = 0x7ff<<4,
Dpid       = 0x3<<15,
Pktsts     = 0xf<<17,
    PKTSTS_IN = 2<<17,
    PKTSTS_IN_XFER_COMP = 3<<17,
    PKTSTS_DATA_TOGGLE_ERR = 5<<17,
    PKTSTS_CH_HALTED = 7<<17,

/* hptxfsiz, gnptxfsiz */
Startaddr  = 0xffff<<0,
Depth      = 0xffff<<16,
    ODepth   = 16,

/* gnptxsts */
Nptxfspcavail = 0xffff<<0,
Nptxqspcavail = 0xff<<16,
Nptxqtop_terminate = 1<<24,
Nptxqtop_token = 0x3<<25,
Nptxqtop_chnep = 0xf<<27,

/* gpvndctl */
Regdata     = 0xff<<0,
Vctrl       = 0xff<<8,
Regaddr16_21 = 0x3f<<16,
Regwr       = 1<<22,
Newregreq   = 1<<25,
Vstsbsy     = 1<<26,
Vstdone     = 1<<27,
Disulpidrvr = 1<<31,

/* ggpio */
Gpi         = 0xffff<<0,
Gpo         = 0xffff<<16,

/* ghwcfg2 */
Op_mode     = 0x7<<0,
    HNP_SRP_CAPABLE_OTG = 0<<0,
    SRP_ONLY_CAPABLE_OTG = 1<<0,
    NO_HNP_SRP_CAPABLE = 2<<0,
    SRP_CAPABLE_DEVICE = 3<<0,
    NO_SRP_CAPABLE_DEVICE = 4<<0,
    SRP_CAPABLE_HOST = 5<<0,

```

```

    NO_SRP_CAPABLE_HOST = 6<<0,
Architecture    = 0x3<<3,
    SLAVE_ONLY        = 0<<3,
    EXT_DMA           = 1<<3,
    INT_DMA           = 2<<3,
Point2point     = 1<<5,
Hs_phy_type     = 0x3<<6,
    PHY_NOT_SUPPORTED = 0<<6,
    PHY_UTMI          = 1<<6,
    PHY_ULPI          = 2<<6,
    PHY_UTMI_ULPI     = 3<<6,
Fs_phy_type     = 0x3<<8,
Num_dev_ep      = 0xf<<10,
Num_host_chan   = 0xf<<14,
    ONum_host_chan    = 14,
Perio_ep_supported= 1<<18,
Dynamic_fifo    = 1<<19,
Nonperio_tx_q_depth= 0x3<<22,
Host_perio_tx_q_depth= 0x3<<24,
Dev_token_q_depth= 0x1f<<26,
Otg_enable_ic_usb= 1<<31,

/* ghwcfg3 */
Xfer_size_cntr_width    = 0xf<<0,
Packet_size_cntr_width  = 0x7<<4,
Otg_func                 = 1<<7,
I2c                     = 1<<8,
Vendor_ctrl_if          = 1<<9,
Optional_features       = 1<<10,
Synch_reset_type        = 1<<11,
Adp_supp                = 1<<12,
Otg_enable_hsic         = 1<<13,
Bc_support              = 1<<14,
Otg_lpm_en              = 1<<15,
Dfifo_depth             = 0xffff<<16,
    ODfifo_depth        = 16,

/* ghwcfg4 */
Num_dev_perio_in_ep      = 0xf<<0,
Power_optimiz            = 1<<4,
Min_ahb_freq             = 1<<5,
Hiber                   = 1<<6,
Xhiber                  = 1<<7,
Utmi_phy_data_width     = 0x3<<14,
Num_dev_mode_ctrl_ep    = 0xf<<16,
Iddig_filt_en           = 1<<20,
Vbus_valid_filt_en      = 1<<21,
A_valid_filt_en         = 1<<22,
B_valid_filt_en         = 1<<23,
Session_end_filt_en     = 1<<24,
Ded_fifo_en             = 1<<25,
Num_in_eps              = 0xf<<26,
Desc_dma                = 1<<30,
Desc_dma_dyn            = 1<<31,

/* glpmcfg */
Lpm_cap_en              = 1<<0,
Appl_resp               = 1<<1,
Hird                    = 0xf<<2,
Rem_wkup_en             = 1<<6,

```

```

En_utmi_sleep   = 1<<7,
Hird_thres     = 0x1f<<8,
Lpm_resp       = 0x3<<13,
Prt_sleep_sts  = 1<<15,
Sleep_state_resumeok= 1<<16,
Lpm_chan_index = 0xf<<17,
Retry_count    = 0x7<<21,
Send_lpm       = 1<<24,
Retry_count_sts = 0x7<<25,
Hsic_connect   = 1<<30,
Inv_sel_hsic   = 1<<31,

```

```
/* gpwrdsn */
```

```

Pmuintsel      = 1<<0,
Pmuactv        = 1<<1,
Restore        = 1<<2,
Pwrdsnclmp     = 1<<3,
Pwrdsnrstn     = 1<<4,
Pwrdsnswtch    = 1<<5,
Dis_vbus       = 1<<6,
Lnstschng      = 1<<7,
Lnstchnng_msk  = 1<<8,
Rst_det        = 1<<9,
Rst_det_msk    = 1<<10,
Disconn_det    = 1<<11,
Disconn_det_msk = 1<<12,
Connect_det    = 1<<13,
Connect_det_msk = 1<<14,
Srp_det        = 1<<15,
Srp_det_msk    = 1<<16,
Sts_chngint    = 1<<17,
Sts_chngint_msk = 1<<18,
Linestate     = 0x3<<19,
Idsts          = 1<<21,
Bsessvld      = 1<<22,
Adp_int        = 1<<23,
Mult_val_id_bc = 0x1f<<24,

```

```
/* gdfifocfg */
```

```

Gdfifocfg      = 0xffff<<0,
Epinfobase     = 0xffff<<16,

```

```
/* adpctl */
```

```

Prb_dschg      = 0x3<<0,
Prb_delta      = 0x3<<2,
Prb_per        = 0x3<<4,
Rtim           = 0x7ff<<6,
Enaprb         = 1<<17,
Enasns         = 1<<18,
Adpres         = 1<<19,
Adpen          = 1<<20,
Adp_prb_int    = 1<<21,
Adp_sns_int    = 1<<22,
Adp_tmout_int  = 1<<23,
Adp_prb_int_msk = 1<<24,
Adp_sns_int_msk = 1<<25,
Adp_tmout_int_msk= 1<<26,
Ar             = 0x3<<27,

```

```
/* hcfg */
```

```

Fslspclksel = 0x3<<0,
    HCFG_30_60_MHZ = 0<<0,
    HCFG_48_MHZ = 1<<0,
    HCFG_6_MHZ = 2<<0,
Fslssupp = 1<<2,
Ena32khzs = 1<<7,
Resvalid = 0xff<<8,
Descdma = 1<<23,
Frlisten = 0x3<<24,
Modechtimen = 1<<31,

/* hfir */
Frint = 0xffff<<0,
Hfirrldctrl = 1<<16,

/* hfnum */
Frnum = 0xffff<<0,
    MAX_FRNUM = 0x3FFF<<0,
Frrem = 0xffff<<16,

/* hptxsts */
Ptxfspcavail = 0xffff<<0,
Ptxqspcavail = 0xff<<16,
Ptxqtop_terminate= 1<<24,
Ptxqtop_token = 0x3<<25,
Ptxqtop_chnum = 0xf<<27,
Ptxqtop_odd = 1<<31,

/* haint, haintmsk */
#define CHANINT(n) (1<<(n))

/* hport0 */
Prtconnsts = 1<<0, /* connect status (RO) */
Prtconndet = 1<<1, /* connect detected R/W1C */
Prtena = 1<<2, /* enable (R/W1C) */
Prtenchng = 1<<3, /* enable/disable change (R/W1C) */
Prtovrcurract = 1<<4, /* overcurrent active (RO) */
Prtovrcurrchng = 1<<5, /* overcurrent change (R/W1C) */
Prtres = 1<<6, /* resume */
Prtsusp = 1<<7, /* suspend */
Prtrst = 1<<8, /* reset */
PrtlInsts = 0x3<<10, /* line state {D+,D-} (RO) */
Prtpwr = 1<<12, /* power on */
Prttstctl = 0xf<<13, /* test */
Prtspd = 0x3<<17, /* speed (RO) */
    HIGHSPPEED = 0<<17,
    FULLSPEED = 1<<17,
    LOWSPEED = 2<<17,

/* hcchar */
Mps = 0x7ff<<0, /* endpoint maximum packet size */
Epnun = 0xf<<11, /* endpoint number */
    OEpnun = 11,
Epdire = 1<<15, /* endpoint direction */
    Epout = 0<<15,
    Epin = 1<<15,
Lspddev = 1<<17, /* device is lowspeed */
Eptype = 0x3<<18, /* endpoint type */
    Epctl = 0<<18,
    Episo = 1<<18,

```

```

    Epbulk      = 2<<18,
    Epintr      = 3<<18,
Multicnt      = 0x3<<20, /* transactions per  $\mu$ frame */
                /* or retries per periodic split */
    OMulticnt   = 20,
Devaddr       = 0x7f<<22, /* device address */
    ODevaddr    = 22,
Oddfrm        = 1<<29,    /* xfer in odd frame (iso/interrupt) */
Chdis         = 1<<30,    /* channel disable (write 1 only) */
Chen          = 1<<31,    /* channel enable (write 1 only) */

/* hcsplt */
Prtaddr       = 0x7f<<0,  /* port address of recipient */
                /* transaction translator */
Hubaddr       = 0x7f<<7,  /* dev address of transaction */
                /* translator's hub */
    OHubaddr    = 7,
Xactpos       = 0x3<<14,  /* payload's position within transaction */
    POS_MID     = 0<<14,
    POS_END     = 1<<14,
    POS_BEGIN   = 2<<14,
    POS_ALL     = 3<<14, /* all of data (<= 188 bytes) */
Compsplt      = 1<<16,    /* do complete split */
Spltena       = 1<<31,    /* channel enabled to do splits */

/* hcint, hcintmsk */
Xfercomp      = 1<<0,    /* transfer completed without error */
Chhltd        = 1<<1,    /* channel halted */
Ahberr        = 1<<2,    /* AHB dma error */
Stall         = 1<<3,
Nak           = 1<<4,
Ack           = 1<<5,
Nyet          = 1<<6,
Xacterr       = 1<<7, /* transaction error (crc, t/o, bit stuff, eop) */
Bblerr        = 1<<8,
Frmovrun      = 1<<9,
Datatglerr    = 1<<10,
Bna           = 1<<11,
Xcs_xact      = 1<<12,
Frm_list_roll = 1<<13,

/* hctsiz */
Xfersize      = 0x7ffff<<0, /* expected total bytes */
Pktcnt        = 0x3fff<<19, /* expected number of packets */
    OPktcnt     = 19,
Pid           = 0x3<<29, /* packet id for initial transaction */
    DATA0      = 0<<29,
    DATA1      = 2<<29,    /* sic */
    DATA2      = 1<<29,    /* sic */
    MDATA       = 3<<29,    /* (non-ctl ep) */
    SETUP       = 3<<29,    /* (ctl ep) */
Dopng         = 1<<31,    /* do PING protocol */

/* pgcctl */
Stoppclk      = 1<<0,
Gatehclk      = 1<<1,
Pwrclmp       = 1<<2,
Rstpdwnmodule = 1<<3,
Enbl_sleep_gating = 1<<5,
Phy_in_sleep  = 1<<6,

```

```

Deep_sleep      = 1<<7,
Resetaftsusp    = 1<<8,
Restoremode     = 1<<9,
Enbl_extnd_hiber = 1<<10,
Extnd_hiber_pwrclmp = 1<<11,
Extnd_hiber_switch = 1<<12,
Ess_reg_restored = 1<<13,
Prt_clk_sel     = 0x3<<14,
Port_power      = 1<<16,
Max_xcvrselect  = 0x3<<17,
Max_termsel     = 1<<19,
Mac_dev_addr    = 0x7f<<20,
P2hd_dev_enum_spd = 0x3<<27,
P2hd_prt_spd    = 0x3<<29,
If_dev_mode     = 1<<31,
};

```

E.29.5 buses/arm/devusb.c

`<enum _anon_((buses/arm/devusb.c)(arm)) 742>`≡ (767b)

```

enum
{
    /* Qid numbers */
    Qdir = 0,          /* #u */
    Qusbdir,           /* #u/usb */
    Qctl,              /* #u/usb/ctl - control requests */

    Qep0dir,           /* #u/usb/ep0.0 - endpoint 0 dir */
    Qep0io,            /* #u/usb/ep0.0/data - endpoint 0 I/O */
    Qep0ctl,           /* #u/usb/ep0.0/ctl - endpoint 0 ctl. */
    Qep0dummy,         /* give 4 qids to each endpoint */

    Qepdir = 0,        /* (qid-qep0dir)&3 is one of these */
    Qepio,              /* to identify which file for the endpoint */
    Qepctl,

    /* ... */

    /* Usb ctls. */
    CMdebug = 0,        /* debug on/off */
    CMdump,             /* dump (data structures for debug) */

    /* Ep. ctls */
    CMnew = 0,          /* new nb ctl|bulk|intr|iso r|w|rw (endpoint) */
    CMnewdev,           /* newdev full|low|high portnb (allocate new devices) */
    CMhub,              /* hub (set the device as a hub) */
    CMspeed,            /* speed full|low|high|no */
    CMmaxpkt,           /* maxpkt size */
    CMntds,             /* ntds nb (max nb. of tds per pframe) */
    CMclrhalt,          /* clrhalt (halt was cleared on endpoint) */
    CMpollival,         /* pollival interval (interrupt/iso) */
    CMhz,              /* hz n (samples/sec; iso) */
    CMsamplesz,         /* samplesz n (sample size; iso) */
    CMinfo,             /* info infostr (ke.ep info for humans) */
    CMdetach,           /* detach (abort I/O forever on this ep). */
    CMaddress,          /* address (address is assigned) */
    CMdebugep,          /* debug n (set/clear debug for this ep) */
    CMname,             /* name str (show up as #u/name as well) */
    CMtmout,            /* timeout n (activate timeouts for ep) */

```

```

    CMpreset,          /* reset the port */

    /* Hub feature selectors */
    Rportenable = 1,
    Rportreset  = 4,

};

<struct Hcitype(arm) 743a)≡ (767b)
    struct Hcitype
    {
        char*   type;
        int (*reset)(Hci*);
    };

<macro QID(arm) 743b)≡ (767b)
    #define QID(q) ((int)(q).path)

<global usbctls(arm) 743c)≡ (767b)
    static Cmdtab usbctls[] =
    {
        {CMdebug,   "debug",   2},
        {CMdump,    "dump",    1},
    };

<global epctls(arm) 743d)≡ (767b)
    static Cmdtab epctls[] =
    {
        {CMnew,      "new",      4},
        {CMnewdev,   "newdev",   3},
        {CMhub,      "hub",      1},
        {CMspeed,    "speed",    2},
        {CMmaxpkt,   "maxpkt",   2},
        {CMntds,     "ntds",     2},
        {CMpollival, "pollival", 2},
        {CMSamplesz, "samplesz", 2},
        {CMhz,       "hz",       2},
        {CMinfo,     "info",     0},
        {CMdetach,   "detach",   1},
        {CMaddress,  "address",  1},
        {CMdebugep,  "debug",    2},
        {CMclrhalt,  "clrhalt",  1},
        {CMname,     "name",     2},
        {CMtmout,    "timeout",   2},
        {CMPreset,   "reset",    1},
    };

<global usbdire(arm) 743e)≡ (767b)
    static Dirtab usbdire[] =
    {
        "ctl",      {Qctl},      0, 0666,
    };

<global usbmodename(arm) 743f)≡ (767b)
    char *usbmodename[] =
    {
        [OREAD] "r",
        [OWRITE] "w",
        [ORDWR] "rw",
    };

```



```

⟨global ttname(arm) 744a⟩≡ (767b)
    static char *ttname[] =
    {
        [Tnone] "none",
        [Tctl]  "control",
        [Tiso]  "iso",
        [Tintr] "interrupt",
        [Tbulk] "bulk",
    };

⟨global spname(arm) 744b⟩≡ (767b)
    static char *spname[] =
    {
        [Fullspeed] "full",
        [Lowspeed]  "low",
        [Highspeed] "high",
        [Nospeed]   "no",
    };

⟨global debug(arm) 744c⟩≡ (767b)
    static int  debug;

⟨global hcities(arm) 744d⟩≡ (767b)
    static Hctype hcities[Nhcis];

⟨global hcis(arm) 744e⟩≡ (767b)
    static Hci* hcis[Nhcis];

⟨global epslck(arm) 744f⟩≡ (767b)
    static QLock  epslck;    /* add, del, lookup endpoints */

⟨global eps(arm) 744g⟩≡ (767b)
    static Ep*  eps[Neps]; /* all endpoints known */

⟨global epmax(arm) 744h⟩≡ (767b)
    static int  epmax;    /* 1 + last endpoint index used */

⟨global usbidgen(arm) 744i⟩≡ (767b)
    static int  usbidgen; /* device address generator */

⟨function seprindata(arm) 744j⟩≡ (767b)
    /*
     * Is there something like this in a library? should it be?
     */
    char*
    seprindata(char *s, char *se, uchar *d, int n)
    {
        int i, l;

        s = seprint(s, se, " %#p[%d]: ", d, n);
        l = n;
        if(l > 10)
            l = 10;
        for(i=0; i<l; i++)
            s = seprint(s, se, " %.2ux", d[i]);
        if(l < n)
            s = seprint(s, se, "...");
        return s;
    }

```

*<function name2speed(*arm*) 745a>*≡ (767b)

```
static int
name2speed(char *name)
{
    int i;

    for(i = 0; i < nelem(spname); i++)
        if(strcmp(name, spname[i]) == 0)
            return i;
    return Nospeed;
}
```

*<function name2ttype(*arm*) 745b>*≡ (767b)

```
static int
name2ttype(char *name)
{
    int i;

    for(i = 0; i < nelem(ttname); i++)
        if(strcmp(name, ttname[i]) == 0)
            return i;
    /* may be a std. USB ep. type */
    i = strtol(name, nil, 0);
    switch(i+1){
    case Tctl:
    case Tiso:
    case Tbulk:
    case Tintr:
        return i+1;
    default:
        return Tnone;
    }
}
```

*<function name2mode(*arm*) 745c>*≡ (767b)

```
static int
name2mode(char *mode)
{
    int i;

    for(i = 0; i < nelem(usbmodename); i++)
        if(strcmp(mode, usbmodename[i]) == 0)
            return i;
    return -1;
}
```

*<function qid2epidx(*arm*) 745d>*≡ (767b)

```
static int
qid2epidx(int q)
{
    q = (q-Qep0dir)/4;
    if(q < 0 || q >= epmax || eps[q] == nil)
        return -1;
    return q;
}
```

*<function isqtype(*arm*) 745e>*≡ (767b)

```
static int
isqtype(int q, int type)
{

```

```

    if(q < Qep0dir)
        return 0;
    q -= Qep0dir;
    return (q & 3) == type;
}

```

```

⟨function addhcitype(arm) 746a⟩≡ (767b)
void
addhcitype(char* t, int (*r)(Hci*))
{
    static int ntype;

    if(ntype == Nhcis)
        panic("too many USB host interface types");
    hcitypes[ntype].type = t;
    hcitypes[ntype].reset = r;
    ntype++;
}

```

```

⟨function seprintep(arm) 746b⟩≡ (767b)
static char*
seprintep(char *s, char *se, Ep *ep, int all)
{
    static char* dsnames[] = { "config", "enabled", "detached", "reset" };
    Udev *d;
    int i;
    int di;

    d = ep->dev;

    qlock(ep);
    if(waserror()){
        qunlock(ep);
        nexterror();
    }
    di = ep->dev->nb;
    if(all)
        s = seprint(s, se, "dev %d ep %d ", di, ep->nb);
    s = seprint(s, se, "%s", dsnames[ep->dev->state]);
    s = seprint(s, se, " %s", ttname[ep->ttype]);
    assert(ep->mode == OREAD || ep->mode == OWRITE || ep->mode == ORDWR);
    s = seprint(s, se, " %s", usbmodename[ep->mode]);
    s = seprint(s, se, " speed %s", spname[d->speed]);
    s = seprint(s, se, " maxpkt %ld", ep->maxpkt);
    s = seprint(s, se, " pollival %ld", ep->pollival);
    s = seprint(s, se, " samplesz %ld", ep->samplesz);
    s = seprint(s, se, " hz %ld", ep->hz);
    s = seprint(s, se, " hub %d", ep->dev->hub);
    s = seprint(s, se, " port %d", ep->dev->port);
    if(ep->inuse)
        s = seprint(s, se, " busy");
    else
        s = seprint(s, se, " idle");
    if(all){
        s = seprint(s, se, " load %uld", ep->load);
        s = seprint(s, se, " ref %ld addr %#p", ep->ref, ep);
        s = seprint(s, se, " idx %d", ep->idx);
        if(ep->name != nil)
            s = seprint(s, se, " name '%s'", ep->name);
        if(ep->tmout != 0)

```

```

        s = seprint(s, se, " tmout");
    if(ep == ep->ep0){
        s = seprint(s, se, " ctrlrno %#x", ep->hp->ctrlrno);
        s = seprint(s, se, " eps:");
        for(i = 0; i < nelem(d->eps); i++)
            if(d->eps[i] != nil)
                s = seprint(s, se, " ep%d.%d", di, i);
    }
}
if(ep->info != nil)
    s = seprint(s, se, "\n%s %s\n", ep->info, ep->hp->type);
else
    s = seprint(s, se, "\n");
qunlock(ep);
poperror();
return s;
}

```

*<function epalloc(*arm*) 747a>*≡ (767b)

```

static Ep*
epalloc(Hci *hp)
{
    Ep *ep;
    int i;

    ep = smalloc(sizeof(Ep));
    ep->ref = 1;
    qlock(&epslock);
    for(i = 0; i < Neps; i++)
        if(eps[i] == nil)
            break;
    if(i == Neps){
        qunlock(&epslock);
        free(ep);
        print("usb: bug: too few endpoints.\n");
        return nil;
    }
    ep->idx = i;
    if(epmax <= i)
        epmax = i+1;
    eps[i] = ep;
    ep->hp = hp;
    ep->maxpkt = 8;
    ep->ntds = 1;
    ep->samplesz = ep->pollival = ep->hz = 0; /* make them void */
    qunlock(&epslock);
    return ep;
}

```

*<function getep(*arm*) 747b>*≡ (767b)

```

static Ep*
getep(int i)
{
    Ep *ep;

    if(i < 0 || i >= epmax || eps[i] == nil)
        return nil;
    qlock(&epslock);
    ep = eps[i];
    if(ep != nil)

```

```

        incref(ep);
        qunlock(&epslck);
        return ep;
    }

⟨function putep(arm) 748a⟩≡ (767b)
    static void
    putep(Ep *ep)
    {
        Udev *d;

        if(ep != nil && decref(ep) == 0){
            d = ep->dev;
            deprint("usb: ep%d.%d %#p released\n", d->nb, ep->nb, ep);
            qlock(&epslck);
            eps[ep->idx] = nil;
            if(ep->idx == epmax-1)
                epmax--;
            if(ep == ep->ep0 && ep->dev != nil && ep->dev->nb == usbidgen)
                usbidgen--;
            qunlock(&epslck);
            if(d != nil){
                qlock(ep->ep0);
                d->eps[ep->nb] = nil;
                qunlock(ep->ep0);
            }
            if(ep->ep0 != ep){
                putep(ep->ep0);
                ep->ep0 = nil;
            }
            free(ep->info);
            free(ep->name);
            free(ep);
        }
    }

⟨function dumpeps(arm) 748b⟩≡ (767b)
    static void
    dumpeps(void)
    {
        int i;
        static char buf[512];
        char *s;
        char *e;
        Ep *ep;

        print("usb dump eps: epmax %d Neps %d (ref=1+ for dump):\n", epmax, Neps);
        for(i = 0; i < epmax; i++){
            s = buf;
            e = buf+sizeof(buf);
            ep = getep(i);
            if(ep != nil){
                if(waserror()){
                    putep(ep);
                    nexterror();
                }
                s = seprint(s, e, "ep%d.%d ", ep->dev->nb, ep->nb);
                seprintep(s, e, ep, 1);
                print("%s", buf);
                ep->hp->seprintep(buf, e, ep);
            }
        }
    }

```

```

        print("%s", buf);
        perror();
        putep(ep);
    }
}
print("usb dump hcis:\n");
for(i = 0; i < Nhcis; i++)
    if(hcis[i] != nil)
        hcis[i]->dump(hcis[i]);
}

```

```

⟨function newusbid(arm) 749a⟩≡ (767b)
static int
newusbid(Hci *)
{
    int id;

    qlock(&epslock);
    id = ++usbigen;
    if(id >= 0x7F)
        print("#u: too many device addresses; reuse them more\n");
    qunlock(&epslock);
    return id;
}

```

```

⟨function newdev(arm) 749b⟩≡ (767b)
/*
 * Create endpoint 0 for a new device
 */
static Ep*
newdev(Hci *hp, int ishub, int isroot)
{
    Ep *ep;
    Udev *d;

    ep = epalloc(hp);
    d = ep->dev = smalloc(sizeof(Udev));
    d->nb = newusbid(hp);
    d->eps[0] = ep;
    ep->nb = 0;
    ep->toggle[0] = ep->toggle[1] = 0;
    d->ishub = ishub;
    d->isroot = isroot;
    if(hp->highspeed != 0)
        d->speed = Highspeed;
    else
        d->speed = Fullspeed;
    d->state = Dconfig; /* address not yet set */
    ep->dev = d;
    ep->ep0 = ep; /* no ref counted here */
    ep->ttype = Tctl;
    ep->tmout = Xfertimeout;
    ep->mode = ORDWR;
    dprint("newdev %#p ep%d.%d %#p\n", d, d->nb, ep->nb, ep);
    return ep;
}

```

```

⟨function newdevep(arm) 749c⟩≡ (767b)
/*
 * Create a new endpoint for the device

```

```

* accessed via the given endpoint 0.
*/
static Ep*
newdevop(Ep *ep, int i, int tt, int mode)
{
    Ep *nep;
    Udev *d;

    d = ep->dev;
    if(d->eps[i] != nil)
        error("endpoint already in use");
    nep = epalloc(ep->hp);
    incref(ep);
    d->eps[i] = nep;
    nep->nb = i;
    nep->toggle[0] = nep->toggle[1] = 0;
    nep->ep0 = ep;
    nep->dev = ep->dev;
    nep->mode = mode;
    nep->ttype = tt;
    nep->debug = ep->debug;
    /* set defaults */
    switch(tt){
    case Tctl:
        nep->tmout = Xfertmout;
        break;
    case Tintr:
        nep->pollival = 10;
        break;
    case Tiso:
        nep->tmout = Xfertmout;
        nep->pollival = 10;
        nep->samplesz = 4;
        nep->hz = 44100;
        break;
    }
    deprint("newdevop ep%d.%d %#p\n", d->nb, nep->nb, nep);
    return ep;
}

```

*<function epdataperm(*arm*) 750a>* ≡ (767b)

```

static int
epdataperm(int mode)
{
    switch(mode){
    case OREAD:
        return 0440|DMEXCL;
        break;
    case OWRITE:
        return 0220|DMEXCL;
        break;
    default:
        return 0660|DMEXCL;
    }
}

```

*<function usbgen(*arm*) 750b>* ≡ (767b)

```

static int
usbgen(Chan *c, char *, Dirtab*, int, int s, Dir *dp)

```

```

{
    Qid q;
    Dirtab *dir;
    int perm;
    char *se;
    Ep *ep;
    int nb;
    int mode;

    if(0)ddprint("usbgen q %#x s %d...", QID(c->qid), s);
    if(s == DEVDOTDOT){
        if(QID(c->qid) <= Qusbdir){
            mkqid(&q, Qdir, 0, QTDIR);
            devdir(c, q, "#u", 0, eve, 0555, dp);
        }else{
            mkqid(&q, Qusbdir, 0, QTDIR);
            devdir(c, q, "usb", 0, eve, 0555, dp);
        }
        if(0)ddprint("ok\n");
        return 1;
    }

    switch(QID(c->qid)){
    case Qdir:                /* list #u */
        if(s == 0){
            mkqid(&q, Qusbdir, 0, QTDIR);
            devdir(c, q, "usb", 0, eve, 0555, dp);
            if(0)ddprint("ok\n");
            return 1;
        }
        s--;
        if(s < 0 || s >= epmax)
            goto Fail;
        ep = getep(s);
        if(ep == nil || ep->name == nil){
            if(ep != nil)
                putep(ep);
            if(0)ddprint("skip\n");
            return 0;
        }
        if(waserror()){
            putep(ep);
            nexterror();
        }
        mkqid(&q, QepOio+s*4, 0, QTFILE);
        devdir(c, q, ep->name, 0, eve, epdataperm(ep->mode), dp);
        putep(ep);
        poperror();
        if(0)ddprint("ok\n");
        return 1;

    case Qusbdir:            /* list #u/usb */
    Usbdir:
        if(s < nelem(usbdir)){
            dir = &usbdir[s];
            mkqid(&q, dir->qid.path, 0, QTFILE);
            devdir(c, q, dir->name, dir->length, eve, dir->perm, dp);
            if(0)ddprint("ok\n");
            return 1;
        }
    }
}

```



```

s -= nelem(usbdir);
if(s < 0 || s >= epmax)
    goto Fail;
ep = getep(s);
if(ep == nil){
    if(0)ddprint("skip\n");
    return 0;
}
if(waserror()){
    putep(ep);
    nexterror();
}
se = up->genbuf+sizeof(up->genbuf);
seprint(up->genbuf, se, "ep%d.%d", ep->dev->nb, ep->nb);
mkqid(&q, Qep0dir+4*s, 0, QTDIR);
putep(ep);
poperror();
devdir(c, q, up->genbuf, 0, eve, 0755, dp);
if(0)ddprint("ok\n");
return 1;

case Qctl:
    s = 0;
    goto Usbdir;

default:
    /* list #u/usb/epN.M */
    nb = qid2epidx(QID(c->qid));
    ep = getep(nb);
    if(ep == nil)
        goto Fail;
    mode = ep->mode;
    putep(ep);
    if(isqtype(QID(c->qid), Qepdir)){
        Emdir:
        switch(s){
            case 0:
                mkqid(&q, Qep0io+nb*4, 0, QTFILE);
                perm = epdataperm(mode);
                devdir(c, q, "data", 0, eve, perm, dp);
                break;
            case 1:
                mkqid(&q, Qep0ctl+nb*4, 0, QTFILE);
                devdir(c, q, "ctl", 0, eve, 0664, dp);
                break;
            default:
                goto Fail;
        }
    }else if(isqtype(QID(c->qid), Qepctl)){
        s = 1;
        goto Emdir;
    }else{
        s = 0;
        goto Emdir;
    }
    if(0)ddprint("ok\n");
    return 1;
}

Fail:
if(0)ddprint("fail\n");
return -1;

```

}

<function hciprobe(arm) 753a>≡ (767b)

```
static Hci*
hciprobe(int cardno, int ctrlnrno)
{
    Hci *hp;
    char *type;
    char name[64];
    static int epnb = 1;    /* guess the endpoint nb. for the controller */

    ddprint("hciprobe %d %d\n", cardno, ctrlnrno);
    hp = smalloc(sizeof(Hci));
    hp->ctrlnrno = ctrlnrno;

    if(cardno < 0)
        for(cardno = 0; cardno < Nhcis; cardno++){
            if(hcitypes[cardno].type == nil)
                break;
            type = hp->type;
            if(type==nil || *type==0)
                type = "uhci";
            if(cistrcmp(hcitypes[cardno].type, type) == 0)
                break;
        }

    if(cardno >= Nhcis || hcitypes[cardno].type == nil){
        free(hp);
        return nil;
    }
    dprint("%s...", hcitypes[cardno].type);
    if(hcitypes[cardno].reset(hp) < 0){
        free(hp);
        return nil;
    }

    snprint(name, sizeof(name), "usb%s", hcitypes[cardno].type);
    arch_intrenable(hp->irq, hp->interrupt, hp, UNKNOWN, name);

    print("#u/usb/ep%d.0: %s: port %#luX irq %d\n",
        epnb, hcitypes[cardno].type, hp->port, hp->irq);
    epnb++;

    return hp;
}
```

<function usbreset(arm) 753b>≡ (767b)

```
static void
usbreset(void)
{
    int cardno, ctrlnrno;
    Hci *hp;

    dprint("usbreset\n");

    for(ctrlnrno = 0; ctrlnrno < Nhcis; ctrlnrno++)
        if((hp = hciprobe(-1, ctrlnrno)) != nil)
            hcis[ctrlnrno] = hp;
    cardno = ctrlnrno = 0;
    while(cardno < Nhcis && ctrlnrno < Nhcis && hcitypes[cardno].type != nil)
```

```

        if(hcis[ctrlrno] != nil)
            ctrlrno++;
        else{
            hp = hciprobe(cardno, ctrlrno);
            if(hp == nil)
                cardno++;
            hcis[ctrlrno++] = hp;
        }
    if(hcis[Nhcis-1] != nil)
        print("usbreset: bug: Nhcis too small\n");
}

⟨function usbinit(arm) 754a⟩≡ (767b)
static void
usbinit(void)
{
    Hci *hp;
    int ctrlrno;
    Ep *d;
    char info[40];

    dprint("usbinit\n");
    for(ctrlrno = 0; ctrlrno < Nhcis; ctrlrno++){
        hp = hcis[ctrlrno];
        if(hp != nil){
            if(hp->init != nil)
                hp->init(hp);
            d = newdev(hp, 1, 1);      /* new root hub */
            d->dev->state = Denabled;  /* although addr == 0 */
            d->maxpkt = 64;
            snprintf(info, sizeof(info), "ports %d", hp->nports);
            kstrdup(&d->info, info);
        }
    }
}

⟨function usbattach(arm) 754b⟩≡ (767b)
static Chan*
usbattach(char *spec)
{
    return devattach(L'u', spec);
}

⟨function usbwalk(arm) 754c⟩≡ (767b)
static Walkqid*
usbwalk(Chan *c, Chan *nc, char **name, int nname)
{
    return devwalk(c, nc, name, nname, nil, 0, usbgen);
}

⟨function usbstat(arm) 754d⟩≡ (767b)
static int
usbstat(Chan *c, uchar *db, int n)
{
    return devstat(c, db, n, nil, 0, usbgen);
}

```

*<function usbload(*arm*) 755a>*≡ (767b)

```
/*
 *  $\mu$ s for the given transfer, for bandwidth allocation.
 * This is a very rough worst case for what 5.11.3
 * of the usb 2.0 spec says.
 * Also, we are using maxpkt and not actual transfer sizes.
 * Only when we are sure we
 * are not exceeding b/w might we consider adjusting it.
 */
static ulong
usbload(int speed, int maxpkt)
{
    enum{ Hostns = 1000, Hubns = 333 };
    ulong l;
    ulong bs;

    l = 0;
    bs = 10UL * maxpkt;
    switch(speed){
    case Highspeed:
        l = 55*8*2 + 2 * (3 + bs) + Hostns;
        break;
    case Fullspeed:
        l = 9107 + 84 * (4 + bs) + Hostns;
        break;
    case Lowspeed:
        l = 64107 + 2 * Hubns + 667 * (3 + bs) + Hostns;
        break;
    default:
        print("usbload: bad speed %d\n", speed);
        /* let it run */
    }
    return l / 1000UL; /* in  $\mu$ s */
}
```

*<function usbopen(*arm*) 755b>*≡ (767b)

```
static Chan*
usbopen(Chan *c, int omode)
{
    int q;
    Ep *ep;
    int mode;

    mode = openmode(omode);
    q = QID(c->qid);

    if(q >= Qep0dir && qid2epidx(q) < 0)
        error(Eio);
    if(q < Qep0dir || isqtype(q, Qepctl) || isqtype(q, Qepdir))
        return devopen(c, omode, nil, 0, usbgen);

    ep = getep(qid2epidx(q));
    if(ep == nil)
        error(Eio);
    deprint("usbopen q %#x fid %d omode %d\n", q, c->fid, mode);
    if(waserror()){
        putep(ep);
        nexterror();
    }
    qlock(ep);
}
```

```

    if(ep->inuse){
        qunlock(ep);
        error(Einuse);
    }
    ep->inuse = 1;
    qunlock(ep);
    if(waserror()){
        ep->inuse = 0;
        nexterror();
    }
    if(mode != OREAD && ep->mode == OREAD)
        error(Eperm);
    if(mode != OWRITE && ep->mode == OWRITE)
        error(Eperm);
    if(ep->ttype == Tnone)
        error(Enotconf);
    ep->clrhalt = 0;
    ep->rhrepl = -1;
    if(ep->load == 0)
        ep->load = usbload(ep->dev->speed, ep->maxpkt);
    ep->hp->epopen(ep);

    poperror(); /* ep->inuse */
    poperror(); /* don't putep(): ref kept for fid using the ep. */

    c->mode = mode;
    c->flag |= COPEN;
    c->offset = 0;
    c->aux = nil; /* paranoia */
    return c;
}

```

```

⟨function epclose(arm) 756a⟩≡ (767b)
static void
epclose(Ep *ep)
{
    qlock(ep);
    if(waserror()){
        qunlock(ep);
        nexterror();
    }
    if(ep->inuse){
        ep->hp->epclose(ep);
        ep->inuse = 0;
    }
    qunlock(ep);
    poperror();
}

```

```

⟨function usbclose(arm) 756b⟩≡ (767b)
static void
usbclose(Chan *c)
{
    int q;
    Ep *ep;

    q = QID(c->qid);
    if(q < Qep0dir || isqtype(q, Qepctl) || isqtype(q, Qepdir))
        return;
}

```

```

    ep = getep(qid2epidx(q));
    if(ep == nil)
        return;
    deprint("usbclose q %#x fid %d ref %ld\n", q, c->fid, ep->ref);
    if(waserror()){
        putep(ep);
        nexterror();
    }
    if(c->flag & COPEN){
        free(c->aux);
        c->aux = nil;
        epclose(ep);
        putep(ep); /* release ref kept since usbopen */
        c->flag &= ~COPEN;
    }
    poperror();
    putep(ep);
}

```

⟨function ctlread(*arm*) 757⟩≡ (767b)

```

static long
ctlread(Chan *c, void *a, long n, vlong offset)
{
    int q;
    char *s;
    char *us;
    char *se;
    Ep *ep;
    int i;

    q = QID(c->qid);
    us = s = smalloc(READSTR);
    se = s + READSTR;
    if(waserror()){
        free(us);
        nexterror();
    }
    if(q == Qctl)
        for(i = 0; i < epmax; i++){
            ep = getep(i);
            if(ep != nil){
                if(waserror()){
                    putep(ep);
                    nexterror();
                }
                s = seprint(s, se, "ep%d.%d ", ep->dev->nb, ep->nb);
                s = seprintep(s, se, ep, 0);
                poperror();
            }
            putep(ep);
        }
    else{
        ep = getep(qid2epidx(q));
        if(ep == nil)
            error(Eio);
        if(waserror()){
            putep(ep);
            nexterror();
        }
        if(c->aux != nil){

```

```

        /* After a new endpoint request we read
        * the new endpoint name back.
        */
        strcpy(s, se, c->aux);
        free(c->aux);
        c->aux = nil;
    }else
        seprintep(s, se, ep, 0);
    perror();
    putep(ep);
}
n = readstr(offset, a, n, us);
perror();
free(us);
return n;
}

```

⟨function rhubread(*arm*) 758a⟩≡ (767b)

```

/*
 * Fake root hub emulation.
 */
static long
rhubread(Ep *ep, void *a, long n)
{
    char *b;

    if(ep->dev->isroot == 0 || ep->nb != 0 || n < 2)
        return -1;
    if(ep->rhrepl < 0)
        return -1;

    b = a;
    memset(b, 0, n);
    PUT2(b, ep->rhrepl);
    ep->rhrepl = -1;
    return n;
}

```

⟨function rhubwrite(*arm*) 758b⟩≡ (767b)

```

static long
rhubwrite(Ep *ep, void *a, long n)
{
    uchar *s;
    int cmd;
    int feature;
    int port;
    Hci *hp;

    if(ep->dev == nil || ep->dev->isroot == 0 || ep->nb != 0)
        return -1;
    if(n != Rsetuplen)
        error("root hub is a toy hub");
    ep->rhrepl = -1;
    s = a;
    if(s[Rtype] != (Rh2d|Rclass|Rother) && s[Rtype] != (Rd2h|Rclass|Rother))
        error("root hub is a toy hub");
    hp = ep->hp;
    cmd = s[Rreq];
    feature = GET2(s+Rvalue);
    port = GET2(s+Rindex);
}

```

```

if(port < 1 || port > hp->nports)
    error("bad hub port number");
switch(feature){
case Rportenable:
    ep->rhrepl = hp->portenable(hp, port, cmd == Rsetfeature);
    break;
case Rportreset:
    ep->rhrepl = hp->portreset(hp, port, cmd == Rsetfeature);
    break;
case Rgetstatus:
    ep->rhrepl = hp->portstatus(hp, port);
    break;
default:
    ep->rhrepl = 0;
}
return n;
}

```

*<function usbread(*arm*) 759>*≡ (767b)

```

static long
usbread(Chan *c, void *a, long n, vlong offset)
{
    int q;
    Ep *ep;
    int nr;

    q = QID(c->qid);

    if(c->qid.type == QTDIR)
        return devdirread(c, a, n, nil, 0, usbgen);

    if(q == Qctl || isqtype(q, Qepctl))
        return ctlread(c, a, n, offset);

    ep = getep(qid2epidx(q));
    if(ep == nil)
        error(Eio);
    if(waserror()){
        putep(ep);
        nexterror();
    }
    if(ep->dev->state == Ddetach)
        error(Edetach);
    if(ep->mode == OWRITE || ep->inuse == 0)
        error(Ebadusefd);
    switch(ep->ttype){
case Tnone:
    error("endpoint not configured");
case Tctl:
    nr = rhubread(ep, a, n);
    if(nr >= 0){
        n = nr;
        break;
    }
    /* else fall */
default:
    ddeprint("\nusbread q %#x fid %d cnt %ld off %lld\n",q,c->fid,n,offset);
    n = ep->hp->epread(ep, a, n);
    break;
}
}

```



```

        poperror();
        putep(ep);
        return n;
    }

⟨function pow2(arm) 760a⟩≡ (767b)
    static long
    pow2(int n)
    {
        return 1 << n;
    }

⟨function setmaxpkt(arm) 760b⟩≡ (767b)
    static void
    setmaxpkt(Ep *ep, char* s)
    {
        long spp;    /* samples per packet */

        if(ep->dev->speed == Highspeed)
            spp = (ep->hz * ep->pollival * ep->ntds + 7999) / 8000;
        else
            spp = (ep->hz * ep->pollival + 999) / 1000;
        ep->maxpkt = spp * ep->samplesz;
        deprint("usb: %s: setmaxpkt: hz %ld poll %ld"
            " ntds %d %s speed -> spp %ld maxpkt %ld\n", s,
            ep->hz, ep->pollival, ep->ntds, sname[ep->dev->speed],
            spp, ep->maxpkt);
        if(ep->maxpkt > 1024){
            print("usb: %s: maxpkt %ld > 1024. truncating\n", s, ep->maxpkt);
            ep->maxpkt = 1024;
        }
    }

⟨function epctl(arm) 760c⟩≡ (767b)
    /*
     * Many endpoint ctls. simply update the portable representation
     * of the endpoint. The actual controller driver will look
     * at them to setup the endpoints as dictated.
     */
    static long
    epctl(Ep *ep, Chan *c, void *a, long n)
    {
        int i, l, mode, nb, tt;
        char *b, *s;
        Cmdbuf *cb;
        Cmdtab *ct;
        Ep *nep;
        Udev *d;
        static char *Info = "info ";

        d = ep->dev;

        cb = parsecmd(a, n);
        if(waserror()){
            free(cb);
            nexterror();
        }
        ct = lookupcmd(cb, epctls, nelem(epctls));
        if(ct == nil)
            error(Ebadctl);
    }

```

```

i = ct->index;
if(i == CMnew || i == CMspeed || i == CMhub || i == CMpreset)
    if(ep != ep->ep0)
        error("allowed only on a setup endpoint");
if(i != CMclrhalt && i != CMdetach && i != CMdebugep && i != CMname)
    if(ep != ep->ep0 && ep->inuse != 0)
        error("must configure before using");
switch(i){
case CMnew:
    deprint("usb epctl %s\n", cb->f[0]);
    nb = strtoul(cb->f[1], nil, 0);
    if(nb < 0 || nb >= Ndeveps)
        error("bad endpoint number");
    tt = name2ttype(cb->f[2]);
    if(tt == Tnone)
        error("unknown endpoint type");
    mode = name2mode(cb->f[3]);
    if(mode < 0)
        error("unknown i/o mode");
    newdev(ep, nb, tt, mode);
    break;
case CMnewdev:
    deprint("usb epctl %s\n", cb->f[0]);
    if(ep != ep->ep0 || d->ishub == 0)
        error("not a hub setup endpoint");
    l = name2speed(cb->f[1]);
    if(l == Nospeed)
        error("speed must be full|low|high");
    nep = newdev(ep->hp, 0, 0);
    nep->dev->speed = l;
    if(nep->dev->speed != Lowspeed)
        nep->maxpkt = 64; /* assume full speed */
    nep->dev->hub = d->nb;
    nep->dev->port = atoi(cb->f[2]);
    /* next read request will read
     * the name for the new endpoint
     */
    l = sizeof(up->genbuf);
    snprintf(up->genbuf, l, "ep%d.%d", nep->dev->nb, nep->nb);
    kstrdup(&c->aux, up->genbuf);
    break;
case CMhub:
    deprint("usb epctl %s\n", cb->f[0]);
    d->ishub = 1;
    break;
case CMspeed:
    l = name2speed(cb->f[1]);
    deprint("usb epctl %s %d\n", cb->f[0], l);
    if(l == Nospeed)
        error("speed must be full|low|high");
    qlock(ep->ep0);
    d->speed = l;
    qunlock(ep->ep0);
    break;
case CMmaxpkt:
    l = strtoul(cb->f[1], nil, 0);
    deprint("usb epctl %s %d\n", cb->f[0], l);
    if(l < 1 || l > 1024)
        error("maxpkt not in [1:1024]");
    qlock(ep);

```

```

    ep->maxpkt = 1;
    qunlock(ep);
    break;
case CMntds:
    l = strtoul(cb->f[1], nil, 0);
    deprint("usb epctl %s %d\n", cb->f[0], l);
    if(l < 1 || l > 3)
        error("ntds not in [1:3]");
    qlock(ep);
    ep->ntds = 1;
    qunlock(ep);
    break;
case CMpollival:
    if(ep->ttype != Tintr && ep->ttype != Tiso)
        error("not an intr or iso endpoint");
    l = strtoul(cb->f[1], nil, 0);
    deprint("usb epctl %s %d\n", cb->f[0], l);
    if(ep->ttype == Tiso ||
        (ep->ttype == Tintr && ep->dev->speed == Highspeed)){
        if(l < 1 || l > 16)
            error("pollival power not in [1:16]");
        l = pow2(l-1);
    }else
        if(l < 1 || l > 255)
            error("pollival not in [1:255]");
    qlock(ep);
    ep->pollival = 1;
    if(ep->ttype == Tiso)
        setmaxpkt(ep, "pollival");
    qunlock(ep);
    break;
case CMsamplesz:
    if(ep->ttype != Tiso)
        error("not an iso endpoint");
    l = strtoul(cb->f[1], nil, 0);
    deprint("usb epctl %s %d\n", cb->f[0], l);
    if(l <= 0 || l > 8)
        error("samplesz not in [1:8]");
    qlock(ep);
    ep->samplesz = 1;
    setmaxpkt(ep, "samplesz");
    qunlock(ep);
    break;
case CMhz:
    if(ep->ttype != Tiso)
        error("not an iso endpoint");
    l = strtoul(cb->f[1], nil, 0);
    deprint("usb epctl %s %d\n", cb->f[0], l);
    if(l <= 0 || l > 100000)
        error("hz not in [1:100000]");
    qlock(ep);
    ep->hz = 1;
    setmaxpkt(ep, "hz");
    qunlock(ep);
    break;
case CMclrhalt:
    qlock(ep);
    deprint("usb epctl %s\n", cb->f[0]);
    ep->clrhalt = 1;
    qunlock(ep);

```

```

        break;
case CMinfo:
    deprint("usb epctl %s\n", cb->f[0]);
    l = strlen(Info);
    s = a;
    if(n < l+2 || strncmp(Info, s, l) != 0)
        error(Ebadctl);
    if(n > 1024)
        n = 1024;
    b = smalloc(n);
    memmove(b, s+1, n-1);
    b[n-1] = 0;
    if(b[n-1-1] == '\n')
        b[n-1-1] = 0;
    qlock(ep);
    free(ep->info);
    ep->info = b;
    qunlock(ep);
    break;
case CMAccess:
    deprint("usb epctl %s\n", cb->f[0]);
    ep->dev->state = Denabled;
    break;
case CMdetach:
    if(ep->dev->isroot != 0)
        error("can't detach a root hub");
    deprint("usb epctl %s ep%d.%d\n",
        cb->f[0], ep->dev->nb, ep->nb);
    ep->dev->state = Ddetach;
    /* Release file system ref. for its endpoints */
    for(i = 0; i < nelem(ep->dev->eps); i++)
        putep(ep->dev->eps[i]);
    break;
case CMdebugep:
    if(strcmp(cb->f[1], "on") == 0)
        ep->debug = 1;
    else if(strcmp(cb->f[1], "off") == 0)
        ep->debug = 0;
    else
        ep->debug = strtoul(cb->f[1], nil, 0);
    print("usb: ep%d.%d debug %d\n",
        ep->dev->nb, ep->nb, ep->debug);
    break;
case CMname:
    deprint("usb epctl %s %s\n", cb->f[0], cb->f[1]);
    validname(cb->f[1], 0);
    kstrdup(&ep->name, cb->f[1]);
    break;
case CMtmout:
    deprint("usb epctl %s\n", cb->f[0]);
    if(ep->ttype == Tiso || ep->ttype == Tctl)
        error("ctl ignored for this endpoint type");
    ep->tmout = strtoul(cb->f[1], nil, 0);
    if(ep->tmout != 0 && ep->tmout < Xfertmout)
        ep->tmout = Xfertmout;
    break;
case CMpreset:
    deprint("usb epctl %s\n", cb->f[0]);
    if(ep->ttype != Tctl)
        error("not a control endpoint");

```

```

        if(ep->dev->state != Denabled)
            error("forbidden on devices not enabled");
        ep->dev->state = Dreset;
        break;
default:
    panic("usb: unknown epctl %d", ct->index);
}
free(cb);
poperror();
return n;
}

```

<function usbctl(arm) 764a>≡ (767b)

```

static long
usbctl(void *a, long n)
{
    Cmdtab *ct;
    Cmdbuf *cb;
    Ep *ep;
    int i;

    cb = parsecmd(a, n);
    if(waserror()){
        free(cb);
        nexterror();
    }
    ct = lookupcmd(cb, usbctls, nelem(usbctls));
    dprint("usb ctl %s\n", cb->f[0]);
    switch(ct->index){
case CMdebug:
    if(strcmp(cb->f[1], "on") == 0)
        debug = 1;
    else if(strcmp(cb->f[1], "off") == 0)
        debug = 0;
    else
        debug = strtol(cb->f[1], nil, 0);
    print("usb: debug %d\n", debug);
    for(i = 0; i < epmax; i++){
        if((ep = getep(i)) != nil){
            ep->hp->debug(ep->hp, debug);
            putep(ep);
        }
    }
    break;
case CMdump:
    dumpeps();
    break;
    }
    free(cb);
    poperror();
    return n;
}

```

<function ctlwrite(arm) 764b>≡ (767b)

```

static long
ctlwrite(Chan *c, void *a, long n)
{
    int q;
    Ep *ep;

    q = QID(c->qid);

```

```

if(q == Qctl)
    return usbctl(a, n);

ep = getep(qid2epidx(q));
if(ep == nil)
    error(Eio);
if(waserror()){
    putep(ep);
    nexterror();
}
if(ep->dev->state == Ddetach)
    error(Edetach);
if(isqtype(q, Qepctl) && c->aux != nil){
    /* Be sure we don't keep a cloned ep name */
    free(c->aux);
    c->aux = nil;
    error("read, not write, expected");
}
n = epctl(ep, c, a, n);
putep(ep);
poperror();
return n;
}

```

*<function usbwrite(*arm*) 765>*≡ (767b)

```

static long
usbwrite(Chan *c, void *a, long n, vlong off)
{
    int nr, q;
    Ep *ep;

    if(c->qid.type == QTDIR)
        error(Eisdir);

    q = QID(c->qid);

    if(q == Qctl || isqtype(q, Qepctl))
        return ctlwrite(c, a, n);

    ep = getep(qid2epidx(q));
    if(ep == nil)
        error(Eio);
    if(waserror()){
        putep(ep);
        nexterror();
    }
    if(ep->dev->state == Ddetach)
        error(Edetach);
    if(ep->mode == OREAD || ep->inuse == 0)
        error(Ebadusefd);

    switch(ep->ttype){
    case Tnone:
        error("endpoint not configured");
    case Tctl:
        nr = rhubwrite(ep, a, n);
        if(nr >= 0){
            n = nr;
            break;
        }
    }
}

```

```

        /* else fall */
default:
    ddeprint("\nusbwrite q %#x fid %d cnt %ld off %lld\n",q, c->fid, n, off);
    ep->hp->epwrite(ep, a, n);
}
putep(ep);
poperror();
return n;
}

⟨function usbbread(arm) 766a⟩≡ (767b)
Block*
usbbread(Chan *c, long n, ulong offset)
{
    Block *bp;

    bp = allocb(n);
    if(bp == 0)
        error(Enomem);
    if(waserror()) {
        freeb(bp);
        nexterror();
    }
    bp->wp += usbread(c, bp->wp, n, offset);
    poperror();
    return bp;
}

⟨function usbbwrite(arm) 766b⟩≡ (767b)
long
usbbwrite(Chan *c, Block *bp, ulong offset)
{
    long n;

    if(waserror()) {
        freeb(bp);
        nexterror();
    }
    n = usbwrite(c, bp->rp, BLEN(bp), offset);
    poperror();
    freeb(bp);

    return n;
}

⟨function usbshutdown(arm) 766c⟩≡ (767b)
void
usbshutdown(void)
{
    Hci *hp;
    int i;

    for(i = 0; i < Nhcis; i++){
        hp = hcis[i];
        if(hp == nil)
            continue;
        if(hp->shutdown == nil)
            print("#u: no shutdown function for %s\n", hp->type);
        else
            hp->shutdown(hp);
    }
}

```

```

    }
}

<global usbdevtab(arm) 767a>≡ (767b)
Dev usbdevtab = {
    .dc = L'u',
    .name = "usb",

    .reset = usbreset,
    .init = usbinit,
    .shutdown = usbshutdown,
    .attach = usbattach,
    .walk = usbwalk,
    .stat = usbstat,
    .open = usbopen,
    .create = devcreate,
    .close = usbclose,
    .read = usbread,
    .bread = usbbread,
    .write = usbwrite,
    .bwrite = usbbwrite,
    .remove = devremove,
    .wstat = devwstat,
};

<buses/arm/devusb.c 767b>≡
/*
 * USB device driver framework.
 *
 * This is in charge of providing access to actual HCIs
 * and providing I/O to the various endpoints of devices.
 * A separate user program (usbd) is in charge of
 * enumerating the bus, setting up endpoints and
 * starting devices (also user programs).
 *
 * The interface provided is a violation of the standard:
 * you're welcome.
 *
 * The interface consists of a root directory with several files
 * plus a directory (epN.M) with two files per endpoint.
 * A device is represented by its first endpoint, which
 * is a control endpoint automatically allocated for each device.
 * Device control endpoints may be used to create new endpoints.
 * Devices corresponding to hubs may also allocate new devices,
 * perhaps also hubs. Initially, a hub device is allocated for
 * each controller present, to represent its root hub. Those can
 * never be removed.
 *
 * All endpoints refer to the first endpoint (epN.0) of the device,
 * which keeps per-device information, and also to the HCI used
 * to reach them. Although all endpoints cache that information.
 *
 * epN.M/data files permit I/O and are considered DMEXCL.
 * epN.M/ctl files provide status info and accept control requests.
 *
 * Endpoints may be given file names to be listed also at #u,
 * for those drivers that have nothing to do after configuring the
 * device and its endpoints.
 *
 * Drivers for different controllers are kept at usb[oue]hci.c

```



```

* It's likely we could factor out much from controllers into
* a generic controller driver, the problem is that details
* regarding how to handle toggles, tokens, Tds, etc. will
* get in the way. Thus, code is probably easier the way it is.
*/

```

<kernel basic includes 35a>

```

#include    "io.h"
#include    "../port/usb.h"

```

```

typedef struct Hcctype Hcctype;

```

<enum _anon_((buses/arm/devusb.c)(arm)) 742>

<struct Hcctype(arm) 743a>

<macro QID(arm) 743b>

<global usbctls(arm) 743c>

<global epctls(arm) 743d>

<global usbdir(arm) 743e>

<global usbmodename(arm) 743f>

<global ttname(arm) 744a>

<global spname(arm) 744b>

<global debug(arm) 744c>

<global hcictypes(arm) 744d>

<global hcis(arm) 744e>

<global epslck(arm) 744f>

<global eps(arm) 744g>

<global epmax(arm) 744h>

<global usbidgen(arm) 744i>

<function seprintdata(arm) 744j>

<function name2speed(arm) 745a>

<function name2ttype(arm) 745b>

<function name2mode(arm) 745c>

<function qid2epidx(arm) 745d>

<function isqtype(arm) 745e>

<function addhcctype(arm) 746a>

<function seprintep(arm) 746b>

<function epalloc(arm) 747a>

<function getep(arm) 747b>

<function putep(arm) 748a>

`<function dumpeps(arm) 748b>`
`<function newusbid(arm) 749a>`
`<function newdev(arm) 749b>`
`<function newdevop(arm) 749c>`
`<function epdataperm(arm) 750a>`
`<function usbgen(arm) 750b>`
`<function hciprobe(arm) 753a>`
`<function usbreset(arm) 753b>`
`<function usbinit(arm) 754a>`
`<function usbattach(arm) 754b>`
`<function usbwalk(arm) 754c>`
`<function usbstat(arm) 754d>`
`<function usbload(arm) 755a>`
`<function usbopen(arm) 755b>`
`<function epclose(arm) 756a>`
`<function usbclose(arm) 756b>`
`<function ctlread(arm) 757>`
`<function rhubread(arm) 758a>`
`<function rhubwrite(arm) 758b>`
`<function usbread(arm) 759>`
`<function pow2(arm) 760a>`
`<function setmaxpkt(arm) 760b>`
`<function epctl(arm) 760c>`
`<function usbctl(arm) 764a>`
`<function ctlwrite(arm) 764b>`
`<function usbwrite(arm) 765>`
`<function usbbread(arm) 766a>`
`<function usbbwrite(arm) 766b>`
`<function usbshutdown(arm) 766c>`
`<global usbdevtab(arm) 767a>`

E.29.6 buses/arm/usbdwc.c

```
<enum _anon_((buses/arm/usbdwc.c)(arm)) 770a>≡ (787b)
enum
{
    Enabledelay = 50,
    Resetdelay  = 10,
    ResetdelayHS = 50,

    Read      = 0,
    Write     = 1,
};

<struct Ctlr((buses/arm/usbdwc.c)(arm)) 770b>≡ (787b)
struct Ctlr {
    Lock;
    Dwcregs *regs; /* controller registers */
    int nchan; /* number of host channels */
    ulong chanbusy; /* bitmap of in-use channels */
    QLock chanlock; /* serialise access to chanbusy */
    QLock split; /* serialise split transactions */
    int splitretry; /* count retries of Nyet */
    int sofchan; /* bitmap of channels waiting for sof */
    int wakechan; /* bitmap of channels to wakeup after fiq */
    int debugchan; /* bitmap of channels for interrupt debug */
    Rendez *chanintr; /* sleep till interrupt on channel N */
};

<struct Epio(arm) 770c>≡ (787b)
struct Epio {
    QLock;
    Block *cb;
    ulong lastpoll;
};

<global dwc(arm) 770d>≡ (787b)
static Ctlr dwc;

<global debug((buses/arm/usbdwc.c)(arm)) 770e>≡ (787b)
static int debug;

<global Ebadlen(arm) 770f>≡ (787b)
static char Ebadlen[] = "bad usb request length";

<function filock(arm) 770g>≡ (787b)
static void
filock(Lock *l)
{
    int x;

    x = splfhi();
    ilock(l);
    l->sr = x;
}

<function fiunlock(arm) 770h>≡ (787b)
static void
fiunlock(Lock *l)
{
    iunlock(l);
}
```

*<function chanalloc(*arm*) 771a>*≡ (787b)

```
static Hostchan*
chanalloc(Ep *ep)
{
    Ctlr *ctlr;
    int bitmap, i;

    ctlr = ep->hp->aux;
    qlock(&ctlr->chanlock);
    bitmap = ctlr->chanbusy;
    for(i = 0; i < ctlr->nchan; i++)
        if((bitmap & (1<<i)) == 0){
            ctlr->chanbusy = bitmap | 1<<i;
            qunlock(&ctlr->chanlock);
            return &ctlr->regs->hchan[i];
        }
    qunlock(&ctlr->chanlock);
    panic("miller is a lazy git");
    return nil;
}
```

*<function chanrelease(*arm*) 771b>*≡ (787b)

```
static void
chanrelease(Ep *ep, Hostchan *chan)
{
    Ctlr *ctlr;
    int i;

    ctlr = ep->hp->aux;
    i = chan - ctlr->regs->hchan;
    qlock(&ctlr->chanlock);
    ctlr->chanbusy &= ~(1<<i);
    qunlock(&ctlr->chanlock);
}
```

*<function chansetup(*arm*) 771c>*≡ (787b)

```
static void
chansetup(Hostchan *hc, Ep *ep)
{
    int hcc;
    Ctlr *ctlr = ep->hp->aux;

    if(ep->debug)
        ctlr->debugchan |= 1 << (hc - ctlr->regs->hchan);
    else
        ctlr->debugchan &= ~(1 << (hc - ctlr->regs->hchan));
    switch(ep->dev->state){
    case Dconfig:
    case Dreset:
        hcc = 0;
        break;
    default:
        hcc = ep->dev->nb<<ODevaddr;
        break;
    }
    hcc |= ep->maxpkt | 1<<OMulticnt | ep->nb<<OEpnun;
    switch(ep->ttype){
    case Tctl:
        hcc |= Epctl;
        break;
    }
```

```

case Tiso:
    hcc |= Episo;
    break;
case Tbulk:
    hcc |= Epbulk;
    break;
case Tintr:
    hcc |= Epintr;
    break;
}
switch(ep->dev->speed){
case Lowspeed:
    hcc |= Lspddev;
    /* fall through */
case Fullspeed:
    if(ep->dev->hub > 1){
        hc->hcsplt = Spltena | POS_ALL | ep->dev->hub<<OHubaddr |
            ep->dev->port;
        break;
    }
    /* fall through */
default:
    hc->hcsplt = 0;
    break;
}
hc->hcchar = hcc;
hc->hcint = ~0;
}

```

⟨function sofdone(*arm*) 772a⟩≡ (787b)

```

static int
sofdone(void *a)
{
    Dwcregs *r;

    r = a;
    return (r->gintmsk & Sofintr) == 0;
}

```

⟨function sofwait(*arm*) 772b⟩≡ (787b)

```

static void
sofwait(Ctlr *ctlr, int n)
{
    Dwcregs *r;

    r = ctlr->regs;
    do{
        filock(ctlr);
        r->gintsts = Sofintr;
        ctlr->sofchan |= 1<<n;
        r->gintmsk |= Sofintr;
        fiunlock(ctlr);
        sleep(&ctlr->chanintr[n], sofdone, r);
    }while((r->hfnum & 7) == 6);
}

```

⟨function chandone(*arm*) 772c⟩≡ (787b)

```

static int
chandone(void *a)
{

```

```

Hostchan *hc;

hc = a;
if(hc->hcint == (Chhltd|Ack))
    return 0;
return (hc->hcint & hc->hcintmsk) != 0;
}

```

⟨function chanwait(*arm*) 773⟩≡ (787b)

```

static int
chanwait(Ep *ep, Ctlr *ctlr, Hostchan *hc, int mask)
{
    int intr, n, ointr;
    ulong start, now;
    Dwcregs *r;

    r = ctlr->regs;
    n = hc - r->hchan;
    for(;;){
restart:
        filock(ctlr);
        r->haintmsk |= 1<<n;
        hc->hcintmsk = mask;
        fiunlock(ctlr);
        sleep(&ctlr->chanintr[n], chandone, hc);
        hc->hcintmsk = 0;
        intr = hc->hcint;
        if(intr & Chhltd)
            return intr;
        start = arch_fastticks(0);
        ointr = intr;
        now = start;
        do{
            intr = hc->hcint;
            if(intr & Chhltd){
                if((ointr != Ack && ointr != (Ack|Xfercomp)) ||
                    intr != (Ack|Chhltd|Xfercomp) ||
                    (now - start) > 60)
                    dprint("await %x after %ld %x -> %x\n",
                        mask, now - start, ointr, intr);
                return intr;
            }
            if((intr & mask) == 0){
                dprint("ep%d.%d await %x intr %x -> %x\n",
                    ep->dev->nb, ep->nb, mask, ointr, intr);
                goto restart;
            }
            now = arch_fastticks(0);
        }while(now - start < 100);
        dprint("ep%d.%d halting channel %8.8ux hcchar %8.8ux "
            "grxsts %8.8ux gnptxsts %8.8ux hptxsts %8.8ux\n",
            ep->dev->nb, ep->nb, intr, hc->hcchar, r->grxsts,
            r->gnptxsts, r->hptxsts);
        mask = Chhltd;
        hc->hcchar |= Chdis;
        start = cpu->ticks;
        while(hc->hcchar & Chen){
            if(cpu->ticks - start >= 100){
                print("ep%d.%d channel won't halt hcchar %8.8ux\n",
                    ep->dev->nb, ep->nb, hc->hcchar);
            }
        }
    }
}

```

```

        break;
    }
}
logdump(ep);
}
}

```

⟨function chanintr(*arm*) 774a⟩≡ (787b)

```

static int
chanintr(Ctlr *ctlr, int n)
{
    Hostchan *hc;
    int i;

    hc = &ctlr->regs->hchan[n];
    if(ctlr->debugchan & (1<<n))
        clog(nil, hc);
    if((hc->hcsplt & Spltena) == 0)
        return 0;
    i = hc->hcint;
    if(i == (Chhltd|Ack)){
        hc->hcsplt |= Compsplt;
        ctlr->splitretry = 0;
    }else if(i == (Chhltd|Nyet)){
        if(++ctlr->splitretry >= 3)
            return 0;
    }else
        return 0;
    if(hc->hcchar & Chen){
        iprint("hcchar %8.8ux hcint %8.8ux", hc->hcchar, hc->hcint);
        hc->hcchar |= Chen | Chdis;
        while(hc->hcchar&Chen)
            ;
        iprint(" %8.8ux\n", hc->hcint);
    }
    hc->hcint = i;
    if(ctlr->regs->hfnum & 1)
        hc->hcchar &= ~Oddfrm;
    else
        hc->hcchar |= Oddfrm;
    hc->hcchar = (hc->hcchar &~ Chdis) | Chen;
    return 1;
}

```

⟨global chanlog(*arm*) 774b⟩≡ (787b)

```

static Reg chanlog[32][5];

```

⟨global nchanlog(*arm*) 774c⟩≡ (787b)

```

static int nchanlog;

```

⟨function logstart(*arm*) 774d⟩≡ (787b)

```

static void
logstart(Ep *ep)
{
    if(ep->debug)
        nchanlog = 0;
}

```

```

⟨function clog(arm) 775a)≡ (787b)
static void
clog(Ep *ep, Hostchan *hc)
{
    Reg *p;

    if(ep != nil && !ep->debug)
        return;
    if(nchanlog == 32)
        nchanlog--;
    p = chanlog[nchanlog];
    p[0] = dwc.regs->hfnum;
    p[1] = hc->hcchar;
    p[2] = hc->hcint;
    p[3] = hc->hctsiz;
    p[4] = hc->hcdma;
    nchanlog++;
}

```

```

⟨function logdump(arm) 775b)≡ (787b)
static void
logdump(Ep *ep)
{
    Reg *p;
    int i;

    if(!ep->debug)
        return;
    p = chanlog[0];
    for(i = 0; i < nchanlog; i++){
        print("%5.5d.%5.5d %8.8ux %8.8ux %8.8ux %8.8ux\n",
            p[0]&0xFFFF, p[0]>>16, p[1], p[2], p[3], p[4]);
        p += 5;
    }
    nchanlog = 0;
}

```

```

⟨function chanio(arm) 775c)≡ (787b)
static int
chanio(Ep *ep, Hostchan *hc, int dir, int pid, void *a, int len)
{
    Ctlr *ctlr;
    int nleft, n, nt, i, maxpkt, npkt;
    uint hcdma, hctsiz;

    ctlr = ep->hp->aux;
    maxpkt = ep->maxpkt;
    npkt = HOWMANY(len, ep->maxpkt);
    if(npkt == 0)
        npkt = 1;

    hc->hcchar = (hc->hcchar & ~Epdire) | dir;
    if(dir == Epin)
        n = ROUND(len, ep->maxpkt);
    else
        n = len;
    hc->hctsiz = n | npkt<<OPktcnt | pid;
    hc->hcdma = dmaaddr(a);

    nleft = len;
}

```



```

logstart(ep);
for(;;){
    hcdma = hc->hcdma;
    hctsiz = hc->hctsiz;
    hc->hctsiz = hctsiz & ~Dopng;
    if(hc->hcchar&Chen){
        dprint("ep%d.%d before chanio hcchar=%8.8ux\n",
            ep->dev->nb, ep->nb, hc->hcchar);
        hc->hcchar |= Chen | Chdis;
        while(hc->hcchar&Chen)
            ;
        hc->hcint = Chhltd;
    }
    if((i = hc->hcint) != 0){
        dprint("ep%d.%d before chanio hcint=%8.8ux\n",
            ep->dev->nb, ep->nb, i);
        hc->hcint = i;
    }
    if(hc->hcsplt & Spltena){
        qlock(&ctrl->split);
        sofwait(ctrl, hc - ctrl->regs->hchan);
        if((dwc.regs->hfnum & 1) == 0)
            hc->hcchar &= ~Oddfrm;
        else
            hc->hcchar |= Oddfrm;
    }
    hc->hcchar = (hc->hcchar &~ Chdis) | Chen;
    clog(ep, hc);
    if(ep->ttype == Tbulk && dir == Epin)
        i = chanwait(ep, ctrl, hc, /* Ack| */ Chhltd);
    else if(ep->ttype == Tintr && (hc->hcsplt & Spltena))
        i = chanwait(ep, ctrl, hc, Chhltd);
    else
        i = chanwait(ep, ctrl, hc, Chhltd|Nak);
    clog(ep, hc);
    hc->hcint = i;

    if(hc->hcsplt & Spltena){
        hc->hcsplt &= ~Compsplt;
        qunlock(&ctrl->split);
    }

    if((i & Xfercomp) == 0 && i != (Chhltd|Ack) && i != Chhltd){
        if(i & Stall)
            error(Estalled);
        if(i & (Nyet|Frmovrun))
            continue;
        if(i & Nak){
            if(ep->ttype == Tintr)
                tsleep(&up->sleepr, returnfalse, 0, ep->pollival);
            else
                tsleep(&up->sleepr, returnfalse, 0, 1);
            continue;
        }
        logdump(ep);
        print("usbotg: ep%d.%d error intr %8.8ux\n",
            ep->dev->nb, ep->nb, i);
        if(i & ~(Chhltd|Ack))
            error(Eio);
        if(hc->hcdma != hcdma)

```

```

        print("usbotg: weird hcdma %x->%x intr %x->%x\n",
              hcdma, hc->hcdma, i, hc->hcint);
    }
    n = hc->hcdma - hcdma;
    if(n == 0){
        if((hc->hchtsiz & Pktcnt) != (hchtsiz & Pktcnt))
            break;
        else
            continue;
    }
    if(dir == Epin && ep->ttype == Tbulk && n == nleft){
        nt = (hchtsiz & Xfersize) - (hc->hchtsiz & Xfersize);
        if(nt != n){
            if(n == ROUND(nt, 4))
                n = nt;
            else
                print("usbotg: intr %8.8ux "
                      "dma %8.8ux-%8.8ux "
                      "hchtsiz %8.8ux-%8.ux\n",
                      i, hcdma, hc->hcdma, hchtsiz,
                      hc->hchtsiz);
        }
    }
    if(n > nleft){
        if(n != ROUND(nleft, 4))
            dprint("too much: wanted %d got %d\n",
                  len, len - nleft + n);
        n = nleft;
    }
    nleft -= n;
    if(nleft == 0 || (n % maxpkt) != 0)
        break;
    if((i & Xfercomp) && ep->ttype != Tctl)
        break;
    if(dir == Epout)
        dprint("too little: nleft %d hcdma %x->%x hchtsiz %x->%x intr %x\n",
              nleft, hcdma, hc->hcdma, hchtsiz, hc->hchtsiz, i);
}
logdump(ep);
return len - nleft;
}

```

*<function multitrans(*arm*) 777>*≡ (787b)

```

static long
multitrans(Ep *ep, Hostchan *hc, int rw, void *a, long n)
{
    long sofar, m;

    sofar = 0;
    do{
        m = n - sofar;
        if(m > ep->maxpkt)
            m = ep->maxpkt;
        m = chanio(ep, hc, rw == Read? Epin : Epout, ep->toggle[rw],
                  (char*)a + sofar, m);
        ep->toggle[rw] = hc->hchtsiz & Pid;
        sofar += m;
    }while(sofar < n && m == ep->maxpkt);
    return sofar;
}

```

*<function eptrans(*arm*) 778a>*≡ (787b)

```
static long
eptrans(Ep *ep, int rw, void *a, long n)
{
    Hostchan *hc;

    if(ep->clrhalt){
        ep->clrhalt = 0;
        if(ep->mode != OREAD)
            ep->toggle[Write] = DATA0;
        if(ep->mode != OWRITE)
            ep->toggle[Read] = DATA0;
    }
    hc = chanalloc(ep);
    if(waserror()){
        ep->toggle[rw] = hc->hchtsiz & Pid;
        chanrelease(ep, hc);
        if(strcmp(up->errstr, Estalled) == 0)
            return 0;
        nexterror();
    }
    chansetup(hc, ep);
    if(rw == Read && ep->ttype == Tbulk)
        n = multitrans(ep, hc, rw, a, n);
    else{
        n = chanio(ep, hc, rw == Read? Epin : Epout, ep->toggle[rw],
            a, n);
        ep->toggle[rw] = hc->hchtsiz & Pid;
    }
    chanrelease(ep, hc);
    poperror();
    return n;
}
```

*<function ctltrans(*arm*) 778b>*≡ (787b)

```
static long
ctltrans(Ep *ep, uchar *req, long n)
{
    Hostchan *hc;
    Epio *epio;
    Block *b;
    uchar *data;
    int datalen;

    epio = ep->aux;
    if(epio->cb != nil){
        freeb(epio->cb);
        epio->cb = nil;
    }
    if(n < Rsetuplen)
        error(Ebadlen);
    if(req[Rtype] & Rd2h){
        datalen = GET2(req+Rcount);
        if(datalen <= 0 || datalen > Maxctlflen)
            error(Ebadlen);
        /* XXX cache madness */
        epio->cb = b = allocb(ROUND(datalen, ep->maxpkt));
        assert(((uintptr)b->wp & (BLOCKALIGN-1)) == 0);
        memset(b->wp, 0x55, b->lim - b->wp);
        cachedwbinvse(b->wp, b->lim - b->wp);
    }
```

```

        data = b->wp;
    }else{
        b = nil;
        datalen = n - Rsetuplen;
        data = req + Rsetuplen;
    }
    hc = chanalloc(ep);
    if(waserror()){
        chanrelease(ep, hc);
        if(strcmp(up->errstr, Estalled) == 0)
            return 0;
        nexterror();
    }
    chansetup(hc, ep);
    chanio(ep, hc, Epout, SETUP, req, Rsetuplen);
    if(req[Rtype] & Rd2h){
        if(ep->dev->hub <= 1){
            ep->toggle[Read] = DATA1;
            b->wp += multitrans(ep, hc, Read, data, datalen);
        }else
            b->wp += chanio(ep, hc, Epin, DATA1, data, datalen);
        chanio(ep, hc, Epout, DATA1, nil, 0);
        n = Rsetuplen;
    }else{
        if(datalen > 0)
            chanio(ep, hc, Epout, DATA1, data, datalen);
        chanio(ep, hc, Epin, DATA1, nil, 0);
        n = Rsetuplen + datalen;
    }
    chanrelease(ep, hc);
    poperror();
    return n;
}

```

⟨function ctldata(*arm*) 779a⟩≡ (787b)

```

static long
ctldata(Ep *ep, void *a, long n)
{
    Epio *epio;
    Block *b;

    epio = ep->aux;
    b = epio->cb;
    if(b == nil)
        return 0;
    if(n > BLEN(b))
        n = BLEN(b);
    memmove(a, b->rp, n);
    b->rp += n;
    if(BLEN(b) == 0){
        freeb(b);
        epio->cb = nil;
    }
    return n;
}

```

⟨function greset(*arm*) 779b⟩≡ (787b)

```

static void
greset(Dwcregs *r, int bits)
{

```

```

    r->grstctl |= bits;
    while(r->grstctl & bits)
        ;
    arch_microdelay(10);
}

⟨function init(arm) 780a⟩≡ (787b)
static void
init(Hci *hp)
{
    Ctlr *ctlr;
    Dwcregs *r;
    uint n, rx, tx, ptx;

    ctlr = hp->aux;
    r = ctlr->regs;

    ctlr->nchan = 1 + ((r->ghwcfg2 & Num_host_chan) >> ONum_host_chan);
    ctlr->chanintr = malloc(ctlr->nchan * sizeof(Rendez));

    r->gahbcfg = 0;
    setpower(PowerUsb, 1);

    while((r->grstctl&Ahbidle) == 0)
        ;
    greset(r, Csftrst);

    r->gusbcfg |= Force_host_mode;
    tsleep(&up->sleepr, returnfalse, 0, 25);
    r->gahbcfg |= Dmaenable;

    n = (r->ghwcfg3 & Dfifo_depth) >> Odfifo_depth;
    rx = 0x306;
    tx = 0x100;
    ptx = 0x200;
    r->grxfsiz = rx;
    r->gnptxfsiz = rx | tx<<ODepth;
    tsleep(&up->sleepr, returnfalse, 0, 1);
    r->hptxfsiz = (rx + tx) | ptx << ODepth;
    greset(r, Rxfflsh);
    r->grstctl = TXF_ALL;
    greset(r, Txfflsh);
    dprint("usbotg: FIFO depth %d sizes rx/nptx/ptx %8.8ux %8.8ux %8.8ux\n",
        n, r->grxfsiz, r->gnptxfsiz, r->hptxfsiz);

    r->hport0 = Prtpwr|Prtconndet|Prtenchn|Prtovrcurrchn;
    r->gintsts = ~0;
    r->gintmsk = Hcintr;
    r->gahbcfg |= Glblintrmsk;
}

⟨function dump((buses/arm/usbdwc.c)(arm)) 780b⟩≡ (787b)
static void
dump(Hci*)
{
}

⟨function fiqintr(arm) 780c⟩≡ (787b)
static void
fiqintr(Ureg*, void *a)

```

```

{
    Hci *hp;
    Ctlr *ctlr;
    Dwcregs *r;
    uint intr, haint, wakechan;
    int i;

    hp = a;
    ctlr = hp->aux;
    r = ctlr->regs;
    wakechan = 0;
    filock(ctlr);
    intr = r->gintsts;
    if(intr & Hcintr){
        haint = r->haint & r->haintmsk;
        for(i = 0; haint; i++){
            if(haint & 1){
                if(chanintr(ctlr, i) == 0){
                    r->haintmsk &= ~(1<<i);
                    wakechan |= 1<<i;
                }
            }
            haint >>= 1;
        }
    }
    if(intr & Sofintr){
        r->gintsts = Sofintr;
        if((r->hfnum&7) != 6){
            r->gintmsk &= ~Sofintr;
            wakechan |= ctlr->sofchan;
            ctlr->sofchan = 0;
        }
    }
    if(wakechan){
        ctlr->wakechan |= wakechan;
        armtimeraset(1);
    }
    fiunlock(ctlr);
}

```

<function armtimeraset(arm) 781a>≡ (715b)

```

void
armtimeraset(int n)
{
    Armtimer *tm;

    tm = (Armtimer*)ARMTIMER;
    if(n > 0){
        tm->ctl |= TmrEnable|TmrIntEnable;
        tm->load = n;
    }else{
        tm->load = 0;
        tm->ctl &= ~(TmrEnable|TmrIntEnable);
        tm->irq = 1;
    }
}

```

<function irqintr(arm) 781b>≡ (787b)

```

static void
irqintr(Ureg*, void *a)

```

```

{
    Ctlr *ctlr;
    uint wakechan;
    int i;

    ctlr = a;
    filock(ctlr);
    armtimerreset(0);
    wakechan = ctlr->wakechan;
    ctlr->wakechan = 0;
    fiunlock(ctlr);
    for(i = 0; wakechan; i++){
        if(wakechan & 1)
            wakeup(&ctlr->chanintr[i]);
        wakechan >>= 1;
    }
}

⟨function epopen(arm) 782a)≡ (787b)
static void
epopen(Ep *ep)
{
    ddprint("usbotg: epopen ep%d.%d ttype %d\n",
        ep->dev->nb, ep->nb, ep->ttype);
    switch(ep->ttype){
    case Tnone:
        error(Enotconf);
    case Tintr:
        assert(ep->pollival > 0);
        /* fall through */
    case Tbulk:
        if(ep->toggle[Read] == 0)
            ep->toggle[Read] = DATA0;
        if(ep->toggle[Write] == 0)
            ep->toggle[Write] = DATA0;
        break;
    }
    ep->aux = malloc(sizeof(Epio));
    if(ep->aux == nil)
        error(Enomem);
}

⟨function epclose((buses/arm/usbdwc.c)(arm)) 782b)≡ (787b)
static void
epclose(Ep *ep)
{
    ddprint("usbotg: epclose ep%d.%d ttype %d\n",
        ep->dev->nb, ep->nb, ep->ttype);
    switch(ep->ttype){
    case Tctl:
        freeb(((Epio*)ep->aux)->cb);
        /* fall through */
    default:
        free(ep->aux);
        break;
    }
}

⟨function epread(arm) 782c)≡ (787b)
static long

```

```

epread(Ep *ep, void *a, long n)
{
    Epio *epio;
    Block *b;
    uchar *p;
    ulong elapsed;
    long nr;

    ddprint("epread ep%d.%d %ld\n", ep->dev->nb, ep->nb, n);
    epio = ep->aux;
    b = nil;
    qlock(epio);
    if(waserror()){
        qunlock(epio);
        if(b)
            freeb(b);
        nexterror();
    }
    switch(ep->ttype){
default:
    error(Egreg);
case Tctl:
    nr = ctldata(ep, a, n);
    qunlock(epio);
    poperror();
    return nr;
case Tintr:
    elapsed = TK2MS(cpu->ticks) - epio->lastpoll;
    if(elapsed < ep->pollival)
        tsleep(&up->sleepr, returnfalse, 0, ep->pollival - elapsed);
    /* fall through */
case Tbulk:
    /* XXX cache madness */
    b = allocb(ROUND(n, ep->maxpkt));
    p = b->rp;
    assert(((uintptr)p & (BLOCKALIGN-1)) == 0);
    cachedinvse(p, n);
    nr = eptrans(ep, Read, p, n);
    cachedinvse(p, nr);
    epio->lastpoll = TK2MS(cpu->ticks);
    memmove(a, p, nr);
    qunlock(epio);
    freeb(b);
    poperror();
    return nr;
    }
}

```

*<function epwrite(*arm*) 783>*≡ *(787b)*

```

static long
epwrite(Ep *ep, void *a, long n)
{
    Epio *epio;
    Block *b;
    uchar *p;
    ulong elapsed;

    ddprint("epwrite ep%d.%d %ld\n", ep->dev->nb, ep->nb, n);
    epio = ep->aux;
    b = nil;

```



```

qlock(epio);
if(waserror()){
    qunlock(epio);
    if(b)
        freeb(b);
    nexterror();
}
switch(ep->ttype){
default:
    error(Egreg);
case Tintr:
    elapsed = TK2MS(cpu->ticks) - epio->lastpoll;
    if(elapsed < ep->pollival)
        tsleep(&up->sleepr, returnfalse, 0, ep->pollival - elapsed);
    /* fall through */
case Tctl:
case Tbulk:
    /* XXX cache madness */
    b = allocb(n);
    p = b->wp;
    assert(((uintptr)p & (BLOCKALIGN-1)) == 0);
    memmove(p, a, n);
    cachedwbse(p, n);
    if(ep->ttype == Tctl)
        n = ctltrans(ep, p, n);
    else{
        n = eptrans(ep, Write, p, n);
        epio->lastpoll = TK2MS(cpu->ticks);
    }
    qunlock(epio);
    freeb(b);
    poperror();
    return n;
}
}

```

```

⟨function seprintep((buses/arm/usbdwc.c)(arm)) 784a)≡ (787b)
static char*
seprintep(char *s, char*, Ep*)
{
    return s;
}

```

```

⟨function portenable(arm) 784b)≡ (787b)
static int
portenable(Hci *hp, int port, int on)
{
    Ctlr *ctlr;
    Dwcregs *r;

    assert(port == 1);
    ctlr = hp->aux;
    r = ctlr->regs;
    dprint("usbotg enable=%d; sts %#x\n", on, r->hport0);
    if(!on)
        r->hport0 = Prtpwr | Prtena;
    tsleep(&up->sleepr, returnfalse, 0, Enabledelay);
    dprint("usbotg enable=%d; sts %#x\n", on, r->hport0);
    return 0;
}

```

*<function portreset(*arm*) 785a>*≡ (787b)

```
static int
portreset(Hci *hp, int port, int on)
{
    Ctlr *ctlr;
    Dwcregs *r;
    int b, s;

    assert(port == 1);
    ctlr = hp->aux;
    r = ctlr->regs;
    dprint("usbotg reset=%d; sts %#x\n", on, r->hport0);
    if(!on)
        return 0;
    r->hport0 = Prtpwr | Prtrst;
    tsleep(&up->sleepr, returnfalse, 0, ResetdelayHS);
    r->hport0 = Prtpwr;
    tsleep(&up->sleepr, returnfalse, 0, Enabledelay);
    s = r->hport0;
    b = s & (Prtconndet|Prtenchnng|Prtovrcurrchnng);
    if(b != 0)
        r->hport0 = Prtpwr | b;
    dprint("usbotg reset=%d; sts %#x\n", on, s);
    if((s & Prtena) == 0)
        print("usbotg: host port not enabled after reset");
    return 0;
}
```

*<function portstatus(*arm*) 785b>*≡ (787b)

```
static int
portstatus(Hci *hp, int port)
{
    Ctlr *ctlr;
    Dwcregs *r;
    int b, s;

    assert(port == 1);
    ctlr = hp->aux;
    r = ctlr->regs;
    s = r->hport0;
    b = s & (Prtconndet|Prtenchnng|Prtovrcurrchnng);
    if(b != 0)
        r->hport0 = Prtpwr | b;
    b = 0;
    if(s & Prtconnsts)
        b |= HPpresent;
    if(s & Prtconndet)
        b |= HPstatuschg;
    if(s & Prtena)
        b |= HPenable;
    if(s & Prtenchnng)
        b |= HPchange;
    if(s & Prtovrcurract)
        b |= HPovercurrent;
    if(s & Prtsusp)
        b |= HPSuspend;
    if(s & Prtrst)
        b |= HPPreset;
    if(s & Prtpwr)
        b |= HPPower;
```

```

switch(s & Prtspd){
case HIGHSPPEED:
    b |= HPhigh;
    break;
case LOWSPEED:
    b |= HPslow;
    break;
}
return b;
}

```

```

⟨function shutdown(arm) 786a⟩≡ (787b)
static void
shutdown(Hci*)
{
}

```

```

⟨function setdebug(arm) 786b⟩≡ (787b)
static void
setdebug(Hci*, int d)
{
    debug = d;
}

```

```

⟨function reset(arm) 786c⟩≡ (787b)
static int
reset(Hci *hp)
{
    Ctlr *ctlr;
    uint id;

    ctlr = &dwc;
    if(ctlr->regs != nil)
        return -1;
    ctlr->regs = (Dwcregs*)USBREGS;
    id = ctlr->regs->gsnpsid;
    if((id>>16) != ('0'<<8 | 'T'))
        return -1;
    dprint("usbotg: rev %d.%3x\n", (id>>12)&0xF, id&0xFFF);

    arch_intrenable(IrqtimerArm, irqintr, ctlr, 0, "dwc");

    hp->aux = ctlr;
    hp->port = 0;
    hp->irq = IRQusb;
    hp->tbdn = 0;
    hp->nports = 1;
    hp->highspeed = 1;

    hp->init = init;
    hp->dump = dump;
    hp->interrupt = fiqintr;
    hp->epopen = epopen;
    hp->epclose = epclose;
    hp->epread = epread;
    hp->epwrite = epwrite;
    hp->seprintep = seprintep;
    hp->portenable = portenable;
    hp->portreset = portreset;
    hp->portstatus = portstatus;
}

```

```

    hp->shutdown = shutdown;
    hp->debug = setdebug;
    hp->type = "dwcotg";
    return 0;
}

⟨function usbdwclink(arm) 787a⟩≡ (787b)
void
usbdwclink(void)
{
    addhcitype("dwcotg", reset);
}

⟨buses/arm/usbdwc.c 787b⟩≡
/*
 * USB host driver for BCM2835
 * Synopsis DesignWare Core USB 2.0 OTG controller
 *
 * Copyright © 2012 Richard Miller <r.miller@acm.org>
 *
 * This is work in progress:
 * - no isochronous pipes
 * - no bandwidth budgeting
 * - frame scheduling is crude
 * - error handling is overly optimistic
 * It should be just about adequate for a Plan 9 terminal with
 * keyboard, mouse, ethernet adapter, and an external flash drive.
 */
⟨kernel basic includes 35a⟩

#include "io.h"
#include "../port/usb.h"

#include "dwcotg.h"

⟨constant USBREGS(arm) 32e⟩
⟨enum _anon_((buses/arm/usbdwc.c)(arm)) 770a⟩

typedef struct Ctlr Ctlr;
typedef struct Epio Epio;

⟨struct Ctlr((buses/arm/usbdwc.c)(arm)) 770b⟩

⟨struct Epio(arm) 770c⟩

⟨global dwc(arm) 770d⟩
⟨global debug((buses/arm/usbdwc.c)(arm)) 770e⟩

⟨global Ebadlen(arm) 770f⟩

static void clog(Ep *ep, Hostchan *hc);
static void logdump(Ep *ep);

⟨function filock(arm) 770g⟩

⟨function fiunlock(arm) 770h⟩

⟨function chanalloc(arm) 771a⟩

⟨function chanrelease(arm) 771b⟩

```

*<function chansetup(*arm*) 771c>*

*<function sofdone(*arm*) 772a>*

*<function sofwait(*arm*) 772b>*

*<function chandone(*arm*) 772c>*

*<function chanwait(*arm*) 773>*

*<function chanintr(*arm*) 774a>*

*<global chanlog(*arm*) 774b>*
*<global nchanlog(*arm*) 774c>*

*<function logstart(*arm*) 774d>*

*<function clog(*arm*) 775a>*

*<function logdump(*arm*) 775b>*

*<function chanio(*arm*) 775c>*

*<function multitrans(*arm*) 777>*

*<function eptrans(*arm*) 778a>*

*<function ctltrans(*arm*) 778b>*

*<function ctldata(*arm*) 779a>*

*<function greset(*arm*) 779b>*

*<function init(*arm*) 780a>*

*<function dump((*buses/arm/usbdwc.c*) (*arm*)) 780b>*

*<function fiqintr(*arm*) 780c>*

*<function irqintr(*arm*) 781b>*

*<function epopen(*arm*) 782a>*

*<function epclose((*buses/arm/usbdwc.c*) (*arm*)) 782b>*

*<function epread(*arm*) 782c>*

*<function epwrite(*arm*) 783>*

*<function seprintep((*buses/arm/usbdwc.c*) (*arm*)) 784a>*

*<function portenable(*arm*) 784b>*

*<function portreset(*arm*) 785a>*

*<function portstatus(*arm*) 785b>*

*<function shutdown(*arm*) 786a>*

<function setdebug(arm) 786b>

<function reset(arm) 786c>

<function usbdwclink(arm) 787a>

E.29.7 buses/arm/dma.c

<macro IRQDMA(arm) 789a>≡ (685a)
#define IRQDMA(chan) (IRQdma0+(chan))

<enum Dma(arm) 789b>≡ (685a)
enum DmaFlags {
 DmaD2M = 0, /* device to memory */
 DmaM2D = 1, /* memory to device */
 DmaM2M = 2, /* memory to memory */

 DmaChanEmmc = 4, /* can only use 2-5, maybe 0 */
 DmaChanSpiTx= 2,
 DmaChanSpiRx= 0,

 DmaDevSpiTx = 6,
 DmaDevSpiRx = 7,
 DmaDevEmmc = 11,
};

<enum _anon_((buses/arm/dma.c)(arm)) 789c>≡ (793b)
enum {
 Nchan = 7, /* number of dma channels */
 Regsize = 0x100, /* size of regs for each chan */
 Calign = 32, /* control block byte alignment */
 Dbg = 0,

 /* registers for each dma controller */
 Cs = 0x00>>2,
 Conblkad = 0x04>>2,
 Ti = 0x08>>2,
 Sourcead = 0x0c>>2,
 Destad = 0x10>>2,
 Txfrlen = 0x14>>2,
 Stride = 0x18>>2,
 Nextconbk = 0x1c>>2,
 Debug = 0x20>>2,

 /* collective registers */
 Intstatus = 0xfe0>>2,
 Enable = 0xff0>>2,

 /* Cs */
 Reset = 1<<31,
 Abort = 1<<30,
 Error = 1<<8,
 Waitwrite = 1<<6,
 Waitdreq = 1<<5,
 Paused = 1<<4,
 Dreq = 1<<3,
 Int = 1<<2,
 End = 1<<1,
 Active = 1<<0,

```

/* Ti */
Permapshift= 16,
Srcignore   = 1<<11,
Srcdreq     = 1<<10,
Srcwidth128 = 1<<9,
Srcinc      = 1<<8,
Destignore  = 1<<7,
Destdreq    = 1<<6,
Destwidth128 = 1<<5,
Destinc     = 1<<4,
Waitresp    = 1<<3,
Tdmode      = 1<<1,
Inten       = 1<<0,

/* Debug */
Lite        = 1<<28,
Clrerrors   = 7<<0,
};

⟨struct Ctlr(arm) 790a⟩≡ (793b)
struct Ctlr {
    u32int *regs;
    Cb *cb;
    Rendez r;
    int dmadone;
};

⟨struct Cb(arm) 790b⟩≡ (793b)
struct Cb {
    u32int ti;
    u32int sourcead;
    u32int destad;
    u32int txfrlen;
    u32int stride;
    u32int nextconbk;
    u32int reserved[2];
};

⟨global dma(arm) 790c⟩≡ (793b)
static Ctlr dma[Nchan];

⟨global dmaregs(arm) 790d⟩≡ (793b)
static u32int *dmaregs = (u32int*)DMAREGS;

⟨Soc bus fields 790e⟩≡ (54c) 791a▷
uintptr busdram;

⟨soc(raspberry pi1) other fields(arm) 790f⟩+≡ (54d) ◁475e 791b▷
.busdram = 0x40000000,

⟨soc(raspberry pi2) other fields(arm) 790g⟩+≡ (473f) ◁475f 791c▷
.busdram = 0xC0000000,

⟨function dmaaddr(arm) 790h⟩≡ (793b)
uintptr
dmaaddr(void *va)
{
    return soc.busdram | (PTR2UINT(va) & ~KSEGM); // PADDR?
}

```

<pre> ⟨Soc bus fields 791a⟩+≡ uintptr busio; </pre>	(54c) <790e
<pre> ⟨soc(raspberry pi1) other fields(arm) 791b⟩+≡ .busio = 0x7E000000, </pre>	(54d) <790f
<pre> ⟨soc(raspberry pi2) other fields(arm) 791c⟩+≡ .busio = 0x7E000000, </pre>	(473f) <790g
<pre> ⟨function dmaioaddr(arm) 791d⟩≡ static uintptr dmaioaddr(void *va) { return soc.busio (PTR2UINT(va) & ~VIRTIO); } </pre>	(793b)
<pre> ⟨function dump(arm) 791e⟩≡ static void dump(char *msg, uchar *p, int n) { print("%s", msg); while(n-- > 0) print(" %2.2x", *p++); print("\n"); } </pre>	(793b)
<pre> ⟨function dumpdregs(arm) 791f⟩≡ static void dumpdregs(char *msg, u32int *r) { int i; print("%s: %#p =", msg, r); for(i = 0; i < 9; i++) print(" %8.8uX", r[i]); print("\n"); } </pre>	(793b)
<pre> ⟨function dmadone(arm) 791g⟩≡ static int dmadone(void *a) { return ((Ctlr*)a)->dmadone; } </pre>	(793b)
<pre> ⟨function dmainterrupt(arm) 791h⟩≡ static void dmainterrupt(Ureg*, void *a) { Ctlr *ctlr; ctlr = a; ctlr->regs[Cs] = Int; ctlr->dmadone = 1; wakeup(&ctlr->r); } </pre>	(793b)

<function dmastart(*arm*) 792)≡ (793b)

```
void
dmastart(int chan, int dev, int dir, void *src, void *dst, int len)
{
    Ctlr *ctlr;
    Cb *cb;
    int ti;

    ctlr = &dma[chan];
    if(ctlr->regs == nil){
        ctlr->regs = (u32int*)(DMAREGS + chan*Regsize);
        ctlr->cb = xspanalloc(sizeof(Cb), Cbalign, 0);
        assert(ctlr->cb != nil);
        dmaregs[Enable] |= 1<<chan;
        ctlr->regs[Cs] = Reset;
        while(ctlr->regs[Cs] & Reset)
            ;
        arch_intrenable(IRQDMA(chan), dmainterrupt, ctlr, 0, "dma");
    }
    cb = ctlr->cb;
    ti = 0;
    switch(dir){
    case DmaD2M:
        cachedinvse(dst, len);
        ti = Srcdreq | Destinc;
        cb->sourcead = dmaioaddr(src);
        cb->destad = dmaaddr(dst);
        break;
    case DmaM2D:
        cachedwbse(src, len);
        ti = Destdreq | Srcinc;
        cb->sourcead = dmaaddr(src);
        cb->destad = dmaioaddr(dst);
        break;
    case DmaM2M:
        cachedwbse(src, len);
        cachedinvse(dst, len);
        ti = Srcinc | Destinc;
        cb->sourcead = dmaaddr(src);
        cb->destad = dmaaddr(dst);
        break;
    }
    cb->ti = ti | dev<<Permapshift | Inten;
    cb->txfrlen = len;
    cb->stride = 0;
    cb->nextconbk = 0;
    cachedwbse(cb, sizeof(Cb));
    ctlr->regs[Cs] = 0;
    arch_microdelay(1);
    ctlr->regs[Conblkad] = dmaaddr(cb);
    DBG print("dma start: %ux %ux %ux %ux %ux %ux\n",
        cb->ti, cb->sourcead, cb->destad, cb->txfrlen,
        cb->stride, cb->nextconbk);
    DBG print("intstatus %ux\n", dmaregs[Intstatus]);
    dmaregs[Intstatus] = 0;
    ctlr->regs[Cs] = Int;
    arch_microdelay(1);
    arch_coherence();
    DBG dumpdregs("before Active", ctlr->regs);
    ctlr->regs[Cs] = Active;
```

```

        DBG dumpdregs("after Active", ctrl->regs);
    }

<function dmawait(arm) 793a>≡ (793b)
    int
    dmawait(int chan)
    {
        Ctlr *ctrl;
        u32int *r;
        int s;

        ctrl = &dma[chan];
        tsleep(&ctrl->r, dmadone, ctrl, 3000);
        ctrl->dmadone = 0;
        r = ctrl->regs;
        DBG dumpdregs("after sleep", r);
        s = r[Cs];
        if((s & (Active|End|Error)) != End){
            print("dma chan %d %s Cs %ux Debug %ux\n", chan,
                (s&End)? "error" : "timeout", s, r[Debug]);
            r[Cs] = Reset;
            r[Debug] = Clrerrors;
            return -1;
        }
        r[Cs] = Int|End;
        return 0;
    }

<buses/arm/dma.c 793b>≡
/*
 * bcm2835 dma controller
 *
 * simplest to use only channels 0-6
 * channels 7-14 have reduced functionality
 * channel 15 is at a weird address
 * channels 0 and 15 have an "external 128 bit 8 word read FIFO"
 *   for memory to memory transfers
 *
 * Experiments show that only channels 2-5,11-12 work with mmc
 */
<kernel basic includes 35a>
#include "io.h"

<constant DMAREGS(arm) 31h>

#define DBG if(Db)

<enum _anon_((buses/arm/dma.c)(arm)) 789c>

typedef struct Ctlr Ctlr;
typedef struct Cb Cb;

<struct Ctlr(arm) 790a>

<struct Cb(arm) 790b>

<global dma(arm) 790c>
<global dmaregs(arm) 790d>

<function dmaaddr(arm) 790h>

```

`<function dmaioaddr(arm) 791d>`
`<function dump(arm) 791e>`
`<function dumpdregs(arm) 791f>`
`<function dmadone(arm) 791g>`
`<function dmainterrupt(arm) 791h>`
`<function dmastart(arm) 792>`
`<function dmawait(arm) 793a>`

E.29.8 buses/arm/uartmini.c

`<constant OkLed(arm) 794a>≡ (802)`
`#define OkLed 16`

`<constant TxPin(arm) 794b>≡ (802)`
`#define TxPin 14`

`<constant RxPin(arm) 794c>≡ (802)`
`#define RxPin 15`

`<enum _anon_((buses/arm/uartmini.c)(arm)) 794d>≡ (802)`
`/* GPIO regs */`
`enum {`
`Fsel0 = 0x00>>2,`
`FuncMask= 0x7,`
`Input = 0x0,`
`Output = 0x1,`
`Alt0 = 0x4,`
`Alt1 = 0x5,`
`Alt2 = 0x6,`
`Alt3 = 0x7,`
`Alt4 = 0x3,`
`Alt5 = 0x2,`
`Set0 = 0x1c>>2,`
`Clr0 = 0x28>>2,`
`Lev0 = 0x34>>2,`
`PUD = 0x94>>2,`
`Off = 0x0,`
`Pullldown= 0x1,`
`Pullup = 0x2,`
`PUDclk0 = 0x98>>2,`
`PUDclk1 = 0x9c>>2,`
`};`

`<enum _anon_((buses/arm/uartmini.c)2(arm)) 794e>≡ (802)`
`/* AUX regs */`
`enum {`
`Irq = 0x00>>2,`
`UartIrq = 1<<0,`
`Enables = 0x04>>2,`
`UartEn = 1<<0,`
`MuIo = 0x40>>2,`

```

    MuIer    = 0x44>>2,
    RxIen    = 1<<0,
    TxIen    = 1<<1,
    MuIir    = 0x48>>2,
    MuLcr    = 0x4c>>2,
    Bitsmask= 3<<0,
    Bits7    = 2<<0,
    Bits8    = 3<<0,
    MuMcr    = 0x50>>2,
    RtsN     = 1<<1,
    MuLsr    = 0x54>>2,
    TxDone   = 1<<6,
    TxRdy    = 1<<5,
    RxRdy    = 1<<0,
    MuCntl   = 0x60>>2,
    CtsFlow  = 1<<3,
    TxEn     = 1<<1,
    RxEn     = 1<<0,
    MuBaud   = 0x68>>2,
};

```

```

⟨global miniuart(arm) 795a⟩≡ (802)
static Uart miniuart = {
    .regs    = (u32int*)AUXREGS,
    .name     = "uart0",
    .freq    = 250000000,
    .phys    = &miniphysuart,
};

```

```

⟨function gpiosel(arm) 795b⟩≡ (802)
void
gpiosel(uint pin, int func)
{
    u32int *gp, *fsel;
    int off;

    gp = (u32int*)GPIOREGS;
    fsel = &gp[Fsel0 + pin/10];
    off = (pin % 10) * 3;
    *fsel = (*fsel & ~(FuncMask<<off)) | func<<off;
}

```

```

⟨function gpiopull(arm) 795c⟩≡ (802)
static void
gpiopull(uint pin, int func)
{
    u32int *gp, *reg;
    u32int mask;

    gp = (u32int*)GPIOREGS;
    reg = &gp[PUDclk0 + pin/32];
    mask = 1 << (pin % 32);
    gp[PUD] = func;
    arch_microdelay(1);
    *reg = mask;
    arch_microdelay(1);
    *reg = 0;
}

```

```

⟨function gpiopulloff(arm) 796a)≡ (802)
void
gpiopulloff(uint pin)
{
    gpiopull(pin, Off);
}

⟨function gpiopullup(arm) 796b)≡ (802)
void
gpiopullup(uint pin)
{
    gpiopull(pin, Pullup);
}

⟨function gpiopulldown(arm) 796c)≡ (802)
void
gpiopulldown(uint pin)
{
    gpiopull(pin, Pulldown);
}

⟨function gpioout(arm) 796d)≡ (802)
void
gpioout(uint pin, int set)
{
    u32int *gp;
    int v;

    gp = (u32int*)GPIOREGS;
    v = set? Set0 : Clr0;
    gp[v + pin/32] = 1 << (pin % 32);
}

⟨function gpioin(arm) 796e)≡ (802)
int
gpioin(uint pin)
{
    u32int *gp;

    gp = (u32int*)GPIOREGS;
    return (gp[Lev0 + pin/32] & (1 << (pin % 32))) != 0;
}

⟨function interrupt(arm) 796f)≡ (802)
static void
interrupt(Ureg*, void *arg)
{
    Uart *uart;
    u32int *ap;

    uart = arg;
    ap = (u32int*)uart->regs;

    arch_coherence();
    if(0 && (ap[Irq] & UartIrq) == 0)
        return;
    if(ap[MuLsr] & TxRdy)
        uartkick(uart);
    if(ap[MuLsr] & RxRdy){
        if(uart->console){

```

```

        if(uart->opens == 1)
            uart->putc = kbdcr2nl;
        else
            uart->putc = nil;
    }
    do{
        uartrecv(uart, ap[MuIo] & 0xFF);
    }while(ap[MuLsr] & RxRdy);
}
arch_coherence();
}

⟨function pnp(arm) 797a)≡ (802)
static Uart*
pnp(void)
{
    Uart *uart;

    uart = &miniuart;
    if(uart->console == 0)
        kbdq = qopen(8*1024, 0, nil, nil);
    return uart;
}

⟨function enable(arm) 797b)≡ (802)
static void
enable(Uart *uart, int ie)
{
    u32int *ap;

    ap = (u32int*)uart->regs;
    arch_delay(10);
    gpiosel(TxPin, Alt5);
    gpiosel(RxPin, Alt5);
    gpiopulldown(TxPin);
    gpiopulldown(RxPin);
    ap[Enables] |= UartEn;
    ap[MuIir] = 6;
    ap[MuLcr] = Bits8;
    ap[MuCntl] = TxEn|RxEEn;
    ap[MuBaud] = uart->freq/(115200*8) - 1;
    if(ie){
        arch_intrenable(IRQaux, interrupt, uart, 0, "uart");
        ap[MuIer] = RxIen|TxIen;
    }else
        ap[MuIer] = 0;
}

⟨function disable(arm) 797c)≡ (802)
static void
disable(Uart *uart)
{
    u32int *ap;

    ap = (u32int*)uart->regs;
    ap[MuCntl] = 0;
    ap[MuIer] = 0;
}

```

```

⟨function kick(arm) 798a)≡ (802)
    static void
    kick(Uart *uart)
    {
        u32int *ap;

        ap = (u32int*)uart->regs;
        if(uart->blocked)
            return;
        arch_coherence();
        while(ap[MuLsr] & TxRdy){
            if(uart->op >= uart->oe && uartstageoutput(uart) == 0)
                break;
            ap[MuIo] = *(uart->op++);
        }
        if(ap[MuLsr] & TxDone)
            ap[MuIer] &= ~TxIen;
        else
            ap[MuIer] |= TxIen;
        arch_coherence();
    }

```

```

⟨function dobreak(arm) 798b)≡ (802)
    /* TODO */
    static void
    dobreak(Uart *uart, int ms)
    {
        USED(uart, ms);
    }

```

```

⟨function baud(arm) 798c)≡ (802)
    static int
    baud(Uart *uart, int n)
    {
        u32int *ap;

        ap = (u32int*)uart->regs;
        if(uart->freq == 0 || n <= 0)
            return -1;
        ap[MuBaud] = (uart->freq + 4*n - 1) / (8 * n) - 1;
        uart->baud = n;
        return 0;
    }

```

```

⟨function bits(arm) 798d)≡ (802)
    static int
    bits(Uart *uart, int n)
    {
        u32int *ap;
        int set;

        ap = (u32int*)uart->regs;
        switch(n){
        case 7:
            set = Bits7;
            break;
        case 8:
            set = Bits8;
            break;
        default:

```

```

        return -1;
    }
    ap[MuLcr] = (ap[MuLcr] & ~Bitsmask) | set;
    uart->bits = n;
    return 0;
}

⟨function stop(arm) 799a⟩≡ (802)
static int
stop(Uart *uart, int n)
{
    if(n != 1)
        return -1;
    uart->stop = n;
    return 0;
}

⟨function parity(arm) 799b⟩≡ (802)
static int
parity(Uart *uart, int n)
{
    if(n != 'n')
        return -1;
    uart->parity = n;
    return 0;
}

⟨function modemctl(arm) 799c⟩≡ (802)
/*
 * cts/rts flow control
 * need to bring signals to gpio pins before enabling this
 */

static void
modemctl(Uart *uart, int on)
{
    u32int *ap;

    ap = (u32int*)uart->regs;
    if(on)
        ap[MuCntl] |= CtsFlow;
    else
        ap[MuCntl] &= ~CtsFlow;
    uart->modem = on;
}

⟨function rts(arm) 799d⟩≡ (802)
static void
rts(Uart *uart, int on)
{
    u32int *ap;

    ap = (u32int*)uart->regs;
    if(on)
        ap[MuMcr] &= ~RtsN;
    else
        ap[MuMcr] |= RtsN;
}

```


<function status(arm) 800a>≡ (802)

```
static long
status(Uart *uart, void *buf, long n, long offset)
{
    char *p;

    p = malloc(READSTR);
    if(p == nil)
        error(Enomem);
    snprintf(p, READSTR,
        "b%d\n"
        "dev(%d) type(%d) framing(%d) overruns(%d) "
        "berr(%d) serr(%d)\n",

        uart->baud,
        uart->dev,
        uart->type,
        uart->ferr,
        uart->oerr,
        uart->berr,
        uart->serr
    );
    n = readstr(offset, buf, n, p);
    free(p);

    return n;
}
```

<function donothing(arm) 800b>≡ (802)

```
static void
donothing(Uart*, int)
{
}
```

<function putc(arm) 800c>≡ (802)

```
void
putc(Uart*, int c)
{
    u32int *ap;

    ap = (u32int*)AUXREGS;
    while((ap[MuLsr] & TxRdy) == 0)
        ;
    ap[MuIo] = c;
    while((ap[MuLsr] & TxRdy) == 0)
        ;
}
```

<function getc(arm) 800d>≡ (802)

```
int
getc(Uart*)
{
    u32int *ap;

    ap = (u32int*)AUXREGS;
    while((ap[MuLsr] & RxRdy) == 0)
        ;
    return ap[MuIo] & 0xFF;
}
```

*<function uartconsinit(*arm*) 801a*) $\equiv$ (802)

```
void
uartconsinit(void)
{
    Uart *uart;
    int n;
    char *p, *cmd;

    if((p = getconf("console")) == nil)
        return;
    n = strtoul(p, &cmd, 0);
    if(p == cmd)
        return;
    switch(n){
    default:
        return;
    case 0:
        uart = &miniuart;
        break;
    }

    if(!uart->enabled)
        (*uart->phys->enable)(uart, 0);
    uartctl(uart, "b9600 18 pn s1");
    if(*cmd != '\0')
        uartctl(uart, cmd);

    consuart = uart;
    uart->console = 1;
}
```

*<global miniphysuart(*arm*) 801b*) $\equiv$ (802)

```
PhysUart miniphysuart = {
    .name      = "miniuart",
    .pnp       = pnp,
    .enable    = enable,
    .disable   = disable,
    .kick      = kick,
    .dobreak   = dobreak,
    .baud      = baud,
    .bits      = bits,
    .stop      = stop,
    .parity    = parity,
    .modemctl  = donothing,
    .rts       = rts,
    .dtr       = donothing,
    .status    = status,
    .fifo      = donothing,
    .getc      = getc,
    .putc      = putc,
};
```

*<function okay(*arm*) 801c*) $\equiv$ (802)

```
void
okay(int on)
{
    static int first;
    static int okled, polarity;
    char *p;
```

```

    if(!first++){
        p = getconf("bcm2709.disk_led_gpio");
        if(p == nil)
            p = getconf("bcm2708.disk_led_gpio");
        if(p != nil)
            okled = strtol(p, 0, 0);
        else
            okled = 0kLed;
        p = getconf("bcm2709.disk_led_active_low");
        if(p == nil)
            p = getconf("bcm2708.disk_led_active_low");
        polarity = (p == nil || *p == '1');
        gpiosel(okled, Output);
    }
    gpioout(okled, on^polarity);
}

⟨buses/arm/uartmini.c 802⟩≡
/*
 * bcm2835 mini uart (UART1)
 */
⟨kernel basic includes 35a⟩

#include "io.h"

⟨constant GPIOREGS(arm) 32g⟩
⟨constant AUXREGS(arm) 32a⟩
⟨constant 0kLed(arm) 794a⟩
⟨constant TxPin(arm) 794b⟩
⟨constant RxPin(arm) 794c⟩

⟨enum _anon_((buses/arm/uartmini.c)(arm)) 794d⟩
⟨enum _anon_((buses/arm/uartmini.c)2(arm)) 794e⟩

extern PhysUart miniphysuart;

⟨global miniuart(arm) 795a⟩
⟨function gpiosel(arm) 795b⟩
⟨function gpiopull(arm) 795c⟩
⟨function gpiopulloff(arm) 796a⟩
⟨function gpiopullup(arm) 796b⟩
⟨function gpiopulldown(arm) 796c⟩
⟨function gpioout(arm) 796d⟩
⟨function gpioin(arm) 796e⟩
⟨function interrupt(arm) 796f⟩
⟨function pnp(arm) 797a⟩
⟨function enable(arm) 797b⟩
⟨function disable(arm) 797c⟩

```

`<function kick(arm) 798a>`
`<function dobreak(arm) 798b>`
`<function baud(arm) 798c>`
`<function bits(arm) 798d>`
`<function stop(arm) 799a>`
`<function parity(arm) 799b>`
`<function modemctl(arm) 799c>`
`<function rts(arm) 799d>`
`<function status(arm) 800a>`
`<function donothing(arm) 800b>`
`<function putc(arm) 800c>`
`<function getc(arm) 800d>`
`<function uartconsinit(arm) 801a>`
`<global miniphysuart(arm) 801b>`
`<function okay(arm) 801c>`

Glossary

UP = User Process
PC = Program Counter
SPL = Set Priority Level
PD = Page Directory
PT = Page Table
PDB = Page Directory Base
PDE = Page Directory Entry
PTE = Page Table Entry
PPN = Physical Page Number?
EDF = Earliest Deadline First
RPC = Remote Procedure Call
EGRP = Environment GRouP
FGRP = File descriptor GRouP (channels)
PGRP = Process GRouP (namespace)
QID = uniQue IDentifier (uid is taken for user id)
TOD = Time Of Day
UART = Universal Asynchronous Receive/Transmitter

(ARM)

ARM = Acorn Risc Machines
CPSR = Current Program Status Register
SPSR = Saved Program Status Register
MCR = Move to Coprocessor from ARM Register
MRC = Move to ARM Register from Coprocessor
TTB = Translation Table Base
TLB = Translation Lookup Table
AP = Access Permission
DAC = Domain Access Control
RFE = Return From Exception
IRQ = Interrupt Request

(x86)

IRQ = Interrupt Request
INTR = INTeRrupt (not INT, probably to avoid ambiguity with integer)
VNO = Vector Number (interrupt)
ISR = Interrupt Service Routine
NMI = Non Maskable Interrupts (hardware error)
AP = Application Processor
RMAP = RAM map
UMB = Upper Memory Block

GDT = Global Descriptor Table
VPT = Virtual Page Table
APIC = Advanced Programmable Interrupt Controller (a next gen 8259)
ACPI = Advanced Configuration & Power Interface
TSC = Time Stamp Counter

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Bibliography

- [Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 20
- [KR88] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988. cited page(s) 19
- [Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. <https://github.com/aryx/syncweb>. cited page(s) 20
- [Pad15a] Yoann Padioleau. *Principia Softwarica: The ARM Assembler 5a*. 2015. cited page(s) 19, 571
- [Pad15b] Yoann Padioleau. *Principia Softwarica: The ARM Emulator 5i*. 2015. cited page(s) 19
- [Pad15c] Yoann Padioleau. *Principia Softwarica: The ARM Linker 5l*. 2015. cited page(s) 43, 571
- [Pad16a] Yoann Padioleau. *Principia Softwarica: The Build System mk*. 2016. cited page(s) 571
- [Pad16b] Yoann Padioleau. *Principia Softwarica: The Plan 9 C Compiler 5c*. 2016. cited page(s) 33, 571
- [Pad16c] Yoann Padioleau. *Principia Softwarica: The Plan 9 Core Libraries*. 2016. cited page(s) 21, 22, 79, 571
- [Pad16d] Yoann Padioleau. *Principia Softwarica: The Plan 9 Graphics System draw*. 2016. cited page(s) 18, 32, 81, 571, 572
- [Pad16e] Yoann Padioleau. *Principia Softwarica: The Plan 9 Network Stack /net*. 2016. cited page(s) 18
- [Pad18] Yoann Padioleau. *Principia Softwarica: The Plan 9 Shell rc*. 2018. cited page(s) 571